The FEMU Manual

Reference Manual and User Guide for the FEMU
NVMe and CXL SSD Emulator

Huaicheng Li and FEMU contributors

`https://github.com/MoatLab/FEMU`

Built November 6, 2026 from revision `43655371a`

Abstract

FEMU is an NVMe and CXL SSD emulator built on QEMU/KVM. A guest operating system sees an emulated SSD as a real PCIe device and drives it with its own NVMe or CXL drivers, while FEMU charges NAND, channel and garbage collection time from a configurable timing model. FEMU emulates several kinds of SSD: a fast DRAM-backed NVMe device with no media timing, a conventional SSD with a device FTL, Zoned Namespace, Open-Channel, key-value and computational storage SSDs, and a CXL Type-3 memory device backed by flash.

This manual describes how to build and run FEMU, how it is built inside, each mode and feature with its parameters, the counters it reports and how to measure with them, and how to test and change it. The chapters are generated from the Markdown documentation in the FEMU repository, which remains the reference when the two differ.

Keywords: SSD emulation, NVMe, flash translation layer, zoned storage, CXL, QEMU

Acknowledgement. FEMU is supported by the U.S. National Science Foundation through NSF POSE award #2550145, *Toward a Community-Driven Fast Emulator (FEMU) Ecosystem for Next-Generation Storage Systems Research and Innovation*. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

Contents

1 Introduction **1**
- 1.1 What FEMU does . . . 1
- 1.2 What FEMU does not do . . . 1
- 1.3 Organization of this manual . . . 2
- 1.4 Conventions . . . 2
- 1.5 How to cite FEMU . . . 3

2 Getting started **4**
- 2.1 Requirements . . . 4
- 2.2 Build FEMU . . . 6
- 2.3 Guest image . . . 8
- 2.4 Quick start . . . 11

3 Architecture and design **14**
- 3.1 FEMU architecture . . . 14
- 3.2 FEMU design . . . 23
- 3.3 NVMe frontend . . . 32
- 3.4 The BlackBox FTL . . . 49
- 3.5 NAND media and timing model . . . 75

4 Modes and features **102**
- 4.1 Choosing a mode . . . 102
- 4.2 NoSSD . . . 109
- 4.3 NoSSD: the mode without a media model . . . 111
- 4.4 BlackBox SSD (BBSSD) . . . 115
- 4.5 Flexible Data Placement (FDP) . . . 122
- 4.6 FDP: Flexible Data Placement . . . 129
- 4.7 Zoned Namespace SSD (ZNS) . . . 147
- 4.8 ZNS: the zoned namespace model . . . 154
- 4.9 Open-Channel SSD (OCSSD) . . . 167
- 4.10 OCSSD: the Open-Channel extension . . . 171
- 4.11 Key-value SSD (KV) . . . 181
- 4.12 KV: the key-value extension . . . 188
- 4.13 Computational storage (CSD) . . . 198
- 4.14 CSD: the computational storage extension . . . 205
- 4.15 Several namespaces and devices . . . 216
- 4.16 Subsystems, controllers and namespaces . . . 219
- 4.17 Namespace management, metadata and protection information . . . 226
- 4.18 CXL SSD . . . 231
- 4.19 CXL SSD (`femu-cxl-ssd`) . . . 242
- 4.20 CXL caching API (CCA) . . . 264
- 4.21 CXL NVMe link . . . 268
- 4.22 Computational Programs and Subsystem Local Memory . . . 272
- 4.23 LSM-tree key index for KV namespaces . . . 276
- 4.24 KV value packing . . . 280

4.25 Tiered NAND: a pSLC pool in front of dense NAND . . . 283

5 Running and configuring **288**
5.1 Parameter manual . . . 288
5.2 Tutorials . . . 305
5.3 Tutorial 01: your first SSD . . . 307
5.4 Tutorial 02: garbage collection and write amplification . . . 309
5.5 Tutorial 03: zoned namespaces . . . 313
5.6 Tutorial 04: Flexible Data Placement . . . 317
5.7 Tutorial 05: latency tuning . . . 321
5.8 Tutorial 06: several namespaces, several modes . . . 324
5.9 Tutorial 07: key-value SSD . . . 326
5.10 Tutorial 08: a CXL SSD as memory . . . 329
5.11 Tutorial 09: configuration files . . . 333
5.12 Performance tuning . . . 335
5.13 Security and limits . . . 339

6 Measuring and validation **344**
6.1 Measuring . . . 344
6.2 Timing model . . . 350

7 Troubleshooting **356**
7.1 Troubleshooting and FAQ . . . 356
7.2 Debugging . . . 362

8 Contributing **367**
8.1 Code structure . . . 367
8.2 Testing . . . 368
8.3 Keeping the documentation correct . . . 377

A Property reference **380**
A.1 Device property reference . . . 380
A.2 Runtime properties and counters . . . 397

B Log pages and counters **402**
B.1 Vendor log page C0h . . . 402
B.2 Computational programs and reachability . . . 403
B.3 Asynchronous events . . . 404
B.4 End of life . . . 404
B.5 Thermal model . . . 405
B.6 Persistent Event log retention . . . 405

C Scripts and tools **407**
C.1 Build and setup . . . 407
C.2 Guest image and access . . . 407
C.3 Launchers . . . 407
C.4 Configuration files . . . 409
C.5 Guest-side test tools . . . 410
C.6 Host monitoring tools . . . 412

C.7 Host tuning helpers . . . 412
C.8 Documentation tooling . . . 413
C.9 CXL caching API tools: `hw/femu/tools/cca/` . . . 413
C.10 Legacy scripts . . . 414
C.11 Related pages . . . 414

D Changelog **415**
D.1 Unreleased (since femu-v9.0.1) . . . 415

List of Figures

1 Where FEMU sits: the guest drives FEMU's NVMe or CXL device, the timing model decides when each request completes, and the data goes straight to the memory backend in host memory. 3
2 Quick start: build FEMU, make the guest image, boot a BBSSD guest, then use the SSD over SSH. Host steps are blue, steps inside the guest green. 11
3 FEMU's six layers, the code that implements each one, and the thread that runs it; data is copied from the frontend straight to the memory backend, past the FTL and NAND model. . 15
4 FEMU's threads and the queues between them: vCPUs store doorbells, each poller owns its submission queues and its `to_ftl`/`to_poller` rings, and one FTL thread per controller times BBSSD, ZNS and CSD requests. 20
5 One 4 KiB write on BBSSD over host time: the poller copies the data at fetch, the FTL thread books a 200 µs program on the LUN from that moment, and the poller holds the completion until it is due. 21
6 One 4 KiB read on BBSSD: the guest buffer is filled at fetch and only the completion waits; below, the checks `ssd_read()` makes for each page and what each one costs. 22
7 The four BBSSD mapping schemes (`mapping=`): the full page table, DFTL's cache of map pages, BAST's log block per data block, and FAST's shared log pool. 53
8 BBSSD line states and garbage collection: when a line moves between the free, written, full and victim sets, when background and foreground GC run, and how each `gc_policy` picks a victim. 58
9 The BBSSD write buffer with `buffer_size=10` and `buffer_thres_pcent=80`: a write that finds the buffer at its watermark first programs the two oldest pages and pays for them. . . . 62
10 The NAND hierarchy (channel, LUN, plane, block, page, sector), a line as one block index across every plane of every LUN, and the order in which the write pointer hands out pages. . 78
11 How the media layer times an operation: the phases of a read, program and erase, and four reads issued together showing LUN serialization, a shared channel bus and parallel channels. 80
12 Program suspend on one LUN: with `pe_suspend=1` two reads go ahead of a running program, the first pays `tsusp_ns`, and the program finishes later by that overhead plus both reads. . . . 90
13 Choosing a mode: answer the questions top to bottom; the first "yes" names the mode, its `femu_mode` value and its launcher in `hw/femu/scripts/`. 104
14 Modes against layers: the interface each mode offers, where its FTL runs, which timing model it uses, where its data lives and which thread charges its time; below, the features each mode supports (CUs are CSD compute units). 108
15 FDP placement on a BBSSD: a write's placement identifier picks a placement handle, the namespace maps it to a reclaim unit handle, and the handle fills its current reclaim unit (one FTL line) before taking a free one. 135
16 How zones sit on NAND with the default geometry: zone k is block k on every channel and LUN, and its pages are placed channel first, then LUN. 157
17 The zone state machine in FEMU's ZNS mode: writes open zones implicitly, Open/Close/Finish/Reset move them explicitly, a zone becomes Full when its write pointer reaches the capacity, and only an injected write fault makes it Read Only. 159
18 Open-Channel SSD: the FTL runs in the guest and sends commands with physical addresses; FEMU checks them, tracks chunk state (2.0), stores the data and charges chip and channel time. 172
19 Key-value SSD: each KV namespace keeps a hash index and a value arena, and a private BBSSD model places the value pages and charges NAND time. 189

20 Computational storage: load a program, stage namespace data in a device memory buffer, execute it on a compute unit, and read the result back. . . . 206
21 One controller with three namespaces of different modes: each BBSSD or ZNS namespace builds its own FTL and NAND model, and all of them share slices of one DRAM backend. Sharing a namespace between controllers is limited to NoSSD or BBSSD without `namespace_modes`. . . . 220
22 Where `femu-cxl-ssd` sits: a CXL type-3 device under `pxb-cxl` and `cxl-rp` on a q35 machine, backed by a RAM memory backend; guest loads and stores to the fixed memory window trap into the device unless `der` has mapped the page. An optional NVMe controller can serve the same medium. . . . 243
23 One guest access to CXL SSD memory: a page mapped by `der=memslot` or `der=cylon` never exits; otherwise the device looks the page up in its DRAM cache, and a miss evicts a victim, reads the page through the FTL and makes the vCPU wait out the NAND time. . . . 246
24 The CXL caching API in BAR5: the guest puts a command in a slot, queues the slot on the request ring and rings the doorbell; the `femu-cxl-cca` thread runs it and queues the response. 256
25 One medium, two views: a linked BBSSD controller serves the CXL SSD's medium as an NVMe namespace, sharing its FTL and memory backend, and each path keeps the other's view consistent. . . . 258
26 Where FEMU's counters come from and how you read them: FTL media counters in vendor log C0h, command and byte counts in the SMART log, FDP statistics, and CXL SSD counters as QOM properties on the host. . . . 345
27 The FEMU source tree under `hw/femu/`: the NVMe controller files at the top level, one directory per mode, the shared media and library code, and the scripts, tests, tools and docs around them. . . . 370
28 Contributing a change: one logical change with a failing test and its docs, checked locally with checkpatch, `make check` and `make check-docs`, then the four CI jobs run on the pull request. 370
29 FEMU's test layers, fastest at the bottom: unit tests without QEMU, doc and config checks, qtests that run QEMU without a guest, and guest tests run by hand. . . . 371

1 Introduction

FEMU is an SSD emulator built into QEMU. It adds three devices to QEMU: `femu`, an NVMe controller; `femu-subsys`, an NVMe subsystem that groups controllers and namespaces; and `femu-cxl-ssd`, a CXL Type-3 memory device. A guest operating system finds the controllers and the CXL device on its PCIe bus and drives them with its own NVMe and CXL drivers; the subsystem is visible only through the controllers it groups. In the block modes, applications, file systems and the block layer in the guest run unchanged, so a whole storage stack can be studied on an SSD whose internals you choose (Figure 1).

This manual is modelled on the DiskSim reference manual: it explains how to run FEMU, then describes each component with its parameters, the statistics it produces and how far it has been validated, and closes with how to extend it. Its chapters are generated from the Markdown pages under `hw/femu/docs/` in the FEMU repository. Those pages are the reference; when this manual and the repository differ, the repository is right.

1.1 What FEMU does

- **Emulates several kinds of SSD.** The `femu_mode` property of `-device femu` selects one: 0 Open-Channel SSD (OCSSD), 1 BlackBox SSD with a device FTL (BBSSD), 2 NoSSD, a DRAM device with no media timing and the default, 3 Zoned Namespace (ZNS), 4 computational storage (CSD), and 5 key-value (KV). Flexible Data Placement, several namespaces per controller, namespace management, and per-block metadata with protection information are features layered on these modes. `femu-cxl-ssd` puts a DRAM page cache in front of the BBSSD FTL and exposes the result as CXL memory.
- **Charges flash time.** The modes with flash compute how long each command would take on the emulated device: NAND reads, programs and erases, channel transfers, garbage collection, and optional host-link and controller-firmware costs. For NVMe commands the data itself is copied at once; FEMU holds the completion until the host clock reaches the computed time, so the guest sees the modelled latency on its own clock. On the CXL SSD, the vCPU that made an access waits out the media time before the access completes.
- **Reports what the device did.** For BBSSD, CSD and KV, a vendor log page (C0h) gives the write amplification factor and media counters. The SMART log gives host read and write totals, wear and media errors, and the CXL SSD exposes cache counters as QOM properties readable over QMP.
- **Is configured from the command line.** Every geometry, timing and FTL parameter is a device property, documented in Appendix A. Launcher scripts in `hw/femu/scripts/` start a guest with a working configuration for most modes; KV mode has none.

1.2 What FEMU does not do

- **It does not keep data across runs.** The emulated medium lives in host memory. A guest reboot or a controller reset keeps it (unless you trigger a simulated power loss on a device with `power_loss=on`, which drops the write buffer); QEMU exiting loses it. FEMU never writes an NVMe namespace to a file. A `femu-cxl-ssd` given a file-backed memory backend leaves its bytes in that file.
- **It does not support migration or snapshots.** The devices are marked unmigratable, so QEMU refuses `migrate` and `savevm`.

- **It does not slow the host down to the model.** Latencies are lower bounds: a completion is posted on the first poller sweep after it is due, so a poller thread that does not get a host CPU adds delay on top. Section 6 explains how to get repeatable numbers.
- **It does not reproduce a particular commercial SSD.** Its timing comes from the parameters you set. Calibrating them against a real drive is your part of the experiment.
- **It does not run everywhere.** FEMU needs an x86_64 Linux host with hardware virtualization and KVM. Open-Channel mode needs a guest kernel from 4.16 to 5.14 (4.17 for Open-Channel 2.0, the default), because Linux removed LightNVM in 5.15.

1.3 Organization of this manual

Section 2, Getting started
Host requirements, building FEMU, preparing a guest image, and a first run that measures write amplification.

Section 3, Architecture and design
The layers of FEMU from the guest interface to the memory backend, its threads, and the design of the NVMe front end, the FTL and the NAND timing model.

Section 4, Modes and features
How to choose a mode, then one part per mode or feature: what it emulates, how to turn it on, its parameters and its limits.

Section 5, Running and configuring
The parameter manual, tutorials, performance tuning, and the security model and host sizing.

Section 6, Measuring and validation
The counters FEMU reports, recipes for repeatable measurements, and the timing model in detail.

Section 7, Troubleshooting
Answers to common questions and how to debug FEMU.

Section 8, Contributing
The source tree, the tests, and the checks that keep this documentation in step with the code.

Appendices
The device property reference generated from the binary, log pages and counters, the scripts shipped with FEMU, and the changelog.

1.4 Conventions

Commands, file names, properties and values are set in `monospace`. A reference such as (§4.2) points to another part of this manual. A footnote gives the GitHub address of a file in the FEMU repository that the manual does not include, or of an outside page. Command blocks are taken from the Markdown pages, where continuous integration runs most `-device femu` examples to check that they still work. Diagrams drawn in text keep their original layout and may be set in smaller type to fit the page.

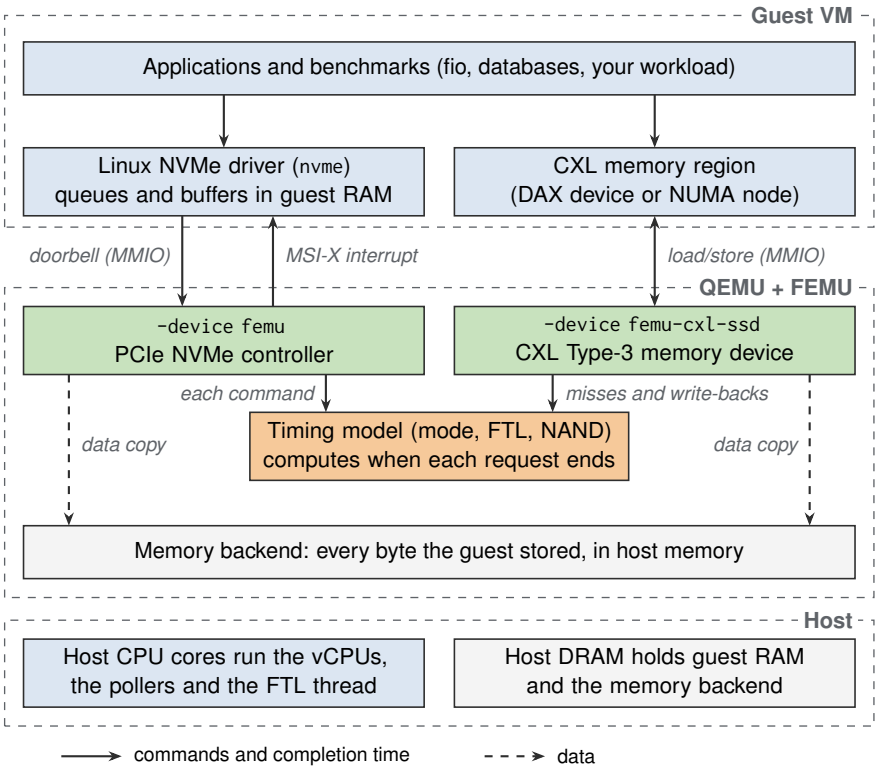


Figure 1: Where FEMU sits: the guest drives FEMU's NVMe or CXL device, the timing model decides when each request completes, and the data goes straight to the memory backend in host memory.

1.5 How to cite FEMU

If you use FEMU in your research, cite the FAST '18 paper. The same reference is in `CITATION.cff` at the top of the repository.

```
@inproceedings{Li+18-FEMU,
  author    = {Huaicheng Li and Mingzhe Hao and Michael Hao Tong and
               Swaminathan Sundararaman and Matias Bj{\o}rling and Haryadi S. Gunawi},
  title     = {{The CASE of FEMU: Cheap, Accurate, Scalable and Extensible Flash Emulator}},
  booktitle = {16th USENIX Conference on File and Storage Technologies (FAST 18)},
  year      = {2018},
}
```

If you use one of the following modes, also cite the paper it comes from: FDP comes from WARP (FAST '26, Section 4.5.8), the CXL SSD from Cylon (FAST '26, Section 4.18.11) and CSD from CEMU (ASPLOS '26, Section 4.13.9).

2 Getting started

2.1 Requirements

What the host and the guest need before you build and run FEMU. The quick start (§2.4) assumes everything on this page.

2.1.1 Host

Operating system and CPU

FEMU runs on an x86_64 Linux host with hardware virtualization (Intel VT-x or AMD-V). Run it on a physical machine. Nested virtualization works but distorts the emulated latency, and WSL is not supported.

FEMU is based on QEMU 10.1, which needs Python 3.9 or newer and GLib 2.66 or newer. Check yours:

```
python3 --version
pkg-config --modversion glib-2.0
```

Distribution	Python	GLib	Status
Ubuntu 24.04 LTS	3.12	2.80	Built in CI
Ubuntu 22.04 LTS	3.10	2.72	Built in CI
Ubuntu 20.04 and older	3.8	2.64	Cannot build with stock packages

Other distributions with new enough Python and GLib should build FEMU, but CI does not test them. `pkgdep.sh` installs dependencies on Debian and Ubuntu only; on other distributions install the equivalents of the packages listed in build.md (§2.2.2).

CI builds FEMU with `femu-compile.sh` on Ubuntu 22.04 and 24.04 (`.github/workflows/ci.yml`). It runs the unit tests and qtests and starts every mode without a guest. It does not boot a guest. The guest-level quick start (§2.4) was last run end to end on a Pop!_OS 24.04 host (Ubuntu 24.04 base) with Linux 6.18.

KVM access

The run scripts start QEMU with `-enable-kvm`. Check that KVM is loaded and that you can open it:

```
ls -l /dev/kvm
id -nG | grep -w kvm
```

If `/dev/kvm` is missing, enable virtualization in the BIOS and load the module (`sudo modprobe kvm_intel` or `sudo modprobe kvm_amd`). If you are not in the `kvm` group, add yourself and log in again:

```
sudo usermod -aG kvm "$USER"
```

The `run-*.sh` launchers run QEMU with `sudo`, so they work without the group. `make-guest-image.sh` runs without root and needs the group.

Memory

The emulated SSD lives in host DRAM. Nothing is stored in a file, and nothing written to it survives a shutdown. Plan host memory as:

```
host RAM needed = devsz_mb (the emulated SSD) + guest RAM (-m) + about 1 GiB for QEMU
```

Launcher	devsz_mb	Guest -m	Free host RAM needed
run-blackbox.sh (BBSSD)	12288	4G	about 17 GiB
run-nossd.sh, run-zns.sh, run-whitebox.sh, run-csd.sh, run-kvssd.sh	4096	4G	about 9 GiB

To fit a smaller host, lower ssd_size in run-blackbox.sh and the geometry with it (see the property reference (§A.1)).

FEMU locks the device memory (mlock) so page faults do not distort latency. Under sudo the limit does not apply. If you run QEMU as a normal user, raise the limit with ulimit -l unlimited (or /etc/security/limits.conf). Without it the device still starts and prints a warning, and latency is less precise.

Hugepages

The standard run-*.sh launchers do not need hugepages. femu-cxl-ssd with der=cylon needs a shared, preallocated hugetlb memory backend and a modified host kernel (see cxlssd.md[1]).

Disk

The source tree and one build take about 3 GiB. The guest image built by make-guest-image.sh uses about 2.5 GiB on disk (32 GiB virtual), plus a 0.6 GiB download cache.

CPU cores

The launchers give the guest 4 vCPUs (-smp 4). FEMU adds its own polling threads. 8 host cores is a comfortable minimum.

2.1.2 Guest

Kernel per mode

Mode	femu_mode	Guest kernel	Notes
NoSSD	2 (default)	any with NVMe	
BlackBox SSD (BBSSD)	1	any with NVMe	
OpenChannel SSD 1.2 / 2.0	0	4.16 to 5.14 (2.0 needs 4.17 or newer)	LightNVM was removed in Linux 5.15. On newer kernels use SPDK.
Zoned Namespace (ZNS)	3	5.9 or newer, CONFIG_BLK_DEV_ZONED=y	
Key-value (KV)	5	6.0 or newer	No block device. The namespace appears as the generic character device /dev/ngXnY. Linux 5.9 to 5.19 do not attach the namespace at all.
Computational storage (CSD)	4	any with NVMe	Guest tools in hw/femu/tests/csd/.
Flexible Data Placement (FDP)	1, with femu-subsys,fdp=on	any with NVMe	Use passthrough commands or io_uring to send placement hints.

[1] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md

Mode	femu_mode	Guest kernel	Notes
CXL SSD (femu-cxl-ssd)	not a femu_mode	CXL region support: CONFIG_CXL_BUS, CXL_PCI, CXL_ACPI, CXL_MEM, CXL_PORT, CXL_REGION, DEV_DAX, DEV_DAX_KMEM	Needs cxl-cli and daxctl in the guest.

The image from make-guest-image.sh runs Ubuntu 24.04 with Linux 6.8. It covers every mode except OCSSD.

Guest tools

- nvme-cli and fio for every NVMe mode. make-guest-image.sh installs both.
- ZNS needs nvme-cli 1.12 or newer for nvme zns.
- nvme-cli 2.8, the version in Ubuntu 24.04, double-frees and crashes after every successful Persistent Event Log read (nvme persistent-event-log). Build nvme-cli 3.x from source in the guest if you need that log. Other commands work.
- CXL: cxl-cli, daxctl and ndctl (make-guest-image.sh --cxl).

2.2 Build FEMU

How to get the source, install dependencies and build the FEMU binary. Check requirements.md (§2.1) first.

2.2.1 Clone

```
git clone https://github.com/MoatLab/FEMU.git
cd FEMU
```

The repository has no git submodules. QEMU's build downloads a few meson subprojects (for example keycodemapdb and berkeley-softfloat-3) during configure, so the first build needs network access.

2.2.2 Dependencies

On Debian and Ubuntu, pkgdep.sh installs what the build needs. It must run as root:

```
mkdir build-femu
cd build-femu
cp ../femu-scripts/femu-copy-scripts.sh .
./femu-copy-scripts.sh
sudo ./pkgdep.sh
```

femu-scripts is a link to hw/femu/scripts. femu-copy-scripts.sh copies the build helpers, the image builder and the run scripts into build-femu/.

pkgdep.sh installs gcc pkg-config git libglib2.0-dev libfdt-dev libpixman-1-dev zlib1g-dev libdw-dev libaio-dev libslirp-dev libnuma-dev ninja-build. QEMU's configure also needs python3-venv, flex and bison; install them if configure asks:

```
sudo apt install python3-venv flex bison
```

CI installs a longer list, which is a known-good superset (.github/workflows/ci.yml, step "Install dependencies"):

```
sudo apt install -y build-essential pkg-config libglib2.0-dev \
  libpixman-1-dev libfdt-dev zlib1g-dev libaio-dev \
  libcap-ng-dev libattr1-dev ninja-build python3-pip \
  libslirp-dev libseccomp-dev libcurl4-gnutls-dev \
  libiscsi-dev libnfs-dev librbd-dev librados-dev \
  libssh-dev liblzo2-dev libsnappy-dev libbz2-dev \
  liblzma-dev libzstd-dev libgcrypt20-dev libgnutls28-dev \
  uuid-dev libcap-dev libxml2-dev libmount-dev \
  liburing-dev flex bison
```

2.2.3 Compile

From `build-femu/`:

```
./femu-compile.sh
```

The script runs `make clean`, then

```
../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp \
    --disable-libnfs --disable-libiscsi --disable-curl
```

and `make` with one job per CPU. A first build takes 3 to 15 minutes, depending on the number of cores.

The binary is `build-femu/qemu-system-x86_64`. The run scripts expect it in the current directory, so run them from `build-femu/`. The build also produces `build-femu/qemu-img`, which `make-guest-image.sh` uses.

Check that the FEMU devices are registered:

```
./qemu-system-x86_64 -device help | grep femu
```

You should see the NVMe controller, the CXL SSD and the NVMe subsystem:

```
name "femu", bus PCI, desc "FEMU Non-Volatile Memory Express"
name "femu-cxl-ssd", bus PCI, desc "FEMU CXL SSD"
name "femu-subsys", desc "FEMU NVMe Subsystem (FDP)"
```

`./qemu-system-x86_64 -device femu,help` lists every property with its default; the property reference (§A.1) explains them.

2.2.4 Optional features

CSD with uBPF programs

Computational storage mode (`femu_mode=4`) loads shared-library programs with no extra build option. To also run uBPF programs, build with uBPF:

```
./femu-compile.sh --enable-csd-ubpf                    # libubpf found by pkg-config
./femu-compile.sh --enable-csd-ubpf=/path/to/ubpf-cemu  # a ubpf-cemu source tree
```

With a path, the build links `<path>/build/lib/libubpf.a`. These are the only options `femu-compile.sh` accepts.

CXL SSD

`femu-cxl-ssd` is built by default for `x86_64-softmmu`. It needs no option. Check it with:

```
./qemu-system-x86_64 -device help | grep femu-cxl-ssd
```

Debug build

`femu-compile.sh` has no debug switch. Run `configure` yourself from `build-femu/`:

```
../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp \
    --disable-libnfs --disable-libiscsi --disable-curl \
    --enable-debug --enable-debug-info
make -j"$(nproc)"
```

Add `--extra-cflags=-DFEMU_FTL_ASSERT` to turn on the FTL consistency checks that are compiled out by default.

2.2.5 Rebuild

After you change the source, rerun `make` from `build-femu/`. It rebuilds only what changed:

```
make -j"$(nproc)"
```

`./femu-compile.sh` always starts with `make clean` and reruns `configure`, so use it only for a full rebuild.

After you pull new scripts, run `./femu-copy-scripts.sh` again. It overwrites the copies in `build-femu/`, including any run script you edited there.

2.2.6 Common build errors

Message	Cause	Fix
`ERROR: Cannot find Ninja`	`ninja-build` is missing	`sudo apt install ninja-build`
`ERROR: python venv creation failed`	Debian and Ubuntu ship `venv` in a separate package	`sudo apt install python3-venv`
`ERROR: Cannot use 'python3', Python >= 3.9 is required.`	Python is older than 3.9 (Ubuntu 20.04)	Use Ubuntu 22.04 or newer
`Dependency "glib-2.0" not found` or a version below 2.66	GLib is missing or too old	`sudo apt install libglib2.0-dev`; on Ubuntu 20.04, upgrade the distribution
Errors around `nfs_pread_async`	Older FEMU trees used libnfs 6, whose API changed; the current QEMU base ignores libnfs 6	Build current `master`. `femu-compile.sh` also passes `--disable-libnfs`.
A meson subproject fails to download	No network during `configure`	Build once with network access
`pkgdep: unsupported system type`	Not Debian or Ubuntu	Install the equivalent packages by hand
A warning stops the build (`-Werror`)	A newer compiler warns where QEMU 10.1's did not	Add `--disable-werror` to the `configure` line, and please report the warning

2.3 Guest image

FEMU emulates an SSD inside a virtual machine, so you need a guest disk image. The run scripts boot `~/images/u20s.qcow2` by default. The name is historical; the image can hold any Linux release.

2.3.1 Build one with make-guest-image.sh

From `build-femu/` (after the build (§2.2)):

```
./make-guest-image.sh
```

The script:

1. downloads the Ubuntu 24.04 cloud image and checks it against Ubuntu's SHA256 sums (and their GPG signature when `gpgv` and the Ubuntu cloud-image keyring are installed);
2. boots it once without a display to create user `femu` with passwordless `sudo`, install `nvme-cli` and `fio`, and turn on the serial console;
3. powers the guest off and writes the result.

It writes these files to `~/images/`:

File	Purpose
`u20s.qcow2`	The guest disk the run scripts boot
`femu-guest-key`, `femu-guest-key.pub`	SSH key for user `femu`
`cache/`	The downloaded cloud image, reused on the next run
`seed.iso`, `provision.log`	Provisioning input and serial log, kept for debugging

It takes a few minutes and needs no root access. The host needs `curl`, `sha256sum`, `ssh-keygen`, `timeout`, `qemu-img`, and one of `cloud-localds`, `genisoimage`, `xorriso` or `mkisofs`. It uses the `qemu-system-x86_64` and `qemu-img` next to it in `build-femu/` when they exist. If tools are missing, the script names them. On Ubuntu:

```
sudo apt install curl cloud-image-utils
```

It also needs read and write access to `/dev/kvm` (see requirements.md (§2.1.1)).

Options:

```
./make-guest-image.sh --size 64G                    # disk size (default 32G)
./make-guest-image.sh --ssh-key ~/.ssh/id_ed25519.pub
./make-guest-image.sh --password femu               # also allow console login
./make-guest-image.sh --cxl                         # also install ndctl, daxctl, cxl-cli
./make-guest-image.sh --packages build-essential,git # extra Ubuntu packages
./make-guest-image.sh -o /data/images               # another output directory
./make-guest-image.sh --force                       # replace an existing image
```

`./make-guest-image.sh --help` lists every option. The script refuses to overwrite an existing image unless you pass `--force`. If provisioning fails, read `provision.log` in the output directory.

Without `--password`, user `femu` has no password, so you can log in only over SSH. The serial console in the run script's terminal shows the boot and a login prompt you cannot use. Pass `--password` if you want console login too.

2.3.2 Use an image in another place

Every `run-*.sh` script reads two variables:

- `IMGDIR`: the image directory (default `~/images`);
- `OSIMGF`: the image file (default `$IMGDIR/u20s.qcow2`).

```
OSIMGF=/data/images/u20s.qcow2 ./run-blackbox.sh
```

`make-guest-image.sh` and `run-guest-ssh.sh` read `IMGDIR` too, so one `export IMGDIR=/data/images` covers all three.

2.3.3 Kernel per mode

The cloud image runs Linux 6.8. That covers NoSSD, BBSSD, ZNS, KV, CSD, FDP and CXL. OCSSD needs a guest kernel older than 5.15, because Linux removed LightNVM in 5.15. See the kernel table (§2.1.2) for every mode.

To run OCSSD with LightNVM, install an older release (Ubuntu 20.04 ships 5.4) by hand as below, or use SPDK in the guest.

2.3.4 Log in with SSH

The run scripts forward host port 8080 to the guest's SSH port. From `build-femu/`, while a run script is running:

```
./run-guest-ssh.sh                  # interactive shell
./run-guest-ssh.sh sudo nvme list   # one command
```

`run-guest-ssh.sh` uses the key `$IMGDIR/femu-guest-key` and user `femu`. Set `SSH_PORT`, `SSH_KEY` or `GUEST_USER` to change them. It skips host key checks, because the guest is rebuilt often and listens only on localhost. The plain equivalent is:

```
ssh -i ~/images/femu-guest-key -p 8080 femu@localhost
```

Only one guest can use a port at a time. To run a second guest, or when another program holds 8080, set `SSH_PORT` for both the run script and `run-guest-ssh.sh`:

```
SSH_PORT=8081 ./run-blackbox.sh        # terminal 1
SSH_PORT=8081 ./run-guest-ssh.sh       # terminal 2
```

2.3.5 Alternatives

Prebuilt image

The FEMU team shares a prebuilt Ubuntu 20.04 image on request through the FEMU VM image form[2]. Extract it to `~/images/` and rename it to `u20s.qcow2`. It holds Ubuntu 20.04; issue #57 reports Linux 5.4, which runs OCSSD but not ZNS. Log in with `ssh -p 8080 <user>@localhost`, using the account described with the image.

Install from an ISO

This needs a display for the installer.

```
mkdir -p ~/images
cd ~/images
wget https://releases.ubuntu.com/24.04/ubuntu-24.04.3-live-server-amd64.iso
qemu-img create -f qcow2 u20s.qcow2 80G
qemu-system-x86_64 -enable-kvm -cpu host -smp 8 -m 8192 \
    -cdrom ubuntu-24.04.3-live-server-amd64.iso -boot d \
    -hda u20s.qcow2 -net nic -net user
```

If the ISO link has moved, pick the current one from `https://releases.ubuntu.com`. Install `openssh-server`, `nvme-cli` and `fio`.

The run scripts use `-nographic`, so the guest must use the serial console. Inside the installed guest, edit `/etc/default/grub`:

[2] `https://forms.gle/nEZaEe2fkj5B1bxt9`

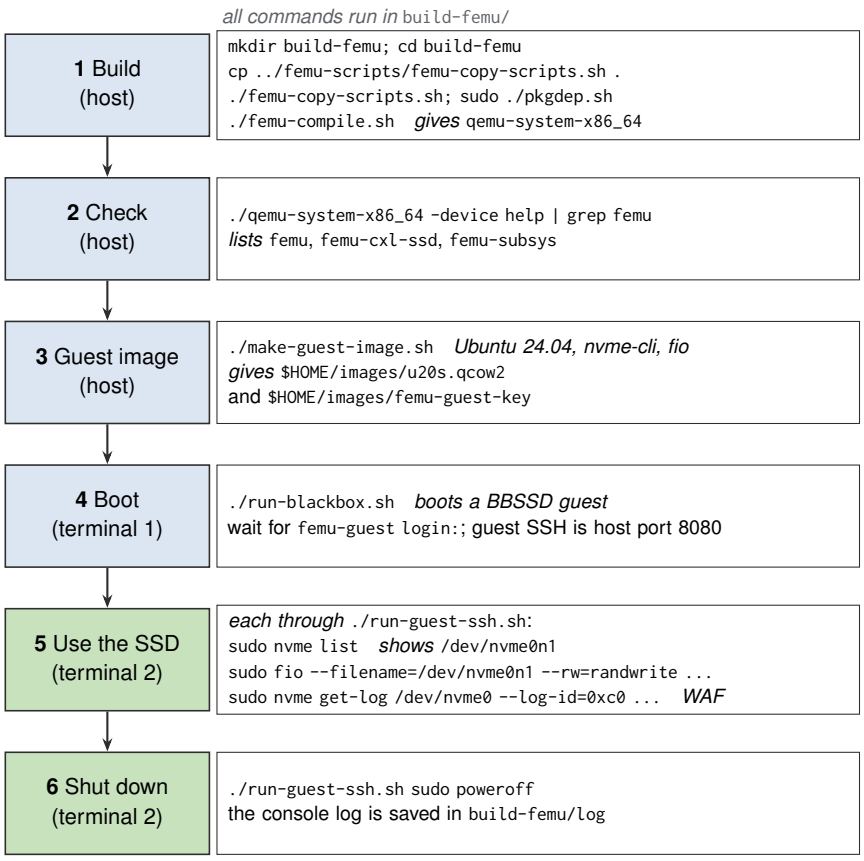


Figure 2: Quick start: build FEMU, make the guest image, boot a BBSSD guest, then use the SSD over SSH. Host steps are blue, steps inside the guest green.

```
GRUB_CMDLINE_LINUX="console=tty0 console=ttyS0,115200n8"
GRUB_TERMINAL="console serial"
GRUB_SERIAL_COMMAND="serial --unit=0 --speed=115200"
```

Then run `sudo update-grub` and power the guest off. Log in later with `ssh -p 8080 <user>@localhost`.

2.4 Quick start

From a fresh clone to a running BlackBox SSD (BBSSD). On a 20-core host the whole run took 5 minutes, most of it compiling. You build FEMU, build a guest image, boot the guest with an emulated SSD, write to the SSD with fio and read its write amplification factor (WAF).

You need an x86_64 Ubuntu or Debian host with KVM, sudo, network access, and about 17 GiB of free RAM. requirements.md (§2.1) has the details.

You use two terminals. Terminal 1 runs the virtual machine. Terminal 2 talks to it over SSH.

2.4.1 Build FEMU (terminal 2)

```
git clone https://github.com/MoatLab/FEMU.git
cd FEMU
mkdir build-femu
```

```
cd build-femu
cp ../femu-scripts/femu-copy-scripts.sh .
./femu-copy-scripts.sh
sudo ./pkgdep.sh
./femu-compile.sh
```

`femu-compile.sh` takes 3 to 15 minutes, depending on the number of cores, and ends with `===> FEMU compilation done ....` Check that the build has the FEMU device:

```
./qemu-system-x86_64 -device help | grep femu
```

```
name "femu", bus PCI, desc "FEMU Non-Volatile Memory Express"
name "femu-cxl-ssd", bus PCI, desc "FEMU CXL SSD"
name "femu-subsys", desc "FEMU NVMe Subsystem (FDP)"
```

If the build fails, see build.md (§2.2.6).

2.4.2 Build the guest image (terminal 2)

Still in `build-femu/`:

```
sudo apt install curl cloud-image-utils
./make-guest-image.sh
```

This downloads the Ubuntu 24.04 cloud image (about 600 MB) and provisions it, which takes one to a few minutes. It ends with:

```
Image ready: /home/<you>/images/u20s.qcow2
```

If it fails, read `~/images/provision.log`. guest-image.md (§2.3) explains the options and alternatives.

2.4.3 Boot the guest with a BBSSD (terminal 1)

Open a second terminal, go to the same `build-femu/` directory, and run:

```
./run-blackbox.sh
```

The script prints the `-device femu,...` line it uses, asks for your `sudo` password, and boots the guest on this terminal. Wait for the login prompt, about 30 seconds:

```
femu-guest login:
```

You cannot log in here (user `femu` has no password). Leave the terminal running.

The guest's SSH port is forwarded to host port 8080. If another program already uses 8080, QEMU stops with `Could not set up host forwarding rule`. Pick a free port and set it for both scripts, in both terminals:

```
export SSH_PORT=8081
```

2.4.4 Look at the SSD (terminal 2)

Back in terminal 2, in `build-femu/`:

```
./run-guest-ssh.sh sudo nvme list
```

```
Node                  Generic               SN                   Model                                    Namespace
    Usage                      Format           FW Rev
--------------------- --------------------- -------------------- ---------------------------------------
    ---------- -------------------------- ---------------- --------
/dev/nvme0n1          /dev/ng0n1            vSSD0                FEMU BlackBox-SSD Controller             0x1
    12.88  GB /  12.88  GB    512   B +  0 B   1.0
```

`/dev/nvme0n1` is the emulated SSD. The guest's own disk is `/dev/sda`. If SSH says `Connection refused`, the guest is still booting; wait and retry.

2.4.5 Write to it with fio (terminal 2)

```
./run-guest-ssh.sh sudo fio --name=qs --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio
   --rw=randwrite --bs=4k --iodepth=16 --size=1G
```

fio writes 1 GiB of random 4 KiB blocks and prints the bandwidth and latency it saw. The latency includes the emulated NAND program time.

2.4.6 Read the write amplification factor (terminal 2)

FEMU reports its media counters in the vendor log page C0h. The first 4 bytes are the WAF times 1000:

```
./run-guest-ssh.sh "sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4"
```

```
       1000
```

1000 means a WAF of 1.000: every page the host wrote was programmed once, and garbage collection has not run yet. The next three 8-byte counters are the pages the host wrote, the pages garbage collection moved, and the pages programmed in total:

```
./run-guest-ssh.sh "sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8
   -N 24 -w24"
```

```
               262144                    0               262144
```

Write more than the drive's capacity, and the second counter and the WAF grow. log-pages-and-counters.md (§B) lists every counter.

2.4.7 Shut down (terminal 2)

```
./run-guest-ssh.sh sudo poweroff
```

The guest powers off and `run-blackbox.sh` returns in terminal 1. The data on the emulated SSD is gone: FEMU keeps it only in host memory.

If the guest hangs, stop QEMU from terminal 1 with `Ctrl-a` then `x`.

`run-blackbox.sh` saves the console output of the run in `build-femu/log` and overwrites it on the next run.

2.4.8 Next steps

- Change the geometry and latency at the top of `run-blackbox.sh`, or pick another mode with another `run-*.sh` script. The doc map[3] lists the guides.
- Every property: reference/properties.md (§A.1).

[3] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/README.md`

3 Architecture and design

3.1 FEMU architecture

FEMU is QEMU with a set of emulated storage devices under `hw/femu/`. This page describes those devices as a stack of layers, from what the guest sees down to the host memory that holds the data. Each section says what the layer does, where its code lives, which threads run it, and where it charges time.

Two ideas explain most of the design:

- **Data and time are separate.** The guest's data is copied into a host memory buffer as soon as a command is parsed. The FTL and NAND model never touch the data; they only compute how long the command would take on a real device, and the completion is held back until that time has passed. (The power-loss model is the exception: with `power_loss=on` the FTL thread applies the data.)
- **NVMe I/O commands run on FEMU's own threads, not on the vCPU.** A doorbell write only records the new queue tail. Polling threads fetch and execute the commands, and an FTL thread computes their timing.

The CXL SSD (`femu-cxl-ssd`) is the exception to the second rule: its loads and stores are served on the vCPU thread that issued them. The CXL SSD design note[4] covers it in depth.

3.1.1 The layers

Figure 3 draws this diagram.

Solid arrows carry commands and timing. Dotted arrows carry the data, which goes straight to the memory backend.

Layer	Main source files	Threads
1. Guest-visible interface	`hw/femu/femu.c`, `hw/femu/femu-props.c`, `hw/femu/intr.c`, `hw/femu/cxlssd/qemu-adapter.c`	vCPU threads, QEMU main loop
2. Frontend	`hw/femu/nvme-io.c`, `hw/femu/nvme-admin.c`, `hw/femu/dma.c`, `hw/femu/lib/`, `hw/femu/cxlssd/cxlssd.c`, `hw/femu/cxlssd/cache.c`	`femu-poller`, vCPU threads
3. Mode backends	`hw/femu/nossd/`, `hw/femu/bbssd/bb.c`, `hw/femu/zns/zns.c`, `hw/femu/ocssd/`, `hw/femu/kvssd/`, `hw/femu/csd/`	`femu-poller`
4. FTL	`hw/femu/bbssd/`, `hw/femu/zns/zftl.c`, `hw/femu/kvssd/kvssd-ftl.c`	`FEMU-FTL-Thread`, `femu-poller` (KV), `femu-cxl-ftl`
5. NAND media timing	`hw/femu/nand/`, `hw/femu/ocssd/oc-timing.c`	the caller's thread
6. Memory backend	`hw/femu/backend/dram.c`; for CXL, a QEMU memory backend object	the thread that copies the data

3.1.2 Guest-visible interface

FEMU registers three device types.

`-device femu` is a PCIe NVMe controller. It reports NVMe version 1.4 and PCI ID `1d1d:1f1f` by default (`vid`, `did`). BAR0 holds the controller registers and the doorbells, MSI-X has one vector per queue plus the admin

[4] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

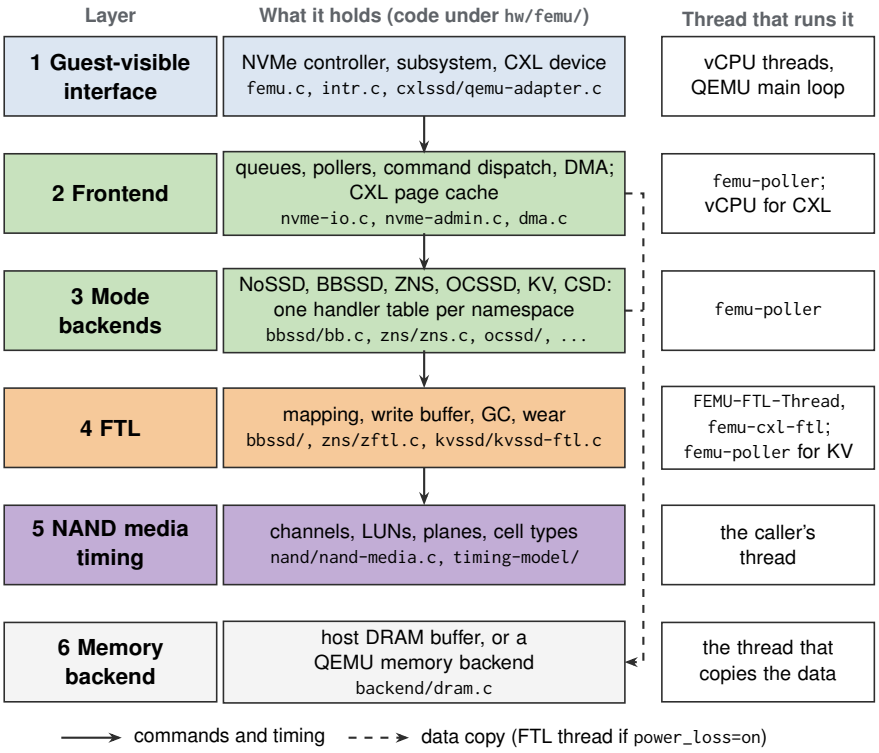


Figure 3: FEMU's six layers, the code that implements each one, and the thread that runs it; data is copied from the frontend straight to the memory backend, past the FTL and NAND model.

queue, and an optional Controller Memory Buffer (`cmbsz`) must be placed on BAR2 with `cmbloc`. `femu_mode` selects which kind of SSD the controller emulates (see layer 3 (§3.1.4) and choosing a mode (§4.1)).

Namespaces are not separate devices. There is no FEMU namespace device to put on the command line; the controller builds its namespaces at realize from `namespaces`, `namespace_sizes` and `namespace_modes`, and Namespace Management can add more at run time when `ns_mgmt=on`. Migration and snapshots are blocked for this device.

`-device femu-subsys` is an NVMe subsystem. It is not on any bus. A controller joins it with `subsys=<id>`, and the subsystem must appear first on the command line. It is needed for Flexible Data Placement (`fdp=on`) and for namespaces shared between controllers (`ns_mgmt=on` on the subsystem).

`-device femu-cxl-ssd` is a CXL Type-3 volatile memory device, a subclass of QEMU's `cxl-type3`. Its data lives in the memory backend named by `volatile-memdev`. FEMU installs an I/O region named `femu-cxl-media` over every CXL fixed memory window that can reach the device, so guest loads and stores to the device's address range reach FEMU. Optional extras are a cache control interface on BAR5 (`cca=on`) and an experiment control channel through the label storage area (`lsa-control=on`). A `femu` controller can serve the same medium as an NVMe namespace with `cxl_ssd=<id>`.

Where it lives:

- `hw/femu/femu.c`: QOM types `femu` and `femu-subsys`, `femu_realize()`, `nvme_check_constraints()`, PCI and BAR setup, MMIO and doorbell handlers (`nvme_mmio_write()`).
- `hw/femu/femu-props.c`: the help text of the `femu` and `femu-subsys` properties. The properties and their defaults are defined in `femu.c` (`femu_props[]`). The generated property reference (§A.1) is built from

the binary.
- hw/femu/intr.c: MSI-X, MSI and pin interrupts (nvme_isr_notify_io()).
- hw/femu/cxlssd/qemu-adapter.c: QOM type femu-cxl-ssd, realize checks, window overlay and address decoding, and its properties (cxl_props[]). hw/femu/cxlssd/props.c: their help text.

Threads: register and doorbell writes are MMIO exits handled on the vCPU thread that made them, under QEMU's big lock (BQL).

3.1.3 Frontend

NVMe queues and the admin path

The admin queue is handled synchronously. A write to the admin submission queue doorbell calls nvme_process_sq_admin() on the vCPU thread, which runs nvme_admin_cmd() in hw/femu/nvme-admin.c for each new entry. Commands the generic code does not know go to the mode's admin hook, for example the BBSSD vendor command 0xEF.

I/O queues are handled asynchronously. A write to an I/O submission queue doorbell only stores the new tail (nvme_process_db_io()). The host can also set up shadow doorbells with the standard Doorbell Buffer Config command (nvme_set_db_memory()); a queue with a shadow doorbell is then read from that buffer, and FEMU publishes EventIdx values so the host can skip doorbell writes it does not need.

Pollers

When the host enables the controller, nvme_start_dataplane() creates the poller threads (femu-poller) once. Each poller loops over the I/O queues it owns (nvme_poller() in hw/femu/nvme-io.c). How many pollers there are is set by two properties:

multipoller_enabled	Pollers	Queues per poller
0 (default)	1	all I/O queues
1	ceil(queues / poller_ratio)	queues i, i + N, i + 2N, ... for poller i of N

Any other value fails realize. A poller fetches commands from a queue without a lock, so every queue must have exactly one owner.

Each poller owns three structures, all indexed by poller number:

- to_ftl[i]: a ring of requests waiting for timing.
- to_poller[i]: a ring of timed requests coming back from the FTL thread.
- pq[i]: a priority queue of timed requests, ordered by completion time.

The rings are in hw/femu/lib/rte_ring.c and the priority queue in hw/femu/lib/pqueue.c.

I/O dispatch

For each new submission queue entry, nvme_process_sq_io():

1. copies the command and stamps the request with the current host time (QEMU_CLOCK_REALTIME) as both its start time and its completion time;
2. calls nvme_io_cmd(), which handles the commands every block mode shares (Flush, Dataset Management, Compare, Write Zeroes, Copy, Verify, Write Uncorrectable, I/O Management) and passes the rest to the namespace's mode (ns->ext_ops.io_cmd);

3. puts the request on `to_ftl[i]`.

For Read and Write on NoSSD, BBSSD and CSD, the mode handler calls `nvme_rw()`, which checks the command, maps the guest's PRP or SGL list (`hw/femu/dma.c`) and copies the data between guest memory and the memory backend with `backend_rw()`. The data transfer is finished at this point, on the poller thread, before any time has been charged. ZNS and OCSSD have their own read and write paths that copy to the same backend, and KV copies values into a store of its own.

On a NoSSD controller, NoSSD namespaces skip the rings: the poller posts the completion inside the same sweep, unless the host-link or firmware-CPU model is enabled.

Completion

`nvme_process_cq_cpl()` runs at the end of every sweep. It drains the timed requests (from `to_poller[i]` when an FTL thread exists, otherwise straight from `to_ftl[i]`), adds the optional host-link and firmware-CPU time, and inserts them into `pq[i]`. Then it posts every request whose completion time has passed and raises one interrupt per completion queue it posted to. A request that is not due yet stays in the queue for the next sweep. This is where FEMU enforces latency: nothing sleeps, the completion is simply not posted before its time.

CXL accesses

The CXL device has no queues. A guest load or store to its range is an MMIO access on the vCPU thread, routed through the host bridge and endpoint decoders to `femu_cxl_access()` in `hw/femu/cxlssd/cxlssd.c`. A page-granular cache (`hw/femu/cxlssd/cache.c`, policies `fifo`, `lifo`, `clock`, `s3-fifo`) decides whether the access needs the FTL. The cache keeps only page numbers and dirty bits; the data is always in the memory backend. With `der=memslot` or `der=cylon`, cached pages can be mapped into the guest so that later accesses do not reach QEMU at all (see the CXL load walkthrough (§3.1.11)).

3.1.4 Mode backends

A mode is a table of handlers (`FemuExtCtrlOps` in `hw/femu/nvme.h`). `nvme_register_extensions()` in `hw/femu/femu.c` installs the table for the controller's `femu_mode`, and `nvme_register_extensions_ns()` gives each namespace the table for its own mode, so one controller can mix modes with `namespace_modes`. I/O commands are dispatched on the namespace's mode, not the controller's.

Mode	`femu_mode`	Code	What the I/O handler does	Where time is computed
OCSSD	0	`hw/femu/ocssd/oc12.c` (`lver=1`), `hw/femu/ocssd/oc20.c` (`lver=2`)	Open-Channel vector commands; the host runs the FTL	in the poller, from chip and channel busy-until times (`hw/femu/nand/nand-media.c`)
BBSSD	1	`hw/femu/bbssd/bb.c`	Read and Write through `nvme_rw()`	FTL thread, `bb_ftl_process_req()`
NoSSD	2	`hw/femu/nossd/nop.c`	Read and Write through `nvme_rw()`	none
ZNS	3	`hw/femu/zns/zns.c`	zoned command set, zone state machine	FTL thread, `zns_ftl_process_req()`
CSD	4	`hw/femu/csd/csd.c`	computational storage commands plus block I/O	FTL thread for NAND; programs and compute-unit time on the `femu-csd-cu` threads

Mode	femu_mode	Code	What the I/O handler does	Where time is computed
KV	5	hw/femu/kvssd/kvssd.c	key-value command set: Store, Retrieve, Delete, Exist, List	in the poller, through the BBSSD NAND model

Features that are not modes:

- **Flexible Data Placement**: enabled on femu-subsys; only BBSSD places data by reclaim unit (hw/femu/bbssd/ftl-fdp.c).
- **Several namespaces and per-namespace modes**: namespaces, namespace_sizes, namespace_modes (nvme_init_namespaces() in hw/femu/femu.c).
- **Namespace Management**: ns_mgmt=on on a NoSSD or BBSSD controller.
- **Metadata and protection information**: meta, mc, pi (hw/femu/nvme-pi.c), NoSSD and BBSSD only.
- **Streams**: streams=on (hw/femu/nvme-streams.c).
- **Persistent Event Log**: always on, kept in pel_file if set (hw/femu/nvme-pel.c).

Choosing a mode (§4.1) lists which of these combine.

Threads: one FEMU-FTL-Thread per controller, started at realize only when a namespace is BBSSD, ZNS or CSD, or the controller shares a BBSSD subsystem (femu_needs_ftl_thread()). It reads every poller's to_ftl[i] ring, calls femu_ftl_process_req(), adds the returned latency to the request's completion time and puts it on to_poller[i]. A controller whose namespaces are all NoSSD, OCSSD or KV has no FTL thread.

3.1.5 FTL

BBSSD

The BBSSD FTL in hw/femu/bbssd/ is a page-level device FTL. CSD uses it unchanged, and the CXL SSD runs a private instance of it. Its state is struct ssd (hw/femu/bbssd/ftl.h).

- **Geometry.** Channels (nchs) hold LUNs (luns_per_ch), LUNs hold planes (pls_per_lun), planes hold blocks (blks_per_pl), blocks hold pages (pgs_per_blk) of secs_per_pg sectors of secsz bytes. With the defaults a page is 4 KiB. A line (superblock) is the same block index on every plane of every LUN, so there are blks_per_pl lines.
- **Write pointer.** By default new pages are allocated from the current line in the order channel, then LUN, then plane, then page, so consecutive pages land on different channels and LUNs and can be programmed in parallel. plane_first puts the plane first, so pages in a row share a LUN and can form one multi-plane program.
- **Mapping** (mapping): page (default, a full table in memory), dftl (a cached mapping table of mapping_cache_mb whose misses cost NAND reads), hybrid and fast (log-block schemes). Code: hw/femu/bbssd/ftl-map*.c.
- **Write buffer** (buffer_size, in pages): a DRAM buffer that accepts writes. Once it reaches buffer_thres_pcent, the next write that needs a slot programs a batch of the least recently written pages. Without power_loss it holds page numbers only, since the data is already in the memory backend. FUA writes and stream writes go straight to NAND, and Flush drains it. Code: hw/femu/bbssd/ftl-datapath.c.
- **Read cache** (read_cache_mb, cache_evict): a DRAM cache of recently read pages (hw/femu/bbssd/ftl-cache.c).
- **Garbage collection.** After every request the FTL thread runs one background GC pass if the share of

lines in use has reached `gc_thres_pcent`. A write that finds it at `gc_thres_pcent_high` first runs GC in the foreground until it drops below. The victim line is chosen by `gc_policy` (`greedy` by default). GC reads, programs and erases occupy the same LUNs as host I/O. Code: `hw/femu/bbssd/ftl-line-gc.c`.
- **Placement extras**: `hot_cold_sep` writes overwritten pages to separate lines, `read_reclaim_limit` and `retention_limit_sec` rewrite lines that were read too often or hold old data, and FDP maps reclaim units to lines.
- **Wear**: each block counts its erases. SMART Percentage Used compares the average erase count with `pe_cycles_rated`, `nand_bad_blocks` lowers Available Spare, and `ecc_step_ns` makes reads of worn or old blocks slower.

Entry point: `bb_ftl_process_req()` in `hw/femu/bbssd/ftl.c`, which dispatches to `ssd_read()`, `ssd_write()`, `ssd_trim()` and the others in `hw/femu/bbssd/ftl-datapath.c`.

Other FTLs

- **ZNS** (`hw/femu/zns/zftl.c`): zones map onto blocks striped over `zns_chnls_per_zone` channels. There is no device GC; the host resets zones. Writes collect in per-zone SRAM write caches (`zns_num_wc`) and are programmed when a cache fills or is evicted. Zone Reset erases the zone's blocks.
- **KV** (`hw/femu/kvssd/kvssd-ftl.c`): a hash index of keys, with values stored on NAND lines laid out with the BBSSD geometry and timing.
- **OCSSD** has no device FTL. The host addresses channels, LUNs, blocks and pages directly.

Threads: `FEMU-FTL-Thread` runs the BBSSD, CSD and ZNS FTLs. The CXL SSD's FTL runs on its own `femu-cxl-ftl` worker; when an NVMe controller is linked with `cxl_ssd=`, both threads use the same FTL and take turns on one mutex.

3.1.6 NAND media timing

`hw/femu/nand/nand-media.c` turns one NAND operation into a completion time. `nand_media_op()` takes a location (channel, LUN, plane, block, page), an operation (read, program, erase) and a start time. It keeps a busy-until time per LUN (BBSSD, CSD, KV) or per plane (ZNS): the operation starts when both the request and that unit are ready, and the unit stays busy until it ends. Units that differ run in parallel. A channel bus with command, data and status phases is modelled only when one of those phases has a non-zero time.

Read, program and erase times come either from flat properties or from built-in per-cell-type tables (`hw/femu/nand/nand.h` for BBSSD, CSD and KV; `hw/femu/zns/zns.h` for ZNS). OCSSD also uses the media layer, with its `flash_type` times set in `hw/femu/ocssd/oc-timing.c`.

Timing model (§6.2) explains the rules and the properties that control them.

Threads: none of its own. It runs on whichever thread asks: the FTL thread for BBSSD, CSD and ZNS, the poller for KV and OCSSD, and the `femu-cxl-ftl` worker for the CXL SSD.

3.1.7 Memory backend

For `femu`, the data lives in one host buffer of `devsz_mb` MiB (the full NAND capacity for BBSSD with `op_pcent`), allocated zero-filled at realize and locked into RAM when `RLIMIT_MEMLOCK` allows (`init_dram_backend()` in `hw/femu/backend/dram.c`). Namespaces are slices of it, packed back to back. Controllers that share namespaces through a subsystem share one buffer. The block modes read and write it with `backend_rw()`; KV keeps its values in a separate buffer of its own. Nothing is written to a file, so the contents are lost when QEMU exits. `FEMU_MBE_INTERLEAVE` controls its NUMA placement (see the environment variables (§A.1.5)).

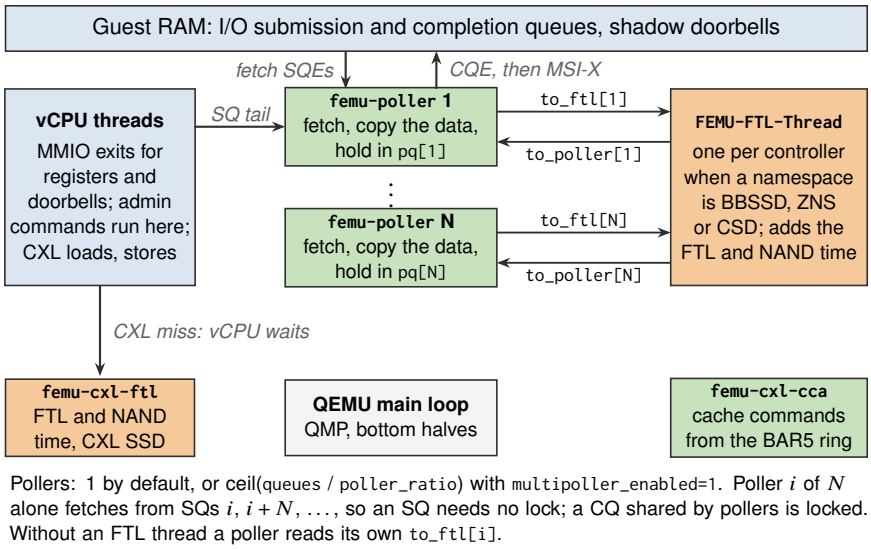


Figure 4: FEMU's threads and the queues between them: vCPUs store doorbells, each poller owns its submission queues and its `to_ftl`/`to_poller` rings, and one FTL thread per controller times BBSSD, ZNS and CSD requests.

For `femu-cxl-ssd`, the data lives in the QEMU memory backend object named by `volatile-memdev`, for example `-object memory-backend-ram,id=cxlmem,size=4G`. It must be a non-zero multiple of 256 MiB and at most 120 GiB. `der=cylon` needs a shared, preallocated hugetlbfs backend. The vCPU thread copies data to and from it directly. A `femu` controller linked with `cxl_ssd=` uses this backend instead of allocating its own.

3.1.8 Threads

Thread name	Created by	How many	Runs
vCPU threads and the QEMU main loop	QEMU	one per vCPU, plus one main loop	MMIO: controller registers, doorbells, admin commands; CXL accesses and their media wait; QMP and monitor commands
`femu-poller`	`nvme_start_dataplane()` when the host enables the controller	1, or ceil(`queues` / `poller_ratio`)	I/O command fetch and execution, data copy, completion and interrupts
`FEMU-FTL-Thread`	`femu_realize()`	0 or 1 per controller	BBSSD, CSD and ZNS FTL and NAND timing
`femu-cxl-ftl`	`femu_cxl_start()`	1 per `femu-cxl-ssd` with `ftl=on`	FTL and NAND timing for cache misses and write-backs
`femu-cxl-cca`	`femu_cxl_cca_start()`	1 per `femu-cxl-ssd` with `cca=on`	cache control commands from BAR5
`femu-csd-cu`	`csd_init()`	`nr_cu` per controller with a CSD namespace	CSD programs (Execute) and their compute-unit time

3.1.9 Where latency is charged and enforced

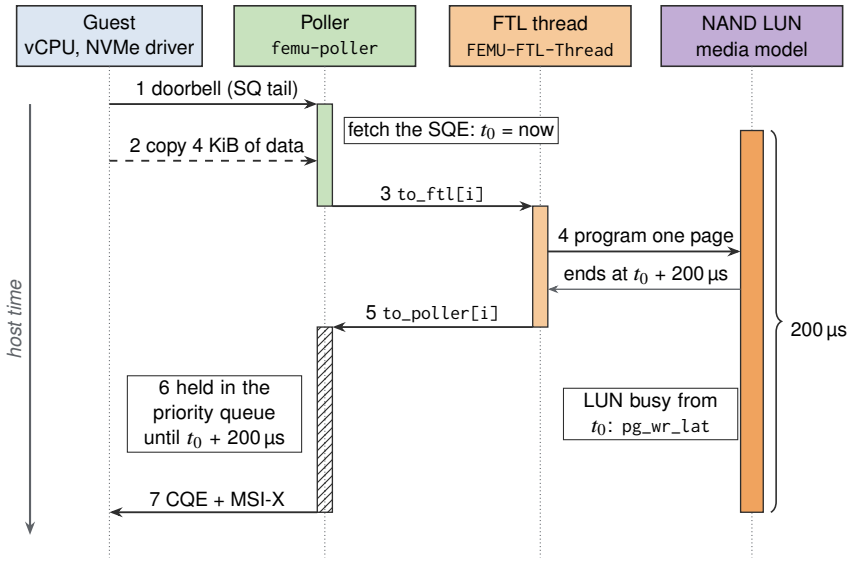


Figure 5: One 4 KiB write on BBSSD over host time: the poller copies the data at fetch, the FTL thread books a 200 µs program on the LUN from that moment, and the poller holds the completion until it is due.

Device or mode	Computed in	Enforced by
NoSSD	nothing to compute	completion posted in the same sweep
BBSSD, CSD, ZNS	`FEMU-FTL-Thread` adds the FTL's latency to the completion time; for a CSD Execute, the `femu-csd-cu` thread that ran the program adds its compute-unit time instead	the poller posts once the time has passed
OCSSD, KV	the poller, while executing the command	the poller posts once the time has passed
Host link, firmware CPU (`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`, `fw_cpu_ns`)	the poller, before queueing the completion	same
`femu-cxl-ssd`	`femu-cxl-ftl` returns the media time of a miss or write-back	the vCPU waits it out before the load or store completes

3.1.10 I/O walkthrough: one 4 KiB write on BBSSD

Assume the defaults: `femu_mode=1`, one poller, `buffer_size=0`, 4 KiB pages, `pg_wr_lat=200000` (200 us), no channel bus model.

1. **Submit.** The guest NVMe driver writes the command into its submission queue in guest memory and writes the new tail to the queue's doorbell. The vCPU exits to QEMU, and `nvme_process_db_io()` stores the tail. Nothing else happens on the vCPU.
2. **Fetch and copy.** The poller sees the new tail on its next sweep. `nvme_process_sq_io()` copies the command and stamps it with the current time `t0`. `nvme_io_cmd()` passes it to `bb_io_cmd()`, which calls `nvme_rw()`: the LBA range and size are checked, the PRPs are mapped, and `backend_rw()` copies the 4 KiB from guest memory into the memory backend at the namespace's offset. The data is now where later reads will find it.
3. **Hand to the FTL.** The poller puts the request on `to_ftl[1]` and goes on with other queues.

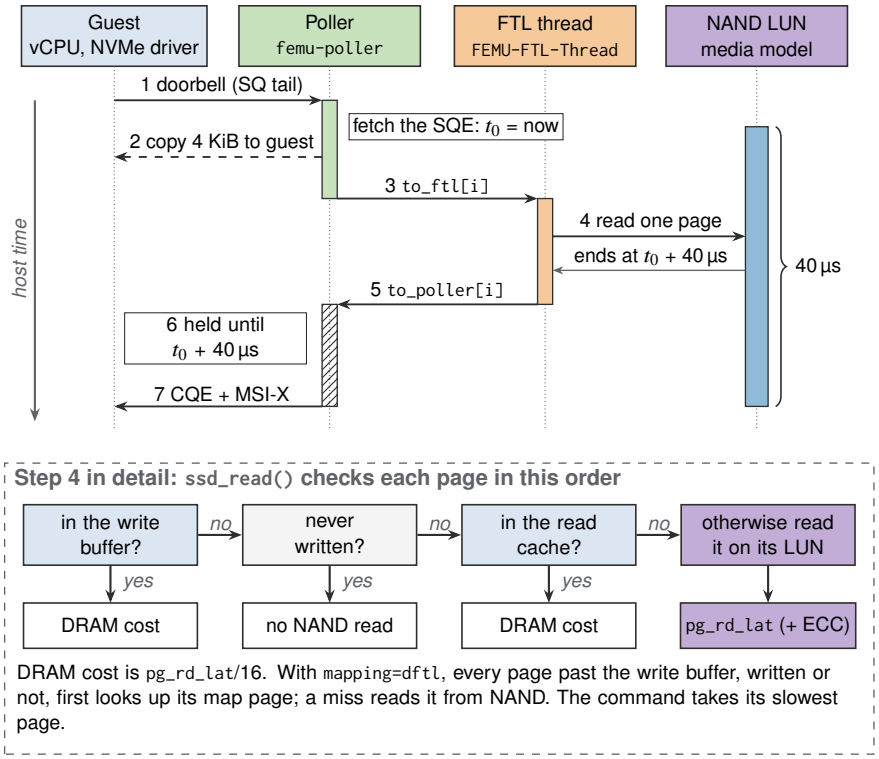


Figure 6: One 4 KiB read on BBSSD: the guest buffer is filled at fetch and only the completion waits; below, the checks `ssd_read()` makes for each page and what each one costs.

4. **Time it.** The FTL thread takes the request and calls `bb_ftl_process_req()`, which calls `ssd_write()`. If too few lines are free, foreground GC runs first. The one logical page is mapped to the next physical page at the write pointer, the old copy (if any) is marked invalid, and `ssd_advance_status()` asks the media layer for a program on that page's LUN. If the LUN is idle the program takes 200 us from `t0`; if it is busy until `t1`, it ends at `t1` + 200 us. The LUN stays busy until then. If enough lines are now in use, one background GC pass follows; its NAND operations make the LUNs they use busy for later requests.
5. **Return.** The FTL thread adds the latency to the request's completion time and puts it on `to_poller[1]`.
6. **Hold.** On a following sweep, `nvme_process_cq_cpl()` moves the request into the priority queue. It stays there until the host clock passes its completion time.
7. **Complete.** The poller writes the completion entry into the guest's completion queue and, at the end of the sweep, raises the queue's interrupt. The guest sees the write complete about 200 us after the poller fetched it on an idle device.

With `buffer_size` set, step 4 puts the page in the write buffer instead and charges a DRAM access (`pg_rd_lat` / 16, 2.5 us with the defaults). The program happens on a later write that pushes the buffer past its threshold, and that write pays for it.

3.1.11 CXL load walkthrough

The guest loads 8 bytes from a `femu-cxl-ssd` region at host physical address `hpa`. With the cache on (`cache-pages` > 0):

1. **Trap.** Unless the page is mapped directly (below), the load exits to QEMU on the vCPU thread. The `femu-cxl-media` overlay routes it through the window, host bridge and endpoint decoders to a device

address, and calls `femu_cxl_access()`.

2. **Hit.** If the page is in the cache, no media time is charged. The vCPU copies the 8 bytes from the memory backend and returns. The cost is the MMIO exit itself, paid on every access.

3. **Miss.** The page must be brought into the cache:

   - The page is read from NAND: the vCPU drops the BQL, queues a read for the `femu-cxl-ftl` worker, and waits for the latency it returns. A page the FTL has never mapped costs nothing to read.
   - If the page's set is full, the policy evicts a victim. A dirty victim is written back, which costs a NAND program.
   - `femu_cxl_delay()` waits out the rest of the media time, sleeping until 100 us before the deadline and spinning for the rest, with the BQL dropped so other vCPUs keep running.
   - The vCPU copies the data from the memory backend. With `prefetch-degree` set, the next pages are inserted into the cache with no NAND read.

   A store follows the same path and marks the page dirty. A store miss reads the page first, as a write-allocate cache does.

What happens to the next access to the same page depends on `der`:

`der`	After a miss or hit	Next access to the page
`off` (default)	nothing	traps again; a hit with no media time
`memslot`	the page is mapped as a one-page KVM memory slot alias (at most 1024 for all devices)	goes straight to host memory with no exit and no time charged; the page counts as dirty, so its eviction costs a program
`cylon`	the page is mapped by writing the guest's EPT entry through a Cylon host kernel	goes straight to host memory; dirty state comes from EPT dirty bits

A direct mapping is removed when its page is evicted, flushed or invalidated, and the next access traps again. Both direct modes map pages only in the single-endpoint topology the design note describes; elsewhere accesses stay on MMIO. `der=memslot` fails realize under TCG. `der=cylon` fails realize without `cylon-kernel-ack=on`, and falls back to MMIO with a warning when the Cylon host kernel or the hugetlbfs backend is missing. The CXL SSD design note[5] gives the details.

3.1.12 Related pages

- Choosing a mode (§4.1)
- Timing model (§6.2)
- CXL SSD design note[6]
- Device property reference (§A.1)

3.2 FEMU design

This is the design document for FEMU's emulated storage devices. It explains how FEMU is built, one component per chapter, so you can predict what a configuration will do, read the numbers it reports, and

[5] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`
[6] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

change the code.

Read this page first. It says what FEMU models and what it leaves out, shows how the components fit together, follows one request from the guest to the memory backend and back, and gives the reasons for the main design choices. Each component then has its own chapter (see Design pages (§3.2.6)). Every chapter has the same parts: purpose, place in the hierarchy, data structures, algorithms, threads and where time is charged, parameters, counters, validation, limits, how to extend it, and a source map.

For a shorter tour, read Architecture (§3.1). For the property list, read the generated property reference (§A.1). Where this document and the code disagree, the code is right; please report it.

3.2.1 What FEMU models

FEMU is QEMU with extra device models under `hw/femu/`. A guest operating system runs unmodified on KVM and sees a storage device built from these models. FEMU models:

- **The host interface.** A PCIe NVMe controller (`-device femu`) with admin and I/O queues, doorbells, shadow doorbells, MSI-X, MSI and pin interrupts, Identify, features, log pages and asynchronous events. A CXL Type-3 memory device (`-device femu-cxl-ssd`) whose loads and stores reach an SSD model.
- **Several kinds of SSD.** `femu_mode` selects one: a conventional SSD with a device FTL (BBSSD), a Zoned Namespace SSD (ZNS), an Open-Channel SSD (OCSSD), a key-value SSD (KV), computational storage (CSD), or a plain memory-backed drive with no media timing (NoSSD). Namespaces of one controller can run different modes.
- **The FTL.** Address mapping (page-level, DFTL, two log-block schemes), a write buffer, a read cache, garbage collection with several victim policies, wear, read and retention reclaim, Flexible Data Placement.
- **NAND timing.** Channels, LUNs, planes, blocks and pages; per-cell-type read, program and erase times; a channel bus; program and erase suspend; ECC read retries as extra time.
- **Optional host-side costs.** A host link with a bandwidth and a propagation delay, and a single controller CPU with a fixed cost per command.

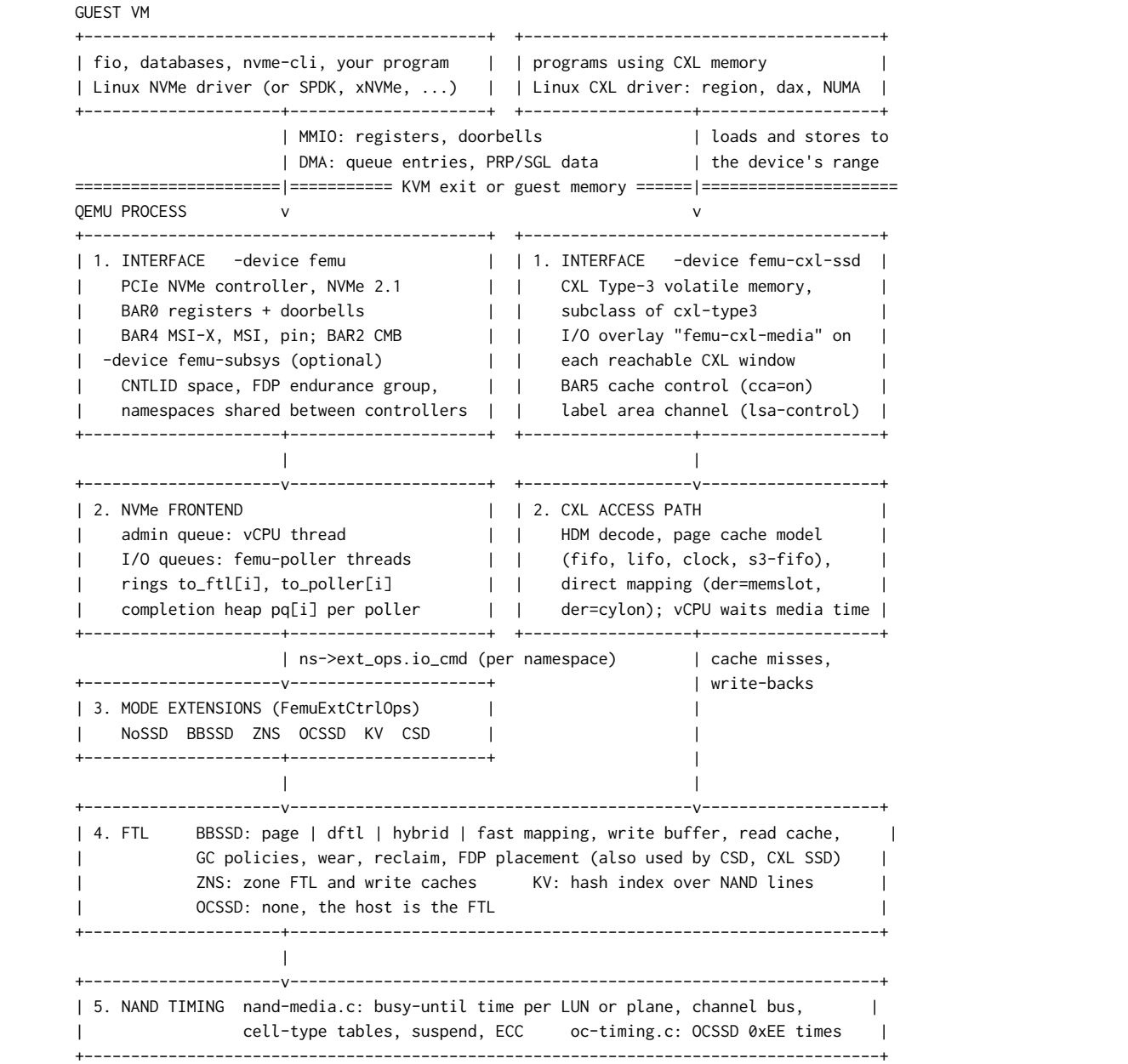
3.2.2 What FEMU does not model

- **Cycles.** Nothing is simulated instruction by instruction or clock by clock. Each operation has a cost in nanoseconds, and FEMU holds the completion until that much host time has passed (see Design choices (§3.2.7)).
- **Persistent data.** Device data lives in host memory. A guest reboot or a controller reset keeps it; QEMU exiting loses it. Migration and snapshots are refused (Security and limits (§5.13.2)).
- **Data errors from the medium.** The media model never changes a byte. ECC shows up as extra read time (`ecc_step_ns`). The medium never fails a command on its own: failures come from the injection properties (`err_read_unc_ppm`, `err_write_fail_ppm`) or from rules the host sees (Write Uncorrectable, protection information, Open-Channel chunk state).
- **The PCIe protocol.** There are no TLPs, credits or link training. The optional host-link model is a bandwidth plus a fixed delay.
- **Controller firmware internals.** The optional firmware-CPU model is one fixed cost per Read, Write and Zone Append on one modelled core.
- **Power and heat.** The controller reports one power state, and the temperature is the value of the `temperature` property.

- **Firmware update and security commands.** Firmware Download and Commit, Security Send and Security Receive are refused.
- **Exact guest-visible latency.** FEMU enforces a lower bound. A completion is posted on the first poller sweep after it is due, so a poller that does not get a host core adds time (Timing model (§6.2)).

3.2.3 Component hierarchy

Everything FEMU adds sits inside the QEMU process. The guest reaches it in two ways: NVMe through PCIe registers and DMA, and CXL through loads and stores to host-managed device memory. Below the interface the two paths share the same FTL and NAND timing code, each device with its own instance.



Layer	What it does	Chapter
1. Interface	PCI function, registers, CXL decode, subsystem	nvme-frontend.md (§3.3), cxl-ssd.md (§4.19), namespaces.md (§4.16)
2. Frontend	queues, pollers, dispatch, completion timing, interrupts	nvme-frontend.md (§3.3)

Layer	What it does	Chapter
3. Mode extensions	the command set and behaviour of each kind of SSD	ocssd.md (§4.10), zns.md (§4.8), kvssd.md (§4.12), csd.md (§4.14), nossd.md (§4.3), fdp.md (§4.6)
4. FTL	mapping, buffering, GC, wear	ftl.md (§3.4)
5. NAND timing	how long each NAND operation takes	nand-timing.md (§3.5)
6. Memory backend	where the bytes live	this page, namespaces.md (§4.16)

Device types

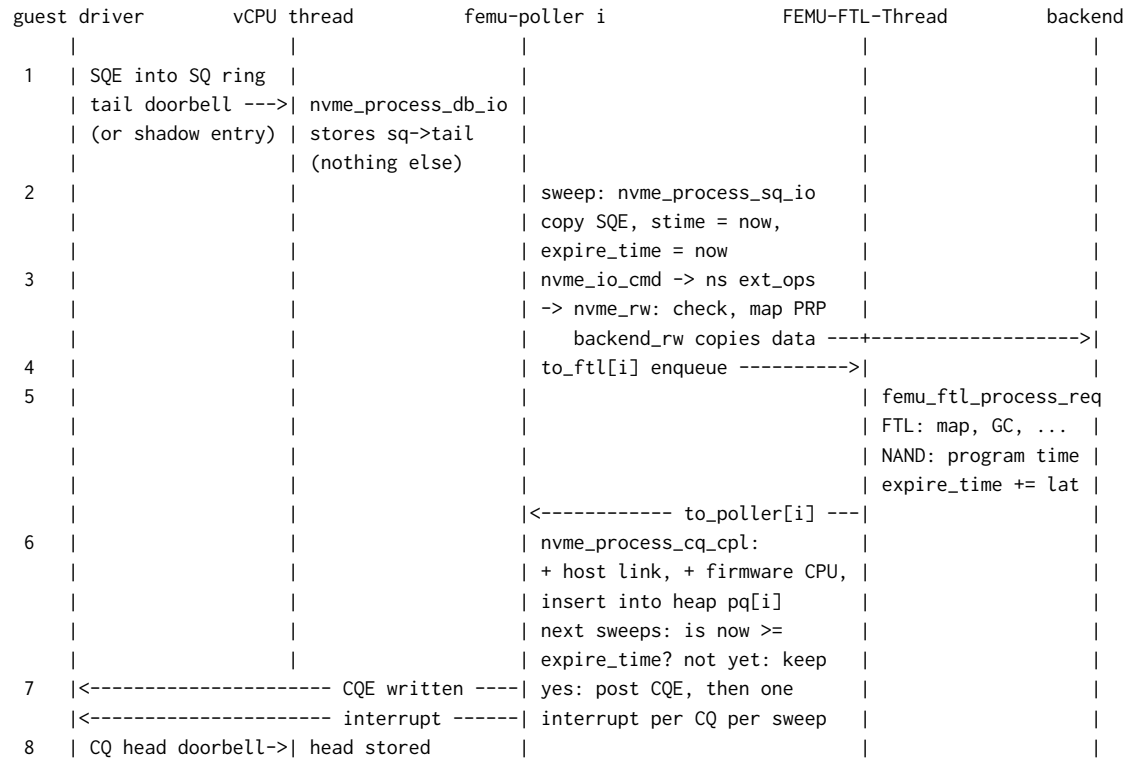
QOM type	Parent	On the command line	Role
`femu`	`TYPE_PCI_DEVICE`, PCIe endpoint	`-device femu,...`	the NVMe controller and all its namespaces
`femu-subsys`	`TYPE_DEVICE`, on no bus	`-device femu-subsys,id=s0,...` before the controllers	joins controllers (`subsys=s0`), holds FDP and shared namespaces
`femu-cxl-ssd`	`cxl-type3`	`-device femu-cxl-ssd,volatile-memdev=...` on a CXL root port	a CXL memory device backed by an SSD model

There is no namespace device: a controller builds its namespaces from `namespaces`, `namespace_sizes` and `namespace_modes`, and Namespace Management adds more at run time. `femu-subsys` creates an `nvme-bus` (a type FEMU borrows from QEMU's own NVMe controller) but no FEMU device plugs into it. A `femu` controller can also serve a `femu-cxl-ssd` medium as its one namespace with `cxl_ssd=<id>`; it then uses the medium's memory and FTL instead of its own.

3.2.4 How a request flows

One NVMe write on a BBSSD namespace

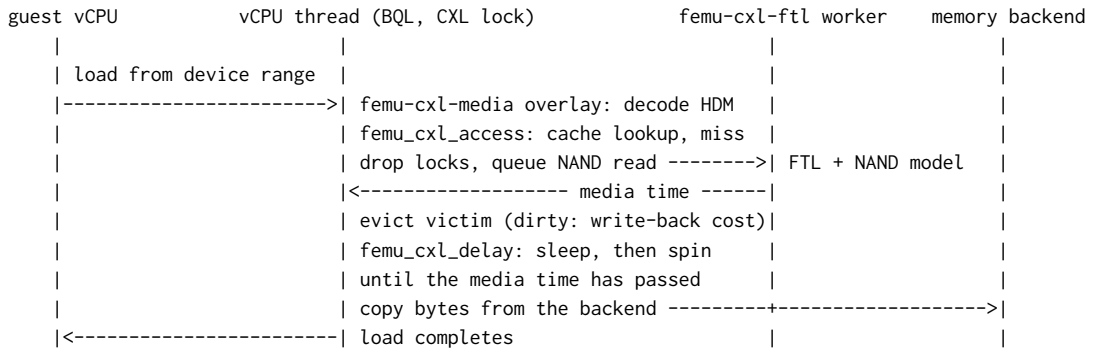
The data moves first; the time is computed afterwards and enforced at completion. Time runs downward.



1. The guest writes a submission queue entry and rings the tail doorbell. The vCPU exits to QEMU, which stores the new tail and returns. With shadow doorbells the guest also writes the value to its own memory and rings the register only when the EventIdx protocol asks for it.
2. The poller that owns the queue finds the new entry on its next sweep and stamps the request with the host time.
3. The command is checked and dispatched to the namespace's mode. For a block Read or Write the data is copied between guest memory and the backend now, on the poller thread. (With `power_loss=on` the FTL thread executes the command, data copy included, in step 5.)
4. The request goes to the FTL thread through a lock-free ring.
5. The FTL decides where the page goes, runs GC if needed, and asks the NAND model when the program would end. The difference is added to the request's completion time.
6. Back on the poller, optional host-link and firmware-CPU time is added, and the request waits in a min-heap ordered by completion time.
7. Once the host clock passes that time, the poller writes the completion entry and raises the queue's interrupt.
8. The guest consumes the entry and moves the completion queue head.

Modes differ only in steps 3 to 5. NoSSD charges nothing and, on a NoSSD controller with the default settings, posts the completion inside step 3's sweep. OCSSD and KV compute their time on the poller in step 3 and skip the FTL thread. ZNS and CSD use the FTL thread like BBSSD. nvme-frontend.md (§3.3) has the full command lifecycle.

One CXL load that misses



A hit skips the worker and the wait. With direct mapping (`der`), subsequent accesses to a cached page do not leave the guest at all. cxl-ssd.md (§4.19) covers the cache, the decoders and direct mapping.

3.2.5 Threads and where time is charged

```
vCPU threads ----- MMIO: registers, doorbells, admin queue, AER, CXL accesses (+ wait)
main loop -------- QMP, interrupt bottom halves without irqfd, AER posting, CXL BHs
femu-poller x N -- I/O fetch, command execution and data copy (not with power_loss=on),
                   OCSSD/KV/CSD-compute time,
                   host-link and firmware-CPU time, completion posting, interrupts
FEMU-FTL-Thread -- BBSSD, CSD and ZNS FTL and NAND time (0 or 1 per controller);
                   with power_loss=on it also executes each I/O command; for a
                   cxl_ssd link it calls into the medium's FTL
femu-cxl-ftl ----- FTL and NAND time for CXL misses and write-backs (1 per medium)
femu-cxl-cca ----- cache control commands from BAR5 (only with cca=on)
```

Device or mode	Time computed in	Time enforced by
NoSSD	nothing to compute	poller heap, usually in the same sweep
BBSSD, ZNS, CSD NAND time	`FEMU-FTL-Thread`	poller heap
OCSSD, KV, CSD compute units and CSD memory-copy reads	the poller, while executing the command	poller heap
Host link, firmware CPU	the poller, before the heap	poller heap
`femu-cxl-ssd`	`femu-cxl-ftl`	the vCPU waits before the access completes

3.2.6 Design pages

Page	Component
nvme-frontend.md (§3.3)	The NVMe controller: PCI and BAR layout, doorbells and shadow doorbells, queues, pollers, dispatch, completion timing, interrupts, reset and teardown
ftl.md (§3.4)	The BBSSD FTL: mapping schemes, write buffer, read cache, lines, GC and victim policies, wear and reclaim
nand-timing.md (§3.5)	The NAND media layer: geometry, cell types, per-LUN and per-plane busy time, channel bus, suspend, ECC
zns.md (§4.8)	Zoned Namespaces: zone state machine, zone FTL, write caches, Zone Append, ZRWA
fdp.md (§4.6)	Flexible Data Placement: endurance group, reclaim units and handles, placement and GC

Page	Component
namespaces.md (§4.16)	Namespaces and the subsystem: sizing, per-namespace modes, Namespace Management, shared namespaces
ocssd.md (§4.10)	Open-Channel SSD 1.2 and 2.0: the host-managed interface and its timing
kvssd.md (§4.12)	The NVMe Key Value command set and its FTL
csd.md (§4.14)	Computational storage: compute units, device memory, programs
nossd.md (§4.3)	NoSSD: memory-backed NVMe with no media timing
cxl-ssd.md (§4.19)	The CXL SSD: address decoding, page cache, direct mapping, cache control, NVMe link

3.2.7 Design choices

Each choice below says what FEMU does and why.

Charge latency, then hold the completion. FEMU does not simulate hardware cycles. A command's data is moved at memory speed, a model computes how long the command would take, and the completion is withheld until the host clock passes that time. The guest therefore runs at native speed with its real driver and file system, and still sees device-like latencies. The price is that FEMU sets a lower bound: if the poller does not run, the completion is posted after it is due. Give pollers and the FTL thread their own cores when you measure (performance tuning (§5.12.1)).

Host wall-clock time. Requests are stamped with `QEMU_CLOCK_REALTIME`, the clock the guest also observes. A 200 us program makes the guest wait 200 us. There is no virtual time that could diverge from what the guest measures.

Data in host memory, separate from timing. The FTL and NAND model keep metadata only: mappings, valid bits, busy-until times. The bytes go straight to one host buffer at the namespace's offset. This keeps the models simple and fast, makes a read return the last write no matter what the FTL did, and lets a mapping bug change timing but never corrupt data. The cost is that capacity is bounded by host RAM and data does not survive QEMU. The power-loss model (`power_loss=on`) is the one place the FTL thread applies data, because it has to roll back unflushed writes.

Polling threads instead of trapping each I/O. A doorbell write only stores a number. Dedicated `femu-poller` threads fetch commands and post completions. Doing the work on the vCPU would make every command pay a VM exit and stall the guest CPU, and a trap-driven design cannot post a completion at a chosen microsecond. The cost is host cores that spin while the controller is enabled.

One owner per queue. A poller fetches from a submission queue without a lock, so each queue has exactly one poller. `multipoller_enabled` accepts only 0 (one poller, all queues) or 1 (queues split round-robin). An older value greater than 1 started several pollers that each swept every queue, and two of them then executed and completed the same command; realize now refuses it.

A separate FTL thread with single-threaded state. One `FEMU-FTL-Thread` per controller owns the FTL and NAND state of every BBSSD, ZNS and CSD namespace, so that state needs no locks. Pollers hand requests over through single-consumer rings and keep fetching while the FTL computes.

Modes as handler tables. A mode is a `FemuExtCtrlOps` table: init, exit, start, admin, I/O and log hooks. The frontend handles everything shared (queues, Identify, features, common NVM commands) and passes the rest to the table of the namespace a command addresses. Adding a kind of SSD means adding a table, and one controller can serve namespaces of different modes.

Refuse, do not guess. Settings that cannot work together fail realize with a message that names the property,

with a few exceptions the property reference states (for example `ns_mgmt`, which stays off without an error on a controller that cannot support it), and properties that are accepted but have no effect print a warning. A configuration that starts is one FEMU can serve as described.

CXL accesses wait on the vCPU. A load or store cannot complete out of order the way an NVMe command can, so the `femu-cxl-ssd` path charges media time on the vCPU thread that made the access, with the BQL released so other vCPUs keep running.

3.2.8 Glossary

Term	Meaning
BAR	PCI Base Address Register: a window of device registers or memory. `femu` uses BAR0 (registers, doorbells), BAR2 (CMB), BAR4 (MSI-X)
BBSSD	black-box SSD, `femu_mode=1`: a conventional SSD with a device FTL
BQL	QEMU's big lock. vCPU MMIO handlers and the main loop hold it; pollers and FTL threads do not
CC, CSTS	Controller Configuration and Controller Status registers; `CC.EN` enables the controller, `CSTS.RDY` says it is ready
CMB	Controller Memory Buffer, device memory the host can place queues or data in (`cmbsz`, `cmbloc`)
CNTLID	controller identifier, unique within a subsystem
CQ, SQ, CQE, SQE	completion and submission queues and their entries
CSD	computational storage device, `femu_mode=4`
DER	direct mapping of cached CXL SSD pages into the guest (`der=memslot` or `der=cylon`)
DFTL	a demand-loaded page mapping table with a cached mapping table (`mapping=dftl`)
Doorbell	a BAR0 register the host writes a queue's new tail (SQ) or head (CQ) to
EventIdx	in the Doorbell Buffer Config scheme, the value the controller publishes so the host knows when it must still ring the real doorbell
`expire_time`	a request's completion time in host nanoseconds; the poller posts it no earlier
`ext_ops`	a mode's `FemuExtCtrlOps` handler table, per controller and per namespace
FDP	Flexible Data Placement: reclaim units and placement handles on `femu-subsys`
FTL	flash translation layer: maps logical pages to NAND pages and reclaims space
GC	garbage collection: copying valid pages out of a line so it can be erased
HDM	host-managed device memory, the address range a CXL Type-3 device decodes
irqfd	a KVM mechanism that lets a thread inject an MSI-X interrupt without the BQL
KV	key-value SSD, `femu_mode=5`, the NVMe Key Value command set
Line	a superblock: the same block index on every plane of every LUN; the unit of GC
LUN	a NAND die that runs one operation at a time; channels hold LUNs
NoSSD	`femu_mode=2`, the default: NVMe backed by memory with no media timing
OCSSD	Open-Channel SSD, `femu_mode=0`: the host addresses NAND directly
`pq[i]`	poller `i`'s min-heap of timed requests, ordered by `expire_time`
PRP, SGL	the two ways an NVMe command describes its data buffers
Poller	a `femu-poller` thread that fetches I/O commands and posts completions
Realize	QEMU's device construction step; property checks happen here
Shadow doorbell	a host-memory copy of the doorbells set up with Doorbell Buffer Config; FEMU polls it
Sweep	one pass of a poller over its queues and its completion heap
`to_ftl[i]`, `to_poller[i]`	the rings between poller `i` and the FTL thread

Term	Meaning
WAF	write amplification factor: NAND pages programmed per host page written
ZNS	Zoned Namespace SSD, `femu_mode=3`

3.2.9 Source map

Path	Contents
`hw/femu/femu.c`	QOM types `femu` and `femu-subsys`, realize and exit, property checks, registers and doorbells, the FTL thread, mode registration
`hw/femu/femu-props.c`	property help text
`hw/femu/nvme-admin.c`, `nvme-io.c`, `nvme-util.c`, `dma.c`, `intr.c`	the NVMe frontend
`hw/femu/nvme-pel.c`, `nvme-pi.c`, `nvme-streams.c`	Persistent Event Log, protection information, Streams
`hw/femu/nossd/`, `bbssd/`, `zns/`, `ocssd/`, `kvssd/`, `csd/`	the modes; `bbssd/` also holds the FTL
`hw/femu/nand/`, `hw/femu/ocssd/oc-timing.c`	NAND timing
`hw/femu/backend/dram.c`	the memory backend
`hw/femu/cxlssd/`	the CXL SSD
`hw/femu/lib/`	rings and the priority queue
`hw/femu/nvme.h`	shared structures, constants and the mode enum

Code structure (§8.1) lists every file.

3.2.10 Related pages

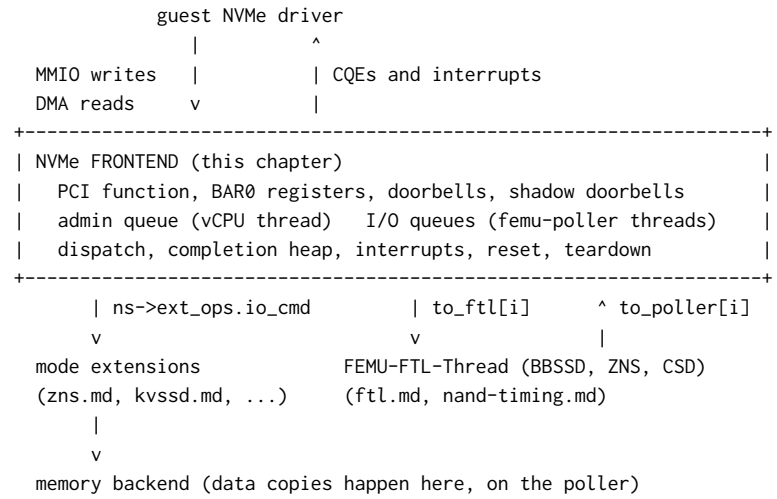
- Architecture (§3.1)
- Timing model (§6.2)
- Choosing a mode (§4.1)
- Security and limits (§5.13)
- Device property reference (§A.1)
- Log pages and counters (§B)

3.3 NVMe frontend

The NVMe frontend is the part of `-device femu` that a guest driver talks to: the PCI function, the controller registers, the admin and I/O queues, the poller threads that fetch and complete commands, and the interrupts. It turns NVMe commands into calls to a mode (BBSSD, ZNS, KV and the others) and holds each completion until the time the mode charged has passed.

This chapter describes the frontend as the code in this tree implements it. The design overview (§3.2) shows where it sits among the other components.

3.3.1 Place in the hierarchy



3.3.2 PCI function and BAR layout

The controller is a PCIe endpoint with class code 01 08 02 (NVMe). Its vendor and device IDs come from `vid` and `did` (by default `1d1d:1f1f`), and it reports NVMe version 1.4. It has INTA, an MSI capability with up to 32 vectors at config offset 0x50, a PCIe capability at 0x80, and MSI-X with `queues` + 1 vectors in a BAR of its own. `nvme_init_pci()` sets this up.

BAR	Contents	When
0-1	controller registers and doorbells (64-bit)	always
2-3	Controller Memory Buffer (64-bit)	`cmbsz` non-zero; `cmbloc` must select BAR 2
4-5	MSI-X table and pending bits	always

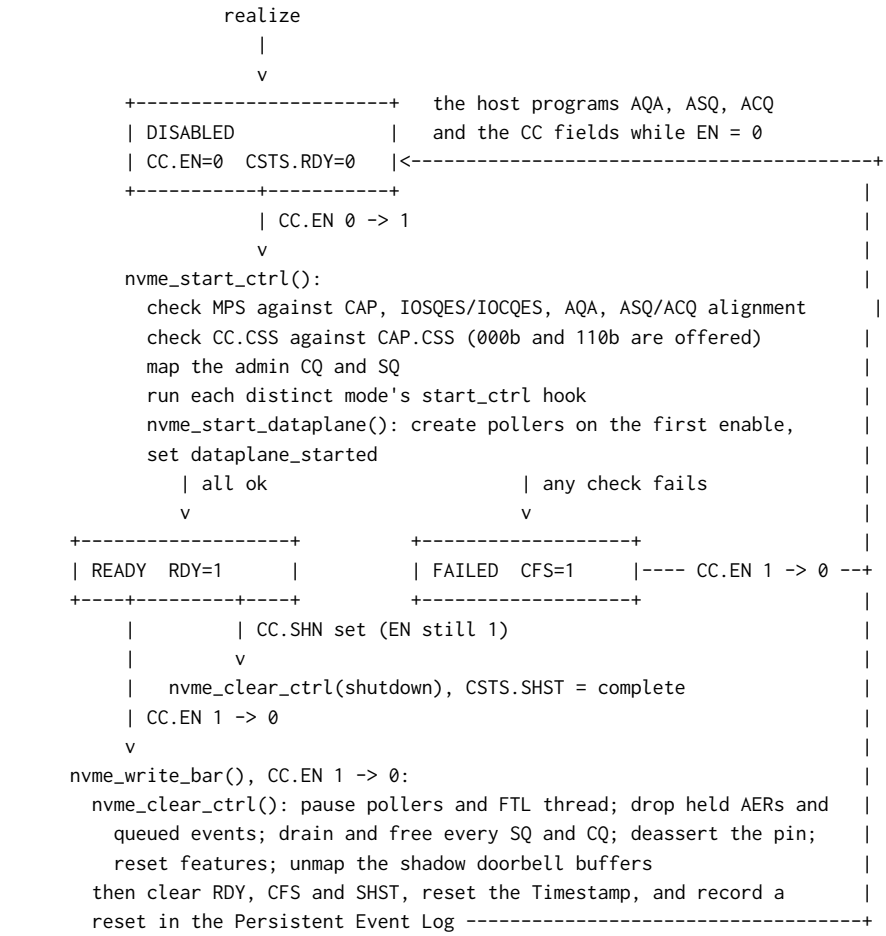
```
BAR0, reg_size bytes
offset   register   FEMU behaviour
0x0000   CAP        MQES = entries, CQR = cqr, AMS = 0, TO = 0xf, DSTRD = stride,
                    NSSRS = 0, CSS = NVM + I/O command sets by CSI,
                    MPSMIN = mpsmin, MPSMAX = mpsmax,
                    CRMS = CRWMS (not with OCSSD)                        read only
0x0008   VS         0x00020100 (2.1); 0x00010400 (1.4) with OCSSD        read only
0x000c   INTMS      set interrupt mask bits (MSI and pin; ignored with MSI-X)
0x0010   INTMC      clear interrupt mask bits; a held MSI is sent when unmasked
0x0014   CC         EN, SHN, MPS, IOSQES, IOCQES, CSS; AMS must be 0 (round
                    robin); see the state machine below
0x001c   CSTS       RDY, CFS, SHST                                       read only
0x0020   NSSR       reads 0; writes ignored (no subsystem reset)
0x0024   AQA        admin ASQS and ACQS, 1..4095 (0's based): 2..4096 entries
0x0028   ASQ        admin SQ base, 64 bits, may be written as two dwords
0x0030   ACQ        admin CQ base, 64 bits, may be written as two dwords
0x0038   CMBLOC     cmbloc                                               read only
0x003c   CMBSZ      cmbsz                                                read only
0x0068   CRTO       CRWMT = CAP.TO, CRIMT = 0 (reads 0 with OCSSD)       read only
0x1000   doorbells: DB = 4 << stride bytes
         0x1000 + (2 * qid)     * DB   SQ qid tail
         0x1000 + (2 * qid + 1) * DB   CQ qid head
         qid 0 is the admin pair, 1 .. queues are I/O queues

reg_size = max(16 KiB, pow2ceil(0x1000 + 2 * (queues + 1) * DB))
```

BAR0 is at least 16 KiB because the PCIe transport makes bits 13:4 of the BAR read only. Writes go to `nvme_mmio_write()`: offsets inside the register block go to `nvme_write_bar()`, the admin doorbell pair to `nvme_process_db_admin()`, and everything else to `nvme_process_db_io()`. All of them run on the vCPU thread that made the access, holding the BQL.

The CMB is an I/O memory region (`nvme_cmb_ops`) backed by a host buffer of the size `cmbsz` describes. PRP and SGL data pointers, and the entries of PRP-list queues, that fall inside it are served from that buffer (`nvme_addr_is_cmb()` in `hw/femu/dma.c`). A physically contiguous queue is mapped directly and has no CMB case.

3.3.3 Controller states



A shutdown with CC.EN still 1 runs `nvme_clear_ctrl()` but leaves RDY set and resets neither the Timestamp nor the Persistent Event Log context.

Pollers are created once and survive a reset; a reset only clears `dataplane_started` until the next enable. With `power_loss=on`, a normal shutdown (CC.SHN = 01b) first flushes the BBSSD write buffer, and a failed flush sets CSTS.CFS.

3.3.4 Queues

Data structures

Structure	Fields that matter
`NvmeSQueue`	`sqid`, `cqid` (the CQ it reports to), `head`, `tail`, `size`; `dma_addr` and `dma_addr_hva` (contiguous ring) or `prp_list`; `io_req` (one `NvmeRequest` per slot) and `req_list` (the free ones); `db_addr`/`db_addr_hva` and `eventidx_addr`/`eventidx_addr_hva` (shadow doorbell); `is_active`
`NvmeCQueue`	`cqid`, `head`, `tail`, `size`, `phase`; `vector`, `irq_enabled`, `virq` and `guest_notifier` (irqfd route), `irq_bh` (fallback); `sq_list`; `post_lock`; the same four shadow doorbell fields; `is_active`
`NvmeRequest`	`cmd`, `cqe`, `sq`, `ns`, `status`; `slba`, `nlb`, `is_write`, `xfer_bytes`; `stime`, `expire_time`, `reqlat` (ns); `cxl_seq`

Structure	Fields that matter
FemuCtrl	sq[], cq[] (index 0 is admin); nr_pollers, poller[]; to_ftl[], to_poller[], pq[], cpl_backlog[], should_isr, poller_in_sweep[], poller_ctr[] (all indexed by poller, 1-based); dataplane_started, ftl_in_sweep, use_ftl_thread

A submission queue owns one request per slot, so it can have at most size commands outstanding; when its free list is empty the poller leaves the rest of the queue for a following sweep.

Admin queue

The admin queue pair is created from AQA, ASQ and ACQ when the controller is enabled. It is processed synchronously: a write to the admin SQ tail doorbell calls nvme_process_sq_admin() on the vCPU thread, which fetches each new entry, runs nvme_admin_cmd(), writes the completion and notifies the admin CQ's interrupt. It fetches only while the admin CQ has a free slot; a CQ head doorbell write resumes it. The admin queue always uses its doorbell registers, even after Doorbell Buffer Config.

Admin commands that free or replace state the I/O path reads (Delete SQ and CQ, Format NVM, Namespace Management and Attachment, Sanitize, Streams, some Set Features) first stop the pollers and the FTL thread with nvme_pause_pollers() and restart them after (see Quiescing the data plane (§3.3.10)).

I/O queues

The host creates I/O queues with Create I/O CQ and Create I/O SQ. Queue ids run from 1 to queues, sizes from 2 to entries + 1. With cqr=1 (the default) queues must be physically contiguous; with cqr=0 a queue may be a PRP list. A completion queue's interrupt vector must be at most queues, and below the number of MSI vectors the guest enabled when MSI is in use. Several SQs may share one CQ.

A new SQ is published with a write barrier before is_active is set, and the poller reads is_active with a read barrier before using the queue, so a poller never sees a half-built queue. Delete I/O SQ pauses the data plane, returns the queue's requests to it, and frees it. Delete I/O CQ is refused while an SQ still reports to it.

Set Features Number of Queues reports queues for both SQs and CQs and is accepted only before any I/O queue exists.

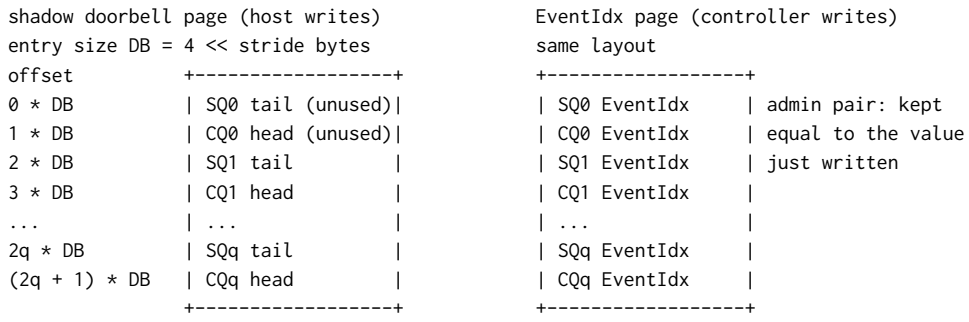
3.3.5 Doorbells and shadow doorbells

Doorbell registers

A doorbell write records the new value and returns. For an I/O SQ the vCPU stores sq->tail; the poller notices it on its next sweep. For an I/O CQ it stores cq->head, notifies the CQ again if entries remain, and on the pin updates the interrupt level. A write to a doorbell that does not exist, or of a value past the end of its queue, is ignored and, while the controller is ready, raises an Error asynchronous event (event information 00h, write to an invalid doorbell register, or 01h, invalid doorbell write value). An I/O doorbell write that is not aligned to DB, and a register write to a queue that has a shadow doorbell, are dropped without an event (for a shadowed CQ the write only updates the pin level).

Doorbell Buffer Config

The controller always advertises Doorbell Buffer Config (OACS bit 8). The host gives two page-aligned pages of its memory with this admin command: a shadow doorbell page, where it writes tails and heads instead of the registers, and an EventIdx page, where the controller writes back hints. nvme_set_db_memory() maps both pages and points every existing I/O queue at its two entries:



Rules the code enforces:

- Both addresses must be non-zero and page aligned, and both pages must map completely; otherwise the command fails with Invalid Field and nothing is recorded.
- The pages are set once per enable. A second command before a reset fails.
- All entries must fit in one page: (2 x queues + 1) x DB bytes at most. With a 4 KiB page and `stride=0` that is 511 I/O queues.
- Each queue that already exists is seeded with its current tail or head before it is switched to its entry, so a poller never reads a stale zero. A queue created afterwards starts at 0 and points at its entry at once, relying on the host's zeroed entry.
- A controller reset unmaps both pages and forgets them.

Once a queue has a shadow doorbell, its register doorbell no longer moves it. The poller reads the SQ tail from the shadow at the start of every sweep (`nvme_update_sq_tail()`) and the CQ head whenever it checks for space or posts an interrupt (`nvme_update_cq_head()`). A value past the end of the queue is ignored.

EventIdx

With shadow doorbells the host rings the real doorbell only when its new value passes the EventIdx the controller published. FEMU's pollers read the shadow values of every active I/O queue on every sweep, so I/O queues do not depend on doorbell writes. Two queues are handled differently:

- **The admin pair** keeps its EventIdx equal to the value just written, so a host that follows the EventIdx protocol still rings it (the admin queue is driven only by its registers).
- **Pin interrupts.** With neither MSI-X nor MSI enabled, the CQ EventIdx is the head itself. The pin is level triggered and can only drop after the host reports its new head, so the host must ring the CQ head doorbell after every batch; the register write then updates the pin level.

3.3.6 Pollers

How many and who owns what

`nvme_init_poller()` creates the pollers the first time the host enables the controller. Their number depends on two properties:

`multipoller_enabled`	Pollers	Poller `i` of `N` owns
`0` (default)	1	every I/O SQ
`1`	ceil(`queues` / `poller_ratio`); `poller_ratio=0` counts as 1	SQs `i`, `i + N`, `i + 2N`, ...
anything else	realize fails	

```
multipoller_enabled=0                multipoller_enabled=1, queues=8, poller_ratio=3
                                     N = ceil(8 / 3) = 3

femu-poller 1: SQ1 SQ2 ... SQ8       femu-poller 1: SQ1 SQ4 SQ7
                                     femu-poller 2: SQ2 SQ5 SQ8
                                     femu-poller 3: SQ3 SQ6

per poller i:  to_ftl[i]    ring, poller i -> FTL thread, holds 65535
               to_poller[i] ring, FTL thread -> poller i, holds 65535
               pq[i]        min-heap on expire_time, starts at 65536, grows
               cpl_backlog[i]  completions waiting for CQ space
               should_isr row  which CQs to interrupt after this sweep
               poller_in_sweep[i], poller_ctr[i]
```

```
-device femu,femu_mode=2,queues=8,multipoller_enabled=1,poller_ratio=3
```

Every SQ has exactly one owner because a poller fetches from it without a lock. Values of `multipoller_enabled` greater than 1 used to start one poller per shard but had each of them sweep every queue; two pollers then fetched, executed and completed the same command. Realize now refuses them.

A CQ is not owned. Its SQs may belong to different pollers, so with more than one poller a completion is posted under the CQ's `post_lock`, and each poller keeps its own row of `should_isr` flags.

The sweep

```
nvme_poller(i), forever until poller_stopping:
  if !dataplane_started: in_sweep = false, sleep 1 ms, retry
  in_sweep = true; smp_mb; recheck dataplane_started
  for each SQ this poller owns, active and with an active CQ:
      nvme_process_sq_io(sq, i)          fetch and execute new commands
  nvme_process_cq_cpl(i)                 time and post completions
  smp_mb; in_sweep = false
```

A poller never sleeps while the data plane runs: it spins on a host core. Plan one core per poller and one for the FTL thread (performance tuning (§5.12.1)).

Data copies and write atomicity

Identify reports AWUN 0, so a one-block write must reach the media whole with respect to every other command. With more than one poller, two commands on the same blocks copy at the same time. Each access to the memory backend therefore holds the locks of the 4 KiB granules it touches. The granules share 1024 locks: granule `g` takes lock `g % 1024`.

```
poller 1: Write LBA 0 ---+                      +--> memcpy into backend
                         |   lock granules of   |
poller 2: Read LBA 0  ---+-> [off, off + len)  --+--> memcpy out of backend
                             in index order          (waits for the write)
```

- Every path that moves block data takes the locks:
  - Read, Write and Zone Append (`backend_rw()`);
  - the NoSSD inline path;
  - Copy, for each source range and for the destination;
  - Compare;
  - Write Zeroes and deallocate;
  - the protection information and metadata paths;
  - the power-loss write cache;

  - CSD memory reads.
- Read, Write and Compare hold the locks of their whole range, so a multi-block Read or Write is also atomic. OCSSD takes the locks per vector entry.
- Copy takes one source range at a time, then the destination. Its write to the destination is atomic, but a Write between two of its source reads can leave old data from one range and new data from the next. Deallocate and Write Zeroes also take one range at a time.
- A bbssd controller linked to a `femu-cxl-ssd` medium (`cxl_ssd=`) brings the locks to the medium's backend. CXL.mem loads and stores, and pages mapped straight into the guest, do not take them.
- The data-remanence debug output (`FEMU_EXP_LOG`, `FEMU_SECRET`, `FEMU_DUMP_LPN`) reads the backend without the locks.
- The tests `rw-atomic-split-write` and `rw-atomic-split-copy` stop a write halfway through its copy (`x-copy-hold`, qtest builds only) and read the block meanwhile. The write goes on once `x-range-waits` shows the read waiting for its lock. Without the locks the read returns half old, half new data.
- Locks are taken in index order, so two commands never wait in a cycle. A namespace with metadata takes its `mdata_lock` first.
- Two commands on different blocks wait for each other only when their granules share a lock. For random 4 KiB commands that happens once in 1024.

3.3.7 Command lifecycle

```
SQ slot in guest memory (or the CMB)
  |  nvme_process_sq_io(): copy the 64-byte SQE, take a free NvmeRequest,
  |  stime = expire_time = now (QEMU_CLOCK_REALTIME)
  v
nvme_io_cmd()
  |  nsid -> nvme_ns(): allocated and attached, else Invalid Namespace
  |  Flush, Dataset Management, Compare, Write Zeroes, Copy, Verify,
  |  Write Uncorrectable, I/O Management Send/Receive: handled here
  |  any other opcode -> ns->ext_ops.io_cmd() of the namespace's mode
  |     NoSSD, BBSSD, CSD Read/Write: nvme_rw() maps PRP/SGL, copies data
  |     ZNS, OCSSD, KV: their own paths; OCSSD and KV add NAND time now
  |  status stored; on success the SMART host counters are updated
  v
to_ftl[i] (every request, failed ones included)
  |
  +-- controller has an FTL thread: it dequeues, calls
  |   femu_ftl_process_req(), adds the latency to expire_time,
  |   enqueues on to_poller[i]
  +-- no FTL thread: the poller reads to_ftl[i] itself
  v
nvme_process_cq_cpl(i)
  |  add host-link time (Read, Write) and firmware-CPU time
  |  (Read, Write, Zone Append), insert into pq[i]
  |  retry cpl_backlog[i]
  |  while the heap's earliest request is due (now >= expire_time)
  |  and, for a CXL-linked request, the medium has dropped stale pages:
  |      CQ inactive -> request back to the free list
  |      CQ full     -> cpl_backlog[i]
  |      otherwise   -> post CQE
  v
request back on sq->req_list; should_isr[cqid] = true
end of sweep: for each flagged CQ, raise its interrupt
```

Things to note:

- **Data moves before time is charged.** By the time a request reaches `to_ftl[i]`, a Read's data is already in guest memory and a Write's data is already in the backend. The FTL thread only computes time. (With `power_loss=on` the poller skips `nvme_io_cmd()` and the FTL thread executes every I/O command, data copy included, through `nvme_power_io()`.)
- **Failed commands take the same path.** A request with an error status is still enqueued; the mode charges it nothing, and it completes as soon as the poller sees it again (in the same sweep when there is no FTL thread), plus any host-link or firmware-CPU time, which is charged regardless of status. Skipping the ring would leak the request.
- **Order.** Completions are posted in `expire_time` order per poller, not in submission order.
- **Full completion queues.** A due completion whose CQ has no free slot waits in the poller's backlog, and the poller retries it every sweep.
- **Abort** looks for the command in the SQ between head and tail with the pollers paused. If it is there, the controller records it in its own state (the SQ itself is never written) and the Abort completes with dword 0 bit 0 clear; the command then completes with Command Abort Requested when it is fetched, without running. A command already fetched is not aborted, and the Abort completes with bit 0 set. Admin commands run one at a time, so the Aborts outstanding together are the one running and those still in the admin SQ; with more than `acl` of them waiting, the running one fails with Abort Command Limit Exceeded. A command queued behind an Abort on the admin queue can be aborted too.
- **Media errors carry DNR.** Unrecovered Read Error (a block marked by Write Uncorrectable, or an injected read error) and Compare Failure always set Do Not Retry, on every command that can return them: a retry would fail again.
- **Admin commands use PRPs.** An admin command whose PSDT is not 00b fails with Invalid Field in Command.

3.3.8 Completion timing

`nvme_process_cq_cpl()` is where FEMU enforces latency. A request's `expire_time` starts at its fetch time and grows as each model adds to it:

Added by	When	Property
OCSSD, KV, CSD compute units	while the command executes on the poller	mode properties
FTL thread (BBSSD, ZNS, CSD NAND time)	before `to_poller[i]`	FTL and NAND properties
host link	in `nvme_process_cq_cpl()`; a per-direction next-free time	`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`
firmware CPU	in `nvme_process_cq_cpl()`; one next-free time per firmware core (`nvme_fw_cost()` picks the time per opcode)	`fw_cpu_ns`, `fw_read_ns`, `fw_write_ns`, `fw_other_ns`, `fw_cores`

The heap `pq[i]` is a binary heap (`hw/femu/lib/pqueue.c`) keyed on `expire_time`. Each sweep pops every request that is due and stops at the first that is not. Nothing sleeps: a request simply stays in the heap until a sweep finds it due. The guest therefore sees at least the charged time, plus however long the poller takes to come round.

The clock is `QEMU_CLOCK_REALTIME`. Only the qtest-only property `x-oc12-clock` switches it to the virtual clock, for deterministic tests.

3.3.9 Interrupts

Each I/O CQ created with interrupts enabled gets its vector. After a sweep that posted to a CQ through the heap, the poller calls `nvme_isr_notify_io()` once for that CQ:

```
nvme_isr_notify_io(cq)                    (poller thread, no BQL)
  |
  +-- cq->virq > 0 (MSI-X under KVM, route added at Create I/O CQ):
  |      event_notifier_set() -> KVM irqfd injects the MSI-X vector
  |
  +-- otherwise: qemu_bh_schedule(cq->irq_bh)      (main loop, BQL)
         nvme_isr_notify_legacy():
           MSI-X enabled  -> msix_notify(vector)
           MSI enabled    -> msi_notify(vector), or hold the bit in
                             irq_status while INTMS masks it
           neither (pin)  -> nvme_irq_update(): assert INTA while any
                             enabled CQ has head != tail and INTMS allows
```

The admin CQ is notified on the vCPU thread through the same legacy path. MSI-X mask, unmask and pending-bit polling are handled by vector notifiers (`nvme_vector_mask()`, `nvme_vector_unmask()`, `nvme_vector_poll()`) that move the irqfd routes.

On the heap path a poller raises at most one interrupt per CQ per sweep.

Interrupt coalescing

Interrupt Coalescing (Feature 08h) holds the interrupt of an I/O completion queue's vector until enough entries wait, or until the oldest waits too long. The threshold THR (0's based entries) and the time TIME (100 us steps) both must be non-zero, or coalescing is off. The threshold counts per interrupt vector, so queues that share a vector share one count. Coalescing Disable in Feature 09h turns it off for one vector.

These are never held: the admin queue, completions with an error, and the pin. The pin is a level the controller recomputes from the queues, and the threshold is only a recommendation. `intc_thresh`, `intc_time` and `intc` set the reset values.

```
entry posted:  vector pending++; first entry: first_ns = now; error: urgent
end of a sweep, CQs it posted to:     off, urgent, pending >= THR + 1,
and every sweep, vectors in its share: or now - first_ns >= TIME x 100 us
                                          --> interrupt, pending = 0
Set Features 08h or 09h: pause the pollers, send what is held, apply
```

Each vector is checked for its time by one poller. A CQ head doorbell write that leaves entries behind still interrupts at once, which the spec allows, and it clears the vector's count. With Feature 08h at 0h nothing is counted.

A Set Features 08h or 09h, and the delete of a queue whose vector holds entries or carries other queues, signal the vector at once on the thread that holds the BQL. They do not use the queue's bottom half or eventfd, which a delete would close. A controller reset sends nothing. The rule is that an interrupt may come early but is never lost. One extra interrupt can follow when an admin completion on vector 0 races a poller that has just posted an I/O entry; the host then finds no new entry.

3.3.10 Quiescing the data plane

Admin commands, controller reset and some features must change state the pollers and the FTL thread read without locks. `nvme_pause_pollers()` waits until neither is inside a sweep:

```
caller (vCPU, BQL)                  poller i                        FTL thread
dataplane_started = false           in_sweep[i] = true              ftl_in_sweep = true
smp_mb                              smp_mb                          smp_mb
wait while any in_sweep[i]          dataplane_started? no:          dataplane_started? no:
  or ftl_in_sweep is true             in_sweep[i] = false, back off   ftl_in_sweep = false, skip
... change queues or namespaces ... ... sweep ...                  ... one request ...
nvme_resume_pollers():              smp_mb                          ftl_in_sweep = false
  dataplane_started = true          in_sweep[i] = false
```

Either the caller sees a sweep in progress and waits for it, or the thread sees the cleared flag and backs off. With shared namespaces (`femu-subsys,ns_mgmt=on`) the pause covers every controller in the subsystem, and a nested pause does not resume its caller's work.

3.3.11 Teardown

`femu_exit()` runs when the device is unplugged (`device_del`). QEMU does not call it when the process exits. It stops the threads before freeing anything they read:

1. Set `poller_stopping` and join every poller. Pollers go first because they feed the FTL thread.
2. Set `ftl_stopping` and join the FTL thread.
3. Close the Persistent Event Log, detach a linked CXL medium.
4. Run each distinct mode's `exit` hook once (not for a controller on a shared subsystem's storage).
5. `nvme_clear_ctrl(shutdown)`, then free the rings, heaps and per-poller arrays, the AER bottom half and the memory backend (unless it is shared or lent by a CXL medium).

3.3.12 Commands, namespaces and modes

Dispatch into modes

A mode is a `FemuExtCtrlOps` table (`hw/femu/nvme.h`):

Hook	Called from	For
`init`	realize, once per namespace of that mode, and Namespace Management create	build the mode's state for the namespace
`init_ctrl_name`	realize, only with `ns_mgmt` (otherwise each mode's `init` sets the name); only a namespace of the controller's `femu_mode` names it, and the controller's own table names it when no namespace does	Identify model number and serial
`start_ctrl`	controller enable, once per distinct mode	refuse settings the mode cannot serve; derive values from CC
`io_cmd`	poller, for opcodes the frontend does not handle	the mode's I/O commands
`admin_cmd`, `admin_cmd_cqe`	vCPU, for admin opcodes the frontend does not handle	mode admin commands, such as the Open-Channel commands, BBSSD 0xEF and the CSD commands
`get_log`	vCPU, for log ids the frontend does not handle	OC 2.0 chunk information, the ZNS Changed Zone List
`rw_check_req`	not called through the table	OC 2.0 sets it and calls its checker directly
`exit`, `ns_exit`	teardown, namespace deletion	free the mode's state

`nvme_register_extensions()` installs the table for the controller's `femu_mode`, and `nvme_register_extensions_ns()` gives each namespace the table of its own mode, so I/O dispatches on the namespace, not

the controller. Admin commands go to the controller's table; Get Log Page tries the named namespace's table first, then the controller's.

Capability registry

`hw/femu/nvme-caps.c` is the one place that says what the controller handles. `nvme_admin_effects()` and `nvme_io_effects()` return the Commands Supported and Effects entry of an admin opcode, or of an I/O opcode in one command set, and zero for an opcode that is not handled. `nvme_log_support()` returns the Supported Log Pages entry of a log id in one command set.

An I/O opcode's entry for a command set is what the namespaces of that set handle between them (`nvme_ns_io_effects()` for one namespace): Flush, the optional commands and I/O Management, which `nvme_io_cmd()` serves itself, and then the mode's own table, so log 05h lists the Open-Channel vector commands, the CSD commands and the key value commands of the modes present, and leaves Read and Write out for Open-Channel 1.2, which refuses them. A command set with no namespace reports what a namespace of it would handle. The admin entries add the commands of the controller's mode: BBSSD 0xEF, the Open-Channel and CSD admin commands. `nvme_caps_id_ctrl()` fills OACS, ONCS, OCFS, LPA and SANICAP from those three each time Identify Controller is answered. LPA bit 5 stays clear: setting it would oblige a Command Scope in every log 05h entry, and Linux warns about any I/O entry bit beyond CSUPP and LBCC as an unusual effect.

Identify CNS 06h reports the version of each command set: NVM 1.1 in bytes 23:20 (CSI 00h) and ZNS 1.2 in bytes 35:32 (CSI 02h). The NVM structure also sets AOCS bit 0, as Get LBA Status serves action type 02h. KV reports 1.3, and SLM and CP 1.2. An Open-Channel controller keeps its 1.4 replies: the NVM and ZNS VER fields and AOCS stay 0, and CSI 01h still reports KV 1.3.

Logs 05h and 00h are built from the same functions, and `nvme_io_cmd()` runs an optional NVM command (Compare, Dataset Management, Write Zeroes, Copy, Verify, Write Uncorrectable) only where `nvme_io_effects()` lists it for the namespace's command set. Features follow the same rule through `nvme_fid_supported()`, which Get Features, Set Features and log 12h share.

The `caps-*` qtests check the registry against dispatch. Vendor command 0xEE answers Invalid Command Opcode outside Open-Channel, as any unlisted admin opcode does. I/O Management Send and Receive answer Invalid Command Opcode where log 05h does not list them: without FDP, and on namespaces of other command sets. Two differences are outside the test's reach. Get Features serves the Select field while ONCS bit 4 is clear. CNS 1Ch always lists the NVM, zoned and key value sets.

Namespace routing

`nvme_ns()` maps an NSID to a namespace that is both allocated and attached to this controller. NSIDs run from 1 to `namespaces` (or to 256 with Namespace Management). An unknown NSID fails with Invalid Namespace, and with Namespace Management an unallocated or detached NSID within range fails with Invalid Field. A Flush to every namespace (NSID FFFFFFFFh) is refused, which Identify reports as Flush Behavior 10b.

The FTL thread routes each request by its namespace: a namespace borrowed from a linked CXL medium goes to the medium's FTL; with `power_loss=on` the command is first executed; then ZNS namespaces go to `zns_ftl_process_req()`, BBSSD and CSD namespaces to `bb_ftl_process_req()`, and others charge nothing. One FTL thread serves all of them, and it exists only when a namespace needs it. With shared namespaces, the subsystem's `ns_lock` is held around command execution and FTL processing. namespaces.md (§4.16) covers sizing, per-namespace modes and Namespace Management.

Log pages and features at a glance

Log page	Id	Notes
Supported Log Pages	00h	lists the ids the controller advertises (04h only with Namespace Management)
Error Information	01h	newest `elpe` + 1 entries; each names the command set (byte 30) and opcode (byte 31) of its command, and every entry, an empty one too, has Log Page Version 1h (byte 63); Open-Channel entries keep the 1.4 layout
SMART / Health	02h	host totals from the pollers, media wear, `temperature`
Firmware Slot	03h	one slot, read-only; Identify FRMW reports the same
Changed Namespace List	04h	namespaces whose attributes changed
Commands Supported and Effects	05h	per command set (NVM, zoned, KV)
Device Self-test	06h	tests complete at once
Telemetry Host / Controller	07h, 08h	07h: header plus the C0h counters captured by the last Create; 08h: header only
Endurance Group	09h	with `femu-subsys`; Identify reports the group (CTRATT bit 4, ENDGIDMAX, ENDGID, and the Endurance Group List, CNS 19h) with it
Persistent Event	0Dh	kept in `pel_file` if set
LBA Status	0Eh	
Feature Identifiers Supported and Effects	12h	built from the check Get and Set Features use; per command set (CSI) when CC.CSS is 110b
NVMe-MI Commands Supported and Effects	13h	all zero: no NVMe-MI Send or Receive
FDP Configurations, RUH Usage, Statistics, Events	20h-23h	with FDP
Sanitize Status	81h	
Changed Zone List	BFh	ZNS, from the mode
Chunk Information	CAh	Open-Channel 2.0, from the mode
FEMU media counters	C0h	WAF and FTL counters

`nvme_log_csi()` gives the command set of a Get Log Page. It is CDW14.CSI when CC.CSS is 110b, and NVM when CC.CSS is 000b (Base 2.3, Figure 205). Get Log Page refuses an id that log 00h does not list for that command set with Invalid Log Page. So the Changed Zone List (BFh) answers only for the zoned set, and the Open-Channel 2.0 Chunk Information (CAh) only for NVM. When CC.CSS selects by CSI, logs 00h, 05h, 12h and BFh refuse an unknown CSI with I/O Command Set Not Supported. With FDP off there is no FDP capability, so the FDP pages are neither listed nor answered.

Get and Set Features answer Arbitration (01h), Power Management (02h, one power state), LBA Range Type (03h), Temperature Threshold (04h), Error Recovery (05h), Volatile Write Cache (06h, only with `vwc=1`), Number of Queues (07h), Interrupt Coalescing (08h), Interrupt Vector Configuration (09h), Write Atomicity (0Ah), Asynchronous Event Configuration (0Bh), Timestamp (0Eh), Host Behavior Support (16h), Command Set Profile (19h), FDP (1Dh, 1Eh, only with FDP on), Key Value Configuration (20h, only with a KV namespace) and Software Progress Marker (80h). Any other identifier, or one of these without what it needs, fails with Invalid Field in Command, and log 12h lists exactly the identifiers that answer. Only the FDP features report a saved value: FDP Mode is fixed at realize, FDP Events can be changed at run time. A controller reset (`nvme_reset_features()`) restores Arbitration, Power Management, the temperature thresholds, Volatile Write Cache, Number of Queues, Interrupt Coalescing, Interrupt Vector Configuration, Write Atomicity, Asynchronous Event Configuration, Host Behavior Support and each namespace's Error

Recovery; the Software Progress Marker, LBA Range Type data, Key Value Configuration and FDP event filters keep their values.

Asynchronous events

The controller holds up to `aerl` + 1 Asynchronous Event Requests; one more fails with Asynchronous Event Request Limit Exceeded. An event is queued (at most 16; more are dropped) and posted by `nvme_process_aers()` from a main-loop bottom half, because a poller may raise it and must not touch the admin CQ. A request that arrives while events are queued is matched at once on the vCPU thread. The event types and their log pages are listed in log pages and counters (§B.3).

3.3.13 Parameters

The frontend reads these `-device femu` properties. Their defaults and ranges are in the generated reference; follow the link of each group.

Property	Effect on the frontend	Reference
`queues`	number of I/O queue pairs; sizes BAR0, MSI-X vectors, `sq[]`, `cq[]` and the shadow doorbell page	queues, pollers and interrupts (§A.1.1)
`entries`	CAP.MQES; the largest queue is `entries` + 1	same
`max_sqes`, `max_cqes`	Identify SQES and CQES; only 64-byte SQEs and 16-byte CQEs are accepted	same
`stride`	doorbell spacing, 4 « `stride` bytes; also the shadow doorbell entry size	same
`multipoller_enabled`, `poller_ratio`	number of pollers and queue ownership	same
`hiops_inline`	NoSSD only: performance option, on by default; set off only when debugging	same
`aerl`, `elpe`	held AER limit, Error log length	same
`mdts`	largest data transfer per command	same
`intc`, `intc_thresh`, `intc_time`	reset values of the coalescing features	same
`vid`, `did`	PCI IDs	controller identity and capabilities (§A.1.1)
`cqr`	whether queues must be physically contiguous	same
`mpsmin`, `mpsmax`	memory page sizes CC.MPS may choose	same
`oacs`, `oncs`, `vwc`	which optional admin and NVM commands are answered	same
`sgl`	accept SGL data pointers	same
`cmbsz`, `cmbloc`	Controller Memory Buffer on BAR 2	same
`femu_mode`, `namespaces`, `namespace_modes`	which mode tables are installed, NSID range	mode, capacity and namespaces (§A.1.1)
`ns_mgmt`, `streams`, `power_loss`	admin commands answered; FTL thread applies data with `power_loss`	namespace management, streams and power loss (§A.1.1)
`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`, `fw_cpu_ns`	time added in `nvme_process_cq_cpl()`	host link and controller firmware (§A.1.1)
`subsys`	CNTLID, FDP, shared namespaces	femu-subsys (§A.1.2)

Interactions:

- With the defaults (`queues=8`, `stride=0`) BAR0 is 16 KiB. Large `queues` or `stride` values grow it; `queues` above 511 with `stride=0` and 4 KiB pages leaves Doorbell Buffer Config unusable (the host then rings the registers).
- `multipoller_enabled=1` with `poller_ratio=1` starts one poller per queue, which needs that many spare host cores.
- The host's CC.MPS must fall within `mpsmin`..`mpsmax`, or enabling the controller sets CSTS.CFS.

3.3.14 Counters

Counter	Where kept	How to read it
Host read and write commands and bytes	`poller_ctr[i]`, counted on the poller after execution, successful commands only, for every mode (the FTL thread counts them with `power_loss`)	SMART log: Host Read/Write Commands, Data Units Read/Written
Completions, and completions posted 20 us or more past their due time	`poller_ctr[i]`	BBSSD vendor command 0xEF, CDW10=5, prints both and resets them (timing model (§6.2.11))
Error log entries	`num_errors`	SMART log, Error Information log

The per-poller counters are padded to separate cache lines so pollers never share one; readers sum them. Media counters (WAF, GC pages, buffer hits) come from the FTL, not the frontend; see log pages and counters (§B.1).

3.3.15 Validation

The frontend is covered by the qtest suite in `hw/femu/tests/qtest/femu-test.c`, which drives the controller through its registers with no guest. Cases that target this chapter include:

Case	Checks
`bar0-size`	BAR0 size for the queue count and stride
`cc-states`	enable, disable, shutdown and failure transitions of CC and CSTS
`doorbell-errors`	invalid doorbell writes raise Error events
`io-by-doorbell`, `io-by-shadow-doorbell`	I/O through registers and through Doorbell Buffer Config
`dbbuf-too-many-queues`	Doorbell Buffer Config refused when the entries do not fit
`intx-shadow-doorbell`	the pin drops with shadow doorbells
`queue-mapping`	Create I/O queue refused when the ring cannot be mapped
`shared-cq`, `shared-cq-pollers`	SQs that share one CQ, with one and several pollers
`poller-burst`, `poller-burst-per-queue`, `poller-burst-sharded`	full-queue bursts complete exactly once, in their own queue, under each poller layout
`cq-full`	completions wait for CQ space
`delete-sq-in-flight`, `ns-retire-pollers`	queue and namespace removal with I/O in flight
`aer-limit`	the AER limit
`abort`	Abort leaves the SQ unwritten, aborts queued admin and I/O commands, and enforces ACL
`cc-css`	an unoffered CC.CSS fails the enable
`admin-psdt`	admin commands naming SGLs are refused
`media-dnr`	Unrecovered Read and Compare Failure set DNR

Case	Checks
`fid-effects`, `fid-effects-kv`, `fid-effects-fdp`	log 12h agrees with Get Features; log 13h is zero
`frmw`	FRMW and the firmware slot log agree
`caps-*` (22 configurations: every mode, the optional commands on, off and in pairs, Namespace Management, Streams, PI, FDP, mixed namespace modes)	every admin opcode, every I/O opcode on every namespace and every log id per command set is answered exactly when logs 05h and 00h list it; log 12h agrees with Get Features; OACS, ONCS, OCFS, LPA, SANICAP, VWC, FRMW, SGLS, CNS 1Ch and the Copy limits in Identify Namespace agree with the logs and with what the controller does; ONCS and the Format bit match what the configuration asks for; VS, CAP.CRMS, CRTO and BPCAP match the reported version (2.1, or 1.4 for Open-Channel); CNS 06h reports the version of each command set it offers and AOCS bit 0, CAP.AMS is 0, and LPA bit 5 and every command scope are clear
`ns-mgmt-before-identify`	a managed bbssd namespace is addressed correctly before the host reads Identify Controller
`v2-refusals`	CC.AMS other than round robin fails the enable; Identify CNS 00h on a KV namespace fails with Invalid I/O Command Set while CNS 08h answers; logs 00h and 05h refuse an unknown CSI; CNS 07h for KV is refused under CC.CSS 000b while CNS 1Ah is not; CNS 1Fh answers an allocated NSID and refuses 0 and FFFFFFFFh
`endgrp-reported`	with a subsystem and FDP off, CTRATT bit 4, ENDGIDMAX and the ENDGID of a block and a KV namespace (CNS 08h, and CNS 05h for KV) report the one endurance group; CNS 19h lists group 1 for a start of 0 or 1 and no group for 2 or FFFFh; the FDP pages, FDP Mode and I/O Management are refused
`endgrp-list-absent`	without a subsystem, CNS 19h fails with Invalid Field
`features-reset`, `features-reset-vwc`	features return to defaults on reset
`admin-fuzz`, `io-fuzz` and its variants	structured fuzzing of admin and I/O commands

Run them all, or one by name, from a build directory (testing (§8.2)):

```
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
    -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests/io-by-shadow-doorbell
```

The documentation example above (`frontend-sharded-pollers`) is started by `check-docs`, which enables the controller and moves one block.

3.3.16 Limits

- At most 2047 I/O queue pairs (the MSI-X table size), at most 65535 entries per queue.
- SQEs are 64 bytes and CQEs 16 bytes; other sizes are refused at realize.
- One controller per PCI function. No SR-IOV, no NVM Subsystem Reset, no controller memory for PMR.
- No interrupt coalescing; the coalescing features only store values.
- Weighted round robin arbitration is not implemented: CAP.AMS advertises it, but each poller sweeps its queues in id order.
- A queue can have at most `size` commands outstanding, and a poller's rings hold 65535 requests each.
- Abort only reaches commands not yet fetched.
- Latency is a lower bound that depends on poller scheduling.

- The controller is not migratable.

3.3.17 Extension points

- **A new mode**: write a `FemuExtCtrlOps` table and a `nvme_register_*()` function, add the mode to the enum in `hw/femu/nvme.h`, to the `femu_mode` bound in `nvme_check_constraints()`, to `nvme_mode_from_token()` for `namespace_modes`, to `nvme_register_extensions()`, and to `femu_needs_ftl_thread()` and `femu_ftl_process_req()` if it charges time on the FTL thread. A mode that charges time on the poller adds it to `req->expire_time` in its `io_cmd`.
- **A new I/O command shared by all block modes**: add it to `nvme_io_cmd()` and to `nvme_io_effects()` in `hw/femu/nvme-caps.c`.
- **A new admin command, feature or log page**: `nvme_admin_cmd()` and `nvme_admin_effects()`; `nvme_set_feature()`, `nvme_get_feature()` and the support tables `nvme_feature_support[]` and `nvme_feature_cap[]` (with `nvme_fid_supported()` for a feature that needs something else present); `nvme_get_log()` and `nvme_log_support()`. An Identify bit that sums up commands or pages is derived in `nvme_caps_id_ctrl()`, not set by hand.
- **A new host-side cost**: add it next to the host-link and firmware-CPU models in `nvme_process_cq_cpl()`.
- **Anything that changes state the I/O path reads**: wrap it in `nvme_pause_pollers()` and `nvme_resume_pollers()`.
- **A new property**: define it in `femu_props[]` in `hw/femu/femu.c`, describe it in `hw/femu/femu-props.c`, check it in `nvme_check_constraints()`, and regenerate the property reference (docs maintenance (§8.3)).

3.3.18 Source map

File	Functions
`hw/femu/femu.c`	`nvme_init_pci()`, `nvme_init_cmb()`, `nvme_init_ctrl()` (Identify Controller, CAP), `nvme_check_constraints()`, `nvme_mmio_write()`, `nvme_write_bar()`, `nvme_process_db_admin()`, `nvme_process_db_io()`, `nvme_start_ctrl()`, `nvme_clear_ctrl()`, `nvme_reset_features()`, `femu_ftl_thread()`, `femu_ftl_process_req()`, `femu_needs_ftl_thread()`, `nvme_register_extensions()`, `nvme_register_extensions_ns()`, `femu_realize()`, `femu_exit()`
`hw/femu/nvme-admin.c`	`nvme_create_sq()`, `nvme_create_cq()`, `nvme_del_sq()`, `nvme_del_cq()`, `nvme_init_poller()`, `nvme_start_dataplane()`, `nvme_set_db_memory()`, `nvme_identify()`, `nvme_get_feature()`, `nvme_set_feature()`, `nvme_get_log()`, `nvme_abort_req()`, `nvme_admin_cmd()`, `nvme_process_aers()`, `nvme_process_sq_admin()`
`hw/femu/nvme-caps.c`	`nvme_admin_effects()`, `nvme_io_effects()`, `nvme_ns_io_effects()`, `nvme_log_support()`, `nvme_log_csi()`, `nvme_caps_id_ctrl()`
`hw/femu/nvme-io.c`	`nvme_poller()`, `nvme_process_sq_io()`, `nvme_update_sq_eventidx()`, `nvme_process_cq_cpl()`, `nvme_post_cqe()`, `nvme_rw()`, `nvme_io_cmd()`
`hw/femu/nvme-util.c`	`nvme_pause_pollers()`, `nvme_resume_pollers()`, `nvme_update_sq_tail()`, `nvme_update_cq_head()`, `nvme_update_cq_eventidx()`, `nvme_init_sq()`, `nvme_init_cq()`
`hw/femu/intr.c`	`nvme_isr_notify_io()`, `nvme_isr_notify_admin()`, `nvme_irq_update()`, `nvme_irq_mask_changed()`, `nvme_setup_virq()`, vector notifiers
`hw/femu/dma.c`	PRP and SGL mapping, `femu_dma_rw`
`hw/femu/lib/rte_ring.c`, `hw/femu/lib/pqueue.c`	the rings and the completion heap
`hw/femu/nvme.h`	`NvmeSQueue`, `NvmeCQueue`, `NvmeRequest`, `FemuCtrl`, `FemuExtCtrlOps`, `FemuPollerCtr`, register layout

3.3.19 Related pages

- Design overview (§3.2)
- Architecture (§3.1)
- Timing model (§6.2)
- Device property reference (§A.1)
- Log pages and counters (§B)
- Performance tuning (§5.12)

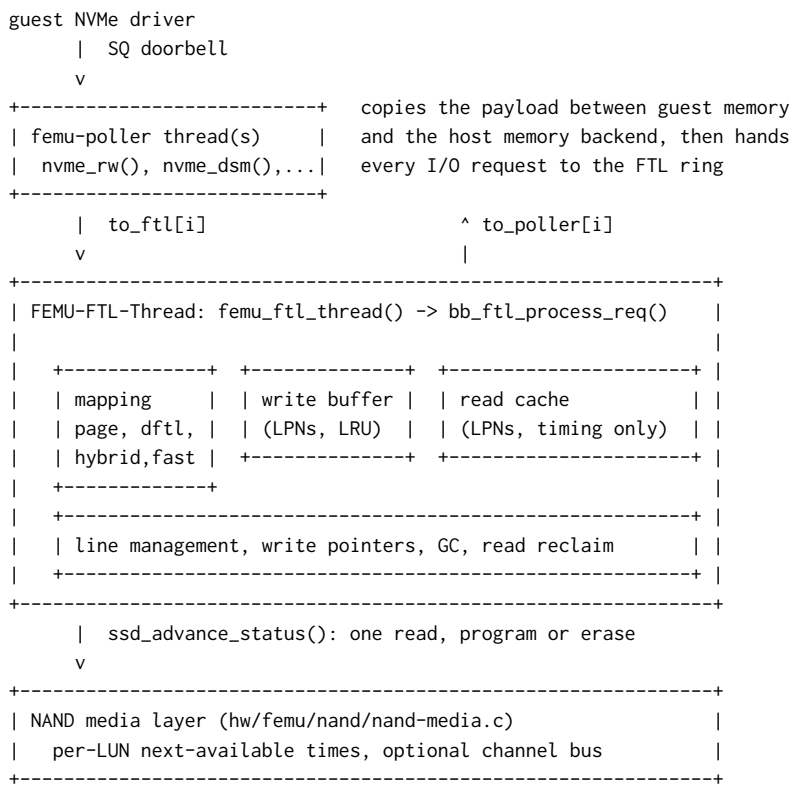
3.4 The BlackBox FTL

This chapter describes the flash translation layer (FTL) behind BlackBox mode (`femu_mode=1`): how it maps logical pages to NAND pages, how it organises NAND into lines and write pointers, how it collects garbage, and how its write buffer, read cache, wear and refresh models work. It is written for readers who want to know what the model does, where its time goes, and how to change it. To run the mode, read the BlackBox guide (§4.4) first; this chapter does not repeat its launch and guest steps.

The code lives in `hw/femu/bbssd/`. The same FTL serves CSD namespaces, which build it through the same `ssd_init()`. KV namespaces call `ssd_init()` for the geometry and NAND timing but run their own key-value FTL. A `femu-cxl-ssd` builds a private instance on its own worker thread.

NAND operation timing (per-LUN timelines, the channel bus, suspend, ECC tiers) is the subject of the NAND timing chapter (§3.5) in this directory, and of the timing model (§6.2). This chapter covers what the FTL asks of the NAND, not how long each operation takes.

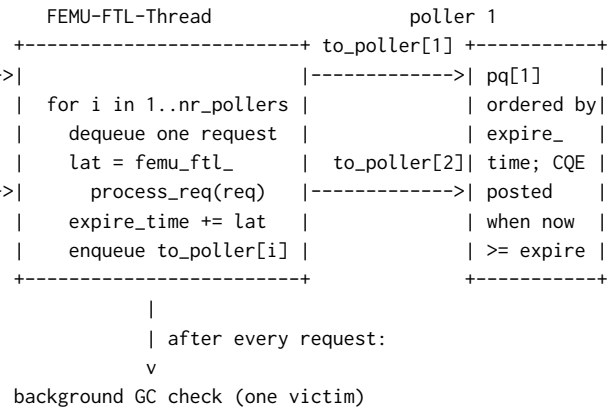
3.4.1 Place in the hierarchy



Two things follow from this picture.

- **The FTL models time and placement, not data.** The poller copies the payload to the host memory backend before the request reaches the FTL. The FTL tracks which NAND page each logical page would occupy and how long each operation takes, but it never stores the payload. The one exception is the power-loss model (`power_loss=on`), which runs the data copy on the FTL thread and keeps undo copies of buffered pages; see Power loss (§3.4.7).
- **The FTL returns a duration.** Each handler returns the time the request spends in the device, measured from the request's arrival time `stime`. The FTL thread adds it to `req->expire_time`, and the poller completes the request when the host clock reaches that time.

3.4.2 The FTL thread and its queues



- **One thread per controller.** `femu_ftl_thread()` in `hw/femu/femu.c` starts at realize when a namespace is bbssd, ZNS or CSD (or the controller shares a bbssd subsystem), after every namespace is built. It is joined at device removal.
- **One pair of rings per poller.** Each poller `i` feeds `to_ftl[i]` and reads `to_poller[i]`. The FTL thread visits the rings in turn and takes one request from each non-empty ring per pass, so no poller can starve another.
- **Every I/O command goes through the ring**, including Flush, Dataset Management and commands that already failed in the I/O layer. A failed request is not processed: `bb_ftl_process_req()` returns 0 for it, and the poller posts the carried error status.
- **No lock protects the FTL state.** The FTL thread is the only writer of the mapping, the lines, the write buffer and the caches. With Streams on, each request runs under `n->streams_lock`; on a shared subsystem namespace it runs under the subsystem's `ns_lock`. An NVMe controller linked to a `femu-cxl-ssd` runs its requests on its own FTL thread against the medium's FTL, under the medium's lock, which also serializes the medium's worker.
- **Admin commands that change FTL state pause the dataplane.** `nvme_pause_pollers()` waits until the FTL thread is outside a request (the `ftl_in_sweep` flag), so the run-time switch (admin opcode 0xEF), the volatile write cache feature, Format and Sanitize change FTL state with nothing in flight.

`bb_ftl_process_req()` in `hw/femu/bbssd/ftl.c` dispatches on the opcode:

Opcode	Handler	What the FTL does
Read	`ssd_read()`	translate each page; buffer hit, read cache hit or NAND read
Write	`ssd_write()`	foreground GC, read reclaim, then buffer or program each page

Opcode	Handler	What the FTL does
Write Zeroes	`ssd_write_zeroes()`	with the Deallocate bit: unmap; without it: program the range
Dataset Management	`ssd_trim()`	deallocate each range
Copy	`ssd_copy()`	read every source range, then write the destination
Compare, Verify	`ssd_read()`	read the range as a Read does, without the injected read faults; also after a miscompare, or a protection error of a Verify
Flush	`ssd_buffer_destage(ssd, 0, ...)`	program everything the write buffer holds
I/O Management Send	`ssd_fdp_update_ruhs()`	FDP only: update reclaim unit handles

Read and Write latency is the largest per-page latency of the request: pages on different LUNs proceed in parallel, and pages on one LUN queue on its timeline. Copy issues the destination write when the slowest source read ends, so it costs that read plus the write. A source in a NoSSD namespace costs nothing, and a Copy into a NoSSD namespace charges no source read. After the opcode handler, the thread runs one background GC step if the device is past the background watermark (Triggers (§3.4.6)).

3.4.3 Data structures

All of the FTL's state hangs off `struct ssd` (`hw/femu/bbssd/ftl.h`), one per bbssd or CSD namespace.

Physical page address

A `struct ppa` packs a NAND location into 64 bits:

```
 63  62          51 50      44 43  40 39      32 31            16 15             0
+---+--------------+----------+------+----------+----------------+----------------+
|rsv|  ch (12)     | lun (7)  |pl (4)| sec (8)  |   pg (16)      |   blk (16)     |
+---+--------------+----------+------+----------+----------------+----------------+
```

The field widths bound the geometry: at most 4096 channels, 128 LUNs per channel, 16 planes per LUN, 65536 blocks per plane, 65536 pages per block and 256 sectors per page. `bb_check_geometry()` refuses larger values, and also refuses a geometry whose total sector count exceeds `INT_MAX`, because the derived totals in `struct ssdparams` are `int`. The FTL always addresses whole pages; the `sec` field is unused by the mapping.

NAND state

```
ssd->ch[nchs]
  .next_ch_avail_time
  .lun[luns_per_ch]
     .next_lun_avail_time          <- the timeline every NAND op is bound by
     .pl[pls_per_lun]
        .blk[blks_per_pl]
           .vpc, .ipc              valid and invalid page counts
           .erase_cnt              P/E cycles of this block
           .read_cnt               reads since the last erase
           .pg[pgs_per_blk].status FREE, VALID or INVALID
```

A page goes FREE -> VALID when it is programmed (`mark_page_valid()`), VALID -> INVALID when its logical page is overwritten, deallocated or moved by a log-block merge (`mark_page_invalid()`), and back to FREE when its block is erased (`mark_block_free()`). A page that line GC relocates stays VALID until its

block is erased. Closing a stream's line marks its unwritten pages INVALID directly.

Mapping tables

```
maptbl: logical page -> physical page        rmap: physical page -> logical page
(one struct ppa per logical page)            (indexed by ppa2pgidx(), one u64 each)

 LPN    PPA                                   pgidx                    LPN
+-----+--------------------------+          +------------------------+-----+
|  0  | ch0 lun0 pl0 blk7 pg3    |--------->| ch0 lun0 pl0 blk7 pg3  |  0  |
|  1  | UNMAPPED                 |          | ch1 lun0 pl0 blk7 pg3  |  5  |
|  2  | ch3 lun1 pl0 blk2 pg0    |          | ...                    | ... |
| ... | ...                      |          | (stale copy)           | INV |
+-----+--------------------------+          +------------------------+-----+
```

`maptbl` is the forward map and `rmap` the reverse map that GC uses to find which logical page a valid physical page holds. Both have one entry per NAND page (`tt_pgs`), 8 bytes each: 32 MiB apiece for the default 16 GiB geometry. Every mapping scheme keeps these two tables as the source of truth.

Lines

```
typedef struct line {
    int id;              /* the block index this line spans */
    int ipc, vpc;        /* invalid and valid pages over the whole line */
    size_t pos;          /* slot in the victim priority queue, 0 if none */
    uint64_t close_time; /* when the line filled; age-based policies */
    uint64_t close_seq;  /* order in which lines filled; the FIFO key */
    uint64_t stream_tag; /* Streams: the stream whose data it holds */
    bool reclaiming;     /* being rewritten by read reclaim */
    ...
} line;
```

`struct line_mgmt` keeps a free list (a FIFO), a victim priority queue keyed by valid page count (by `close_seq` under `gc_policy=fifo`), and a full list.

3.4.4 Address mapping

From LBA to logical page

`ssd_lpn_range()` turns an LBA range into a range of logical page numbers (LPNs). A page is `secsz * secs_per_pg` bytes. The byte offset is the LBA times the LBA size, plus the namespace's offset in the backend:

```
LPN = (backend_offset + slba * lba_size) / page_size
```

So a namespace's LPNs start where its data starts in the backend. A controller with Namespace Management on uses offset 0, because each managed namespace has a private FTL. A request whose last LPN is at or beyond `tt_pgs` fails with LBA Out of Range.

Page mapping (`mapping=page`)

The default. `translate()` returns `maptbl[lpn]`. A write invalidates the old physical page, if any, and points the LPN at the new one. A read of an LPN with no mapping costs no NAND time.

DFTL (`mapping=dftl`)

DFTL keeps the page-level mapping on NAND in translation pages (TPs) and caches the TPs in use in a cached mapping table (CMT). The flat `maptbl` still answers every lookup. Without FDP the TPs also live on

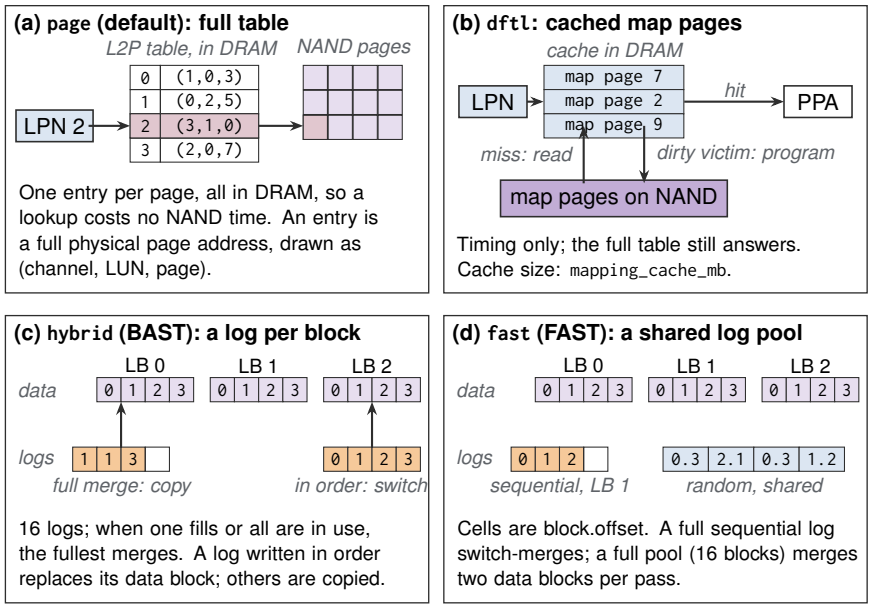


Figure 7: The four BBSSD mapping schemes (`mapping=`): the full page table, DFTL's cache of map pages, BAST's log block per data block, and FAST's shared log pool.

flash, in `hw/femu/bbssd/ftl-map-tp.c`:

```
DRAM (power-loss protected)              NAND
  CMT: whole TPs, CLOCK, dirty bit         TP lines (line kind LINE_TP),
  GTD: tp -> page of its current copy       written only by the TP pointer
  pending: changed TPs with no CMT slot     rmap of a TP page: tag | tp

host read/write of lpn, tp = lpn / (page_size / 8)
  CMT hit  --> no NAND cost; a write makes the slot dirty
  CMT miss --> evict a slot by CLOCK
                 dirty victim --> program it at the TP pointer, GTD = new
               --> read the GTD copy of tp (none: no read), fill the slot
```

- A TP holds `page_size / 8` mappings, 512 on a 4 KiB page. With `mapping_cache_mb` left at 0 the CMT is 4 MiB: 1024 TPs, 2 GiB of logical space on a 4 KiB page.
- The CMT caches whole TPs. This is a DFTL variant with a TP-granularity cache, as many real FTLs use. It is not the entry-granularity CMT of Gupta et al. (ASPLOS '09).
- Every change of a mapping reaches the TPs through one interface, `tp_note()`. A cached TP gets dirty. Any other TP goes pending, and the FTL publishes it later with one read and one program for all its changes.
- Collection publishes at the end of each victim. A paced victim publishes when it is empty, and its budget counts those programs. The end of each request publishes what is left.
- DSM Deallocate and Write Zeroes with Deallocate publish at the end of the command, and the command pays for it. A cached TP they changed only gets dirty. An uncached TP takes one program, and one read first when it has a flash copy. At the forced reserve the TP stays pending for a later publication. Write Zeroes without Deallocate translates each page as a write.
- Format and Sanitize discard every TP with no write-back: their pages go invalid, and the GTD, the CMT and the pending set empty. A CSD namespace takes neither command.
- Collection takes TP lines and data lines from one victim queue. It moves each valid page by its owner: a TP through the TP pointer and the GTD, a data page through the data path. A TP line never moves by

copyback.

- A TP program never takes one of the free lines that forced collection keeps, except to move a TP line. A dirty eviction or a publication with no page leaves the TP dirty or pending. So a host command does not fail for lack of TP space.
- The mapping DRAM is modelled as capacitor-backed, as on enterprise SSDs. The pending changes stand for a log of changes in that DRAM. The model does not size it: in the worst case it holds one entry per logical page, and `mapping_cache_mb` does not bound it. A power loss keeps the GTD, the CMT and the pending TPs. Flush and FUA need no extra metadata barrier; ordinary evictions still run.

Timing. Each translation has a ready time: the end of the TP read, or the start of the access when the TP needs no read.

```
read of lpn:   |-- TP read (tp's LUN) --|-- data read (lpn's LUN) --|
write of lpn:  |-- TP read --|
               |------------ data program -------------|   ends at the later
hot/cold on:   |-- TP read --|------------ data program -------------|
```

- A read starts its media read when the translation is ready, on any LUN. Translations of different TPs in one command run in parallel.
- A write programs at once, since placement does not need the old mapping; it ends when both are done. With `hot_cold_sep`, placement reads the old mapping, so the program waits for the translation.
- A CMT hit is ready when the fill of its slot ends. A dirty eviction starts after the fill of the victim, and runs in parallel with the read.
- A TP read waits for a program of the same TP that is still running.

Under FDP, TPs stay timing only. A miss reads a placeholder page on LUN `tp_id % tt_luns`, and a dirty eviction programs one. That traffic counts in no counter, and collection and deallocate cost no TP access.

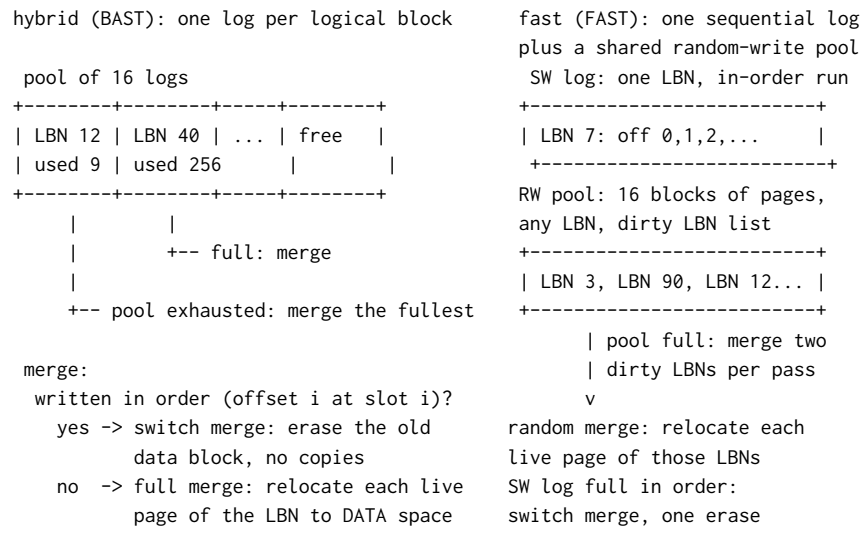
Space. With TPs on flash, realize keeps lines for the live TPs of the namespace, rounded up to whole lines, and for the TP pointer. Forced collection keeps at least two free lines. So the largest size a geometry exposes falls with `mapping=dftl`, and realize refuses a size without room.

Counters. `map-write-pages` and `map-read-pages` (query-femu, C0h bytes 280 and 288) count TP programs and reads. The write amplification factor adds the TP programs:

```
WAF = (nand_write_pages + gc_write_pages + map_write_pages + pad pages)
      / host_write_pages
```

Log-block mapping: BAST (`mapping=hybrid`) and FAST (`mapping=fast`)

The two log-block schemes write every page through a separate LOG write pointer and model the merges a log-block FTL has to run. Like DFTL, they use `maptbl` for correctness, so a read always finds the newest copy. A logical block (LBN) is `pgs_per_blk` consecutive LPNs.



- **hybrid** follows BAST (Kim et al. 2002): one log per logical block, from a pool of 16. Each program consumes a log slot, overwritten or not; neither an overwrite nor a deallocate frees a slot, only a merge does. After every programmed page, if a log is full or every log in the pool is taken, the fullest log is merged. A log written strictly in order switch-merges for the cost of one erase. Any other log full-merges: every live page of its LBN is read and programmed into DATA-class space, counted as GC writes. Hybrid counts its merges for log page C0h. The scheme models merge cost, not BAST's physical block placement: logs of different LBNs share physical lines, and line GC still runs underneath and can add copies.
- **fast** follows FAST (Lee et al. 2007): a single sequential-write log for an in-order run that starts at offset 0 of an LBN, and a shared pool of 16 blocks' worth of pages for everything else. When the pool fills (or the dirty-LBN list reaches its 4095 limit), a reclaim relocates the live pages of up to two dirty LBNs, and repeats on later writes until the pool is no longer full. A full in-order sequential log switch-merges with one erase. The sequential log is released only when it fills in order; a broken run keeps holding it, and later sequential runs go to the shared pool. FAST reclaims once per request, not once per page, and its merge counts are not exported.

Merge reads, programs and erases are charged to the request that triggered the merge, and the relocated pages count in the write amplification factor.

Choosing a scheme

`mapping`	Models	Extra cost charged	Extra write pointer	Counters
`page`	a full page map in DRAM	none	none	none
`dftl`	a page map on NAND with a cache of translation pages	translation page read on a miss, program on a dirty eviction, publication of the changes collection and deallocate make	TP (without FDP)	C0h bytes 280 and 288
`hybrid`	BAST log blocks	switch and full merges per log	LOG	C0h bytes 88 to 111
`fast`	FAST log blocks	bounded random merges, sequential switch merges	LOG	none exported

Streams and FDP need `page` or `dftl`.

3.4.5 Lines, superblocks and write pointers

A line is a superblock

A line is the block with the same index on every plane of every LUN of every channel. There are `blks_per_pl` lines, each `nchs * luns_per_ch * pls_per_lun` blocks wide. By default a write pointer fills a line in this order: channel first, then LUN, then plane, then page. `plane_first` changes it for bbssd and CSD, as described below.

```
           ch0     ch1     ch2   ...  ch7
lun0  pg0 [ 0 ]   [ 1 ]   [ 2 ]       [ 7 ]     numbers are the order in
lun1  pg0 [ 8 ]   [ 9 ]   [10 ]       [15 ]     which the write pointer
...                                               hands out pages of line N
lun7  pg0 [56 ]   [57 ]   [58 ]       [63 ]     (one plane per LUN)
lun0  pg1 [64 ]   [65 ]   ...
...
lun7  pg255                          [16383]   -> line full, take the next
                                                   free line
```

Consecutive pages land on different channels, then different LUNs, so a large write or a burst of small ones spreads over the whole device. With several planes, the pointer sweeps every channel and LUN on plane 0, then on plane 1, and so on, before the page index moves on.

`plane_first` (bbssd and CSD without FDP, off by default) moves the plane to the front: plane, then channel, then LUN, then page. Pages in a row then fill the planes of one LUN, so with `mp_program` two pages in a row are one multi-plane program. The cost is less spread for short writes.

```
2 channels x 1 LUN x 2 planes, one page row
default:      ch0 pl0 [0]  ch1 pl0 [1]  ch0 pl1 [2]  ch1 pl1 [3]
plane_first:  ch0 pl0 [0]  ch0 pl1 [1]  ch1 pl0 [2]  ch1 pl1 [3]
              \___ one multi-plane program __/
```

Why there are several write pointers

Each write pointer holds one line open and appends to it. Data written through one pointer shares lines, so the lines die together or not. Keeping data of different lifetimes apart is what makes GC victims mostly invalid. The FTL has these pointers:

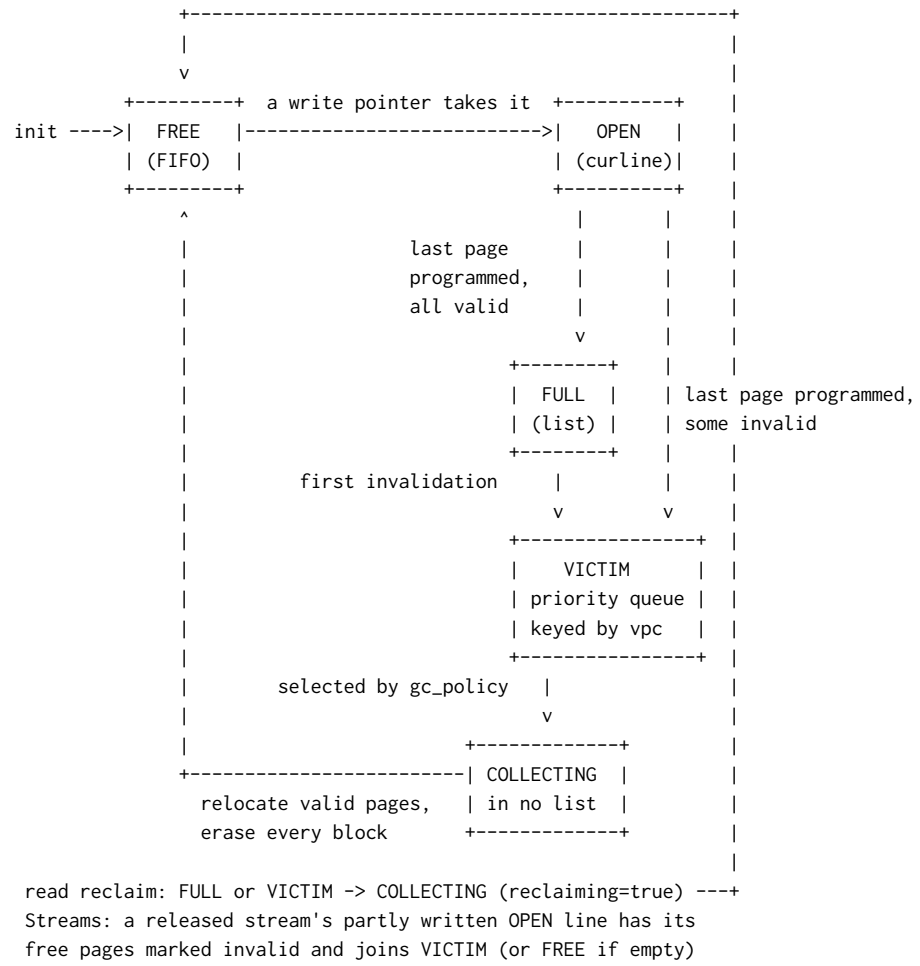
Pointer	In `struct ssd`	Takes	Opened
data	`wp`	host writes (only first writes with `hot_cold_sep=on`), GC relocations, read reclaim, merge relocations	at init
hot	`hot_wp`	overwrites of mapped pages, with `hot_cold_sep=on`	on first use
log	`log_wp`	every host write under `hybrid` or `fast`	on first use
stream	`stream_wp[i]`	writes of open stream `i`	on first use
stream GC	`stream_gc_wp`	relocated pages of a released stream	on first use
reclaim unit	per handle	FDP placement; see FDP (§4.5)	FDP init

When a hot or log pointer cannot get a line, the write falls back to the data pointer. Every open pointer pins a line, so `bb_check_capacity()` adds one reserved line per pointer the configuration can hold open (Capacity (§3.4.12)).

Hot/cold separation

With `hot_cold_sep=on` and page or DFTL mapping, `prepare_write()` sends a write to the hot pointer when its LPN is already mapped. An overwrite is the cheapest predictor of another overwrite, so hot lines fill with pages that are invalidated together. GC relocations go through the data pointer, so pages that survive a collection join the cold data. The log-block schemes ignore the setting, but the extra reserved line is still counted. FDP refuses it.

Line state machine



An open line belongs to no list; invalidations only lower its valid count. When a line fills, `close_time` and `close_seq` are set. The free list is a FIFO: lines are taken from the head and returned to the tail. The FTL has no explicit wear levelling; the FIFO spreads erases over the free lines.

3.4.6 Garbage collection

The code is in `hw/femu/bbssd/ftl-line-gc.c`.

Triggers and watermarks

The two watermarks are stored as free-line counts:

```
gc_thres_lines      = (int)((1 - gc_thres_pcent/100)      * tt_lines)
gc_thres_lines_high = (int)((1 - gc_thres_pcent_high/100) * tt_lines),
```

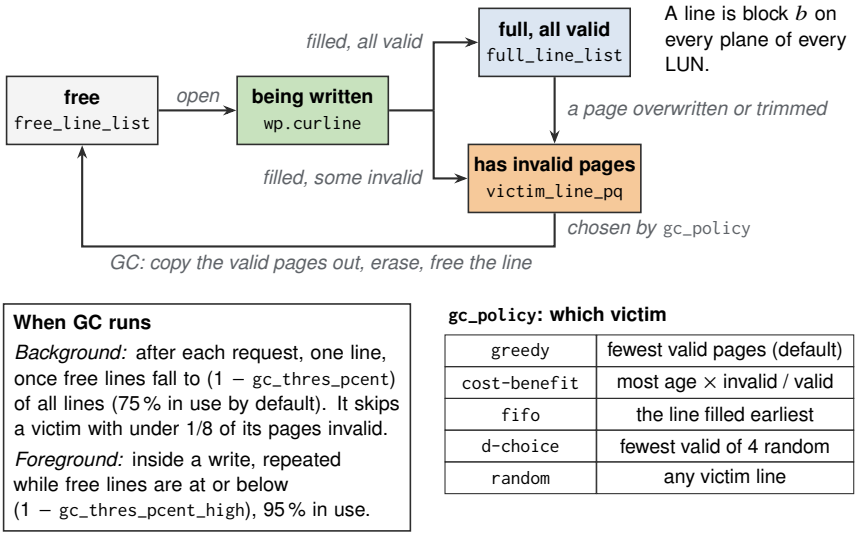


Figure 8: BBSSD line states and garbage collection: when a line moves between the free, written, full and victim sets, when background and foreground GC run, and how each `gc_policy` picks a victim.

```
                    at least 1 with hot_cold_sep, Streams, or a hybrid
                    or fast mapping (bb_gc_forced_lines())
```

The floor matters below 20 lines, where the default high watermark rounds to zero. A second write pointer can then take the last free line while the data pointer, which GC writes into, is nearly full, and GC would have nowhere to put a victim's pages (with Streams, GC refuses such a victim outright). One free line always holds a whole victim. The floor costs those small geometries one line of exposable capacity.

Kind	When	Where	Victim filter	How many
Background	free lines <= `gc_thres_lines`	after every request that reaches the FTL without an error, any opcode	the chosen line must have at least 1/8 of its pages invalid	one line
Foreground (forced)	free lines <= `gc_thres_lines_high`	at the start of a write, and before every page that a write, a write-back of the buffer, or Write Zeroes without Deallocate programs	none	repeats until above the watermark or no victim

Forced GC runs per page, not once per command: one command can program more lines than the watermark keeps free, and a collection that starts after the last line is gone has nowhere to move pages to.

A GC step returns -1 when there is no victim, when the background filter rejects it, or when the victim's valid pages cannot all be moved; the forced loop then stops. With Streams on, a step also fails when it frees nothing, because lines of distinct retired streams cannot be combined, or when the victim holds valid pages and no line is free.

Victim policies

`gc_policy` selects one entry of `femu_ftl_policies[]`:

gc_policy	Chooses	Cost per choice
greedy	the line with the fewest valid pages: the top of the priority queue	O(log n)
random	a uniformly random line of the queue, drawn from the seeded generator; a background step puts it back if it fails the 1/8 filter	O(log n)
cost-benefit	the largest age x (1 - u) / 2u, with u = vpc / pgs_per_line and age = now - close_time, compared in 128-bit integers; a line with no valid pages always wins	O(n) scan
fifo	the line that filled first (lowest close_seq), whatever its valid count: the top of the queue, which this policy orders by close_seq	O(1) to find, O(log n) to remove
d-choice	the fewest valid pages among 4 queue slots drawn from the seeded generator; a slot can be picked twice	O(1) sample

Only lines in the victim queue are candidates. A line that closed with every page valid stays in the full list until something invalidates one of its pages. Under fifo the queue is keyed by close_seq, which does not change while a line is queued, so an invalidation lowers the valid count without moving the line.

random, d-choice and the FDP gc_strategy=2 (random) draw from a splitmix64 generator in each namespace's FTL (ftl_gc_rand()), seeded from gc_seed (default 0). The same configuration, seed and command sequence pick the same victims and give the same write amplification; give each run a different gc_seed to vary it. The other policies use no random numbers. cost-benefit still reads the host clock for line age, so its picks depend on when commands arrive.

Collecting a line

```
do_gc(force)
  |
  v
victim = policy->select_victim_line(force)  -- none? return -1
  |
  v
move: for ch, lun, pl, each page of block (ch, lun, pl, victim->id):
  if VALID:
    new = next page of the data pointer        (stream pointer for
          (takes a free line if it has none)    Streams data)
    none? requeue the victim, return -1
    NAND read at the old page                  (GC_IO)
    lpn = rmap[old]
    maptbl[lpn] = new, rmap[new] = lpn
    old page -> INVALID, rmap[old] = none
    NAND program at the new page               (GC_IO)
    gc_write_pages++
  |
  v
erase: for ch in 0..nchs-1, lun in 0..luns_per_ch-1:
  mark each plane's block free: erase_cnt++, read_cnt = 0
  one multi-plane erase of the LUN's blocks    (GC_IO, tplebsy between
                                                planes)
  |
  v
line -> free list tail
```

The relocated page goes wherever the data pointer is, not to the victim's LUN, unless gc_copyback is on (next section). Every valid page is moved before any block is erased. If the data pointer runs out of lines part way, the victim goes back to the victim queue (or the full list, if nothing was moved from a full line) holding the

pages it still has, and nothing is erased. The pages that were moved are already invalid in the victim, so its counts stay true and a later step moves only what is left. Writes that then find no line fail with Capacity Exceeded. The watermark floor, per-page forced GC and the capacity reserve keep a valid configuration from getting there.

Copyback (`gc_copyback`)

Copyback reads a page into the die's page register and programs it into another block of the same die and plane. The data never crosses the channel. It is off by default.

A copyback skips ECC. ECC decode and correction run in the controller, and the data of a copyback never reaches the controller. On a real SSD the bit errors of the page are not corrected, so they can add up over repeated copybacks:

```
off-chip move:  die --tR--> page register --channel--> controller (ECC decode,
                correct) --channel--> page register --tPROG--> die
copyback:       die --tR--> page register --tPROG--> die
                (no channel, no ECC: bit errors move with the data)
```

FEMU models no ECC decode and no bit errors, so in FEMU a copyback changes only timing. Two rules exist because a copyback skips ECC:

- A page whose read has extra ECC time (`ecc_step_ns` wear or retention tier) moves off-chip, so the controller would correct it.
- Read reclaim and retention refresh always move off-chip. They rewrite data so that the controller corrects it.

By default the data pointer steps to the next die on every page (channel fastest, then LUN, then plane), so a victim walked in its own order meets its destination die only once in `nchs x luns_per_ch x pls_per_lun` moves. With `plane_first` it stays on one die for `pls_per_lun` pages. With `gc_copyback` on, line collection walks the victim in the order of the destination dies:

```
while the victim holds valid pages:
  dst = next page of the destination pointer
  src = next valid victim page on dst's channel, LUN and plane
        else next valid victim page in reclaim order
  same channel, LUN and plane, and zero extra ECC time --> copyback
  otherwise                                            --> read + program
```

Move	Charged
Copyback, staged channel	src cmd/addr, tR, status and program cmd/addr, tPROG; no data phase
Copyback, no channel model	tR then tPROG on the die
Off-chip	the ordinary GC read (with data-out) and program (with data-in)

- The walk changes only which victim page goes to which destination page. The line, the pointer and the pages moved stay the same, so wear levelling still fills the worn line exactly.
- The ECC rules above are timing rules: FEMU stores no bit errors, and nothing limits how often one page moves by copyback. A line queued for refresh that collection takes first also moves off-chip.
- FDP collection uses the same walk, with the pointer of the unit it writes. For a Persistently Isolated handle that is the handle's own collection unit. For an Initially Isolated one it is the current unit of the last handle.

- The later bus phases of a channel share a queue of 32 bookings. A burst of collection fills it, and the channel is then held to the end of each new phase, as for an off-chip burst. Copyback books one window per move, where an off-chip move books two.
- An off-chip GC program does not wait for its own read; copyback chains the two. The saving shows in the channel time, with `pg_xfer_lat` set.
- Log page C0h offset 248 counts the pages moved by copyback, which offset 16 also counts.

Refused with `gc_pace` (its steps keep a cursor in reclaim order), `pe_suspend` (a suspended read would not move the booked program command), the hybrid and fast mappings (they merge), `cxl_ssd`, `ns_mgmt` and shared namespaces (their FTL comes from another device), and `namespace_modes`.

How GC time is charged

GC operations enter the NAND model with `stime = 0`, which the media bridge replaces with the current host time. Each one moves its LUN's next-available time forward, and the latency it returns is discarded. GC time therefore reaches the host only through the LUN timelines: a later host read or program on a LUN that GC is using starts when GC is done with it.

```
LUN 3 timeline   |--host W--|--GC R--|--GC W--|--GC W--|--erase-------|--host R--|
                            ^                                       ^
                            GC starts at "now"                      a read that
                                                                    arrived here
                                                                    waits until
                                                                    the erase ends
```

- In background GC the request that triggered it has already been timed, so it pays nothing. The requests that follow pay, on the LUNs GC occupied.
- In foreground GC the write itself waits, because its own programs queue behind the GC operations on the same LUNs.
- Admin opcode 0xEF code 2 turns GC timing off (`enable_gc_delay`): GC still moves the pages and erases the blocks, but issues no NAND operations. Code 1 turns it back on.

The LUN field `gc_endtime` is updated as GC programs and erases, and the channel field is never written; nothing reads either.

Paced collection (`gc_pace`)

A whole-line pass books every copy of the victim at one instant, so the next writes wait for all of them. With `gc_pace` on, collection copies the victim a few pages at a time instead, in step with host writes.

```
end of a request, free lines <= gc_thres_pcent line count, no paced line:
  victim = policy->select_victim_line(false); mark it reclaiming
  V0 = its valid pages; R = pages_per_line - V0 (the pages it frees)
each host page programmed (direct, write-back, Write Zeroes):
  credit += V0
  while credit >= R: copy the victim's next valid page; credit -= R
every step:
  budget: pages left before free lines reach the forced threshold
  if the copies left and the host pages they need do not fit: copy more now
victim has no valid page left:
  erase it on every LUN (in parallel), then free or retire it
forced pass (free lines <= gc_thres_pcent_high):
  finishes the paced victim first, then collects as before
```

The line is empty after about R host pages, which is the space it gives back. Each copy still enters the NAND

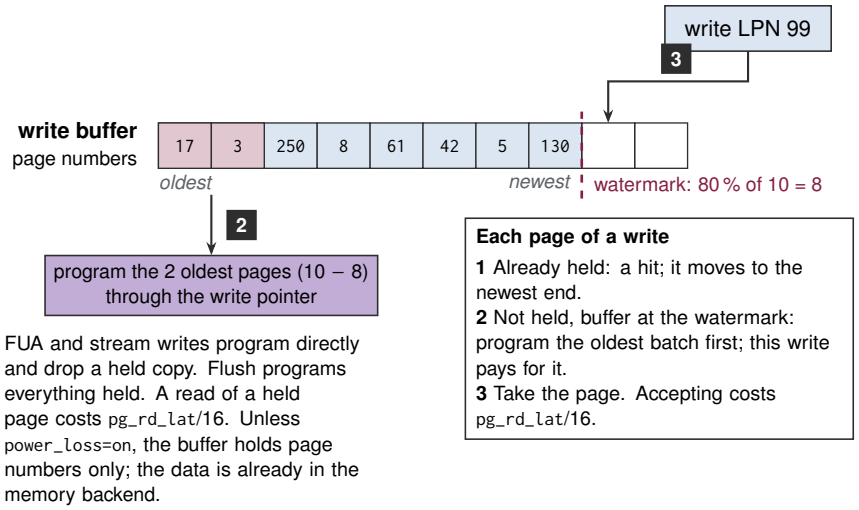


Figure 9: The BBSSD write buffer with `buffer_size=10` and `buffer_thres_pcent=80`: a write that finds the buffer at its watermark first programs the two oldest pages and pays for them.

model at "now", as other GC does, so a host write waits for about one copy instead of a whole line. The end of each request takes the place of the background pass: it chooses the victim, and it finishes one that deallocation or overwrites emptied. Only host programs earn credit. A copy that finds no room stops pacing until the next request.

Pacing spreads the work; it does not bound latency. A large write does its share in one call, and read reclaim and wear levelling still move whole lines. It uses the boundary of `blk_pe_limit` (one plane per LUN, one namespace, no buffer, FDP, Streams, CXL, hot/cold or hybrid mapping). Log page C0h counts the copies at offset 224, the lines it finishes at 232, and the lines the forced pass or the budget had to finish at 240.

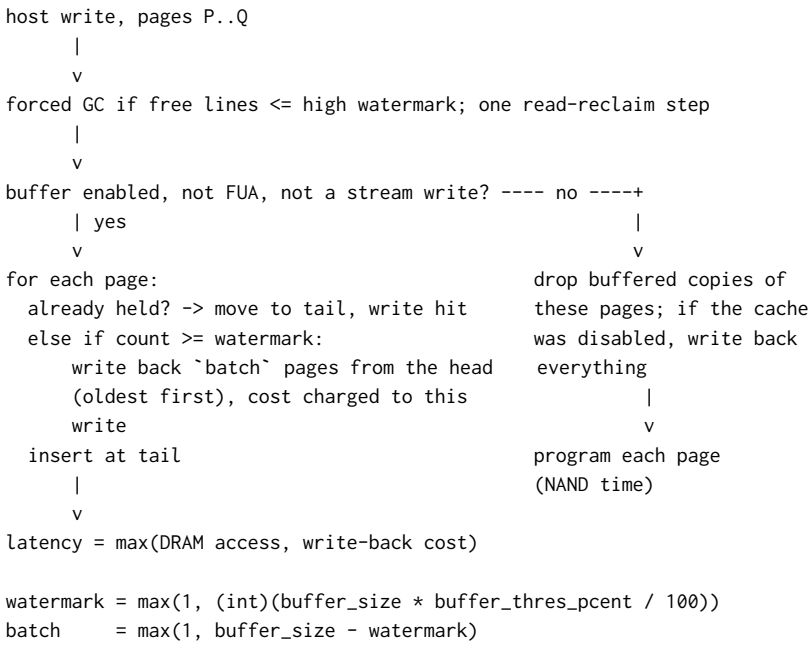
FDP reclaim units

With Flexible Data Placement the FTL places data in reclaim units, one line each, through per-handle write pointers, and collects reclaim units instead of lines (`do_gc_fdp_style()` in `hw/femu/bbssd/ftl-fdp.c`). `gc_strategy` picks that victim policy, or `gc_policy` by name; the FTL knobs listed under Interactions (§3.4.15) are refused. The FDP chapter (§4.6) and the FDP guide (§4.5) describe it.

Both collectors record a page move through one helper, `ssd_gc_move_page()`, and time it through `ssd_gc_charge_move()`. Each keeps its own destination and write frontier: line GC writes to the data or stream pointer, FDP to the handle's collection reclaim unit.

3.4.7 Write buffer

The write buffer models DRAM in front of the NAND. It holds logical page numbers, not data: a write it accepts costs a DRAM access, and the program is charged later to whatever forces the page out. Repeated writes to a buffered page cost one program. The code is in `hw/femu/bbssd/ftl-datapath.c`. Under FDP each held page also keeps the placement handle of its last write, and its write-back goes to that handle's reclaim unit (`fdp_buffer_destage()` in `hw/femu/bbssd/ftl-fdp.c`; see the FDP chapter (§4.6.11)).



- **Structure.** A tail queue in order of last write, LRU (`write_buffer`) and a GLib tree keyed by LPN (`wb_tree`) for lookup. Capacity is `buffer_size` pages.
- **Admission is per page.** A command larger than the buffer cannot push it past its size: it writes back a batch whenever the buffer is at the watermark. If a write-back frees nothing because the device is out of lines, the write fails with Capacity Exceeded.
- **Write-back** (`ssd_buffer_destage()`) takes pages from the head (the least recently written), runs forced GC as needed, charges the DFTL lookup and programs each page exactly as a direct write would. Hybrid merges after each page as usual; FAST merges once per batch. A page costs the same buffered or not; only the request that pays differs.
- **Reads.** A read of a buffered page costs one DRAM access and does not reach NAND, even when the buffer has stopped accepting writes.
- **FUA and stream writes** program directly. A direct write first drops any buffered copy of its own pages, so an older version is never programmed after the newer one.
- **Deallocate** drops buffered copies before unmapping, so discarded data is never programmed later.
- **Flush** reaches the FTL like any other command and writes back the whole buffer, charged to the Flush.
- **Volatile write cache.** With `vwc=1` the controller advertises a cache. When the host turns it off with feature 06h, the buffer stops accepting writes and the next write drains it. With `vwc=0` the buffer still works, and the host has no way to turn it off; Linux sends no Flush to a controller that advertises no cache.

`host_write_pages` counts pages when the host writes them and `nand_write_pages` when they are programmed, so a buffer that absorbs overwrites makes the write amplification factor drop below 1.

Power loss

With `power_loss=on`, the poller no longer touches payload: the FTL thread runs the whole I/O command (`nvme_power_io()`), so the data copy and the buffer change together. The first time a page enters the buffer, the FTL saves the page's previous bytes and per-LBA state as an undo record. When the page is programmed the record is dropped. Setting the QOM property `simulate-power-loss` (runtime properties (§A.2.1)) resets

the controller, restores every buffered page from its undo record, empties the buffer and counts an unsafe shutdown.

In this mode a normal shutdown, turning the cache off and Sanitize first write back every namespace's buffer; Flush, Format, Dataset Management, Copy, Write Zeroes (except on a PI-formatted namespace) and Write Uncorrectable write back their own namespace's. If the device has no line to write to, they fail with Capacity Exceeded (a shutdown sets Controller Fatal Status instead) rather than claim the data is durable. An FUA write programs a whole NAND page, so buffered neighbours in that page become durable with it.

3.4.8 Read cache

`read_cache_mb` adds a DRAM read cache in front of NAND reads (`hw/femu/bbssd/ftl-cache.c`). Like the write buffer it holds LPNs only.

```
host read of a mapped lpn (not in the write buffer)
      |
      v
lookup in an open-addressed hash over capacity slots
   hit:  cost pg_rd_lat / 16, no NAND read, no block read count
   miss: insert (evict by cache_evict if full), then NAND read
```

- Capacity is `read_cache_mb` MiB divided by the page size.
- Only reads fill it. A host overwrite, a deallocate and a power cut remove the page's entry. A GC relocation does not, since the content is the same.
- `cache_evict` picks the victim: `clock` (second chance), `random` (a fixed-seed generator, so runs repeat), `lru` (exact, by scan) or `arc`, a scan-resistant 2Q variant that evicts pages read once before pages read twice. It is not full ARC.
- Hit and miss counts are kept in `ssd->rcache` but not exported.

3.4.9 Deallocate, Write Zeroes, Format and Sanitize

- **Dataset Management deallocate** (`ssd_trim()`) unmaps every LPN of every range: it drops the buffered copy, invalidates the physical page, clears `maptbl` and `rmap`, and drops the read cache entry. A log-block scheme does this through its `trim()` hook. A range past the device is skipped. The latency is `trim_lat_ns` times the number of ranges, or 0.
- **Write Zeroes with Deallocate** unmaps the range in the same way and costs no time. **Without Deallocate** it programs the range directly, after forced GC if needed, and counts the pages as host writes.
- **Format** and **Sanitize** call `bbssd_deallocate_all()` on BlackBox namespaces (not CSD), which unmaps every LPN of the namespace's FTL. The lines keep their wear.

ONCS bit 0x8 (Write Zeroes) and 0x100 (Copy) are on by default; `oncs` can clear them.

3.4.10 Wear, read reclaim and retention refresh

Program/erase cycles

Every erase increments the block's `erase_cnt` and the FTL-wide `total_erases`. SMART Percentage Used is

```
percentage_used = min(255, total_erases * 100 / (tt_blks * rated_pe_cycles))
```

where `rated_pe_cycles` is `pe_cycles_rated`, or the rating of `nand_cell_type` (SLC 100000, MLC 3000, TLC 1000, QLC 300), or 0, in which case the field reads 0. With `blk_pe_limit`, the denominator is the sum of

every block's own erase limit at start, spare lines included, so the figure only grows. `nand_bad_blocks` marks a number of blocks as factory bad for SMART Available Spare only: `100 - bad * 100 / tt_blks`. Placement ignores them. With several namespaces the controller reports the most worn namespace and the lowest spare.

With `ecc_step_ns` set, a block's erase count (one tier per 750 erases) and the age of its line add time to each read. That model belongs to the NAND timing chapter.

Read-only end of life (`eol_read_only`)

With `blk_pe_limit`, a block wears out on the erase that takes it to its limit. The check runs after the erase of a collected line, when the line holds no data:

```
erase of line L takes a block to its limit
  spare in every plane that needs one   --> swap blocks, L stays
  eol_read_only=off                     --> retire L, or keep the block
                                            (overworn, warning bit 2)
  spare_lines > 0, a pool is empty      --> READ-ONLY, L retires
  no pools, U - 1 >= floor              --> retire L
  no pools, U - 1 <  floor              --> READ-ONLY, L retires
a host write finds no line to program   --> READ-ONLY (write goes on)

U     = lines - spare - reserve - retired
floor = namespace lines + forced collection lines + 2
```

Pollers change the backing store before the FTL sees a write, so a write that got past admission must be able to finish. Realize holds back reserve lines: enough pages for the largest command, which MDTS, WZSL and MCL bound. One 64-bit word holds the read-only bit, the admitted requests and the pages they may still program.

```
poller, before a write-class command changes data
  cost = pages it may program (DSM, DEAC Write Zeroes, Write Uncorrectable: 0)
  CAS on the word: read-only bit set     --> fail 182h, nothing changed
                   pages + cost > reserve --> leave it in the queue
                   else add 1 request and cost pages
FTL thread
  after the request, or nvme_free_sq() for a dropped one: subtract them
  at the end of life: atomic OR of the read-only bit, then the reserve
                   lines join the free list
```

After the OR no admission succeeds, so the admitted writes hold no more pages than the reserve has. Collection, read reclaim, wear levelling, pacing and padding stop for good, so the drain needs no erase. Format NVM fails with Command Sequence Error while admitted requests wait for the FTL, since their cost was set in the old LBA format.

Read reclaim and retention refresh

```
host NAND read of block B in line L
      |
      +-- read_reclaim_limit set and B.read_cnt >= limit   --+
      +-- retention_limit_sec set and now - L.close_time >= -+--> queue L
          limit                                               (one line at
                                                               a time)
next host write:
      |
      v
do_read_reclaim(): queued line, high watermark not reached,
at least 2 free lines, L full or in the victim queue?
      | yes
      v
take L out of its list, reclaiming = true, collect L like GC
read_reclaims++ or retention_refreshes++
```

Every NAND read the FTL issues, host, GC or merge, increments its block's `read_cnt` (GC reads are issued only while GC timing is on); an erase resets it. Reads answered by the write buffer, the read cache or a DFTL translation page do not count. The rewrite happens on a write, where relocation already costs something, so a read never stalls behind a whole line. One line is refreshed per write at most. A line that nothing reads is never refreshed, and a workload that only reads refreshes nothing: FEMU runs no background media scan. The request is dropped if GC collects the line first, or if the line is still open for writing when the next write checks it.

3.4.11 Fault insertion

`err_read_unc_ppm` and `err_write_fail_ppm` turn a rate into a period, `1000000 / ppm`, at least 1. Every Nth Read command completes with Unrecovered Read Error (with DNR, as every Unrecovered Read Error in FEMU is) and every Nth Write command with Write Fault. The counters run per namespace FTL, so a run repeats exactly. The command is still timed and, for a write, still programmed and mapped; only its status changes. The injected counts feed SMART Media and Data Integrity Errors. Write Zeroes and Copy are never failed.

3.4.12 Over-provisioning and capacity

The reserve

`bb_check_capacity()` in `hw/femu/bbssd/bb.c` refuses a namespace that would leave GC no room. The reserve, in lines:

```
reserve = gc_thres_lines_high      forced watermark, with its floor
        + 1                        data pointer
        + 1 if hot_cold_sep        hot pointer
        + 1 if mapping is hybrid or fast   log pointer
        + streams.max + 1 if streams       stream and stream GC pointers

usable  = (blks_per_pl - reserve) * pages_per_line * page_size
refused when the namespace is larger than usable,
or when blks_per_pl <= reserve
```

Each namespace has its own FTL built from the whole geometry, so the check applies per namespace.

Sizing the namespace

Without `op_pcent`, the backend is `devsz_mb` MiB, split across the namespaces, and the spare area is whatever the geometry has beyond that. With `op_pcent`, the backend is the raw NAND capacity and each namespace gets `raw * 100 / (100 + op_pcent) / namespaces`, rounded down to 512 bytes (an even split unless `namespace_sizes` is given); `devsz_mb` is ignored.

Worked example: the default geometry

The default properties describe 8 channels x 8 LUNs x 1 plane x 256 blocks x 256 pages x 8 sectors x 512 bytes:

```
page            = 8 x 512                    = 4 KiB
NAND pages      = 8 x 8 x 1 x 256 x 256      = 4,194,304
raw capacity    = 4,194,304 x 4 KiB          = 16 GiB
lines           = blks_per_pl                = 256
blocks per line = 8 x 8 x 1                  = 64
pages per line  = 64 x 256                   = 16,384   (64 MiB)

background GC   = (int)(0.25 x 256)          = 64 free lines  (75% used)
forced GC       = (int)(0.05 x 256)          = 12 free lines  (95% used)
victim filter   = 16,384 / 8                 = 2,048 invalid pages

reserve         = 12 + 1                     = 13 lines
usable          = 243 x 64 MiB               = 15,552 MiB

devsz_mb=12288 (the launcher):  12 GiB = 192 lines of data
   -> the data pointer holds a line from init, so free lines reach
      64 and background GC starts as the last of the 192 lines is
      opened, just before one full pass completes
op_pcent=25:  16 GiB x 100 / 125             = 13,107.2 MiB exposed
op_pcent=7:   16 GiB x 100 / 107             = 15,312.1 MiB exposed

host memory for the FTL: maptbl 32 MiB + rmap 32 MiB
                         + page status 16 MiB + block and line arrays
```

With the default `devsz_mb` of 1024 the same geometry exposes 1 GiB, about 6% of the NAND, and GC barely runs. To give GC work on a small device, shrink the geometry instead; this one has 512 MiB of NAND and exposes about 410 MiB:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25
```

3.4.13 Run-time controls

Admin opcode 0xEF changes the FTL while the guest runs. Only a BlackBox controller accepts it. Codes 1 to 4 apply to every namespace with an FTL, with the dataplane paused; codes 5 to 7 act on the controller.

CDW10	Effect
1	GC operations take NAND time (the default)
2	GC operations take no NAND time
3	flat read, program, erase times back to 40 us, 200 us, 2 ms
4	flat read, program and erase times to 0
5	reset the poller completion counters
6, 7	turn the per-request log lines on or off

Codes 3 and 4 set the built-in flat times, not the values given on the command line, and do not affect `nand_cell_type` tables. See changing timing at run time (§6.2.11).

3.4.14 Statistics

Vendor log page C0h

`nvme_collect_media_stats()` in `hw/femu/nvme-admin.c` sums the FTL counters over the controller's bbssd, CSD and KV namespaces. log-pages-and-counters.md (§B.1) lists the offsets.

C0h field	FTL counter	Moves when
WAF x 1000	(nand + gc) x 1000 / host	after the first host write
host write pages	`host_write_pages`	Write, the destination of Copy, and Write Zeroes without Deallocate, per page, buffered or not
GC write pages	`gc_write_pages`	GC, read reclaim and log-block merges relocate a page
NAND write pages	`nand_write_pages`	a host page is programmed
max block reads	largest `read_cnt`	NAND reads; reset by erase
read reclaims, retention refreshes	`read_reclaims`, `retention_refreshes`	a queued line is rewritten
buffer reads and read hits	`sp.read_cnt`, `sp.read_hit_cnt`	host read pages, and those the buffer held
buffer writes and write hits	`sp.write_cnt`, `sp.write_hit_cnt`	host write pages, and those already buffered
hybrid switch, full merges, merge erases	`struct femu_map_hybrid`	`mapping=hybrid` only

The buffer read and write counts move with or without a buffer, so their ratio is the hit rate. Telemetry log 07h captures the same 512 bytes.

SMART and Endurance Group

Field	Source
Percentage Used	`ssd_percentage_used()`, and `zns_percentage_used()` for a zoned namespace; the most worn namespace
Available Spare	`ssd_available_spare()`, lowest namespace; below 20 sets the spare critical warning; 0 once the media is read-only
Critical Warning	bit 0 spare below 20, bit 1 temperature, bit 2 an overworn block, bit 3 media read-only; byte 6 (EGCWS) and the Endurance Group log repeat bits 0, 2 and 3
Media and Data Integrity Errors	injected read and write faults (and ZNS write faults)
Data units, host commands	the pollers' host I/O counters, not the FTL
Endurance Group Media Units Written	(NAND + GC + translation + pad write pages) x page size, in units of 10^9 bytes rounded up; needs a `femu-subsys`

Counters kept but not exported

DFTL hits and misses (`ssd->cmt`), read cache hits and misses (`ssd->rcache`), FAST merge counts and the per-block `erase_cnt` stay inside the FTL. `debug_ftl=on` prints hybrid and FAST merge counts to stdout.

3.4.15 Parameters

Every property is listed with its type, default and range in the property reference (§A.1); this table says what each does inside the FTL and what it interacts with.

Geometry and capacity

NAND geometry (§A.1.1), mode and capacity (§A.1.1)

Property	Effect in the FTL	Interacts with
`secsz`, `secs_per_pg`	page size; LPN = byte offset / page size	the LBA format; DFTL entries per translation page
`pgs_per_blk`	pages per block and per line column; the log-block merge unit	at most 512 with `nand_cell_type`
`blks_per_pl`	the number of lines	the watermarks and the reserve are fractions of it
`pls_per_lun`, `luns_per_ch`, `nchs`	line width and write striping	parallelism; `tplebsy` for multi-plane erase; `mp_program`, `mp_read` for multi-plane program and read; `plane_first` for the write order
`devsz_mb`, `namespaces`, `namespace_sizes`	the exposed capacity	must fit the usable lines per namespace
`op_pcent`	exposes a fixed fraction of raw NAND	overrides `devsz_mb`; refused with `cxl_ssd`

Garbage collection, mapping and caches

Garbage collection, mapping and caches (§A.1.1)

Property	Effect in the FTL	Interacts with
`gc_thres_pcent`	background GC watermark	must not exceed `gc_thres_pcent_high`
`gc_thres_pcent_high`	forced GC watermark; sizes the reserve	lower values cost exposed capacity
`gc_policy`	line victim selection	under FDP, the reclaim unit strategy; needs `gc_strategy=0`
`gc_seed`	seed for the `random` and `d-choice` policies and FDP random reclaim	no effect on the other policies
`gc_pace`	paced collection: copies the victim in step with host writes (paced collection (§3.4.6))	replaces the background pass; refused where `blk_pe_limit` is, and under FDP
`gc_strategy`	reclaim unit victim selection	FDP only; a `gc_policy` other than `greedy` replaces it
`mapping`	L2P scheme	`hybrid` and `fast` reserve a line and refuse Streams and FDP
`mapping_cache_mb`	DFTL cache size	used only with `mapping=dftl`
`read_cache_mb`, `cache_evict`	read cache size and eviction	hit time is `pg_rd_lat` / 16 at realize; 0xEF does not change it
`hot_cold_sep`	hot write pointer for overwrites	page or DFTL only; reserves a line; refused with FDP
`buffer_size`, `buffer_thres_pcent`	write buffer capacity in pages, and the write-back watermark	`vwc`, `power_loss`; under FDP each page goes to its handle's unit
`fdp_trim_erase_all`	FDP deallocate resets every reclaim unit	FDP only
`debug_ftl`	prints invalid page transitions and merge counts	none

Reliability and wear

Reliability and wear (§A.1.1)

Property	Effect in the FTL	Interacts with
`pe_cycles_rated`	Percentage Used denominator	overrides the `nand_cell_type` rating
`nand_bad_blocks`	Available Spare	placement ignores it
wear events	the FTL thread sets a pending SMART warning bit when the spare crosses below 20, a block is overworn while none was, or the media goes read-only; the controller's event bottom half raises it if Asynchronous Event Configuration enables it. The FTL thread also writes the warning to the Persistent Event log at the change	once at each start of a condition; a controller reset drops a pending bit, not the log entry
`spare_lines`	per-plane pool of replacement blocks: a worn-out block swaps its whole state with a spare block of its plane, so addresses do not change; Available Spare follows the emptiest plane	needs `blk_pe_limit`; the namespace must fit without the spare lines
`blk_pe_limit`, `blk_pe_spread`, `blk_pe_seed`	per-block erase limit; after a line erase that takes a block to it, the line retires while usable lines stay at or above the namespace's lines + forced collection lines + 2; otherwise the media goes read-only (end of life (§3.4.10))	refused with the settings the parameter manual lists
`eol_read_only`	on (default): read-only end of life, reserve lines and page-credit admission; off: retirement also needs an open line and a free line, and a block that cannot retire stays in service (overworn) and sets SMART critical warning bit 2	has an effect only with `blk_pe_limit`; on needs `mdts` 1 or more
`ecc_step_ns`, `ecc_retention_sec`	read time grows with erase count and line age	under FDP, a line's age counts from the retirement of its reclaim unit
`pd_edge_wls`, `pd_edge_tiers`, `pd_open_tiers`, `pd_open_age`	program disturb: extra ECC tiers for bottom wordlines and part-programmed blocks; age from the first program	chosen tier values; `nand_block.wp` counts programs since the erase; refused with ZNS, OCSSD, NoSSD, `cxl_ssd` and shared namespaces (NAND timing (§3.5.10))
`pd_open_read_limit`, `pd_open_close_sec`	program disturb actions: a lower read limit for part-programmed blocks, which takes the line out of its write frontier and rewrites it; pad and close a frontier left open too long	bbssd and CSD, also FDP units; checked at each request, served on the next Write; pads count in WAF and media bytes; refused with KV, `mapping=fast` or `hybrid` and the phase 1 refusals (NAND timing (§3.5.10))
`err_read_unc_ppm`, `err_write_fail_ppm`	fixed-period command failures	counted in SMART media errors
`read_reclaim_limit`	read count that queues a line for rewrite	needs host writes to act; under FDP it refreshes the whole reclaim unit (FDP (§4.6))
`wl_spread`	static wear levelling (`do_wear_level()`): at a fresh data line, swap it for the most worn free line and move the least worn full line into it	no write pointer of its own; credit of one per host page programmed, four lines per move; the earlier attempts are described in the function comment

Property	Effect in the FTL	Interacts with
age_scale	multiplies data age (ssd_data_age_ns()) for retention refresh and the ECC age tier	not for cost-benefit collection, whose order one factor on every age does not change
retention_limit_sec	line age that queues a line for rewrite	same

NAND timing the FTL charges

NAND timing (§A.1.1)

The FTL charges trim_lat_ns per deallocate range (also under FDP) and issues a multi-plane erase per LUN during GC, whose inter-plane time is tplebsy. All the other timing properties are applied by the NAND media layer.

Controller properties the FTL reads

Property	Where documented	Effect in the FTL
vwc	controller identity (§A.1.1)	lets the host turn the write buffer off
oncs	same	Write Zeroes and Copy reach the FTL only when enabled (the default)
power_loss	power loss (§A.1.1)	undo records and simulate-power-loss
streams, streams.max	same	per-stream write pointers; reserve streams.max + 1 lines

Interactions and refusals

- **FDP** refuses a gc_policy other than greedy together with a gc_strategy other than 0. It refuses hot_cold_sep and the hybrid and fast mappings with a message that says why: they write through pointers of their own, past the unit the placement handle names.
- **Unknown names** for mapping, gc_policy and cache_evict are refused at realize.
- **Watermarks** outside [1, 100], or a high watermark below the low one, are refused.
- **The write buffer** is off when power_loss=on and vwc=0, and while the host has the advertised cache turned off.
- **DFTL cache size** has no effect under page, hybrid or fast.

Examples, each checked by check-doc-examples.py:

```
-device femu,devsz_mb=1024,femu_mode=1,mapping=dftl,mapping_cache_mb=8
-device femu,devsz_mb=1024,femu_mode=1,hot_cold_sep=on,gc_policy=cost-benefit
-device femu,devsz_mb=1024,femu_mode=1,buffer_size=4096,buffer_thres_pcent=75,vwc=1
-device femu,devsz_mb=1024,femu_mode=1,mapping=hybrid
```

3.4.16 Validation status

What the automated tests check:

Area	Test	Checks
BAST merges	unit test `test-femu-hybrid-oracle`; qtests `hybrid-oracle-*`, `hybrid-batch-occupancy`, `hybrid-destage-occupancy`, `hybrid-switch-trim-erase`, `hybrid-trim-occupancy`	the unit test checks the reference model; the qtests compare FEMU's switch, full merge and erase counts against it, including deallocate and the write buffer
Victim queue	unit test `test-femu-pqueue`	priority queue operations, including random pop driven by the caller's number; pop, remove and random pop leave the detached element's index at 0; pop equals removing the top and random pop equals removing the drawn slot, slot for slot
Reproducible GC	qtests `gc-seed-d-choice`, `gc-seed-random`, `gc-seed-fifo`	two devices with the same configuration and 2048 random page writes report the same relocation count every 32 writes and the same WAF; a third with another `gc_seed` differs (FIFO: matches)
NAND timing	unit test `test-femu-nand-media`	the media layer the FTL calls
C0h counters	qtest `media-counters`	host and NAND page counts and the WAF move with writes
Write buffer	qtest `buffer-counters`, `flush-without-vwc` and `power-loss-*`	hit counts; Flush (also with `vwc=0`), FUA, write-back, cache disable, shutdown and power-cut rollback
Streams	qtest `streams-gc` and the other `streams-*` cases	stream placement and GC of stream lines
GC with no free line	qtests `gc-no-destination`, `gc-no-destination-hot-cold`, `streams-gc-floor`, `gc-no-destination-fdp`	on a geometry whose forced watermark rounds to zero, random single-page and 64-page writes never fail, no mapping names an erased page and no valid page is orphaned (read through the qtest-only `x-ftl-check` property)
Charged work for a fixed workload	qtests `ftl-trace-bbssd`, `ftl-trace-hot-cold`, `ftl-trace-fdp`	a seeded queue-depth-one write and read workload with collection gives exact command, host, NAND, relocated-page and erase counts and exact read, program and erase commands charged to the media layer, and a digest of the order in which lines or reclaim units were collected (qtest-only `x-ftl-trace`; the digest is the last field, a running FNV-style hash of each collected victim's id, updated once per collection, read reclaim included); a refactor of the allocator, collection or media charge must not move them. A different victim policy or a dropped or duplicated charge fails it. The modelled-latency sum depends on host load and is only held to 25%, so NAND timing values are left to `test-femu-nand-media`

Area	Test	Checks
Victim order of each policy	qtests `ftl-trace-random`, `ftl-trace-d-choice`, `ftl-trace-fifo`, `ftl-trace-fdp-random`, `ftl-trace-fdp-noisy`, `ftl-trace-fdp-noisy-ii`, `ftl-trace-fdp-reread`	the same workload under `gc_policy=random`, `d-choice` and `fifo`, and under FDP `gc_strategy=2` and `4`, gives exact counts, with a fixed `gc_seed` where the policy samples. The NOISY cases write two placement identifiers, so the per-handle heaps compare their tops; in `-ii` one handle is Initially Isolated and has no heap of its own. `ftl-trace-fdp-reread` reads a page after each overwrite, which adds background passes whose refusals move the greedy tie order. A changed draw, tie break or heap key fails the matching case: the victim digest catches a change that leaves the counts equal, such as FDP random with `gc_seed=7` and each draw one higher; cost-benefit has no exact pin, because it reads the host real-time clock
Read reclaim	qtest `ftl-trace-read-reclaim`	with `read_reclaim_limit=2` and a read after each overwrite, 83 lines are rewritten, some of them taken from the victim heap, and the trace counts are exact
Refused FDP strategies	qtest `fdp-gc-strategy-refused`	`gc_strategy` values other than 0, 1, 2 and 4 are refused at realize
Multi-plane program and read	qtests `ftl-trace-mp-off`, `ftl-trace-mp-on`, `ftl-trace-mp-one-plane`, `ftl-trace-mp-fdp-off`, `ftl-trace-mp-fdp-read`, `mp-program-scopes-off`, `mp-program-scopes-on`, `multiplane-warnings`, `config-refused`	with two planes per LUN and reads of 16 pages, `mp_program=1,mp_read=1` keeps every command, page and erase count of the same workload with them off, issues fewer read and program commands, and models less time before collection starts, where that time is exact; with one plane the trace equals `ftl-trace-bbssd`; under FDP the trace with `mp_read=1` equals the one without it; a full row written by Write Zeroes or by a buffer write-back takes 4 program commands instead of 8; a setting that changes nothing warns and a negative busy time is refused
Format, Sanitize	qtests `format-ftl`, `sanitize`	after Format, GC relocates nothing; Sanitize status and zeroed data (the FTL state is not checked)
Robustness	qtests `io-fuzz`, `io-fuzz-fdp`, `config-refused`	malformed I/O, refused configurations
Start-up	`doc-examples`	every tagged example on this page and the BlackBox guide starts and moves one block

Apart from the exact counts above, no automated test checks the behaviour of the `cost-benefit` policy, `dftl` and `fast` mapping, hot/cold separation, the read cache, retention refresh, read and write fault insertion on BlackBox, or the wear and spare figures. They are covered only by the start-up examples and by guest runs during development; `femu-test.sh` (guest-side tests (§8.2.6)) checks that the counters move. FEMU's latencies and write amplification have not been calibrated against a specific commercial drive.

3.4.17 Limits

- Data placement is modelled per page; the FTL never tracks sectors within a page, so writes smaller than a page cost a whole page program.
- GC relocations go through the data pointer (or a stream pointer for stream data). Only line and FDP collection with `gc_copyback` move a page on its die; paced collection does not. GC runs on the FTL thread, one line at a time.
- There is no explicit wear levelling, and bad blocks do not affect placement.

- Read reclaim and retention refresh act only on reads followed by writes.
- `cost-benefit` GC uses the host clock for age, so it is not reproducible run to run.
- An injected write fault still programs and maps the data.
- FAST merge counts and DFTL and read cache hit rates are not exported.
- Each namespace's FTL is built from the whole geometry, so host memory for the FTL grows with the namespace count.

3.4.18 Extending the FTL

Adding a GC victim policy

1. Write `static struct line *select_victim_line_<name>(struct ssd *ssd, bool force)` in `hw/femu/bbssd/ftl-line-gc.c`. Candidates are the lines in `ssd->lm.victim_line_pq`; slots `1` to `size - 1` of `pq->d[]` hold them.
2. Apply the background filter: when `!force` and the line has fewer than `pgs_per_line / 8` invalid pages, return NULL and leave the queue as it was.
3. Remove the chosen line with `pqueue_remove()` (or `pqueue_pop()` for the top), which sets `line->pos` to 0. The victim count is the queue size, so there is no counter to keep. `reclaim_line()` expects a line that is in no list.
4. Add `{ .name = "<name>", .select_victim_line = ... }` to `femu_ftl_policies[]`. `femu_ftl_policy_known()` then accepts the name.
5. Describe it in the `gc_policy` description in `hw/femu/femu-props.c`, regenerate the property reference, and add a qtest that tells the policy apart from greedy.

Adding a mapping scheme

1. Implement `struct femu_mapping_ops` in a new `ftl-map-<name>.c`: at least `translate`, `prepare_write`, `commit_write` and `gc_relocate_commit`. Keep `maptbl` and `rmap` correct: `commit_write` must invalidate the old page and set both tables, because GC and reads depend on them.
2. Keep private state in `ssd->map_priv`, allocated in `init` and freed in `exit`.
3. Set `uses_log_class` if writes go through the LOG pointer; the reserve then grows by one line and Streams are refused. Set `uses_cmt` to get the DFTL cost model.
4. For merges, provide `needs_reclaim` and `reclaim(ssd, budget)`. `reclaim()` charges its own NAND operations and returns the latency to add to the triggering request. `reclaim_per_page` runs it after every page instead of once per request. Allocate a destination before invalidating the source, so a full device never loses the only copy.
5. Provide `trim` if the scheme keeps per-page state.
6. Add the ops to `femu_mapping_extra[]` in `hw/femu/bbssd/ftl-map.c`, declare them in `ftl-internal.h`, and add the file to the FEMU source list in `hw/femu/meson.build`.

Run the unit tests and the qtests (testing guide (§8.2)), and check that the C0h counters move for the new code path: a counter that never changes in a mode means the mode is not reached.

3.4.19 Source map

File	Contents
`hw/femu/bbssd/bb.c`	mode registration, `bb_check_capacity()`, FDP refusals, the 0xEF switch, teardown
`hw/femu/bbssd/ftl.c`	`ssd_init()`, `bb_ftl_process_req()`, Copy, counters, Percentage Used, Available Spare, `ssd_free()`

File	Contents
hw/femu/bbssd/ftl.h	struct ssd, struct ppa, lines, write pointers, the mapping and policy ops
hw/femu/bbssd/ftl-internal.h	address helpers, ssd_lpn_range(), GC watermark tests
hw/femu/bbssd/ftl-geom.c	bb_check_geometry(), ssd_init_params(), NAND array allocation
hw/femu/bbssd/ftl-datapath.c	read, write, write buffer, deallocate, Write Zeroes, power-loss rollback
hw/femu/bbssd/ftl-line-gc.c	lines, write pointers, Streams pointers, GC, victim policies, read reclaim
hw/femu/bbssd/ftl-map.c	maptbl, rmap, page and DFTL schemes, scheme registry
hw/femu/bbssd/ftl-map-cmt.c	DFTL cached mapping table
hw/femu/bbssd/ftl-map-tp.c	DFTL translation pages on flash: GTD, TP pointer, publication, TP moves, checks
hw/femu/bbssd/ftl-map-hybrid.c	BAST log-block scheme and its C0h counters
hw/femu/bbssd/ftl-map-fast.c	FAST log-block scheme
hw/femu/bbssd/ftl-cache.c	read cache and its eviction policies
hw/femu/bbssd/ftl-media.c	bridge to the NAND media layer, block read counts
hw/femu/bbssd/ftl-fdp.c	FDP reclaim units, handles and their GC
hw/femu/bbssd/ftl-exp.c	debug tracing of marked pages, off unless FEMU_EXP_LOG or FEMU_DUMP_LPN is set
hw/femu/femu.c	the FTL thread, op_pcent sizing, simulate-power-loss
hw/femu/nvme-admin.c	C0h, SMART and Endurance Group counters, Sanitize, the cache feature

3.4.20 Related pages

- BlackBox guide (§4.4): launch, guest use, refusals
- Timing model (§6.2)
- Architecture (§3.1.5)
- Measuring (§6.1.2)
- Log pages and counters (§B)
- Property reference (§A.1)

3.5 NAND media and timing model

This chapter describes the component that decides how long a flash operation takes in FEMU. It covers the geometry the model works on, the time it charges for a page read, a page program and a block erase, how operations on the same die, plane or channel wait for each other, and how those times become the completion time of an NVMe command. It also covers the two models that sit after the media: the host link and the controller firmware CPU.

The timing model concept page (§6.2) explains the same model from the user's side. This page is for readers who want the algorithms, the data structures and the source.

3.5.1 Contents

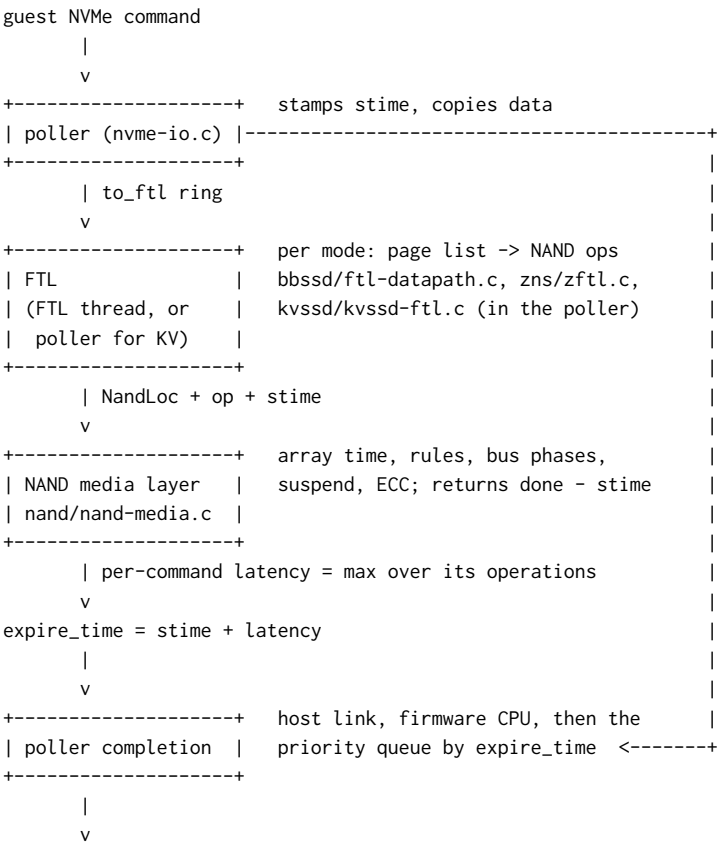
- Purpose and place in FEMU (§3.5.2)
- Geometry (§3.5.3)
- Data structures (§3.5.4)
- Operation timing (§3.5.5)
- Cell types and page types (§3.5.6)

- Channel bus (§3.5.7)
- Plane parallelism and multi-plane operations (§3.5.8)
- Program and erase suspend (§3.5.9)
- ECC read time (§3.5.10)
- From operations to a completion time (§3.5.11)
- How garbage collection is charged (§3.5.12)
- Host link and firmware CPU (§3.5.13)
- OCSSD timing model (§3.5.14)
- Runtime switches (§3.5.15)
- Parameters (§3.5.16)
- Statistics (§3.5.17)
- Calibrating against a real device (§3.5.18)
- Validation status (§3.5.19)
- What is not modelled (§3.5.20)
- Adding a timing feature (§3.5.21)
- Source map (§3.5.22)

3.5.2 Purpose and place in FEMU

FEMU copies a command's data as soon as it parses the command. Timing is computed separately: a model works out when the command would have finished on the emulated device, and the poller holds the completion until the host clock reaches that time. The NAND media model is the part of that computation that charges flash operations.

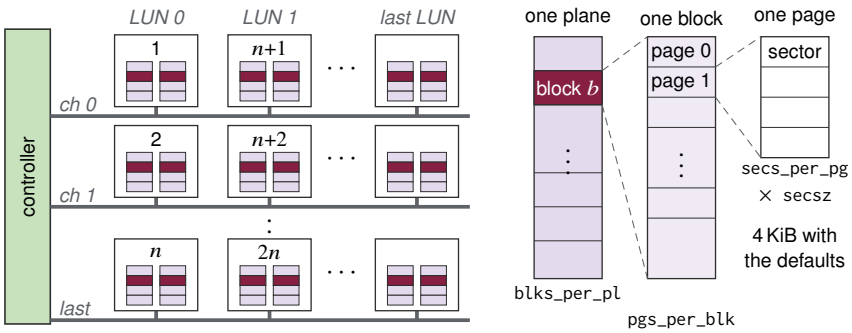
The **media layer**, `nand_media_op()` in `hw/femu/nand/nand-media.c`, is the one media engine. BBSSD, CSD, KV, ZNS, the FTL behind a `femu-cxl-ssd`, and Open-Channel 1.2 and 2.0 use it. It never includes a controller header. Each mode decodes its own address into a `NandLoc`, gives the layer a configuration, and lends it pointers to its busy-until fields. `hw/femu/ocssd/oc-timing.c` keeps the Open-Channel table times, the geometry check and 0xEE (OCSSD timing model (§3.5.14)).



Mode	Engine	Built in	Called from
BBSSD, CSD	media layer	`bb_nand_media_init()` in `bbssd/ftl-media.c`	FTL thread
KV	media layer, through the BBSSD wrapper	`ssd_init()` from `kvssd/kvssd-ftl.c`	poller
ZNS	media layer	`zns_nand_media_init()` in `zns/zftl.c`	FTL thread
`femu-cxl-ssd`	media layer, through the BBSSD wrapper	`femu_cxl_start()` in `cxlssd/cxlssd.c`	`femu-cxl-ftl` worker
OCSSD 1.2, 2.0	media layer	`oc12_media_init()`, `oc20_media_init()` in `ocssd/`	poller
NoSSD	none		

3.5.3 Geometry

Every mode that models NAND describes it with the same hierarchy. Only the names of the properties and the field widths differ.



nchs channels, luns_per_ch LUNs per channel and pls_per_lun planes per LUN (two drawn). A **line** is block b on every plane of every LUN, so there are blks_per_pl lines. The numbers are the order in which the write pointer hands out pages (n = nchs): channel first, then LUN, then plane, then page, so the pages of a large write land on different LUNs.

Figure 10: The NAND hierarchy (channel, LUN, plane, block, page, sector), a line as one block index across every plane of every LUN, and the order in which the write pointer hands out pages.

```
device
+-- channel 0 .. nchs-1             one shared bus per channel
    +-- LUN 0 .. luns_per_ch-1      a die: runs one array operation at a time
        |                           (one per plane under the ZNS plane rule)
        +-- plane 0 .. pls_per_lun-1
            +-- block 0 .. blks_per_pl-1   erase unit
                +-- page 0 .. pgs_per_blk-1  program and read unit
                    +-- sector 0 .. secs_per_pg-1  (secsz bytes each)

BBSSD line (superblock) k = block k on every (channel, LUN, plane)
lines = blks_per_pl     pages per line = nchs * luns_per_ch * pls_per_lun * pgs_per_blk
```

Address formats

Mode	Address	Fields, low bit first	Where
BBSSD, CSD, KV	`struct ppa`	blk 16, pg 16, sec 8, pl 4, lun 7, ch 12, reserved 1	`bbssd/ftl.h`
ZNS	`struct ppa`	spg 2, pg 16, blk 16, fc (LUN) 8, pl 3, ch 7, valid 1, reserved 8	`zns/zns.h`
OCSSD 1.2	64-bit PPA with masks and offsets from the geometry	ch, lun, pln, blk, pg, sec	`PPA_*` macros in `nand/nand.h`
OCSSD 2.0	64-bit LBA with group, parallel unit, chunk, sector fields	group, PU, chunk, sector	`OC20_LBA_GET_*` in `ocssd/`

A field width is an upper bound on its axis. `bb_check_geometry()` in `bbssd/ftl-geom.c` refuses a count larger than its field can hold, and refuses a geometry whose total sector count does not fit in a signed 32-bit integer. `zns_check_params()` bounds each ZNS axis by its field. `oc_timing_geometry_ok()` requires every OCSSD axis to be non-zero, `lnum_ch` at most 32 and `lnum_ch * lnum_lun` at most 128, the sizes of the per-channel and per-LUN arrays.

How pages land on the geometry

The timing a workload sees depends on where the FTL puts its pages, because the media model only charges what it is asked to.

- **BBSSD, CSD, KV** allocate from a write pointer that moves channel first, then LUN, then plane, then page (plane first with `plane_first` on bbssd and CSD) (`ssd_advance_write_pointer_common()` in `bbssd/ftl-line-gc.c`). Consecutive pages of a large write therefore land on different channels, then different LUNs, and are programmed in parallel.
- **ZNS** buffers writes in a per-zone write cache. A flush programs one program unit per plane: `zns_flash_type` pages of 16 KiB (`ZNS_PAGE_SIZE`) per plane, charged as one program operation per plane (`zns_wc_flush()` in `zns/zftl.c`). A 4 KiB logical page is a sub-page (`spg`) of a 16 KiB page.
- **OCSSD** takes physical addresses from the host, so the host chooses the parallelism.

3.5.4 Data structures

The media layer's types are in `hw/femu/nand/nand-media.h`.

Type	Fields that matter	Role
`NandLoc`	`ch`, `lun`, `pl`, `blk`, `pg`, `flash_type`, `page_type`, `pe_cycles`, `age_sec`, `xfer_secs`	One operation's position and the facts timing depends on. Filled by the mode's decoder (`bb_decode_loc()`, `zns_advance_status()`, `oc12_media_op()`).
`NandMediaTiming`	`rd_ns`, `wr_ns`, `er_ns` (flat); `rd_table_ns`, `wr_table_ns`, `er_table_ns` (by cell and page type); `pgtype_mult`; `cmd_addr_ns`, `page_xfer_ns`, `status_ns`; `tplpbsy_ns`, `tplrbsy_ns`, `tplebsy_ns`; `ecc_*`; `tsusp_ns`	Every duration.
`NandMediaPolicy`	`array_contend`, `channel_mode`, `pe_suspend`, `ecc_on_read`, `use_flat_timing`	Which mechanisms are on.
`NandTimelineOps`	`ch_avail`, `lun_avail`, `plane_avail`, `lock_lun`, `unlock_lun`	Accessors that return pointers into the mode's own busy-until fields.
`NandMedia`	`cfg`, `bus_res` (per-channel booked windows), `susp` (per-position suspend state)	One instance per namespace FTL.
`NandOpCompletion`	`done_ns`, `latency_ns`	Absolute end time, and `done_ns - stime`.

The busy-until times ("availability timestamps") live in the mode's structures, not in the media layer:

Resource	BBSSD, CSD, KV	ZNS	OCSSD
Channel bus	`ssd_channel.next_ch_avail_time`	`zns_ch.next_ch_avail_time`	`Oc12Ctrl.ch_avail[]` (1.2 only)
LUN (die)	`nand_lun.next_lun_avail_time`	`zns_fc.next_fc_avail_time` (only with `zns_lun_contention`)	`Oc12Ctrl.lun_avail[]`, `Oc20Ctrl.lun_avail[]`
Plane	not kept	`zns_plane.next_plane_avail_time`	not kept

Each mode configures the layer differently:

Setting	BBSSD, CSD, KV, CXL	ZNS
`array_contend`	`NAND_CONTEND_LUN_ONLY`	`NAND_CONTEND_PLANE_ONLY`; `NAND_CONTEND_LUN_AND_PLANE` with `zns_lun_contention`
`channel_mode`	`NAND_CH_STAGED` when any bus phase is non-zero, else `NAND_CH_OFF`	same rule

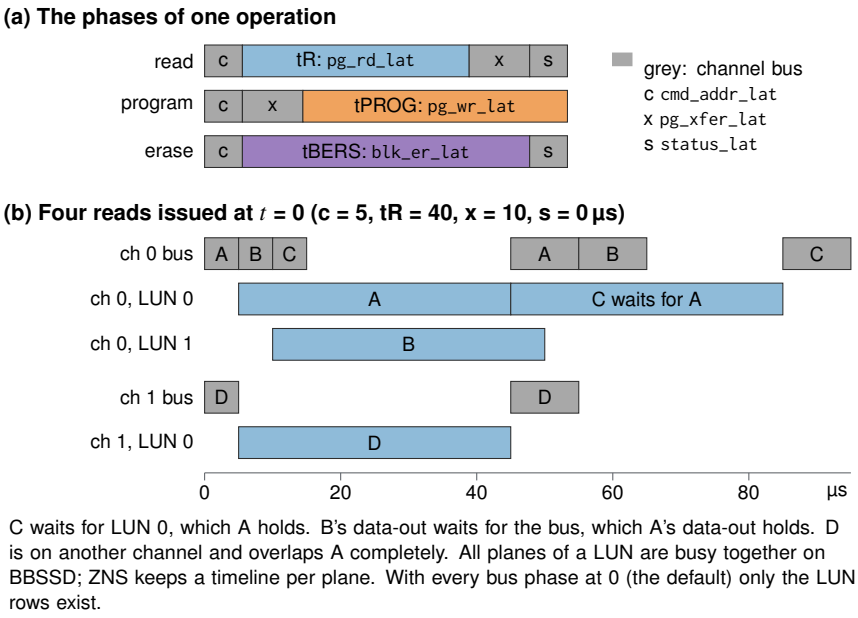


Figure 11: How the media layer times an operation: the phases of a read, program and erase, and four reads issued together showing LUN serialization, a shared channel bus and parallel channels.

Setting	BBSSD, CSD, KV, CXL	ZNS
`use_flat_timing`	true when `nand_cell_type=0`, false when it is set	false (per-type table; page type 0, or `pg % zns_flash_type` for reads with `zns_page_types`)
`pe_suspend`	`pe_suspend` property	`zns_pe_suspend` property
`ecc_on_read`	true; the adder is still 0 unless `ecc_step_ns` is set	true; the adder is still 0 unless `zns_ecc_step_ns` is set

The enum also has `NAND_CH_NOOP`. No mode selects it today.

3.5.5 Operation timing

Array time

`array_lat()` picks the time the die spends on the operation:

```
erase   : flat ? er_ns : er_table_ns[flash_type]
read    : (flat ? rd_ns : rd_table_ns[flash_type][page_type]) + ecc_extra
program : flat ? wr_ns : wr_table_ns[flash_type][page_type]
          if flat and pgtype_lat:  * pgtype_mult[page_type] / 1000
```

Phase sequence of one operation

With the channel bus off (the default), an operation is one array phase:

```
start = max(stime, rule busy-until)
done  = start + array time
rule busy-until = done
```

With the bus on (`NAND_CH_STAGED`), each operation also has bus phases. "Now" phases queue on the bus in arrival order; a read's data-out happens after the array finishes, so it is booked as a future window (Channel bus (§3.5.7)).

```
READ     |cmd/addr|..wait for die..|=== tR array ===|data-out|status|
          bus now                  die              bus later  bus later

PROGRAM  |cmd/addr|data-in|..wait for die..|======= tPROG array =======|
          bus now  bus now                  die  (done = array end)

ERASE    |cmd/addr|..wait for die..|=========== tBERS array ===========|status|
          bus now                  die                                 bus later
```

A program completes when its array phase ends. A read completes after its status phase, an erase after its status phase. A zero-length phase touches nothing, so a bus with only `pg_xfer_lat` set has no command or status cost.

Default times

Mode	Read	Program	Erase	Source
BBSSD, CSD, KV flat (`nand_cell_type=0`)	`pg_rd_lat` 40 us	`pg_wr_lat` 200 us	`blk_er_lat` 2 ms	property defaults in `femu.c`
`femu-cxl-ssd`	`read-ns` 40 us	`program-ns` 200 us	`erase-ns` 2 ms	`cxlssd/qemu-adapter.c`
ZNS SLC	4 us	75 us	2 ms	`zns/zns.h`, erase from `nand/nand.h`
ZNS TLC	32 us	937.5 us	3 ms	same
ZNS QLC (default `zns_flash_type`)	85 us	12.196 ms	3 ms	same
ZNS MLC, PLC	none built in	none built in	none built in	realize refuses them unless all three `zns_*_lat` are set

The ZNS program time covers a whole program unit (`zns_flash_type` pages of 16 KiB on one plane). The header cites the ISSCC papers the figures come from.

3.5.6 Cell types and page types

BBSSD, CSD, KV: `nand_cell_type`

`nand_cell_type`	Cell	Read by page type (us)	Program by page type (us)	Erase
0	flat	`pg_rd_lat`	`pg_wr_lat`, optionally scaled by `pgtype_lat`	`blk_er_lat`
1	SLC	40	800	2 ms
2	MLC	lower 48, upper 64	lower 850, upper 2300	3 ms
3	TLC	56.5, 77.5, 106	820.5, 2225, 5734	3 ms
4	QLC	59.325, 85.25, 127.2, 169.6	861.525, 2447.5, 6880.8, 9174.4	3 ms

Values come from `nand/nand.h`: MLC from a profile of Micron L95B parts, TLC from SimpleSSD, QLC scaled from the TLC values. Any other value is reset to 0 with a message (`ssd_init()` in `bbssd/ftl.c`); PLC has no table here. With a cell type set, `pg_rd_lat`, `pg_wr_lat` and `blk_er_lat` no longer set NAND times (`pg_rd_lat` still sets the write buffer and read cache hit cost), and `pgs_per_blk` must be at most 512, the size of the page-type tables.

The page type of page `pg` in a block comes from the pairing tables `mlc_tbl`, `tlc_tbl` and `qlc_tbl`, built by `init_nand_flash()` in `nand/nand.c`; `slc_tbl` is all zeros. They follow a shadow programming order: the first

pages of a block are lower pages, then consecutive pairs of pages cycle through the page types from lower to upper.

BBSSD, CSD, KV: `pgtype_lat` and `cell_pages`

With `nand_cell_type=0` and `pgtype_lat` non-zero, the flat program time is multiplied by a factor that depends on the page's position in its wordline. The page type is `pg % cell_pages`; `cell_pages` 0 is set to 3 when `pgtype_lat` is on (`ssd_init()`). The factors, in thousandths, are in `bb_nand_media_init()`:

`cell_pages`	Page type 0	1	2	3	4
1	1000				
2	600	1400			
3	600	1000	1400		
4	500	850	1150	1500	
5	400	750	1000	1250	1600

Each row averages to 1000, so the mean program time stays `pg_wr_lat` when `pgs_per_blk` is a multiple of `cell_pages`. Reads and erases are not scaled.

ZNS and OCSSD

ZNS keeps one read, program and erase time per flash type. It uses page type 0, except for reads with `zns_page_types` (see ZNS (§4.7.3)). Each `zns_*_lat` property replaces the built-in value for the configured `zns_flash_type` only. OCSSD uses the `nand_cell_type` tables above, selected by `flash_type`; Open-Channel 1.2 looks up the page type of each page. Open-Channel 2.0 uses page type 0. With `oc20_page_types=on` a chip operation covers one NAND page and uses its page type.

3.5.7 Channel bus

The bus is modelled only when a phase has a duration: `cmd_addr_lat`, `pg_xfer_lat` (or `ch_xfer_lat` when `pg_xfer_lat` is 0) or `status_lat` for BBSSD, CSD and KV; `zns_cmd_addr_lat`, `zns_pg_xfer_lat` or `zns_status_lat` for ZNS. Otherwise transfers take no time and channels never contend.

Two kinds of phase share a channel:

- `bus_now()`: command/address, a program's data-in. It starts at `max(earliest, ch_avail)`, moves past any booked window it would overlap, and advances `ch_avail` to its end.
- `bus_later()`: a read's data-out and status, which happen after the array finishes. It starts no earlier than `ch_avail` and is booked as a window `[start, end)` in `bus_res[ch]` and does not advance `ch_avail`, so another die's "now" phase can use the bus before the window opens.

Windows that ended before the current operation's `stime` are pruned. A channel holds at most `NAND_BUS_RES_MAX` (32) windows; when it is full, the data-out is charged FIFO from `ch_avail` instead. With `policy.bus_res_unbounded` (OCSSD 1.2) the list grows and nothing falls back.

A data phase moves one page in `page_xfer_ns`. When the mode sets `secs_per_page` and the operation's `NandLoc.xfer_secs`, it moves only those sectors and takes `DIV_ROUND_UP(page_xfer_ns * xfer_secs, secs_per_page)`, as OCSSD 1.2 charges a partial page.

```
channel 0 bus, cmd/addr 1 us, data 10 us (times in us)

0   1        11 12 13                 52        62
|P c|P data-in|R1c|R2c|.... idle .....|R1 out   |
                                      ^ booked window, R1's array ended at 52
a phase that arrives at 20 fits in the idle gap; one that would overlap
[52,62) moves to 62
```

ONFI 6.0 profile (BBSSD, CSD, KV)

`onfi_mode` (0 to 26) sets every channel bus phase from an ONFI 6.0 NV-LPDDR4 timing mode. ONFI 6.0 gives no array times, so `pg_rd_lat`, `pg_wr_lat`, `blk_er_lat` and `nand_cell_type` still set tR, tPROG and tBERS. The default, -1, turns the profile off and changes nothing.

```
onfi_mode, onfi_mts --> tDSC --> page_xfer = bytes * tDSC / 2 (x8, DDR)
Table 4-34 constants --> per-op command, status and burst phases
pg_rd_lat, pg_wr_lat, blk_er_lat ------------------> array times (unchanged)

read     |00h addr 30h tWB|..... tR .....|70h status|00h pre| page |post|
program  |80h addr tADL|edges| page |10h tWB|...... tPROG ......|70h status|
erase    |60h addr D0h tWB|.............. tBERS ..............|70h status|
          ^ sent only when the LUN is ready (RDY)       LUN held to here ^
```

The profile derives these phases (`nand/onfi.c`). Each one is a sum in ps, rounded up to whole ns once. The table shows mode 26 with 2 column and 3 row address cycles.

Phase	Sequence	Mode 26
read command	00h, 5 address cycles, 30h, tWB	275 ns
program command	80h, 5 address cycles, tADL	562 ns
data in edges	tWPRE2 + tWPST2 + tWPSTH2	57 ns
program confirm	10h, tWB	125 ns
erase command	60h, 3 address cycles, D0h, tWB	225 ns
status	70h (78h + 3 cycles with several LUNs), tWHR, tRPRE2, one byte, tRPSTH, tRHW	246 ns (321 ns)
data out lead	00h, tRPRE2; with several LUNs 06h, 5 cycles, E0h, tCCS, tRPRE2	50 ns (200 ns)
data out tail	tRPSTH, tRHW	115 ns
page of 4 KiB	4096 bytes at 0.417 ns per 2 bytes	855 ns

- `onfi_mts` replaces the mode's rate for the page only: `ceil(bytes * 1000 / mts)` ns. The status byte keeps the mode's tDSC.
- `onfi_spare_bytes` adds spare bytes to each page on the bus.
- `onfi_tccs_ns` gives tCCS. ONFI 6.0 has no value, so 0 charges nothing.
- A `tplpbsy`, `tplrbsy` or `tplebsy` left at 0 takes 500 ns, the Table 4-30 typical value.
- A read sends its status before its data out (Section 6.14). A program, erase or copyback ends at its status.
- An op sends its command only when its LUN is ready, because the page register is in use until then. The LUN stays busy to the end of the read's data out, or of the program, erase or copyback status.
- A command that waits for a busy LUN books a future window, so an op on an idle LUN can use the bus first.
- Op start times are not monotonic. A copy's write starts at the end of its reads, and garbage collection starts at the wall clock. An op with an older start time can still use an idle bus gap before the latest start time on its channel.

```
latest start time on the channel: high
dropped      | kept history (ends <= high)  | future windows
[=] [==]     | [==]  [=]   [===] ... [==]   | [====]  [==]
             low                           high
an op with start t >= low:  can use any idle gap after t
an op with start t <  low:  starts no earlier than low (the fallback)
```

- At the start of each op, a grown list drops old windows. When more than 1024 windows end before `high` in time, it keeps the newest 512. The `NAND_BUS_HISTORY_MAX` value sets this cap. Then `low` moves to the end of the newest dropped window, and the op can add a few windows of its own.
- A window that ends before `ch_avail` also goes, because no phase can start before it. A bounded 32-entry list has no room for history, so it keeps `low` at `high`.
- An op that starts at or after `low` gets the gaps it would get from a list that never drops a window. Its LUN and `ch_avail` can still delay it.
- A phase that happens now (a command to a ready LUN) still starts after `ch_avail` in arrival order, as without the profile.
- Future windows stay until they end, so the list grows with the media ops in flight: one set of windows per queued op.
- A multi-plane program sends every plane's command and data, with the tPLPBSY gaps, as one bus window (Section 3.1.3). A multi-plane read or erase books each later plane command after its gap.
- With several LUNs, a multi-plane program or erase reads one 78h status per plane. With one LUN, one 70h status reads the composite.

Approximations:

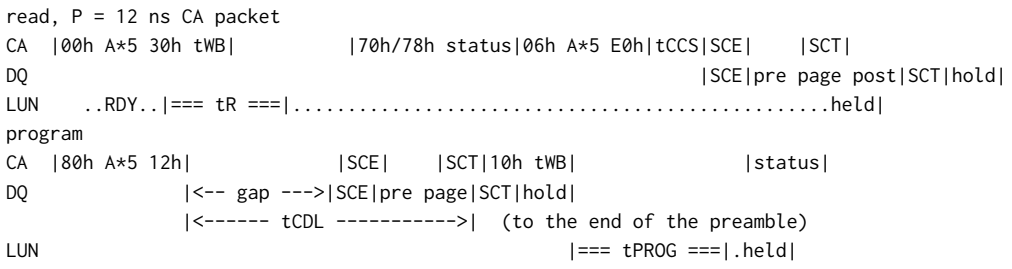
- The status and data out phases charge tRPSTH (15 ns) and not tRPST. tRPST is tDQSRE plus half a tRC, and tDQSRE is 1 to 25 ns by device (Table 4-38).
- Warmup cycles are not modelled. Section 4.12 calls them optional, but Section 4.15 note 12 asks for them above 800 MT/s.
- The profile assumes the program page register clear enhancement is on.
- With the profile, the channel keeps every future window it books, so a full list never books the bus from now.

Realize refuses these settings:

- `onfi_mode` outside -1 to 26, or `onfi_mts` above 100000;
- a negative `onfi_mts`, `onfi_spare_bytes` or `onfi_tccs_ns`;
- an odd page size on the bus (`secsz * secs_per_pg + onfi_spare_bytes`), because a DDR burst moves bytes in pairs;
- `cmd_addr_lat`, `pg_xfer_lat`, `ch_xfer_lat` or `status_lat` with the profile;
- a namespace that is not bbssd, CSD or KV;
- `cxl_ssd`, `ns_mgmt` or shared namespaces;
- `pe_suspend`, because ONFI 6.0 has no suspend and a held page register has no suspend rule.

`onfi_mts`, `onfi_spare_bytes` or `onfi_tccs_ns` without `onfi_mode` warns.

SCA protocol (`onfi_sca`). `onfi_sca=1` selects the Separate Command Address protocol (Section 5). Commands, addresses and status then move to a CA bus. Each channel gets its own CA bus. The channel bus (DQ) carries only data bursts, so a command of one LUN can run during the data of another.



- A packet is a 4-bit header and an 8-bit body on two DDR pins: 3 tCACI, 12 ns (Section 5.7.1, Table 5-9).
- A data burst owns the DQ bus from its SCE packet to the 50 ns hold after its SCT packet. A new SCE cannot come during another burst (Section 5.9.2).
- FEMU looks for the first start where DQ is free for the whole burst and the CA bus is free for both packets. Then it books all three.
- Every other CA phase takes the first idle CA gap after its earliest time.
- Array data out needs Change Read Column Enhanced (06h-E0h) before SCE, also with one LUN (Section 5.9.6). `onfi_tccs_ns` gives tCCSR.
- tCDL (400 ns) runs from the 12h packet to the end of the write preamble. It replaces tADL.
- A status read is a CA output: the header, tW2R_CA, one byte at tCACO 10 ns (two bits per cycle), then tR2W_CA.
- The CA bus keeps its own window history and its own `low` mark, as the channel bus does.

Phase	Sequence	Mode 26
read command (CA)	00h, 5 address packets, 30h, tWB	184 ns
program command (CA)	80h, 5 address packets, 12h	84 ns
tCDL gap	tCDL - (SCE + tSCD + tWPRE2)	313 ns
write burst (DQ)	SCE, tSCD, tWPRE2, page, SCT, tWPSTH_CA	149 ns + page
program confirm (CA)	10h, tWB	112 ns
erase command (CA)	60h, 3 address packets, D0h, tWB	160 ns
status (CA)	70h (78h + 3 packets), tWHR, header, tW2R_CA, 4 tCACO, tR2W_CA	241 ns (277 ns)
column select (CA)	06h, 5 address packets, E0h	84 ns, then tCCSR
read burst (DQ)	SCE, tSCR, tRPRE2, page, then SCT ending tRPST_CA after the page, tRPSTH_CA	168 ns + page

tRPST_CA is tDQSRE plus half a tRC. FEMU takes tDQSRE at its 25 ns maximum.

A multi-plane program sends 11h after each plane but the last, then waits tPLPBSY. The SCA figures call this busy tDBSY (Sections 5.12.5 to 5.12.7), and ONFI 6.0 gives it no value. So `tplpbsy` sets it, and 0 takes the 500 ns FEMU default. A value of 0 cannot model a device with no busy at all. Another LUN can use both buses between the planes (Figure 5-34).

The confirm opcodes are 11h and 10h (program), 32h and 30h (read), and D1h and D0h (erase). Each pair has the same length.

Approximations:

- tCDL ends at the end of the write preamble. With an idle bus the write burst, SCE first, starts 313 ns after the 12h packet.

- The 10h confirm waits for the end of the hold after SCT.
- LUNSel and NTO packets, ODT and CA training are not modelled.
- CA_CE# setup and hold appear only in the status turnaround.

With a parameter page, SCA needs two things:

- Change Read Column Enhanced (bytes 8-9 bit 6);
- at most 16 LUNs per target (byte 100), because SCE and SCT carry 4 LUN bits.

Read cache sequence (`onfi_cache_read`). `onfi_cache_read=1` reads the pages of one read command that land on one LUN as a read cache sequence (Section 6.15). The array reads the next page while the bus moves the current one.

```
00h addr 30h   read page 0 into the data register (tR)
31h            copy page i to the page register, start page i + 1
               (the next page of the same block and plane)
00h addr 31h   the same, for any other page of the LUN
3Fh            copy the last page, start nothing
each copy keeps SR[6] busy for tRCBSY after the array read ends

array |== tR 0 ==|cp|== tR 1 ==|cp|== tR 2 ==|cp|
bus     st 31h --^   |st out 0|31h |st out 1|3Fh  |st out 2|
```

- A sequence sends the command of the next page before the data out of the current page. So the media layer needs every page of the sequence at once. The bbssd read path holds the media reads of one command, groups them by LUN in LPN order, and gives each group to `nand_media_read_seq()`.
- A group of one page is a plain read. Reads of two commands never join one sequence. With 8 channels of 8 LUNs, a command needs more than 64 pages before a LUN gets two of them.
- Each copy ends at `max(trigger, array end) + tRCBSY`. The next array read starts there. `trcbsy` sets tRCBSY; 0 takes 3000 ns (Table 4-30 typical).
- With several LUNs, a 78h status selects the LUN before each 31h or 3Fh. Each page reads its status, then its data out, after its copy.
- The LUN is held from the 30h command to the last data out.
- Garbage collection reads, mapping reads and copyback stay plain reads.
- A KV namespace reads a value one page at a time, so it keeps plain reads. Realize warns, and `query-femu` reports `cache-read` false for it.
- The read counts and the wear checks of the held pages run in LPN order, as without the sequence.

The period per page on one LUN, with no other traffic:

```
plain read   = cmd + tR + status + data out
in sequence  = max(tR, data out + status + 31h + select) + tRCBSY
```

A sequence is faster only when the bus part of a plain read is longer than tRCBSY. tRCBSY as a copy time after the array read is a model choice. ONFI gives it only as a typical busy time.

FEMU reads a group as plain reads when the sequence would end later, as a controller that knows the timing parameters would choose:

```
nand_media_read_seq(group of one LUN)
  |
  +-- one page, or no cache read --> plain read
  +-- cost on an idle LUN and idle buses (closed form, no booking):
  |     PLAIN = sum (cmd + tR(i) + status + data out(i))
  |     SEQ   = the sequence recurrence above, with every phase at its
  |             earliest time
  +-- SEQ <= PLAIN --> one read cache sequence
  +-- SEQ >  PLAIN --> plain reads of the group, in LPN order
```

Mode, page	8 pages, one LUN, plain	as a sequence	FEMU uses
0, 4 KiB	817240 ns	560590 ns	the sequence
26, 4 KiB	332328 ns	345912 ns	plain reads

- The choice uses only the timing fields and the pages of the group. These give tR with the ECC time of each page, the data out with its partial transfer, and the 31h or full read command of each step.
- The choice does not look at the bus, so the same group always gets the same choice.
- With other traffic both forms can end later than their idle cost. The choice does not change then.
- The plain reads stay inside the LUN group. With several LUNs the FTL still books group by group, so the order differs from a device with `onfi_cache_read` off.
- The choice needs the ONFI order and the page register hold (the profile sets both) and LUN contention, as bbssd uses.

Realize refuses `onfi_cache_read` with these settings:

- `mp_read`: FEMU does not model the multi-plane cache read;
- `mapping=dftl`: a mapping read can make a page ready after the command start;
- `read_cache_mb`: a DRAM hit can come before the NAND read of the page ends;
- a negative `trcbsy`.

Parameter page (`onfi_param_page`). `onfi_param_page` names a file with an ONFI parameter page image (Section 6.7.1, Table 6-4). The page turns the profile on, also with `onfi_mode=-1`. Then the mode is the highest one that the page lists.

```
file --> copy 0, 1, 2, ... (256 bytes each)
           signature: 2 or more of "ONFI" (6.7.1.1)
           CRC-16: poly 8005h, init 4F4Eh, bytes 0-253 (6.7.1.42)
       --> first good copy
       --> checks against the device --> mode, spare bytes, address cycles
```

Bytes	Field	Use
4-5	revision	bit 15 (ONFI 6.0) must be set
6-7	features	bit 9 (NV-LPDDR4) must be set and bit 0 (x16) clear; see the rules below
8-9	optional commands	Read Status Enhanced, Change Read Column Enhanced and Copyback, when the device needs them
80-83, 84-85	data and spare bytes per page	data must equal `secsz * secs_per_pg`; spare bytes go on the bus
92-95, 96-99	pages per block, blocks per LUN	must equal `pgs_per_blk` and `blks_per_pl * pls_per_lun`
100	LUNs per target	must divide `luns_per_ch`
101	address cycles	column cycles (bits 4-7) and row cycles (bits 0-3) set the command lengths

Bytes	Field	Use
102	bits per memory entry	must equal `nand_cell_type` when both are set
113, 114	plane address bits, multi-plane attributes	`pls_per_lun` must be `1 << bits`; several planes need the concurrent form
118-121	NV-LPDDR4 modes	bit 0 is modes 0 to 3, bit k is mode k + 3

Realize refuses the page in these cases. The error names the field and both values.

- The file is missing or shorter than 256 bytes, or no copy has a signature and a good CRC.
- The data bytes per page are not a power of two of 512 or more.
- The pages per block are not a multiple of 32.
- A geometry field differs from the device, or an address field is too short for it (Section 3.1).
- `onfi_spare_bytes` is set and differs from bytes 84-85.
- `onfi_mode` is set and the page does not list it, or the page lists no mode.
- The listed modes are not one range, as Section 6.7.1.30 asks.
- Byte 102 (bits per memory entry) is 0.
- Several LUNs per target (byte 100) without multiple LUN operations (bytes 6-7 bit 1) and the program page register clear enhancement (bit 8). The held page register model needs both.
- Several LUNs per channel without Read Status Enhanced (bytes 8-9 bit 3), or several LUNs or planes without Change Read Column Enhanced (bit 6).
- Several planes without multi-plane program and erase (bytes 6-7 bit 3), which garbage collection uses, or with overlapped multi-plane only (byte 114 bit 0). FEMU models the concurrent form.
- `mp_read` without multi-plane read (bytes 6-7 bit 6), or `gc_copyback` without Copyback (bytes 8-9 bit 4).

A page can clear odd to even copyback (bytes 6-7 bit 4). Then `gc_copyback` moves a page on the die only when the source and the destination page have the same parity. Other moves go off the die. The page gives no array times in ONFI 6.0, so `pg_rd_lat`, `pg_wr_lat` and `blk_er_lat` still set them. Pages of older ONFI revisions are refused.

QMP `query-femu`[7] reports the phases a namespace charges in its `media` member, for every mode with a media layer, with the profile on or off. The values come after the fallbacks: a command phase of 0 takes `cmd_addr_ns`, and with the bus off every bus phase is 0.

3.5.8 Plane parallelism and multi-plane operations

The array rule decides which operations exclude each other:

```
LUN rule (BBSSD, CSD, KV)            plane rule (ZNS)
LUN 0: [pl0 op][pl1 op][pl0 op]      LUN 0 pl0: [op][op]
       one at a time per LUN                pl1: [op][op]  in parallel
```

Under the LUN rule, all planes of a LUN are busy together, so `pls_per_lun > 1` adds capacity and line width but no single-operation parallelism, except for the batched operations below.

`nand_media_multiplane()` runs one operation on several planes of a LUN:

```
per plane i:  cmd/addr on bus (+ data-in for a program); tplXbsy between planes
array:        start = max(bus end, rule of every plane); done = start + array time
read:         status, then per plane cmd/addr + data-out (booked windows)
program/erase: status (if set)
```

[7] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md#media-timing`

Operation	Batched across planes	Caller
Erase of a line's block on every plane of a LUN	yes, one erase time plus `tplebsy` per extra plane	BBSSD GC (`bbssd/ftl-line-gc.c`), FDP GC (`bbssd/ftl-fdp.c`), KV reclaim (`kvssd/kvssd-ftl.c`)
Host programs, and buffer write-back	with `mp_program`: pages of the same block and page on distinct planes of a LUN, one program time plus `tplpbsy` per extra plane	`ssd_program_lpn()` in `bbssd/ftl-datapath.c`
Host reads	with `mp_read`: as above, one read time plus `tplrbsy` per extra plane	`ssd_read()` in `bbssd/ftl-datapath.c`
FDP reads and writes, KV writes	no, one operation per page	`bbssd/ftl-fdp.c`, `kvssd/kvssd-ftl.c`
GC page moves	no	`gc_read_page()`, `gc_write_page()`
ZNS zone reset	no batching needed: each plane has its own rule, so per-plane erases overlap	`zns_zone_reset()`
Copyback (`nand_media_copyback()`)	no: one read and one program on one die, no data phase, no ECC adder (it skips the controller's ECC)	BBSSD line GC and FDP GC with `gc_copyback` (`bbssd/ftl-line-gc.c`, `bbssd/ftl-fdp.c`)

`trcbsy` applies only to the read cache sequence of `onfi_cache_read` (see the ONFI 6.0 profile). Without it, a value warns at realize. With the defaults (no bus, no `tplebsy`) a two-plane erase takes exactly one erase time.

Multi-plane program and read (BBSSD, CSD)

`mp_program` and `mp_read` are off by default. They have no effect with one plane per LUN, in other modes, or under FDP, whose reads and writes keep their single-page timing; each setting warns at realize in those cases. By default the write pointer walks channel, LUN, plane, page, so the pages of one block and page on the planes of a LUN arrive `nchs * luns_per_ch` pages apart. With `plane_first` it walks plane, channel, LUN, page, and those pages arrive in a row. `plane_first` follows the same rules and warnings as `mp_program`. The datapath holds each LUN's pages in a batch and charges the batch through `ssd_advance_status_multiplane()` when one of these happens:

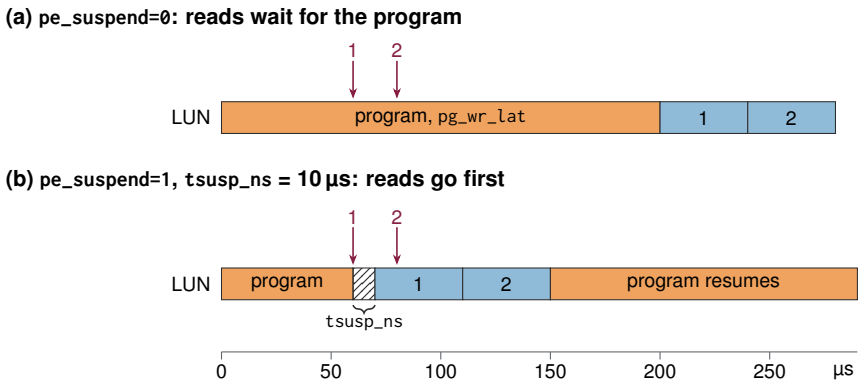
- the batch has one page on every plane;
- the next page for that LUN is in another block or page, or on a plane the batch already has;
- the command ends (write, Write Zeroes, buffer write-back, read);
- garbage collection or a mapping merge is about to run.

A batch of one page is an ordinary single-page operation. The media layer counts a batch as one command, so the read and program command counts drop while host, NAND, relocated-page and erase counts stay the same. Garbage collection moves pages one at a time.

`tplpbsy` and `tplrbsy` are durations: the busy time between the planes of a multi-plane program or read. Each warns at realize when set without its enable, and realize refuses a negative value.

3.5.9 Program and erase suspend

With `pe_suspend` (BBSSD, CSD, KV) or `zns_pe_suspend` (ZNS), a read that finds its LUN (ZNS: plane) running a program or erase goes ahead of it. The layer keeps, per position, `pe_end` (when the newest program or erase there ends) and `rd_end` (when the newest read there ends). `suspend_read_start()` decides:



Read 1 takes 50 µs instead of 180; read 2 takes 70 instead of 200. Only the read that opens a suspension pays `tsusp_ns`; later reads queue behind it. The program ends at 290 µs, late by `tsusp_ns` plus every read that went ahead. An erase is suspended the same way. BBSSD suspends per LUN, ZNS per plane.

Figure 12: Program suspend on one LUN: with `pe_suspend=1` two reads go ahead of a running program, the first pays `tsusp_ns`, and the program finishes later by that overhead plus both reads.

```
read arrives at t, rule busy until B
if t >= B or t >= pe_end:           ordinary rule (nothing to suspend)
elif rd_end > t:                    join the open suspension
    start = rd_end,      shift = tR
else:                               open a suspension
    start = t + tsusp,   shift = tR + tsusp
done = start + tR
pe_end += shift; every rule busy-until > t += shift; rd_end = done
```

Only programs and erases are suspended: a read that finds the die busy with another read waits for it. Reads that queue behind an open suspension pay `tsusp_ns` once. A read that arrives after the suspension's reads have ended opens a new one and pays it again.

Worked example, one LUN, `pg_wr_lat` 200 us, `pg_rd_lat` 40 us, `tsusp_ns` 5 us, bus off. Numbers produced by `nand_media_op()`:

```
time (us)   0        50 55    95     135  150 155   195             330            530
LUN busy    |== P ===|ss|  R1  |  R2  |= P =|ss| R3 |====== P ======|===== W2 =====|

P  program  arrives 0     runs 0-50, 135-150, 195-330 (suspended twice)
R1 read     arrives 50    opens a suspension: starts 50 + 5, done 95
R2 read     arrives 60    joins it behind R1: starts 95, done 135, no tsusp
R3 read     arrives 150   R1 and R2 are over, so it opens a new one: 155-195
W2 program  arrives 160   waits for the LUN until P ends: 330-530
(ss = tsusp_ns)

P's die time: 200 alone; with the reads 200 + (5+40) + 40 + (5+40) = 330
```

P's own command still completes at 200. Its latency was computed when it was issued and is never revisited. Suspension moves only the die's busy-until time, from 200 to 330, so operations issued afterwards, such as W2, start after 330. A suspended program or erase never makes its own command complete afterwards; it delays the commands that follow.

`nand_media_multiplane()` does not consult the suspend state, so a read issued through it never suspends; this applies to erases and to the batches of `mp_program` and `mp_read`. The state is read and written without a lock, so operations on one position must be serialized. BBSSD and ZNS run them on the single FTL thread. A

device with suspend on also leaves the lock-free fast path described in Concurrency (§3.5.11).

3.5.10 ECC read time

ecc_step_ns adds read time for worn or old data (ecc_extra_ns()):

```
tiers = pe_cycles / 750  +  age_sec / ecc_retention_sec   (second term only if set)
tiers = min(tiers, 4)
extra read time = tiers * ecc_step_ns
```

pe_cycles is the block's erase count, age_sec the time since its line was filled (line.close_time). The constants are NAND_ECC_PE_PER_TIER and NAND_ECC_MAX_TIERS in nand/nand-media.h. A multi-plane read pays the largest extra of its planes. Read-retry as a sequence of separate array reads is not modelled; this adder stands in for it. ZNS has the wear term with zns_ecc_step_ns, and no retention term. OCSSD has no ECC time.

Program disturb terms

Four opt-in properties add a program disturb sensitivity model to the same adder. They apply to bbssd, CSD, FDP and KV. These modes share the decoder bb_decode_loc() in bbssd/ftl-media.c. The FTL computes extra tiers. ecc_extra_ns() adds them before the shared cap of 4.

```
bb_decode_loc() (FTL)                    ecc_extra_ns() (media)
---------------------------------        ---------------------------------
pe_cycles   = block erase count  ----->  tiers  = pe / 750
age_sec     = age of the line    ----->        + age / ecc_retention_sec
extra_tiers = edge + open        ----->        + extra_tiers
                                         tiers  = min(tiers, 4)
                                         extra  = tiers * ecc_step_ns
```

- **Edge term.** A read of one of the bottom pd_edge_wls wordlines of a block pays pd_edge_tiers. A wordline is B contiguous pages. B is the pages per wordline of nand_cell_type (1 SLC to 4 QLC), else the effective cell_pages, else 1. The effective cell_pages is 3 when pgtype_lat is on and cell_pages is unset. The pairing tables give page types, not wordlines, so B is a synthetic grouping.
- **Open term.** A read of a block that is only part programmed pays pd_open_tiers. nand_block.wp counts the data pages programmed since the last erase. Invalidation and a stream seal do not change it.
- **Open age.** With pd_open_age, the age counts from the first program of the line (line.open_time), not from the line close. Open data ages, and a close does not start the age again. One time per line makes later pages look older than they are.

```
mark_page_valid(), mark_page_valid_fdp()  --> blk->wp++, open_time once
mark_block_free() (erase)                 --> blk->wp = 0
mark_line_free(), wear retirement, FDP    --> open_time = 0
0 < blk->wp < pgs_per_blk                 --> block part programmed
```

The tier counts are chosen values for sensitivity studies. No source measurement maps an error rate to a number of tiers. Translation page reads of dftl and the KV index address a placeholder block, so they pay the wear and close-time age only. A copyback needs a source with no ECC cost, so a page with a program disturb term moves off chip. The model does not carry error tiers to the copyback destination.

Program disturb actions

Two more opt-in properties add the firmware actions of the model. They apply to bbssd and CSD, and to FDP reclaim units. Both are 0 (off) by default, and neither needs ecc_step_ns.

trigger	action
a read of a part-programmed block takes its read count to pd_open_read_limit	queue the line; the next Write seals the frontier that fills it and rewrites the line
a frontier stays open pd_open_close_sec after its first program	program pad pages into its free pages and close it

- **Open-block read limit.** `pd_open_read_limit` is a read limit for a block with `0 < wp < pgs_per_blk`. It must not be above a non-zero `read_reclaim_limit`. The read path queues the line in the same single slot as `read_reclaim_limit` and `retention_limit_sec`. It is checked first, and it takes over a request already queued for the same line. The next Write command serves the slot, as on master. If a write pointer still fills the line, the FTL seals its free pages with no program, as a stream release does. It then rewrites the line with no copyback and counts it in C0h `pd_open_reclaims`. On master, a request for a line under a write pointer is dropped.
- **Pad and close.** `pd_open_close_sec` limits how long a frontier stays open after its first program (`line.open_time`, also set for this property). The time is device time, so `age_scale` speeds it up. When the FTL takes a request, it checks every write pointer: data, hot, stream and stream collection. For each line past the limit, it programs every free page with pad data and closes the line into the victim queue. The data pointer takes a new line at once. The others take one at their next write.

```
bb_ftl_process_req()          (every request, inside the streams lock)
  pd_open_close_sec set?
    for each write pointer with a line:
      age(open_time) >= limit, not at the high watermark, 2 free lines
        for each free page: PG_FREE -> PG_INVALID, wp++, ipc++,
                            one single-plane program at the request time
        close the line --> victim queue; C0h pd_pad_pages, pd_pad_closes
```

A pad is a NAND program with no data and no mapping. It counts in the plane programs and the energy, in the write amplification and in the media bytes. It does not count as a host write. The pads are not added to the latency of the request that finds the old line. They hold the LUNs, so that request and later ones can wait behind them. After the pad the blocks are full, so their reads no longer pay `pd_open_tiers`.

The check runs only when the FTL takes a request. A device that gets no request pads nothing, as a device that gets no read refreshes nothing for age. The single queue slot is best effort: while one line waits, reads of other lines queue nothing. Write and the write phase of Copy serve the slot. In line mode, Write Zeroes and Flush do not. Under FDP, Write Zeroes without Deallocate serves it too. KV, ZNS, OCSSD and NoSSD namespaces, `mapping=fast` and `mapping=hybrid` are refused. The ZNS form of this action, a finish recommended limit or a zone active excursion, is not modelled.

3.5.11 From operations to a completion time

The per-operation rule

For every resource the operation needs, it starts no earlier than that resource's busy-until time, and it moves the resource's busy-until time to when it stops using it. With the bus off and the LUN rule this reduces to:

```
done(op)      = max(stime, lun_avail) + array_time(op)
lun_avail     = done(op)
latency(op)   = done(op) - stime
```

`stime` is the command's submission time, stamped by the poller from `QEMU_CLOCK_REALTIME` (`nvme-io.c`; a test-only property switches it to the virtual clock). Background work (GC) passes `stime = 0`, which the

wrappers replace with the current time.

The per-command rule

The FTL issues all of a command's NAND operations with the command's `stime` and returns the largest latency:

```
latency(cmd) = max over the command's operations of latency(op)
               (and of write buffer, read cache and mapping cache costs)
expire_time  = stime + latency(cmd)           femu_ftl_thread(), femu.c
```

Then the poller adds the optional host link and firmware CPU (§3.5.13) models and queues the request in a priority queue ordered by `expire_time`. The completion is posted on the first poller sweep at or after it.

Worked example: operations overlapped across LUNs and channels

Geometry 2 channels by 2 LUNs, 1 plane, flat 40 us read and 200 us program, staged bus with `cmd_addr_lat=1000` and `pg_xfer_lat=10000`. Four operations arrive at time 0 in this order. The numbers are what `nand_media_op()` returns.

```
op   target       arrives
P    ch0 LUN0     0       program
R1   ch0 LUN1     0       read
R2   ch0 LUN0     0       read (same LUN as P)
R3   ch1 LUN0     0       read (other channel)

time (us)   0 1   11 12 13      51 52  62          211        251 261
ch0 bus     |c|-d--|c |c |       .  |o1-|            .          |o2-|
            P P    R1 R2             R1             .           R2
ch0 LUN0      .    [===== P tPROG 200 ======================)[R2 tR )
ch0 LUN1           [R1 tR 40)
ch1 bus     |c|                 |o3-|
ch1 LUN0      [R3 tR 40)

done:  P 211   R1 62   R2 261   R3 51
```

- P: command 0-1, data-in 1-11, array 11-211.
- R1: its command waits for P's data-in (11-12), its array runs on the idle LUN1 (12-52), its data-out is booked 52-62.
- R2: command 12-13, then waits for LUN0 until P ends at 211, array 211-251, data-out 251-261.
- R3: on channel 1, which nothing else uses: 0-1, 1-41, 41-51.

With the bus off, the same four operations end at P 200, R1 40, R2 240, R3 40. With `pe_suspend=1` and `tsusp_ns=5000`, R2 suspends P instead: it starts at 13 + 5 = 18, ends its array at 58, waits for R1's booked window and moves its data at 62-72 (done 72), and the die stays busy until 256. P's own completion stays at 211.

If R1 and R2 were the two pages of one 8 KiB read command, its media latency would be max(62, 261) = 261 us.

Worked example: full command completion

A host read of two written 4 KiB pages on idle LUNs of the default BBSSD (bus off), with `pcie_bandwidth_mbps=4000`, `pcie_prop_delay_ns=500` and `fw_cpu_ns=1000`, submitted at time S:

```
S                       stime stamped by the poller
S + 40 000              FTL: max(40 000, 40 000) -> expire_time
S + 40 000 .. 42 048    link: 8192 B * 1000 / 4000 MB/s = 2 048 ns on the
                        device-to-host queue (starts at max(queue free, expire))
S + 42 548              + propagation 500 ns
S + 42 548 .. 43 548    firmware core: max(core free, expire) + 1 000 ns
S + 43 548              expire_time; CQE posted on the first sweep at or after it
```

The link and firmware queues are shared by every command of the controller, so under load each command also waits for the ones queued before it on those queues.

Concurrency

- BBSSD and ZNS call the layer from the single `FEMU-FTL-Thread`, so their timelines need no lock. KV calls it from the poller and `femu-cxl-ssd` from its `femu-cxl-ftl` worker, each under its own device lock.
- With the LUN rule, no bus and no suspend, `nand_media_op()` updates `next_lun_avail_time` with a compare-and-swap loop that gives the same result as a locked read-max-add-store. Any other configuration takes the general path, which calls `lock_lun`/`unlock_lun` when the mode provides them; BBSSD and ZNS do not.
- OCSSD 1.2 with channel timing takes the channel's spinlock (`chnl_locks[]`) through `lock_lun`/`unlock_lun`; without it, and OCSSD 2.0, use the compare-and-swap path.

3.5.12 How garbage collection is charged

GC operations go through the same timelines as host operations, with `stime = 0` (the current time) and `type = GC_IO`:

- `gc_read_page()` charges a read of each valid page, `gc_write_page()` a program at its new location, and `reclaim_line()` one multi-plane erase per LUN (`bbssd/ftl-line-gc.c`). All of a victim's pages are moved before any of its blocks is erased, so the erases follow the last relocation. FDP GC does the same in `bbssd/ftl-fdp.c`.
- The returned latency is discarded. GC never adds to a command's latency directly. It makes LUNs and channels busy, and the host operations that need them afterwards wait.
- **Foreground GC** runs when the share of used lines has reached `gc_thres_pcent_high`: at the start of `ssd_write()`, and before each page that a write, a write buffer destage or a Write Zeroes programs (`bbssd/ftl-datapath.c`), until the share drops below it. The command's own programs then queue behind the GC operations on the LUNs they share.
- **Read reclaim** (`read_reclaim_limit`, `retention_limit_sec`) rewrites one queued line through `reclaim_line()` at the start of a write, charged the same way as GC.
- **Background GC** runs one victim after a request finishes in `bb_ftl_process_req()` (`bbssd/ftl.c`), once the latency of that request has been computed. It delays the commands that follow.
- Because GC is stamped with the current time and a host command with its submission time, a command submitted before a GC pass can still wait for it when the FTL thread gets to the command after the pass.
- GC time can be turned off with 0xEF code 2 (Runtime switches (§3.5.15)). GC still moves pages and erases blocks; its operations are then not charged at all.

```
LUN 3 timeline during foreground GC for one write (bus off)
|-- GC read --|-- GC program --| ... |== multi-plane erase ==|-- host program --|
^ write arrives                                                 ^ write's done
```

KV is different. Its reclaim erases and compaction moves are stamped with the triggering command's `stime`,

the command completes when the last of them and of its own operations does (`kvssd/kvssd-ftl.c`), and 0xEF does not apply to KV, so they cannot be switched off.

ZNS has no device GC. A Zone Reset charges one erase per block of the zone on every plane it spans, with the command's `stime`, and returns the largest (`zns_zone_reset()`).

Costs that are not NAND operations

The FTL adds these to a command's latency alongside its NAND operations:

- **BBSSD write buffer** (`buffer_size` > 0): a write the buffer accepts costs `pg_rd_lat / 16` (1 us when `pg_rd_lat` is 0), plus the programs of any destage it forces. A Flush pays for programming everything buffered (`bbssd/ftl.c`). A read of a buffered page costs the same DRAM access.
- **BBSSD read cache and DFTL mapping cache**: a hit costs a DRAM access; a mapping miss adds a NAND read of the translation page, after a program if the evicted entry is dirty.
- **ZNS write cache**: a write only enters the zone's write cache and pays `SRAM_WRITE_LATENCY_NS` (1 us) per 4 KiB page, summed over the command (`zns_write()` in `zns/zftl.c`). It pays program time only when the cache fills, or when it has to evict another zone's cache. A cache holds one stripe: 16 KiB * `zns_flash_type` * `zns_num_plane` * `zns_num_ch` * `zns_num_lun`. A read of data still in the cache is not mapped yet and costs nothing.

3.5.13 Host link and firmware CPU

Both are applied in `nvme_process_cq_cpl()` in `hw/femu/nvme-io.c`, after the media latency is in `expire_time` and before the request enters the priority queue. Both are off by default.

Model	Applies to	Computation
Link transfer (`pcie_bandwidth_mbps`)	NVMe Read and Write (opcodes 01h and 02h) in BBSSD, CSD, KV, ZNS and NoSSD; KV store and retrieve share those opcodes and use the bytes moved. Not Zone Append, and not the Open-Channel vector commands, so OCSSD 1.2 never pays it	`trans = bytes * 1000 / MBps` ns. One queue per direction: writes on `pcie_rx_next_avail_time`, reads on `pcie_tx_next_avail_time`. `start = max(queue, expire)`, `queue = start + trans`
Propagation (`pcie_prop_delay_ns`)	same	`expire = queue + delay`; does not occupy the queue
Firmware CPU (`fw_cpu_ns`, `fw_read_ns`, `fw_write_ns`, `fw_other_ns`, `fw_cores`)	Read (`fw_read_ns`), Write and Zone Append (`fw_write_ns`), each falling back to `fw_cpu_ns` when 0; every other I/O command, the Open-Channel vector commands included, pays `fw_other_ns`	`fw_cores` modelled cores: the command takes the core free soonest, `start = max(core, expire)`, `core = start + cost`, `expire = core`

The link is modelled after the media for both directions. A read's transfer to the host never overlaps its NAND time, and a write's transfer from the host is charged after its programs, not before them. The firmware cost is charged at the end of a command, which caps the command rate at about `fw_cores` per firmware time but does not delay the start of the media operations. The link model is enabled when either link property is non-zero (`pcie_enabled` in `femu.c`).

3.5.14 OCSSD timing model

Open-Channel devices run on the media layer, each with its own `NandMedia` and the LUN rule (`oc12_media_init()`, `oc20_media_init()`):

- Per chip (flat LUN id `ch * num_lun + lun`), an operation ends at `max(now, busy) + time`. Times come from the `flash_type` table: 1.2 as a table by page type, 2.0 as flat lower-page times (by page type with `oc20_page_types`).
- Open-Channel 1.2 with channel timing uses the staged bus with no command, address or status phase: `bus_now()` for a program's data-in and `bus_later()` for a read's data-out.
- A write moves its data over the channel, then programs the chip. A read occupies the chip, then moves its data out. Erase charges each chip in the address list.
- Open-Channel 1.2 charges a channel transfer only with `oc12_channel_timing=on`: `ch_xfer_lat` ns per page, or the table value for `flash_type` when it is 0, scaled by the sectors used out of `lsecs_per_pg`. Open-Channel 2.0 charges no channel time and uses page type 0 unless `oc20_page_types` is on.
- Open-Channel 1.2 charges one chip operation per NAND page of the address list. Open-Channel 2.0 groups addresses that differ only in the sector field, which spans the whole chunk, so a run of consecutive sectors in one chunk pays one chip operation whatever its length.
- The latency is `max over operations of (done - stime)`, written straight into `expire_time` in the poller (`oc12_advance_status()`, `oc20_advance_status()`).
- Open-Channel 1.2 keeps its data-out windows in an unbounded list per channel (`policy.bus_res_unbounded`), where the other modes cap it at 32.

3.5.15 Runtime switches

A BBSSD controller accepts the vendor admin command 0xEF (`bb_flip()` in `bbssd/bb.c`). The action is in CDW10. For codes 1 to 4 it pauses the pollers, applies the change to every namespace of the controller that has a BBSSD FTL, and resumes them. Controllers of every other mode return Invalid Opcode.

CDW10	Constant (`bbssd/ftl.h`)	Effect on timing
1	`FEMU_ENABLE_GC_DELAY`	GC operations are charged (default)
2	`FEMU_DISABLE_GC_DELAY`	GC operations are not charged
3	`FEMU_ENABLE_DELAY_EMU`	Flat read, program and erase set to the built-in 40 us, 200 us and 2 ms, not to the command-line values
4	`FEMU_DISABLE_DELAY_EMU`	Flat read, program and erase set to 0
5	accounting reset	Prints and resets the per-poller completion counters; no timing change
6, 7	`FEMU_ENABLE_LOG`, `FEMU_DISABLE_LOG`	Per-command debug log on or off; no timing change

Codes 3 and 4 refresh only the flat `rd_ns`, `wr_ns` and `er_ns` in the media layer (`bb_nand_media_refresh_timing()`). With `nand_cell_type` 1 to 4 the flat times are not used, so the codes have no effect on NAND time. They never change the channel bus phases, `pgtype_lat`, ECC time or `trim_lat_ns`. Code 4 makes a write buffer hit cost 1 us; the read cache keeps the hit cost it computed at start. On a controller linked to a `femu-cxl-ssd`, codes 1 to 4 change the medium's FTL instead, and code 3 restores its `read-ns`, `program-ns` and `erase-ns`.

From the guest:

```
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=2   # stop charging GC
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=4   # zero NAND times
```

The vendor command 0xEE (`NVME_ADM_CMD_FEMU_DEBUG`) sets an Open-Channel controller's NAND times at run time, in nanoseconds: CDW10 upper page read, CDW11 lower page read, CDW12 upper page program, CDW13 lower page program, CDW14 block erase and CDW15 channel transfer per page. The lower page is page type 0 and the upper page the highest page type of `flash_type`; the centre pages of TLC and QLC keep

their table times, and SLC, with one page type, takes the lower page values. The times belong to that controller (`FemuCtrl.oc_pg_rd_lat` and the fields beside it, filled from the `flash_type` table by `set_latency()`), so other devices are not affected. Other modes refuse 0xEE with Invalid Command Opcode, and log 05h does not list it.

```
sudo nvme admin-passthru /dev/nvme0 --opcode=0xee --cdw10=64000 --cdw11=48000 \
    --cdw12=2300000 --cdw13=850000 --cdw14=3000000 --cdw15=52433
```

3.5.16 Parameters

The generated property reference (§A.1) is the authority for types, defaults and ranges. This table groups the timing properties and states how they interact.

Group	Properties	Interactions
Geometry (§A.1.1)	`secsz`, `secs_per_pg`, `pgs_per_blk`, `blks_per_pl`, `pls_per_lun`, `luns_per_ch`, `nchs`	`nchs * luns_per_ch` is the number of independent dies. `pls_per_lun` adds no read or program parallelism under the LUN rule unless `mp_program` or `mp_read` is set.
Array times (§A.1.1)	`pg_rd_lat`, `pg_wr_lat`, `blk_er_lat`	Ignored when `nand_cell_type` is 1 to 4. 0xEF codes 3 and 4 overwrite them. `pg_rd_lat / 16` is also the cost of a write buffer hit.
Cell type (§A.1.1)	`nand_cell_type`, `cell_pages`, `pgtype_lat`	`pgtype_lat` applies only with `nand_cell_type=0`. `nand_cell_type` limits `pgs_per_blk` to 512.
Bus phases (§A.1.1)	`cmd_addr_lat`, `pg_xfer_lat`, `ch_xfer_lat`, `status_lat`	Any non-zero value turns the bus on. `pg_xfer_lat` takes precedence over `ch_xfer_lat`.
Multi-plane (§A.1.1)	`tplebsy`, `mp_program`, `tplpbsy`, `mp_read`, `tplrbsy`, `plane_first` (`trcbsy` applies only with `onfi_cache_read`)	Only with `pls_per_lun > 1`. `tplebsy` applies to the multi-plane erase of GC; `tplpbsy` only with `mp_program`; `tplrbsy` only with `mp_read`.
Suspend (§A.1.1)	`pe_suspend`, `tsusp_ns`	`tsusp_ns` matters only with `pe_suspend`.
ECC (§A.1.1)	`ecc_step_ns`, `ecc_retention_sec`	`ecc_retention_sec` matters only with `ecc_step_ns`.
Program disturb (§A.1.1)	`pd_edge_wls`, `pd_edge_tiers`, `pd_open_tiers`, `pd_open_age`	All need `ecc_step_ns`; `pd_open_age` also needs `ecc_retention_sec`.
Other FTL costs (§A.1.1)	`trim_lat_ns`	Charged per Dataset Management range by the FTL, not by the media layer, also under FDP.
ZNS (§A.1.1)	`zns_num_ch`, `zns_num_lun`, `zns_num_plane`, `zns_num_blk`, `zns_flash_type`, `zns_pg_rd_lat`, `zns_pg_wr_lat`, `zns_blk_er_lat`, `zns_cmd_addr_lat`, `zns_pg_xfer_lat`, `zns_status_lat`, `zns_pe_suspend`, `zns_tsusp_ns`	The `zns_*_lat` overrides apply to the configured cell type only. `zns_num_plane` scales the program unit.
OCSSD (§A.1.1)	`flash_type`, `oc12_channel_timing`, `ch_xfer_lat`, `lnum_ch`, `lnum_lun`, `lnum_pln`, `lsecs_per_pg`, `lpgs_per_blk`	`ch_xfer_lat` is shared with BBSSD. `lnum_ch` is at most 32 and `lnum_ch * lnum_lun` at most 128.
Host link and firmware (§A.1.1)	`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`, `fw_cpu_ns`	Applied after the media time, to the opcodes listed in Host link and firmware CPU (§3.5.13).
`femu-cxl-ssd` (§A.1.3)	`channels`, `luns-per-channel`, `pages-per-block`, `blocks-per-plane`, `read-ns`, `program-ns`, `erase-ns`, `channel-ns`	One plane per LUN; feeds the same media layer.

A BBSSD configuration with the bus and suspend on:

```
-device femu,devsz_mb=1024,femu_mode=1,cmd_addr_lat=1000,pg_xfer_lat=10000,pe_suspend=1,tsusp_ns=5000
```

A BBSSD with TLC page-type timing:

```
-device femu,devsz_mb=1024,femu_mode=1,nand_cell_type=3
```

3.5.17 Statistics

The media layer keeps no counters of its own. What it charges shows up in:

- **Latency seen by the guest**: `fio` completion latency. This is the primary output of the model.
- **Vendor log page C0h** (BBSSD, CSD): host, NAND and GC page counts, write amplification, buffer hits. See log pages and counters (§B).
- **Per-block counters** used by the model: `nand_block.erase_cnt` (ECC tiers, SMART Percentage Used) and `nand_block.read_cnt` (read reclaim), updated in `ssd_advance_status()` and on erase.
- **`femu-cxl-ssd`**: the QOM counter `media-time-ns`, the sum of the latencies the FTL returned. See runtime properties (§A.2).
- **Debug**: `nand_lun.gc_endtime` records when the last GC operation on a LUN ends; nothing reports it.

3.5.18 Calibrating against a real device

1. **Take array times from the datasheet.** `pg_rd_lat` is tR, `pg_wr_lat` is tPROG, `blk_er_lat` is tBERS. For a part with different lower and upper page times, either use `nand_cell_type` if one of the built-in tables is close, or set the mean tPROG and turn on `pgtype_lat` with `cell_pages` equal to bits per cell.
2. **Derive the bus phases from the interface rate.** `pg_xfer_lat` is the page size divided by the interface rate: a 16 KiB page on an 8-bit bus at 1600 MT/s takes 16384 / 1.6e9 s, about 10.2 us. `cmd_addr_lat` and `status_lat` are the command, address and status cycle times, usually well under 1 us.
3. **Match the parallelism.** Set `nchs` and `luns_per_ch` to the device's channels and dies per channel, and `secs_per_pg * secsz` to its page size.
4. **Check the unloaded latency.** Write the range first (a read of an unwritten page costs nothing), then run a queue depth 1 4 KiB random read. The median completion latency should be about tR plus the data-out and status phases plus guest and poller overhead. Measure that overhead once with flat timing, no bus phases, no caches and 0xEF code 4, and subtract it.
5. **Check the saturated throughput.** A random read at high queue depth should approach `nchs * luns_per_ch / tR` operations per second when the bus is not the limit, or `nchs / (pg_xfer_lat + cmd_addr_lat + status_lat)` when it is. Compare with the real device at the same queue depth.
6. **Add the host side.** If the real device's peak is below what the media allows, set `pcie_bandwidth_mbps` to its link rate and `fw_cpu_ns` to the inverse of its peak command rate.
7. **Check GC separately.** Precondition the device, run sustained random writes, and compare the write amplification in log page C0h and the steady-state write bandwidth. Toggle 0xEF codes 1 and 2 to see how much of the gap GC time explains.

Give each poller and the FTL thread its own host core while measuring; performance tuning (§5.12) and measuring (§6.1) cover the setup.

3.5.19 Validation status

- **Unit tests**: `hw/femu/tests/unit/test-nand-media.c` checks the media layer's arithmetic without QEMU: ECC tiers, the staged bus and its booked windows, the plane rule with the bus, multi-plane

erase against serial erases, copyback, and program/erase suspend. Run it with `make -C hw/femu/tests check`.

- **qtests**: `hw/femu/tests/qtest/femu-test.c` covers the OCSSD 1.2 channel model (`oc12-channel-timing`, `oc12-channel-gap`, `oc12-channel-default`, `oc12-channel-off`, `oc12-ppa-timing`, `oc12-flash-type`, `oc12-page-count`, `oc12-transfer-cost`, and the exact per-command times of `oc12-trace-on` and `oc12-trace-off`), the warning for the timing properties that have no effect (`ignored-props`), the 0xEF flips on a linked `femu-cxl-ssd` (`cxl-nvme-flip`), the two example configurations on this page (`doc-examples`), and program/erase suspend on a BBSSD, CSD, KV and ZNS device (`pe-suspend-<mode>-on` and `pe-suspend-<mode>-off`): a read that arrives during a program completes before the program only when suspend is on.
- The firmware CPU model per opcode and per core (`fw-cost-read`, `fw-cost-other`, `fw-cost-cores1`, `fw-cost-cores2`).
- **Not covered by automated tests**: other BBSSD and ZNS timing inside QEMU, the host link, the 0xEF flips on a plain BBSSD controller, and end-to-end guest latency against a reference device. The built-in MLC table is a profile of real parts; the other tables are from published papers, as cited in the headers. Check numbers that matter to you with the calibration steps above.

```
make -C hw/femu/tests check
```

3.5.20 What is not modelled

- Read-retry as repeated array reads, soft-decision decoding, and read disturb errors. The ECC adder is a fixed time per tier, and data is never corrupted.
- Multi-plane reads and programs outside BBSSD and CSD. `mp_read` and `mp_program` batch host pages there; they are off by default and refused with FDP. Line erases are always batched.
- Cache read outside the ONFI profile, and cache program. `onfi_cache_read` models the read cache sequence of one command (Read cache sequence (§3.5.7)).
- Copyback outside BBSSD line GC and FDP GC. Paced collection, read reclaim and retention refresh move pages off-chip.
- Bit errors of a copyback. A real copyback skips ECC, so its bit errors are not corrected and can add up; FEMU models no bit errors, so a copyback changes only timing.
- Program suspend for programs, or erase suspend by another erase. Only reads suspend.
- Per-die power or thermal limits, and temperature-dependent timing. The package thermal model (`thermal_tau_ms`, `thermal_r`) reports a temperature but does not slow the NAND.
- Plane-level parallelism for BBSSD, CSD and KV reads and programs (the rule is per LUN), and the channel bus for OCSSD 2.0.
- PLC outside ZNS. ZNS accepts PLC with times you supply; BBSSD resets `nand_cell_type=5` to 0 and OCSSD refuses `flash_type=5`.
- Queueing inside the controller other than the link and the single firmware core: there is no DRAM bandwidth or NAND controller queue model.
- Timing accuracy below the poller sweep. A completion is posted on the first sweep after it is due, and a poller without its own host core adds delay.

3.5.21 Adding a timing feature

1. **Decide the resource.** If the feature makes an existing resource busy for longer, it is an `array_lat()` or bus-phase change. If it adds a resource, add an accessor to `NandTimelineOps` and keep the state in the mode's structures, like the existing busy-until fields.
2. **Add the duration to `NandMediaTiming` and a switch to `NandMediaPolicy`** in `nand/nand-media.h`.

Default it to 0 or off so that existing configurations keep their exact timing. The layer must not include a controller header or branch on the mode.

3. **Implement it in `nand-media.c`.** Keep the max() ordering of the existing paths. A phase that happens at issue goes through `bus_now()`; one that happens after the array goes through `bus_later()`.
4. **Wire the property.** Add it with `DEFINE_PROP_*` in `femu.c` and a description in `femu-props.c`, copy it into `ssdparams` in `ssd_init_params()` (`bbssd/ftl-geom.c`), validate it in `bb_check_geometry()`, and set the media config in `bb_nand_media_init()`. For ZNS, the same in `zns_init_params()` and `zns_nand_media_init()`.
5. **Test the arithmetic** in `tests/unit/test-nand-media.c`. Include a case that fails if the feature is ignored, so the test cannot pass with the feature off.
6. **Update the docs.** Regenerate the property reference with `gen-property-docs.py`, add the parameter to this page and to the timing model concept page (§6.2), and add a CHANGELOG entry. Keeping the documentation correct (§8.3) lists the checks.

3.5.22 Source map

File	What it holds
`hw/femu/nand/nand-media.h`[8]	Media layer types and API: `NandLoc`, `NandMediaTiming`, `NandMediaPolicy`, `NandTimelineOps`, `nand_media_op()`, `nand_media_multiplane()`, `nand_media_copyback()`
`hw/femu/nand/nand-media.c`[9]	Array time, rules, bus booking (`bus_now()`, `bus_later()`), suspend, ECC, multi-plane, copyback
`hw/femu/nand/nand.h`[10], `nand.c`[11]	Cell-type timing tables, page pairing tables, rated P/E cycles
`hw/femu/bbssd/ftl-media.c`[12]	BBSSD adapter: `bb_decode_loc()`, `bb_nand_media_init()`, `ssd_advance_status()`, `ssd_advance_status_multiplane()`, `bb_nand_media_refresh_timing()`
`hw/femu/bbssd/ftl-geom.c`[13]	Geometry checks and parameter copy (`bb_check_geometry()`, `ssd_init_params()`)
`hw/femu/bbssd/ftl-datapath.c`[14]	Host read and write: per-page operations and max latency
`hw/femu/bbssd/ftl-line-gc.c`[15]	Write pointer order, GC reads, programs and multi-plane erase
`hw/femu/bbssd/bb.c`[16]	0xEF handler (`bb_flip()`, `bb_flip_apply()`)
`hw/femu/zns/zftl.c`[17]	ZNS adapter, write cache flush, zone reset erase
`hw/femu/zns/zns.c`[18], `zns.h`[19]	ZNS timing values and property overrides (`zns_init_params()`)
`hw/femu/ocssd/oc-timing.c`[20]	OCSSD `flash_type` times, geometry check, 0xEE

[8] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand-media.h
[9] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand-media.c
[10] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand.h
[11] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand.c
[12] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl-media.c
[13] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl-geom.c
[14] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl-datapath.c
[15] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl-line-gc.c
[16] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/bb.c
[17] https://github.com/MoatLab/FEMU/blob/master/hw/femu/zns/zftl.c
[18] https://github.com/MoatLab/FEMU/blob/master/hw/femu/zns/zns.c
[19] https://github.com/MoatLab/FEMU/blob/master/hw/femu/zns/zns.h
[20] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc-timing.c

File	What it holds
`hw/femu/ocssd/oc12.c`[21], `oc20.c`[22]	OCSSD per-command timing (`oc12_advance_status()`, `oc20_advance_status()`)
`hw/femu/femu.c`[23]	FTL thread: `expire_time += latency`; timing properties
`hw/femu/nvme-io.c`[24]	`stime` stamp, host link and firmware CPU models, priority queue and completion
`hw/femu/tests/unit/test-nand-media.c`[25]	Media layer unit tests

[21] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc12.c
[22] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc20.c
[23] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu.c
[24] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-io.c
[25] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tests/unit/test-nand-media.c

4 Modes and features

4.1 Choosing a mode

An NVMe `femu` device emulates one kind of SSD, chosen with `femu_mode`. Features such as Flexible Data Placement or several namespaces are added on top of a mode with more properties. The CXL SSD is a separate device type, `femu-cxl-ssd`, not a `femu_mode`.

If you set nothing, you get NoSSD: `femu_mode` defaults to 2.

4.1.1 `femu_mode` values

Value	Name	Emulates
0	OCSSD	An Open-Channel SSD. The host runs the FTL. `lver=1` selects Open-Channel 1.2, `lver=2` (default) selects 2.0.
1	BBSSD	A conventional ("black-box") SSD with a device FTL, garbage collection and NAND timing.
2	NoSSD	An NVMe drive with no media timing. Default.
3	ZNS	A Zoned Namespace SSD (NVMe Zoned Namespace command set).
4	CSD	A computational storage drive that runs programs next to the data, on top of the BBSSD FTL.
5	KV	A key-value SSD (NVMe Key Value command set).

Any other value fails realize with "femu_mode must be 0 (OpenChannel), 1 (black-box), 2 (no-SSD), 3 (zoned), 4 (computational storage) or 5 (key-value)". With `femu_mode=0`, an `lver` other than 1 or 2 fails too.

4.1.2 Decision table

Settings are added to `-device femu,...` unless the row says otherwise. Each launcher is in `hw/femu/scripts/`. Each guide covers launch, configuration, guest-side use, refusals and troubleshooting for its mode or feature.

Goal	Mode or feature	Key settings	Launcher	Guide
Fastest emulated NVMe drive, for host software stack work	NoSSD	`femu_mode=2` (or nothing)	`run-nossd.sh`	NoSSD (§4.2)
A conventional SSD with realistic latency, GC and write amplification	BBSSD	`femu_mode=1`; geometry `nchs`, `luns_per_ch`, `pls_per_lun`, `blks_per_pl`, `pgs_per_blk`; timing `pg_rd_lat`, `pg_wr_lat`, `blk_er_lat`	`run-blackbox.sh`	BlackBox (§4.4)
A zoned drive for ZNS-aware filesystems and databases	ZNS	`femu_mode=3`; `zns_num_ch`, `zns_num_lun`, `zns_num_plane`, `zns_num_blk`, `zns_flash_type`	`run-zns.sh`	ZNS (§4.7)
Host-managed flash: your own FTL in the host	OCSSD	`femu_mode=0`, `lver=2` (or 1); `lnum_ch`, `lnum_lun`, `lnum_pln`, `lpgs_per_blk`	`run-whitebox.sh`	OCSSD (§4.9)
Store and fetch values by key, with no file system	KV	`femu_mode=5`	none	KV (§4.11)

Goal	Mode or feature	Key settings	Launcher	Guide
Run filters or other programs inside the drive	CSD	`femu_mode=4`, `fdm_size` (required), `nr_cu`, `csd_program_dir`	`run-csd.sh`	CSD (§4.13)
Steer writes to reclaim units to reduce write amplification	FDP on BBSSD	`-device femu-subsys,id=s0,fdp=on,fdp.nruh=N` before the controller, then `femu_mode=1`, `subsys=s0`	`run-blackbox-fdp.sh`	FDP (§4.5)
Several namespaces on one controller, possibly of different modes	multi-namespace	`namespaces=N`, optional `namespace_sizes=...`, `namespace_modes=...`	none	Several namespaces (§4.15)
Create and delete namespaces from the guest	Namespace Management	`ns_mgmt=on` on a NoSSD or BBSSD controller; `bbssd_ns_limit` for BBSSD	none	Namespace management (§4.17.2)
Metadata and end-to-end protection information	metadata, PI	`meta=8` (or more), `mc`, `pi=on`	none	Metadata and PI (§4.17.3)
SSD capacity that the guest uses as memory, with SSD timing on cache misses	CXL SSD	`-device femu-cxl-ssd,volatile-memdev=...` on a CXL topology; `cache-pages`, `cache-policy`, `der`	`run-cxlssd.sh`	CXL SSD (§4.18); design note[26]
The same CXL medium also as an NVMe block device	CXL NVMe front end	`femu-cxl-ssd` first, then `-device femu,femu_mode=1,cxl_ssd=<id>`	none	CXL NVMe link (§4.21)

Commas inside a property value are doubled on the QEMU command line, for example `namespace_modes=bbssd,,znssd`.

4.1.3 Which features combine

The checks below are made at realize; a combination they refuse stops QEMU with an error naming the rule.

Flexible Data Placement. `fdp=on` is a property of `femu-subsys`. Only BBSSD has an FDP write path. NoSSD counts the host bytes of FDP but has no media to place on. ZNS, OCSSD and KV namespaces are refused: an FDP endurance group holds NVM command set namespaces only. FDP requires a single controller in the subsystem. It also requires `fdp.nruh` from 1 to `fdp.nru`. With BBSSD, `fdp.nrg` above 1 makes each group a set of dies. More than one namespace needs every namespace in BBSSD mode, in whole NAND pages, or every one in NoSSD mode. `namespace_phls` gives each namespace its placement handle list. It cannot be combined with a KV namespace, `meta`, Streams or a subsystem with `ns_mgmt=on`. A BBSSD controller under FDP also refuses `hot_cold_sep` and the `hybrid` and `fast` mappings.

Several namespaces and per-namespace modes. `namespaces` is 1 to 256. The backend of `devsz_mb` MiB is split evenly, or by `namespace_sizes`. `namespace_modes` lists one mode per namespace from `nossd`, `bbssd`, `znssd`, `ocssd`, `csd` and `kvssd`; without it every namespace runs `femu_mode`. Limits:

- OCSSD supports one namespace only, and the controller must be OCSSD too.
- At most one CSD namespace per controller.
- ZNS, KV, BBSSD and NoSSD namespaces can be mixed.

[26] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

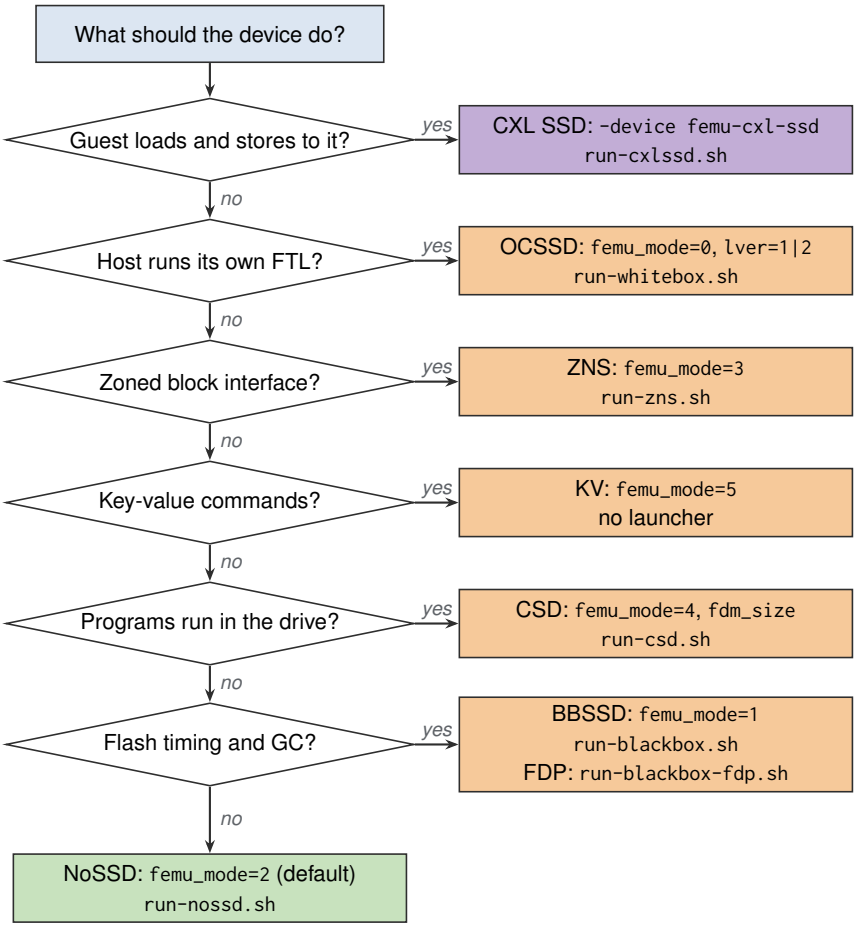


Figure 13: Choosing a mode: answer the questions top to bottom; the first "yes" names the mode, its `femu_mode` value and its launcher in `hw/femu/scripts/`.

- meta and Streams need every namespace to be NoSSD or BBSSD.

Namespace Management. ns_mgmt=on takes effect only when the controller and every namespace are NoSSD, or every one is BBSSD; with other modes, or with dps, it is accepted and stays off. On a controller that joins a subsystem without ns_mgmt, it fails realize. To share namespaces between controllers, set ns_mgmt=on on the femu-subsys instead. A shared subsystem takes NoSSD or BBSSD controllers that all have the same mode, meta, mc, pi, dpc, nlbaf, vwc and oncs, and no Streams, dps or namespace_modes.

Metadata and protection information. meta (bytes per block) works on NoSSD and BBSSD namespaces only, not with FDP, and needs a matching mc bit. pi=on offers protection information types 1 to 3 when meta is at least 8; the guest selects one with Format NVM or Namespace Management. pi cannot be combined with power_loss or cxl_ssd.

CXL NVMe front end. A femu controller with cxl_ssd=<id> must be BBSSD with one namespace, and must not use namespace_modes, namespace_sizes, ns_mgmt, subsys, streams, power_loss, buffer_size, op_pcent, meta, pi or dps. The femu-cxl-ssd must come first on the command line and have ftl=on. Its geometry and timing apply; the controller's own are ignored, and devsz_mb must be left at its default or equal the CXL medium's size.

Other combinations.

- Streams (streams=on): NoSSD or BBSSD, not with FDP or a shared subsystem; BBSSD needs mapping page or dftl.
- Power-loss model (power_loss=on): BBSSD with buffer_size > 0, page-aligned namespaces, and no meta, pi, ns_mgmt, subsys, namespace_modes or cxl_ssd. Without vwc=1 the write buffer holds nothing, so there is nothing to lose.
- ZNS needs a logical block size of 4 KiB or less (lba_index).

4.1.4 Every mode at a glance

Generated from modes.py[27]: how to turn each mode or feature on, the guest kernel and tools it needs, what the host needs, and what CI checks. requirements.md (§2.1.2) has the guest kernel configuration in more detail.

Mode or feature	Use it for	Turn it on with	Guest kernel	Guest tools	Host needs	Launcher	Checked
NoSSD (§4.2)	fast NVMe device in DRAM, no flash timing	femu_mode=2 (the default)	any with the NVMe driver	nvme-cli, fio	none beyond the common ones	run-nossd.sh	CI: realize, Identify, write and read back
BlackBox SSD (BBSSD) (§4.4)	a commercial SSD: device FTL, GC, NAND timing	femu_mode=1	any with the NVMe driver	nvme-cli, fio	about 17 GiB free RAM for the launcher's 12 GiB device	run-blackbox.sh	CI: realize, Identify, write and read back; guest: quick start, run end to end (§2.4)
Zoned Namespace (ZNS) (§4.7)	zoned storage research	femu_mode=3	5.9 or newer with CONFIG_BLK_DEV_ZONED=y; 4 KiB guest pages	nvme-cli 1.12 or newer for nvme zns	none beyond the common ones	run-zns.sh	CI: realize, Identify, write and read back

[27] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/modes.py

Mode or feature	Use it for	Turn it on with	Guest kernel	Guest tools	Host needs	Launcher	Checked
Open-Channel SSD 1.2 (§4.9)	host-managed FTL research	`femu_mode=0`, `lver=1`	4.16 to 5.14 (LightNVM was removed in 5.15)	LightNVM tools, or SPDK on newer kernels	none beyond the common ones	`run-whitebox.sh`	CI: realize, Identify
Open-Channel SSD 2.0 (§4.9)	host-managed FTL research	`femu_mode=0` (`lver=2` is the default)	4.17 to 5.14 (LightNVM was removed in 5.15)	LightNVM tools, or SPDK on newer kernels	none beyond the common ones	`run-whitebox.sh`	CI: realize, Identify
Key-value SSD (KV) (§4.11)	key-value store research	`femu_mode=5`	6.0 or newer; no block device, the namespace is `/dev/ngXnY`	nvme-cli `io-passthru`, `hw/femu/scripts/kv-probe.c`	none beyond the common ones	`run-kvssd.sh`	CI: realize, Identify, store and retrieve
Computational storage (CSD) (§4.13)	running programs next to the data	`femu_mode=4`, `fdm_size=<MiB>`	any with the NVMe driver	`hw/femu/tests/csd` tools	`csd_program_dir` for shared-library programs; `--enable-csd-ubpf` build for eBPF programs	`run-csd.sh`	CI: realize, Identify, write and read back
Flexible Data Placement (FDP) (§4.5)	placement hints on a BBSSD	`femu-subsys`, `fdp=on,fdp.nruh=<n>` and `femu,femu_mode=1`, `subsys=<id>`	any with the NVMe driver; placement hints need passthrough or io_uring commands	nvme-cli with `nvme fdp`	none beyond the common ones	`run-blackbox-fdp.sh`	CI: realize, Identify, write and read back
Multiple namespaces (§4.15)	several namespaces, each with its own mode	`namespaces=<n>`, optionally `namespace_sizes` and `namespace_modes`	any with the NVMe driver (ZNS namespaces need what ZNS needs)	nvme-cli	none beyond the common ones	none	CI: realize, Identify, write and read back
Namespace management (§4.17.2)	create, delete and attach namespaces at run time	`ns_mgmt=on` on a NoSSD or BBSSD controller; `femu-subsys`, `ns_mgmt=on` to share namespaces	any with the NVMe driver	nvme-cli `create-ns`, `attach-ns`	none beyond the common ones	none	CI: realize, Identify, write and read back
Metadata and protection information (§4.17.3)	per-block metadata, PI types 1 to 3	`meta=<bytes>,mc=<mask>`, plus `pi=on` with `meta` of 8 or more	`CONFIG_BLK_DEV_INTEGRITY=y` to use metadata formats through the block layer	nvme-cli `format`	none beyond the common ones	none	CI: realize, Identify, write and read back

Mode or feature	Use it for	Turn it on with	Guest kernel	Guest tools	Host needs	Launcher	Checked
CXL SSD, `der=off` (§4.18)	CXL memory backed by flash, all accesses trapped	`femu-cxl-ssd` below `pxb-cxl` and `cxl-rp` on `-machine q35,cxl=on`	`CONFIG_CXL_BUS`, `CXL_PCI`, `CXL_ACPI`, `CXL_MEM`, `CXL_PORT`, `CXL_REGION`, `CXL_REGION_INVALIDATION_TEST` (in a VM), `DEV_DAX`, `DEV_DAX_CXL`, `DEV_DAX_KMEM`	`cxl-cli`, `daxctl`, `ndctl`	a build with `CONFIG_CXL_MEM_DEVICE`	`run-cxlssd.sh`	CI: realize
CXL SSD, `der=memslot` (§4.18.7)	cached pages mapped into the guest as KVM memory slots	`der=memslot` on `femu-cxl-ssd`	as for `der=off`	as for `der=off`	KVM (TCG is refused)	`run-cxlssd.sh`	CI: realize
CXL SSD, `der=cylon` (§4.18.7)	cached pages mapped by a Cylon host kernel	`der=cylon`, `cylon-kernel-ack=on` on `femu-cxl-ssd`	as for `der=off`	as for `der=off`	Cylon host kernel; KVM with EPT A/D bits and the TDP MMU; 4 KiB host pages; a shared, preallocated hugetlb backend. Without them the device warns and uses MMIO	`run-cxlssd.sh`	CI: realize
CXL caching API (CCA) (§4.20)	guest pins, unpins and invalidates cached pages	`cca=on` on `femu-cxl-ssd`	as for `der=off`; a devdax region	`hw/femu/tools/cca` (`ccactl`, `cca-test`), run as root	as for `der=off`	`run-cxlssd.sh`	CI: realize
NVMe front end on a CXL SSD (§4.21)	the same media as CXL memory and as an NVMe namespace	`femu,bus=pcie.0,femu_mode=1,cxl_ssd=<id>` after the `femu-cxl-ssd`	as for `der=off`, plus the NVMe driver	as for `der=off`, plus nvme-cli	as for `der=off`	none	CI: realize, Identify, write and read back

The image built by `make-guest-image.sh` runs Linux 6.8 and covers every mode except OCSSD.

4.1.5 Related pages

- Architecture (§3.1): how the modes fit into FEMU.
- Timing model (§6.2): what each mode charges time for.
- Device property reference (§A.1)
- Mode guides: NoSSD (§4.2), BlackBox (§4.4), ZNS (§4.7), OCSSD (§4.9), KV (§4.11), CSD (§4.13)
- Feature guides: FDP (§4.5), several namespaces (§4.15), namespace management and PI (§4.17)

Mode `femu_mode`	**NoSSD** 2	**BBSSD** 1	**ZNS** 3	**OCSSD** 0	**KV** 5	**CSD** 4	**CXL SSD** device
Interface	NVMe NVM	NVMe NVM	NVMe Zoned	OC vector	NVMe KV	NVM and CSD	CXL.mem loads
FTL	none	map and GC	zone FTL	in the host	hash index	BBSSD FTL	cache + FTL
NAND timing	none	`nand/`	`nand/`	`timing-model/`	BBSSD model	`nand/`, CUs	`nand/`
Data store	DRAM backend	DRAM backend	DRAM backend	DRAM backend	value arena	DRAM backend	memory backend
Timing on	poller	FTL thread	FTL thread	poller	poller	FTL + poller	vCPU, cxl-ftl

FEATURES

FDP		●					
Write buffer		●				●	
Namespaces	●	●	●		●	○	
NS mgmt	●	●					
Metadata, PI	●	●					
Streams	●	●					
NVMe link							●
CCA							●

● supported ○ one CSD namespace per controller, mixed with others

Figure 14: Modes against layers: the interface each mode offers, where its FTL runs, which timing model it uses, where its data lives and which thread charges its time; below, the features each mode supports (CUs are CSD compute units).

4.2 NoSSD

NoSSD mode (`femu_mode=2`, the default) emulates an NVMe drive with no media model. Reads and writes copy data between guest memory and a host DRAM buffer and complete as soon as the poller finishes the copy. There is no FTL, no GC and no NAND time.

Use it when the storage device should not be the bottleneck: work on the host I/O stack, the NVMe driver, polling, SPDK or io_uring, or as an upper bound to compare a timed mode against. For a device with SSD latency and garbage collection, use BlackBox (§4.4).

4.2.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). Any guest kernel with the NVMe driver works.
- Host memory: `devsz_mb` plus guest RAM plus about 1 GiB for QEMU, about 9 GiB for `run-nossd.sh` (a 4 GiB device and a 4 GiB guest).
- Host cores: the pollers spin on host cores. For throughput work, give each `femu-poller` thread a core of its own.

4.2.2 Launch

From `build-femu/`:

```
./run-nossd.sh
```

The FEMU device in that script is:

```
-device femu,devsz_mb=4096,id=nvme0
```

`femu_mode` is not set, so it is 2. Writing `femu_mode=2` has the same effect.

4.2.3 Configuration

NoSSD ignores the NAND geometry, timing and FTL properties. These are the ones that change its behaviour.

Capacity and namespaces

Properties: mode, capacity and namespaces (§A.1.1).

`devsz_mb` is the namespace size. `namespaces`, `namespace_sizes` and `namespace_modes` split it into several namespaces (multi-namespace guide (§4.15)).

Queues and pollers

Properties: queues, pollers and interrupts (§A.1.1).

- `queues` sets the number of I/O queue pairs (default 8).
- By default one poller thread serves every I/O queue. With `multipoller_enabled=1`, FEMU runs `ceil(queues / poller_ratio)` pollers, each owning a round-robin share of the queues:

```
-device femu,devsz_mb=4096,queues=8,multipoller_enabled=1,poller_ratio=2
```

This starts four pollers. Give each one a host core.

Host link and controller firmware

Properties: host link and controller firmware (§A.1.1).

`pcie_bandwidth_mbps`, `pcie_prop_delay_ns` and `fw_cpu_ns` add link and firmware time to each Read and Write. With any of them set, NoSSD completes commands through the same timed queue as the other modes instead of in the poller sweep, which adds work per command.

```
-device femu,devsz_mb=4096,pcie_bandwidth_mbps=3500,pcie_prop_delay_ns=1000
```

The preset `hw/femu/scripts/configs/nossd.conf` sets all three to the figures of the NVMeCHA FPGA controller with a null backend (Qiu et al., IEEE TCAD, doi:10.1109/TCAD.2021.3088784): a 7000 MB/s link and about 2.4 us per 4 KiB read. A 4 KiB transfer at 7000 MB/s takes 585 ns, so with 1300 ns of link delay and 500 ns of firmware a 4 KiB Read or Write costs 2385 ns, and the link holds 4 KiB I/O near the controller's 1.7 million IOPS. FEMU charges writes the same as reads, below the controller's 3.2 us, and the guest also sees FEMU's own per-command time on top. `ssd-config.sh` (§5.11) expands it to:

```
-device femu,id=nvme0,devsz_mb=4096,namespaces=1,pcie_bandwidth_mbps=7000,pcie_prop_delay_ns=1300,fw_cpu_ns=500,femu_mode=2
```

Optional commands and features

Properties: controller identity and capabilities (§A.1.1), LBA formats, metadata and protection (§A.1.1).

- `oncs` turns on optional NVM commands. The default (0x19d) offers Compare, Dataset Management, Write Zeroes, Save/Select, Verify and Copy. Add 0x2 for Write Uncorrectable.
- NoSSD supports namespace management, metadata and protection information (§4.17) and Streams (`streams=on`, which tracks streams but places nothing).

4.2.4 Use it from the guest

Check the device:

```
sudo nvme list
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(mn|sn) '
```

The model is `FEMU NoSSD NVMe Controller` and the serial number starts with `vNoSSD`.

Measure latency and throughput with fio. Unlike BlackBox, unwritten blocks read at the same speed as written ones:

```
sudo fio --name=lat --filename=/dev/nvme0n1 --direct=1 --ioengine=io_uring \
    --rw=randread --bs=4k --iodepth=1 --runtime=30 --time_based
sudo fio --name=tput --filename=/dev/nvme0n1 --direct=1 --ioengine=io_uring \
    --rw=randread --bs=4k --iodepth=64 --numjobs=4 --group_reporting \
    --runtime=30 --time_based
```

The SMART log counts host reads and writes in every mode, NoSSD included:

```
sudo nvme smart-log /dev/nvme0
```

The vendor log page C0h stays zero: NoSSD has no media to count.

4.2.5 Limits and refusals

Message	Cause and fix
`multipoller_enabled must be 0 (one poller for all queues) or 1 (each poller owns poller_ratio queues)`	Any other value.
`queues must be in [1, 2047]`	`queues` out of range.
`oncs may only set Compare, Write Uncorrectable, DSM, Write Zeroes, Save/Select Feature Support, Verify and Copy`	A bit FEMU does not implement.

If the host cannot allocate `devsz_mb` of memory, QEMU aborts in GLib with `failed to allocate N bytes`.

4.2.6 Verify

1. `sudo nvme list` shows the model `FEMU NoSSD NVMe Controller`.
2. A queue depth 1 random read reports a completion latency of a few microseconds to tens of microseconds, depending on the host, with no difference between written and unwritten blocks.
3. `sudo nvme smart-log /dev/nvme0` shows the read and write command counts moving.

4.2.7 Troubleshooting

- **Throughput does not grow with more guest jobs.** A single poller serves every queue by default. Set `multipoller_enabled=1` with a `poller_ratio` that leaves one poller per host core you can spare, and pin the threads.
- **Latency varies from run to run.** The pollers compete with vCPUs and other host work. Pin vCPUs and pollers to separate cores, and keep the host CPU at a fixed frequency (see performance tuning (§5.12)).
- **Data is gone after QEMU exits.** FEMU keeps the device only in host memory. A guest reboot keeps the data.

Related issues: #52, #69.

4.2.8 Related pages

- Choosing a mode (§4.1)
- Architecture: pollers (§3.1.3)
- Timing model (§6.2)

4.3 NoSSD: the mode without a media model

This chapter describes NoSSD (`femu_mode=2`, the default): what it leaves out compared with the other modes, the path a command takes, and when to use it. To run the mode, see the NoSSD guide (§4.2).

4.3.1 Purpose

NoSSD is a plain NVMe namespace stored in host memory. It keeps FEMU's whole NVMe frontend (queues, pollers, admin commands, optional commands, namespaces) and drops everything below it:

Layer	BlackBox (`femu_mode=1`)	NoSSD
NVMe frontend, pollers	yes	yes
FTL: mapping, write buffer, GC, wear	yes, on the FTL thread	none
NAND timing: chips, channels, planes	yes	none

Layer	BlackBox (`femu_mode=1`)	NoSSD
FTL thread	started	not started for NoSSD namespaces
Data	memory backend	memory backend
Completion time	arrival + FTL and NAND time	arrival, plus optional link and firmware time

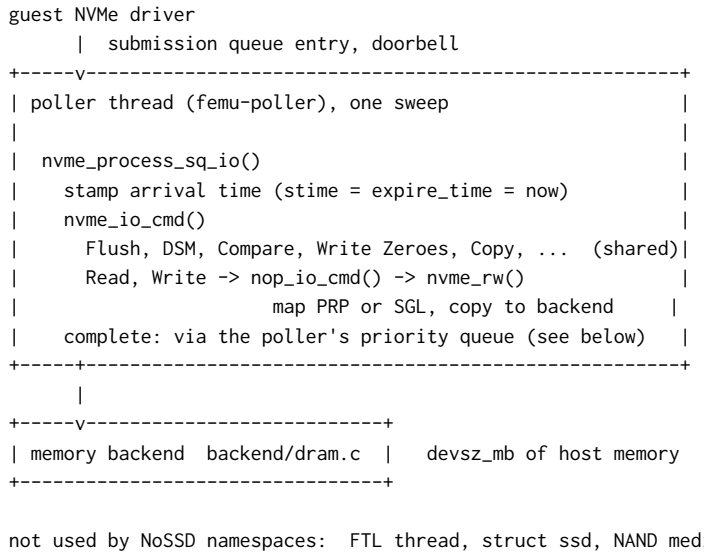
A command completes as soon as the poller has copied its data. What the guest then measures is the cost of the emulation itself: the guest driver, the doorbell exit, the poller, the copy and the interrupt.

Use NoSSD when:

- the storage device should not be the bottleneck, for example to work on the guest's I/O stack, the NVMe driver, io_uring or SPDK;
- you need an upper bound to compare a timed mode against. The difference between a BlackBox run and a NoSSD run with the same queues and pollers is mostly the time FEMU charges for the media and the FTL, but it also includes the cost of BlackBox's longer path through the FTL thread;
- you test NVMe features that do not depend on media, such as namespace management, metadata and protection information, or Streams.

Use BlackBox (§4.4) when you need SSD latency, garbage collection or write amplification.

4.3.2 Place in the hierarchy



`nvme_register_nossd()` (`hw/femu/nossd/nop.c`) installs a handler table with three entries that do work: `init` and `init_ctrl_name`, which set the model and serial strings, and `io_cmd`, which sends Read and Write to the shared `nvme_rw()` and returns Invalid Opcode for anything else that reaches it. The optional commands (Dataset Management, Compare, Write Zeroes, Copy, Verify, Write Uncorrectable) are handled in `nvme_io_cmd()` before the mode is consulted, when `oncs` turns them on. With `streams=on`, a successful write also records its stream, though NoSSD places nothing differently.

4.3.3 Data path

`nvme_rw()` checks the LBA range and MDTS, maps the guest's PRP or SGL list, and copies between guest memory and the memory backend, normally with `backend_rw()`. The copy is done on the poller thread before the command completes. The backend is one `devsz_mb` buffer allocated at realize; FEMU tries to pin it with `mlock()` and prints a notice when `RLIMIT_MEMLOCK` is too small, since page faults would then add to the measured latency. Namespaces are slices of that buffer (multi-namespace guide (§4.15)).

The data survives a guest reboot and is lost when QEMU exits.

4.3.4 Completion path

Every request is stamped with its arrival time as both its start time and its completion time (`expire_time`). NoSSD adds nothing to it, so a NoSSD request is due the moment it is stamped.

```
fetch -> execute -> copy -> to_ftl ring
      -> nvme_process_cq_cpl(): + link time, firmware time
      -> priority queue -> post when due -> interrupt
```

With no FTL thread, the poller drains its own `to_ftl` ring at the end of the sweep, so the request is posted in the same sweep unless a link or firmware time pushes it into the future or the completion queue is full. A completion that finds the queue full waits in a per-poller backlog. When a black-box, ZNS or CSD namespace on the same controller has started the FTL thread, the request goes through that thread and back.

`hiops_inline` (§A.1.1) applies to NoSSD only: performance option, on by default; set off only when debugging.

4.3.5 Threads and pollers

NoSSD runs on the poller threads alone. How many there are, and which queues each owns, is set by `multipoller_enabled` and `poller_ratio` (§A.1.1) as in every mode; the architecture page (§3.1.3) explains the split. Because there is no media time to hide behind, the pollers' CPU is the device's speed limit in this mode. Give each poller a host core of its own, away from the vCPUs, and see performance tuning (§5.12).

4.3.6 Timing

None from the media. The optional models in the host link and controller firmware (§A.1.1) group apply to NoSSD as to every mode:

- `pcie_bandwidth_mbps`: transfer time per Read and Write on one device-wide link queue per direction;
- `pcie_prop_delay_ns`: a fixed delay after each transfer;
- `fw_cpu_ns`: a fixed time per Read, Write and Zone Append on one modelled controller core; `fw_read_ns`, `fw_write_ns` and `fw_other_ns` set the time per opcode, and `fw_cores` sets the number of cores.

4.3.7 Parameters

NoSSD ignores the NAND geometry, NAND timing and FTL properties. The ones that matter:

Group	Properties
Mode, capacity and namespaces (§A.1.1)	`devsz_mb`, `namespaces`, `namespace_sizes`, `namespace_modes`
Queues, pollers and interrupts (§A.1.1)	`queues`, `multipoller_enabled`, `poller_ratio`, `hiops_inline`, `mdts`

Group	Properties
Host link and controller firmware (§A.1.1)	pcie_bandwidth_mbps, pcie_prop_delay_ns, fw_cpu_ns
Controller identity and capabilities (§A.1.1)	oncs, cmbsz, vwc
LBA formats, metadata and protection (§A.1.1)	meta, mc, pi, nlbaf, lba_index
Namespace management, streams and power loss (§A.1.1)	ns_mgmt, streams

A NoSSD device with four pollers, each owning two of eight queues:

```
-device femu,devsz_mb=1024,queues=8,multipoller_enabled=1,poller_ratio=2
```

4.3.8 Counters

- The SMART log counts host read and write commands and bytes, kept per poller and summed when the log is read.
- The vendor log page C0h stays zero on a controller whose namespaces are all NoSSD: there is no media to count. Black-box, CSD and KV namespaces on the same controller fill it.

4.3.9 Validation status

- NoSSD is the default mode, so most qtest cases in hw/femu/tests/qtest/femu-test.c run on it through the default device options. Cases named for it include io-fuzz-nossd, a fuzzer over I/O command fields, and the namespace management cases ns-retire-nossd, ns-shared-remove-nossd and ns-shared-retire-nossd. The poller cases poller-burst, poller-burst-per-queue, poller-burst-sharded and shared-cq-pollers run on NoSSD because it is the default.
- The documentation check starts each NoSSD example and writes and reads back one block.
- In a guest, femu-test.sh runs its block checks on a NoSSD namespace.

4.3.10 Limits

- No latency model, so results say nothing about SSD behaviour.
- Throughput is bounded by poller CPU and by how the host schedules the pollers and vCPUs, which varies from run to run unless they are pinned.
- Host memory equal to devsz_mb is allocated at realize. If the allocation fails, QEMU aborts. If it succeeds but cannot be pinned, the pages are faulted in as they are used, and a host short of memory may run out then instead.
- Data is not kept across QEMU restarts.

Refusal messages are listed in the NoSSD guide (§4.2.5).

4.3.11 Extending the mode

- **A fixed device latency.** Add a constant to req->expire_time in nop_io_cmd() and run with hiops_inline=off; the completion step then holds the request until it is due.
- **Another I/O command.** Add a case to nop_io_cmd() for a NoSSD-only command, or to nvme_io_cmd() in hw/femu/nvme-io.c when every block mode should get it.
- **Starting point for a new mode.** hw/femu/nossd/nop.c is the smallest complete mode. Copy it and register it in nvme_register_extensions() in hw/femu/femu.c. Then add the new femu_mode value to the enum in hw/femu/nvme.h, raise the bound in the realize check (nvme_check_constraints(), which

refuses values above the KV mode), teach `backend_rw()` in `hw/femu/backend/dram.c` the mode (it asserts on an unknown one), add a token to `nvme_mode_from_token()` for `namespace_modes`, and add a `modes.py` entry.

4.3.12 Source map

File	Contents
`hw/femu/nossd/nop.c`[28]	the mode: handler table, Read and Write dispatch, model string
`hw/femu/nvme-io.c`[29]	`nvme_process_sq_io()`, `nvme_io_cmd()`, `nvme_rw()`, `nvme_process_cq_cpl()`, `nvme_poller()`
`hw/femu/backend/dram.c`[30]	the memory backend and `backend_rw()`
`hw/femu/femu.c`[31]	mode registration, FTL thread decision

4.3.13 Related pages

- NoSSD guide (§4.2): launching, fio examples, troubleshooting
- Architecture: pollers (§3.1.3)
- Timing model: host link and controller firmware (§6.2.9)

4.4 BlackBox SSD (BBSSD)

BlackBox mode (`femu_mode=1`) emulates a conventional NVMe SSD: the device runs its own flash translation layer (FTL), garbage collection (GC) and NAND timing, and the guest sees an ordinary block device. Use it to study how a host workload behaves on a real-looking SSD: latency under GC, write amplification, the effect of over-provisioning, caches and write buffers.

If you want a drive with no media timing at all, use NoSSD (§4.2). If you want the host to run the FTL, use OCSSD (§4.9) or ZNS (§4.7).

4.4.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). Any guest kernel with the NVMe driver works.
- Host memory: the whole namespace lives in host DRAM. `run-blackbox.sh` creates a 12 GiB device, so it needs about 17 GiB of free RAM (memory planning (§2.1.1)).
- Guest tools: `nvme-cli` and `fio`, both in the image from `make-guest-image.sh`.

4.4.2 Launch

From `build-femu/`, with a guest image in place:

```
./run-blackbox.sh
```

The script prints the `-device` line it builds. It is equivalent to:

[28] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nossd/nop.c
[29] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-io.c
[30] https://github.com/MoatLab/FEMU/blob/master/hw/femu/backend/dram.c
[31] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu.c

```
-device femu,devsz_mb=12288,namespaces=1,femu_mode=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,
    pls_per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,ch_xfer_lat=0,gc_
    thres_pcent=75,gc_thres_pcent_high=95
```

To change the geometry or latency, edit the variables at the top of `run-blackbox.sh`. Its console output goes to `build-femu/log`.

4.4.3 Configuration

Every property below is listed with its default and range in the property reference (§A.1). This section says which ones matter for BlackBox and how they interact.

Capacity and geometry

Properties: mode and capacity (§A.1.1), NAND geometry (§A.1.1).

The NAND geometry is `nchs` channels, each with `luns_per_ch` LUNs, each with `pls_per_lun` planes of `blks_per_pl` blocks of `pgs_per_blk` pages. A page is `secs_per_pg` sectors of `secsz` bytes. The raw NAND capacity is the product of all seven. With the defaults it is 16 GiB (8 x 8 x 1 x 256 x 256 x 8 x 512 = 17,179,869,184 bytes), and a page is 4 KiB.

The namespace the guest sees is `devsz_mb` MiB, which can be smaller than the raw NAND. The rest is spare area for GC. FEMU refuses a namespace that leaves GC no free lines (see Limits and refusals (§4.4.5)). A line is one block on every plane of every LUN, so there are `blks_per_pl` lines.

To state the over-provisioning directly, set `op_pcent`. FEMU then backs the device with the full raw NAND and exposes `raw / (1 + op_pcent/100)`, and `devsz_mb` is ignored. This example has 512 MiB of NAND and exposes about 410 MiB:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25
```

NAND timing

Properties: NAND timing (§A.1.1). How the time is computed: timing model (§6.2.3).

- `pg_rd_lat`, `pg_wr_lat` and `blk_er_lat` set flat read, program and erase times in nanoseconds.
- `nand_cell_type` (1 SLC, 2 MLC, 3 TLC, 4 QLC) replaces the flat times with built-in per-page-type timing:

```
-device femu,devsz_mb=1024,femu_mode=1,nand_cell_type=3
```

- `cmd_addr_lat`, `pg_xfer_lat` (or `ch_xfer_lat`) and `status_lat` add a shared channel bus. It is modelled only when one of them is non-zero.
- `pe_suspend` and `tsusp_ns` let a read suspend a program or erase on its LUN.
- `trim_lat_ns` charges each Dataset Management deallocate range.

Garbage collection

Properties: garbage collection, mapping and caches (§A.1.1).

Background GC starts when the share of lines in use reaches `gc_thres_pcent` (75). Foreground GC runs inside writes from `gc_thres_pcent_high` (95). `gc_policy` picks the victim line:

- `greedy` (the default): the line with the fewest valid pages.
- `random`: a random line among the GC candidates.

- `cost-benefit`: the line with the largest age x (1 - u) / 2u, where u is the share of valid pages.
- `fifo`: the oldest closed line.
- `d-choice`: samples 4 candidate lines at random (a line can be drawn twice) and takes the one with the fewest valid pages.

`random` and `d-choice` draw from a generator seeded by `gc_seed` (default 0), so the same configuration and workload pick the same victims and give the same write amplification on every run. Set a different `gc_seed` per run to vary them.

Mapping and caches

Properties: garbage collection, mapping and caches (§A.1.1).

- `mapping`: `page` (default, a full table in DRAM), `dftl` (a cached table of `mapping_cache_mb` MiB whose misses cost NAND reads), `hybrid` or `fast` (log-block schemes).
- `read_cache_mb` and `cache_evict` add a DRAM read cache. It models timing only: a hit costs DRAM time and skips the NAND read, but the cache holds no data, so NAND stays the source of truth. `cache_evict` is `clock` (the default), `random`, `lru` or `arc` (a scan-resistant 2Q variant, not full ARC).

The two log-block schemes follow published designs:

`mapping`	Model
`hybrid`	BAST log-block mapping (Kim 2002): one log block per data block, merged when that log block fills or the log pool runs out
`fast`	FAST log-block mapping (Lee et al. 2007): a sequential log block plus a shared, fully associative pool for random writes

Both suit some workloads more than others: sequential overwrites merge cheaply, random overwrites force full merges. A merge is charged to the NAND timeline and counted as relocated pages, so it shows in latency and in the write amplification factor.

- `hot_cold_sep` writes overwrites of mapped pages to separate lines.

Write buffer and power loss

Properties: garbage collection, mapping and caches (§A.1.1), namespace management, streams and power loss (§A.1.1).

`buffer_size` is the DRAM write buffer capacity in NAND pages, not bytes. Once the buffer is `buffer_thres_pcent` full, the next write programs a batch of the least recently written pages. A write the buffer absorbs costs no NAND time; the cost moves to the write that evicts it. Flush drains the buffer. Set `vwc=1` so that the guest sees a volatile write cache: Linux then sends Flush, and the guest can turn the buffer off with feature 06h. A read of a page the buffer still holds costs no NAND time, and deallocating such a page drops it instead of writing it out later.

```
-device femu,devsz_mb=1024,femu_mode=1,buffer_size=1024,vwc=1
```

`power_loss=on` makes the buffer hold data, and the QOM property `simulate-power-loss` drops what it holds (runtime properties (§A.2.1)).

Wear, retention and errors

Properties: reliability and wear (§A.1.1).

All of these are off by default.

- `ecc_step_ns` and `ecc_retention_sec` make reads of worn or old blocks slower.
- `pe_cycles_rated` and `nand_bad_blocks` feed SMART Percentage Used and Available Spare.
- Log page C0h counts plane reads, programs and erases. With `energy_read_nj`, `energy_prog_nj` and `energy_erase_nj` set from a part's datasheet, it also reports their energy in uJ. With `thermal_tau_ms`, `thermal_r` and `idle_mw` too, that energy heats a first-order thermal model, and the SMART temperature follows it (thermal model (§B.5)).
- `wl_spread` turns on static wear levelling. When the lines in service differ by more than that many erases, the least worn full line moves into the most worn free line, so cold data rests on worn blocks and young blocks rejoin the rotation. It runs only when the data write pointer has just taken an empty line, which it exchanges for the worn one, so it adds no write pointer, and it copies at most a quarter of what the host writes.
- `gc_pace` collects the victim line a few pages per host page written, in proportion to its valid pages, instead of in one pass. A write then waits for about one copy instead of a whole line. The forced pass stays as the last resort. It is refused where `blk_pe_limit` is.
- `gc_copyback` (off by default) moves collected pages on their die, with no data on the channel, and works with FDP too. A copyback skips ECC: on a real SSD its bit errors are not corrected and can add up. FEMU models no bit errors, so only timing changes; pages with extra ECC time and refresh moves stay off-chip (design (§3.4.6)).
- `age_scale` makes data age faster than wall time for `retention_limit_sec` and `ecc_retention_sec`, so a study of months of retention runs in minutes. I/O timing and collection order stay as they are.
- `blk_pe_limit` gives each block an erase limit (`blk_pe_spread` varies it per block, from `blk_pe_seed`). When a line's erase takes a block to its limit, the line leaves service if enough lines remain: the namespace's lines, the free lines forced collection keeps, and two more.
- With `eol_read_only` (on by default), the media goes read-only when no block can replace a worn-out one. A real SSD does the same at the end of its life. Reads go on. The writes in flight at that point finish from the reserve lines. The SMART log shows critical warning bit 3 and Available Spare 0; see end of life (§B.4).
- In read-only state, Write, Write Zeroes, Write Uncorrectable, Copy and a deallocating Dataset Management change nothing. They fail with Attempted Write to Read Only Range (182h). Flush and Dataset Management without deallocation succeed. Format NVM and Sanitize deallocate the data, and the media stays read-only.
- With `eol_read_only=off`, a worn-out block that cannot leave service stays in service and sets the SMART reliability warning (critical warning bit 2). Retirement also needs an open line and a free line then, and wear makes no write fail.
- `spare_lines` holds lines back as spare blocks. A worn-out block is first replaced by a spare of its plane, when every plane with a worn-out block in the line has one. With `eol_read_only`, a pool that cannot replace it is the end of life; with it off, the line retires as without spare lines. SMART Available Spare is then the spare blocks the emptiest plane has left; without spare lines, it is the lines retirement can still take.

- With `blk_pe_limit`, `eol_read_only` (on by default) keeps reserve lines erased for the writes in flight at the end of life. Before a write-class command changes data, it takes credits for the NAND pages it may program. It waits in its queue while the reserve cannot take them. So one command needs a bound: `mdts` must be 1 or more. Write Zeroes is limited to MDTS (WZSL, with ONCS bit 3 clear). Copy is limited to MDTS (MCL), or to one block of the largest LBA format when that is larger.
- With `mapping=dftl`, read-only media opens no new translation page line. The reserve stays for the writes in flight. Their mapping changes stay in the CMT or pending, in power-loss-protected DRAM.
- The controller raises a SMART asynchronous event each time one of these starts, if the host enabled its bit of Asynchronous Event Configuration:

Condition	AEC bit	Information
the spare falls below its threshold	0	02h
a block stays in service past its limit while none did	2	00h
the media goes read-only	3	00h

- The read-only end of life covers bbssd and CSD (the NVM namespace).
- FDP takes `blk_pe_limit` too. A worn-out block retires its reclaim unit while the unit's group keeps the units the namespaces and placement need. Otherwise the media goes read-only (`eol_read_only`), or the block stays in service with critical warning bit 2 (`eol_read_only=off`). Each reclaim group then holds its own reserve units, and RUH Update fails after the end of life (FDP wear-out (§4.6.8)).
- ZNS ends the life of a zone with the Read Only and Offline zone states. This model does not drive them. KV has no NVM write path. Both refuse `blk_pe_limit`.
- With `blk_pe_limit`, SMART Percentage Used counts erases against the sum of the blocks' limits. `query-femu` shows retired and spare lines with their own states.
- `err_read_unc_ppm` and `err_write_fail_ppm` fail a fixed share of reads or writes. The failures come at a fixed period, so a run repeats exactly.
- `read_reclaim_limit` and `retention_limit_sec` rewrite lines that were read too often or hold old data. The line is picked on a read and rewritten on a following write, so a workload that never writes never triggers it.

This small device wears out after a few thousand writes and then goes read-only. Its 16 KiB MDTS keeps the reserve at one line:

```
-device femu,femu_mode=1,devsz_mb=1,secsz=512,secs_per_pg=8,pgs_per_blk=4,blks_per_pl=40,luns_per_ch=2,nchs=2,blk_pe_limit=8,spare_lines=8,mdts=2
```

Both refresh knobs act only when something reads the line. A region nothing reads is never refreshed: modelling a background media scan would need a timer that FEMU does not run.

`read_reclaim_limit` is the number of reads a block may take before its line is refreshed:

```
-device femu,devsz_mb=1024,femu_mode=1,read_reclaim_limit=100000
```

The line is chosen on the read but rewritten on the next write, where relocation already costs something, so a read never waits behind a whole line. One line is queued at a time and at most one is refreshed per write, so the rate follows how often the host writes, not how hard it reads. The cost shows up as write amplification. On a 512 MiB region read six times over with a low limit, 5 lines were refreshed and 77824 pages relocated, and the write amplification factor went from 1.000 to 1.542. Lowering the limit from 500 to 10 moved it only

from 1.542 to 1.628. A workload that only reads never refreshes anything, where a real drive would do it in the background, so model read-only ageing some other way.

Host link and controller firmware

Properties: host link and controller firmware (§A.1.1).

`pcie_bandwidth_mbps`, `pcie_prop_delay_ns` and `fw_cpu_ns` add link and firmware time to every Read and Write. See the timing model (§6.2.9).

Features on top of BlackBox

- Flexible Data Placement (§4.5) (`femu-subsys,fdp=on`).
- Several namespaces (§4.15), each with its own FTL.
- Namespace management, metadata and protection information (§4.17).
- Streams (`streams=on`), which needs `mapping page` or `dftl`.

4.4.4 Use it from the guest

These commands run inside the guest. From the host, prefix each with `./run-guest-ssh.sh` as in the quick start (§2.4).

Check the device:

```
sudo nvme list
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(mn|sn) '
```

The model is `FEMU BlackBox-SSD Controller` and the serial number starts with `vSSD`.

Write before you read. A read of a page that was never written costs no NAND time, so a read benchmark on a fresh device measures nothing. Fill the range first, then read it:

```
sudo fio --name=fill --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=write --bs=128k --iodepth=16 --size=4G
sudo fio --name=rr --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randread --bs=4k --iodepth=1 --size=4G --runtime=30 --time_based
```

With an idle device and queue depth 1, the completion latency (`clat`) is about `pg_rd_lat` plus guest and poller overhead.

To make GC run, write more than the free lines hold. GC starts when 75% of the lines are in use, so on the default 16 GiB of NAND that takes about 12 GiB of writes, one pass over the 12 GiB namespace. Random writes over the namespace several times keep GC busy:

```
sudo fio --name=age --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=32 --loops=5
```

Read the write amplification factor and media counters from the vendor log page C0h. The first four bytes are the WAF times 1000, bytes 4 to 7 are reserved, and the three 8-byte counters from byte 8 are host pages written, pages GC moved, and pages programmed:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
```

log-pages-and-counters.md (§B.1) lists every field. Wear and host totals are in the SMART log:

```
sudo nvme smart-log /dev/nvme0
```

The vendor admin command 0xEF switches GC time and the NAND times on or off while the guest runs, for example to fill a device quickly:

```
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=4   # NAND times to 0
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=3   # back to 40 us, 200 us, 2 ms
```

Code 3 restores the built-in times, not the ones on your command line. Codes 3 and 4 change only the flat times, so they have no effect with `nand_cell_type` set. The timing model (§6.2.11) lists every code.

4.4.5 Limits and refusals

FEMU checks the configuration at realize and stops QEMU with a message. The common ones:

Message	Cause and fix
`FEMU bbssd: namespace 1 exposes 1024 MiB of the 1024 MiB this geometry has, leaving garbage collection no room; expose at most 960 MiB per namespace (lower devsz_mb, or set op_pcent)`	`devsz_mb` (or the namespace size) does not leave the free lines GC needs. Lower `devsz_mb`, grow the geometry, or set `op_pcent`.
`FEMU bbssd: the geometry has only N lines, fewer than the M garbage collection needs free`	`blks_per_pl` is too small for the reserve that `gc_thres_pcent_high`, `hot_cold_sep`, Streams and log-block mapping add up to.
`FEMU bbssd: nchs must be greater than 0, got 0`	A geometry property is 0 or negative. The same message names any of them.
`FEMU bbssd: the geometry describes N sectors, which exceeds the 2147483647 the FTL can address; ...`	The NAND is too large; reduce one of the geometry properties.
`FEMU bbssd: unknown mapping "x", unknown gc_policy "x", unknown cache_evict "x"; ...`	A misspelled policy name.

If the host cannot allocate `devsz_mb` (or, with `op_pcent`, the raw NAND) of memory, QEMU aborts in GLib with `failed to allocate N bytes` and a core dump.

Reserve per namespace: `(1 - gc_thres_pcent_high/100) * blks_per_pl` lines, rounded down, plus one for the write pointer, one more with `hot_cold_sep`, one more with `mapping=hybrid` or `fast`, and `streams.max + 1` with Streams. The namespace must fit in the remaining lines.

4.4.6 Verify

1. `sudo nvme list` in the guest shows `/dev/nvme0n1` with model `FEMU BlackBox-SSD Controller` and the size you set.
2. After a 4 KiB random write run, the C0h WAF reads 1000 or more and the host page counter (byte 8) equals the pages fio wrote.
3. A queue depth 1 random read of written data takes about `pg_rd_lat`.

4.4.7 Troubleshooting

- **Reads are faster than the configured latency.** The range was never written. Unwritten pages have no mapping and cost no NAND time. Fill the range first.
- **The WAF stays at 1.000.** GC has not run, or found only lines with no valid pages, as after a sequential overwrite. GC starts after about 12 GiB of writes on the default geometry. Run random writes over the namespace more than once, or use `op_pcent` to make the namespace a known fraction of the NAND.

- **`Property 'femu.cell_type' not found`, or `flash_type` has no effect.** The BlackBox cell type is `nand_cell_type`. `flash_type` belongs to OCSSD.
- **QEMU aborts with `failed to allocate`.** The device is in host DRAM. Lower `devsz_mb` or free host memory.
- **Latency is higher than the model.** Completions are posted by the poller, which needs a host core. Give each `femu-poller` thread and the `FEMU-FTL-Thread` a core of its own (timing model (§6.2.1)).
- **Data is gone after QEMU exits.** FEMU keeps the device only in host memory and writes nothing to a file. A guest reboot keeps the data.

Related issues: #16, #50, #52, #92, #113, #130, #132, #137.

4.4.8 Related pages

- Timing model (§6.2)
- Architecture: the BBSSD FTL (§3.1.5)
- Quick start (§2.4), which runs this mode end to end
- Device property reference (§A.1)

4.5 Flexible Data Placement (FDP)

Flexible Data Placement lets the host tell the SSD which writes belong together. The device groups its NAND into reclaim units and offers a few reclaim unit handles. Each write may carry a placement identifier that names a handle, and the device puts all writes of one handle into the same open reclaim unit. When data with similar lifetimes shares a reclaim unit, GC finds it mostly invalid and copies less, so write amplification drops.

FDP is not a `femu_mode`. It is a property of an NVMe subsystem, `femu-subsys`, which a BlackBox (§4.4) controller joins. In FEMU a reclaim unit is one superblock of the BlackBox FTL. With one reclaim group it is a whole line. With more, it holds the blocks of the dies of its group. FEMU's FDP support comes from WARP; if you use it, cite WARP as well as FEMU (citation (§4.5.8)).

Use it to measure how placement hints from an application, a file system or fio change write amplification and tail latency.

4.5.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). Any guest kernel with the NVMe driver sees the device.
- Placement hints: the guest has to send them. Ordinary writes through the block device carry no placement identifier unless the guest kernel adds one, so use passthrough commands (`nvme write` with a directive), or fio's `io_uring_cmd` engine on the generic device `/dev/ng0n1`.
- Guest tools: nvme-cli with the `fdp` commands (2.x), and fio built with `io_uring_cmd` support for placement from fio.

4.5.2 Launch

From `build-femu/`:

```
./run-blackbox-fdp.sh
```

The script builds two devices. The subsystem must come first on the command line, because the controller names it with `subsys=`:

```
-device femu-subsys,id=femu-subsys-0,nqn=subsys0,fdp=on,fdp.nruh=4,fdp.nrg=1,fdp.nru=256 -device
    femu,devsz_mb=12288,namespaces=1,femu_mode=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,
    pls_per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,ch_xfer_lat=0,gc_
    thres_pcent=75,gc_thres_pcent_high=95,subsys=femu-subsys-0
```

The shortest form, with the default 64 MiB reclaim units and 128 of them:

```
-device femu-subsys,id=fdp0,fdp=on,fdp.nruh=4 -device femu,devsz_mb=1024,femu_mode=1,subsys=fdp0
```

`run-blackbox-fdp.sh` writes its console output to `/tmp/femu-fdp.log`.

4.5.3 Configuration

Subsystem

Properties: Flexible Data Placement (§A.1.2).

- `fdp=on` turns FDP on for the controller that joins the subsystem. An FDP subsystem takes a single controller.
- `fdp.nruh`: number of reclaim unit handles, that is, placement handles the host can name. Required, at least 1.
- `fdp.nru`: reclaim units per reclaim group (default 128). FEMU uses at most one per superblock, so the effective count is the smaller of `fdp.nru` and `blks_per_pl`. It needs at least `2 * fdp.nruh + 1` of them: one open unit per handle, one spare per handle for GC, and one more. The namespace gets what is left once each handle has a unit open, each Persistently Isolated handle one to collect into, and forced GC its free units (at least one unless `gc_thres_pcent_high` is 100); a larger `devsz_mb` is refused. See the reserve (§4.6.3).
- `fdp.nrg`: reclaim groups (default 1, at most 128). With BlackBox each group is a set of dies; see Reclaim groups (§4.5.3).
- `fdp.runs`: reclaim unit size in bytes. Leave it at 0, or set it to the size of one superblock of a group: `nchs * luns_per_ch * pls_per_lun * pgs_per_blk * secs_per_pg * secsz / fdp.nrg` (64 MiB with the default geometry and one group).
- `fdp.isolation_mode`: 0 makes every handle Persistently Isolated; any other value makes the last handle Initially Isolated.

Controller

The controller is an ordinary BlackBox controller with `subsys=<id>`, so the BlackBox geometry, timing and GC properties (§4.4.3) apply. Two properties are specific to FDP (garbage collection, mapping and caches (§A.1.1)):

- `gc_strategy`: how GC picks a victim reclaim unit: 0 greedy (default), 1 cost-benefit, 2 random, 4 per-handle. The random strategy draws from a generator seeded by `gc_seed`, so it repeats run to run.
- `gc_policy`: the same choice by name. `cost-benefit` and `random` are strategies 1 and 2; `fifo` and `d-choice` exist only by name. A name other than `greedy` needs `gc_strategy=0`. `d-choice` draws from the same generator.
- `buffer_size`, `buffer_thres_pcent`, `vwc`: the write buffer holds placed writes. Each held page goes to the unit of the handle and group that last wrote it. RUH Update first writes back the held pages of the handles it names (design (§4.6.11)).
- `mapping=dftl`: a placed write charges the translation page of each page it programs.
- `fdp_trim_erase_all`: non-zero makes a deallocate reset every reclaim unit instead of the given ranges.

`FEMU_FDP_DEBUG` in QEMU's environment prints placement and reclaim traces (environment variables (§A.1.5)).

Reclaim groups

A reclaim group is a set of dies that work in parallel. Each handle has an open reclaim unit in every group. A write names the group in the top bits of its placement identifier (Base 2.3, Figure 290). Collection moves data only inside its group. The groups do not share dies, so the data of one group never waits on the erases of another.

```
dies in the order ch x luns_per_ch + lun; group g = the g-th of nrg runs

nchs=2, luns_per_ch=2, nrg=2          nchs=2, luns_per_ch=4, nrg=4
       LUN0 LUN1                              LUN0 LUN1 LUN2 LUN3
 ch0  [ g0 ][ g0 ]                     ch0  [ g0 ][ g0 ][ g1 ][ g1 ]
 ch1  [ g1 ][ g1 ]                     ch1  [ g2 ][ g2 ][ g3 ][ g3 ]

PID = RGID << (16 - RGIF) | placement handle
RGIF = the bits that hold nrg - 1 (1 for 2 groups, 2 for 3 or 4 groups)
```

```
-device femu-subsys,id=fdp0,fdp=on,fdp.nruh=4,fdp.nrg=2 -device
    femu,devsz_mb=16,femu_mode=1,nchs=2,luns_per_ch=2,blks_per_pl=64,pgs_per_blk=64,lba_index=3,subsys=fdp0
```

With BlackBox, the rules are:

- A group is whole channels or LUNs of one channel: `fdp.nrg` divides `nchs`, or is a multiple of `nchs` that divides `nchs * luns_per_ch`.
- A reclaim unit is one block of each plane of each die of its group. So RUNS is a line divided by `fdp.nrg`. Each group has a unit per block index, up to `fdp.nru`.
- A logical block is whole NAND pages (`lba_index=3` with 4 KiB pages). The FTL maps whole pages. A write of part of a page would move the rest of the page into the group of the write. A Format to a smaller block fails with Invalid Format.
- Each group must hold the whole namespace. A host may send all of its data to one group, and a write within the namespace capacity must not fail. So a device with `fdp.nrg` groups needs about `fdp.nrg` times the namespace in NAND.

A write without a directive goes through placement handle 0. So does a write with an invalid identifier. It goes to the group with the most free reclaim units, and groups with the same count take turns.

RUH Status lists a descriptor for each placement handle and group, by handle and then by group. A RUH Update names one pair and moves only its unit. Events carry the group of the unit they are about. `query-femu`[32] lists each group with its dies and free units. NoSSD takes any `fdp.nrg` up to 128.

Several namespaces

Reclaim units and handles belong to the endurance group, so namespaces of one FDP controller share them. Each namespace has its own placement handle list (PHL): placement handle `i` of the namespace selects reclaim unit handle `phl[i]`. Two namespaces may list the same handle; their writes then go to the same reclaim unit.

```
namespace_phls=0:1,,2:1
ns 1: PH 0 -> RUH 0, PH 1 -> RUH 1  --+
ns 2: PH 0 -> RUH 2, PH 1 -> RUH 1  --+--> RUH 1 is shared; RUH 3 is unused
```

```
-device femu-subsys,id=fdp0,fdp=on,fdp.nruh=4 -device femu,devsz_mb=1024,femu_mode=1,subsys=fdp0,namespaces=2,namespace_phls=0:1,,2:1
```

- `namespace_phls`: one entry per namespace, separated by commas (`,` , on the QEMU command line). An entry lists handle IDs separated by `:`. Each entry has 1 to `MIN(fdp.nruh, 128)` IDs, each below `fdp.`

[32] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md`

nruh, none twice. Unset, every namespace gets every handle in order.

- Every namespace is BlackBox, or every one is NoSSD. BlackBox namespaces share one FTL, each in its own slice of the logical pages. Thus a namespace size must be a whole number of NAND pages. The sum of the sizes must fit in the reclaim units, as one namespace must.
- `nvme fdp status /dev/nvme0n2` lists the placement identifiers of namespace 2 with the handle each selects. `nvme fdp usage` reports a handle no list names as unused (RUHA 0).
- Statistics (log 22h) and C0h count the endurance group: a shared handle adds the writes of every namespace that uses it.
- Namespaces that share a handle keep one format. A Format of one of them to another LBA format fails with Invalid Format; format them together with NSID FFFFFFFFh. Namespaces with no shared handle may differ, and RUAMW counts the blocks of each handle's namespaces.
- Event 80h names the namespace of the blocks it counts. It carries the placement identifier of that namespace; a collection that moves blocks of two namespaces logs one 80h for each run of them. Events without moved data (01h, and 81h of a refresh) name the first namespace that lists the handle.
- The write buffer (`buffer_size`) belongs to the shared FTL. A Flush of one namespace writes back the held pages of every namespace, which the specification allows. A RUH Update first writes back the held pages of the handles it names, of every namespace.
- NoSSD namespaces count their writes as NoSSD namespaces (§4.5.3) says.

NoSSD namespaces

A NoSSD namespace has no NAND model. It still reports the FDP counters of its writes. The rules are those of the FDP Statistics log (Base 2.3, Figure 293, and NVM Command Set 1.2, 4.1.4.6):

Command	HBMW	MBMW and RUAMW
Write, Copy, Write Zeroes without Deallocate	bytes of the blocks	the same bytes; RUAMW drops by the blocks
Write Zeroes with Deallocate, Write Uncorrectable	bytes of the blocks	unchanged: no data is stored

MBE stays 0, because nothing is erased. A unit holds `fdp.runs` bytes (96 MiB unless set). A write that reaches the end of the unit leaves the handle on a fresh unit. A write longer than the room left goes on in the next unit.

RUH Update of a part-written unit logs event 00h and gives the handle a fresh unit; an empty unit stays, with no event. A write with an invalid placement identifier logs event 03h. The namespace's logical block must fit in `fdp.runs`.

4.5.4 Use it from the guest

Check that the controller supports FDP. Bit 19 (0x80000) of CTRATT is FDP:

```
sudo nvme id-ctrl /dev/nvme0 | grep ctratt
```

Read the configuration, handle usage, statistics and events of endurance group 1:

```
sudo nvme fdp configs /dev/nvme0 -e 1
sudo nvme fdp usage /dev/nvme0 -e 1
sudo nvme fdp stats /dev/nvme0 -e 1
sudo nvme fdp events /dev/nvme0 -e 1 -E
sudo nvme fdp status /dev/nvme0n1
```

`configs` lists the handles and the reclaim unit size, `usage` gives each handle's attributes, `stats` gives host and media bytes written, and `status` lists the placement identifiers the namespace accepts with the space left in each one's reclaim unit (RUAMW). Every page programmed into the unit counts. This includes pages that collection moves into the unit of an Initially Isolated handle. A Format does not give the room back. By default every handle backs a placement handle of the namespace, so `usage` reports each one as host specified:

```
Reclaim Unit Handle 0 Attributes: 0x1 (Host Specified)
Reclaim Unit Handle 1 Attributes: 0x1 (Host Specified)
Reclaim Unit Handle 2 Attributes: 0x1 (Host Specified)
Reclaim Unit Handle 3 Attributes: 0x1 (Host Specified)
```

`events -E` shows host events, such as a write with an invalid placement identifier; every event type is enabled by default. Without `-E`, `events` shows controller events. When collection or a read refresh moves data of the Initially Isolated handle, it logs 80h (Media Reallocated) with the number of blocks moved. When such a move gives that handle a new reclaim unit, it logs 81h (Implicitly Modified Reclaim Unit Handle).

Write with a placement identifier

A write with directive type 2 (data placement) uses the directive specific field as its placement identifier. This writes 8 blocks of 512 bytes at LBA 0 through handle 1:

```
head -c 4096 /dev/urandom > data.bin
sudo nvme write /dev/nvme0n1 -s 0 -c 7 -z 4096 -d data.bin -T 2 -S 1
```

A write with no directive goes through placement handle 0, in a reclaim group the controller selects. So does a write with an identifier the namespace does not have. An invalid identifier also logs host event 03h, with the group and handle the controller selected.

Enable the Data Placement Directive

The spec asks the host to enable the Data Placement Directive per namespace before it writes with directive type 2. FEMU reports the directive in the Identify directive and accepts the enable. It also places writes with directive type 2 when the host did not enable the directive, because the Linux driver does not enable it.

```
# enable (Directive Send, Identify directive, Enable Directive, type 2)
sudo nvme dir-send /dev/nvme0n1 -n 1 -D 0 -O 1 -T 2 -e 1
# check: Data Placement is supported, enabled and kept across a reset
sudo nvme dir-receive /dev/nvme0n1 -n 1 -D 0 -O 1 -H
```

```
 Directive Receive, Identify (bytes 0, 32, 64; bit 2 = Data Placement)
   supported  01h | 04h        enabled  01h, or 05h after the enable
   kept across a controller reset: 04h
 Directive Send, enable type 2:  nsid FFFFFFFFh -> Invalid Namespace or Format
                                 no FDP          -> FDP Disabled
 any command with directive type 2 -> Invalid Field (it has no operations)
 enabled: Write, Write Zeroes or Copy naming a type other than 0 or 2 -> Invalid Field
```

fio places data from its `io_uring_cmd` engine. It reads the placement identifier list from the device and cycles through the entries whose indexes are listed in `fdp_pli`:

```
sudo fio --name=fdp --filename=/dev/ng0n1 --ioengine=io_uring_cmd --cmd_type=nvme \
    --fdp=1 --fdp_pli=0,1,2,3 --rw=randwrite --bs=4k --iodepth=16 --size=4G
```

Measure the effect

The vendor log page C0h reports the write amplification factor for FDP as for plain BlackBox. Compare a run with placement against the same run without it:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
```

The QMP command `query-femu`[33] reads the same state from the host, with no guest tools. It shows the bytes that each reclaim unit handle wrote and moved, and the state of each reclaim unit:

```
{"execute": "query-femu", "arguments": {"kind": "reclaim-units"}}
```

4.5.5 Limits and refusals

Message	Cause and fix
`fdp.nruh must be non-zero`	Set `fdp.nruh`.
`fdp.nruh (200) must not exceed fdp.nru (128)`	More handles than reclaim units.
`fdp.nrg (129) must not exceed 128`	Lower `fdp.nrg`.
`fdp.nruh (16367) must not exceed 16366: the size of an FDP configuration descriptor is 16 bits`	Lower `fdp.nruh`.
`FEMU bbssd: fdp.nrg (3) must divide nchs (2), or be a multiple of nchs that divides the 4 dies of nchs x luns_per_ch: ...`	The groups would not be whole channels or LUNs of one channel.
`FEMU bbssd: fdp.nrg above 1 needs a logical block of whole NAND pages (4096 bytes); namespace 1 has 512-byte blocks: set lba_index`	Set `lba_index=3` with 4 KiB pages.
`FEMU bbssd: a reclaim unit (16384 bytes) must hold a logical block; namespace 1 has 32768-byte blocks: lower lba_index or fdp.nrg`	A unit of a group is smaller than a block.
`FEMU bbssd: a reclaim group must hold every namespace, since a host may place all of its data in one: ...`	Lower `devsz_mb` or `fdp.nrg`, or add NAND.
`fdp.runs (1024) is smaller than a logical block of namespace 1 (4096 bytes)`	Raise `fdp.runs` (NoSSD) or lower the block size.
`FDP needs namespaces of the NVM command set; namespace 1 is zoned (znssd)`	ZNS, and the same for OCSSD.
`FEMU bbssd: placement needs N reclaim units in each group for M handles and this geometry gives K; raise blks_per_pl or fdp.nru, or lower fdp.nruh`	Too few reclaim units for the handles.
`FEMU bbssd: namespace 1 exposes 1024 KiB, but placement keeps 9 of the 24 reclaim units for 4 handles and collection, leaving 960 KiB; lower devsz_mb or fdp.nruh, raise blks_per_pl, or raise op_pcent`	The namespace does not fit in what placement leaves.
`FEMU bbssd: fdp.runs must be 67108864, the size of one superblock of the dies of a reclaim group of this geometry, or unset`	`fdp.runs` does not match the geometry.
`FDP with more than one namespace needs every namespace in bbssd mode or every one in nossd mode`	Namespaces of other modes, or of mixed modes, with `namespaces` above 1.
`FDP with more than one namespace needs namespace sizes in whole NAND pages (4096 bytes); namespace 1 has 526336 bytes`	Set `namespace_sizes` to multiples of the page size.

[33] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md`

Message	Cause and fix
`fdp_trim_erase_all erases every namespace; it needs a single namespace`	The test knob with more than one namespace.
`FEMU bbssd: the 2 namespaces expose 2048 KiB, but placement keeps 9 of the 25 reclaim units ...`	The namespaces together do not fit in the reclaim units.
`namespace_phls needs fdp=on on the subsystem`	Placement handle lists without FDP.
`namespace_phls count must equal namespaces=2`	One entry per namespace.
`namespace_phls: namespace 1 names handle '4'; handles are 0 to 3`	An ID that is not a handle.
`namespace_phls: namespace 1 lists handle 0 twice`	An ID twice in one list.
`namespace_phls: namespace 2 lists no handle`	An empty entry.
`femu-subsys with fdp=on takes a single controller`	A second controller joins the FDP subsystem.
`FEMU bbssd: set gc_policy or gc_strategy under FDP, not both`	Both pick the reclaim unit strategy. Set one of them, or leave `gc_policy` at `greedy`.
`FEMU bbssd: hot_cold_sep is not supported under FDP: the placement handle of a write chooses its reclaim unit, ...`	The placement handle of a write names its reclaim unit, and the hot write pointer writes past it. Give hot data a placement handle of its own.
`FEMU bbssd: mapping=hybrid is not supported under FDP: the log-block mappings write and merge through lines that are not FDP-aware; use page or dftl`	The same for `mapping=fast`. The log-block mappings keep their own log and data lines. A CSD controller gets these messages with `FEMU csd:`.
`meta: not supported with placement (fdp)`	`meta` (with a valid `mc`) under FDP.
`streams requires bbssd or NoSSD with FDP disabled`	`streams=on` under FDP.
`the key-value command set and FDP cannot share a controller: ...`	A KV namespace under FDP.
`namespace management does not support FDP`	`ns_mgmt=on` together with `fdp=on` on the subsystem.
`Device 'fdp0' not found`	The controller comes before its subsystem on the command line.

NoSSD controllers accept an FDP subsystem and count its writes, with no media to place on. ZNS, OCSSD and KV are refused: an FDP endurance group holds namespaces of the NVM command set only (Base 2.3, 8.1.11). CSD goes through the BlackBox FDP write path, with the same refusals and reclaim unit size as BlackBox. Its memory and compute namespaces may sit next to FDP, because they are in no endurance group (SLM 1.2, 2.1.2; CP 1.2, 2.1.1).

When every reclaim unit is in use and GC cannot free one, a write fails with Capacity Exceeded and QEMU prints `ssd_stream_write: device full, no RU for ruh N`. Deallocating or overwriting data gives GC units to free again, and writes succeed once it has.

4.5.6 Verify

1. `ctratt` has bit 19 set, and `nvme fdp configs` shows `fdp.nruh` handles in `fdp.nrg` reclaim groups.
2. After writes with `-T 2 -S 1`, `nvme fdp status /dev/nvme0n1` shows the RUAMW of placement identifier 1 going down.
3. `nvme fdp stats` shows host bytes written growing with your writes.

4.5.7 Troubleshooting

- **`Property 'femu.fdp' not found`, or FDP does not show up.** FDP is a property of `femu-subsys`, not of `femu`. Create the subsystem first and join it with `subsys=`.
- **QEMU crashed in GC under a long write workload (`select_victim_ru`, `victim_ru_get_pri`, `No free RUs left`).** These were bugs in the reclaim unit victim queue, fixed on master. Update FEMU.
- **The WAF is the same with and without placement.** Check that the writes really carry directive type 2: block-device writes do not unless the guest kernel sets placement. Use `nvme write -T 2 -S N` or fio with `--ioengine=io_uring_cmd --fdp=1`.

Related issues: #153, #186, #189, #191.

4.5.8 Citation

FDP support in FEMU comes from WARP, the FDP SSD emulator described in Characterizing and Emulating FDP SSDs with WARP[34] (FAST '26). If you use the FDP mode, please also cite:

```
@inproceedings{Song+26-WARP,
  author    = {Inho Song and Shoaib Asif Qazi and Javier Gonz{\'a}lez and
               Matias Bj{\o}rling and Sam H. Noh and Huaicheng Li},
  title     = {{Characterizing and Emulating FDP SSDs with WARP}},
  booktitle = {24th USENIX Conference on File and Storage Technologies (FAST 26)},
  pages     = {347--362},
  year      = {2026},
}
```

4.5.9 Related pages

- BlackBox SSD (§4.4)
- Choosing a mode: which features combine (§4.1.3)
- Architecture: features that are not modes (§3.1.4)

4.6 FDP: Flexible Data Placement

This chapter describes how FEMU implements NVMe Flexible Data Placement (FDP): how the subsystem builds reclaim groups, reclaim units and reclaim unit handles, how a write is placed, how reclaim units map onto the BlackBox FTL's NAND, how garbage collection works under placement, and what the FDP log pages and features report. To launch an FDP device and write with placement identifiers from a guest, read the FDP feature guide (§4.5) first. FEMU's FDP support comes from WARP (FAST '26); the guide has its citation (§4.5.8).

4.6.1 Purpose

FDP lets the host group writes it expects to die together. The device offers reclaim unit handles (RUHs); each write may name one through a placement identifier, and the device puts all writes of one handle into the same open reclaim unit (RU). When an RU holds data of similar lifetime, it is mostly invalid by the time garbage collection picks it, so GC copies less and write amplification drops. FEMU models the host-visible FDP interface and backs it with a placement-aware write and GC path in the BlackBox FTL, so you can measure that effect.

[34] https://www.usenix.org/conference/fast26/presentation/song

FDP is not a `femu_mode`. It is turned on with `fdp=on` on a `femu-subsys` device, and a controller joins the subsystem with `subsys=`. Only a BlackBox (bbssd) namespace has the placement data path. A NoSSD namespace counts the host side of FDP (NoSSD namespaces (§4.6.6)).

4.6.2 Place in the hierarchy

```
femu-subsys device, fdp=on              NVMe subsystem
  |  endurance group 1 (NvmeEnduranceGroup.fdp)
  |    configuration: nrg reclaim groups x nru reclaim units, nruh handles
  |    host-visible state: RUH type and attributes, RUAMW per RU,
  |    HBMW/MBMW/MBE, host and controller event rings
  |
  +-- femu device, femu_mode=1, subsys=   one controller
        |
        +-- namespace 2 ...                 its own handle list, the same FTL
        +-- namespace 1                     placement handles PH 0..nruh-1
              |                             (default: PH i -> RUH i)
              +-- struct ssd (BlackBox FTL)
                    FemuRuHandle[nruh]       per group: active RU and
                                             collection RU; counters
                    FemuReclaimGroup[nrg]    its dies, free list, full
                                             list, victim queues
                    FemuReclaimUnit[nrg][nru] one block index on the dies
                                             of its group, write pointer,
                                             valid pages
                    lines -> blocks on the channels, LUNs and planes of
                             the group (every die with one group)
```

Two layers of state exist side by side. The subsystem holds what the host sees (`NvmeRuHandle`, `NvmeReclaimUnit` with its RUAMW, the statistics and events) in `nvme.h`. The FTL holds what places and collects data (`FemuRuHandle`, `FemuReclaimGroup`, `FemuReclaimUnit` in `bbssd/ftl.h`). Each FTL object points at its host-visible twin, and the write path updates both.

4.6.3 Configuration

`nvme_subsys_setup_fdp()` in `femu.c` builds one endurance group (ID 1) with a single FDP configuration when the subsystem is realized:

Item	Value	Property
Reclaim groups (NRG)	1 to 128	`fdp.nrg`
Reclaim units per group	as set, at most 65536	`fdp.nru`
Reclaim unit handles (NRUH)	1 to `fdp.nru`, at most 16366 (the descriptor size of log 20h is 16 bits); must be set	`fdp.nruh`
Handle type (RUHT)	Persistently Isolated; with `fdp.isolation_mode` non-zero the last handle is Initially Isolated	`fdp.isolation_mode`
Reclaim unit size (RUNS)	one BlackBox superblock of the dies of a group (see below); the subsystem default of 96 MiB applies only when no bbssd controller overwrites it	`fdp.runs`
Reclaim group identifier format (RGIF)	the bits that hold `nrg - 1`: 0 with one group, 1 with two; the placement handle keeps the rest, at least enough for `MIN(nruh, 128)` handles	derived
MAXPIDS	`MIN(128, NRG x NRUH)`, reported zero-based, so below NRG x NRUH (Base 2.3, Figure 287)	derived

Item	Value	Property
Event filter	every supported event type enabled on every handle	Set Features 1Eh

With FDP on, a controller that joins runs `nvme_ns_init_fdp()` for each namespace. It takes the placement handle list of the namespace from its entry in `namespace_phls`, if set. Unset, the list holds the first `MIN(fdp.nruh, 128)` handles in order. Each placement handle then selects the RUH of the same number. A list holds at most 128 handles (NVM Command Set 1.2, Figure 125). The namespace's ENDGID is 1, and each handle's RUAMW starts at a full unit. The controller then sets CTRATT bit 19 (FDP), and the effects log lists I/O Management Send and Receive for NVM namespaces. Without FDP, and on namespaces of other command sets, the two commands fail with Invalid Command Opcode. CTRATT bit 4 (Endurance Groups), ENDGIDMAX = 1 and each namespace's ENDGID = 1 come with any subsystem, FDP or not, since its one endurance group is what log 09h reports. The Endurance Group List (Identify CNS 19h) lists group 1 when CDW11 asks for identifiers from 0 or 1; without a subsystem, CNS 19h is refused.

The BlackBox side (`bb.c`, `ftl-fdp.c`) adds the following, for a CSD namespace as for a bbssd one, since CSD runs the same FTL (`bb_init_fdp()`):

- RUNS is overwritten with the bytes of one superblock of a group: `nchs * luns_per_ch * pls_per_lun * pgs_per_blk * secs_per_pg * secsz / fdp.nrg`. `fdp.runs` must be unset or exactly that.
- Each group has `min(lines, fdp.nru)` reclaim units (`lines_per_ru = 1`). A unit is one block index on the dies of the group. Lines left over are not used by the placement write path.
- Each group needs at least `2 * fdp.nruh + 1` units: one open unit per handle, one collection unit per handle, and one spare.
- `gc_thres_pcent` and `gc_thres_pcent_high` set the free-unit watermarks instead of free-line watermarks. The forced watermark keeps at least one unit free, even where the percentage rounds to none, unless `gc_thres_pcent_high` is 100 (`bb_fdp_forced_units()`): a pass can fill its destination part way through a victim and needs a free unit to go on.
- The namespace must fit in the units left after the reserve (`bb_check_capacity()`), or the controller is refused at realize:

```
in each group:
reserve = fdp.nruh                    one open unit per handle
        + Persistently Isolated handles one collection unit each
        + forced watermark            free units forced GC keeps
usable  = (units - reserve) * unit size     (refused if units <= reserve)
every namespace together must fit in usable: the host may send all of
its data to one group (Base 2.3, 8.1.11)
```

  The Initially Isolated handle collects into its open unit and adds nothing. The namespaces are checked against the whole pool together: every namespace of the controller places into the one FTL, FDP takes a single controller, and namespace management is refused with it. A 1 MiB namespace on 64 KiB superblocks with four Persistently Isolated handles needs 16 + 4 + 4 + 1 = 25 units; with 19 or 23 lines, both accepted before, random writes over all four handles failed nearly all of them. A partly exposed last page counts as a whole one. bbssd refuses a namespace that does not fit rather than shrink it, as it does without placement and with `op_pcent`.

4.6.4 Several namespaces

Reclaim units and handles belong to the endurance group (Base 2.3, 8.1.11). Each namespace reaches the handles through its own placement handle list (PHL), and namespaces may share a handle. FEMU follows that model: every bbssd namespace of an FDP controller places into one FTL, the one namespace 1 builds.

```
namespace_phls=0:1,,1:2           endurance group 1, one struct ssd
ns 1  PH 0 -> RUH 0  ------------>  RUH 0  units ...
      PH 1 -> RUH 1  --+---------->  RUH 1  units ...   (shared)
ns 2  PH 0 -> RUH 1  --+
      PH 1 -> RUH 2  ------------>  RUH 2  units ...
                                    RUH 3  units ...   (unused: RUHA 0)
LPN = (backend_offset of the namespace + LBA * block size) / page size
      [ ns 1 slice      | ns 2 slice      ]   no page holds two namespaces
```

Part	How it works	Code
FTL	namespace 1 builds it; later namespaces point at it (`ssd_shared`); the owner's exit clears their pointers	`bb_init()`, `bb_ns_exit()`
Address	each namespace keeps its slice of the logical pages: `ssd_ns_byte_base()` adds its backend offset	`ssd_lpn_range()`
Handles	the FTL builds every handle `0 .. fdp.nruh - 1`; a handle no list names holds its open unit too	`femu_fdp_ssd_init_ru_handles()`
Placement	a write's PID selects a placement handle of its namespace's list	`fdp_place()`
RUAMW	counts blocks of the format of the namespaces that use the handle (`NvmeRuHandle.lbads`)	`fdp_ru_ruamw()`
Format, Sanitize	drop the pages of the slice of the namespace only	`ssd_deallocate_fdp_ns()`
SMART, C0h, log 09h, thermal	count the shared FTL once	`nvme_collect_media_stats()`
Release	a handle no remaining namespace lists is unused again; its byte counters stay	`nvme_ns_detach_fdp()`

Rules at realize (`nvme_check_phls()`, `nvme_check_fdp_namespaces()`):

- `namespace_phls` needs `fdp=on`, one entry per namespace, 1 to `MIN(fdp.nruh, 128)` handles per entry, each below `fdp.nruh`, none twice. Unset gives every namespace every handle. That list can be longer than 128 when `fdp.nruh` is, as it always could.
- With more than one namespace, every namespace runs bbssd, or every one NoSSD. A bbssd namespace size is a whole number of NAND pages, so no page holds blocks of two namespaces. `fdp_trim_erase_all` is refused, since it erases every namespace.
- The sum of the namespace sizes must fit in the units left after the reserve, as one namespace does.

Namespaces that share a handle share the format (NVM Command Set 1.2, 5.2.5 and 5.2.21). A Format of one of them to another index fails with Invalid Format. A Format of every namespace (NSID FFFFFFFFh) takes the bitmaps of all of them first, so it changes all or none.

Events from the FTL name a namespace by the data:

```
move of page lpn  --> namespace whose slice holds it
80h  NSID and LBA of that namespace; PID from its list
     a run of another namespace's pages ends the event: one 80h per run
81h  the namespace of the move that changed the handle
01h, 81h of an open-unit refresh   no data: the first namespace that
                                   lists the handle
a namespace that does not list the handle --> the first one that does
no namespace lists it               --> no NSID, no PID
```

The write buffer is part of the shared FTL. A Flush of one namespace writes back the held pages of all of them, which Flush allows.

Statistics stay per endurance group and per handle: a shared handle adds the writes of every namespace that uses it. query-femu reports the PHL of each namespace (`placement-handles`) and the namespace whose FTL holds the units (`ftl-nsid`).

NoSSD namespaces have no FTL. They report their lists in RUH Status and count their writes as NoSSD namespaces (§4.6.6) says.

4.6.5 Reclaim groups

A reclaim group is a set of dies. Base 2.3 (3.2.4) gives each handle a reclaim unit in each group and keeps collection inside a group (Figure 288). It does not say which media a group holds. FEMU follows physical FDP SSDs: a group is a set of dies that work in parallel, and a unit is a superblock of those dies.

```
dies d = ch * luns_per_ch + lun, D = nchs * luns_per_ch
group of d = d / (D / nrg)
  nrg divides nchs            --> nchs / nrg whole channels per group
  nchs divides nrg, and
  nrg / nchs divides luns_per_ch --> luns_per_ch * nchs / nrg LUNs of one
                                     channel per group
  anything else               --> refused at realize (bb_check_fdp_groups())

unit (g, j) = block index b_j on every plane of every die of group g
write pointer of a unit: steps over the window ch0..ch0+nch-1,
                         lun0..lun0+nlun-1 of its group (ssd_wp_step())
```

Part	How it works	Code
Lines	with more than one group, each group has its own line per block index; `get_line()` takes the group of the die; with one group the lines are those of line mode	`femu_fdp_ssd_init_reclaim_group()`
Frontiers	each handle has an active unit and a collection unit in each group	`FemuRuHandle.rus[]`, `gc_rus[]`, `fdp_set_active()`
Placement	a valid PID names the group; a write without one goes to the group with the most free units, in turn on a tie	`fdp_place()`, `fdp_select_rg()`
Collection	runs per group: foreground in the group of the write, background in every group under its watermark; victims and destinations stay in the group	`fdp_gc_until_clear()`, `bb_ftl_process_req()`
Erase, pad, copyback	walk the dies of the unit's group only	`fdp_collect_ru()`, `ssd_pad_line()`, `fdp_reclaim_ru_copyback()`
Write buffer	each held page keeps its group; a write-back skips a group with no room; a write at the watermark whose buffer holds only such pages goes to its own unit at once	`fdp_buffer_destage_some()`, `fdp_buffered_write()`
RUH Update	writes back the held pages of the named pairs only	`fdp_buffer_destage_update()`

Part	How it works	Code
Refresh	the unit's group must have room; with more than one group a queued refresh that cannot run is dropped, and the next read queues it again	`fdp_do_read_reclaim()`
Check	`x-fdp-check` adds, for each unit, the valid pages of its blocks on the dies of its group, and compares them with its valid count	`fdp_check_dies()`

A logical block must be whole NAND pages with more than one group (`bb_check_fdp_block()`). The FTL maps whole pages. A write of part of a page would program the whole page into the group of the write. The other blocks of the page would then change group with it. With one group the FDP-2 rule stays: the handle of the last write to a page takes the page.

4.6.6 NoSSD namespaces

A NoSSD namespace has no NAND model. The backend stores each byte of data once and no controller write exists. `nvme_fdp_host_write()` counts on the poller, under `stats_lock` of the endurance group, since several pollers write at once:

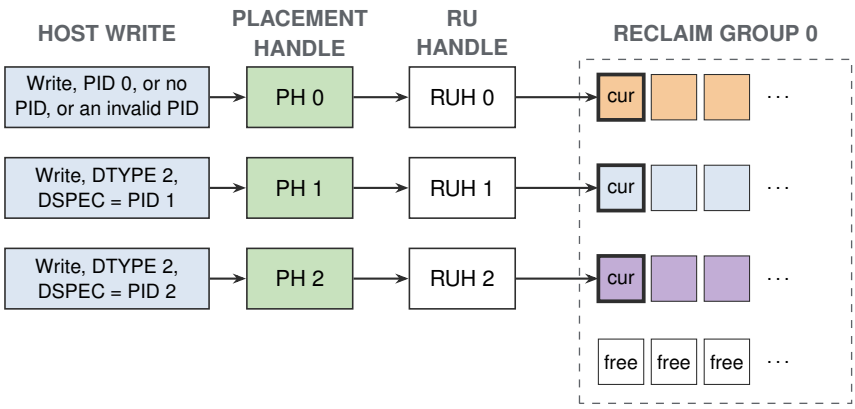
```
Write, Copy, Write Zeroes (DEAC 0)  --> HBMW += bytes, MBMW += bytes,
                                        RUAMW of rus[g] of the handle -= blocks
Write Zeroes (DEAC 1), Write Uncorrectable --> HBMW += bytes only
unit full (RUAMW would reach 0)     --> fresh unit: RUAMW = full - rest
RUH Update: unit part written       --> event 00h, fresh unit
            unit empty              --> stays, no event (Base 2.3, 7.4.1.1)
invalid PID                         --> placement handle 0, group in turn,
                                        event 03h
```

MBE stays 0. Realize refuses a logical block larger than `fdp.runs`. A Format to such a block fails with Invalid Format. So a unit holds at least one block. RUH Update holds `stats_lock` across its whole list, and RUH Status and the FDP Statistics log read under it.

4.6.7 Placement of writes

```
Write: CDW12 bits 23:20 DTYPE, CDW13 bits 31:16 DSPEC
  |
  | DTYPE == 2 (data placement) and DSPEC is a valid placement identifier?
  |   no  -> placement handle 0, in the group with the most free units
  |          (if DTYPE was 2: host event "Invalid Placement Identifier")
  |   yes -> PID = (RG << (16 - RGIF)) | PH ;  with RGIF 0, PID = PH
  v
PH --ns->fdp.phs[]--> RUH --rus[RG]--> active RU --wptr--> next page
  |
  | per FTL page (secsz * secs_per_pg, 4 KiB by default): invalidate the old copy, map LPN -> new page,
  | RUAMW -= logical blocks written, advance the write pointer,
  | charge a NAND program
  v
RU full -> retire it (full list or victim queue), take a free RU
           for this handle; none free -> Capacity Exceeded
```

`ssd_stream_write_lpns()` does this on the FTL thread. Write, Write Zeroes without the Deallocate bit, and the destination of Copy all go through it; Copy and Write Zeroes read their placement fields from the same CDW12 and CDW13 bits.



With one reclaim group (`fdp.nrg=1`, the only value) the PID has no group bits: it picks a placement handle, which the namespace maps to a reclaim unit handle. Each handle writes into its current RU and takes the next free RU when that one is full. One RU is one BBSSD line: one block on every channel, LUN and plane. With the default `fdp.isolation_mode=0` every handle is persistently isolated, so garbage collection moves its valid pages into a GC RU of the same handle.

Figure 15: FDP placement on a BBSSD: a write's placement identifier picks a placement handle, the namespace maps it to a reclaim unit handle, and the handle fills its current reclaim unit (one FTL line) before taking a free one.

The order inside `ssd_stream_write_lpns()` is:

1. If the handle has no active unit (the last one filled and no free unit was left), take a free unit without running GC; if there is none, fail with Capacity Exceeded. A unit that fills on a command's last page therefore makes the next write to that handle fail when the device is full.
2. Run foreground GC while the free-unit count is at or below the high watermark (see Garbage collection (§4.6.8)).
3. Place the pages, running foreground GC again before each one: a command can take more units than the watermark keeps free. If the device runs out part way, the command completes with Capacity Exceeded and only the pages already placed count.

Writes of different handles never share an RU, but they share the NAND of their group. Every RU of a group spans the dies of the group. So concurrent handles of one group compete for the same planes and channel buses.

Reclaim units over NAND

An RU is one BlackBox line: block `b` on every channel, LUN and plane. Its write pointer moves across channels first, then LUNs, then planes, then down the pages of the block, so consecutive pages of one handle land on different dies:

```
RU k = line k (block index k), nchs=4, luns_per_ch=2, pls_per_lun=2

page order:   ch0/L0/pl0  ch1/L0/pl0  ch2/L0/pl0  ch3/L0/pl0
              ch0/L1/pl0  ch1/L1/pl0  ch2/L1/pl0  ch3/L1/pl0
              ch0/L0/pl1  ...                     ch3/L1/pl1   <- page 0 done
              ch0/L0/pl0 (page 1) ...                          <- page 1
              ...                                 last page    <- RU full

RUNS = all of that = one superblock
```

`fdp_advance_ru_pointer()` implements the order; `fdp_get_new_page()` turns the pointer into an address.

The NAND timing of each program is the BlackBox timing (see NAND timing (§3.5)).

Reclaim Unit Handle Update

I/O Management Send with operation 1 (RUH Update) lists placement identifiers. The poller checks every identifier first and fails the command with Invalid Field if one is invalid, so nothing changes. On a bbssd namespace `ssd_fdp_update_ruhs()` then, for each handle:

- gives a handle that has no current unit (its last unit filled with nothing free to follow it) an empty unit, after the same passes as below, or fails with Capacity Exceeded when none can be freed;
- keeps the current unit if nothing was written to it yet. The handle does not move, so no event follows (Base 2.3, 7.4.1.1 lets an empty unit stay);
- otherwise first runs foreground GC passes in the group, as a write does, while its free units are at or below the forced watermark. An update that took the last free units would leave collection no unit to move pages into. A pass into an Initially Isolated handle can fill the old unit and move the handle, which then counts as the update;
- then takes a fresh unit, with one GC pass if none is free, and retires the old one as if it had filled. If no unit can be freed, the command fails with Capacity Exceeded;
- when the handle moves off a unit that still had room (RUAMW not zero), records host event "Reclaim Unit Not Fully Written". The handle must have that event enabled.

Each identifier names a handle and a group, and moves that pair only.

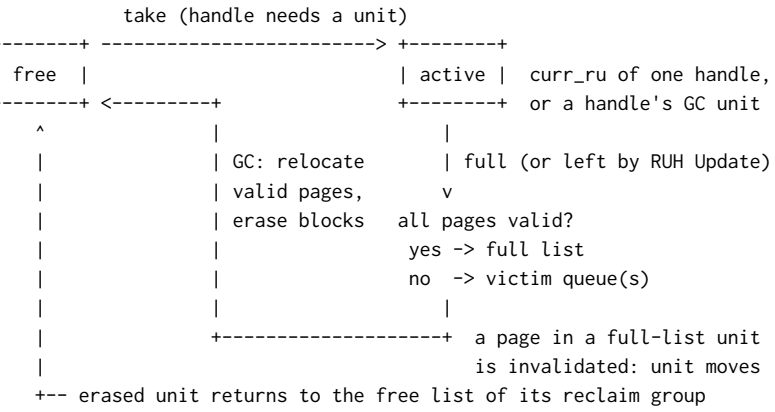
I/O Management Receive with operation 1 (RUH Status) returns one descriptor per placement handle and reclaim group: PID, RUHID, EARUTR (always 0) and the RUAMW of the handle's current unit, or 0 while the handle has none because its last unit filled with no free unit to follow it.

RUAMW is the room left in a unit, in logical blocks. Each unit counts the bytes programmed into it since its erase, from every path:

```
host program (direct, FUA, Write Zeroes) --+
buffer write-back                         --+--> ru->prog_bytes += page size
relocation (collection, refresh, copyback) -+         |
pad (pd_open_close_sec)                   --+         v
                        RUAMW = (RU size - prog_bytes) >> LBA size bits
unit freed (erase)  --> prog_bytes = 0, RUAMW = a full unit
Format              --> RUAMW from prog_bytes again, in the new LBA size
```

Collection writes into the active unit of the last handle of an Initially Isolated configuration, so that handle's RUAMW drops for the moved pages too. The remaining bytes round down once, so a page smaller than a block counts the same on every path. `x-fdp-check` (a test hook) counts a unit whose bytes differ from its valid and invalid pages, or whose RUAMW differs from the room they leave.

4.6.8 Garbage collection



When it runs

- Background: after every request the FTL thread handles, each reclaim group whose free units are at or below `(1 - gc_thres_pcent / 100) * units` runs one pass. A background pass puts its victim back, whatever the policy, unless the victim is empty or an erase would give back at least 1/8 of its pages. Those pages are the invalid ones plus, for a unit that RUH Update retired part written, the ones it never wrote. Cost-benefit skips units that fail this check and takes the best-scoring unit that passes, so an old unit with little to free does not stall the pass.
- Foreground: before a placed write and before each of its pages, it checks the free units of the write's group. While they are at or below `(1 - gc_thres_pcent_high / 100) * units` (at least one unless `gc_thres_pcent_high` is 100), it runs passes in that group. It stops when the pressure clears or no victim is left. The host write waits for them. RUH Update runs the same passes before it takes a unit. Foreground passes, and the passes RUH Update runs, always collect their victim.

Victim selection

`gc_strategy` selects the policy, or `gc_policy` by name. A `gc_policy` other than `greedy` needs `gc_strategy=0`; unset or `greedy`, it leaves the choice to `gc_strategy`.

gc_strategy	gc_policy	Policy
0 (default)	`greedy`	greedy: the unit with the fewest valid pages
1	`cost-benefit`	cost-benefit: an empty unit first, otherwise the largest `(1 - u) * age / u`, with `u` the valid pages over all the unit's pages (pages a part-written unit never wrote count as freed) and `age` the time since a page in the unit was last invalidated, computed at selection time; a unit retired with no page invalidated is aged from its retirement
2	`random`	random among the victims
-	`fifo`	the unit retired first; an empty unit goes before it, and a background pass takes the oldest unit it accepts
-	`d-choice`	the fewest valid pages among 4 random draws from the victims, with replacement; the first of equal counts wins; an empty unit goes first
4	-	per handle: the unit with the fewest valid pages among the per-handle queues, in the group, of Persistently Isolated handles that hold units there, falling back to greedy

Where relocated data goes

The handle type decides the destination of the valid pages, which is what "isolation" means here:

```
victim from a Persistently Isolated handle h
    -> h's own GC unit in the victim's group (gc_rus[g], taken from the
       free list of the group on demand)
       data of h stays apart from every other handle, before and after GC

victim from an Initially Isolated handle
    -> the current unit of the last handle (RUH nruh-1) in the group
       data may be mixed with other relocated data after GC
```

With the default `fdp.isolation_mode=0` every handle is Persistently Isolated. With it set, only the last handle is Initially Isolated, so its relocated data goes back into its own current unit.

A pass copies every valid page of the victim, invalidating each old copy as it goes, and only then erases the planes of each LUN as one multi-plane erase when `enable_gc_delay` is on (the default). It adds the copied bytes to MBMW and the erased bytes to MBE, records the controller events of Events (§4.6.9), and returns the unit to the free list. If the destination runs out of space part way, the pass stops before it erases anything. The pages already moved live at their new location, and the rest stay in the victim. The victim goes back on the queue for a later pass.

With `gc_copyback` on, the pass takes the victim's pages in the order of the destination dies. A page whose destination is on its own die and plane moves by copyback, with no data on the channel. A copyback skips ECC, so on a real SSD its bit errors are not corrected; FEMU models no bit errors, so only timing changes (copyback (§3.4.6)).

Both destinations can be left without a unit: one that fills with no free unit to follow it is dropped. The next pass with a page to move takes a free unit for it from the victim's reclaim group, whatever state the handle's own current unit is in. A victim with no valid page left needs no destination, so it is collected even when no unit is free; that is how a device that ran out gets a unit back once the host deallocates or overwrites data.

Deallocate

Dataset Management deallocate on an FDP namespace unmaps the given ranges; the pages become invalid and GC reclaims their units later. Write Zeroes with the Deallocate bit does the same. `fdp_trim_erase_all` (test only) instead resets every unit, handle and mapping on any Dataset Management deallocate, ignoring the ranges; Write Zeroes with Deallocate still unmaps only its range.

Block wear-out

With `blk_pe_limit`, each block has an erase limit, as in bbssd (end of life (§3.4.10)). The check runs in `fdp_collect_ru()` after the erase of every block of the victim unit. The unit holds no data then. Collection, read reclaim and retention refresh all reach it there.

```
erase of unit U in group g takes a block of U to its limit
  eol_read_only=off:
    U(g) - 1 >= floor and a free unit besides U  --> U retires
    otherwise                                    --> U goes back to the
                                                     free list; its worn-out
                                                     blocks stay in service
                                                     (overworn, bit 2)
  eol_read_only=on (default):
    U(g) - 1 >= floor                            --> U retires
    otherwise                                    --> U retires, READ-ONLY

U(g)  = units of group g - retired units of group g - reserve units R
floor = units the namespaces fill (ceil(exposed pages / unit pages))
      + one unit per handle + one per persistently isolated handle
      + the units forced collection keeps (at realize) + 1
```

- The floor is what `bb_check_capacity()` asks of the pool at realize, plus one unit. Each group keeps its own floor, since a host may place all of its data in one group (Base 2.3, 8.1.11).
- The free unit besides U is where the next collection moves pages to.
- A retired unit is on no list, holds no data and has no handle. It keeps its blocks; their erase counts no longer change.
- SMART Available Spare is the worst group's share of the units it may still retire: `(units - R - floor - retired) * 100 / (units - R - floor)`, or 100 when the group starts at or below the floor. It is 0 once the media is read-only.
- SMART critical warning bit 2, the Endurance Group Critical Warning Summary, and the Endurance Group Information log (09h) critical warning, Available Spare and Percentage Used all follow the same state. Crossing into a warning raises the SMART asynchronous event the host enabled.
- No FDP event reports a retired unit; the specification has none. The moves before the erase are reported as before. The configuration log and RUH Status do not change.
- Log page C0h offset 128 counts the lines of retired units. `query-femu` shows them in the `retired` state.

Read-only end of life. With `eol_read_only` (on by default), the media goes read-only when a worn-out block has no unit to replace it, as in line mode. The page credits, the 182h refusal, WZSL, MCL and the Format and Sanitize rules are the same. Each reclaim group holds R erased units back for the writes in flight:

```
B = most bytes one command may program: MDTS, or MCL in the largest
    LBA format when that is larger (femu_eol_max_bytes())
C = credit cap in pages = ceil((ceil(B / page) + 1) / Q) * Q
R = C / Q + handles                      in EVERY reclaim group
    Q = pages of one unit; realize counts R out of each group's pool

transition (FTL thread)
  read-only bit set in the admission word; no write is admitted again
  each group's R reserve units join its free list
  collection, read reclaim, retention refresh and padding stop
  admitted writes finish into their handles' units and the reserve
```

- A handle that takes c pages takes at most c / Q + 1 new units: one for a missing unit and one at each fill. A host may place every page in one group, so each group holds the whole R.
- A host write that finds no unit for its handle, after collection, also ends the life of the media. The write then finishes from the reserve. This covers a device whose collection cannot make room, such as one with `gc_thres_pcent_high=100`.
- RUH Update after the transition fails with Capacity Exceeded, as one with no empty unit to give: a move would take a reserve unit that no data can fill. An update the poller passed before, or whose own

collection ends the life of the media, fails the same way. The handles it moved before that stay moved (Base 2.3, 7.4.1.1).

- `query-femu` shows the reserve units in the `reserve` state and counts them in `unit-counts`. Its `end-of-life` object shows the credits.

```
-device femu-subsys,id=fdpe,fdp=on,fdp.nruh=4 -device femu,devsz_mb=1,femu_mode=1,secsz=512,secs_per_pg=8,␣
    pgs_per_blk=4,blks_per_pl=40,luns_per_ch=2,nchs=2,subsys=fdpe,blk_pe_limit=8,mdts=2
```

With `eol_read_only=off`, this small device retires units after a few thousand writes, then keeps worn-out blocks in service:

```
-device femu-subsys,id=fdpw,fdp=on,fdp.nruh=4 -device femu,devsz_mb=1,femu_mode=1,secsz=512,secs_per_pg=8,␣
    pgs_per_blk=4,blks_per_pl=40,luns_per_ch=2,nchs=2,subsys=fdpw,blk_pe_limit=8,eol_read_only=off
```

4.6.9 Log pages and features

All four FDP log pages take endurance group 1 in the Log Specific Identifier, and need a subsystem with FDP on. FEMU fixes FDP at realize. So with FDP off, the controller has no FDP capability (CTRATT bit 19 is clear). The pages then fail with Invalid Log Page, and log 00h does not list them.

Log	Content	Code
20h FDP Configurations	header with NUMFDPC 0 (one configuration), then one descriptor: FDPA (valid, RGIF), NRG, NRUH, MAXPIDS, NNSS 256, RUNS, and NRUH handle descriptors with the RUHT of each handle	`nvme_fdp_confs()`
21h Reclaim Unit Handle Usage	header with NRUH, then one descriptor per handle with its RUHA (1 = host-specified for a handle some namespace lists, 0 for one no list names)	`nvme_fdp_ruh_usage()`
22h FDP Statistics	HBMW, MBMW and MBE as 128-bit counters, which stop at the 128-bit maximum	`nvme_fdp_stats()`
23h FDP Events	number of events, then the events of the ring selected by LSP bit 0 (1 = host events, 0 = controller events)	`nvme_fdp_events()`

The 21h layout

The log is an 8-byte header followed by one 8-byte Reclaim Unit Handle Usage Descriptor per handle, as the specification defines it. Each descriptor holds the handle's RUHA in byte 0, and bytes 1 to 7 are reserved. `NvmeRuhuDescr` in `nvme.h` has that layout, and a `QEMU_BUILD_BUG_ON` keeps it at 8 bytes. The log is `8 + 8 * NRUH` bytes:

```
offset      content
0           NRUH (2 bytes), reserved (6 bytes)
8           handle 0: RUHA, reserved
16          handle 1: RUHA, reserved
8 + 8 * i   handle i: RUHA, reserved
```

The log carries no per-handle byte counters. The endurance-group totals are in 22h.

Statistics

Counter	Grows by	Where
HBMW (host bytes with metadata written)	bytes of the logical blocks a Write, Copy or Write Zeroes without Deallocate wrote, in the pages it actually programmed or the buffer took (`fdp_host_bytes()`); and the bytes of the blocks of a Write Zeroes with Deallocate or a Write Uncorrectable, which NVM Command Set 1.2, 4.1.4.6 counts too, though they program no page	`fdp_count_write()`, `fdp_buffered_write()`, `ssd_fdp_count_unprogrammed()`
MBMW (media bytes with metadata written)	the bytes of the pages programmed for the host, plus bytes relocated by GC, plus pad bytes (`pd_open_close_sec`)	`fdp_count_write()`, `do_gc_fdp_style()`, `fdp_pd_leave_ru()`
MBE (media bytes erased)	bytes of every block erased by GC	`do_gc_fdp_style()`

The ratio MBMW / HBMW is the write amplification of the endurance group. A write smaller than a page programs the whole page, so it raises the ratio above 1, as it does on a real device. The counters are charged after the write, from the pages it placed, so a write that ran out of space adds only what it wrote. They are reset by a deallocate with `fdp_trim_erase_all` set.

Events

There are two rings of 63 events each, one for host events and one for controller events. When a ring is full the oldest event is overwritten. Each event carries the Timestamp feature's current value.

Type	Ring	Generated when
0h Reclaim Unit Not Fully Written	host	RUH Update moves a handle off a unit that still had room; an empty unit stays and logs nothing
3h Invalid Placement Identifier	host	a placed write (Write, Copy, Write Zeroes, Write Uncorrectable) on a bbssd or NoSSD namespace has DTYPE 2 and a DSPEC that is not a valid PID; PIV with that DSPEC, NSIDV, and LV with the group and handle the controller selected (Base 2.3, Figure 296); checked against the event filter of placement handle 0
80h Media Reallocated	controller	a collection or refresh pass moved pages of a unit of the Initially Isolated handle; one event per pass, split around each 81h the pass causes, also for a pass that stops part way; checked against that handle's filter
81h Implicitly Modified Reclaim Unit Handle	controller	a move, not the host, gave the Initially Isolated handle a new unit: the moves filled its active unit and a free unit follows, or the pass takes a unit for a handle left with none; checked against that handle's filter
1h Reclaim Unit Active Time Limit Exceeded	host	`pd_open_close_sec`: the controller padded a handle's active unit and gave the handle a fresh one; LV, NSIDV and PIV as for 81h
81h, program disturb	controller	`pd_open_read_limit`: the controller took a handle's active unit away to refresh it; this one also for a Persistently Isolated handle
2h		accepted by the event filter, never generated

A Persistently Isolated handle moves its data into a collection unit that the host never sees, so its moves log no event. A host write that fills a unit is a host action and logs no 81h either.

```
pass over victim unit V of handle h
  |
```

```
+-- h Persistently Isolated --> moves into h's gc_ru ------> no event
|
+-- h Initially Isolated ----> moves into h's active unit
       |  a move fills it, next unit follows --> 80h (moves so far)
       |                                         81h (h)
       |  pass ends, moves not yet reported --> 80h (the rest)
```

Both events have LV set, with the victim's reclaim group and handle. The FTL belongs to one namespace, so NSIDV and PIV are set too. PID encodes the victim's reclaim group and the lowest placement handle of that namespace that maps to the handle. The Event Type Specific field of 80h is in NVM Command Set 1.2, Figure 112. It holds LBAV, the number of blocks moved (at most FFFFh) and one of them.

The FTL moves whole pages, so the count takes only the blocks of a moved page that a host write programmed into it. The FTL notes those blocks as it programs each page, in 512-byte pieces. The block reported is the first such block. With a page size that is not a multiple of 512 bytes, the FTL keeps no such record, and 80h reports no count and no block. The FTL drops a handle's active unit when it fills with no free unit. That logs nothing, and the unit taken later logs the 81h.

The FTL maps whole pages. A host write of part of a page programs the whole page through the write's handle. The other blocks of that page then change handle with it. That is a host action, and FEMU logs no 80h for it.

Get and Set Features 1Eh (FDP Events) read and change the enabled event types of one placement handle, named in CDW11; an unsupported type in a Set fails with Invalid Field. Get Features 1Dh (FDP Mode) for endurance group 1 reports that FDP is on. Both features exist only with FDP on; otherwise they fail with Invalid Field, and log 12h does not list them. Set Features 1Dh fails with Invalid Field for an endurance group other than 1, and otherwise with Command Sequence Error, since the mode may only change while the endurance group has no namespaces and FEMU builds the namespace at realize.

4.6.10 Threads, locks and where latency is charged

Work	Thread	Notes
Parsing DTYPE and DSPEC, validating RUH Update identifiers, I/O Management Receive	poller	data copy to the DRAM backend happens here too
Placement, RU rotation, GC, deallocate, RUH Update on bbssd	FTL thread	sole owner of the FTL structures
Event append	poller or FTL thread	`events_lock` in the endurance group
Counters of log 22h; NoSSD RUAMW	FTL thread, poller	`stats_lock` in the endurance group, taken before `events_lock`
Log pages 20h to 23h, Features 1Dh and 1Eh	admin path	23h reads under `events_lock`

A placed write is charged the largest NAND program latency of its pages, after any foreground GC passes, whose erases and relocations occupy the same LUNs. GC programs and erases use the BlackBox timing with `enable_gc_delay` (see the FTL chapter (§3.4)).

4.6.11 Parameters

Subsystem properties are in the Flexible Data Placement section (§A.1.2) of the property reference; the controller-side ones are under garbage collection, mapping and caches (§A.1.1).

Property	On	Effect and interactions
fdp	femu-subsys	Turns FDP on. Refused together with ns_mgmt=on.
fdp.nruh	femu-subsys	Handles and placement handles. Required. Raises the units the geometry must supply.
fdp.nru	femu-subsys	Units per group; the FTL uses at most one per line.
fdp.nrg	femu-subsys	Groups; with bbssd a set of dies each, see Reclaim groups (§4.6.5).
fdp.runs	femu-subsys	Unset, or the superblock size of a group.
fdp.isolation_mode	femu-subsys	Non-zero makes the last handle Initially Isolated.
gc_strategy	femu	Victim policy, above. No effect without FDP.
fdp_trim_erase_all	femu	Whole-device reset on deallocate. No effect without FDP.
gc_thres_pcent, gc_thres_pcent_high	femu	Background and foreground watermarks, as fractions of the units.
gc_copyback	femu	Moves collected pages on their die; off by default. A copyback skips ECC (timing only in FEMU).
BlackBox geometry (nchs, luns_per_ch, pls_per_lun, blks_per_pl, pgs_per_blk, secs_per_pg, secsz)	femu	Sets RUNS (one superblock of a group) and the number of lines, hence units.

Refused under FDP on a bbssd controller:

- hot_cold_sep: the placement handle of a write names its unit, and the hot write pointer writes past it;
- mapping=hybrid and mapping=fast, whose log and data lines are not FDP-aware;
- a gc_policy other than greedy together with a gc_strategy other than 0.

The FTL applies trim_lat_ns and ecc_retention_sec as without FDP. A unit's lines take their close time when the unit retires, and lose it when collection frees the unit.

read_reclaim_limit and retention_limit_sec refresh a whole reclaim unit. A read queues the unit of its line, as in line mode. The next write moves its valid pages to where collection would move them, then erases the unit.

```
read of unit U: read count >= read_reclaim_limit, or age >= retention_limit_sec
  --> U queued (one at a time)
next write: U's group not at the high watermark, 2 or more free units
  (with more than one group, else the request is dropped)
  --> U active (rus[g] or gc_rus[g]) --> request dropped
  --> U full or a victim            --> taken off its list
  --> move every valid page, no copyback --> erase U, free U
  --> C0h read_reclaims or retention_refreshes + 1, MBMW + moved bytes
```

The refresh logs the same FDP events as a collection pass (see Events (§4.6.9)).

buffer_size puts the write buffer in front of placed writes. The buffer models time only: it holds page numbers, and the backend holds the data. Each held page keeps the handle and group of its last write. A write-back programs each page into the active unit of its own handle in its own group, so a unit takes only the pages of its handle.

```
Write or Copy, no FUA, cache on
  --> each page: held?  no, and buffer at the watermark
        --> write back the oldest (buffer_size - watermark) pages
        --> still at the watermark: the page goes to its own unit at
```

```
            once if its group has room, else Capacity Exceeded
  --> entry {lpn, handle, group, written 512-byte pieces} := this write
  --> HBMW += bytes of the blocks written; C0h host pages, 72 and 80 count
write-back of entry e
  --> collect in e.group; e.handle has no unit there and none is free:
      e stays, and so do the later pages of e.group; go on with the rest
  --> program into e.handle's active unit, RUAMW of that unit drops
  --> MBMW of the group and of e.handle += page size; e leaves the buffer
```

The FTL maps whole pages. The handle of the last write to a page takes the whole page, with or without the buffer, also when two handles wrote different blocks of it. The other commands:

Command	Effect on held pages
Flush	writes every held page back; Capacity Exceeded if one finds no room, and those stay held
Write with FUA, Write Zeroes without Deallocate	program at once; a held copy of a page goes when this command programs that page
Volatile Write Cache off (feature 06h)	writes every held page back first; if one finds no room, the command fails and the cache stays on
DSM Deallocate, Write Zeroes with Deallocate, Format	drop the held copy, which never reaches the media
I/O Management Send, RUH Update	writes back the held pages of the named (handle, group) pairs first, so they land in the units the handles named before the update; if one of them finds no room, no handle moves and the command returns Capacity Exceeded
Read	a held page is served from the buffer

HBMW counts the blocks a write accepts, and MBMW pages that reach the media, so MBMW can be below HBMW when the buffer absorbs rewrites. RUAMW drops when a page reaches its unit, for written back and direct pages alike (see RUAMW above). The admin commands above charge no modelled time for their write-back.

The program disturb actions work on units too (NAND timing (§3.5.10)). A unit today is one line, so its open time is the open time of that line.

```
pd_open_read_limit: unit U queued for a part-programmed block
  next write --> U a handle's curr_ru --> fresh unit to the handle first,
                                          U retired with its free pages
                                          unprogrammed; 81h (handle)
             --> U a handle's gc_ru   --> U retired, gc_ru cleared; no event
             --> then the refresh above; C0h pd_open_reclaims + 1
pd_open_close_sec: curr_ru or gc_ru open past the limit, at a request
  --> its free pages padded, U retired, MBMW + pad bytes
  --> curr_ru: fresh unit to the handle; host event 1h (handle)
  --> ERUTL stays 0: the limit counts from the first program
```

The fresh unit is taken before anything changes. With no free unit to spare, or a group at the high watermark, nothing changes. The handle's `curr_ru`, `rus[rg]` and the unit the host is shown change together. The configuration descriptor keeps ERUTL at 0 (not reported). ERUTL times a unit from when the handle takes it, but the limit here counts from the first program, so an empty unit is never padded. Collection that frees a queued unit first drops the request, and the counters do not move. A collection of a queued unit does not use copyback.

Refused on any controller with FDP: a second controller in the subsystem, `meta`, `streams`, a KV namespace. Each message is listed in the feature guide (§4.5.5).

A device with eight handles, every one Persistently Isolated except the last:

```
-device femu-subsys,id=fdp8,fdp=on,fdp.nruh=8,fdp.isolation_mode=1 -device femu,devsz_mb=1024,femu_mode=1,subsys=fdp8,gc_strategy=1
```

4.6.12 Statistics and outputs

Output	What it shows
Logs 20h to 23h	configuration, handle usage, endurance-group byte counters, events
Vendor log C0h	write amplification and page counts of the FTL; placed writes count as host and programmed pages, relocations as GC pages (log pages and counters (§B.1))
SMART / Health (02h)	host data units and commands, as for every mode
`FEMU_FDP_DEBUG` in QEMU's environment	traces of RU rotation and GC passes on stderr

4.6.13 Validation

Check	What it covers
qtest cases in `hw/femu/tests/qtest/femu-test.c`	`fdp-ns-phl`, `fdp-ns-cmds`, `fdp-ns-format-shared`, `fdp-ns-format-apart`, `fdp-ns-events`, `fdp-ns-knobs`, `fdp-ns-nossd`, `fdp-ns-refused`, `fdp-ns-replace`, `fdp-replace-full` (several namespaces, and a replaced controller whose handle had no unit; `x-fdp-ns-mapped` gives each namespace's mapped pages), `fdp-events`, `fdp-features`, `fdp-report-length`, `fdp-ruh-usage`, `fdp-write-zeroes`, `fdp-write-zeroes-placed`, `fdp-ruh-update`, `fdp-ruh-update-full`, `fdp-background-gc`, `fdp-cb-unwritten-age`, `fdp-cb-utilization`, `fdp-cb-background`, `wide-lba-fdp`, `io-fuzz-fdp`, `copy-fdp`, `log-contents-fdp`, `ns-mgmt-unavailable-fdp`, `fdp-csd-knobs`, `fdp-knobs-refused`, `fdp-policy-fifo`, `fdp-policy-fifo-greedy`, `fdp-policy-fifo-empty`, `fdp-policy-fifo-csd`, `fdp-policy-fifo-background`, `fdp-policy-cb`, `fdp-policy-cb-strategy`, `fdp-policy-cb-greedy`, `fdp-policy-refused`, `fdp-dftl`, `fdp-dftl-page`, `fdp-dftl-csd`, `fdp-buffer-isolation`, `fdp-buffer-isolation-csd`, `fdp-buffer-rehandle`, `fdp-buffer-dealloc`, `fdp-buffer-fua`, `fdp-buffer-vwc`, `fdp-buffer-update`, `fdp-buffer-full`, `fdp-buffer-copy`, `fdp-events-gc-ii-part-buffer`, `fdp-events-gc-ii-mix-buffer`, `fdp-events-gc-ii-fua-buffer`, `fdp-events-gc-ii-redo-buffer`, `fdp-csd-runs`, `fdp-gc-strategy-refused`, `fdp-events-gc-pi`, `fdp-events-gc-ii`, `fdp-events-gc-ii-cb`, `fdp-events-gc-ii-4k`, `fdp-events-gc-ii-part`, `fdp-events-gc-ii-rot`, `fdp-events-gc-ii-mask`, `fdp-events-gc-ii-no80`, `fdp-events-gc-ii-no81`, `fdp-events-gc-ii-other`, `fdp-events-gc-ii-empty`, `fdp-events-rf-pi`, `fdp-events-rf-ii`, `fdp-events-rf-ii-part`, `fdp-confs` (log 20h), `fdp-invalid-pid` and `fdp-invalid-pid-nossd` (event 03h), `fdp-unprogrammed`, `fdp-nossd-counts`, `fdp-runs-block`; reclaim groups: `fdp-confs-rg2`, `fdp-rg2-place`, `fdp-rg3-place`, `fdp-rg4-place` (PIDs, RUH Status, placement by group from query-femu, group selection, RUH Update of one pair), `fdp-rg2-gc`, `fdp-rg2-gc-cb`, `fdp-rg2-gc-ii`, `fdp-rg4-gc` (the whole namespace through one group, collection in it, no page and no erase in another), `fdp-rg2-buffer` (a full group does not stop another)

Check	What it covers
Exact traces and victim-order digests (qtest-only `x-ftl-trace`)	`ftl-trace-fdp` and `ftl-trace-fdp-reread` (greedy), `ftl-trace-fdp-random` (`gc_strategy=2`), `ftl-trace-fdp-policy-random` (`gc_policy=random`, the same trace), `ftl-trace-fdp-fifo` and `ftl-trace-fdp-d-choice` (`gc_policy`), `ftl-trace-fdp-knobs-off` (`gc_policy=greedy`, `mapping=page` and `buffer_size=0` given: the `ftl-trace-fdp` trace), `ftl-trace-fdp-noisy` and `ftl-trace-fdp-noisy-ii` (`gc_strategy=4` on two handles, the second one Initially Isolated in `-ii`), `ftl-trace-fdp-trim-erase-all` (`fdp_trim_erase_all=1` while both kinds of heap hold victims; afterwards the mapping holds exactly the pages written since)
Documentation examples	each tagged FDP example starts under qtest and moves one block
`hw/femu/scripts/fdp-test-nvme-admin.sh`	in-guest nvme-cli checks against the `run-blackbox-fdp.sh` configuration; manual
`hw/femu/tests/unit/test-pqueue.c`	the priority queue the victim queues are built on

4.6.14 Limits

- One FDP configuration and one controller per FDP subsystem.
- An RU is exactly one block index on the dies of its group; RUNS cannot be smaller or larger than that superblock.
- Only bbssd places data. A NoSSD controller counts the host side (NoSSD namespaces (§4.6.6)). CSD uses the bbssd path, with the same refused knobs and RUNS, and without its memory and compute namespaces. ZNS, OCSSD and KV are refused (Base 2.3, 8.1.11).
- With more than one group, each group holds the whole exposed capacity, and a logical block is whole NAND pages.
- query-femu shows the FDP state of bbssd namespaces only.
- The FTL thread writes the RUAMW of bbssd units, and pollers read it for RUH Status with no lock, as before. The counters of log 22h change and are read under `stats_lock`.
- A bbssd write that runs out of room counts the pages it programmed or the buffer took before the failure. A NoSSD command that fails counts nothing, except a two-page write whose second transfer fails: the whole blocks of its first page count.
- Page or DFTL mapping only.
- The CSD NVM-to-AFDM transfer reads the mapping only, without FDP as well: a page the write buffer holds and never wrote back costs no read.
- EARUTR is always 0; ERUTL is not reported.
- The handle counters of query-femu are 64-bit and stop at their largest value.
- Metadata, Streams and Namespace Management do not combine with FDP.
- `spare_lines`, `fdp_trim_erase_all` and `buffer_size` are refused with `blk_pe_limit`.
- With `gc_thres_pcent_high=100` no unit is kept free for collection. A persistently isolated handle then may find no unit to move pages into, and a write can fail with Capacity Exceeded before the namespace is full. With `blk_pe_limit` and `eol_read_only`, that write ends the life of the media instead.

4.6.15 How to extend it

- Larger reclaim units: `lines_per_ru` in `ssd_init_fdp_params()` is the hook, but `fdp_advance_ru_pointer()` retires a unit after its first line; it must walk `ru->lines[]` first.
- A new victim policy: add a value to the GC strategy enum in `bbssd/ftl.h`, a case in `select_victim_ru()` that chooses a unit without removing it, and accept the value in the geometry checks of `bbssd/`

ftl-geom.c. Queue, remove and reorder victims only through fdp_victim_enqueue(), fdp_victim_dequeue() and fdp_victim_reprioritize(), which keep the group heap (pos) and the handle heap (ruh_pos) in step.
- A new event: generate it with nvme_fdp_record_event() after checking the handle's filter, and add the type to nvme_fdp_events_supported[] and nvme_fdp_evf_shifts[] in nvme.h.

4.6.16 Source map

File	What is there
hw/femu/femu.c	femu-subsys properties, nvme_subsys_setup_fdp(), nvme_ns_init_fdp(), nvme_ns_refresh_fdp(), namespace_phls, single-controller and multi-namespace checks, CTRATT
hw/femu/nvme.h	FDP wire structures (configuration, usage, statistics, events, RUH status), endurance group and handle state, event tables
hw/femu/nvme-io.c	DTYPE and DSPEC parsing, PID helpers, event recording, I/O Management Send and Receive
hw/femu/nvme-admin.c	log pages 20h to 23h, Features 1Dh and 1Eh, supported logs, effects log
hw/femu/bbssd/ftl-fdp.c	reclaim group, unit and handle setup, placed write, RU rotation, RUH Update, GC, deallocate
hw/femu/bbssd/bb.c	FDP checks on the bbssd geometry and refused knobs, RUNS
hw/femu/bbssd/ftl.c	FTL-thread dispatch to the FDP paths, background GC trigger
hw/femu/scripts/run-blackbox-fdp.sh	launcher

4.6.17 Related pages

- FDP feature guide (§4.5)
- BlackBox mode (§4.4)
- Namespaces and subsystems (§4.16)
- Property reference: Flexible Data Placement (§A.1.2)

4.7 Zoned Namespace SSD (ZNS)

ZNS mode (femu_mode=3) emulates an NVMe SSD with the Zoned Namespace command set. The namespace is divided into zones. The host writes each zone sequentially at its write pointer (or with Zone Append), and resets a zone before writing it again. The device runs no garbage collection: reclaiming space is the host's job. FEMU models the zone state machine, open and active zone limits, a pool of zone write caches and the NAND time of each read, program and erase.

Use it to develop and measure zoned software: zoned file systems (btrfs, f2fs), zonefs, RocksDB or other log-structured stores, and zone-aware schedulers. For a conventional SSD with a device FTL, use BlackBox (§4.4).

4.7.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). The guest kernel needs zoned block device support (CONFIG_BLK_DEV_ZONED=y) and must be new enough to drive NVMe ZNS. The Linux 6.8 image from make-guest-image.sh works.

- The host driver must set a 4 KiB NVMe memory page size (CC.MPS), as Linux does. The controller refuses to become ready with another page size.
- Guest tools: `nvme-cli` with the `zns` commands, `blkzone` from util-linux, and `fio` for zoned workloads (`--zonemode=zbd`).

4.7.2 Launch

From `build-femu/`:

```
./run-zns.sh
```

The FEMU device in that script is:

```
-device femu,devsz_mb=4096,namespaces=1,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=32,zns_flash_type=4,femu_mode=3
```

That is a 4 GiB namespace with 16 zones of 256 MiB and 512-byte logical blocks. The variables at the top of `run-zns.sh` set the geometry.

4.7.3 Configuration

ZNS has its own geometry and timing properties, all prefixed `zns_`. The BlackBox geometry and timing properties (`nchs`, `pg_rd_lat` and so on) do not apply. Every property is in the ZNS section of the property reference (§A.1.1).

Zone size and zone count

There is no zone size property. The zone size follows from the namespace size and the geometry:

```
pages per block = namespace size / 16 KiB / (zns_num_ch * zns_num_lun * zns_num_blk)
zone width      = zns_chnls_per_zone, or zns_num_ch when it is 0
zone LUNs       = zns_luns_per_zone, or zns_num_lun when it is 0
zone size       = zone width * zone LUNs * zns_num_plane * pages per block * 16 KiB
zone count      = namespace size / zone size
                = zns_num_ch * zns_num_lun * zns_num_blk
                  / (zone width * zone LUNs * zns_num_plane)
```

So, when the divisions are exact, the zone count does not depend on the namespace size, and the zone size grows with it. Pages per block are rounded down, so an uneven size gives a smaller zone and a leftover. Some configurations:

Settings	Namespace	Zones	Zone size
no `zns_` properties, `devsz_mb=1024`	1 GiB	16	64 MiB
`run-zns.sh`	4 GiB	16	256 MiB
`run-zns.sh` with `zns_num_blk=128`	4 GiB	64	64 MiB
`run-zns.sh` with `zns_chnls_per_zone=2`	4 GiB	64	64 MiB

To get more, smaller zones, raise `zns_num_blk` or narrow the zones with `zns_chnls_per_zone`, which must divide `zns_num_ch`:

```
-device femu,devsz_mb=4096,femu_mode=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=128

-device femu,devsz_mb=4096,femu_mode=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=32,zns_chnls_per_zone=2
```

A narrower zone spans fewer channels, so a single zone has less internal parallelism.

`zns_luns_per_zone` narrows a zone on the LUN axis in the same way. It must divide `zns_num_lun`. With both at 1, each zone owns one die, as in the single-die zone of ConfZNS (SYSTOR '23):

```
-device femu,devsz_mb=4096,femu_mode=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=32,zns_chnls_per_zone=1,zns_luns_per_zone=1
```

`zns_zone_order` sets which axis a zone fills first. `ch` (the default) puts consecutive programs on different channels. `lun` puts them on the LUNs of one channel, so their page transfers take turns on that channel:

```
zns_chnls_per_zone=2, zns_luns_per_zone=2: the slots of one zone

order ch:   slot 0 ch0/LUN0   slot 1 ch1/LUN0   slot 2 ch0/LUN1   slot 3 ch1/LUN1
order lun:  slot 0 ch0/LUN0   slot 1 ch0/LUN1   slot 2 ch1/LUN0   slot 3 ch1/LUN1
```

Linux uses a ZNS namespace only when its zone size is a power of two. Keep `devsz_mb` (or each ZNS namespace's size), `zns_num_ch`, `zns_num_lun`, `zns_num_plane` and `zns_num_blk` powers of two, or check the result with the formula above.

`zns_zone_cap` sets a zone capacity below the zone size, in bytes. The host can write only the first `zns_zone_cap` bytes of each zone.

Logical block size

The namespace starts with 512-byte blocks. `lba_index` picks another of the `nlbaf` formats (512 bytes doubling with each index), so `lba_index=3` gives 4 KiB blocks. ZNS accepts block sizes up to 4 KiB.

```
-device femu,devsz_mb=4096,femu_mode=3,lba_index=3
```

Properties: LBA formats (§A.1.1).

Open and active zone limits

`zns_max_open` and `zns_max_active` set the Maximum Open and Maximum Active Resources. Both default to 0, which means no limit. Neither may exceed the zone count, and `zns_max_open` may not exceed `zns_max_active`:

```
-device femu,devsz_mb=4096,femu_mode=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=128,zns_max_active=16,zns_max_open=8
```

`zns_num_wc` sets the size of the pool of zone write caches. A zone being written takes a cache from the pool; a write goes into it and costs 1 us per 4 KiB page. The cached pages are programmed when the cache fills, or when another zone needs a cache and none is free. The default is `zns_max_open`, or 3 when that is 0.

NAND timing

`zns_flash_type` picks the cell type: 1 SLC, 2 MLC, 3 TLC, 4 QLC (the default) or 5 PLC. SLC, TLC and QLC have built-in read, program and erase times. MLC and PLC have none, so they need `zns_pg_rd_lat`, `zns_pg_wr_lat` and `zns_blk_er_lat`. Those three also override the built-in times for the other types. `zns_cmd_addr_lat`, `zns_pg_xfer_lat` and `zns_status_lat` add a shared channel bus, and `zns_pe_suspend` with `zns_tsusp_ns` lets reads suspend a program or erase. By default each plane runs on its own. `zns_lun_contention=1` makes the planes of one LUN take turns, as on one die. A zone reset then erases the planes of a LUN in one multi-plane operation. By default a reset of a non-empty zone erases every block of the zone. `zns_reset_by_fill=1` erases only the blocks that hold programmed pages, so the reset time and the wear follow the zone's fill. See the timing model (§6.2.6).

A read takes one latency per flash type by default. `zns_page_types=1` gives each read the latency of its page type, from SLC to QLC. A program writes a whole wordline, one page of each type, so it keeps its one latency. A flush after a partial eviction can start in the middle of a wordline and still pays one program time. The

spread between types comes from the shared NAND table. It is scaled so that the mean over a wordline is the read latency of the flash type, which `zns_pg_rd_lat` can set.

```
QLC wordline, read latency 85 us (the default mean)
page type    lower   lower-center   upper-center   upper
shared (us)  59.3    85.3           127.2          169.6
ZNS (us)     45.7    65.7            98.0          130.6    mean 85
```

`zns_ecc_step_ns` adds ECC read time as the block wears, as bbssd `ecc_step_ns` does: one step per 750 erases of the block, up to 4 steps. ZNS counts the wear only, with no retention term. Both are off by default. `zns_ecc_step_ns`, and `zns_pg_rd_lat` with `zns_page_types`, must be at most one hour.

Optional zone features

All of these are off by default.

- **Zone Random Write Area.** Set `zns_zrwa_size` and `zns_zrwafg_size` (in logical blocks) and `zns_zrwa_num` (zones that may hold one at once). A zone opened with the ZRWA flag accepts writes anywhere in a window above the write pointer. The write pointer advances only in whole flush-granularity units: when a write crosses the end of the window, or when the host flushes explicitly. Finishing or resetting the zone returns its ZRWA resource. With all three properties at 0 the namespace advertises no ZRWA.

```
-device femu,devsz_mb=1024,femu_mode=3,zns_zrwa_size=64,zns_zrwafg_size=8,zns_zrwa_num=2
```

- **Conventional zones.** `zns_num_conv_zones=N` makes the first N zones accept writes anywhere inside the zone. They keep no write pointer (reported as all ones) and reject zone management and Zone Append. Leave it at 0 for a Linux guest: the NVMe ZNS command set defines only the sequential-write-required zone type, so Linux rejects a conventional zone, fails the whole zone report with `EINVAL`, and the namespace reports no zones, which leaves it unusable for zoned btrfs, f2fs, zonefs or dm-zoned. To mix random-write and zoned capacity, give the controller a BlackBox namespace and a ZNS namespace (§4.15) instead.
- **Reads across zone boundaries.** By default a read that runs past the end of its zone fails with a zone boundary error. `zns_cross_zone_read=on` allows it and advertises OZCS bit 0, which tells the host it may issue one. Every zone the read spans must still be in a readable state.
- **Zone Append size limit.** `zns_zasl_bs` (default 128 KiB) caps one Zone Append, and must be a power-of-two multiple of 4 KiB, because Identify reports the limit (ZASL) as a power-of-two count of 4 KiB pages; other values are refused, not rounded down. 0 follows `mdts`. The host reads ZASL to size its appends, and a larger append fails with Invalid Field in Command.
- **Zone descriptor extensions.** `zns_zd_ext_size` in bytes, a multiple of 64.
- **Write failures.** `err_write_fail_ppm` fails a fixed share of writes. The zone of a failed write becomes read only and is added to the Changed Zone List log page.
- **Finish Zone Recommended and Zone Active Excursions.** A zone that is written but not finished holds part-programmed blocks, which a real device does not keep open for long. `zns_fzr_sec=N` sets Finish Zone Recommended (FZR) on a zone N seconds of guest time after its first write. `zns_frl_sec=M` reports FRL M in Identify and sets ZOC.ZAE. M seconds after FZR, the controller finishes the zone itself and sets Zone Finished by Controller (ZFC). Both changes enter the Changed Zone List (ZNS 1.4, 5.5 and 5.6).

```
first write --(zns_fzr_sec)--> FZR, FZRTL 00b, changed zone
  host finishes, fills or resets the zone --> FZR cleared, no event
  else --(zns_frl_sec)--> zone Full, ZFC, FZR cleared, changed zone
checks run once a second of guest time; a paused VM does not age zones
```

A finished zone takes no more writes, so the device programs the pages its write cache still holds for the zone:

```
host Finish        --> zone Full --> FTL programs the cache --> command completes
controller finish  --> zone Full --> next command on the FTL thread programs it,
                                     from the time of the finish
Finish of a Full zone --> completes when those programs end
```

A host Finish completes after these programs. After a finish by the controller, later commands wait for the programs on the planes. Writes that reach the FTL thread after the finish of their zone are programmed together, by the next command that is not a read and has another target. A deleted queue or a controller reset drops a Finish command, but its programs still happen. A Reset that it drops before the FTL thread runs it erases nothing, as before. That zone's Finish programs resume after the next Reset that the FTL thread runs for it; a Reset of an Empty zone does not count. A zone with a write still in flight waits for the next check. A zone that writes fill keeps its cache until eviction, as before.

```
-device femu,devsz_mb=1024,femu_mode=3,zns_fzr_sec=60,zns_frl_sec=300
```

- **Reset Zone Recommended.** A real device refreshes the data of a Full zone after some time, to keep it readable. A host Reset makes that work needless, so the device recommends one first. `zns_rzr_sec=N` sets Reset Zone Recommended (RZR) on a zone that has been Full for N seconds of guest time, with RZRTL 00b.

  `zns_rrl_sec=M` reports RRL M in Identify. When RZR has been set for M seconds, the controller refreshes the zone itself and clears RZR. The zone stays Full and ages again from the refresh. Both changes enter the Changed Zone List (ZNS 1.4, 5.4). A change comes at the first check once its time is up, so up to a second late. A check that finds the zones busy waits for the next one.

```
zone Full --(zns_rzr_sec)--> RZR, RZRTL 00b, changed zone
  host resets the zone          --> Empty, RZR cleared, no event
  zone goes Read Only or Offline --> RZR cleared
  else --(zns_rrl_sec)--> refresh: RZR cleared, changed zone,
                          zone stays Full and ages again
without zns_rrl_sec, RZR stays until the host resets the zone
```

Only Full zones get RZR, as the specification defines it. The refresh changes the attribute only: it takes no media time and no wear.

```
-device femu,devsz_mb=1024,femu_mode=3,zns_rzr_sec=600,zns_rrl_sec=3600
```

4.7.4 Use it from the guest

Check that Linux sees a zoned device:

```
sudo nvme list
cat /sys/block/nvme0n1/queue/zoned        # host-managed
cat /sys/block/nvme0n1/queue/nr_zones
cat /sys/block/nvme0n1/queue/chunk_sectors   # zone size in 512-byte sectors
sudo nvme zns id-ns /dev/nvme0n1
```

The model is `FEMU ZMS-SSD Controller [by Misao]` and the serial number starts with `vZNSSD`.

List the zones, with their state and write pointer:

```
sudo nvme zns report-zones /dev/nvme0n1 -d 4
sudo blkzone report /dev/nvme0n1
```

Manage one zone by its start LBA (`-s`), or every zone (`-a`):

```
sudo nvme zns open-zone /dev/nvme0n1 -s 0
sudo nvme zns finish-zone /dev/nvme0n1 -s 0
sudo nvme zns reset-zone /dev/nvme0n1 -s 0
sudo nvme zns reset-zone /dev/nvme0n1 -a
```

Append 4 KiB to zone 0 and let the device pick the LBA:

```
head -c 4096 /dev/urandom > data.bin
sudo nvme zns zone-append /dev/nvme0n1 -s 0 -z 4096 -d data.bin
```

Write sequentially with fio, read the written zones back at random, then reset the zones before the next run:

```
sudo fio --name=zns --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=write --bs=128k --size=1G
sudo fio --name=zr --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=randread --bs=4k --size=1G --runtime=30 --time_based
sudo blkzone reset /dev/nvme0n1
```

With ZRWA configured, open a zone with the ZRWA flag, write inside its window with passthrough write commands (the block layer always writes at the write pointer), then commit the window up to an LBA. With `zns_zrwafg_size=8`, the flush must end on a multiple of 8 blocks:

```
sudo nvme zns open-zone /dev/nvme0n1 -s 0 --zrwaa
sudo nvme zns zrwa-flush-zone /dev/nvme0n1 -l 63
```

Read the Changed Zone List log page (BFh). It needs an explicit namespace:

```
sudo nvme get-log /dev/nvme0 --log-id=0xbf --log-len=4096 --namespace-id=1
```

Changed Zone List

The page is per namespace, which is why the command above names one: nvme-cli otherwise sends the broadcast identifier. It holds an 8-byte count followed by up to 511 zone start LBAs, and lists only zone changes the host did not cause. As the ZNS specification requires, it leaves out changes that follow a Zone Management Send command, writes that open or fill a zone, and the controller closing a zone to free a resource. Reading it without `--rae` clears both the list and the event behind it, once the page reached the host. A read whose transfer fails clears nothing, and a zone that changes during a read keeps its entry for the next one. A notice names its namespace in dword 1 of the completion. Notice events stay masked until the host reads the list of that namespace; a read of another namespace's list leaves them masked.

With `err_write_fail_ppm`, `zns_fzr_sec` and `zns_rzr_sec` unset nothing adds to the list, and it stays empty. With it set, one write in every 1,000,000 / `err_write_fail_ppm` fails and makes its zone read only, the way a controller does when it can no longer program the zone. The failing write completes with Write Fault, later writes to that zone are refused as read only, the zone is added to this list. A host that enabled Zone Descriptor Changed notices (bit 27 of Asynchronous Event Configuration, Set Features 0Bh) and has an Asynchronous Event Request outstanding then gets the notice; without that bit the zone is still listed, but no event is raised. The failures come at a fixed count, so a run repeats exactly. `hw/femu/scripts/zone-aen-probe.c` checks this path and sets bit 27 itself, since Linux leaves it clear. Add `,err_write_fail_ppm=10000` to the device in `run-zns.sh`, copy the probe into the guest, and run it on a fresh device; it prints `ZAEN PASS`:

```
gcc -O2 -o zone-aen-probe zone-aen-probe.c   # inside the guest
sudo ./zone-aen-probe /dev/nvme0 /dev/nvme0n1
```

Do not run `mkfs.ext4` on the namespace: it needs random writes. Use a file system with zoned support, zonefs, or zone-aware applications.

4.7.5 Limits and refusals

Message	Cause and fix
`max_open_zones value 20 exceeds the number of zones 16`	`zns_max_open` above the zone count.
`max_active_zones value N exceeds the number of zones M`	`zns_max_active` above the zone count.
`zns_num_wc value N exceeds the number of zones M, zns_num_wc (N) exceeds the M zones this geometry can have at most`	`zns_num_wc` above the zone count.
`max_open_zones value 10 exceeds max_active_zones 8`	`zns_max_open` above `zns_max_active`.
`zns_chnls_per_zone 3 must divide zns_num_ch 2`	The zone width must divide the channel count.
`zns_luns_per_zone 3 must divide zns_num_lun 4`	The LUNs of a zone must divide the LUN count.
`zns_zone_order must be ch or lun, not 'die'`	An unknown fill order.
`zns_flash_type 2 has no built-in timing; set zns_pg_rd_lat, zns_pg_wr_lat and zns_blk_er_lat`	MLC or PLC without the three latencies.
`zns_flash_type must be in [1, 5]`	A cell type outside SLC to PLC.
`zns_frl_sec needs zns_fzr_sec: an excursion follows Finish Zone Recommended`	Set `zns_fzr_sec` too, or drop `zns_frl_sec`.
`zns_rrl_sec needs zns_rzr_sec: the limit follows Reset Zone Recommended`	Set `zns_rzr_sec` too, or drop `zns_rrl_sec`.
`zoned namespaces need a logical block of 4 KiB or less; lba_index selects 8192 bytes`	`lba_index` picks a block larger than 4 KiB.
`zns geometry gives N pages per block; must be in [1, 65536]`	The namespace is too small or too large for `zns_num_ch * zns_num_lun * zns_num_blk`.
`FEMU zns: N zones exceed M blocks per plane ...`	Raise `zns_num_blk` or the namespace size.
`the device is too small to hold a single zone of N logical blocks`	The geometry gives less than one zone: `zns_num_ch * zns_num_lun * zns_num_blk` is below the zone width times the zone LUNs times `zns_num_plane`. Raise `zns_num_blk` or `zns_num_ch`, or lower `zns_num_plane`, `zns_chnls_per_zone` or `zns_luns_per_zone`.
`zns_zrwafg_size must be between 1 and 65535 when ZRWA is enabled, got 0`	ZRWA needs all three `zns_zrwa*` properties. Other messages check that the window is a multiple of the flush granularity and that the zone capacity is too.
`zns_zasl_bs N must be a power-of-two multiple of the 4096 byte controller page size`	Fix `zns_zasl_bs`.

On a zoned namespace, Dataset Management (deallocate), Write Zeroes, Write Uncorrectable and Copy fail with Invalid Opcode: they would change blocks without going through the zone state machine. Reset the zone instead.

A Zone Management Send command waits until every write to its zones that the controller already accepted has its data copied. So a Reset sent while a write to the zone is in flight on another queue acts after that write: the data and the zone descriptor are correct. The FTL timing model takes each queue's requests in that queue's order, and it can still see the two commands in the other order. An older write can then refill the write cache after the Reset, or a newer write can lose its cache entries to it. That changes later mappings, media counters and read latencies, not data. To keep the model exact, wait for every write to a zone to complete before you reset it, and for the Reset to complete before you write it again.

4.7.6 Verify

1. `cat /sys/block/nvme0n1/queue/zoned` prints `host-managed`.
2. `nr_zones` and `chunk_sectors` match the zone count and size from the formula above.
3. `sudo nvme zns report-zones /dev/nvme0n1 -d 1` shows zone 0 empty; after the fio run the first zones are full and their write pointers moved.
4. From the host, the QMP command `query-femu`[35] with `kind zones` shows the same zone states and write pointers. It needs no guest tools.
5. For a check under a real workload, the `zenfs-crash` suite of `guest-conformance.sh` runs a RocksDB crash test on ZenFS. See ZenFS crash test (§8.2.6).

4.7.7 Troubleshooting

- **`nvme list` shows the device but `nr_zones` is 0, or `blkzone` says `unable to determine zone size`.** The guest kernel cannot drive ZNS. Use a kernel from the requirements table; the image from `make-guest-image.sh` works. Check `dmesg` for the reason. A message with `invalid zone size` means the zone size is not a power of two.
- **No zoned device after setting `zns_num_conv_zones`.** Linux rejects conventional zones and fails the whole zone report. Set it back to 0.
- **The device disappears after a geometry change.** The geometry is checked at realize, so a bad one stops QEMU with a message from the table above. Read the console log (`run-zns.sh` writes `build-femu/log`).
- **I want 4 KiB blocks.** Set `lba_index=3`. Older guides patched `zns.c`; that is no longer needed.
- **I want a different zone size or more zones.** Change the geometry, not the code. See Zone size and zone count (§4.7.3).
- **A large ZNS device fails with `failed to allocate N bytes`.** The namespace lives in host DRAM. A 256 GiB device needs 256 GiB of free host memory.
- **fio fails with an I/O error on the first write.** The zone is full or not at its start. Reset the zones (`blkzone reset /dev/nvme0n1`) and run with `--zonemode=zbd`.

Related issues: #57, #76, #112, #115, #126, #144, #161, #177, #182, #193.

4.7.8 Related pages

- Timing model: ZNS write cache (§6.2.6)
- Architecture: other FTLs (§3.1.5)
- Multiple namespaces (§4.15), to mix zoned and conventional namespaces
- Device property reference: ZNS (§A.1.1)
- QMP query-femu[36], to read the zones from the host

4.8 ZNS: the zoned namespace model

This chapter describes how FEMU implements the NVMe Zoned Namespace (ZNS) command set: the zone state machine, how zones are laid out on the emulated NAND, where each cost is charged, and what every property changes. To run a ZNS device and use it from a guest, read the ZNS mode guide (§4.7) first; this page explains what happens inside.

[35] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md`
[36] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md`

4.8.1 Purpose

A zoned namespace splits its logical blocks into zones. The host writes each zone sequentially at the zone's write pointer, or with Zone Append, and resets a zone before it writes it again. The device keeps no page-level remapping for overwrites and runs no garbage collection; reclaiming space is the host's job. FEMU's ZNS mode gives you:

- the zone state machine of the ZNS specification, with open and active zone limits, Zone Append, Zone Random Write Area (ZRWA), conventional zones, zone descriptor extensions and the Changed Zone List log page;
- a zone-to-NAND layout that decides which channels, LUNs and planes a zone occupies, so zone width changes internal parallelism;
- a per-zone write cache and NAND read, program and erase timing, so a zoned workload sees realistic latency.

ZNS is selected with `femu_mode=3` for the whole controller, or per namespace with `znssd` in `namespace_modes` (see namespaces (§4.16.5)).

4.8.2 Place in the hierarchy

```
guest: blkzone, nvme zns, fio --zonemode=zbd, zoned btrfs/f2fs, zonefs
  |
  | NVMe queues (Read, Write, Zone Append, Zone Mgmt Send/Receive)
  v
poller thread  -- nvme-io.c: nvme_io_cmd() --> ns->ext_ops.io_cmd = zns_io_cmd()
  |               zns.c: zone checks, write pointer, state machine,
  |                      data copy to the DRAM backend (backend_rw)
  | to_ftl ring
  v
FTL thread     -- femu.c: femu_ftl_process_req() --> zftl.c: zns_ftl_process_req()
  |               write cache, L2P table, zone erase, NAND timing
  v
NAND media     -- nand/: nand_media_op(), plane-guarded timelines,
                  optional channel bus and program/erase suspend
  |
  | latency added to req->expire_time; to_poller ring
  v
poller completes the command when the clock passes expire_time
```

The data itself always lives in the controller's DRAM backend. The NAND model holds no data: it tracks where each 4 KiB logical page would sit and how long each operation takes. The general request path is described in architecture (§3.1.3).

4.8.3 Data structures

Structure	File	Fields that matter
`NvmeZone`	`zns/zns.h`	`d` (the zone descriptor the host reads: type `zt`, state `zs`, attributes `za`, `zcap`, `zslba`, `wp`), `w_ptr` (the write pointer used to place writes), list `entry`
`NvmeNamespace` (zoned part)	`nvme.h`	`zone_array`, `num_zones`, `zone_size` and `zone_capacity` in logical blocks, `zone_size_log2`, the lists `exp_open_zones`, `imp_open_zones`, `closed_zones`, `full_zones`, the counters `nr_open_zones` and `nr_active_zones` with their limits, the ZRWA fields `zrwa_size`, `zrwafg_size`, `zrwa_num`, `zrwa_avail`, `zd_extensions`, `id_ns_zoned`

Structure	File	Fields that matter
`struct zns_ssd`	`zns/zns.h`	NAND geometry (`num_ch`, `num_lun`, `num_plane`, `num_blk`, `num_page`), the `ch -> fc -> plane -> blk` tree with per-block `page_wp`, `maptbl` (L2P per 4 KiB page), `cache` (write caches), `program_unit`, `stripe_unit`, `chnls_per_zone`, `zone_wp_slot[]`, `zone_wp_plane[]`, `media`, `zone_lock`, the changed zone list
`struct zns_write_cache`	`zns/zns.h`	`sblk` (the zone it serves), `used`, `cap`, `lpns[]`

There are two write pointers per zone. `w_ptr` moves when a write is accepted, before its data is copied, so that two writes on different pollers, or two appends, never get the same LBA. `d.wp` is the pointer the host sees in a zone report; it moves when the write is finalized after the copy. Without a ZRWA they agree once every accepted write has finished.

4.8.4 Zone geometry

Size, capacity and count

There is no zone size property. `zns_init_zone_cap()` derives it from the namespace size and the `zns_` geometry, with a fixed 16 KiB NAND page (`ZNS_PAGE_SIZE`):

```
pages per block  = namespace bytes / 16 KiB / (zns_num_ch * zns_num_lun * zns_num_blk)
zone width       = zns_chnls_per_zone, or zns_num_ch when it is 0
zone LUNs        = zns_luns_per_zone, or zns_num_lun when it is 0
zone size        = zone width * zone LUNs * zns_num_plane * pages per block * 16 KiB
zone count       = namespace bytes / zone size          (rounded down)
zone capacity    = zns_zone_cap, or the zone size when it is 0
```

The namespace exposes `zone count * zone size` blocks (NSZE). Bytes past the last whole zone are not exposed. Each zone's capacity is the same; a zone accepts writes only below `ZSLBA + ZCAP` and reads up to `ZSLBA + ZSZE`. The mode guide (§4.7.3) has a table of zone sizes for common settings.

Zone layout over NAND

Zone `i` occupies one block index in a set of channels. With full-width zones (`zns_chnls_per_zone` unset) the zone owns block `i` on every channel, LUN and plane:

```
full width: zns_num_ch=4, zns_num_lun=2, zns_num_plane=2

           ch0         ch1         ch2         ch3
        LUN0 LUN1   LUN0 LUN1   LUN0 LUN1   LUN0 LUN1
 pl0     b0   b0     b0   b0     b0   b0     b0   b0     <- zone 0 = block 0
 pl1     b0   b0     b0   b0     b0   b0     b0   b0        everywhere
 pl0     b1   b1     b1   b1     b1   b1     b1   b1     <- zone 1 = block 1
 pl1     b1   b1     b1   b1     b1   b1     b1   b1
 ...
```

A narrower zone spans `zns_chnls_per_zone` channels. The channels form `groups = zns_num_ch / zns_chnls_per_zone` groups, and the zones that share one block index sit in different groups:

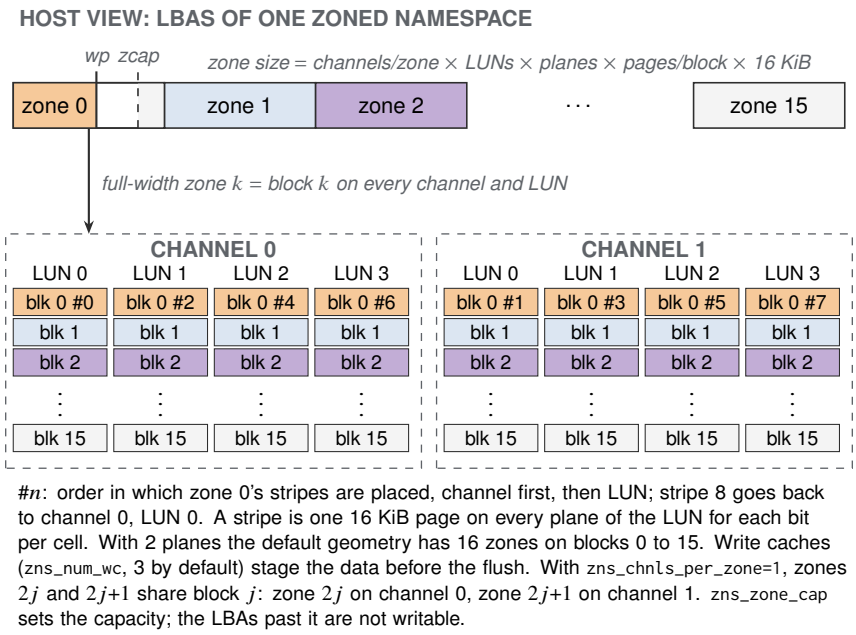


Figure 16: How zones sit on NAND with the default geometry: zone k is block k on every channel and LUN, and its pages are placed channel first, then LUN.

```
zns_num_ch=4, zns_chnls_per_zone=2 -> 2 channel groups

           group 0 (ch0, ch1)      group 1 (ch2, ch3)
 block 0   zone 0                  zone 1
 block 1   zone 2                  zone 3
 block 2   zone 4                  zone 5

 zone i: channels (i % groups) * width .. + width - 1, block i / groups
```

So narrowing the zones doubles the zone count for each halving of the width, and each zone has fewer channels to program in parallel.

`zns_luns_per_zone` splits the LUNs of each channel group the same way. The groups are then pairs of a channel group and a LUN group. `zns_zone_span()` in `zftl.c` computes them, and the write, reset and erase paths all call it:

```
ch_groups = zns_num_ch / width       lun_groups = zns_num_lun / zone LUNs
groups    = ch_groups * lun_groups
g         = zone % groups
channels  = (g % ch_groups) * width     .. + width - 1
LUNs      = (g / ch_groups) * zone LUNs .. + zone LUNs - 1
block     = zone / groups

zns_num_ch=2, zns_num_lun=2, width 1, zone LUNs 1 -> 4 groups

         ch0      ch1
 LUN0    z0, z4   z1, z5      block 0: z0..z3
 LUN1    z2, z6   z3, z7      block 1: z4..z7
```

The geometry check refuses a configuration with more zones than `zns_num_blk * groups`, since every zone needs its own block.

Placement inside a zone

`get_new_page()` in `zftl.c` picks the next location from the zone's slot counter `zone_wp_slot[zone]`, through `zns_zone_die()`. A slot is one (channel, LUN) pair of the zone. Slots go channel first, then LUN, unless `zns_zone_order=lun`:

```
order ch (default):  ch  = first channel + s % width
                     lun = first LUN + (s / width) % zone LUNs
order lun:           lun = first LUN + s % zone LUNs
                     ch  = first channel + (s / zone LUNs) % width

one slot programs, on each plane of that LUN, zns_flash_type pages of
16 KiB (4 logical 4 KiB pages each):

  program unit = 16 KiB * zns_flash_type * zns_num_plane
  stripe unit  = program unit * zone width * zone LUNs
```

With `run-zns.sh` (8 channels, 4 LUNs, 2 planes, QLC) the program unit is 128 KiB and the stripe unit 4 MiB. Each block keeps its own page write pointer `page_wp`; a zone reset sets it, the zone's slot counter and the zone's plane cursor back to zero.

A flush of a full write cache uses every plane of each slot. A flush of a partial cache (an eviction) can stop after the first planes of a slot. The zone then keeps that position in `zone_wp_plane[zone]`, and its next flush continues on the first free plane of the same slot. The slot counter moves only after all planes of the slot are used:

```
eviction 1:  slot 0  [pl 0: data] [pl 1: free ]   zone_wp_plane = 1
eviction 2:  slot 0  [pl 0: data] [pl 1: data ]   slot -> 1, zone_wp_plane = 0
```

Without the cursor, every flush started at plane 0, so plane 0's block ran past its last page while the other planes stayed empty.

A partial flush still programs a whole 16 KiB page for each group of up to 4 logical pages it holds. A workload that evicts caches with fewer than 4 logical pages, or a `zns_flash_type` that does not divide the pages of a block, can still use more pages than a block has. FEMU does not refuse or redirect those writes.

4.8.5 Zone state machine

The states and transitions below are the ones `zns.c` implements. Every transition the host drives goes through `zns_assign_zone_state()`, which also moves the zone between the per-state lists.

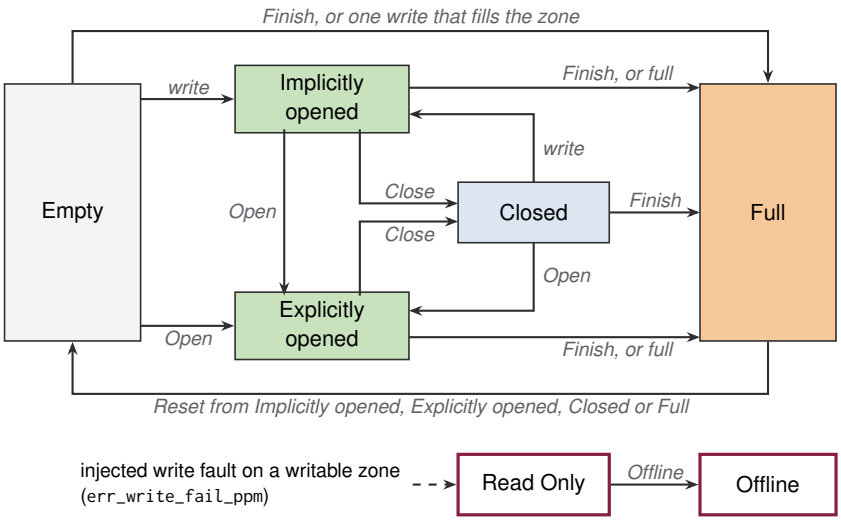


Figure 17: The zone state machine in FEMU's ZNS mode: writes open zones implicitly, Open/Close/Finish/Reset move them explicitly, a zone becomes Full when its write pointer reaches the capacity, and only an injected write fault makes it Read Only.

```
+-------+   write, append
| Empty |-------------------------------------------------+
+-------+                                                 |
 |  |  |      Open      +-----------------+   Open   +-------v---------+
 |  |  +--------------->| Explicitly Open |<---------| Implicitly Open |
 |  |                   +-----------------+          +-----------------+
 |  |                     |           ^                |           ^
 |  |               Close |           | Open    Close, |           | write
 |  |                     v           |     auto close v           |
 |  |  Set ZD Ext     +-----------------------------------------------+
 |  +---------------->|                   Closed                      |
 |                     +-----------------------------------------------+
 |  Finish             +-----------------------------------------------+
 +------------------->|                    Full                       |
                      +-----------------------------------------------+

also to Full: a write that reaches ZCAP (from Empty, either Open state or
Closed), or Finish from either Open state or Closed
Reset: Explicitly Open, Implicitly Open, Closed, Full --> Empty
controller: injected write fault --> Read Only --(Offline action)--> Offline
```

From	Event	To	Code
Empty	write or Zone Append	Implicitly Open	`zns_auto_open_zone()`, `zns_advance_zone_wp()`
Empty, Implicitly Open, Closed	Open Zone	Explicitly Open	`zns_open_zone()`
Empty	Set Zone Descriptor Extension	Closed (extension valid)	`zns_set_zd_ext()`

From	Event	To	Code
Implicitly or Explicitly Open	Close Zone	Closed	`zns_close_zone()`
Implicitly Open	another zone needs an open resource at the open limit	Closed	`zns_auto_transition_zone()`
Closed	write	Implicitly Open	`zns_advance_zone_wp()`
Empty, Open, Closed	Finish Zone	Full	`zns_finish_zone()`
Empty, Open, Closed	write reaches the zone capacity	Full	`zns_finalize_zoned_write()`
Open, Closed, Full	Reset Zone	Empty	`zns_reset_zone()`
any but Read Only	injected write fault on the zone	Read Only	`zns_nvme_rw()`
Read Only	Offline Zone	Offline	`zns_offline_zone()`

Offline is final, and Read Only leaves only through Offline. Open, Close, Finish and Reset refuse both with Invalid Zone State Transition. Reads work in every state except Offline. Writes are refused in Full (Zone Is Full), Read Only (Zone Is Read Only) and Offline (Zone Is Offline).

Reset and Offline also deallocate the zone's slice of the DRAM backend (`zns_deallocate_zone()`), so a read of a reset zone returns zeros, as the specification requires for Empty and Offline zones.

Zone state lives only in QEMU memory. It survives a controller reset and a guest reboot, and is lost when QEMU exits. `zns_ns_shutdown()`, which moves open and closed zones to Closed, or to Empty when they hold no data and no descriptor extension, runs only when the device is removed.

Open and active resources

`zns_max_open` and `zns_max_active` set the limits; 0 means no limit, and then FEMU does not count. An active zone is one that is open or closed; an open zone is implicitly or explicitly open.

```
write to an Empty zone, or Open of an Empty zone:
  1. need an active resource      -> else Too Many Active Zones, nothing changes
  2. at the open limit? close the oldest Implicitly Open zone
  3. need an open resource        -> else Too Many Open Zones
write to a Closed zone: steps 2 and 3 only
```

Only implicitly open zones are closed to make room, oldest first. If every open zone is explicitly open, the command fails with Too Many Open Zones. Open with Select All opens every closed zone or none: it fails up front if the closed zones would not fit under the open limit.

4.8.6 Commands

`zns_io_cmd()` handles Read, Write, Zone Append, Zone Management Send and Zone Management Receive. Other I/O opcodes go through the common path in `nvme-io.c`, with these zoned rules:

- Compare is checked like a Read (`zns_check_compare()`). Compare and Verify take the NAND read time of a Read, also after a miscompare.
- Dataset Management, Write Zeroes, Copy and Write Uncorrectable fail with Invalid Opcode on a zoned namespace: they would change blocks without going through the state machine. The command effects log for CSI 2 leaves them out, and Identify Namespace reports no Copy limits (MSSRL, MCL and MSRC are zero) even when `oncs` turns Copy on.
- Verify runs the common path without zone state or boundary checks.

- Flush behaves as on any namespace. I/O Management Send and Receive fail with Invalid Command Opcode, since log 05h lists them for NVM only.

Read and Write

A Write must start exactly at `w_ptr` and end at or below the zone capacity, or it fails with Zone Invalid Write or Zone Boundary Error. A Read must stay inside its zone unless cross-zone reads are on; see below (§4.8.7).

Zone Append

Zone Append names the zone by its ZSLBA. Under `zone_lock` the poller takes `w_ptr` as the append's LBA and advances the pointer in one step, so appends on different queues never overlap; the data is copied afterwards, outside the lock. The completion carries the LBA where the data landed. An append fails with Invalid Field when it does not name a zone start or exceeds the Zone Append Size Limit, and with Zone Boundary Error when it would cross the zone capacity.

The limit comes from `zns_zasl_bs` in bytes. At controller enable, `zns_start_ctrl()` converts it to ZASL, a power of two in units of the 4 KiB controller page, which Identify Controller for CSI 2 reports. With `zns_zasl_bs=0` Identify reports ZASL 0, and appends are held to MDTS instead. Realize refuses a value that is not a power-of-two multiple of 4 KiB, rather than round it down.

Zone Management Send

Action	Select All acts on	Notes
Close (01h)	open zones	
Finish (02h)	open and closed zones	moves `w_ptr` and `wp` to the capacity; does not program the write cache
Open (03h)	closed zones, all or none	bit 9 of CDW13 also allocates a ZRWA (single zone only)
Reset (04h)	open, closed and full zones	descriptor changes on the poller; the erase runs on the FTL thread
Offline (05h)	read-only zones	
Set Zone Descriptor Extension (10h)	not allowed	data is staged first and copied only if the zone is Empty
ZRWA Flush (11h)	not allowed	see ZRWA (§4.8.7)

A single-zone action must name the zone's start LBA, except ZRWA Flush, which names a boundary inside the zone. Every action runs under the namespace's `zone_lock`.

Zone Management Receive

Report Zones (00h) and Extended Report Zones (01h, only with `zns_zd_ext_size` set) return a 64-byte header with the matching zone count, then one 64-byte descriptor per zone, followed in the extended report by that zone's extension. The filter field takes the states 0 to 7, plus 9 for zones with the Finished by Controller, Finish Recommended or Reset Recommended attribute; FEMU never sets those attributes, so filter 9 always reports none. The Partial Report bit limits the count to the descriptors that fit. The write pointer field is all ones for Full, Read Only and Offline zones and for conventional zones. The report is built under `zone_lock`, so it is a consistent snapshot.

4.8.7 Optional zone features

Zone Random Write Area (ZRWA)

A ZRWA lets the host write anywhere in a window above the write pointer and commit the window later. `zns_zrwa_size` (ZRWAS) and `zns_zrwafg_size` (ZRWAFG, the flush granularity) are in logical blocks; `zns_zrwa_num` is how many zones may hold a ZRWA at once (NUMZRWA).

```
       w_ptr                      w_ptr + ZRWAS             w_ptr + 2*ZRWAS
         |<------- ZRWA window ------->|<-- implicit flush area -->|
zone: [..committed..|.................|...........................|....]
                    ^ a write may start anywhere in here ---------^
a write that ends past the window moves w_ptr up by whole ZRWAFG units
ZRWA Flush to LBA x (inside the window, x - w_ptr + 1 a multiple of
ZRWAFG) sets w_ptr = x + 1
```

- A ZRWA is allocated by Open Zone, or by Set Zone Descriptor Extension, with CDW13 bit 9, on an Empty zone while `zrwa_avail` is non-zero; otherwise the command fails with Invalid Zone Operation or No ZRWA Resources.
- Writes must start in `[w_ptr, w_ptr + 2 * ZRWAS)` and stay below the zone capacity. Zone Append to a ZRWA zone fails with Invalid Zone Operation.
- When `w_ptr` reaches the capacity the zone becomes Full and the resource goes back. Finish, Reset and Offline also return it.
- Identify Namespace for CSI 2 sets OZCS bit 1 (ZRWASUP), NUMZRWA (zero-based), ZRWAS, ZRWAFG and ZRWACAP bit 0 (explicit flush).

Realize refuses a ZRWA configuration where ZRWAS or ZRWAFG exceed 65535, ZRWAS is not a multiple of ZRWAFG, the zone capacity is not a multiple of ZRWAFG, `zns_zrwa_num` is 0, or the granularity or count is set without a window.

Conventional zones

`zns_num_conv_zones=N` makes the first N zones conventional (zone type 1). A conventional zone accepts writes anywhere below ZSLBA + ZCAP and keeps no write pointer. Its state changes only through an injected write fault, which can take it to Read Only like any zone. It rejects Zone Append (Invalid Field), single-zone Open, Close, Finish, Reset, Offline and Set Zone Descriptor Extension (Invalid Zone State Transition), and ZRWA Flush (Invalid Zone Operation). Zone reports describe it as usual. N above the zone count is lowered to the zone count. The ZNS specification defines only sequential-write-required zones, so Linux refuses a namespace that reports a conventional zone; the mode guide (§4.7.3) explains the consequence.

Reads across zone boundaries

`zns_cross_zone_read=on` sets OZCS bit 0 (Read Across Zone Boundaries). A read that runs past its zone is then accepted if every zone it touches is readable (not Offline). Without it, such a read fails with Zone Boundary Error.

Zone descriptor extensions

`zns_zd_ext_size` sets a per-zone extension in bytes, a multiple of 64 and at most 255 units of 64. Identify reports it in LBAFE.ZDES. Setting an extension takes an Empty zone to Closed with the Zone Descriptor Extension Valid attribute, and needs an active resource.

Changed Zone List and Zone Descriptor Changed notices

The Changed Zone List log page (BFh) lists zones whose descriptor changed for a reason the host did not cause. The ZNS specification excludes every change that follows a Zone Management Send, a write that opens or fills a zone, and the controller closing a zone to free a resource. In FEMU the only change left is the injected write fault: with `err_write_fail_ppm` set, one write in `1000000 / err_write_fail_ppm` completes with Write Fault and its zone becomes Read Only. That zone's ZSLBA is added to the list by `zns_record_changed_zone()`.

```
failed write --> zone Read Only --> changed_zones[] (dedup, up to 511)
                                          |
                    AEC bit 27 set? ---+--> Asynchronous Event Notice,
                    (Set Features 0Bh)      Zone Descriptor Changed,
                                            log page BFh, this NSID
Get Log Page BFh, NSID = the zoned namespace:
  8-byte count, then 8-byte ZSLBAs (count FFFFh and no entries on overflow)
  read without RAE: list and pending event cleared
```

- The list is per namespace. The command must name the zoned namespace's NSID: 0, FFFFFFFFh or a non-zoned namespace fail. The offset must be a multiple of 8.
- Identify Controller advertises OAES bit 27 (ZDCN) whenever a zoned namespace exists. The notice itself is sent only when the host has set bit 27 of Asynchronous Event Configuration; the list is kept either way. The mode guide (§4.7.4) notes that Linux leaves the bit clear, which is why `scripts/zone-aen-probe.c` sets it itself.
- The supported log pages list (log 00h) shows BFh for CSI 2 only.

4.8.8 Threads, locks and where latency is charged

Work	Thread	Lock
Zone checks, auto open, `w_ptr` advance, append placement	poller of the submission queue	`zns->zone_lock`
Data copy between guest memory and the DRAM backend	poller	none (the LBA range was reserved under the lock)
`d.wp` update, Full transition, injected fault	poller	`zone_lock`
Zone Management Send and Receive, reset state change	poller	`zone_lock`
Changed Zone List read	admin path	`zone_lock`
Write cache, L2P table, block page pointers, erase, NAND timing	the controller's FTL thread	single owner, no lock

One FTL thread serves every ZNS, bbssd and CSD namespace of a controller (`femu_ftl_thread()` in `femu.c`). It returns a latency for each request, which is added to `expire_time`; the poller posts the completion once the clock passes it. See timing model (§6.2.1).

What each command is charged, in `zftl.c`:

Command	Cost
Write, Zone Append	the larger of two figures: 1 us for each 4 KiB logical page put into the zone's write cache, summed over the pages added since the request's last cache flush, and the program time of any cache this write caused to be programmed: an evicted cache, or this zone's own cache when a page of this write found it full. The write that fills a cache is not charged for programming it; the next write to the zone is
Read	the largest NAND read time over the 4 KiB pages it touches that have been programmed; pages still in a write cache, or never written, cost nothing
Zone Reset	the largest erase time over the blocks of each reset zone; zones reset by one command share planes, so the command ends with its last erase. An Empty zone has no erase. With `zns_reset_by_fill`, only blocks that hold a programmed page are erased, so a zone whose data is all still in its write cache costs nothing
Open, Close, Finish, Report, Offline	nothing

Write cache

A write cache holds the logical page numbers of one zone's partial stripe; its capacity is one stripe unit (`stripe unit / 4 KiB` pages). There are `zns_num_wc` caches, or `zns_max_open` when that is 0, or 3 when both are 0.

```
write to zone z
  cache bound to z?  yes -> append LPNs
                     no  -> take an empty cache, else evict the fullest
                            cache (program it now, cost charged to this
                            write), bind it to z
  for each page:     cache already full? program it first: one program
                     unit per slot, each plane of the slot's LUN one NAND
                     program op, L2P updated; then add the page
Zone Reset of z: drop z's cache without programming it
```

Fewer caches than zones written at once makes writes evict each other's caches and pay a program each time. That is why the default follows `zns_max_open`. The timing model (§6.2.6) puts this in context with the other modes.

NAND timing

`zns_nand_media_init()` hands the timing to the shared NAND media layer (see NAND timing (§3.5)) with:

- one latency per cell type for read, program and erase, from the `zns_flash_type` row of the built-in table in `zns/zns.h` and `nand/nand.h`, replaced by `zns_pg_rd_lat`, `zns_pg_wr_lat` and `zns_blk_er_lat` when set; MLC and PLC have no built-in row and need all three;
- the array held per plane only (`NAND_CONTEND_PLANE_ONLY`) by default: operations on different planes of one LUN overlap, operations on one plane queue;
- with `zns_lun_contention` non-zero, the LUN and the plane both (`NAND_CONTEND_LUN_AND_PLANE`): the planes of one LUN take turns, as on one die, and `zns_zone_reset()` erases each LUN's planes with one `nand_media_multiplane()` call instead of one erase per plane;
- with `zns_reset_by_fill` non-zero, `zns_zone_reset()` skips each block whose `page_wp` is 0, before it clears the block state; the erase time, the block's `erase_cnt` and the device's `total_erases` (SMART Percentage Used) then follow the zone's fill;
- a shared channel bus only if `zns_cmd_addr_lat`, `zns_pg_xfer_lat` or `zns_status_lat` is non-zero;
- program and erase suspend for reads if `zns_pe_suspend` is non-zero, with `zns_tsusp_ns` charged per preempting read.

4.8.9 Parameters

All ZNS properties are on the `femu` device and are listed with types and defaults in the ZNS section of the property reference (§A.1.1). The geometry and limits are controller-wide: every zoned namespace of a controller uses the same `zns_` values with its own size.

Group	Properties	Interactions
Geometry	`zns_num_ch`, `zns_num_lun`, `zns_num_plane`, `zns_num_blk`, `zns_chnls_per_zone`	Set zone size and count with the namespace size. `zns_num_ch` 1 to 128 and `zns_num_plane` 1 to 8 (PPA field widths). Pages per block must come out in 1 to 65536. `zns_chnls_per_zone` must divide `zns_num_ch`.
Zone shape	`zns_zone_cap`, `zns_num_conv_zones`, `zns_zd_ext_size`	`zns_zone_cap` between one logical block and the zone size.
Limits	`zns_max_open`, `zns_max_active`	Each at most the zone count; open at most active when both are set. `zns_max_open` also sets the default cache count.
Write cache	`zns_num_wc`	At most the zone count.
ZRWA	`zns_zrwa_size`, `zns_zrwafg_size`, `zns_zrwa_num`	All three together, or none.
Reads, appends	`zns_cross_zone_read`, `zns_zasl_bs`	`zns_zasl_bs` 0 follows `mdts`.
Timing	`zns_flash_type`, `zns_pg_rd_lat`, `zns_pg_wr_lat`, `zns_blk_er_lat`, `zns_cmd_addr_lat`, `zns_pg_xfer_lat`, `zns_status_lat`, `zns_pe_suspend`, `zns_tsusp_ns`, `zns_lun_contention`, `zns_reset_by_fill`	Cell type also multiplies the program unit. All latencies are in ns and must not be negative.
Shared with other modes	`devsz_mb`, `namespaces`, `namespace_sizes`, `namespace_modes`, `lba_index`, `nlbaf`, `mdts`, `err_write_fail_ppm`	`lba_index` must select a block of 4 KiB or less.

The BlackBox geometry and timing properties (`nchs`, `pg_rd_lat` and so on) do not apply to a zoned namespace. The controller also refuses to become ready unless the host selects a 4 KiB memory page (CC.MPS).

A device with 64 MiB zones of half width on an 8-channel geometry:

```
-device femu,devsz_mb=2048,femu_mode=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=32,zns_chnls_per_zone=4,zns_max_open=8,zns_max_active=16
```

4.8.10 Statistics and outputs

Output	What it shows	Source
Zone report (Zone Management Receive)	state, write pointer, capacity and attributes of each zone	`zns_zone_mgmt_recv()`
Changed Zone List (BFh)	zones taken read only by injected write faults	`zns_changed_zone_list()`
SMART / Health (02h)	host data units and commands read and written, counted on the poller for every mode; Media Errors includes the injected ZNS write faults; Percentage Used counts the block erases of Zone Reset against `pe_cycles_rated`, or the rating of `zns_flash_type`	`nvme-io.c`, `zns_media_errors()`, `zns_percentage_used()`
Vendor log C0h	not filled for zoned namespaces: ZNS runs no garbage collection, so there is no write amplification to report	log pages and counters (§B.1)

Output	What it shows	Source
QEMU log	the ZNS geometry, program unit, stripe unit and cache count at realize (`[FEMU] Log:` lines)	`zns_init_params()`

4.8.11 Validation

Check	What it covers
qtest cases in `hw/femu/tests/qtest/femu-test.c`	`zone-reset`, `zns-reset-fill-off`, `zns-reset-fill-on`, `zns-reset-fill-lun`, `zone-open-limits`, `zone-active-limit`, `zone-append-parallel`, `zoned-append-limit`, `zoned-append-mdts0`, `zone-bad-dptr`, `zone-report-length`, `mdts0-zone-report`, `zoned-format-index`, `zoned-compare`, `compare-verify-media-zns`, `zone-change-notice`, `log-contents-zoned`, `sgl-zoned`, `dma-error-zoned`, `ns-mgmt-unavailable-zoned`, `zrwa-reopen`, `zrwa-write-bounds`, `zrwa-odd-granule`, `zrwa-zd-ext`, `io-fuzz-zoned`
Documentation examples	each tagged ZNS example starts under qtest, identifies the controller and writes and reads one block
`hw/femu/scripts/zone-aen-probe.c`	in-guest, manual: Zone Descriptor Changed notice end to end with `err_write_fail_ppm`
Guest tools	`blkzone`, `nvme zns`, fio `--zonemode=zbd` on a Linux guest, as in the mode guide (§4.7.6)

The NAND timing goes through the same media layer as bbssd, whose timing math has unit tests in `hw/femu/tests/unit/test-nand-media.c`.

4.8.12 Limits

- No zone attribute is ever set by the controller: no Finish Recommended, Reset Recommended or Finished by Controller, and ZOC is 0 (no variable zone capacity, no zone active excursions).
- The only zone change the host does not cause is the injected write fault. Zones never go read only or offline because of wear.
- No wear, retention or read disturb model, and no garbage collection.
- Finish does not program the zone's write cache, and Close does not either. A read of data that is still in a write cache costs nothing.
- One latency per cell type: no page-type (LSB/MSB) difference and no program/erase cycle effect.
- The zone geometry properties are shared by all zoned namespaces of a controller.
- Zone state and data exist only in QEMU memory and are lost when QEMU exits. A large namespace needs that much free host memory.
- Namespace Management cannot create zoned namespaces, and metadata and Streams are refused on them (see namespaces (§4.16)).

4.8.13 How to extend it

- A new zone management action: add a case to `zns_zone_mgmt_send()` and an `op_handler_t` that `zns_do_zone_op()` can apply to one zone or a list. Return resources through the same ladder the existing handlers use (`zns_aor_dec_open()`, `zns_aor_dec_active()`, `zns_zrwa_release()`).
- A new reason for a zone to change on its own: change the state, then call `zns_record_changed_zone()`. Never call it from a host-driven transition; the specification excludes those from the log.

- New timing: extend `zns_nand_media_init()` and `zns_advance_status()` in `zftl.c`; the media layer does the plane and channel bookkeeping. Work that touches the L2P table, block page pointers or timelines must run on the FTL thread, as Zone Reset does through `req->zone_resets`.
- A new property: declare it in `femu.c` next to the other `zns_` ones, add the field to `ZNSCtrlParams`, copy it per namespace in `nvme_init_namespaces()` if the zoned code reads it from the namespace, validate it in `zns_check_params()` or `zns_init_zone_geometry()`, give it a description and topic so `gen-property-docs.py` documents it, and add a qtest case.

4.8.14 Source map

File	What is there
`hw/femu/zns/zns.h`	zone and zoned Identify structures, `struct zns_ssd`, write cache, built-in cell timings, open and active counters
`hw/femu/zns/zns.c`	geometry checks, zone state machine, Read/Write/Append, Zone Management Send and Receive, ZRWA, Changed Zone List, Identify, mode registration (`nvme_register_znssd()`)
`hw/femu/zns/zftl.c`	zone-to-NAND placement, write cache, L2P table, zone reset and erase, media layer setup, `zns_ftl_process_req()`
`hw/femu/zns/zftl.h`	`zns_zone_reset()`, debug and assert macros
`hw/femu/femu.c`	`zns_` property definitions, per-namespace copies of the zone limits, FTL thread dispatch
`hw/femu/nvme-io.c`	opcode filtering for zoned namespaces, host counters
`hw/femu/nvme-admin.c`	Identify Controller CSI 2 (ZASL), OAES.ZDCN, supported log pages, SMART media errors
`hw/femu/nand/`	shared NAND media timing

4.8.15 Related pages

- ZNS mode guide (§4.7)
- Namespaces and subsystems (§4.16)
- Timing model (§6.2)
- Property reference: ZNS (§A.1.1)

4.9 Open-Channel SSD (OCSSD)

OCSSD mode (`femu_mode=0`) emulates an Open-Channel SSD, also called a "white-box" SSD. The device has no FTL. The host sees the NAND geometry (channels or groups, LUNs or parallel units, blocks or chunks, pages) and addresses physical pages directly with vector read, write and erase commands. Mapping, garbage collection and wear levelling run in the host. `lver=2` (the default) selects Open-Channel 2.0, `lver=1` selects 1.2.

Use it for host-side FTL research: LightNVM and pblk in older Linux kernels, or SPDK.

The guest kernel must be older than 5.15. Linux removed LightNVM, its Open-Channel driver, in 5.15. A newer guest kernel, including the Linux 6.8 image from `make-guest-image.sh`, sees the controller but has no driver for the Open-Channel interface. On a newer kernel, drive the device from user space with SPDK.

4.9.1 Requirements

- Guest kernel: older than 5.15, with LightNVM (`CONFIG_NVM`) and pblk (`CONFIG_NVM_PBLK`) built. The minimum version for each Open-Channel release is in the mode table (§4.1.4) and requirements.md (§2.1.2).

guest-image.md (§2.3.3) says how to get such a guest.
- Guest tools: nvme-cli 1.x for `nvme lnvm`. nvme-cli 2.x no longer has the `lnvm` commands.
- Host: nothing beyond the common requirements. `run-whitebox.sh` uses a 4 GiB device and a 4 GiB guest.

4.9.2 Launch

From `build-femu/`:

```
./run-whitebox.sh
```

The FEMU device in that script, for Open-Channel 2.0, is:

```
-device femu,devsz_mb=4096,namespaces=1,lver=2,lmetasize=16,nlbaf=5,lba_index=3,mdts=10,lnum_ch=2,lnum_lun=4,⌋
    lnum_pln=2,lsec_size=4096,lsecs_per_pg=4,lpgs_per_blk=512,femu_mode=0
```

For Open-Channel 1.2, set `OCVER=1` at the top of `run-whitebox.sh`, which changes `lver`:

```
-device femu,devsz_mb=4096,namespaces=1,lver=1,lmetasize=16,nlbaf=5,lba_index=3,mdts=10,lnum_ch=2,lnum_lun=4,⌋
    lnum_pln=2,lsec_size=4096,lsecs_per_pg=4,lpgs_per_blk=512,femu_mode=0
```

`run-whitebox.sh` lets you change `ssd_size`, `num_channels` and `num_chips_per_channel`. Keep the other values unless you have a reason to change them.

The same Open-Channel 2.0 device is the preset `hw/femu/scripts/configs/ocssd.conf` (set `lver = 1` in it for 1.2). `ssd-config.sh` (§5.11) expands it to:

```
-device femu,id=nvme0,lver=2,devsz_mb=4096,namespaces=1,lnum_ch=2,lnum_lun=4,lnum_pln=2,lpgs_per_blk=512,⌋
    lsecs_per_pg=4,lsec_size=4096,lmetasize=16,nlbaf=5,lba_index=3,mdts=10,femu_mode=0
```

4.9.3 Configuration

Properties: OCSSD (§A.1.1). The BlackBox geometry and timing properties do not apply, except `ch_xfer_lat` with `oc12_channel_timing`.

Geometry

- `lnum_ch`: channels (groups in 2.0), 1 to 32.
- `lnum_lun`: LUNs (parallel units in 2.0) per channel. `lnum_ch * lnum_lun` is at most 128.
- `lnum_pln`: planes per LUN. Open-Channel 1.2 accepts 1, 2 or 4.
- `lpgs_per_blk`: pages per block, at most 512 for 1.2. A 2.0 chunk is one block on every plane of a parallel unit.
- `lsecs_per_pg`: sectors per page.
- `lsec_size` and `lmetasize`: sector and out-of-band metadata size for 1.2. 2.0 always uses 4096-byte sectors with 16 bytes of metadata.
- `devsz_mb` sets the capacity, and with it the number of blocks per LUN.

Timing

Read, program and erase times come from the `flash_type` table: 1 SLC, 2 MLC (the default), 3 TLC or 4 QLC. Open-Channel 1.2 also charges channel transfer time when `oc12_channel_timing=on`, using `ch_xfer_lat` per page or the table value when it is 0. Open-Channel 2.0 charges no channel time. Open-Channel 2.0 charges the lower page time by default. With `oc20_page_types=on` each chip operation takes the time of its page type, and a run of sectors splits at each NAND page. A page spans `lsecs_per_pg` sectors on each of `lnum_pln` planes, so sector `s` of a chunk is on page `s / (lsecs_per_pg x lnum_pln)`. The option needs `lpgs_`

per_blk of at most 512, the size of the page type tables. Vendor admin command 0xEE changes these times at run time (NAND timing (§3.5.15)). See the timing model (§6.2.7).

Other

- learly_reset (2.0): report that the host may reset a chunk it has not filled.
- OCSSD supports one namespace, does not take namespace_modes other than ocssd, and ignores sgl.

4.9.4 Use it from the guest

These commands assume a guest kernel with LightNVM and nvme-cli 1.x.

Check the device:

```
sudo nvme list
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(mn|sn) '
sudo nvme lnvm list
```

The model is FEMU OpenChannel-SSD Controller (v2.0) (or (v1.2)), and the serial number starts with vOCSSD.

Open-Channel 1.2 accepts only its vector commands on /dev/nvme0n1; Open-Channel 2.0 also accepts plain Read and Write. Either way the host has to follow the NAND rules below. To get a normal block device, create a pblk target over a range of LUNs. With run-whitebox.sh there are 2 x 4 = 8 LUNs, numbered 0 to 7:

```
sudo nvme lnvm create --device-name=nvme0n1 --target-name=mydev \
    --target-type=pblk --lun-begin=0 --lun-end=7
```

/dev/mydev is then a block device with pblk as its host FTL. Use it like any disk:

```
sudo fio --name=oc --filename=/dev/mydev --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=16 --size=1G
sudo mkfs.ext4 /dev/mydev
```

With SPDK, bind the controller to SPDK's user-space driver and use its Open-Channel support. FEMU's Open-Channel 2.0 controller also accepts the plain NVMe Read and Write commands that SPDK sends.

The 2.0 chunk information log page (CAh) reports each chunk's state, write pointer and wear, 32 bytes per chunk. The Supported Log Pages log (00h) lists it for the NVM command set. It needs an explicit namespace, and --log-len must not exceed 32 times the chunk count. This reads the first 128 chunks:

```
sudo nvme get-log /dev/nvme0 --log-id=0xca --log-len=4096 --namespace-id=1
```

The vendor log page C0h stays zero in OCSSD mode: there is no device FTL to count.

4.9.5 Limits and refusals

Message	Cause and fix
lver must be 1 (Open-Channel 1.2) or 2 (Open-Channel 2.0)	Another lver.
flash_type must be 1 (SLC), 2 (MLC), 3 (TLC) or 4 (QLC)	A cell type without a timing table.
ocssd supports a single namespace	namespaces above 1.
ocssd is a controller mode: femu_mode and namespace_modes must both select it, or neither	namespace_modes=ocssd on a controller of another mode, or the reverse.
FEMU ocssd: lnum_ch must not exceed 32 and lnum_ch * lnum_lun must not exceed 128, got 64 and 8	The geometry is too large.

Message	Cause and fix
`FEMU ocssd: lnum_ch, lnum_lun, lnum_pln, lsecs_per_pg, lpgs_per_blk and lsec_size must all be greater than zero`	A geometry property is 0.
`FEMU ocssd: lnum_pln must be 1, 2 or 4, got 3`	Open-Channel 1.2 plane count.
`OC 1.2 requires SLC, MLC, TLC or QLC and at most 512 pages per block`	`lpgs_per_blk` above 512 with `lver=1`.

At run time, Open-Channel 2.0 enforces the chunk rules: a write must start at the chunk's write pointer, so a chunk must be reset before it is written again. Open-Channel 1.2 does not check; a rewrite of a written page silently replaces its data, which real flash would not allow. A write outside the geometry fails in both.

A 2.0 Vector Chunk Reset (opcode 0x90) handles each entry of its LBA list on its own. An entry that fails does not stop the entries after it:

- Dword 0 and 1 of the completion have one bit for each entry that failed. The command status is the status of the first failure.
- With a metadata pointer, each entry gets its chunk descriptor after the command, 32 bytes each, in list order. An address with no chunk leaves its 32 bytes as they were.
- Each chunk that the command resets takes its parallel unit for one erase. Resets on one unit run one after another, and resets on other units overlap.

```
LBA list:  [u0 c0] [u1 c0] [no chunk] [u0 c1] [free c]
result:      reset   reset   0xC1       reset   0xC1
Dword 0/1:   0       0       1          0       1
unit 0:    |erase c0|erase c1|      unit 1: |erase c0|
```

4.9.6 Verify

1. `sudo nvme list` shows the model `FEMU OpenChannel-SSD Controller`.
2. On a LightNVM guest, `sudo nvme lnvm list` lists `nvme0n1` with the geometry you set, and `nvme lnvm create` gives a `/dev/mydev` that fio can write and read.

4.9.7 Troubleshooting

- **The device shows up but nothing can use it.** The guest kernel is 5.15 or newer, so there is no LightNVM. Boot an older kernel, or use SPDK.
- **`nvme lnvm`: invalid sub-command.** nvme-cli 2.x dropped it. Build nvme-cli 1.x in the guest.
- **Writes fail after rewriting an address (Open-Channel 2.0).** A write must start at the chunk's write pointer; reset the chunk first. Run normal workloads on a pblk target, not on `/dev/nvme0n1`.
- **`dmesg` shows `corrupted read LBA` from pblk.** Users reported this warning on reads of pages that were never written; the maintainers found the data intact.
- **`mkfs` hangs on a pblk target.** Users reported that keeping `lnum_pln=2`, changing only `lnum_ch`, `lnum_lun` and the size, and using a power-of-two size avoided it.
- **SPDK cannot write the device.** Older FEMU accepted only vector commands on Open-Channel 2.0. Current FEMU also accepts NVMe Read and Write.

Related issues: #3, #4, #48, #49, #118.

4.9.8 Related pages

- Timing model: OCSSD (§6.2.7)
- Requirements: kernel per mode (§2.1.2)
- Device property reference: OCSSD (§A.1.1)

4.10 OCSSD: the Open-Channel extension

This chapter describes how FEMU emulates an Open-Channel SSD (`femu_mode=0`). It covers the internals: the two protocol versions, their address formats, the per-chunk state that 2.0 keeps, and how each command is timed. To run the mode, see the OCSSD guide (§4.9).

4.10.1 Purpose

An Open-Channel SSD has no device FTL. The host sees the NAND geometry and addresses physical sectors directly. Mapping, garbage collection, wear levelling and bad-block handling all run in the host: LightNVM and pblk in Linux before 5.15, or SPDK in user space. FEMU's job in this mode is to check the addresses the host sends, store the data, and charge NAND time per chip and per channel.

FEMU implements two versions, selected by `lver` (§A.1.1):

	Open-Channel 1.2 (`lver=1`)	Open-Channel 2.0 (`lver=2`, the default)
Address	physical page address (PPA): channel, LUN, block, page, plane, sector	logical block address: group, parallel unit, chunk, sector
Unit of erase	a block on one plane, named by a PPA	a chunk: one block on every plane of a parallel unit
Write rule	none enforced	sequential within a chunk, at the chunk's write pointer
Per-unit state	sector state and out-of-band bytes	chunk descriptor: state, type, wear index, write pointer
Plain NVMe Read and Write	refused	accepted (SPDK uses them)
Channel transfer time	optional (`oc12_channel_timing`)	none
Page-type timing	yes (lower, center, upper page)	no (every access is charged as a lower page)
Log page	none	chunk information, CAh
Model string	`FEMU OpenChannel-SSD Controller (v1.2)`	`FEMU OpenChannel-SSD Controller (v2.0)`

Guest requirement. Linux removed LightNVM in 5.15, so a guest that uses the kernel's Open-Channel stack must run an older kernel. Requirements: kernel per mode (§2.1.2) gives the range for each version. On a newer kernel the controller still appears, and SPDK can drive it from user space.

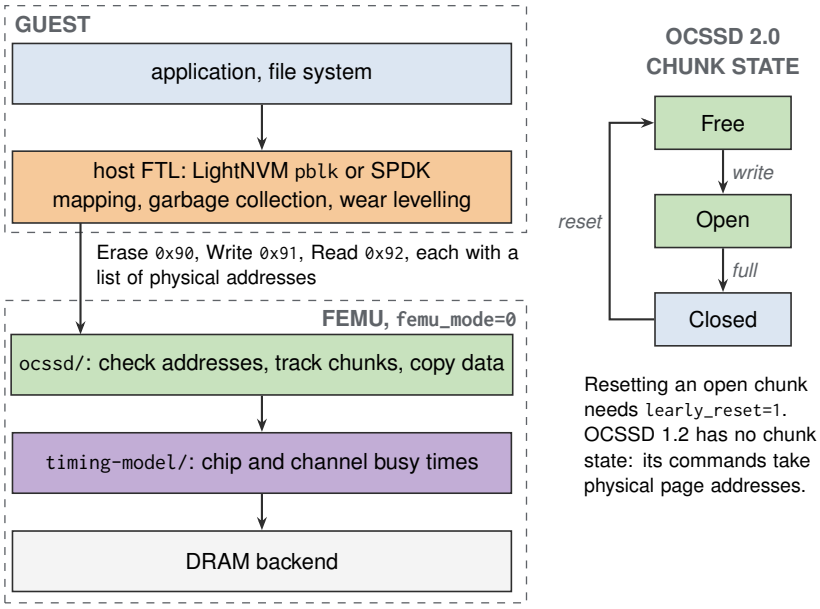
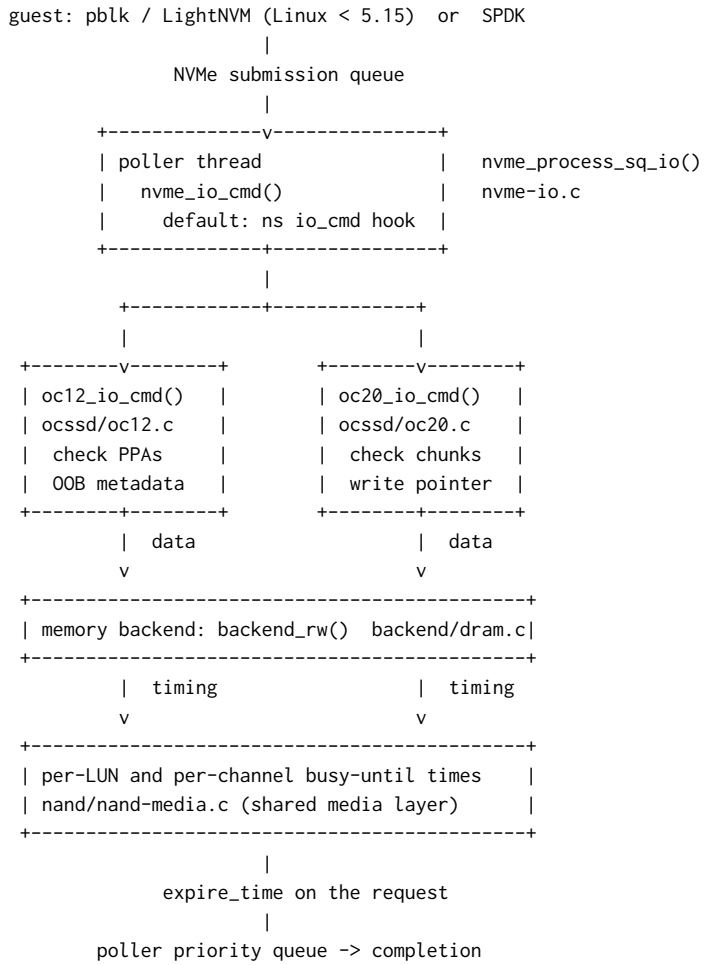


Figure 18: Open-Channel SSD: the FTL runs in the guest and sends commands with physical addresses; FEMU checks them, tracks chunk state (2.0), stores the data and charges chip and channel time.

4.10.2 Place in the hierarchy



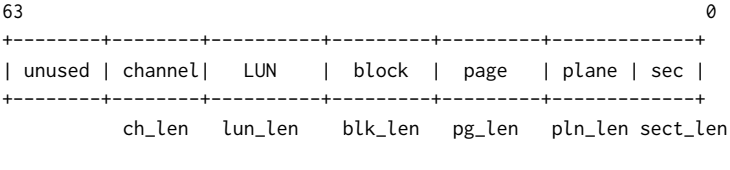
There is no FTL thread in this mode. The whole command, data copy and timing included, runs on the poller thread inside `nvme_io_cmd()`. The request then goes to the poller's own priority queue and is completed when the host clock passes its `expire_time`. The architecture page (§3.1.3) describes that completion step.

Registration happens in `nvme_register_extensions()` in `hw/femu/femu.c`: with `femu_mode=0`, `lver=1` installs `nvme_register_ocssd12()` and `lver=2` installs `nvme_register_ocssd20()`. Each fills the controller's `FemuExtCtrlOps` table: `init`, `exit`, `io_cmd` and `admin_cmd`, plus `get_log` for 2.0.

4.10.3 Open-Channel 1.2

Address format

A 1.2 address is a 64-bit PPA. Each field gets just enough bits for its count, rounded up to a power of two, packed from the sector upward (`oc12_init_id_ctrl()`):




```
field width = ceil(log2(count)); for example lsecs_per_pg=4 -> 2 bits
```

`oc12_ppa_in_geometry()` checks every field against the real count. Because the fields are rounded up to powers of two, a value can fit in its field and still name a unit the device does not have; such an address is refused.

Data structures

`Oc12Ctrl` (`hw/femu/ocssd/oc12.h`) holds the controller state:

- `id_ctrl`: the 1.2 identity structure the host reads with admin command 0xE2. One configuration group: channel, LUN, plane, block, page and sector counts, sector size (`lsec_size`), out-of-band size (`lmetasize`), the multi-plane mask (`mpos`) for 1, 2 or 4 planes, fixed timing hints, and capability bit 0 (bad block table management).
- `params`: the geometry from the properties plus derived strides (`sec_per_pl`, `sec_per_blk`, `pl_units`, `pg_units`, ...).
- `ppaf`: the masks and offsets for each address field.
- `meta_buf`: one entry per sector, `4 + lmetasize` bytes. The first 4 bytes are a sector state (`OC12_SEC_WRITTEN`, `OC12_SEC_ERASED`); the rest are the out-of-band bytes the host passes through the command's metadata pointer. `ppa2secidx()` flattens a PPA to the entry index.

Per namespace (`NvmeNamespace`):

- `bbtbl`: one bad block table per LUN, `blocks * planes` bytes, one byte per plane in the order block 0 plane 0, block 0 plane 1, and so on. The host reads it with admin command 0xF2 and marks blocks with 0xF1, which sets the entry for the block and plane in each address and keeps the header counts (factory bad, grown bad, device reserved, host reserved) in step.
- `tbl`: a table the host can read with admin command 0xEA (Get L2P Table). FEMU fills it with "unmapped" at start-up and never updates it.

Commands

Opcode	Kind	Handler	What it does
0x92	I/O	`oc12_read()`	vector read of up to `lmax_sec_per_rq` sectors
0x91	I/O	`oc12_write()`	vector write; at least `lsecs_per_pg * lnum_pln` sectors
0x90	I/O	`oc12_erase_async()`	vector erase of the listed blocks
0x01, 0x02	I/O		plain Write and Read are refused with Invalid Opcode
0xE2	admin	`oc12_identity()`	identity structure
0xEA	admin	`oc12_get_l2p_tbl()`	the `tbl` described above
0xF2	admin	`oc12_bbt_get()`	bad block table of one LUN
0xF1	admin	`oc12_bbt_set()`	mark one or more blocks

The Commands Supported and Effects log (05h) lists these commands, the timing command 0xEE, and for 2.0 its own vector commands and admin 0xC1; for 1.2 it leaves plain Read and Write out.

A vector command carries either one PPA in the command or a list of PPAs in host memory. Read and write accept only PRP data pointers, check the list against the geometry and MDTS, move the out-of-band bytes, and then copy the data. Each PPA must pair with one sector of the data transfer; a data pointer that does not split into exactly one segment per address is refused with Invalid Field.

The backend stores 1.2 data at the PPA value itself, used as a sector number: `start_block + (ppa << lbads)`. Because the PPA fields are padded to powers of two, this address space can be larger than the device. An

address that lands past the end of the memory backend fails with LBA Out of Range.

Other refusals: a PPA value above the namespace size fails with LBA Out of Range before the backend is reached; an LBA format with metadata (`meta`) makes read and write fail with Invalid Field; and an address outside the geometry on erase or on a bad block command fails with Invalid Field.

Bad blocks

Write and erase check every address against `bbtbl`. An entry with bit 0 (factory bad), bit 1 (grown bad) or bit 2 (device reserved) set makes the block unusable. Bit 3 (host reserved) and bit 4 (media manager reserved) record the host's own allocation, so the device does not refuse them. A single-plane access (control bits 1:0 zero) checks the plane in the address; a dual- or quad-plane access checks the block on every plane.

A command that names an unusable block fails as a whole with Write Fault (status type 2h, code 80h, the 1.2 error code for data that cannot be committed) and Do Not Retry. Nothing in it is written or erased, so every bit of the per-address status in completion dwords 0 and 1 is set. Reads are not checked. Clearing an entry with 0xF1 makes the block usable again. The table starts empty; `nand_bad_blocks` does not seed it.

In both versions, Flush is handled by `nvme_io_cmd()` before the Open-Channel handler. The optional NVM commands (Dataset Management, Compare, Write Zeroes, Copy, Verify, Write Uncorrectable) fail with Invalid Opcode, and ONCS and the command effects log leave them out, whatever `oncs` sets. Their common handlers address the backing store with a flat LBA. An Open-Channel address is sparse, so these handlers would touch the wrong media or memory past the store.

What 1.2 does not enforce

1.2 keeps the sector state but does not act on it:

- A write to a sector that was already written succeeds and replaces the data. Real NAND would refuse it.
- Erase does not reset the sector states: the state update is disabled in `oc12_meta_blk_set_erased()`, which returns at its first line.
- Reads do not check the state, so an unwritten sector returns whatever the backend holds.
- Blocks marked bad in `bbtbl` can still be read; write and erase are refused (see Bad blocks (§4.10.3)).

Write the host FTL as if these rules applied, because real devices apply them.

4.10.4 Open-Channel 2.0

Address format and geometry

A 2.0 address is a logical block address split into four fields (`femu_oc20_init_id_ctrl()`, macros in `hw/femu/ocssd/oc20.h`):

```
63                                                  0
+--------+---------+------------------+--------+-------+
| unused |  group  | parallel unit    | chunk  |sector |
+--------+---------+------------------+--------+-------+
           grp_len   lun_len            chk_len sec_len

group           = channel                 (NUM_GRP = lnum_ch)
parallel unit   = LUN                     (NUM_PU  = lnum_lun)
chunk           = one block on every plane of the LUN
sector          = index inside the chunk  (CLBA = lsecs_per_pg
                                           * lpgs_per_blk * lnum_pln)
NUM_CHK = namespace blocks / (lsecs_per_pg * lpgs_per_blk
                              * lnum_ch * lnum_lun * lnum_pln)
```

`NUM_CHK` is computed from the namespace's block count in the LBA format selected by `lba_index`, before 2.0 switches the namespace to 4096-byte sectors. With the default `lba_index=0` (512-byte blocks), the geometry therefore describes 8 times the memory that `devsz_mb` provides, and valid addresses past the first eighth fail with LBA Out of Range. Set `nlbaf=5,lba_index=3` (4096-byte blocks), as `run-whitebox.sh` does, so the two agree.

The namespace reports its size as `2^(grp_len + lun_len + chk_len + sec_len)` sectors, the whole address space including the padding. The sector size is always 4096 bytes. Unlike 1.2, the data path turns the address into a dense sector index (`oc20_lba_to_sector_index()`) before it touches the backend, so the padding costs no memory.

The geometry structure returned by admin command 0xE2 also carries fixed values: minimum write size 4 sectors, optimal write size 8, cache minimum write size units (`mw_cunits`) 24, no limit on open chunks, and typical and maximum read, write and reset times. Those times are hints for the host; the timing model below does not use them.

Data structures

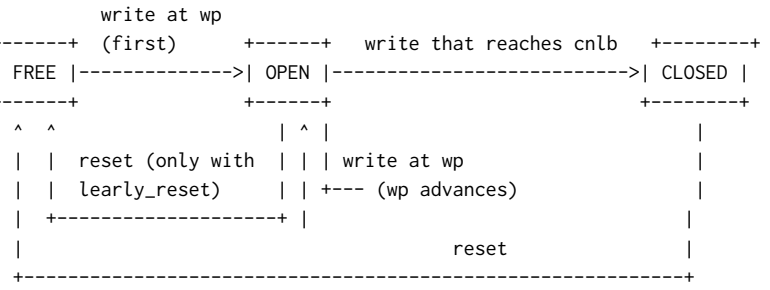
`Oc20Namespace` (`ns->state`) holds:

- `id_ctrl`: the 4096-byte geometry structure.
- `lbaf`: masks and offsets for the four address fields.
- `chunk_info`: one 32-byte descriptor (`Oc20CS`) per chunk, the same layout the chunk information log page returns:

```
Oc20CS (32 bytes)
+-------+------+------------+---------+---------+---------+---------+
| state | type | wear_index | rsvd[5] |  slba   |  cnlb   |   wp    |
|  u8   |  u8  |     u8     |         |  u64    |  u64    |  u64    |
+-------+------+------------+---------+---------+---------+---------+
 state: FREE 0x1, CLOSED 0x2, OPEN 0x4, OFFLINE 0x8
 type : SEQ 0x1 (all chunks at start-up), RAN 0x2
```

`Oc20Ctrl` (`n->ext_ops.state`) holds a header with the sector size (4096) and metadata size (16) the namespace reports.

Chunk state machine




```
reset of a FREE chunk  : refused (multiple resets are not offered)
write to an OFFLINE one: Write Fault
reset of an OFFLINE one: Offline Chunk (0x2C0)
read of an OFFLINE one : succeeds and returns zeros
OFFLINE is only set through Set Log Page
each reset             : wear_index + 1, wp = 0
```

The rules, from `oc20_rw_check_chunk_write()` and `oc20_chunk_set_free()`:

- A write is split into runs, one per chunk it touches. In each run the sectors must be consecutive and the first must equal the chunk's write pointer. Otherwise the command fails with Out of Order Write (0x2F2).
- A write to a closed or offline chunk, or one that would pass the chunk's capacity, fails with Write Fault.
- After the data is stored, `oc20_advance_wp_all()` moves each chunk's write pointer by its run. A free chunk becomes open; a chunk whose write pointer reaches `cnlb` becomes closed.
- Vector erase (0x90) resets each listed chunk. Resetting a closed chunk is always allowed. Resetting an open chunk needs `learly_reset` (§A.1.1). Resetting a free chunk is refused, because FEMU does not advertise multiple resets. When the command has a metadata pointer, FEMU writes the new descriptor of each chunk there.
- A read of a sector that is not yet readable succeeds and returns zeros. A sector is readable when it is below the write pointer of a closed chunk, or more than `mw_cunits` (24) sectors below the write pointer of an open one. The 24-sector window models data still held in the device's write cache.
- Random chunks (type RAN) skip the write pointer: a write run may start at any sector of an open random chunk, though its sectors must still be consecutive, and it moves neither the write pointer nor the state. Any sector of an open random chunk can be read. Chunks only become random through Set Log Page.

The minimum write size (4 sectors) is reported but not checked, so that SPDK works.

Commands

Opcode	Kind	Handler	What it does
0x92	I/O	`oc20_rw(vector=true)`	vector read, list of up to 64 addresses
0x91	I/O	`oc20_rw(vector=true)`	vector write
0x90	I/O	`oc20_erase()`	vector chunk reset
0x02, 0x01	I/O	`oc20_rw(vector=false)`	plain Read and Write over consecutive addresses, with the same chunk rules
0xE2	admin	`oc20_identify()`	geometry
0x02 (Get Log Page), LID CAh	admin	`oc20_get_log()`	chunk descriptors

Opcode	Kind	Handler	What it does
0xC1	admin	`oc20_set_log()`	write chunk descriptors (FEMU extension)

Read and write accept only PRP data pointers. A command with more than 64 addresses fails with Invalid Field.

Set Log Page (0xC1) with LID CAh lets the host put chunks into a given state, type, wear index and write pointer, for example to start a test from a partly used device. FEMU checks every descriptor before it changes any: `slba` and `cnlb` must match the controller's, the type must be SEQ or RAN, and the write pointer must suit the state (0 for FREE, below `cnlb` for OPEN, at most `cnlb` for CLOSED). It pauses the pollers while it copies the descriptors in.

The 2.0 data path does not transfer the metadata buffer, even though the namespace reports 16 metadata bytes per sector.

4.10.5 Timing

Both versions use the shared media layer (`nand_media_op()` in `hw/femu/nand/nand-media.c`), each with its own `NandMedia` and the LUN rule. Open-Channel 2.0 charges no channel time and uses flat lower-page times, through `oc20_chip_op()`. Open-Channel 1.2 uses per-page-type tables, through `oc12_media_op()`; with `oc12_channel_timing=on` the bus is staged, otherwise it is off. Vendor command 0xEE refreshes the times of either version (`oc20_refresh_timing()`, `oc12_refresh_timing()`). With 1.2 channel timing on, one channel lock covers a page's transfer and its chip time together, so two commands that reach the same channel and chip at once complete in an order the host would also see from one queue; before, their phases could interleave. A single command stream gets the same times as before. The controller keeps one time per chip (LUN) and, for 1.2, one per channel:

- `lun_avail[ch * num_lun + lun]` in `Oc12Ctrl` or `Oc20Ctrl`, at most 128 chips.
- `Oc12Ctrl.ch_avail[ch]`, at most 32 channels, and the media layer's list of booked read windows per channel. For 1.2 the list has no cap (`policy.bus_res_unbounded`).

An operation starts at the larger of "now" and the chip's busy-until time, adds the operation's latency, and the end becomes the new busy-until time. Latencies come from the built-in table for `flash_type` (§A.1.1) (1 SLC, 2 MLC, 3 TLC, 4 QLC) in `hw/femu/nand/nand.h`, by operation and page type. The NAND timing properties of the black-box mode (`pg_rd_lat` and the others) do not apply.

A read or write is first grouped into buckets, one per NAND page touched, by `parse_ppa_list()` (1.2) or `oc20_parse_lba_list()` (2.0). Each bucket is then charged on its chip and channel:

```
write bucket:  [ channel transfer ][ program on chip ........ ]
               ^ the channel's first free time at or after now,
                 fitted around reserved read transfers     ^ done

read bucket:   [ read on chip ....... ][ channel transfer ]
               ^ now                                     ^ done
               (the transfer is fitted into a free gap of the
                channel's reservation list)

command done = now + max over buckets of (done - now)
erase       = one block erase on the chip of each listed address;
              command done = the latest of them
```

Version differences:

- **1.2.** A bucket is one page across all planes of a LUN. Its page type (lower, center and upper; QLC has two center types) comes from the page number, so MLC, TLC and QLC pages cost different times. With `oc12_channel_timing` (§A.1.1) on, each bucket also pays channel time: `ch_xfer_lat` per page, or the `flash_type` table's transfer time when `ch_xfer_lat` is 0, scaled by the sectors it carries out of `lsecs_per_pg`.
- **2.0.** A bucket is one run of addresses inside one chunk, so a command that writes 16 consecutive sectors of a chunk is charged one program. Every operation is charged as page type 0 (the lower page). No channel time is charged.

The time is computed on the poller when the command arrives, and stored as the request's `expire_time`. For 1.2 with channel timing, the channel's spin lock (`chnl_locks[]`) covers the bus and its LUNs; otherwise a chip's time is updated with a compare-and-swap. So several pollers can time commands at once.

4.10.6 Parameters

All OCSSD properties are listed in the OCSSD section of the property reference (§A.1.1). How they interact:

Property	1.2	2.0	Notes
`lver`	1	2	Any other value fails realize.
`lnum_ch`	channels	groups	`lnum_ch * lnum_lun` at most 128, `lnum_ch` at most 32: the chip and channel arrays are that size.
`lnum_lun`	LUNs per channel	parallel units per group	
`lnum_pln`	1, 2 or 4	any value above 0	2.0 folds the planes into the chunk size.
`lpgs_per_blk`	at most 512	any value above 0	1.2 looks up the page type in a 512-entry table.
`lsecs_per_pg`	sectors per page	sectors per page	
`lsec_size`	reported sector size	must be above 0, otherwise ignored	
`lmetasize`	out-of-band bytes per sector	ignored (16)	
`lmax_sec_per_rq`	addresses per command	ignored (64)	
`flash_type`	timing table	timing table	1 to 4; others fail realize.
`oc12_channel_timing`, `ch_xfer_lat`	channel time	ignored	`ch_xfer_lat` must not be negative when channel timing is on.
`learly_reset`	ignored	early reset capability	
`devsz_mb`	sets blocks per LUN	sets chunks per parallel unit	1.2 needs 1 to 65535 blocks per LUN.

`ms_max` has no effect; a value other than the default prints a warning. An Open-Channel controller has exactly one namespace, and `namespace_modes`, if given, must say `ocssd`.

A minimal 2.0 device with the geometry of `run-whitebox.sh`:

```
-device femu,devsz_mb=4096,namespaces=1,lver=2,nlbaf=5,lba_index=3,lnum_ch=2,lnum_lun=4,lnum_pln=2,lsecs_per_pg=4,lpgs_per_blk=512,femu_mode=0
```

4.10.7 Counters

OCSSD has no device FTL, so the vendor log page C0h stays zero. What the mode does report:

- The SMART log's host read and write counts. The poller counts only the plain NVMe Read and Write opcodes (0x02 and 0x01), so vector commands are not counted: on 1.2 the counts stay at zero, and on 2.0 they move only for plain Read and Write.
- In 2.0, the chunk information log (CAh): each chunk's state, write pointer and wear index. The wear index counts resets.

4.10.8 Validation status

- The qtest suite in `hw/femu/tests/qtest/femu-test.c`, run in CI, has command-level cases for both versions. For 1.2: `oc12-opcodes`, `oc12-capabilities`, `oc12-small-sectors`, `oc12-ppa-timing`, `oc12-flash-type`, `oc12-page-count`, `oc12-transfer-cost` and the `oc12-channel-*` cases for channel time. For 2.0: `oc20-vector-io`, `oc20-set-chunks`, `oc20-log-length`, `oc20-sgl-refused` and `oc20-fuzz`, a fuzzer over the command fields.
- `oc12-trace-on` and `oc12-trace-off` pin the time of each 1.2 command exactly, with and without channel time, through the qtest-only `x-oc12-trace`. The workload uses TLC pages of each type, part pages, two planes, erases of several blocks, 40 queued reads on one channel and a 0xEE change of the NAND times. A change to the 1.2 timing model must not move these times.
- The documentation check starts each OCSSD example in this guide and in the mode guide, and sends Identify; it does not move data in this mode.
- No guest test runs in CI: LightNVM needs a guest kernel older than 5.15, and the guest image FEMU builds runs Linux 6.8.

4.10.9 Limits

- One namespace per controller.
- 1.2 does not enforce erase-before-write and does not reset sector state on erase (see above). Its bad block table is enforced on write and erase only.
- 2.0 does not model page types or channel transfer, and charges one page time per chunk run regardless of how many pages the run covers.
- 2.0 does not store per-sector metadata.
- The vendor admin command 0xEE can change the read, program and erase times of the lower and upper pages and the channel time at run time, but not those of the centre pages of TLC and QLC (0xEE (§3.5.15)).
- The Get L2P Table command of 1.2 always returns unmapped entries.
- Data lives only in host memory and is lost when QEMU exits.

Refusals at realize are listed in the mode guide (§4.9.5).

4.10.10 Extending the mode

- **A new cell type.** Add its latencies and page-type table in `hw/femu/nand/nand.h` and `hw/femu/nand/nand.c`, then widen the `flash_type` checks in `nvme_check_constraints()` (`hw/femu/femu.c`) and `oc12_init()`.
- **Page types or channel time in 2.0.** `oc20_advance_status()` passes page type 0 and a transfer time of 0. Look up the page from the sector index and pass a transfer time, as `oc12_advance_status()` does.

- **Enforcing 1.2 write-once.** `oc12_meta_state_set_written()` already reads the sector state; refuse a write when it is `OC12_SEC_WRITTEN`, and make `oc12_meta_blk_set_erased()` reset the states instead of returning early.
- **Another log page.** Add a case to `oc20_get_log()`; the generic Get Log Page handler offers unknown log IDs to the namespace's `get_log` hook.

4.10.11 Source map

File	Contents
`hw/femu/ocssd/oc12.c`[37], `oc12.h`[38]	1.2 commands, PPA format, sector metadata, bad block tables, init and exit
`hw/femu/ocssd/oc20.c`[39], `oc20.h`[40]	2.0 commands, chunk descriptors, write pointer rules, geometry, log page
`hw/femu/ocssd/oc-timing.c`[41], `oc-timing.h`[42]	`flash_type` times (`set_latency()`), geometry bound check, 0xEE
`hw/femu/nand/nand-media.c`[43]	chip and channel busy-until times for both versions
`hw/femu/nand/nand.h`[44], `nand.c`[45]	per-cell-type latency tables and page-type tables
`hw/femu/femu.c`[46]	`nvme_register_extensions()`, realize-time checks
`hw/femu/nvme-io.c`[47]	`nvme_io_cmd()` dispatch, completion queue

4.10.12 Related pages

- OCSSD guide (§4.9): launching, guest tools, troubleshooting
- Timing model: OCSSD (§6.2.7)
- Property reference: OCSSD (§A.1.1)

4.11 Key-value SSD (KV)

KV mode (`femu_mode=5`) emulates an SSD with the NVMe Key Value command set. The namespace stores values under keys instead of blocks at addresses: the host sends Store, Retrieve, Delete, Exist and List commands with a key of up to 16 bytes. Values live on emulated NAND laid out with the BlackBox geometry, and each command is charged NAND read and program time.

Use it to develop or measure key-value stores and host software written for key-value devices, without key-value hardware.

4.11.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2).

[37] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc12.c
[38] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc12.h
[39] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc20.c
[40] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc20.h
[41] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc-timing.c
[42] https://github.com/MoatLab/FEMU/blob/master/hw/femu/ocssd/oc-timing.h
[43] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand-media.c
[44] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand.h
[45] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nand/nand.c
[46] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu.c
[47] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-io.c

- Linux has no key-value command set driver. The namespace gets no block device (`/dev/nvme0n1` does not exist for it). You drive it with NVMe passthrough commands, from nvme-cli or your own program. On a controller with one namespace, send them to the controller node `/dev/nvme0`. Linux refuses I/O passthrough on the controller node when the controller has more than one namespace; then use the namespace's generic node `/dev/ngXnY`.
- Guest kernel: 6.0 or newer. Linux 6.0 started to attach namespaces of a command set it has no driver for, with only the generic node (commit eb867ee995bd, "nvme: enable generic interface (/dev/ngXnY) for unknown command sets"). Linux 5.10 to 5.19 log `unknown csi 1 for nsid N` (5.9: `unknown csi:1 ns:N`) and skip the namespace, so passthrough on `/dev/nvme0` fails too.
- The host must enable the controller with CC.CSS set to 110b (all supported I/O command sets). Under 000b (NVM only), every I/O command to a KV namespace fails with Invalid Command Opcode (Base 2.3, Figure 41). Linux sets 110b when CAP.CSS bit 6 is set, and FEMU sets that bit.
- Guest tools: `nvme-cli` (`io-passthru`), and a C compiler to build `kv-probe.c`. The image from `make-guest-image.sh` has no compiler; add one with `./make-guest-image.sh --packages build-essential`, or run `sudo apt install build-essential` in the guest.

4.11.2 Launch

From `build-femu/`:

```
./run-kvssd.sh
```

The FEMU device in that script is:

```
-device femu,devsz_mb=4096,namespaces=1,femu_mode=5,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,
    pls_per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,gc_thres_pcent=75
```

The geometry and timing above are the defaults, so the shortest KV device is:

```
-device femu,devsz_mb=4096,namespaces=1,femu_mode=5
```

The preset `hw/femu/scripts/configs/kvssd.conf` describes the launcher's device; `ssd-config.sh` (§5.11) expands it to:

```
-device femu,id=nvme0,devsz_mb=4096,namespaces=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,pls_
    per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,gc_thres_pcent=75,
    femu_mode=5
```

4.11.3 Configuration

KV uses only part of the BlackBox properties:

- NAND geometry (§A.1.1) (`nchs`, `luns_per_ch`, `pls_per_lun`, `blks_per_pl`, `pgs_per_blk`, `secs_per_pg`, `secsz`) lays out the value store.
- NAND timing (§A.1.1) (`pg_rd_lat`, `pg_wr_lat`, `blk_er_lat`, `nand_cell_type` and the channel bus properties) sets the cost of each command.
- `gc_thres_pcent` is the share of the NAND usable for values. The usable value space is the namespace size or that share of the NAND, whichever is smaller.

The FTL, mapping, cache and write buffer properties do not apply. Each command pays one page read for the index lookup, overlapped with the NAND reads or programs of its value; reclaiming space charges erases to the command that triggers it (timing model (§6.2.8)).

Key index (kv_index)

The device index from key to value is an open-addressed hash in DRAM by default. `kv_index=lsm` makes it an LSM tree after PinK. Lookups then read meta pages of the unpinned levels, merges load the NAND in the background, and List returns keys in byte order. See LSM-tree key index (§4.23) for the knobs and the cost model.

```
-device femu,femu_mode=5,devsz_mb=64,kv_index=lsm,kv_lsm_pinned_levels=2
```

Value packing (kv_pack)

Values whose record (the value and a 32 byte header) fits a NAND page share pages, as real KV-SSDs pack small values. A Store goes to an open page in controller DRAM; the page is programmed when the next record does not fit. With `vwc=1`, Flush programs the open page. `kv_pack=off` gives each value whole pages, the model before packing. See KV value packing (§4.24).

```
-device femu,femu_mode=5,devsz_mb=64,kv_pack=off
```

Key limit (kv_max_keys)

Each KV namespace has a key limit. Identify reports it as MNK, the maximum number of keys of KV format 0 (KV 1.3, Figure 42). A Store of a new key past the limit fails with Capacity Exceeded. An overwrite of a key that exists passes the key check.

With `kv_max_keys=0` (the default), FEMU sizes the limit from the flash:

```
kv_pack=on   MNK = usable pages * floor(page size / 33)
             (one 33-byte record per 1-byte value), 1023 to 4194303
             (a page below 33 bytes holds one value)
kv_pack=off  MNK = 2 * (value space / 4 KiB + 1) - 1, 1023 to 4194303
             (the old limit: a value takes whole pages)
usable pages = NAND pages * gc_thres_pcent / 100
```

So with packing, small values are no longer held to the old two keys for each 4 KiB of value space. MNK is an estimate that sizes the index. The value space, the flash or the limit can stop new keys first. A namespace with more than about 130 MiB of usable flash gets the top of 4194303 keys. Real KV-SSDs also limit the key count by the device DRAM that holds the index. In FEMU the index is a table in host DRAM:

```
hash index: K keys --> K + K/3 + 1 slots * 64 bytes  (about 85 B a key)
            K = 4194303 (the automatic top) --> about 341.3 MiB
kv_pack=off without kv_max_keys: the old table, K + 1 slots
a page of the table costs host memory only when a key lands in it
```

`kv_max_keys=N` (1 to 1610612735) sets the limit of each KV namespace. With the hash index, a small value saves host DRAM, and a large value gives more keys than the automatic top. Realize fails when the host cannot allocate the table. With `kv_index=lsm` the limit counts live keys only. The LSM table holds versions, starts small and grows with the Stores.

```
-device femu,femu_mode=5,devsz_mb=64,kv_max_keys=100000
```

Workload-mix timing (kv_timing=model)

With `kv_timing=model`, Store and Retrieve take their time from a fitted model, not from the NAND model. The fit and its coefficients come from the Samsung KVSSD PDK emulator (BSD-3). Delete, Exist, List and a Retrieve of a missing key keep the NAND model time, as in the PDK.

```
Store / Retrieve hit
       |
       v
window of the last 16 timed commands  (per namespace)
  insert share, read share, update share, mean value size
       |
       v
kIOPS = cubic polynomial (35 terms + intercept)
       |
       v
latency = 1e6 / kIOPS ns   (x size / 16 KiB when the mean is above 16 KiB)
```

- An insert is a Store of a new key. An update is a Store over an existing key. A read is a Retrieve that finds the key, with the bytes it returns.
- Each command gets its own latency. Commands do not queue behind each other, so throughput grows with queue depth, as in the PDK.
- The NAND model still runs, so wear, space and the counters move as in media mode. Only the time of the command changes.
- The fit is not used above a mean value size of 16 KiB, where it grows without bound. The time then scales with the size.
- A result below 0.01 kIOPS, or one that is not finite, becomes 0.01 kIOPS. QEMU then warns once for each namespace.
- The PDK computes the size terms x0^2 and x0^3 in 32-bit integers. x0^3 wraps above 1625 bytes, and x0^2 above 65535 bytes. FEMU computes every term in double. At 4 KiB FEMU gives 0.24 kIOPS more than the PDK.

`kv_model_coeffs` replaces the fit: 36 comma-separated numbers, the intercept first. On a `-device` line, write each comma twice. The 35 coefficients follow the PDK term order:

```
x0 = mean value size, x1 = insert share, x2 = read share, x3 = update share
1, x0, x1, x2, x3,
x0^2, x0 x1, x0 x2, x0 x3, x1^2, x1 x2, x1 x3, x2^2, x2 x3, x3^2,
x0^3, x0^2 x1, ... (every product of three, in the same order), x3^3
```

What the fit does not tell:

- the device and the firmware that produced it;
- the queue depth and the key size at training time;
- the training data, so the fit cannot be checked or trained again here.

Use it to compare workload mixes against one fixed device curve, not as a calibrated device.

```
-device femu,femu_mode=5,devsz_mb=4096,kv_timing=model
```

Limits built into the mode:

- Keys are 1 to 16 bytes.
- Values are at most 2 MiB, and one command also cannot move more than `mdts` allows (4 MiB by default).
- Each KV namespace has its own key space. A controller can have several KV namespaces, or KV next to block namespaces (multi-namespace guide (§4.15)); see the requirements above for which device node to use then.

- KV checks the BlackBox geometry properties, but not the GC capacity rule BlackBox applies: a namespace larger than the NAND allows still starts, and its usable value space is clamped to the NAND.

4.11.4 Use it from the guest

Command format

Command	Opcode
Store	0x01
Retrieve	0x02
List	0x06
Delete	0x10
Exist	0x14

- Key: the low 8 bytes in CDW2 and CDW3, the high 8 bytes in CDW14 and CDW15, little-endian.
- CDW11 bits 7:0: key length in bytes.
- CDW11 bits 15:8, Store only: bit 8 stores only if the key exists, bit 9 stores only if it does not.
- CDW10: the value size for Store, the host buffer size for Retrieve and List.
- The value moves through the normal data pointer.
- The namespace ID must be given. nvme-cli sends 0 by default, which the Linux driver refuses with Invalid argument before the command reaches the device. FEMU answers the broadcast ID (`0xffffffff`) with Invalid Field.

With nvme-cli

Check the controller:

```
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(mn|sn) '
```

The model is `FEMU KV-SSD Controller` and the serial number starts with `vKVSSD`.

Store a 64-byte value under the 4-byte key `BBBB` (0x42424242), read it back, check that it exists, then delete it:

```
head -c 64 /dev/urandom > value.bin
sudo nvme io-passthru /dev/nvme0 -O 0x01 -n 1 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -w -i value.bin
sudo nvme io-passthru /dev/nvme0 -O 0x02 -n 1 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -r -b > out.bin
cmp value.bin out.bin
sudo nvme io-passthru /dev/nvme0 -O 0x14 -n 1 --cdw11=4 --cdw2=0x42424242
sudo nvme io-passthru /dev/nvme0 -O 0x10 -n 1 --cdw11=4 --cdw2=0x42424242
```

Retrieve reports the full value size in the completion's result (Dword 0). If the host buffer is smaller, the device returns the first bytes and the host can retry with a larger buffer.

List up to 4096 bytes of keys from the start of the key space:

```
sudo nvme io-passthru /dev/nvme0 -O 0x06 -n 1 --cdw10=4096 --cdw11=0 \
    -l 4096 -r
```

The returned buffer starts with a 4-byte key count, followed by one entry per key: a 2-byte key length and the key, each entry padded to 4 bytes.

Walk all keys

The Key Value command set (revision 1.3) has no iterator and no command that deletes a group of keys. A host walks the key space with List. The order of the keys stays the same while no Store, Delete, Format or Sanitize runs. A start key that exists comes back as the first entry.

```
List(no key)        --> page 1: k1 k2 ... k8
List(start = k8)    --> page 2: k8 k9 ... k15   (drop k8, it repeats)
List(start = k15)   --> page 3: k15 k16 ...     (drop k15)
...                 --> stop when k1 comes back
```

FEMU returns the keys in the order of its index slots, from the slot of the start key. After the last slot the walk continues from slot 0, so a walk stops when its first key comes back. A start key that is not stored gives the same first key as a List without a key. To delete all keys, walk them and send one Delete for each key. The qtest `kv-list-walk` checks that a walk returns each stored key once.

With kv-probe

`hw/femu/scripts/kv-probe.c` runs the whole lifecycle: store, exist, retrieve (full and short), the conditional stores, delete, the retrieve that must then miss, and a key longer than 16 bytes. It checks status and data. It takes the node to use as its argument (default `/dev/nvme0`) and always addresses namespace 1. From `build-femu/` on the host, copy it to the guest and run it:

```
scp -P 8080 -i ~/images/femu-guest-key ../femu-scripts/kv-probe.c femu@localhost:
./run-guest-ssh.sh gcc -O2 -o kv-probe kv-probe.c
./run-guest-ssh.sh sudo ./kv-probe /dev/nvme0
```

With kv-bench

`hw/femu/scripts/kv-bench.c` is a key-value workload with the knobs of Samsung's kvbench. It checks each Retrieve, Exist and List answer against a model of the keys, and prints the rate and latency of each command. It needs the generic node and an empty namespace:

```
scp -P 8080 -i ~/images/femu-guest-key ../femu-scripts/kv-bench.c femu@localhost:
./run-guest-ssh.sh gcc -O2 -pthread -o kv-bench kv-bench.c
./run-guest-ssh.sh sudo ./kv-bench --keys 20000 /dev/ng0n1
```

The `kv-bench` suite of `guest-conformance.sh` runs `kv-probe` and two `kv-bench` runs in a fresh guest. See KV benchmark (§8.2.6).

Status codes

Status	Meaning
0x85	Invalid Value Size: the value is larger than 2 MiB.
0x86	Invalid Key Size: a key length of 0 on Store, Retrieve, Delete or Exist. For List, length 0 means "from the start".
0x87	Key Does Not Exist: Retrieve or Exist of a missing key, a Store with bit 8 (only if the key exists) for a missing key, or Delete of a missing key when EDNEK is set. It comes with Do Not Retry set.
0x89	Key Exists: a Store with bit 9 (only if the key is absent) found the key.
Capacity Exceeded	The value space is full, or a new key is past the key limit (MNK).
Invalid Field	A key longer than 16 bytes, a List buffer smaller than 4 bytes, or the broadcast namespace ID.
Invalid Command Opcode	The controller runs with CC.CSS 000b, which selects only the NVM command set. Enable it with 110b.

By default, Delete of a missing key succeeds. Setting the EDNEK bit (bit 0) of the Key Value Configuration feature (20h) makes it fail with 0x87:

```
sudo nvme set-feature /dev/nvme0 -n 1 -f 0x20 -V 1
```

Counters

The vendor log page C0h sums its counters over the controller's BlackBox, CSD and KV namespaces. This reads the host pages written, pages moved and pages programmed (log pages and counters (§B.1)):

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
```

4.11.5 Limits and refusals

Message	Cause and fix
`the key-value command set and FDP cannot share a controller: placement owns every reclaim unit and the key-value store is left with no write pointer`	A KV namespace on a controller in an FDP subsystem.
`FEMU bbssd: nchs must be greater than 0, got 0`	KV checks the BlackBox geometry properties with the same messages, but not the GC capacity rule.
`meta: namespace 1 runs a mode without metadata support (block or no-SSD only)`	`meta` with a KV namespace.
`FEMU kvssd: kv_timing=X is not media or model`	Use `media` or `model`.
`FEMU kvssd: kv_model_coeffs: ...; give the intercept and 35 coefficients`	Give 36 finite numbers, separated by commas.
`kv_timing=model and kv_model_coeffs need a key-value namespace (femu_mode=5 or kvssd in namespace_modes)`	The device has no KV namespace, so the knobs would do nothing.
`kv_timing=model and kv_model_coeffs need a key-value namespace; shared namespaces are NoSSD or bbssd`	A controller on shared namespaces (`ns_mgmt=on` subsystem) has no KV namespace.
`kv_index=X is not hash or lsm`	Use `hash` or `lsm`.
`kv_lsm_memtable_keys, kv_lsm_size_ratio and kv_lsm_pinned_levels need kv_index=lsm`	An LSM knob other than its default with the hash index.
`kv_lsm_size_ratio=N is not 2 to 64` (and the other knob ranges)	See the LSM index properties (§4.23.3).
`kv_index=lsm and the kv_lsm_* knobs need a key-value namespace ...`	The device has no KV namespace, or its namespaces are shared.
`FEMU kvssd: kv_index=lsm needs a page of at least 32 bytes; the page is N`	A meta page must hold one 32-byte record.
`kv_pack=off needs a key-value namespace ...`	The device has no KV namespace, or its namespaces are shared.
`kv_max_keys needs a key-value namespace ...`	The device has no KV namespace, or its namespaces are shared.
`kv_max_keys=N is not 0 to 1610612735`	Use 0 (automatic) or a limit in that range.
`FEMU kvssd: no host memory for an index table of N slots (M MiB); set a smaller kv_max_keys`	The host cannot allocate the hash index table. With `kv_index=lsm` the message has no hint: the first LSM table size comes from the value space.

4.11.6 Verify

1. `sudo nvme id-ctrl /dev/nvme0` reports the model `FEMU KV-SSD Controller`.
2. The store, retrieve and `cmp` above succeed, and Exist after Delete returns 0x87.

3. `kv-probe` prints no failures.
4. `kv-bench` prints `KVBENCH_RESULT=PASS`.

4.11.7 Troubleshooting

- **There is no `/dev/nvme0n1`.** Expected: Linux attaches no block driver to a key-value namespace. Send commands to `/dev/nvme0`, or to `/dev/ngXnY` when the controller has several namespaces.
- **nvme-cli says `Invalid argument`.** Add `-n 1` (or the namespace you mean). nvme-cli sends namespace ID 0 by default, and the Linux driver refuses it. On a controller with several namespaces, the controller node refuses I/O passthrough altogether; use the generic node.
- **Retrieve returns 0x87 right after Store.** Check that the key bytes and the key length (CDW11) match exactly; `BBBB` with length 4 is a different key from `BBBB` with length 8.
- **`FEMU_KV_SELFTEST`**: set this environment variable to run the KV FTL self-test once at realize and log the result (environment variables (§A.1.5)). A key limit below 128 skips it, with a `KVSELFTEST SKIP` line.

Related issues: #68.

4.11.8 Related pages

- Timing model: KV and CSD (§6.2.8)
- Architecture: other FTLs (§3.1.5)
- Multiple namespaces (§4.15)
- LSM-tree key index (§4.23)
- KV value packing (§4.24)

4.12 KV: the key-value extension

This chapter describes how FEMU emulates a key-value SSD (`femu_mode=5`): the NVMe Key Value command set as implemented, the device-side index and value store, and what each command is charged. To run the mode, see the KV guide (§4.11).

4.12.1 Purpose

A key-value SSD stores values under keys instead of blocks at addresses. The device owns the mapping from a key to where its value lives, so the host needs no file system or block map to find it. FEMU implements the NVM Express Key Value Command Set, revision 1.3, on top of the same NAND model the black-box mode uses. Values are placed on emulated channels, LUNs and planes, so key-value latency follows flash parallelism and space reclaim rather than a fixed number.

Guest requirement. Linux has no key-value driver. The guest reaches a KV namespace through NVMe passthrough on the generic character device `/dev/ngXnY`, which Linux creates for command sets it does not know since 6.0. Older kernels skip the namespace. See Requirements: kernel per mode (§2.1.2).

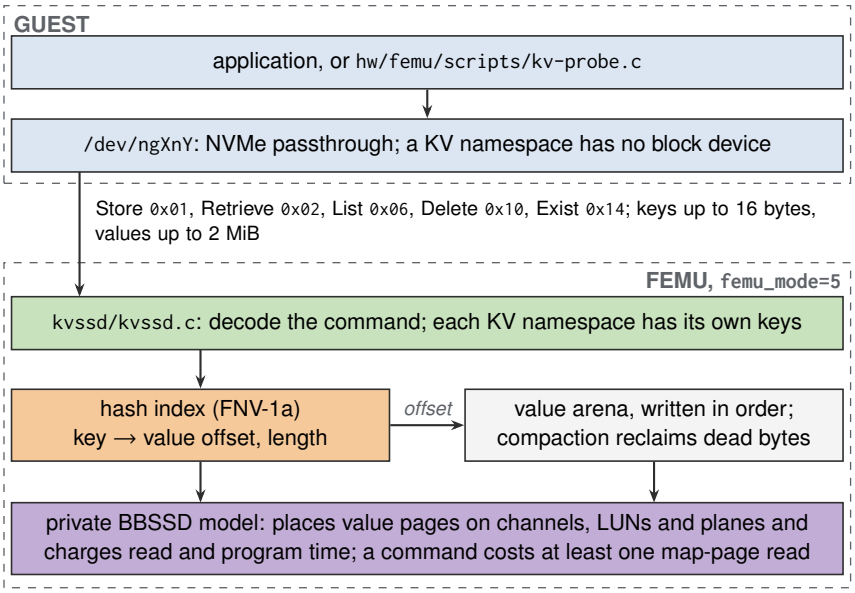


Figure 19: Key-value SSD: each KV namespace keeps a hash index and a value arena, and a private BBSSD model places the value pages and charges NAND time.

4.12.2 Place in the hierarchy

```
guest: nvme io-passthru / kv-probe / own program on /dev/ngXnY
                       |
              NVMe submission queue
                       |
          +------------v---------------+
          | poller thread              |
          |   nvme_io_cmd()            |   nvme-io.c
          |     default: ns io_cmd hook |
          +------------+---------------+
                       |
          +------------v---------------+
          | kvssd_io_cmd()   kvssd.c   |   parse key, lengths, options
          +------------+---------------+
                       |
          +------------v---------------+
          | KV FTL           kvssd-ftl.c |   under the namespace's lock
          |  index: key -> value       |
          |  value arena (DRAM, data)  |
          |  struct ssd (timing only)  |---> NAND media model
          +------------+---------------+     (channels, LUNs, planes)
                       |
                expire_time on the request
                       |
          poller priority queue -> completion
```

KV needs no FTL thread. The command, the data copy and the timing all run on the poller thread inside `nvme_io_cmd()`. If another namespace of the same controller starts the FTL thread (a black-box, ZNS or CSD namespace), KV requests pass through it without further charge.

Each KV namespace owns its own state: key space, value store and NAND model. `kvssd_init()` sets the namespace's command set identifier to KV (`NVME_CSI_KV`, 1h) and allocates the state with `kvssd_ftl_alloc()`.

The first KV namespace also answers controller-wide admin paths that name no namespace. A controller can mix KV namespaces with block namespaces (multi-namespace guide (§4.15)), but not with FDP.

4.12.3 Data structures

`FemuKvssdState` (`hw/femu/kvssd/kvssd.h`), one per KV namespace:

```
FemuKvssdState
+-------------------------------------------------------------------+
| table[hash_slots]   open-addressing index (DRAM)                  |
|   FemuKvssdMappingEntry:                                          |
|   +----------+---------+-----------+--------+---------+--------+ |
|   | key[16]  | key_len | value_off | length | ppas[]  | nr_ppas| |
|   +----------+---------+-----------+--------+---------+--------+ |
|     key_len == 0 marks an empty slot                              |
+-------------------------------------------------------------------+
| values[value_capacity]   byte arena holding the value bytes       |
|   [ live | dead | live | live | dead | ... | free ............ ] |
|                                            ^ value_next           |
|   value_used, key_used, value_reclaimable (dead bytes)            |
+-------------------------------------------------------------------+
| ssd    a black-box struct ssd built from the NAND properties:     |
|        lines, write pointer, page states, LUN busy times.         |
|        Used for placement and timing only; it holds no data.      |
+-------------------------------------------------------------------+
| ednek   Key Value Configuration feature (20h), bit 0              |
| lock    one mutex for all of the above                            |
+-------------------------------------------------------------------+
```

- **Index.** A hash table with linear probing on a 32-bit FNV-1a hash of the key bytes. Deletion shifts following entries back so probe runs stay intact. The table lives in host memory, 64 bytes a slot, and costs no flash reads.
- **Key limit.** `kv_index_size()` sets `kv_max_keys`, which the namespace reports as MNK. The property `kv_max_keys` sets it, or else the flash does (KV mode (§4.11.3)). With `kv_pack` it is the number of 1-byte values the usable flash holds, from 1023 to 2^22 - 1. Without `kv_pack` it is the old limit: `hash_slots - 1` of `min(2^22, 2 * (value_capacity / 4096 + 1))` slots, at least 1024. Otherwise a limit of K keys gets `K + K/3 + 1` slots, so a probe always ends at an empty slot.
- **Value arena.** The value bytes the host stores, appended at `value_next`. An overwrite or delete leaves the old bytes behind as dead space (`value_reclaimable`).
- **Value pages.** Each entry also records the NAND pages that hold its value (`ppas`), `ceil(length / page size)` of them, where the page size is `secsz * secs_per_pg`. Those pages exist only in the timing model. `ssd->rmap` names the owner of each programmed page: the table slot of its value, or the LSM tree for a meta page. A page is released only through its owner.

The two views are separate. The arena is the emulator's store of the host bytes: when its frontier is full, `kv_arena_pack()` moves the live bytes to the front, with no NAND time. The NAND model is the device: value GC collects its lines one victim at a time (value GC (§4.12.8)).

4.12.4 Value packing

With `kv_pack=on` (the default), a value whose record fits a page shares the page with other values. A record is the value and a 32 byte header (`KV_PACK_HDR`): the key, the length, a version and the offset. The user view is in KV value packing (§4.24).

```
s->open (KvPackPage, DRAM)            s->packed: ppa -> KvPackPage
+-------------------------------+     +-------------------------------+
| items: owner slot, off, len,  |     | the same header, for a page   |
|        live                   |     | on NAND; rmap = KV_OWNER_PACK |
| fill, live, live_bytes        |     +-------------------------------+
+-------------------------------+
entry of a packed value: packed = true, nr_ppas = 1,
  ppas[0] = KV_PPA_OPEN (in DRAM) or the page, pack_off = record offset
s->line_live[line]: live bytes; a page for a whole or meta page,
  the live records of a packed page
```

- `kv_place_value()` puts a value in the open page. A value whose record does not fit a page goes to whole pages, as before. When the record does not fit the open page, the dead records go first (`kv_pack_compact()`, DRAM only). Then `kv_pack_program()` programs the page. A page full of live records is programmed with the value that fills it. `kv-pack.c` holds the page header; the unit test `test-femu-kv-pack` checks it.
- `kv_pack_hold()` says whether the open page may stay in DRAM: not with `vwc=1` and WCE cleared. Then the page is programmed with the value in it, before the index takes the value, so a failed program changes nothing.
- The owner of a packed value is its item, found by the offset. Owner moves (`kv_move_owner()`) and releases (`kv_release_value()`) take the entry, so a hash backshift and an LSM reinsertion move the item too.
- `kv_pack_kill()` ends a record. A programmed page with no live record goes invalid and its header is freed.
- `kvssd_ftl_flush()` programs the open page and waits for `pack_persist`, the end of the last packed program. Flush with WCE set, Set Features 06h that clears WCE and CC.SHN call it (`kvssd_flush()`, `kvssd_flush_all()`).
- `kv_room()` keeps one page for the open page while it holds a live record, so that a Flush finds its page.

4.12.5 Key index

`FemuKvIndexOps` (`kvssd.h`) is the index of a namespace: `find`, `upsert`, `remove` and `probe`. The `kv_index` property picks one of two tables in `kvssd_ftl_alloc()`:

`kv_index`	Index	`probe`
`hash` (default)	the open-addressing table above, in DRAM	none: no flash read
`lsm`	an LSM tree after PinK (`kv-lsm.c`)	one meta page read per unpinned level, in sequence

With `lsm`, `table[]` becomes a pool of versions. Each Store takes a new slot from a free stack, and the pool grows when the stack is empty. The tree maps the key to the slot of its newest version:

```
memtable (sorted, DRAM) ---flush---> L1 ---merge---> L2 ---> ... L24
record { key, key_len, tombstone, slot } --------+
                                                 v
table[slot] { key, value_off, length, ppas, stale }   one version
```

- `kv-lsm.c` holds the tree and needs nothing from QEMU. The unit test `tests/unit/test-kv-lsm.c` builds it alone. It calls the FTL through `KvLsmOps`:
  - `io`: a meta page read or program;
  - `alloc`: a new meta page;

  - `free_page`: a page the tree no longer names;
  - `drop`: a record that leaves the tree.
- A meta page is a real NAND page on its own write pointer (`meta_wp`), in lines that hold no value. Its id is its `ppa`. Probes and scans charge it as `USER_IO`, merges and GC as `GC_IO`. Thus read counts, wear and ECC see its real block.
- Each slot has one record. An overwrite or a Delete makes the old version stale: it gives back its arena bytes and NUSE at once, but it keeps its pages. The pages go when `drop` reports its record, from a merge that shadows it or from a memtable put that replaces it.
- `alloc` takes only a free line, and does not program: the program comes as `KV_LSM_IO_COMPACT_PROGRAM` when the merge commits. GC runs only at the start of a command, never inside a merge. GC itself runs the merges of the keys it reinserts, between two victims.
- `room` refuses a merge before it takes a page. Every merge leaves one free line for value GC. A Delete whose flush fails keeps its tombstone in the memtable over its capacity (`kv_lsm_put_tombstone()`), so a full device still takes a Delete.
- `kv_lsm_compact_all()` merges every run into the deepest level in one pass, so its output holds only the live keys. A Store that still has no room after GC runs it, and then GC again.
- The timing rules, from the request arrival time `t0`:

```
lookup (probe)   each meta read starts when the one before ends
Retrieve         value reads start at t0 + probe
List             per level: serial reads; levels side by side
merge I/O        queued during the command, issued at unlock:
                 reads from t0, programs after the last read,
                 the next merge after the last program; not in the latency
value GC         check reads level by level from t0, each move after
                 its read, reinserted keys' merges after the moves
```

The hash table leaves `probe` NULL and takes none of these paths, so its call order and times are unchanged. The user view is in LSM-tree key index (§4.23).

4.12.6 Capacity

`kvssd_ftl_alloc()` sizes the store from two limits and takes the smaller:

```
value_capacity = min( namespace size,
                      NAND pages * page size * gc_thres_pcent / 100 )
```

The share of NAND set aside by `gc_thres_pcent` (§A.1.1) is free space for reclaim, the over-provisioning a real KV-SSD keeps. The NAND size comes from the geometry properties (§A.1.1). Identify reports the result as the namespace size (NSZE) in bytes and the live key and value bytes as NUSE.

A store is refused with Capacity Exceeded when live keys and values would pass `value_capacity`, when the index is full, or when the NAND model has too few free pages for the value and a reserve. The reserve is the room that value GC needs to move a victim. It is one line with the hash index. The LSM index also needs a line for its open meta page, and the pages of its largest merge: one that rewrites every record, with a part page for each level.

4.12.7 Commands

Commands use the common command format. `kvssd.c` reads these fields:

Field	Meaning
CDW2, CDW3	key bytes 0 to 7
CDW14, CDW15	key bytes 8 to 15
CDW11 bits 7:0	key length, 1 to 16 (List also accepts 0)
CDW11 bits 15:8	Store options: bit 8 store only if the key exists, bit 9 store only if it does not
CDW10	value size (Store) or host buffer size (Retrieve, List)
data pointer	the value or the list, PRP or SGL

Opcode	Command	FTL function	Result
01h	Store	`kvssd_ftl_store()`	appends the value, programs its pages, updates the index
02h	Retrieve	`kvssd_ftl_retrieve()`	returns `min(host buffer, value)` bytes; Dword 0 of the completion holds the full value size
10h	Delete	`kvssd_ftl_delete()`	removes the key; a missing key succeeds unless EDNEK is set
14h	Exist	`kvssd_ftl_exist()`	success or Key Does Not Exist
06h	List	`kvssd_ftl_list()`	key count, then one entry per key: 2-byte length and key, padded to 4 bytes

Store and Retrieve share their opcodes with Write and Read, so the generic code counts them in the SMART host command totals, using the bytes really moved.

Limits on the command fields:

- Key length above 16: Invalid Field. Key length 0: Invalid Key Size (86h), except for List, where 0 means "from the start".
- Value above 2 MiB: Invalid Value Size (85h).
- A value, retrieved span or list larger than MDTS allows: Invalid Field. Store checks the value before the FTL runs; Retrieve checks the span inside the FTL, and only when the key exists; List checks its buffer size inside the FTL.
- A List buffer smaller than 4 bytes: Invalid Field.
- The broadcast namespace ID: Invalid Field.
- A controller enabled with CC.CSS other than 110b: Invalid Command Opcode for every I/O command to a KV namespace (`nvme_io_cmd()`). CC.CSS 000b selects only the NVM command set (Base 2.3, Figure 41).

List walks the index in slot order, starting at the start key's slot when the key exists and at slot 0 otherwise. The order is stable while the namespace does not change, which is what the specification asks.

Admin side (`hw/femu/kvssd/kvssd-admin.c`): Identify with CSI 01h answers the I/O command set specific Namespace and Controller structures (the controller reports KV 1.3) and the format query (CNS 0Ah). The format query ignores the NSID and reports only the format: NSZE and the key limit read 0, and an index other than 0 fails with Invalid Field. There is one KV format with a 16-byte key, a 2 MiB value and the key limit above. Identify CNS 00h belongs to the NVM Command Set, so for a KV namespace it fails with Invalid I/O Command Set. A host reads the common attributes and readiness from CNS 08h, as Linux does, and the size and format from the KV structure above, which reports the same NMIC and ENDGID. Set and Get Features 20h (Key Value Configuration) read and write EDNEK.

4.12.8 Store, step by step

```
kvssd_ftl_store(key, vsize)                    lock held throughout
 1. find key in index
 2. option or capacity check fails?  -> charge the index cost, return error
 3. kv_gc_run(): value GC while free lines <= threshold or room < need
    lsm, still no room, old records: kv_lsm_compact_all(), GC again
 4. room < value pages + reserve?    -> Capacity Exceeded
 5. kv_value_alloc(vsize)
      frontier fits?          -> take [value_next, value_next + vsize)
      else dead bytes exist?  -> kv_arena_pack() (memory only), retry
      else                    -> Capacity Exceeded
 6. copy the value from the host into the arena
 7. kv_place_value()
      record fits a page (kv_pack=on): the open page
        no room: drop dead records, then program the open page
        WCE cleared: program the page with the value in it
      else kv_program_ppas(): for each value page
        kv_wp_line()   (a free line when the pointer has none)
        mark valid; charge a NAND program
        kv_wp_step()   (channel, LUN, plane, then page)
 8. index upsert; the new pages or item take the slot as owner
      hash: release the old value's pages or record now
      lsm:  the old version goes stale; its flash waits for the tree
 9. latency: whichever ends last of the GC in step 3, the index cost,
    the page programs, the reclaim erases, and the DRAM time of a
    record that stays in the open page
```

The write pointer walks a line the way the black-box FTL does: across channels, then LUNs, then planes, then down the pages of the block. When a line fills, it goes to the full list if all its pages are still valid, or to the victim queue if some were invalidated.

Reclaim. `kv_reclaim_empty_lines()` takes lines from the victim queue whose valid page count is zero and erases them, one multi-plane erase per LUN. It runs at the start of GC and before a Store programs its value.

Value GC

`kv_gc_run()` runs at the start of a Store, and with the LSM index at the start of a Delete. It collects victims while the free lines are at or below `gc_thres_lines`, or while a Store has too little room:

```
victim = the line with the fewest valid pages (victim queue)
         kv_pack: the closed line with the fewest live bytes, from the
         victim queue and the full list, each once per run
         lsm, no such line, a Store with no room:
         the next full value line of the run, oldest first
value line:
   hash   every valid page is live: read it, program a copy
   lsm    look up each version's key (meta reads, level by level)
            the newest record names it -> move; record on flash -> reinsert
            else                       -> release the page, no move
   packed page: read it once; each live record goes to the GC page
            (lsm: a stale version is released); the GC page is
            programmed when full and before the erase
meta line: read and program each valid meta page; the level list follows
reinsert the keys (lsm), then erase the victim, one erase per LUN
```

- A victim from the queue that gives no usable page ends the run, unless a Store needs room. GC then tries the victims queued at the start of the run, and the full lines. A run takes only the full lines that

existed at its start, so it never moves a line in a loop.
- The victim is marked `reclaiming` before any page changes. When a move finds no free page, the victim goes back to its list by its counts.
- Each move reads its source page, then programs the copy. The erase waits for the last program. The next victim starts when the erase ends. The Store waits for all of it.

4.12.9 Timing

KV charges NAND operations through the black-box media model (`ssd_advance_status()`), so every operation queues on its LUN and channel like a black-box I/O. The NAND timing properties (`pg_rd_lat`, `pg_wr_lat`, `blk_er_lat`, the channel properties (§A.1.1)) set the costs.

With the hash index, every command that reaches the index pays a **base cost**: one NAND page read, charged as a mapping read on the LUN that holds the key's part of the index (`kv_index_ppa()`). The index is spread over every LUN by the key's FNV-1a hash, LUN `hash % (nchs * luns_per_ch)`, so lookups of different keys run in parallel; a List uses its start key. The cost stands for the controller's index work, so that commands that touch no value are not free. The index itself adds no reads. Commands refused for their fields, and most error paths after the index lookup, are charged nothing.

Command	Charged
Store	whichever ends last of any value GC it runs, the base cost, its page programs and any reclaim erases; a packed value costs the DRAM time, or the program of the open page it forces
Flush (`vwc=1`, WCE set)	the program of the open page, and every earlier program of packed values
Retrieve	the reads of the value pages that are transferred, and the base cost; the longest of them; a packed value reads its one page, or costs the DRAM time while it is in the open page
Exist	the base cost
Delete, List	the longer of the base cost and the index cost, which is 0 for the hash index
any command with `kv_index=lsm`	the meta page reads of the key index (§4.12.5) in place of the base cost
a missing key, a conditional Store that does not apply, a full index or a full value space	the base cost

All costs are measured from the command's arrival time on the LUN timelines, so a LUN that is already busy makes the command wait, and a stage that queues behind an earlier stage of the same command already contains that wait. The command therefore ends with its last stage, and costs never add up: a Store whose own programs follow its value GC on a LUN completes when they do. The result is added to the request's `expire_time` (`kv_apply_lat()`). The host-link and firmware models (host link and controller firmware (§A.1.1)) also charge Store and Retrieve as they charge Write and Read. The timing model page (§6.2.8) gives the same rules in short.

4.12.10 Parameters

KV reads only part of the black-box properties:

Group	Properties	Use in KV
NAND geometry (§A.1.1)	nchs, luns_per_ch, pls_per_lun, blks_per_pl, pgs_per_blk, secs_per_pg, secsz	layout of the value pages; NAND size
NAND timing (§A.1.1)	pg_rd_lat, pg_wr_lat, blk_er_lat, cell type and channel properties	cost of each operation
Garbage collection (§A.1.1)	gc_thres_pcent	usable share of the NAND
KV timing (§A.1.1)	kv_pack	small values share pages (default on)
Controller identity and capabilities (§A.1.1)	vwc	with WCE set, the open page waits for Flush
Mode, capacity and namespaces (§A.1.1)	devsz_mb, namespaces, namespace_sizes, namespace_modes	namespace size, KV next to other modes
Queues, pollers and interrupts (§A.1.1)	mdts	largest value, retrieve or list one command moves

The mapping, cache, write buffer and background GC properties do not apply. KV runs the black-box geometry check, but not its capacity check: a namespace larger than the NAND still starts, and its value space is clamped as above. meta and FDP are refused for a KV namespace.

A KV namespace with a block namespace beside it on one controller (on the QEMU command line, the comma inside the value is written , ,):

```
-device femu,devsz_mb=2048,femu_mode=5,namespaces=2,namespace_modes=kvssd,,bbssd
```

4.12.11 Counters

KV keeps its NAND page counts on its struct ssd, split the way the black-box mode splits them, and the vendor log page C0h sums them over the controller's black-box, CSD and KV namespaces (log pages and counters (§B.1)):

- pages the host asked to program: value pages written by Store; with kv_pack=on, the value bytes of the Stores in whole pages, rounded down;
- pages relocated by garbage collection: value pages and meta pages that value GC moved;
- pages programmed: the value pages of the first item, each packed page, and the meta pages that merges program (LSM index only);
- reads of the most-read block: value page reads count against their block.

The SMART log counts Store and Retrieve as host writes and reads, with the bytes actually moved.

4.12.12 Validation status

- qtest cases in hw/femu/tests/qtest/femu-test.c, run in CI: kv-discovery (the command set list and its command effects log), kv-namespaces (separate key spaces), kv-accounting (SMART bytes and the most-read block figure), kv-base-spread (timing), kv-gc-victim (one victim per GC), kv-hash-backshift (page owners after a Delete shifts entries), kv-css-nvm and kv-css-all (CC.CSS), namespace-kv-byte-capacity, kv-mdts and kv-list-mdts0 (transfer limits), kv-identify-reserved (Identify layout), sgl-kv, and kv-fuzz, a fuzzer over the command fields.
- The kv-pack-* qtests cover value packing. They check pages and host bytes, the old model and overwrites in the open page. They also check Flush and WCE, a Flush beside a program, the read time, and GC with both indexes.

- The `kv-lsm-*` qtests cover `kv_index=lsm`: data, List order, probe time, background merges, query output, refusals, value GC (also with two planes and two-page values), old values on flash, meta page programs, Delete, a full device, table growth and random overwrites. The unit test `test-femu-kv-lsm` checks the tree against a reference, and each page and record callback.
- The unit test `test-femu-kv-pack` checks the packed page header: fit, exact fill, offsets, drops, compaction and allocation failures.
- The qtests set `FEMU_KV_CHECK`. After each value GC and every 64 commands, the FTL checks the owner that `rmap` names for each valid page. A broken owner stops QEMU.
- The documentation check stores and retrieves one value in each example whose namespace 1 is a KV namespace.
- `FEMU_KV_SELFTEST`, set in the environment, runs a self-test of the FTL at realize and logs the result (environment variables (§A.1.5)).
- In a guest, `hw/femu/scripts/kv-probe.c` and the KV checks of `femu-test.sh` run the full command set; they need Linux 6.0 or newer and are not run in CI.

4.12.13 Limits

- One key-value format: keys up to 16 bytes, values up to 2 MiB.
- Values are held in host memory as well as in the timing model's page states, so a KV namespace costs host memory equal to its value capacity.
- Value GC runs in the Store that needs room, not in the background.
- With `kv_pack=off`, a value takes whole pages. Small values then use more NAND than their bytes. A Store can get Capacity Exceeded before NUSE reaches NSZE. With packing, a record still takes a 32 byte header, and a value whose record does not fit a page takes whole pages.
- One open page per namespace. Admin commands and the shutdown status do not wait for the program of the open page. FEMU models no power loss for KV, so the index in device DRAM needs no journal.
- The base cost of a key always falls on the same LUN, so a workload that hammers one key queues on that LUN.
- With the hash index, List order follows the hash table, not key order. The LSM index returns byte order.
- Not with FDP or `meta`. A controller with a KV namespace offers no namespace management: it needs a NoSSD or black-box controller whose namespaces all share its mode.

Refusal messages are listed in the KV guide (§4.11.5).

4.12.14 Extending the mode

- **Another index.** `FemuKvIndexOps` in `kvssd.h` is a small table of `find`, `upsert`, `remove` and `probe`. An index kept on flash returns from `probe` the time of its lookup reads, as the LSM tree does. Add a `kv_index` value and set `s->index` to the new table in `kvssd_ftl_alloc()`.
- **Background GC.** `kv_gc_run()` runs in the command; a timer could run it between commands, as the black-box GC in `hw/femu/bbssd/` does.
- **More KV formats.** `kvssd_fill_id_ns()` reports one format; additional formats need a Format NVM path that changes `kv_key_max` and `kv_value_max`.

4.12.15 Source map

File	Contents

File	Contents
hw/femu/kvssd/kvssd.c[48]	command parsing, registration, init and exit
hw/femu/kvssd/kvssd-ftl.c[49]	index, value arena, NAND placement, value packing, page owners, value GC, timing, owner check, self-test
hw/femu/kvssd/kv-lsm.c[50]	the LSM tree of kv_index=lsm: memtable, levels, merges, scan
hw/femu/kvssd/kv-pack.c[51]	the packed page header of kv_pack=on: records, fit, compaction
hw/femu/kvssd/kvssd-admin.c[52]	Identify structures, Key Value Configuration feature
hw/femu/kvssd/kvssd.h[53]	FemuKvssdState, FemuKvssdMappingEntry, FemuKvIndexOps
hw/femu/bbssd/ftl-media.c[54]	ssd_advance_status(), the media model KV charges through
hw/femu/nvme-admin.c[55]	Identify and feature routing to the KV handlers

4.12.16 Related pages

- KV guide (§4.11): command examples, status codes, troubleshooting
- Timing model: KV and CSD (§6.2.8)
- Multiple namespaces (§4.15)
- KV value packing (§4.24)

4.13 Computational storage (CSD)

CSD mode (femu_mode=4) emulates a computational storage drive. On top of a normal NVMe namespace, the device has its own memory (function data memory, FDM) and compute units that run programs next to the data. The guest allocates device memory, copies namespace data into it, loads and runs a program, and reads the result back, all with NVMe commands. Ordinary reads and writes go through the BlackBox (§4.4) FTL, so they have SSD timing.

The mode is a port of CEMU[56]; if you use it, cite CEMU as well as FEMU (citation (§4.13.9)). It does not need CEMU's modified guest kernel, FDMFS or a fixed VM image. CEMU's VM freezing and virtual clock changes are left out of this port. The opt-in csd_pause (§4.13.4) does that job on an unmodified QEMU and guest.

Use it to study offloading filters, scans, compression or other kernels to the drive, and what that does to latency and host CPU use.

[48] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kvssd.c
[49] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kvssd-ftl.c
[50] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kv-lsm.c
[51] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kv-pack.c
[52] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kvssd-admin.c
[53] https://github.com/MoatLab/FEMU/blob/master/hw/femu/kvssd/kvssd.h
[54] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl-media.c
[55] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-admin.c
[56] https://github.com/cs-qyzhang/CEMU

4.13.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). Any guest kernel with the NVMe driver works.
- Guest tools: `hw/femu/tests/csd`, built in the guest with a C compiler. The image from `make-guest-image.sh` has none; add one with `./make-guest-image.sh --packages build-essential`, or run `sudo apt install build-essential` in the guest.
- Programs: shared-library programs are built on the host and placed in the directory named by `csd_program_dir`. eBPF programs also need a FEMU build with uBPF (`./femu-compile.sh --enable-csd-ubpf`, see build.md (§2.2.4)) and clang with the BPF target on the host.

4.13.2 Security: programs run inside QEMU on the host

A program load names a file. QEMU, on the host, opens that file and runs code from it inside the QEMU process. The `run-*.sh` launchers start QEMU with `sudo`, so that code runs as root on the host.

FEMU limits what the guest can name:

- With `csd_program_dir` unset, only the built-in phantom program type loads. Shared-library and eBPF loads fail.
- With it set, the guest may name only a file in that directory: no `/`, no `..`, and the path must still resolve inside the directory after symbolic links are followed.

That protects the rest of the host file system. It does not make the programs in the directory safe: they run on QEMU's compute unit threads with all of QEMU's privileges and memory. Use CSD only with guests you trust, put only programs you built and trust in the directory, and make sure no one else can write to it.

4.13.3 Launch

From `build-femu/`:

```
./run-csd.sh
```

The FEMU device in that script is:

```
-device femu,devsz_mb=4096,namespaces=1,femu_mode=4,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,␣
    pls_per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,ch_xfer_lat=0,gc_␣
    thres_pcent=75,gc_thres_pcent_high=95,fdm_size=64,nr_cu=4,nr_thread=4,time_slice=200000,context_switch_␣
    time=200,csf_runtime_scale=3
```

The preset `hw/femu/scripts/configs/csd.conf` describes the same device without the three scheduler properties that have no effect. `ssd-config.sh` (§5.11) expands it to:

```
-device femu,id=nvme0,devsz_mb=4096,namespaces=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,pls_␣
    per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_wr_lat=200000,blk_er_lat=2000000,gc_thres_pcent=75,gc_␣
    thres_pcent_high=95,fdm_size=64,nr_cu=4,csf_runtime_scale=3,femu_mode=4
```

`run-csd.sh` does not set `csd_program_dir`, so it runs phantom programs only. To load your own programs, add the directory after the other `FEMU_OPTIONS` lines in `run-csd.sh`:

```
FEMU_OPTIONS=${FEMU_OPTIONS}",csd_program_dir=$HOME/csd-programs"
```

The shortest device line with programs enabled is:

```
-device femu,devsz_mb=4096,femu_mode=4,fdm_size=64,csd_program_dir=/home/you/csd-programs
```

FEMU does not check the directory at start-up. A directory that does not exist makes every program load fail.

4.13.4 Configuration

Properties: CSD (§A.1.1). The BlackBox geometry, timing, GC and FTL properties also apply to the namespace; see the BlackBox guide (§4.4.3).

- `fdm_size`: device memory in MiB. Required.
- `nr_cu`: compute units, 1 to 64. Each is a host thread, `femu-csd-cu`, that runs programs. A program waits for the first free unit.
- `csf_runtime_scale`: a program that declares no run time and no scale of its own holds its unit for its measured host run time times this value (default 3). A completion carries the program's result, so it never arrives before the host has run the program. Without `csd_pause`, a run time, declared or scaled, shorter than the host's own is not reached, and QEMU warns once when that happens.
- `csd_pause`: stop the guest while the host runs a program, so the guest sees only the modelled run time (default off). See below (§4.13.4).
- `csd_pause_max_ms`: with `csd_pause`, the longest stop, 1 to 60000 ms (default 1000).
- `csd_program_dir`: the host directory programs load from.
- `nr_thread`, `time_slice` and `context_switch_time` are accepted so CEMU configurations still start. They have no effect, and a value other than the default prints a warning at realize.

A copy from the namespace into device memory costs one `pg_rd_lat`, whatever its size, when any page of the range has been written, and nothing otherwise. See the timing model (§6.2.8).

Program types

Type	What runs	Needs
Phantom	Built in: copies the input memory range to the output range. Useful to test the command flow and timing.	nothing
Shared library	A function with the signature `int64_t fn(struct femu_csd_args *args)` from a `.so` file, named in the load command.	`csd_program_dir`
uBPF	An eBPF ELF object, interpreted or JIT-compiled.	`csd_program_dir` and a build with `--enable-csd-ubpf`

How programs run

An Execute command is checked on the poller and then handed to one of the `nr_cu` compute unit threads, which runs the program and posts the completion. While a program runs, I/O on every queue, other CSD commands and admin commands go on as usual; only that compute unit is busy.

- At most `nr_cu` programs run at once. Runs of one program take turns, so a shared library needs no locking of its own; different programs run side by side, and a run waiting for its own program leaves the compute unit free for another.
- A program that never returns keeps its compute unit for good, and QEMU waits for it when the device is removed. FEMU cannot stop native code it has called.
- Device memory a program is using stays allocated, and counted against `fdm_size`, until the program returns, even after Free device memory.
- Deleting the I/O queue an Execute came from, or resetting the controller, while the program runs drops its result; the program still runs to the end.

Hide the host run time (csd_pause)

A real CSD runs a program on its own cores while the host goes on. FEMU runs it on a host thread, and the completion carries the result. So without help the guest waits for the host run, also when the model says the device is faster.

With `csd_pause=on`, FEMU stops the guest while a program runs, and hides that time from the guest clocks. The device clock of the controller stops with them. The guest then sees the modelled run time.

```
host time   |--Execute--|======= host runs program =======|--CQE--|
guest time  |--Execute--|                                  |--CQE--|
                          <- guest stopped, clocks hidden ->
guest sees  Execute .......... modelled run time .......... CQE
```

- Under KVM, FEMU stops the vCPUs, sets the kvmclock back and moves the TSC of each vCPU back by the same span. Guest readers of the kvmclock, the vDSO and the TSC all skip the stop.
- Overlapped runs on several compute units share one stop.
- All I/O of the controller runs on the device clock, so other commands keep their modelled latency.
- A stop has a cost: the guest runs for a short time before the vCPUs stop. On a 20-CPU host with 4 vCPUs, the guest saw the modelled run time plus 30 to 50 us at the median.
- Time outside the guest does not stop. Network peers, the wall clock and other devices, also other FEMU controllers, move on during a stop.
- A vCPU that stays halted through many stops keeps an old pvclock page. The page can stray from the clock by about 1 ns a stop until that vCPU runs again.
- A LAPIC timer due during a stop fires once, early. Linux with high resolution timers sets it again, so a sleep does not end early.
- A program that runs past `csd_pause_max_ms` lets the guest run again, and the guest sees the rest of the host run. QEMU warns once.
- A VM stop, suspend, reset or hot plug during a program makes that one stop visible to the guest.
- If KVM fails a clock call after the kvmclock is set, the vCPUs would run on clocks that disagree. QEMU then stops the VM with an internal error.

It needs KVM on x86 with Linux 6.0 or later on the host. The guest TSC must run at the host rate, and realize refuses a `tsc-frequency` that changes it. Every namespace of the controller must be CSD or bbssd, and the controller must not share namespaces. The script `hw/femu/scripts/csd-pause-check/run.sh` boots a guest with and without the option, and prints the latency each sees.

```
-device femu,devsz_mb=4096,femu_mode=4,fdm_size=64,csd_program_dir=/home/you/csd-programs,csd_pause=on
```

Commands

I/O commands on the namespace. The opcodes are in the vendor specific range 80h to FFh. Bits 1:0 give the data direction (Base 2.3 Figure 91), and a command that moves no data may carry any value there:

Opcode	Command	Data
0xb0	Allocate device memory	none
0xc0	Free device memory	none
0xd0	Copy namespace data into device memory	none
0xe1	Execute a program	host to device
0xf2	Read device memory	device to host
0xf5	Write device memory	host to device

0xf6, 0xf7, 0xf8	Create a group, set its QoS, delete it	none

Admin commands, in the vendor specific range C0h to FFh. As a mnemonic, 0xc5, 0xc8 and 0xc9 are the CP 1.2 opcodes 0x85, 0x88 and 0x89 of the matching commands plus 40h. CP has no Load Data command, so 0xcd has no CP pair:

Opcode	Command	Data
0xc5	Program load and unload	host to device
0xc8	Program activate and deactivate	none
0xc9	Memory range set management	host to device
0xcd	Load more program data	host to device

The command layouts follow CEMU; `hw/femu/tests/csd/csd-passthru.c` builds each one.

Migration from the CEMU opcodes. CEMU and FEMU before this change used admin opcodes 0x21, 0x22, 0x23 and 0x25. NVMe Base 2.3 gives 0x21, 0x22 and 0x25 to fabrics commands, which a PCIe controller must not support, and reserves 0x23. 0x22 also declares data to the host, but it sends program bytes to the device. The Linux driver reads that bit to map the data buffer. The I/O opcodes do not change.

```
old    new    command
0x21   0xc9   memory range set management
0x22   0xc5   program load and unload
0x23   0xc8   program activate and deactivate
0x25   0xcd   load more program data
```

The device now answers the old opcodes with Invalid Command Opcode. To run an old tool unchanged, set `csd_legacy_opcodes=on`. The device then takes the old opcodes and refuses the new ones. The Commands Supported and Effects log (05h) lists the layout in use. The passthrough helper takes `CSD_LEGACY_OPCODES=1` in its environment to send the old opcodes.

NVMe CP and SLM command sets

`csd_slm_size` adds a memory namespace (SLM 1.2) and `csd_cp=on` a compute namespace (CP 1.2) after the CSD namespace. They take the ratified opcodes, and their programs, range sets and memory are separate from the CEMU ones above. The CEMU commands stay on, on the CSD namespace. See CP and SLM (§4.22).

```
CSD namespace (CSI 00h)   NVM commands + CEMU b0h..f8h, admin c5h..cdh
memory ns     (CSI 03h)   Memory Copy 01h, Read 02h, Fill 04h, Write 05h
compute ns    (CSI 04h)   Execute Program 01h, admin 85h, 88h, 89h
```

4.13.5 Use it from the guest

Check the device:

```
sudo nvme list
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(mn|sn) '
```

The model is `FEMU Computational Storage Controller` and the serial number starts with `vCSD`.

Build the guest tool

From `build-femu/` on the host, copy the tools into the guest and build the passthrough helper:

```
scp -P 8080 -i ~/images/femu-guest-key -r ../hw/femu/tests/csd femu@localhost:
./run-guest-ssh.sh make -C csd csd-passthru
```

Run the phantom smoke test

This needs no program directory. It allocates, writes and reads device memory, loads and runs a phantom program, and frees everything:

```
./run-guest-ssh.sh sudo ./csd/csd-passthru /dev/nvme0n1 smoke
```

Run a shared-library program

On the host, build the example programs and put them in the program directory (`csd-original-kernels.so` needs the lz4 development package):

```
cd hw/femu/tests/csd          # in the FEMU source tree
make csd-vadd.so csd-original-kernels.so
mkdir -p ~/csd-programs
cp csd-vadd.so csd-original-kernels.so ~/csd-programs/
```

Start FEMU with `csd_program_dir` set to that directory, then name the file alone in the guest:

```
sudo ./csd/csd-passthru /dev/nvme0n1 smoke-so csd-vadd.so
sudo ./csd/csd-passthru /dev/nvme0n1 smoke-so-all csd-original-kernels.so
sudo ./csd/csd-passthru /dev/nvme0n1 bench 4096 32
```

`bench` reports the average latency of device memory writes, reads and namespace-to-memory copies. hw/femu/tests/csd/README.md[57] lists every subcommand, the eBPF steps and the program ABI.

The vendor log page C0h counts the namespace's NAND traffic as for BlackBox (log pages and counters (§B.1)).

4.13.6 Limits and refusals

Message	Cause and fix
`CSD mode requires fdm_size to be non-zero`	Set `fdm_size`.
`CSD nr_cu must be in range [1, 64]`	`nr_cu` out of range.
`CSD nr_thread must be non-zero`	`nr_thread=0`.
`CSD csf_runtime_scale must be non-zero`	`csf_runtime_scale=0`.
`CSD csd_pause_max_ms must be in range [1, 60000]`	`csd_pause_max_ms` out of range.
`csd_pause needs a CSD namespace`	`csd_pause` on a controller with no CSD namespace.
`csd_pause: namespace N runs a mode other than bbssd or CSD`	Other modes keep their own clocks; use them on another controller.
`csd_pause cannot be used with shared namespaces: ...`	Two controllers would time one FTL on two clocks.
`csd_pause needs KVM`	Another accelerator; `csd_pause` needs `-enable-kvm`.
`csd_pause needs KVM_CLOCK_HOST_TSC and KVM_CAP_VM_TSC_CONTROL ...`	The host kernel is older than Linux 6.0.
`csd_pause needs the guest TSC at the host rate; ...`	Remove `tsc-frequency` from `-cpu`.
`csd supports at most one namespace per controller`	Two CSD namespaces, from `namespace_modes` or from `femu_mode=4` with `namespaces` above 1. Other namespaces of the controller may use other modes.

[57] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tests/csd/README.md

Message	Cause and fix
`FEMU bbssd: namespace 1 exposes ...`	The namespace does not fit the NAND geometry; see the BlackBox limits (§4.4.5).
`FEMU csd: hot_cold_sep is not supported under FDP: ...`	A knob the FDP write path does not take, on a controller in an FDP subsystem; the same list as for BlackBox (FDP (§4.5.5)).

A program load that fails returns Invalid Field to the guest, or Capacity Exceeded when the program table is full. For a missing or bad program file, QEMU prints the reason on its console after `[FEMU] Err:`, for example:

- `CSD: loading a program needs csd_program_dir to be set`
- `CSD: a program name must be a file in csd_program_dir, got "..."`
- `CSD: <path> does not resolve inside csd_program_dir`
- `CSD: failed to load shared library <path>: <reason>`

An eBPF load on a build without uBPF, a malformed load descriptor, or an unknown program type fails with Invalid Field and prints nothing.

4.13.7 Verify

1. `sudo nvme id-ctrl /dev/nvme0` reports `FEMU Computational Storage Controller`.
2. `csd-passthru /dev/nvme0n1 smoke` completes without errors.
3. With `csd_program_dir` set, `smoke-so csd-vadd.so` completes; without it, the same command fails and the QEMU console says why.

4.13.8 Troubleshooting

- **A program load fails with Invalid Field.** Read the QEMU console (or `build-femu/log`). Usually `csd_program_dir` is unset, the name contains a path, or the file is not in the directory. No message means an eBPF load on a build without uBPF, or a malformed load command.
- **An eBPF load fails.** The FEMU build has no uBPF support. Rebuild with `./femu-compile.sh --enable-csd-ubpf`.
- **`make` in the guest fails on `csd-original-kernels.so`.** It needs the lz4 development package and is meant to be built on the host. Build only `csd-passthru` in the guest.

Related issues: #60, #143, #188.

4.13.9 Citation

CSD mode is derived from CEMU[58]. We thank the CEMU authors, Qiuyang Zhang, Jiapin Wang, You Zhou, Peng Xu, Kai Lu, Jiguang Wan, Fei Wu and Tao Lu, and Emilio (@Emilio597[59]), who ported it to FEMU in #188[60]. If you use the CSD mode, please also cite:

```
@inproceedings{Zhang+26-CEMU,
  author    = {Qiuyang Zhang and Jiapin Wang and You Zhou and Peng Xu and
```

[58] https://github.com/cs-qyzhang/CEMU

[59] https://github.com/Emilio597

[60] https://github.com/MoatLab/FEMU/pull/188

```
               Kai Lu and Jiguang Wan and Fei Wu and Tao Lu},
  title     = {{CEMU: Enabling Full-System Emulation of Computational Storage
               Beyond Hardware Limits}},
  booktitle = {Proceedings of the 31st ACM International Conference on
               Architectural Support for Programming Languages and Operating
               Systems (ASPLOS '26), Volume 2},
  pages     = {323--341},
  year      = {2026},
  doi       = {10.1145/3779212.3790137},
}
```

4.13.10 Related pages

- hw/femu/tests/csd/README.md[61]: the guest tool and program ABI
- BlackBox SSD (§4.4): the FTL under the namespace
- Timing model: KV and CSD (§6.2.8)

4.14 CSD: the computational storage extension

This chapter describes how FEMU emulates a computational storage drive (`femu_mode=4`): the device memory, how programs are loaded and run, how compute time is charged, and what the guest is trusted with. To run the mode, see the CSD guide (§4.13).

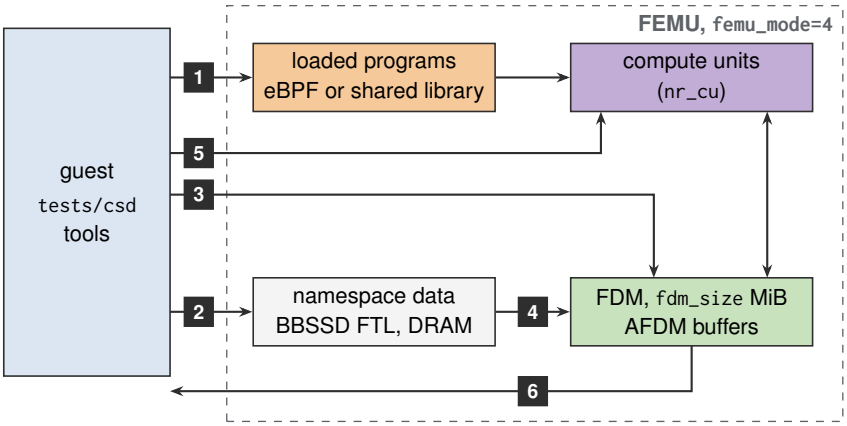
4.14.1 Origin and credit

CSD mode is derived from CEMU, by Qiuyang Zhang, Jiapin Wang, You Zhou, Peng Xu, Kai Lu, Jiguang Wan, Fei Wu and Tao Lu (ASPLOS '26), and was ported to FEMU by Emilio (@Emilio597[62]) in #188[63]. The command layouts follow CEMU's. The port keeps CEMU's device-side model and leaves out its modified guest kernel, its FDMFS file system, and its VM freezing and virtual clock changes. If you use this mode, cite CEMU as well as FEMU; the BibTeX entry is in the CSD guide's citation section (§4.13.9).

4.14.2 Purpose

A computational storage drive runs programs next to the data. The host allocates memory on the device, copies namespace data into it without a trip through host memory, runs a program on that memory, and reads back only the result. FEMU emulates the control path of such a device with NVMe commands, and runs each program inside the QEMU process on the host. The program's time on the device is modelled with a small set of compute units.

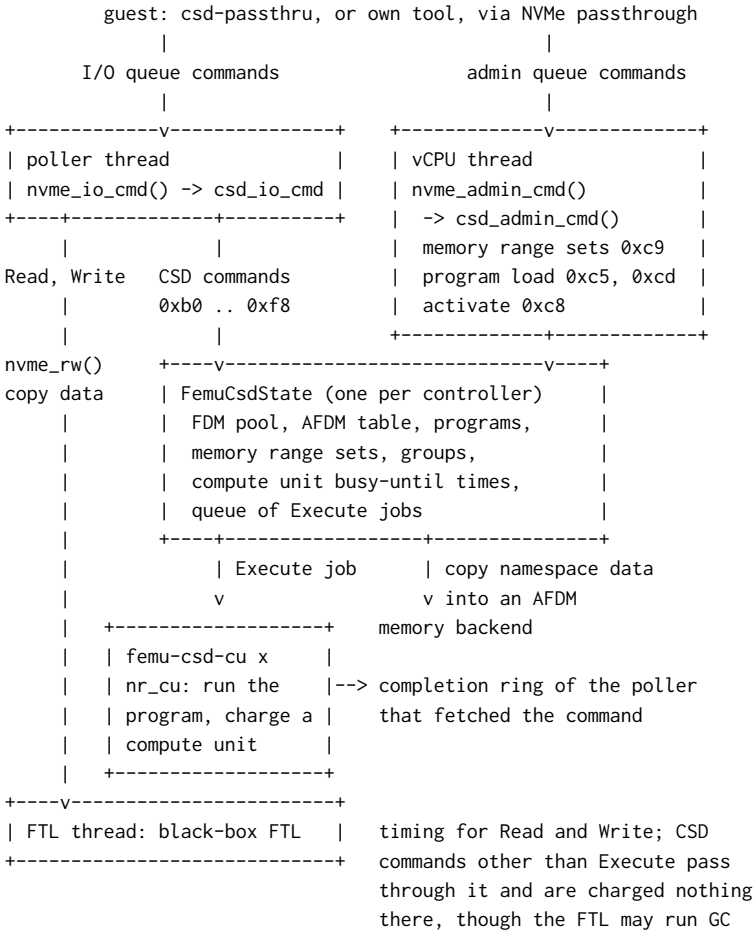
The namespace under the device memory is an ordinary block namespace on the black-box FTL, so plain reads and writes have SSD timing and garbage collection.



1 Compute Load `0x22`, Compute Activate `0x23` (admin): load a program; shared libraries come from `csd_program_dir`. **2** Write the input with ordinary NVMe writes. **3** Alloc FDM `0xb0`: reserve a buffer (AFDM) in device memory. **4** NVM to AFDM `0xd0`: copy namespace data into the buffer, charging a page read when the pages are mapped. **5** Exec `0xe1`: run the program on the buffer on the first free compute unit; it is charged its own runtime, or the host time it took times `csf_runtime_scale`. **6** Read AFDM `0xf2`: copy the result to the guest; Dealloc AFDM `0xc0` frees the buffer.

Figure 20: Computational storage: load a program, stage namespace data in a device memory buffer, execute it on a compute unit, and read the result back.

4.14.3 Place in the hierarchy



`csd_init()` runs the black-box geometry and capacity checks, builds a full black-box FTL for the namespace (`ns->ssd`), and allocates `FemuCsdState`. The state is controller-wide (`n->csd_ctrl_state`), which is why a controller may have only one CSD namespace. Its other namespaces may run other modes, but the CSD admin commands (0xc5, 0xc8, 0xc9, 0xcd) reach `csd_admin_cmd()` only through the controller's own handler table. On a controller whose `femu_mode` is not 4, for example `femu_mode=2` with `namespace_modes=nossd,,csd`, those commands fail with Invalid Opcode, so programs cannot be loaded. Set `femu_mode=4` on a controller that should run programs. A `femu_mode=4` controller with no CSD namespace has no `FemuCsdState` either, so it refuses them the same way, and log 05h does not list them. A CSD namespace makes the controller start its FTL thread.

4.14.4 Data structures

`FemuCsdState` in `hw/femu/csd/csd.c`, guarded by one mutex. No program runs under it:

[61] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tests/csd/README.md
[62] https://github.com/Emilio597
[63] https://github.com/MoatLab/FEMU/pull/188

```
FemuCsdState
+-------------------------------------------------------------------+
| fdm_capacity = fdm_size MiB       fdm_used                        |
| afdms     id   -> FemuCsdAfdm { id, size, data }                  |
| programs  pind -> FemuCsdProgram { type, active, loading,         |
|                     runtime, runtime_scale, size, data,           |
|                     module + function, or uBPF VM + JIT }         |
| mrs       rsid -> FemuCsdMrs { numr, ranges[] }                   |
| groups    id   -> FemuCsdGroup { prio, qos_flags, bandwidth,      |
|                     deadline }                                    |
| cu_next_avail[nr_cu]   when each compute unit is free again       |
| prog_used              bytes held by loaded programs              |
| workers[nr_cu]         femu-csd-cu threads                        |
| pending, running       FemuCsdJob { req, poller, program,         |
|                          AFDMs, args, runtime }                   |
+-------------------------------------------------------------------+
```

AFDMs and programs are reference counted. The table holds one reference, and each Execute that is queued or running holds one on its program and on each AFDM it names. Deallocating an AFDM or unloading or replacing a program takes it out of its table at once, but its memory, and its share of the `fdm_size` quota or of the program limits, is given back only when the last Execute using it has finished. A program whose last reference goes is closed by a compute unit thread without the CSD lock, since closing a library runs its destructors.

- **FDM and AFDM.** Function data memory (FDM) is the device memory pool, `fdm_size` MiB. An allocation from it (AFDM) is a buffer in host memory with an id. `fdm_size` is a quota: FEMU allocates each AFDM when the guest asks for it, and refuses an allocation that would pass the quota with Capacity Exceeded. Only the host-visible type (0) is supported.
- **Memory range.** A 32-byte descriptor `{nsid, len, sb}`. FEMU accepts only `nsid` 0, meaning device memory, where `sb` is an AFDM id and `len` is the bytes to use (0 for the whole AFDM).
- **Memory range set.** A stored list of up to 255 memory ranges with an id (`rsid`), so a program can be run many times without sending its ranges again. The ranges are checked only when a program runs.
- **Program.** Indexed by `pind` (1 to 65535). One program may be up to 16 MiB and all loaded programs together up to 64 MiB.
- **Group.** A priority (1 to 9, 0 means 5), QoS flags, bandwidth and deadline, kept as CEMU defines them. An execute command may name a group, which must exist, but groups do not change scheduling. The group field of Execute is 8 bits wide, so only groups 1 to 255 can be named.

4.14.5 Commands

I/O commands, on the CSD namespace (`csd_io_cmd()`):

Opcode	Command	Effect	Time charged
0x01, 0x02	Write, Read	`nvme_rw()` then the black-box FTL	black-box FTL timing
0xb0	Allocate FDM	new AFDM of `size` bytes; Dword 0 = id	none
0xc0	Deallocate AFDM	frees it, returns its bytes to the quota	none
0xd0	NVM to AFDM	copy `nlb + 1` blocks (`nlb` is 0-based) from `slba` into an AFDM at `offset`	one `pg_rd_lat` if any page of the range is mapped in the FTL, else none
0xe1	Execute	run a program on memory ranges; Dword 0 = its return value	compute unit time, below

Opcode	Command	Effect	Time charged
0xf2	Read AFDM	AFDM bytes to the host	none
0xf5	Write AFDM	host bytes into an AFDM	none
0xf6, 0xf7, 0xf8	Create group, Set QoS, Delete group	group bookkeeping	none

Admin commands (`csd_admin_cmd()`):

Opcode	Command
0xc9	Memory range set management: `sel` 0 creates a set from `numr` ranges in the data buffer (Dword 0 = `rsid`), `sel` 1 deletes one
0xc5, 0xcd	Load program, or load more of its bytes; `sel` 1 unloads
0xc8	Activate (`sel` 1) or deactivate (`sel` 0) a program

Every opcode is vendor specific, and bits 1:0 give the data direction of a command that moves data (Base 2.3 Figure 91). `csd_admin_opcode()` maps the old CEMU admin opcodes onto these handlers when `csd_legacy_opcodes=on`, and refuses the layout that is not in use:

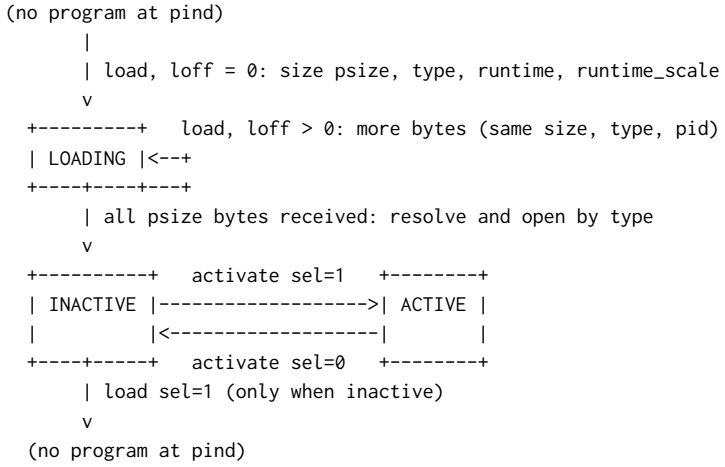
```
legacy off:  admin c5h c8h c9h cdh  --> handlers
             admin 21h 22h 23h 25h  --> Invalid Command Opcode
legacy on:   the reverse
both layouts refuse 24h
```

`hw/femu/tests/csd/csd-passthru.c` builds every one of these commands and is the reference for their layouts; `hw/femu/csd/csd.h` declares them.

NVM to AFDM reads the namespace's slice of the memory backend directly and checks the black-box mapping table only to decide whether to charge the read. It does not go through the FTL's LUN queues. Pages still held in the write buffer and deallocated pages are not mapped, so a range of only those costs nothing.

When `oncs` turns them on, the generic NVM commands (Dataset Management, Compare, Write Zeroes and the others) are handled by the common layer in `nvme_io_cmd()` before `csd_io_cmd()` is consulted.

4.14.6 Loading a program



The diagram simplifies a few edges. A load with `loff = 0` replaces whatever the index held, even an active

program; an Execute already queued or running keeps the old one. A further chunk (`loff > 0`) is accepted only while the program is still loading, so a loaded program never changes. A load with `sel=1` also unloads a program that is still loading. A further chunk must repeat the size and type, and the `pid` only when the command's `pit` field is 1. Every chunk adds its byte count toward the size, so a chunk sent twice counts twice. When the byte count reaches the size, `csd_load_program_data()` prepares the program by type, without the CSD lock held: opening a library runs its constructors, and the pollers must not wait for them. The program is still loading then, so nothing can run it, and admin commands, the only ones that change programs, run one at a time under QEMU's global lock:

Type	Value	Payload	Prepared by
Phantom	0	none needed	nothing; runs built-in code
eBPF	1	`name\0symbol\0`	read the ELF file, load the section into a uBPF VM, JIT-compile it when the command's `jit` bit is set
Shared library	3	`name\0symbol\0`	`g_module_open()` the file and look up the symbol

Bitstream programs (type 2) and any other type fail with Invalid Field, but only when the last byte arrives. If preparing fails, the load command fails and the program stays at that index in the loading state until a new load replaces it.

Program files and the trust model

`csd_program_path()` turns the guest's `name` into a host path:

- With `csd_program_dir` (§A.1.1) unset, every shared-library and eBPF load fails; only phantom programs load.
- The name must be a plain file name: not empty, no `/`, not `.` or `...`
- The joined path is resolved with `realpath()`, symbolic links included, and must still lie inside the resolved directory.

This keeps the guest from naming any other file on the host. It does not make the programs safe. A shared library runs as native code in the QEMU process, with QEMU's privileges and access to all of its memory, and the launchers start QEMU with `sudo`. A uBPF program runs in uBPF's interpreter, or as native code when JIT-compiled, also inside QEMU; FEMU adds no checks of its own to what uBPF does. So the directory must hold only programs you built and trust, and must not be writable by anyone else. FEMU does not check the directory at realize; a missing one makes every load fail.

eBPF needs a build with uBPF (`./femu-compile.sh --enable-csd-ubpf`, meson option `femu_csd_ubpf`). Without it, an eBPF load fails with Invalid Field.

4.14.7 Running a program

```
Execute (0xe1), on the poller thread (csd_exec):
 1. read numr inline ranges (+ optional extra data, dlen <= 1 MiB)
    from the data buffer, or take the stored set named by rsid
 2. lock; check the program exists and is active, and the group exists
 3. map each range to its AFDM buffer (nsid 0, sb = AFDM id), and take
    a reference on each AFDM and on the program
 4. runtime = command runtime, else program runtime; if there is one,
    take the compute unit now (see below); unlock
 5. hand the request to the CSD as a job (req->defer); the poller
    moves on without sending it to the FTL thread

on one of nr_cu femu-csd-cu threads (csd_worker):
 6. take the oldest job whose program is not running already, mark the
    program running, and call it without the CSD lock, timing it with
    the host clock:
      phantom    copy range 1 into range 0
      shared lib fn(&args)
      eBPF       JIT function or ubpf_exec(&args)
 7. lock; Dword 0 = return value (above 2^32 - 1 clamped to that; a
    negative value keeps its low 32 bits)
 8. with no runtime from step 4: runtime = measured host time x scale,
    and take the compute unit now
 9. put the request on the completion ring of the poller that fetched
    it, retrying while the ring is full; drop the references; unlock
```

Taking the compute unit means: pick the unit that is free first, hold it for the runtime from the command's arrival, and set `expire_time` to the end of that hold. A known runtime takes its unit on the poller, in arrival order, so the order in which the host happens to finish programs does not change it; a failed run still holds it. A measured runtime is known only after the run, so those take their unit in the order runs end.

The poller then posts the completion once `expire_time` has passed, as for any other command. A long program therefore holds one `femu-csd-cu` thread and nothing else: the pollers keep serving every queue, CSD admin commands and other CSD I/O commands take the CSD lock only briefly, and the vCPU that sends an admin command never waits for a program.

When a submission queue is deleted, or the controller is reset, while one of its Execute commands is queued or running, `nvme_drain_sq()` first calls `femu_csd_drain_sq()`, which frees its queued jobs and detaches the request from a running one. A running program still runs to the end, but its result is dropped, as the specification allows for commands on a deleted queue, and nothing touches the freed request.

The program sees one argument, `struct femu_csd_args` (`hw/femu/tests/csd/femu-csd-kernel.h`):

```
int        numr          number of memory ranges
void     **mr_addr       one pointer per range; by convention
                         mr_addr[0] is output, mr_addr[1] input
long long *mr_len        bytes per range
long long  cparam1, cparam2   from the command
void      *data_buffer   extra bytes after the ranges, or NULL
long long  buffer_len
```

Compute units and runtime scaling

The program runs to completion on the host as soon as a `femu-csd-cu` thread is free; the compute units' busy-until times then decide when the guest sees the completion. This is the same "compute, then hold" approach the rest of FEMU uses (timing model (§6.2.1)).

```
runtime (ns), first that applies:
  1. the runtime field of the Execute command
  2. the runtime given when the program was loaded
  3. measured host time x program runtime_scale / 10
  4. measured host time x csf_runtime_scale

compute units (nr_cu = 3):
  cu0  |==== A ====|
  cu1  |== B ==|    |===== D =====|
  cu2  |====== C ======|
                   ^ D arrives: cu1 is already free, D starts at once

  cu   = the unit with the smallest cu_next_avail
  start = max(arrival, cu_next_avail[cu])
  cu_next_avail[cu] = start + runtime
  expire_time = cu_next_avail[cu]
```

`nr_cu` (§A.1.1) bounds how many programs run at once in device time; `csf_runtime_scale` (§A.1.1) stands for how much slower the device's cores are than the host's.

The completion carries the program's return value, so the poller cannot post it before the worker has finished the run. A runtime shorter than the host's own run time, declared or scaled below 1, is therefore not reached: the guest sees the host's time. `csd_finish_locked()` warns once when a run's runtime is shorter than its measured host time, unless `csd_pause` is on.

Freeze service (csd_pause)

With `csd_pause=on` the worker hides the host run from the guest. It calls `femu_freeze_begin()` before the run and `femu_freeze_end()` after it, with no lock held. A main loop bottom half in `hw/femu/femu-freeze.c` does the stop and the restart under the BQL, so the worker never takes the BQL.

```
worker                          main loop (BQL)
------                          ---------------
femu_freeze_begin() --BH-->     pause_all_vcpus()
  wait for the answer           save kvmclock C0 + host TSC, TSC offsets
                                stop the device clock, cpu_disable_ticks()
run program (hidden)
femu_freeze_end()  --BH-->      KVM_SET_CLOCK(C0); measure the span lost
                                move each TSC offset back by that span
                                device clock hides the span
                                cpu_enable_ticks(), resume_all_vcpus()
```

- The service is process-wide and counts its users, so runs on several compute units and several controllers share one stop.
- `femu-freeze-kvm.c` reads the kvmclock with the host TSC before and after the set. The difference to the clock the guest would read is the span. Each vCPU TSC offset moves back by the same span. The cycles come from the pvclock scale that KVM derives from the host TSC rate, with the fraction of a cycle carried to the next stop. So an old pvclock page of a halted vCPU reads the time a new one does, up to the rounding of the two clock samples. That rounding is about 1 ns a stop and need not cancel out. An old page keeps the sum until its vCPU next runs, so a reader that mixes the vDSO and a system call can see a step back of that sum.
- With `csd_pause`, `femu_io_now()` reads the device clock. It is the raw monotonic clock less every hidden span. It also stands while the VM is stopped. Under qtest it is the virtual clock.
- A reset, suspend, VM stop, hot plug or other resume during a stop marks it spoiled. The restart then

does no clock step, and the guest sees that span.
- `csd_exit()` sets `csd->stopping` and wakes the waiters; a worker that did not get its stop fails the job instead of running it.
- A REALTIME timer ends a stop after `csd_pause_max_ms`.
- A KVM failure after the set leaves the kvmclock and the TSC apart, and no set can undo it to the instant. The service then stops the VM with `RUN_STATE_INTERNAL_ERROR` instead of a resume.

4.14.8 Parameters

Properties: CSD (§A.1.1). The NAND geometry (§A.1.1), NAND timing (§A.1.1) and garbage collection (§A.1.1) properties apply to the namespace as in the black-box mode.

Property	Role
`fdm_size`	device memory quota in MiB; required
`nr_cu`	compute units, 1 to 64
`csf_runtime_scale`	host-to-device time factor for programs that give no runtime; not 0
`csd_program_dir`	where shared-library and eBPF programs load from
`csd_pause`, `csd_pause_max_ms`	stop the guest during a program run, and the longest stop
`pg_rd_lat`	the cost of an NVM to AFDM copy
`nr_thread`, `time_slice`, `context_switch_time`	accepted for CEMU configurations; no effect, and a non-default value warns at realize (`nr_thread` must still not be 0)

The smallest device line that loads programs:

```
-device femu,devsz_mb=4096,femu_mode=4,fdm_size=64,nr_cu=2,csd_program_dir=/home/you/csd-programs
```

4.14.9 Counters

- The vendor log page C0h counts the namespace's NAND traffic as for the black-box mode (log pages and counters (§B.1)). NVM to AFDM copies do not appear there, because they do not go through the FTL.
- The SMART log counts plain Read and Write. CSD commands are not counted.
- FEMU keeps no counter of programs run or compute unit use. Time the Execute commands from the guest; `csd-passthru benchmark-kernels` times whole program runs, and `csd-passthru bench` times the device memory and NVM to AFDM transfers.

4.14.10 Validation status

- qtest cases in CI: `csd-program-dir`, run with the property set, checks that a missing file, a symbolic link out of the directory and a file that is not a library are refused; `csd-fuzz` fuzzes the fields of the CSD I/O commands; `csd-exec-concurrency` builds a shared-library program that sleeps (it skips when the host has no C compiler), loads and runs it, and checks that a read on another queue, an admin command and another program complete while it runs, that a loaded program refuses a further load piece, and that deleting the program's queue or disabling the controller under it leaves the controller working. No qtest covers the case where `csd_program_dir` is unset, or the `/`, `.` and `..` name rules. Phantom and eBPF programs and compute unit timing are tested only in a guest. `csd-pause` and its control `csd-pause-off` run a program that sleeps 20 ms with a declared 300 ms: with `csd_pause` it completes exactly when the test clock reaches 300 ms, and makes one stop and one restart; without it, it completes on the host clock. A qtest has no KVM, so it cannot show the guest clocks stop.

- The script `hw/femu/scripts/csd-pause-check/run.sh` boots a KVM guest twice, with the option and without it. It times Execute from the guest. It also watches for a clock step back on each vCPU and for a sleep that ends early.
- The documentation check starts each CSD example and writes and reads one block of the namespace.
- Shared-library and eBPF programs run only in a guest, with `hw/femu/tests/csd/csd-passthru` and the example programs in `hw/femu/tests/csd/`. When `femu-test.sh` finds a CSD namespace, it runs a smoke check that allocates device memory, copies into it and reads it back; it runs no program. CI does not build uBPF, so the eBPF path is not tested there.

4.14.11 Limits

- Programs run on `nr_cu` host threads, one per compute unit, so at most `nr_cu` run at once on the host; further Execute commands queue for a thread. Two runs of the same program never overlap; different programs do. A program that never returns keeps its thread and, at device removal (`device_del`), keeps QEMU waiting for it; FEMU cannot stop native code it has called.
- A program runs on the AFDMs it was given while other commands may read or write the same AFDMs; FEMU does not order them. Device memory freed while a program uses it counts against `fdm_size` until the program returns.
- The guest's program runs in the QEMU process. Treat `csd_program_dir` as code you are choosing to run on the host.
- NVM to AFDM charges one page read whatever its size, and does not queue on LUNs or channels.
- Groups and their QoS settings are stored but do not affect scheduling.
- Device memory transfers (Read and Write AFDM) take no device time.
- One CSD namespace per controller. No `meta` and no namespace management.

Refusal messages are listed in the CSD guide (§4.13.6).

4.14.12 Extending the mode

- **A new program type.** Add a case to `csd_load_program_data()` to prepare it, and one to the switch in `csd_run_program()` to run it.
- **Group scheduling.** `csd_exec()` already looks up the group. Choosing the compute unit by group priority or deadline would happen in `csd_finish_locked()`, where it picks the unit with the smallest `cu_next_avail`, and the order jobs leave the queue in `csd_worker()`.
- **Copy cost from the FTL.** `csd_nvm_to_afdm()` has the mapped page count from `csd_check_nvm_ftl_range()`; charging each mapped page through the media model (`ssd_advance_status()`) instead of one `pg_rd_lat` would add LUN and channel contention.
- **Preemption.** A compute unit thread runs a program to the end. A time slice, which CEMU's `time_slice` and `context_switch_time` describe, would need programs that yield.

4.14.13 CP and SLM command sets

`csd_slm_size` and `csd_cp` add the ratified NVMe Subsystem Local Memory (SLM 1.2) and Computational Programs (CP 1.2) command sets. The guide is CP and SLM (§4.22).

Namespaces

`femu_csd_add_namespaces()` appends the memory namespace, then the compute namespace, after the configured ones. They keep `femu_mode` 4, so they share the CSD handler table, and their CSI tells them apart.

They have no backend slice and no FTL. `NS_CSD()` is true only for the block namespace, so no FTL code sees them. `csd_init()` copies its handler table to them.

```
nvme_io_cmd(): NS_SLM / NS_CP --> csd_io_cmd() before the NVM opcodes
csd_io_cmd():  NS_SLM --> csd_slm_io_cmd()   NS_CP --> csd_cp_io_cmd()
               else   --> the CEMU switch (unchanged)
csd_admin_cmd(): 85h 88h 89h --> csd_cp_admin_cmd() with csd_cp only
```

The early hook is necessary. SLM uses 04h and 05h, which the NVM switch takes as Write Uncorrectable and Compare.

State

```
FemuCsdState
  slm, slm_size, slm_lock     the memory; slm_lock nests inside lock
  cp_prog[16]                 PIND 0, 1 device-defined; 2..15 downloaded
  cp_mrs                      RSID -> FemuCsdCpMrs
  lock, workers, job queue    shared with the CEMU commands
```

Execute and its lifetime

Three counts keep a program and its range set alive and in use:

```
fetch      csd_cp_exec():   program->refs++, program->use++, mrs->use++,
                            req->csd_use = {PIND, RSID}
run        csd_worker():    copy ranges to scratch (slm_lock), run unlocked,
                            write back changed bytes if job->req is set
job freed  worker:          program->refs--   (the job's own reference)
CQE posted nvme_post_cqe(): femu_csd_post_cqe(): under lock, write the
                            CQE, then program->use--, mrs->use--
dropped    nvme_free_sq():  femu_csd_req_done()
```

- `use` is the protocol state: Program In Use and Memory Range Set In Use while a command is outstanding.
- `refs` is the object lifetime: a program dropped while it runs stays alive until the worker frees its job.
- `femu_csd_drain_sq()` clears `job->req` under `lock`, and the write-back checks it under the same lock. A command dropped while its program runs therefore writes nothing. A command dropped later, while it waits for its modeled time or for room in its completion queue, keeps what it wrote.

Memory access

Every access to the memory takes the lock `slm_lock`. Host transfers go through a buffer. Thus no DMA runs under the lock, and a failed DMA changes nothing. Programs never see an address in the memory. They work on scratch copies.

Timing

Memory Copy, format 2h, puts its block ranges in `req->dsm_ranges` for the FTL thread. There, `femu_ftl_process_req()` sends such a request to `bb_ftl_read_ranges()`, the read loop that `ssd_copy()` also uses. Copy, format 4h, gives the FTL no source range, so only the write is charged.

4.14.14 Source map

File	Contents
hw/femu/csd/csd.c[64]	state, every CSD command, program loading, compute unit threads, execution
hw/femu/csd/csd-int.h[65]	state shared by the CEMU and the CP and SLM code
hw/femu/csd/cp-slm.c[66]	the SLM and CP command sets, their namespaces, Identify and logs
hw/femu/csd/csd.h[67]	command layouts, opcodes, program types
hw/femu/tests/csd/[68]	guest tool, example programs, program ABI
hw/femu/bbssd/[69]	the black-box FTL under the namespace
hw/femu/femu.c[70]	the one-CSD-namespace check, FTL thread start

4.14.15 Related pages

- CSD guide (§4.13): launching, guest tool, citation
- Timing model: KV and CSD (§6.2.8)
- BlackBox SSD guide (§4.4): the FTL under the namespace

4.15 Several namespaces and devices

One FEMU controller can expose several namespaces, and each namespace can run its own mode. A controller can, for example, offer a BlackBox namespace for random writes next to a ZNS namespace for zoned data. You can also start several FEMU controllers in one guest.

Namespaces created this way exist from boot. To create and delete namespaces from the guest while it runs, see namespace management (§4.17).

4.15.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2). Each namespace needs what its own mode needs: a ZNS namespace needs a guest kernel with ZNS support, and a KV namespace gets no block device.
- Host memory: `devsz_mb` is split between the namespaces, so the total does not grow with the namespace count. Each extra controller adds its own `devsz_mb`.

4.15.2 Launch

There is no launcher for this. Take any `run-*.sh` script and change its `-device femu` line.

Two BlackBox namespaces that split 4 GiB evenly:

```
-device femu,devsz_mb=4096,femu_mode=1,namespaces=2
```

The same with explicit sizes of 3 GiB and 1 GiB. A comma inside a property value is written twice on the QEMU command line:

[64] https://github.com/MoatLab/FEMU/blob/master/hw/femu/csd/csd.c
[65] https://github.com/MoatLab/FEMU/blob/master/hw/femu/csd/csd-int.h
[66] https://github.com/MoatLab/FEMU/blob/master/hw/femu/csd/cp-slm.c
[67] https://github.com/MoatLab/FEMU/blob/master/hw/femu/csd/csd.h
[68] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tests/csd/README.md
[69] https://github.com/MoatLab/FEMU/blob/master/hw/femu/bbssd/ftl.c
[70] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu.c

```
-device femu,devsz_mb=4096,femu_mode=1,namespaces=2,namespace_sizes=3G,,1G
```

Four namespaces, each in a different mode:

```
-device femu,devsz_mb=4096,femu_mode=1,namespaces=4,namespace_modes=nossd,,bbssd,,znssd,,kvssd
```

Two controllers, one BlackBox and one ZNS, each with its own memory:

```
-device femu,id=nvme0,devsz_mb=1024,femu_mode=1 -device femu,id=nvme1,devsz_mb=1024,femu_mode=3
```

4.15.3 Configuration

Properties: mode, capacity and namespaces (§A.1.1).

- `namespaces`: 1 to 256 namespaces.
- `namespace_sizes`: the size of each namespace, in bytes or with a QEMU size suffix (`4G`, `512M`). One entry per namespace, each at least one 512-byte sector; the sum may be all of `devsz_mb`. Unset, `devsz_mb` is split evenly, each share rounded down to 512 bytes. Each size, given or split, is then rounded down to whole logical blocks (`512 << lba_index` bytes; 512 bytes for a KV namespace), and a size under one block makes an empty namespace. Capacity that no namespace takes, because the sizes sum to less than `devsz_mb` or because of this rounding, stays unused and is not reported in `tnvmcap`.
- `namespace_modes`: the mode of each namespace, one of `nossd`, `bbssd`, `znssd`, `ocssd`, `csd` and `kvssd`, one entry per namespace. Unset, every namespace runs `femu_mode`. The controller's model number and serial still come from `femu_mode`.

The namespaces are packed one after another in the controller's memory, so none can overwrite another.

The mode properties apply to each namespace of that mode:

- Each BlackBox or CSD namespace has its own FTL with the full NAND geometry (`nchs`, `blks_per_pl` and so on). Each namespace must fit in that geometry on its own, with room for GC.
- Each ZNS namespace builds its zones from the `zns_` properties and its own size, so a smaller ZNS namespace gets smaller zones (zone size (§4.7.3)). Keep each ZNS namespace's size a power of two, or Linux may refuse its zone size.
- Each KV namespace has its own key space.

4.15.4 Use it from the guest

List the namespaces and look at one:

```
sudo nvme list
sudo nvme list-ns /dev/nvme0
sudo nvme id-ns /dev/nvme0n2
```

Block namespaces appear as `/dev/nvme0n1`, `/dev/nvme0n2` and so on, in namespace order; `nvme list` shows which ID each one has. A ZNS namespace is a zoned block device. In the four-mode example it is the third namespace:

```
cat /sys/block/nvme0n3/queue/zoned
```

A KV namespace has no block device. Address it by namespace ID through the controller, as in the KV guide (§4.11.4), with `-n` set to its ID.

The SMART log and the vendor log page C0h report the controller as a whole: C0h sums its counters over every BlackBox, CSD and KV namespace, and reports the largest block read count among them. The second command below prints the controller's WAF times 1000 (log pages and counters (§B.1)).

```
sudo nvme smart-log /dev/nvme0
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
```

A second controller is `/dev/nvme1` with its own namespaces.

4.15.5 Limits and refusals

Message	Cause and fix
`namespaces must be in [1, 256]`	`namespaces` out of range.
`namespace_sizes count must equal namespaces=2`	Give one size per namespace. Check that every comma inside the value is doubled.
`namespace_sizes: invalid size '3X'`	A size QEMU cannot parse.
`namespace_sizes sum (2147483648 B) exceeds the backend capacity (1073741824 B)`	Raise `devsz_mb` or lower the sizes.
`backend capacity N B is too small for M namespaces`	`devsz_mb=0`.
`namespace_sizes: '100' is smaller than a sector`	Each size must be at least 512 bytes.
`namespace_modes count must equal namespaces=2`	Give one mode per namespace.
`namespace_modes: unknown mode 'foo'`	Use `nossd`, `bbssd`, `znssd`, `ocssd`, `csd` or `kvssd`.
`ocssd supports a single namespace`	OCSSD cannot share a controller.
`csd supports at most one namespace per controller`	Only one CSD namespace. Its neighbours may use other modes.
`meta: namespace 2 runs a mode without metadata support (block or no-SSD only)`	`meta` needs every namespace to be NoSSD or BlackBox.
`streams requires NVM namespaces`	Streams needs every namespace to be NoSSD or BlackBox.
`FDP with more than one namespace needs every namespace in bbssd mode or every one in nossd mode`	FDP namespaces share the endurance group's units, which only these modes place into. See FDP (§4.5.3).
`FDP with more than one namespace needs namespace sizes in whole NAND pages (4096 bytes); namespace 1 has 526336 bytes`	Set `namespace_sizes` to multiples of the page size.
`FEMU bbssd: namespace 1 exposes 4096 MiB of the 2048 MiB this geometry has, ...`	A BlackBox namespace is larger than the NAND geometry allows. Each namespace is checked against the full geometry.

4.15.6 Verify

1. `sudo nvme list-ns /dev/nvme0` lists IDs 1 to `namespaces`.
2. `sudo nvme list` shows one block device per non-KV namespace, with the sizes you set.
3. Data written to one namespace does not appear in another.

4.15.7 Troubleshooting

- **Only one namespace appears.** `namespaces` defaults to 1. Set it, and check that the commas in `namespace_sizes` and `namespace_modes` are doubled. A single comma ends the property and QEMU reads the next item as a new property.

- **I want random-write and zoned capacity on one ZNS device.** Linux rejects ZNS conventional zones, so use a BlackBox namespace next to a ZNS namespace instead.
- **I want several separate SSDs.** Add one `-device femu` per SSD, each with its own `id` and `devsz_mb`.
- **Data is gone after QEMU exits.** Every namespace lives in host memory only. A guest reboot keeps the data.

Related issues: #26, #52, #121.

4.15.8 Related pages

- Namespace management, metadata and protection information (§4.17)
- Choosing a mode: which features combine (§4.1.3)
- Architecture: mode backends (§3.1.4)

4.16 Subsystems, controllers and namespaces

This chapter describes how FEMU arranges NVMe subsystems, controllers and namespaces: how a controller's memory backend is split between namespaces, how each namespace runs its own mode, how Namespace Management creates and deletes namespaces at run time, how a subsystem shares namespaces between controllers, and where metadata and protection information are kept. For usage, see several namespaces (§4.15) and namespace management, metadata and PI (§4.17).

4.16.1 Purpose

An NVMe device is a subsystem of one or more controllers; each controller exposes namespaces, and each namespace is a range of logical blocks with its own command set. FEMU models this so you can:

- give one controller several namespaces, each in a different mode, such as a BlackBox namespace next to a ZNS namespace;
- create, delete, attach and detach namespaces from the guest;
- attach one namespace to several controllers;
- store per-block metadata and protection information.

4.16.2 Place in the hierarchy

```
femu-subsys device id=S [ns_mgmt=on] [fdp=on]     optional; not on any bus
  |  CNTLID table (up to 256), SUBNQN, endurance group (FDP),
  |  shared storage object (ns_mgmt=on only)
  |
  +-- femu device, subsys=S       controller CNTLID 0   (PCIe endpoint)
  |      queues, pollers, one FTL thread
  |      namespace table: 256 slots, NSID = slot + 1
  |        NSID 1  mode A  -> backend bytes [0, size1)
  |        NSID 2  mode B  -> backend bytes [size1, size1 + size2)
  |        ...
  |      DRAM backend (devsz_mb)
  |
  +-- femu device, subsys=S       controller CNTLID 1
         its own namespaces and backend, unless the subsystem has ns_mgmt=on

femu device, no subsys=          a controller without a subsystem: the same,
                                   with CNTLID 0 and no subsystem features
```

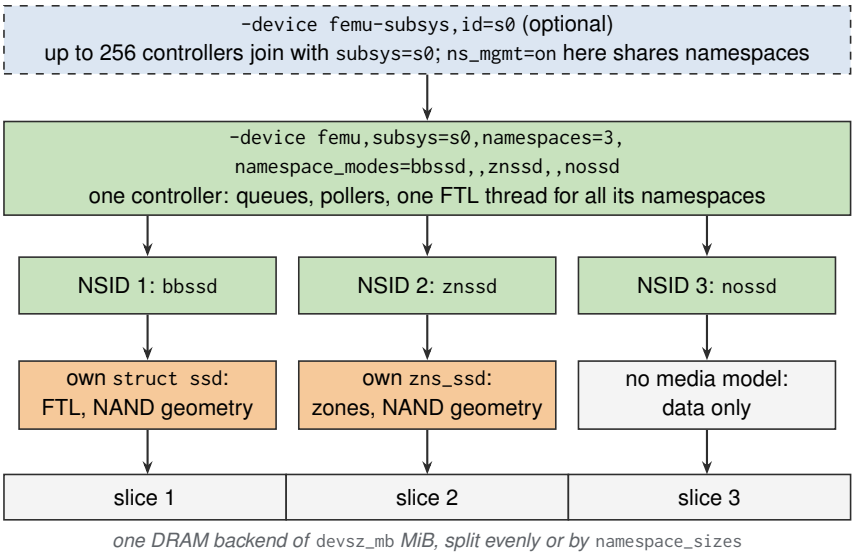


Figure 21: One controller with three namespaces of different modes: each BBSSD or ZNS namespace builds its own FTL and NAND model, and all of them share slices of one DRAM backend. Sharing a namespace between controllers is limited to NoSSD or BBSSD without `namespace_modes`.

There is no namespace `-device`. Namespaces live inside the controller (or the subsystem's storage object) and are built from controller properties. `femu-subsys` creates an `nvme-bus`, but no FEMU device plugs into it.

4.16.3 Data structures

Structure	File	Fields that matter
`NvmeSubsystem`	`nvme.h`	`ctrls[256]` (CNTLID slots), `subnqn`, `endgrp` (FDP), `ns_mgmt`, `storage` (the shared storage object), `ns_lock`
`FemuCtrl`	`nvme.h`	`namespaces` (256 slots), `num_namespaces` (boot count), `namespace_limit` (slots in use: the boot count, or 256 with Namespace Management), `namespace_pool_size`, `mbe` (DRAM backend), `attached_ns` (per-controller attachment bitmap for shared namespaces), `cntlid`, `ext_ops` (the controller's mode table)
`NvmeNamespace`	`nvme.h`	`id`, `size` (bytes the host sees), `extent_size` (bytes reserved in the backend), `backend_offset`, `start_block`, `femu_mode`, `csi`, `ext_ops` (this namespace's mode table), `allocated`, `attached`, `util` and `uncorrectable` bitmaps, `mdata`, the mode state (`ssd` for bbssd and CSD, `zns` for ZNS), `endgrp` and `fdp`

4.16.4 Namespace sizes and backend partitioning

Every controller has one DRAM backend of `devsz_mb` MiB, created in `femu_realize()`. For bbssd with `op_pcent` the backend is the full NAND capacity of the geometry instead, and the exposed capacity is that NAND capacity divided by `1 + op_pcent / 100`, so the spare space is `op_pcent` percent of the exposed size (`op_pcent=25` exposes 80 % of the NAND).

`nvme_resolve_ns_sizes()` then sizes the boot namespaces:

- `namespace_sizes` unset: the exposed capacity is split evenly, each share rounded down to 512 bytes.
- `namespace_sizes` set: one size per namespace, each at least one 512-byte sector; the sum must fit in the exposed capacity (`devsz_mb`, or the over-provisioned NAND capacity with `op_pcent`), all of it, whatever

the namespace count.

Either way each size is then rounded down to whole logical blocks of the format `lba_index` selects (`512 << lba_index` bytes), so no slice ends in part of a block the host cannot address and every slice starts on a block boundary. A KV namespace is sized in bytes and is rounded to 512 bytes only. A size smaller than one logical block leaves an empty namespace.

`nvme_init_namespaces()` packs the slices in NSID order. Each namespace records where its slice starts in `backend_offset`, and every data path adds it to the byte offset of an LBA, so no namespace can reach another's data:

```
devsz_mb=4096, namespaces=4, namespace_modes=nossd,,bbssd,,znssd,,kvssd

backend  0        1 GiB      2 GiB      3 GiB      4 GiB
         +----------+----------+----------+----------+
         | NSID 1   | NSID 2   | NSID 3   | NSID 4   |
         | nossd    | bbssd    | znssd    | kvssd    |
         +----------+----------+----------+----------+
backend_offset: 0    1 GiB      2 GiB      3 GiB

byte of LBA x in NSID n = backend_offset(n) + x * logical block size
```

The namespace's NSZE, NCAP and NUSE start as its slice size divided by the logical block size of its format. A zoned namespace then lowers NSZE to its whole zones (see ZNS (§4.8.4)); a KV namespace measures its capacity in bytes.

The pool reported to the host is the sum of the boot slices: TNVMCAP is the pool size and UNVMCAP the part no namespace holds. Backend space past the last boot slice, including what the rounding above leaves, is not in the pool and no namespace uses it.

4.16.5 Per-namespace modes

`namespace_modes` gives each namespace its own mode, from the tokens `nossd`, `bbssd`, `znssd`, `ocssd`, `csd` and `kvssd`. Unset, every namespace runs `femu_mode`. `nvme_resolve_ns_modes()` parses the list.

```
femu_realize()
  nvme_init_namespaces()        sizes, slices, modes, per-namespace checks
  nvme_register_extensions()    controller ext_ops = table of femu_mode
                                (used by the admin paths)
  for each namespace:
    nvme_register_extensions_ns()   ns->ext_ops = table of the ns's mode
    ns->ext_ops.init(n, ns)         bb_init, zns_init, kvssd_init, ...
  start one FTL thread if any namespace is bbssd, ZNS or CSD
```

At run time the mode is chosen per command, not per controller:

- I/O commands that the common path in `nvme-io.c` does not handle itself go to `ns->ext_ops.io_cmd` of the namespace the command names.
- The controller's single FTL thread routes each request by `req->ns`: ZNS namespaces to `zns_ftl_process_req()`, bbssd and CSD namespaces to `bb_ftl_process_req()`; NoSSD and KV cost nothing there.
- Each namespace reports its own command set (CSI 2 for ZNS, 1 for KV, 0 otherwise). The controller always advertises NVM, Zoned and KV in its I/O Command Set list; only the Changed Zone List entry in the supported log pages depends on a zoned namespace being present.

The controller's model number and serial come from its own femu_mode. Every namespace still advances its mode's serial counter as it starts, so serials keep their values, but only a namespace of the controller's mode names the controller. When none does (a KV controller whose KV namespace is not namespace 1, for example), the controller is named from femu_mode once its namespaces are up (nvme_set_ctrl_name()).

Each namespace keeps its own mode state. Every bbssd or CSD namespace has a complete FTL with the full NAND geometry, and must fit in that geometry on its own with room for GC. Every ZNS namespace builds its own zones from the shared zns_ properties and its own size. Every KV namespace has its own key space.

Rule	Why	Where
ocssd only as the controller's own mode, and only with one namespace	Open-Channel tables are controller-wide	nvme_init_namespaces()
at most one csd namespace per controller	CSD keeps controller-wide state	nvme_init_namespaces()
no kvssd with FDP	placement takes every line, leaving the KV store none	nvme_init_namespaces()
meta and streams need every namespace to be nossd or bbssd	only the block data path carries them	nvme_init_namespaces()
with FDP and more than one namespace: all bbssd in whole NAND pages, or all nossd; no fdp_trim_erase_all	the namespaces share one FTL, one slice each, and the endurance group's handles (FDP (§4.6.4))	nvme_check_fdp_namespaces()

4.16.6 Namespace management

With ns_mgmt=on on a controller, and when the controller and every boot namespace run the same mode, NoSSD or bbssd, with no dps and no FDP (nvme_ns_mgmt_supported()), the controller:

- reports NN = 256 and opens all 256 slots (namespace_limit);
- sets OACS bit 3 (Namespace Management) and OAES Namespace Attribute notices;
- reports each namespace's NVMCAP.

In any other configuration ns_mgmt=on adds no Namespace Management. Realize refuses it outright with a subsystem that lacks ns_mgmt, and with power_loss or cxl_ssd.

Create, attach, detach, delete

```
pool = sum of boot slices          (all of it in use at boot)

  | NSID 1 (2 GiB)        | NSID 2 (2 GiB)        |     Delete NSID 2
  | NSID 1 (2 GiB)        |      free 2 GiB       |     Create 1 GiB
  | NSID 1 (2 GiB)        | NSID 2 (1 GiB) | free |     detached until Attach

create: lowest free NSID; extent = first gap that fits (first fit)
```

Namespace Management Create (nvme_ns_mgmt() in nvme-admin.c, then nvme_ns_create() in femu.c):

- accepts only the NVM command set, with NCAP equal to NSZE (a smaller NCAP fails with Thin Provisioning Not Supported), an LBA format the controller offers, and a PI setting only if pi=on;
- rounds the size up to a NAND page for bbssd (secs_per_pg * secsz) and to 4 KiB for NoSSD;
- takes the lowest free NSID and the first gap in the pool that fits (nvme_ns_find_extent()); no gap fails with Namespace Insufficient Capacity;
- for bbssd, refuses a new namespace once bbssd_ns_limit namespaces are allocated, and refuses one that would leave its FTL no room for GC;

- zeroes the slice, builds the mode state, and leaves the namespace detached.

The controller pauses its pollers and FTL thread around create and delete, so no request runs against a half-built or half-freed namespace. Attach and detach flip `attached` (or a bit of `attached_ns` for shared namespaces) and send Attached Namespace Attribute Changed notices to the controllers they affect. Delete releases the slice; the gap is reused by the next create.

4.16.7 Subsystems and shared namespaces

A controller joins a subsystem with `subsys=<id>`; the subsystem must come first on the command line. `nvme_init_subsys()` gives it the lowest free CNTLID. What else is shared depends on the subsystem's properties:

Subsystem	Controllers	What they share
plain	any number	only the CNTLID space; each keeps its own namespaces and backend, and none can use Namespace Management, so the controller lists (CNS 12h, 13h) are not available
`fdp=on`	one	the endurance group and its FDP state (FDP (§4.6))
`ns_mgmt=on`	any number, NoSSD or bbssd, all alike	one namespace table, backend and set of FTLs, and the SUBNQN

A controller with `streams=on` also cannot share a subsystem.

Shared namespaces

```
femu-subsys,ns_mgmt=on
  storage object (an unrealized femu, a copy of the first controller's
  properties): namespace table, DRAM backend, one FTL per bbssd namespace
      ^                       ^                       ^
      | attached_ns bitmap    | attached_ns bitmap    |
  controller A (CNTLID 0)  controller B (CNTLID 1)   ...
  own queues, pollers, FTL thread (bbssd only), namespace-change log
  all I/O decode and FTL work of every controller under subsys->ns_lock
```

- The subsystem must be realized before its controllers. The first controller builds the boot namespaces; `nvme_subsys_take_storage()` then moves them, with the backend, to a storage object that copies the first controller's properties. That controller starts with every boot namespace attached; later controllers start with none.
- Later controllers must match the first in mode, `meta`, `pi`, `mc`, `dpc`, `nlbaf`, `vwc` and `oncs`. Their own capacity and geometry properties do not create storage.
- Identify Controller reports CMIC bit 1 (multiple controllers) and the subsystem's NQN; each namespace reports NMIC from its creation, and a private namespace cannot be attached to two controllers at once.
- `subsys->ns_lock` serializes command decode on the pollers and request processing on the FTL threads of all controllers, so two controllers never run the same FTL at once.
- The namespaces and backend outlive any one controller. They are released when the subsystem is removed and its last controller has left.

`femu-subsys` with both `ns_mgmt=on` and `fdp=on` is refused.

4.16.8 Metadata and protection information

```
Identify Namespace LBA formats with meta=8, nlbaf=5:
  index 0..4   512 B, 1 KiB, 2 KiB, 4 KiB, 8 KiB      no metadata
  index 5..9   the same sizes                          8 bytes metadata
  boot format = nlbaf + lba_index (5 with lba_index=0)

where the bytes live:
  data      DRAM backend, at backend_offset + LBA * block size
  metadata  ns->mdata, a separate host buffer of (blocks * meta) bytes,
            one entry per LBA, under ns->mdata_lock
  transfer  separate buffer (MPTR, mc bit 1), or interleaved after each
            block in the data buffer (extended LBA, mc bit 0)
```

- `meta` sets the metadata bytes per block and doubles the format list. `nlbaf` is then at most 8.
- With `pi=on` and `meta` of 8 or more, Identify reports DPC 1Fh: PI types 1 to 3, in the first or last 8 bytes of the metadata. The host picks a type with Format NVM or at Namespace Management create. FEMU generates the PI when PRACT is set and checks the guard (CRC-16 T10-DIF), application tag and reference tag as the PRCHK bits ask (`nvme-pi.c`).
- `dpc` and `dps` describe PI without these paths. A non-zero `dps` cannot pass the realize checks, so use `pi`.
- Metadata and PI work only on NoSSD and bbssd namespaces and not with FDP.

4.16.9 Threads and locks

Lock	Protects	Taken by
`subsys->ns_lock`	the shared namespace table and FTLs (shared namespaces only)	pollers around command decode, FTL threads around each request
`ns->mdata_lock`	the metadata buffer of one namespace	every metadata read and write
poller pause (`nvme_pause_pollers()`)	namespace lifecycle, Format, Sanitize	admin path; pauses every controller of a shared subsystem

4.16.10 Parameters

Properties are in the property reference under mode, capacity and namespaces (§A.1.1), namespace management, streams and power loss (§A.1.1), LBA formats, metadata and protection (§A.1.1) and shared namespaces (§A.1.2).

Property	On	Effect and interactions
`devsz_mb`	`femu`	Backend size, split between the boot namespaces.
`namespaces`	`femu`	Boot namespaces, 1 to 256.
`namespace_sizes`	`femu`	Per-namespace sizes; commas doubled on the QEMU command line.
`namespace_modes`	`femu`	Per-namespace modes; see the rules above. Refused with a shared subsystem.
`op_pcent`	`femu`	bbssd: backend = NAND capacity; exposed capacity = NAND capacity / (1 + op_pcent / 100).
`ns_mgmt`	`femu`, `femu-subsys`	Namespace Management; on the subsystem, shared namespaces. A controller with it cannot join a subsystem without it.
`bbssd_ns_limit`	`femu`	bbssd namespaces allocated at once, 1 to 256 and at least `namespaces`. Each has a full FTL in host memory.

Property	On	Effect and interactions
`subsys`	`femu`	Joins a `femu-subsys`.
`nqn`	`femu-subsys`	SUBNQN suffix; reaches Identify only on a shared subsystem.
`meta`, `mc`, `extended`, `pi`, `nlbaf`, `lba_index`, `dpc`, `dps`	`femu`	Formats, metadata and PI, above.

A NoSSD namespace and a ZNS namespace of different sizes on one controller:

```
-device femu,devsz_mb=2048,femu_mode=2,namespaces=2,namespace_sizes=1G,,1G,namespace_modes=nossd,,znssd
```

4.16.11 Statistics and outputs

Output	Scope
Identify Controller TNVMCAP, UNVMCAP	the controller's namespace pool
Identify Namespace NSZE, NCAP, NUSE, NVMCAP	one namespace (NVMCAP with Namespace Management)
Namespace lists (CNS 02h, 10h), controller lists (CNS 12h, 13h)	12h and 13h answer only with Namespace Management; 02h and 10h on any controller
Changed Namespace List (04h)	namespaces attached, detached, deleted or reformatted, per controller; filled only with Namespace Management
SMART / Health (02h)	the controller, summed over its namespaces
Vendor log C0h	summed over the bbssd, CSD and KV namespaces (log pages and counters (§B.1))

4.16.12 Validation

qtest cases in `hw/femu/tests/qtest/femu-test.c`:

- layout and modes: `namespace-capacity`, `namespace-identity`, `namespace-mixed-identity` and its `-bbssd` and `-kv` variants, `namespace-sizes-rounding`, `namespace-large`, `namespace-empty-slice`, `namespace-kv-byte-capacity`, `kv-namespaces`, `namespace-sparse`;
- management: the `ns-mgmt-*` cases (lifecycle, capacity, `bbssd_ns_limit`, notices, validation, unsupported modes and FDP), `namespace-lifecycle`, `namespace-allocated`, `ns-retire-*`;
- shared namespaces: the `ns-shared-*` cases and `ns-mgmt-subsys`;
- metadata and PI: `metadata`, `metadata-bbssd`, `metadata-extended`, and the `pi-*` cases.

Every tagged documentation example with several namespaces or controllers is also started under qtest and moves one block on namespace 1.

4.16.13 Limits

- Namespace Management creates only NoSSD or bbssd namespaces of the controller's own mode, and only on controllers whose namespaces all run that mode. It does not create zoned, KV, CSD or Open-Channel namespaces.
- Each bbssd namespace has a full-size FTL; there is no scaled geometry for small namespaces, which is why `bbssd_ns_limit` exists.
- Allocated Namespace Attribute notices are not advertised.

- Shared namespaces need homogeneous NoSSD or bbssd controllers without Streams, dps or namespace_modes.
- All namespaces and metadata live in host memory and are lost when QEMU exits.

4.16.14 How to extend it

- A new mode token: add it to nvme_mode_from_token(), register its table in nvme_register_extensions(), add any per-namespace restriction to nvme_init_namespaces(), and keep hw/femu/docs/modes.py in step.
- A mode that keeps controller-wide state must refuse a second namespace of that mode, as CSD does, or move the state into the namespace.
- Namespace Management for another mode: nvme_ns_mgmt_supported(), nvme_ns_create() and the capacity checks in nvme_ns_mgmt() are the places that limit it to NoSSD and bbssd.
- Any new data path must add ns->backend_offset to its byte offsets.

4.16.15 Source map

File	What is there
hw/femu/femu.c	femu-subsys device, CNTLID registration, namespace sizing, packing and per-namespace mode setup (nvme_init_namespaces()), nvme_ns_create(), extent search, shared storage (nvme_subsys_take_storage()), capacity reporting
hw/femu/nvme.h	NvmeSubsystem, NvmeNamespace, nvme_ns() and attachment helpers
hw/femu/nvme-admin.c	Namespace Management and Attachment, Identify lists, Format NVM
hw/femu/nvme-io.c	per-namespace dispatch, backend offsets, metadata transfer
hw/femu/nvme-pi.c, hw/femu/nvme-pi.h	protection information generation and checks

4.16.16 Related pages

- Several namespaces and devices (§4.15)
- Namespace management, metadata and PI (§4.17)
- ZNS (§4.8) and FDP (§4.6)
- Architecture (§3.1)

4.17 Namespace management, metadata and protection information

This page covers two optional NVMe features of NoSSD and BlackBox (§4.4) controllers:

- Namespace management (§4.17.2): the guest creates, deletes, attaches and detaches namespaces while it runs.
- Metadata and protection information (§4.17.3): extra bytes stored with every logical block, and end-to-end protection information (PI) types 1 to 3 in them.

Both are off by default. Use them to test host software that manages namespaces, or that writes and checks per-block metadata and PI, without hardware that supports them.

4.17.1 Requirements

- Host and guest: see the mode table (§4.1.4) and requirements.md (§2.1.2).

- Namespace management: any guest kernel with the NVMe driver, and nvme-cli (`create-ns`, `attach-ns`).
- Metadata through the block layer: a guest kernel with `CONFIG_BLK_DEV_INTEGRITY=y`. Without it, use passthrough commands.

4.17.2 Namespace management

What it does

With `ns_mgmt=on`, the controller advertises Namespace Management and Namespace Attachment and reports 256 as its namespace count. The namespaces from `namespaces` and `namespace_sizes` are created at boot, and together they define the capacity pool: the pool is the sum of their sizes, and at boot all of it is in use (`unvmcap` is 0). To create a namespace, the guest first deletes one, then allocates from the space it freed. A namespace the guest creates is detached until the guest attaches it.

It takes effect only when the controller and every boot namespace run the same mode, NoSSD or BlackBox. With another mode it is accepted and stays off.

Launch

A BlackBox controller with two 2 GiB boot namespaces, a 4 GiB pool:

```
-device femu,devsz_mb=4096,femu_mode=1,ns_mgmt=on,namespaces=2
```

A NoSSD controller:

```
-device femu,devsz_mb=4096,femu_mode=2,ns_mgmt=on,namespaces=2
```

To share namespaces between controllers, set `ns_mgmt=on` on a `femu-subsys` and join it with `subsys=`. The subsystem comes first:

```
-device femu-subsys,id=shared,ns_mgmt=on -device femu,id=ctrl-a,subsys=shared,femu_mode=2,devsz_mb=64 -device femu,id=ctrl-b,subsys=shared,femu_mode=2
```

The first controller sets up the pool and the boot namespaces; later ones join with nothing attached. Shared namespaces in detail (§4.17.2) describes the shared model in full.

Configuration

Properties: namespace management, streams and power loss (§A.1.1), shared namespaces (§A.1.2).

- `ns_mgmt`: on a standalone controller, or on `femu-subsys` for shared namespaces. A controller with `ns_mgmt=on` cannot join a subsystem that does not have it.
- `bbssd_ns_limit` (BlackBox only, default 4): the most namespaces that may be allocated at once, detached ones included. Each BlackBox namespace has its own FTL with the full NAND geometry, so this bounds host memory.
- `namespaces` and `namespace_sizes`: the boot namespaces, whose sizes add up to the pool. Space that no boot namespace covers is not in the pool.

A shared subsystem takes NoSSD or BlackBox controllers that all have the same mode, `meta`, `mc`, `pi`, `dpc`, `nlbaf`, `vwc` and `oncs`, and no Streams, `dps` or `namespace_modes`.

Use it from the guest

Check that the controller offers it, and find its controller ID:

```
sudo nvme id-ctrl /dev/nvme0 | grep -E '^(oacs|nn|cntlid|tnvmcap|unvmcap) '
```

`oacs` has bit 3 (0x8) set, `nn` is 256, and `unvmcap` is the free capacity in bytes.

With the BlackBox example above, delete boot namespace 2 to free 2 GiB, create a 1 GiB namespace of 512-byte blocks (LBA format 0) in it, attach it to this controller, use it, then remove it:

```
sudo nvme delete-ns /dev/nvme0 --namespace-id=2
sudo nvme create-ns /dev/nvme0 --nsze=2097152 --ncap=2097152 --flbas=0
sudo nvme attach-ns /dev/nvme0 --namespace-id=2 --controllers=0
sudo nvme ns-rescan /dev/nvme0
sudo nvme list
sudo nvme detach-ns /dev/nvme0 --namespace-id=2 --controllers=0
sudo nvme delete-ns /dev/nvme0 --namespace-id=2
```

`create-ns` prints the new namespace ID; use it in the commands after it. `--controllers` takes the `cntlid` from `id-ctrl`. `nsze` and `ncap` are in logical blocks and must be equal: an `ncap` below `nsze` fails with Thin Provisioning Not Supported, and a zero or larger one with Invalid Field. A BlackBox namespace is rounded up to whole NAND pages.

Limits

Message or status	Cause and fix
`bbssd_ns_limit must be between 1 and 256 and cover the boot namespaces`	`namespaces` above `bbssd_ns_limit`, or the limit out of range.
`ns_mgmt=on does not support subsys; use a standalone controller`	`ns_mgmt=on` on a controller whose subsystem lacks it. Set it on the `femu-subsys` instead.
`shared namespaces require homogeneous NoSSD or bbssd without Streams or default protection`	A ZNS, KV, CSD or OCSSD controller, or Streams, `dps` or `namespace_modes`, in a shared subsystem.
`shared namespace mode and capabilities must match`	Controllers in one shared subsystem differ in mode or format properties.
`namespace management does not support FDP`	`ns_mgmt=on` and `fdp=on` on one subsystem.
Namespace Insufficient Capacity	The pool has no free space (delete a namespace first), or a BlackBox namespace would not leave GC its free lines.
Namespace Identifier Unavailable	All 256 IDs are used, or `bbssd_ns_limit` is reached.
Thin Provisioning Not Supported	`ncap` below `nsze`.
I/O Command Set Not Supported	A command set other than NVM; the guest can create only block namespaces.

Shared namespaces in detail

With `femu-subsys,ns_mgmt=on`, the subsystem owns one namespace table, backend capacity pool and full-geometry FTL per bbssd namespace. This prevents controllers allocating the same NSID independently or deleting only a private copy. Realize the subsystem before its controllers. The first controller establishes the pool and boot namespaces; later controllers join that pool with no initial attachments. Namespace Attachment selects their active namespaces.

The subsystem copies the first controller's properties into an unrealized configuration object as its storage context, independently of PCI transport lifetime. This context has no queues or workers. Its namespace table and backend survive removal of that controller, including removal of every controller. They are released at subsystem teardown after its last controller has left. Namespace identity and the creation sequence belong to this context. Controllers must agree on the storage mode and format capabilities. Runtime construction uses the original storage configuration, including capacity, geometry, media options and the bbssd namespace cap; those properties on later controllers do not create additional storage.

Each controller keeps its own queues, pollers, FTL thread, attachment bitmap and namespace-change log. Pollers reach the common table through their controller; active lookup additionally checks that controller's attachment bitmap. A subsystem mutex serializes data and FTL processing. Existing pause/resume interfaces pause all controllers in a managed subsystem, so Format, Sanitize and lifecycle changes cannot race another controller's workers. Delete retires namespace references in every controller's request containers before releasing storage, without waiting for guest consumption of a full CQ. Detach retires only the selected controllers. Reset preserves attachments. Transport removal stops its workers and drops its attachments without releasing the subsystem's namespaces.

CNS 10h/11h describe the common allocated set, and CNS 1Fh gives the command-set-independent structure (CNS 08h) of an allocated namespace, attached or not. CNS 02h describes the issuing controller's attachments. CNS 12h lists attached controllers and CNS 13h lists eligible controllers, in ascending order with inclusive CNTID filtering. Attach/detach records changes and sends enabled Attached Namespace Attribute Changed notices on each affected controller. Delete does the same except that its issuer receives no notice. Allocated Namespace Attribute notices are not advertised. Create produces a detached namespace and no attached-set notice.

The model is opt-in on the subsystem. Existing standalone management and unmanaged subsystems retain their behavior. Only homogeneous NoSSD or bbssd is supported; FDP and Streams remain excluded. Full-geometry bbssd allocation and its namespace-count cap are retained. Scaled FTL geometry, allocated-namespace notices, persistent storage and other namespace modes are not supported.

With the shared example under Launch (§4.17.2), only ctrl-a initially has the boot namespace attached. Controller lists report the assigned CNTLIDs; use Namespace Attachment to expose it through ctrl-b too. Create honors NMIC: private namespaces can move between controllers but cannot be attached to two simultaneously. Shared namespaces permit both attachments.

4.17.3 Metadata and protection information

What it does

`meta=<bytes>` adds that many metadata bytes to every logical block. The controller then offers each LBA format twice: formats `0` to `nlbaf - 1` without metadata, and formats `nlbaf` to `2 * nlbaf - 1` with it. The namespace boots on the metadata version of `lba_index`. With the default `nlbaf=5` and `lba_index=0`, that is format 5: 512-byte blocks with metadata.

The metadata travels either in a separate buffer (`mc` bit 1, 0x2) or interleaved with the data as extended LBAs (`mc` bit 0, 0x1). `extended=1` boots on the interleaved layout.

With `pi=on` and `meta` of 8 or more, the controller also offers protection information types 1, 2 and 3, in the first or the last 8 bytes of the metadata. The guest selects a type with Format NVM, or with `--dps` when it creates a namespace. The controller generates and checks PI as the command asks (PRACT and PRCHK bits).

Launch

8 bytes of separate metadata with PI available:

```
-device femu,devsz_mb=1024,femu_mode=1,meta=8,mc=2,pi=on
```

8 bytes of interleaved metadata, booting on the extended layout:

```
-device femu,devsz_mb=1024,femu_mode=1,meta=8,mc=1,extended=1
```

Configuration

Properties: LBA formats, metadata and protection (§A.1.1).

- `meta`: metadata bytes per block. NoSSD and BlackBox namespaces only.
- `mc`: which layouts the controller supports, separate (0x2), interleaved (0x1) or both (0x3). Required with `meta`.
- `extended`: boot on the interleaved layout. Needs bit 0 of `mc`.
- `pi`: offer PI types 1 to 3. Needs `meta` of 8 or more; with less it is accepted and offers nothing.
- `nlbaf`: at most 8 with `meta`, since each format is offered twice.
- `dpc` and `dps` describe PI without metadata support; leave them at 0 and use `pi`. `dpc` must not set bit 3: NVM 1.1 does not allow PI in the first bytes of metadata, so realize refuses it (OCSSD, at 1.4, keeps it).

Use it from the guest

Look at the formats and the protection capabilities:

```
sudo nvme id-ns /dev/nvme0n1 -H | grep -E 'LBA Format|dpc|dps|mc'
```

Reformat to 512-byte blocks with 8 bytes of separate metadata and PI type 1 in the last 8 bytes (`--pil=0`), then check that the namespace changed:

```
sudo nvme format /dev/nvme0n1 --lbaf=5 --pi=1 --pil=0 --ms=0 --force
sudo nvme id-ns /dev/nvme0n1 | grep -E '^(flbas|dps) '
```

Write and read one block and let the controller generate and check the PI (`--prinfo=0xf`: PRACT plus the guard, application tag and reference tag checks; the reference tag of LBA 0 is 0):

```
head -c 512 /dev/urandom > blk.bin
sudo nvme write /dev/nvme0n1 -s 0 -c 0 -z 512 -d blk.bin --prinfo=0xf --ref-tag=0
sudo nvme read /dev/nvme0n1 -s 0 -c 0 -z 512 -d out.bin --prinfo=0xf --ref-tag=0
cmp blk.bin out.bin
```

With `CONFIG_BLK_DEV_INTEGRITY=y`, Linux generates and verifies PI for ordinary block I/O on such a namespace by itself.

Limits

Message	Cause and fix
`meta/extended need a matching metadata capability (mc)`	`mc` must have bit 0 when `extended=1`, and bit 1 otherwise.
`meta: at most 8 block sizes (nlbaf), each is also offered with metadata`	Lower `nlbaf`.
`meta: namespace N runs a mode without metadata support (block or no-SSD only)`	Every namespace must be NoSSD or BlackBox.
`meta: not supported with placement (fdp)`	Metadata and FDP do not combine.
`meta: protection information (dpc, dps) is not supported`	`dpc` or `dps` with `meta`; use `pi`.
`dps needs 8 bytes of metadata and a matching dpc`	Use `pi=on` instead of `dps`.
`power_loss requires bbssd, buffer_size > 0, and no metadata, namespace management or subsystem`	`meta` or `pi` with `power_loss`.

At run time, a Format NVM that asks for a PI type when `pi` is off, a PI type with less than 8 bytes of metadata, or a layout `mc` does not allow fails with Invalid Format. On a type 1 namespace, a command whose reference

tag is not the low 32 bits of its LBA fails with Invalid Protection Information. A check against stored PI that fails returns the matching End-to-end Guard, Application Tag or Reference Tag Check Error.

4.17.4 Verify

1. Namespace management: `oacs` has bit 3 set, and a created and attached namespace appears in `nvme list` with the size you asked for.
2. Metadata: `nvme id-ns /dev/nvme0n1` shows a non-zero `ms` for the current format, and `dpc` is `0x17` with `pi=on`: types 1 to 3, with PI in the last 8 bytes of metadata only. Format and Create refuse PIL with a type. With Namespace Management on, Identify CNS 09h and CNS 0Ah (CSI 00h) describe each format index.
3. PI: the write and read above succeed, and a read with `--ref-tag=1` for LBA 0 fails with Invalid Protection Information.

4.17.5 Troubleshooting

- **`ns_mgmt=on` but `oacs` has no Namespace Management bit.** The controller or a boot namespace is not NoSSD or BlackBox, or the modes differ. The property is silently off in that case.
- **`create-ns` fails with Insufficient Capacity.** The pool is the boot namespaces, and they use all of it at boot. Delete a namespace first.
- **A new namespace does not appear.** It is detached after creation. Attach it with the controller ID from `id-ctrl`, then run `nvme ns-rescan`.
- **The namespace shows a size of 0 after formatting with metadata.** The guest kernel cannot use that format through the block layer, usually because it lacks `CONFIG_BLK_DEV_INTEGRITY` or the format interleaves the metadata. Use passthrough commands on `/dev/ng0n1`, or a separate-buffer format on a kernel with integrity support.

Related issues: #121.

4.17.6 Related pages

- Several namespaces (§4.15)
- Changelog (§D)
- Choosing a mode: which features combine (§4.1.3)

4.18 CXL SSD

`femu-cxl-ssd` is a CXL Type-3 memory device whose capacity is backed by emulated NAND flash. The guest sees ordinary CXL memory: it creates a region, then uses it as a DAX device or onlines it as system RAM on its own NUMA node. Behind the memory interface, FEMU keeps a DRAM page cache in front of the same BBSSD FTL and NAND timing model that `femu_mode=1` uses. A cache hit costs no media time; a miss costs a NAND read, and evicting a dirty page costs a NAND program.

This page shows how to run it. The design note[71] explains how it works inside, and the property reference (§A.1.3) lists every property. The device comes from Cylon (FAST '26); if you use it, cite Cylon as well as FEMU (citation (§4.18.11)).

Related pages:

[71] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

- CXL caching API (§4.20): let the guest pin, drop and uncache pages through BAR5.
- CXL NVMe link (§4.21): serve the same medium as an NVMe namespace too.

4.18.1 What it emulates

Part	What FEMU does
Interface	A CXL Type-3 volatile memory device, a subclass of QEMU's `cxl-type3`. The guest CXL driver enumerates it as a memdev (`mem0`).
Capacity	The memory backend named by `volatile-memdev`: 256 MiB to 120 GiB, in steps of 256 MiB. The data lives there.
Cache	`cache-pages` pages of 4 KiB (default 1024, so 4 MiB), `cache-ways` ways per set, policy `fifo`, `lifo`, `clock` or `s3-fifo`.
Media	The BBSSD FTL: page mapping, garbage collection, NAND read, program and erase times, channels and LUNs.
Direct mapping	Optional (`der`). Cached pages are mapped into the guest so hits run at DRAM speed with no exit to QEMU.
Persistence	None. It is volatile memory. Data is lost when QEMU exits.
Migration	Not supported. `migrate` fails with "State blocked by non-migratable device '...'".

Every guest access that is not direct-mapped is an MMIO exit to QEMU. The vCPU that made the access waits out the modelled media time before the load or store completes. The timing model (§6.2.10) lists what each kind of access costs.

4.18.2 Host requirements

- An x86-64 Linux host with KVM. QEMU must be built for `x86_64-softmmu`; `femu-cxl-ssd` is built by default (see build (§2.2)).
- Host RAM for the media size, the guest RAM, and the FTL mapping tables.
- Hugepages are optional for `der=off` and `der=memslot`. `der=cylon` needs a shared, preallocated hugetlbfs backend (see `der=cylon` (§4.18.7)).
- `der=cylon` also needs a modified host kernel. The other modes run on a stock kernel.

To reserve 2 MiB hugepages for a 256 MiB device, for example:

```
echo 128 | sudo tee /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
grep HugePages_Free /proc/meminfo
mount | grep hugetlbfs          # usually /dev/hugepages
```

Give the pages back with `echo 0 | sudo tee ...` after QEMU exits.

4.18.3 Guest requirements

The guest kernel needs the CXL stack with region support and the DAX drivers:

Option	Why
`CONFIG_CXL_BUS`, `CXL_PCI`, `CXL_ACPI`, `CXL_MEM`, `CXL_PORT`	Enumerate the host bridge, root port and memdev
`CONFIG_CXL_REGION`	Create a region over the device

Option	Why
CONFIG_CXL_REGION_INVALIDATION_TEST	Needed inside a VM. A guest cannot invalidate CPU caches by address range, and without this option Linux refuses to commit the region. The option skips that step
CONFIG_DEV_DAX, CONFIG_DEV_DAX_CXL	Expose the region as /dev/daxX.Y
CONFIG_DEV_DAX_KMEM	Online the region as system RAM. kmem may claim a new RAM region on its own; daxctl reconfigure-device switches it back
CONFIG_MEMORY_HOTPLUG, CONFIG_MEMORY_HOTREMOVE, CONFIG_ZONE_DEVICE	Memory hotplug support that kmem and DAX need
CONFIG_FS_DAX	Not needed for RAM regions; the validated guests had it on

The guest runs validated in FEMU's own testing used Linux 6.12 with these options. Check a guest kernel's configuration with:

```
grep -E 'CONFIG_(CXL_REGION|CXL_REGION_INVALIDATION_TEST|DEV_DAX|DEV_DAX_CXL|DEV_DAX_KMEM|FS_DAX)='
    \
     /boot/config-$(uname -r)
```

Guest tools: cxl (cxl-cli), daxctl and ndctl. They come from the ndctl project; ./make-guest-image.sh --cxl installs them (see guest image (§2.3)). numactl helps when you use the region as system RAM.

4.18.4 Launching

With run-cxlssd.sh

hw/femu/scripts/run-cxlssd.sh builds a command line with one pxb-cxl host bridge, one cxl-rp root port, the femu-cxl-ssd and a single-target CXL window. Settings come from environment variables; arguments after the script name go to QEMU unchanged. It does not add a guest disk or network, so pass them yourself. From build-femu/:

```
../femu-scripts/run-cxlssd.sh \
    -drive file=$HOME/images/u20s.qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::8080-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0
```

The script looks for QEMU at ./qemu-system-x86_64; set QEMU to use another binary. DRY_RUN=1 prints the command instead of running it. It never runs sudo and changes no host settings.

Its defaults follow the published Cylon experiments and differ from the device's own defaults:

Variable	Script default	Device default	Property
CXL_SIZE	256M	(required)	backend size, M or G suffix
CACHE_PAGES	size in MiB / 20 * 256 (a cache of 1/20 of the media)	1024	cache-pages
CACHE_WAYS	1 (direct mapped); full means CACHE_PAGES	16	cache-ways
CACHE_POLICY	fifo	fifo	cache-policy
CHANNELS, LUNS_PER_CHANNEL	8, 8	4, 4	channels, luns-per-channel
LSA_CONTROL	on	off	lsa-control

BLOCKS_PER_PLANE	867 for 48G, 1732 for 96G, else 0	0	blocks-per-plane

The other variables (DER, CYLON_KERNEL_ACK, PREFETCH_DEGREE, PREFETCH_STRIDE, PAGES_PER_BLOCK, READ_NS, PROGRAM_NS, ERASE_NS, CHANNEL_NS, GC_THRESHOLD, GC_THRESHOLD_HIGH, FTL, CYLON_FIRST_TOUCH_PROGRAM, CYLON_FREE_WRITEBACK, LOG_DIR, LOG_LIMIT, TRACEFS_DIR, CXL_BACKEND, ACCEL, CPU, CPUS, RAM) keep the device defaults; the comment at the top of the script lists them all. The 48G and 96G presets are the least NAND the device accepts for those sizes. Cylon's own presets have no spare blocks, and the device refuses them: see Full NAND[72].

A property the script has no variable for can be set with -global, which applies to every femu-cxl-ssd:

```
../femu-scripts/run-cxlssd.sh -global femu-cxl-ssd.concurrent-misses=on
```

The equivalent command line

This is equivalent to the script with its defaults. The script also passes the remaining properties at their default values:

```
./qemu-system-x86_64 -machine q35,cxl=on,smm=off -accel kvm \
    -cpu host -smp 4 -m 4G \
    -object memory-backend-ram,id=cxlmem,size=256M \
    -device pxb-cxl,id=cxl.0,bus=pcie.0,bus_nr=52 \
    -device cxl-rp,id=cxl-rp0,bus=cxl.0,chassis=0,slot=0 \
    -device femu-cxl-ssd,id=cxlssd,bus=cxl-rp0,volatile-memdev=cxlmem,cache-pages=3072,cache-ways=
       1,cache-policy=fifo,der=off,channels=8,luns-per-channel=8,lsa-control=on
       \
    -M cxl-fmw.0.targets.0=cxl.0,cxl-fmw.0.size=256M \
    -drive file=$HOME/images/u20s.qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::8080-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0 \
    -nographic
```

The pieces:

- -machine q35,cxl=on turns on CXL support in the machine.
- pxb-cxl is a CXL host bridge, cxl-rp a root port below it, and the femu-cxl-ssd sits below the root port.
- cxl-fmw.0 is the fixed memory window, the host physical address range where the guest maps the region. Its size must be at least the media size.
- volatile-memdev names the backend that holds the data.
- smm=off is required by der=cylon and harmless otherwise.

To give the guest its own kernel, add -kernel bzImage -append "root=/dev/vda1 console=ttyS0" with the root partition of your image.

4.18.5 Creating the region in the guest

Log in (ssh -i ~/images/femu-guest-key -p 8080 femu@localhost with the image from make-guest-image.sh) and work as root. Find the memdev and the root decoder:

[72] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#full-nand

```
cxl list -M          # the memdev, normally mem0, with its ram_size
cxl list -D -d root  # the root decoder, normally decoder0.0
```

Create a RAM region over the whole device:

```
cxl create-region -d decoder0.0 -t ram -w 1 -m mem0
daxctl list
```

`daxctl list` shows one device, for example `dax0.0`. If kmem already claimed it as system RAM (`daxctl list` shows `"mode":"system-ram"`), switch it back with `daxctl reconfigure-device --mode=devdax --force dax0.0`. If `cxl create-region` fails and `dmesg` reports that the CPU cache could not be synchronized, the kernel lacks `CONFIG_CXL_REGION_INVALIDATION_TEST`.

As a DAX device

In devdax mode, programs `mmap()` `/dev/dax0.0` and load and store directly. Mapping offsets and lengths must be multiples of the device's alignment, which `daxctl list` shows as `align` (normally 2 MiB). Every access reaches the device; nothing in the guest page cache sits in front of it. The caching API (§4.20) tools expect this mode.

As system RAM on a NUMA node

`daxctl` hands the region to the kmem driver, which onlines it as memory:

```
daxctl reconfigure-device --mode=system-ram --force dax0.0
numactl -H                    # a new node with memory and no CPUs
numactl --membind=1 ./my-workload
```

The region becomes its own NUMA node only when the guest has an ACPI SRAT, which QEMU generates when the guest RAM is described with `-numa`. Without it the guest has no NUMA information and the new memory gets no node of its own. Pass the guest RAM as a NUMA node; the memory backend size must equal `-m` (4G with the script's default `RAM`):

```
../femu-scripts/run-cxlssd.sh \
    -object memory-backend-ram,id=ram0,size=4G \
    -numa node,nodeid=0,cpus=0-3,memdev=ram0 \
    -drive file=$HOME/images/u20s.qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::8080-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0
```

With this the CXL window becomes node 1. FEMU's guest runs wrote and verified data on node 1 in all three `der` modes; long workloads on it have not been checked. With `der=off` each access to the node exits to QEMU, so expect a workload there to run orders of magnitude slower than on node 0; use `der=memslot` or `der=cylon` to serve cache hits at memory speed.

4.18.6 Cache

Property	Default	Meaning
`cache-pages`	1024	Pages of 4 KiB the cache holds. 0 disables the cache: every read is a NAND read and every write a NAND program. At most the media page count, and divisible by `cache-ways`
`cache-ways`	16	Entries per set. 1 is direct mapped; equal to `cache-pages` is fully associative

Property	Default	Meaning
`cache-policy`	`fifo`	Replacement within a set: `fifo` drops the oldest entry, `lifo` the newest, `clock` the first without a recent reference, `s3-fifo` uses small, main and ghost queues

What an access costs:

- A hit costs no media time.
- A read miss reads the page from NAND and inserts it. A page the FTL has never mapped costs no media time, though it still counts in `media-reads`; its contents come from the backend, which reads as zeros for a fresh `memory-backend-ram`.
- A write miss reads the page as a read miss does, then the write makes it dirty.
- Evicting a dirty page programs it to NAND, and the access that caused the eviction pays for it. A clean eviction is free.

`cache-ways`, `prefetch-degree` and `prefetch-stride` can be changed while the guest runs with `qom-set`. Changing `cache-ways` writes back dirty pages and rebuilds the cache. `flush-cache` writes back every dirty page and drops the cache, waiting for the modelled media time; it is how you start a measurement from a cold cache:

```
{"execute": "qom-set", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "flush-cache", "value": true}}
{"execute": "qom-set", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "cache-ways", "value": 4}}
```

The path ends with the `id=` of the device; `run-cxlssd.sh` uses `cxlssd`.

Fast load

`fast-load=true` speeds up warmup and data loading phases that are not measured. While it is on, an access does not wait for its modelled media time. Everything else still runs: the FTL request, cache inserts and evictions, prefetch, direct mapping and every counter. The NAND timelines still advance, so the skipped time builds up as a backlog on the LUNs.

`fast-load=false` waits only for the accesses in progress, and at most 100 ms for queued device DMA work. It does not wait for the NAND backlog. Device DMA work queued before the switch still books its NAND time before any later access. That wait would stop the QEMU main loop for as long, and a guest disk that the main loop serves would time out. The backlog stays on the LUNs, so the next accesses wait behind it in their own threads. It does not flush the cache. `fast-load-drain-ns` gives the backlog in ns from the switch. If queued device DMA work was not booked within the 100 ms, it reads at least 1 at first, and the device updates it once that work is booked. `fast-load-switch-ns` gives how long the last switch took in the device. After it returns, accesses pay the full media time again.

Before a measured phase, wait until the NAND is idle. Poll `nand-idle-ns` until it reads 0. It gives the ns until the NAND timelines and the queued work are idle, and it never blocks. A harness that cannot poll can sleep `fast-load-drain-ns` instead, but device DMA that arrives meanwhile can add work after it.

```
{"execute": "qom-set", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "fast-load", "value": true}}
{"execute": "qom-set", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "fast-load", "value": false}}
{"execute": "qom-get", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "fast-load-drain-ns"}}
{"execute": "qom-get", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "nand-idle-ns"}}
```

The contract is narrow:

- Only the wait at the end of an access is skipped. Flush, `cache-ways` changes, caching API commands and a linked NVMe controller keep their timing.

- A serial access sequence gives the same cache and media counters with `fast-load` on or off. `media-time-ns` can differ, because accesses that arrive sooner queue behind each other on a LUN.
- With concurrent accesses, the final state is not guaranteed to be the same. Shorter page holds change which evictions succeed, and a guest that runs faster can issue its accesses in a different order.
- Use it for warmup and loading only. Compare measured phases against a normal run if the starting state matters.

Prefetch

On a miss, after inserting the missed page, the device inserts up to `prefetch-degree` more pages starting `prefetch-stride` pages after it: the pages `[page + stride, page + stride + degree)`. Pages already cached or past the end of the media are skipped. Prefetched pages are inserted with no NAND read and no media time; a dirty page they evict is still programmed. The degree is capped at `cache-pages`. The default degree is 0, which turns prefetch off.

```
CACHE_PAGES=4096 CACHE_WAYS=16 CACHE_POLICY=s3-fifo PREFETCH_DEGREE=4 \
    ../femu-scripts/run-cxlssd.sh
```

NAND geometry and timing

Property	Default	Meaning
`channels`	4	NAND channels, 1 to 4096
`luns-per-channel`	4	LUNs per channel, 1 to 128, one plane each
`pages-per-block`	256	4 KiB pages per block
`blocks-per-plane`	0	0 sizes the NAND to 5/4 of the media plus 4 blocks per plane, which leaves room for garbage collection. An explicit value must leave the media plus 7% beyond the lines `gc-threshold-high` keeps free and two more
`read-ns`, `program-ns`, `erase-ns`	40000, 200000, 2000000	Page read, page program and block erase times, at most one second
`channel-ns`	0	Channel transfer time per page
`gc-threshold`, `gc-threshold-high`	75, 95	Percent of lines in use at which garbage collection starts and is forced
`ftl`	`on`	`off` charges no media time at all; the device behaves as plain memory with a cache in front

Sectors are 512 bytes, eight per 4 KiB page. Misses on different LUNs overlap in the NAND model. Two switches reproduce the media model of the published Cylon experiments and are off by default: `cylon-first-touch-program=on` charges a program instead of a free read the first time an unmapped page is read, and `cylon-free-writeback=on` makes dirty write-backs free.

```
CXL_SIZE=1G CHANNELS=8 LUNS_PER_CHANNEL=4 READ_NS=50000 PROGRAM_NS=500000 \
    ../femu-scripts/run-cxlssd.sh
```

4.18.7 Direct mapping: `der`

With `der=off` every load and store traps to QEMU, even a cache hit. That gives exact counts and exact timing, but a hit costs microseconds. The other two modes map cached pages straight into the guest, so hits run at DRAM speed without an exit.

Mode	How hits are served	Needs	Limits
`off` (default)	Every access traps to QEMU	Nothing	Slow hits; every hit is counted and timed
`memslot`	Cached pages become KVM memory slot aliases	KVM. Refused under TCG	At most 1024 mapped pages (4 MiB) across all devices
`cylon`	Cached pages are written into KVM's page tables by a Cylon host kernel	The fixed Cylon host kernel, a hugetlb backend, `cylon-kernel-ack=on`	Only on that kernel; falls back to MMIO elsewhere

As a reference point, one host (Xeon Gold 6548Y+, 256 MiB device, 1024-page cache) measured a median cached load of about 3.0 us with `off` and 105 ns with `memslot` and with `cylon`.

What changes in the direct modes:

- A direct hit never reaches QEMU, so it is not counted in `cache-hits`, does not update the `clock` or `s3-fifo` reference state, and has no media time.
- Direct mapping works only in the topology `run-cxlssd.sh` builds: one endpoint directly below the only root port of a host bridge, in a window with one target, not interleaved. Elsewhere the device still works, all accesses use MMIO, and `der-fallbacks` counts the refused mappings.
- Mappings are revoked before an eviction, a flush, a PCI configuration or decoder change, a CXL mailbox command (except a control Get LSA, below), a reset and unplug. The page maps again on its next access.
- `der-ratio` maps a fixed fraction of all pages, whether cached or not, to reproduce Cylon's direct ratio experiments. See the design note.

`der=memslot`

```
DER=memslot ../femu-scripts/run-cxlssd.sh
```

Each mapped page is a one-page alias in the guest address space. All `femu-cxl-ssd` devices share a budget of 1024 aliases (4 MiB), fewer when KVM runs short of free memory slots. A cached page that finds the budget full is served by MMIO at `der=off` cost and counts in `der-fallbacks`. When the hot set moves, a cached page that takes 256 trapped hits while the budget is full replaces this device's oldest alias that does not hold a pinned page, at most `der-replace-rate` times per second (default 64; 0 disables replacement). Every mapped page is treated as dirty, because writes through an alias are invisible to QEMU, so its eviction costs a program.

Realize refuses `der=memslot` under TCG ("der=memslot is not supported with TCG; use KVM, or der=off"): the alias changes would race with other vCPUs' TLBs. Use `memslot` for hot sets of up to 4 MiB on a stock kernel.

`der=cylon`

`cylon` writes direct entries into KVM's EPT page tables through two ioctls of the Cylon host kernel. It has no per-page alias cost, samples EPT dirty bits so clean pages are not programmed on eviction, and suits hot sets and caches larger than 4 MiB.

It needs all of these on the host; FEMU checks the ones it can:

- A Cylon kernel with the dual-slot fixes on MoatLab/Cylon `master`. The published CylonLinux 6.4.6 corrupts host memory with this interface. FEMU cannot tell the two apart, so realize fails unless you set `cylon-kernel-ack=on` to state that the fixed kernel is running.

- For workloads that use vector instructions (most programs, through glibc), a Cylon kernel from MoatLab/Cylon `master` at 8c13c5cf2 or later, which adds `KVM_CAP_CYLON_FAULT_EXIT`. An older kernel loops forever, or crashes the host, when such an instruction touches an uncached page; FEMU warns at slot install when the capability is missing.
- KVM on Intel with EPT, EPT A/D bits, the TDP MMU and MMIO caching enabled, 4 KiB host base pages and no dirty ring.
- A hugetlbfs memory backend with `share=on` and `prealloc=on`. The device locks it with `mlock()` and reads `/proc/self/pagemap`, so run QEMU as root.
- `-machine smm=off` and the same CPU model on every vCPU (`run-cxlssd.sh` does both).

```
sudo DER=cylon CYLON_KERNEL_ACK=on \
    CXL_BACKEND=memory-backend-file,mem-path=/dev/hugepages,share=on,prealloc=on \
    ../femu-scripts/run-cxlssd.sh
```

When a check fails the device prints one warning naming the first missing requirement, keeps working on MMIO as with `der=off`, and `der-active` reads false. On a stock kernel:

```
DER=cylon CYLON_KERNEL_ACK=on ../femu-scripts/run-cxlssd.sh
```

```
qemu-system-x86_64: -device femu-cxl-ssd,...: warning: FEMU CXL DER unavailable: Cylon GET_LINEAR_SPT/SET_SPTE_FLAG require a Cylon KVM host; using MMIO
```

While a `cylon` VM runs, do not migrate, offline or soft-offline the hugetlb pages, and do not run idle page tracking (`/sys/kernel/mm/page_idle`) or DAMON on the QEMU process. The design note's host and guest restrictions[73] explain why.

Concurrent misses

`concurrent-misses` decides whether misses to different pages wait for the media together or one at a time. `auto` (default) overlaps them only while a direct mode is active; `on` and `off` force it. With `der=off`, a guest's `lock`-prefixed read-modify-write reaches the device as a separate read and write and is never atomic. Overlapping misses make lost updates common (about half of `lock add` increments with four vCPUs in FEMU's tests); one miss at a time keeps them rare but not absent. Do not rely on atomic operations to `der=off` memory, and keep `concurrent-misses` at `auto` or `off` there.

4.18.8 Control channel through the label area

`lsa-control=on` lets the guest send experiment commands with the CXL Get LSA mailbox command, in the form Cylon's scripts use: the read size is the command and the read offset its argument.

```
cxl read-labels mem0 -s 1 -O 42 -o /tmp/status.bin   # command 1, argument 42
od -An -tu1 -N1 /tmp/status.bin                     # 0 ok, 1 error, 2 queued
```

Command	Argument and effect
1	Append a statistics snapshot tagged with the argument to `cxlssd-stats.log`, then reset the event counters as `stats-reset` does
2	Write dirty pages back and drop unpinned pages, as `flush-cache`
3	Set the ways: 0 to 4 mean 1, 2, 4, 8, 16 ways; 5 means fully associative
5	Set `prefetch-degree`
7	Set `prefetch-stride`
9, 11	Flush and clear the cache, as 2
13	Open the next per-access log, `cxlssd-io-N.log` (64 names, reused in turn)

[73] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#host-and-guest-restrictions

Command	Argument and effect
15	Close the per-access log
17	Dump the current direct mappings to `cxlssd-spt.log`
90, 80	Set a direct ratio / remove it. The ratio is one of 50, 75, 90, 95, 97, 98, 99, 995 (99.5%), 999 (99.9%) or 100 percent, and 0 means 100. Setting one needs `der=memslot` or `der=cylon`; `memslot` refuses a ratio needing more aliases than are free, which on a 256 MiB device leaves 99 and up
91, 81	Empty the host trace buffer and start tracing / stop tracing, in `tracefs-dir`

Byte zero of the data returned is the status: 0 success, 1 error, 2 queued. Commands 1, 5, 7, 13, 15, 17, 80, 81, 90 and 91 run at once unless the device is busy, and report 0 or 1. Flushes and way changes (2, 3, 9, 11), unknown commands and anything that finds the device busy are queued and run after the read returns, from QEMU's main loop; their byte zero is 2, and an error shows only in `control-status` over QMP, which reads 2 until the queue drains and then 0 or 1 for the last command. A read larger than the mailbox payload also returns 1. Commands 2, 3, 9 and 11 also clear the cache event counters.

`lsa-control` is off by default on the device and on in `run-cxlssd.sh`. With it on, the device serves a 128 MiB internal label area and refuses an `lsa` backend. Turn it off for guests you do not trust: commands 1, 13 and 17 let the guest create and write files on the host in `log-dir` (the working directory by default), and commands 91 and 81 let it write the host's `tracing_on` and `trace` files when `tracefs-dir` is set. Each file is capped at `log-limit` (64 MiB by default), statistics appends are rate limited, and `log-dropped` counts what was refused.

```
LSA_CONTROL=off LOG_DIR=/tmp/cxl-logs ../femu-scripts/run-cxlssd.sh
```

The same commands are available from the host whatever `lsa-control` says: set `control-argument`, then `control-command`, with `qom-set`. The command runs before `qom-set` returns, and an invalid argument is reported as a QMP error.

4.18.9 Counters

The device's counters are QOM properties. Start QEMU with a QMP socket by adding `-qmp unix:/tmp/qmp.sock,server=on,wait=off` after the script name, then read them with QEMU's `scripts/qmp/qom-get`, from the QEMU source tree:

```
export QMP_SOCKET=/tmp/qmp.sock
scripts/qmp/qom-get /machine/peripheral/cxlssd.cache-misses
scripts/qmp/qom-get /machine/peripheral/cxlssd.media-time-ns
```

or send raw QMP, for example with `socat - UNIX-CONNECT:/tmp/qmp.sock`:

```
{"execute": "qmp_capabilities"}
{"execute": "qom-get", "arguments": {"path": "/machine/peripheral/cxlssd", "property": "media-reads"}}
```

The ones you need most:

Counter	Meaning
`cache-hits`, `cache-misses`, `read-hits`, `read-misses`, `write-hits`, `write-misses`	Trapped lookups; direct hits are not counted
`cache-entries`, `cache-evictions`, `prefetch-inserts`	Cache occupancy and churn
`media-reads`, `media-writes`, `media-time-ns`	NAND page reads, page programs and total modelled media time

Counter	Meaning
`media-full`	NAND programs that found no free page after garbage collection. The program is not timed, and it does not stop an eviction or an insert. The over-provisioning rule keeps it at 0. A measurement is valid only while it is 0
`gc-stalls`, `gc-stall-ns`, `gc-stall-max-ns`	Media requests that waited for forced garbage collection, the total time from their start to the end of that collection on every LUN, and the longest single wait. A wait over one second warns once; more `blocks-per-plane` make such waits shorter
`der-active`, `der-mapped`, `der-fallbacks`	Whether direct mapping is on, how many pages are mapped now, and refused mappings

`qom-set ... stats-reset true` copies the counters to the `last-*` properties and clears the cache, prefetch and caching API event counters. The media counters, `media-full` and the `der-*` counters are never cleared; measure them as differences between two reads. The full list is in runtime properties (§A.2.2).

4.18.10 Limits

- No live migration and no snapshots of device state.
- Volatile only: no persistent memory or dynamic capacity (`memdev`, `persistent-memdev`, `volatile-dc-memdev` and `num-dc-regions` are refused). An `lsa` label backend works only with `lsa-control=off`.
- Direct mapping needs the single-endpoint topology above.
- With `der=off`, every access costs an exit to QEMU; large workloads take hours. Use a small device and cache when you only need correct behaviour.

4.18.11 Citation

`femu-cxl-ssd` comes from Cylon, described in Cylon: Fast and Accurate Full-System Emulation of CXL-SSDs[74] (FAST '26). If you use the CXL SSD mode, please also cite:

```
@inproceedings{Yoon+26-Cylon,
  author    = {Dongha Yoon and Hansen Idden and Jinshu Liu and Berkay Inceisci and
               Sam H. Noh and Huaicheng Li},
  title     = {{Cylon: Fast and Accurate Full-System Emulation of CXL-SSDs}},
  booktitle = {24th USENIX Conference on File and Storage Technologies (FAST 26)},
  pages     = {313--327},
  year      = {2026},
}
```

4.18.12 Related pages

- CXL SSD design note[75]
- CXL caching API (§4.20)
- CXL NVMe link (§4.21)
- Timing model (§6.2.10)
- Device properties (§A.1.3)

[74] https://www.usenix.org/conference/fast26/presentation/yoon

[75] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md

4.19 CXL SSD (`femu-cxl-ssd`)

The device comes from Cylon (FAST '26); the user guide has its citation (§4.18.11).

This chapter describes the CXL SSD as a component: where it sits, what state it keeps, how an access moves through it, which thread charges which cost, and which parameters and counters belong to it. It ties together three existing pages:

- The design note[76] is the detailed reference. It records every locking rule, every KVM corner case and every validation result. This chapter links to it for depth instead of repeating it.
- The CXL SSD guide (§4.18) shows how to run the device.
- The caching API guide (§4.20) and the CXL NVMe link guide (§4.21) cover the two optional features.

4.19.1 Purpose

`femu-cxl-ssd` is a CXL Type-3 memory device whose capacity is backed by emulated NAND flash. The guest sees ordinary volatile CXL memory: it creates a region and uses it as a DAX device or as system RAM. Behind the memory interface the device keeps a DRAM page cache in front of a private instance of the BBSSD FTL and NAND timing model. A cache hit costs no media time; a miss costs a NAND page read; evicting a dirty page costs a NAND page program.

The device is a subclass of QEMU's `cxl-type3`. It keeps the parent's decoder translation, capacity reporting, CDAT and media-disable behaviour, and adds the cache, the media timing, optional direct mapping of cached pages into the guest, a guest-controlled caching API and an optional NVMe front end over the same medium. It changes no other FEMU mode.

Three properties set it apart from the NVMe modes:

1. The data path is a load or store, not a command. Each access that is not direct-mapped is an MMIO exit on a vCPU thread, and that vCPU waits out the modelled media time before the load or store completes.
2. The payload is a QEMU host memory backend (`volatile-memdev`), not FEMU's own DRAM backend. The cache holds metadata only: page numbers, dirty state and replacement state.
3. The FTL never touches payload bytes. It receives page numbers, opcodes and timestamps, and returns a latency.

4.19.2 Place in the CXL topology

The guest reaches the device through a CXL fixed memory window (CFMW), a host physical address range that the machine routes to one or more CXL host bridges. Below a host bridge sit root ports, optionally a CXL switch, and the Type-3 endpoint. Each level has HDM decoders that the guest's CXL driver programs when it creates a region.

[76] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

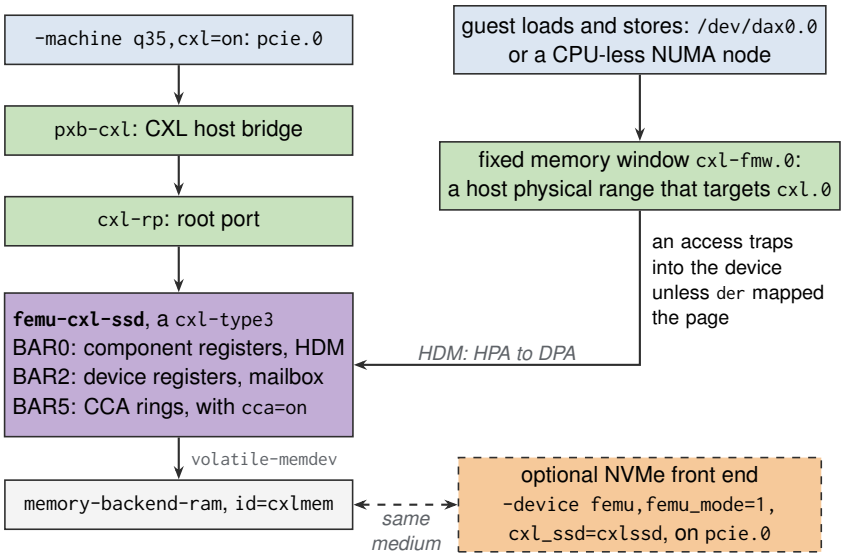


Figure 22: Where `femu-cxl-ssd` sits: a CXL type-3 device under `pxb-cxl` and `cxl-rp` on a q35 machine, backed by a RAM memory backend; guest loads and stores to the fixed memory window trap into the device unless `der` has mapped the page. An optional NVMe controller can serve the same medium.

```
guest physical address space
+-----------------------------------------------------------------+
| system RAM | ... | CXL fixed memory window cxl-fmw.0 (HPA)    |
+-----------------------------------------------------------------+
                              |
                FEMU overlay "femu-cxl-media" (priority 0, I/O)
                installed over every window that can reach a
                femu-cxl-ssd; routes like QEMU's own window router
                              |
        window interleave: pick the target host bridge
                              |
                +-------------------------------+
                | pxb-cxl host bridge           |  HB HDM decoders, or
                |  (cxl.0)                      |  passthrough to its one
                +-------------------------------+  root port when it has none
                              |
                +-------------------------------+
                | cxl-rp root port              |
                +-------------------------------+
                              |
                (optional CXL switch: one level of switch decoders)
                              |
  +-----------------------------------------------------------------+
  | femu-cxl-ssd  (subclass of cxl-type3)                           |
  |   endpoint HDM decoders: HPA -> DPA (interleave, DPA skip)      |
  |   BAR0/1, BAR2/3: parent registers    BAR4: MSI-X               |
  |   BAR5: caching API (cca=on only)                               |
  |   mailbox / CCI: Get LSA control channel (lsa-control=on only)  |
  |                                                                 |
  |   DRAM page cache -> femu-cxl-ftl worker -> BBSSD FTL + NAND model |
  |   payload: host memory backend named by volatile-memdev         |
  +-----------------------------------------------------------------+
```

`run-cxlssd.sh` builds the simplest form: one `pxb-cxl`, one `cxl-rp`, the device, and a single-target window.

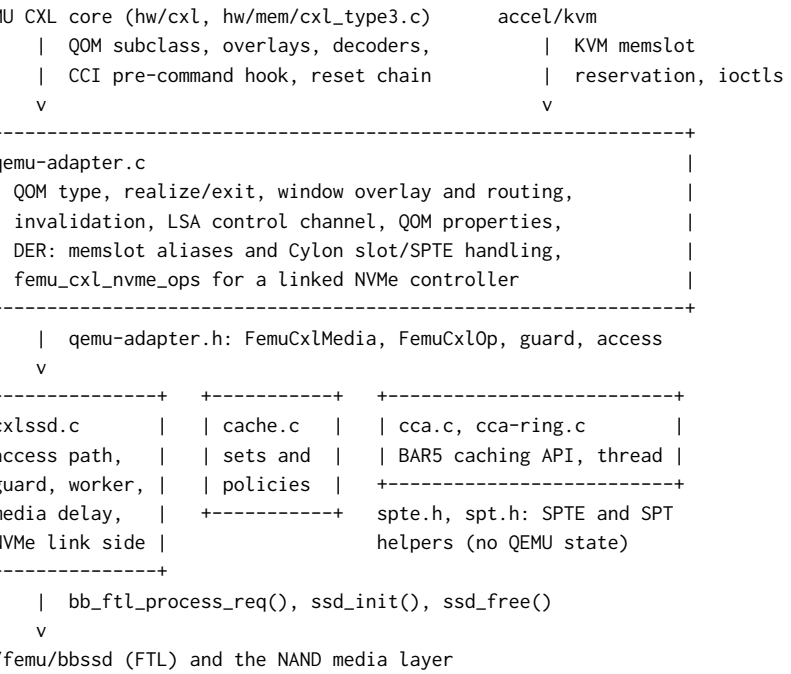
The device realizes in any topology a plain `cxl-type3` accepts: interleaved windows, host bridges with HDM decoders and several root ports, and below a CXL switch. Address routing follows the window interleave, the host bridge decoders (or its single root port when it has none), one level of switch decoders, then endpoint translation, the same order QEMU's own window router uses. A selected endpoint that is not a `femu-cxl-ssd` is forwarded to the window's original dispatch, so a plain `cxl-type3` in the same window keeps working. An address that no decoder claims reads as zero with a transaction error, and a write to it is dropped.

Only the direct mapping modes are tied to one topology: a single endpoint directly below the only root port of a host bridge without HDM decoders, in a single-target window, with no committed interleaved endpoint decoder (`der_windows_scan()`). Elsewhere the device works, accesses stay on MMIO and refused mappings count in `der-fallbacks`.

Where it lives: `adapter_route()`, `adapter_translate()`, `adapter_access()` and `adapter_machine_done()` in `hw/femu/cxlssd/qemu-adapter.c`.

4.19.3 The adapter boundary

All dependencies on QEMU's CXL device layout, fixed windows, decoders, CCI state and KVM ioctls are in one file, `hw/femu/cxlssd/qemu-adapter.c`. The rest of the component talks to it through `hw/femu/cxlssd/qemu-adapter.h`.



Two generic hooks live outside `hw/femu/`, and both are inert unless a `femu-cxl-ssd` uses them:

Hook	Where	What it does
KVM memslot reservation	`kvm_reserve_memslot()`, `kvm_reserved_memslot_id()` and `kvm_release_memslot()` in `accel/kvm/kvm-all.c`, declared in `include/system/kvm.h`	Reserves a slot ID that the KVM memory listener never allocates and that does not count as free capacity. `der=cylon` registers its dual-mode slot under that ID

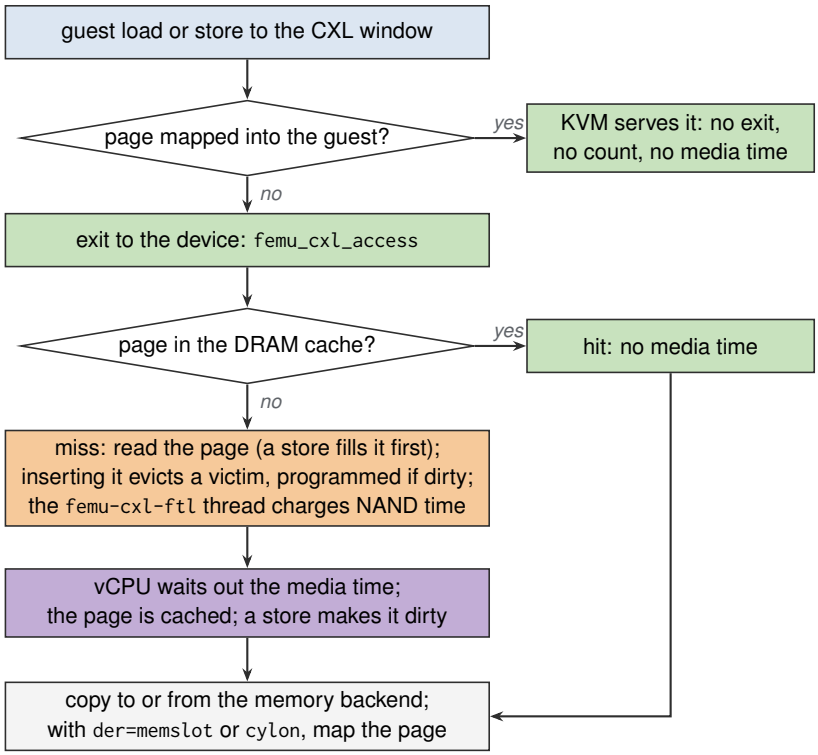
Hook	Where	What it does
CCI command callbacks	pre_command, post_command and pre_command_opaque in CXLCCI (include/hw/cxl/cxl_device.h), called from hw/cxl/cxl-mailbox-utils.c	Run before and after every CCI command on every transport, without replacing any command handler. FEMU uses them to revoke direct mappings and to hold its CXL lock across the command

Everything else is done by chaining parent methods: PCI configuration writes call the inherited method after FEMU revokes, component register writes go through a FEMU overlay on the component register block, reset chains the parent's hold phase, and the LSA class methods are always overridden but pass through to the parent's unless lsa-control=on.

The NVMe side never includes CXL headers. hw/femu/femu.c, and hw/femu/bbssd/bb.c for flips, call the medium through femu_cxl_nvme_ops, a table of five functions (prepare, attach, detach, ftl, flip) that qemu-adapter.c registers. The CXL files are built only when both CONFIG_FEMU_PCI and CONFIG_CXL_MEM_DEVICE are on (hw/femu/meson.build); without them the table stays empty and cxl_ssd= is refused.

4.19.4 Data structures

Structure	File	Fields that matter
FemuCxlSsd	qemu-adapter.c	parent_obj (the CXLType3Dev), media, component_overlay, lsa_queue (queued control commands)
FemuCxlWindow	qemu-adapter.c	One per fixed window that can reach a FEMU endpoint: the window and its io overlay region. Shared by all FEMU devices; removed when the last one leaves
FemuCxlMedia	qemu-adapter.h	backend (payload pointer and size), cache, direct (DER state), cca; the guard (busy, accesses, exclusive_waiters, idle condition variable); pages (pages held by accesses in progress); invalidations (generation); lock, worker_cond, work, worker (the FTL worker); ns.ssd (the private FTL); NVMe link fields nvme, nvme_ranges, nvme_taken, nvme_done, nvme_bh; counters
FemuCxlOp	qemu-adapter.h	One access, flush or eviction chain: the media time ns it has accumulated and the pages it holds itself
FemuCxlWork	qemu-adapter.h	One FTL request on the worker queue: an NvmeRequest, the returned latency, done, and done_cond, the waiter's condition variable
FemuCxlCache, FemuCxlSet, FemuCxlEntry	cache.h	nsets, ways, policy, entries and ghosts hash tables; per set the small, main, ghost and pinned queues; per entry lpn, dirty, freq, queue, der_hits, der_displaced
FemuCxlDer	der.h	maps (mapped pages), available, cylon, fast (Cylon state), ratio, installed (memslot aliases, oldest first), replacement state, DER counters
FemuCxlMap	qemu-adapter.c	One memslot alias: first page, page count, alias region, queue link
FemuCylon, CylonPage	qemu-adapter.c	Slot reservation, memory listener, SPT chunk areas, resolved huge page frames; per mapped page the SPTE pointer and the saved MMIO SPTE
FemuCxlCca	cca.h	BAR5 regions, host ring cursors, status, epoch, uncached_map (one bit per media page), thread, kick, stop, counters



`der=off` (default): nothing is mapped, so every access exits and is counted. `der=memslot`: a cached page becomes a one-page KVM memory slot, up to 1024 for all devices; without KVM realize fails, and a page that finds no free slot stays on MMIO. `der=cylon`: a Cylon host kernel writes the page into KVM's EPT; realize fails without `cylon-kernel-ack=on`, and when the host lacks Cylon support the device warns once and runs as `der=off`. Evicting a page unmaps it first.

Figure 23: One guest access to CXL SSD memory: a page mapped by `der=memslot` or `der=cylon` never exits; otherwise the device looks the page up in its DRAM cache, and a miss evicts a victim, reads the page through the FTL and makes the vCPU wait out the NAND time.

Keys in the cache and DER tables are full 64-bit page numbers (LPN = DPA / 4096). The cache library (`cache.c`) has no QEMU dependency and builds standalone for its unit test.

4.19.5 The media path

From a guest access to the cache

The overlay callback `adapter_access()` runs on the vCPU thread that took the MMIO exit, holding the BQL. A Cylon fault exit reaches it on the vCPU thread without the BQL (see Locking[77]). It routes the address, takes the CXL lock, translates, takes the device's operation guard, routes again (decoders may have changed while it waited), and calls `femu_cxl_access()` in `cxlssd.c`. If media is disabled (the parent's media-disable state), reads return random bytes and writes are discarded, as the parent Type-3 device does.

`femu_cxl_access()` handles an access of 1 to 8 bytes, which may cross one page boundary:

1. Hold each page it touches, in ascending order, in `s->pages`. A second access to a held page waits. This keeps accesses to one page ordered and makes a second miss wait for the first fill instead of repeating it.

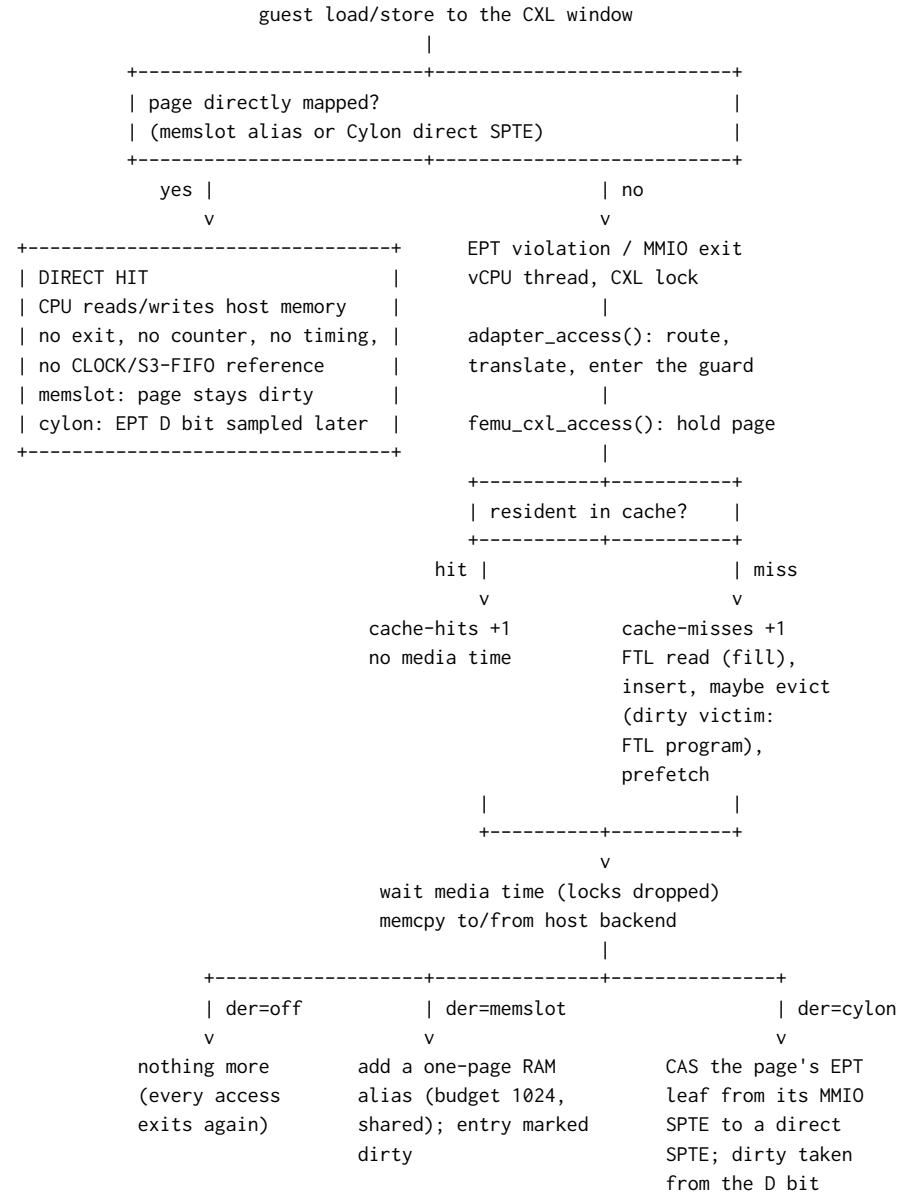
[77] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#locking`

2. Look up each page in the cache. On a miss, ask the FTL worker for a page read (or a program, when the page is not cacheable and the access is a write), then insert the page. Insertion may evict a victim; a dirty victim costs a program. A write marks the entry dirty.
3. On a demand miss, prefetch (see Prefetch (§4.19.6)).
4. Wait out the accumulated media time with the locks released. A fill without prefetch waits it out inside its media read instead, so it takes the locks once less.
5. Copy the bytes between the guest and the host memory backend.
6. For a single-page access to a cached (or ratio-selected) page, try to map the page directly into the guest (see Direct mapping (§4.19.8)).

A page is not cacheable when the cache is off (`cache-pages=0`), when the caching API marked it uncached, or when every way of its set is pinned. Such an access goes to the media every time: a read is a NAND read, a write a NAND program. An access also goes uncached when the only victim its insert could evict is held by another access in progress.

The write miss is write-allocate with fill: a store to a page that is not resident first reads the page from NAND, then dirties it. An unmapped page (one the FTL has never programmed) costs no media time on a read, though it still counts in `media-reads`; its bytes come from the backend, which reads as zeros for a fresh `memory-backend-ram`.

Access path under each `der` mode



With `der=off` the same page exits on every access, so every hit is counted and timed. In the direct modes the first access after a miss completes in QEMU and installs a mapping; the guest's later accesses to that page are direct hits until the page is evicted or an invalidation revokes it.

The operation guard

The guard (`femu_cxl_enter()`, `femu_cxl_enter_access()` and their `leave` pairs in `cxlssd.c`) is a reader/writer style lock built on the CXL lock and the `idle` condition variable:

Holder	Mode
Guest access, `concurrent-misses` in effect	Shared (`femu_cxl_enter_access()`)
Guest access otherwise	Exclusive (`femu_cxl_enter()`)
`flush-cache`, `stats-reset`, `fast-load`, `cache-ways`, prefetch and `der-ratio` changes, every control command	Exclusive

Holder	Mode
Each caching API chunk	Exclusive
The NVMe link bottom half that drops cache entries	Exclusive

An exclusive waiter blocks new shared holders, so a flush is not starved by a stream of accesses. Waiters release the locks while they wait. Invalidation and teardown never wait for the guard (see Invalidation (§4.19.9)).

`concurrent-misses` decides whether accesses share the guard. With `auto`, they share it only while a direct mode is available (`femu_cxl_concurrent()`). Sharing lets misses to different pages wait for the media together, so their NAND reads overlap where they reach different LUNs.

Atomicity caveat. With `der=off`, a guest's `lock`-prefixed read-modify-write reaches the device as a read and a separate write. It is never atomic. One access at a time keeps other vCPUs out from between the two most of the time; overlapping misses make lost updates common. FEMU's tests with four vCPUs lost 0.6% to 0.9% of `lock add` increments with serialized accesses and 51% to 52.5% with overlapping misses. In the direct modes the write lands on the page the read mapped, where KVM's emulator exchanges and retries, and no increments were lost. Do not rely on atomic operations to `der=off` memory, and keep `concurrent-misses` at `auto` or `off` there.

4.19.6 The DRAM page cache

The cache (`hw/femu/cxlssd/cache.c`) is set-associative over LPNs. It holds `cache-pages` entries in `nsets = cache-pages / cache-ways` sets; page `lpn` belongs to set `lpn % nsets`. One way is direct mapped; `cache-ways` equal to `cache-pages` is fully associative.

```
FemuCxlCache
+-------------------------------------------------------------------+
| entries: hash LPN -> FemuCxlEntry   ghosts: hash LPN -> node   |
| nsets = cache-pages / cache-ways    policy                     |
+-------------------------------------------------------------------+
      set index = lpn % nsets
      |
      v
sets[0] ... sets[i] ... sets[nsets-1]
            |
            v   FemuCxlSet (ways = cache-ways)
+-------------------------------------------------------------------+
| small : [head: next victim] e e e e [tail: newest]            |
| main  : [head] e e e [tail]              (S3-FIFO only)       |
| ghost : LPNs recently evicted from small, at most `ways`       |
| pinned: e e        (caching API; never evicted)                |
+-------------------------------------------------------------------+
   small + main + pinned <= ways

FemuCxlEntry: lpn | dirty | freq (0..3) | queue | der_hits | link
```

Replacement policies

`cache-policy` selects one policy for all sets. Every policy evicts only from `small` and `main`; pinned entries have left those queues.

Policy	Insert	Victim
`fifo`	Tail of `small`	Head of `small` (oldest)

Policy	Insert	Victim
`lifo`	Tail of `small`	Tail of `small` (newest)
`clock`	Tail of `small` with its reference set	Rotate from the head, clearing references, until an entry has none
`s3-fifo`	Tail of `small`, or of `main` when the LPN is in the set's ghost queue	Evict from `small` while it holds at least a tenth of the unpinned ways (minimum one) or `main` is empty. An entry leaving `small` with more than one hit moves to `main`; otherwise it is evicted and its LPN joins the ghost queue. In `main`, an entry with hits loses one and rotates

Hits increment `freq` up to 3. With one way, S3-FIFO uses only `small`. Removal of a known entry is constant time because each entry keeps its own queue node. The design note's media and cache[78] section has the remaining details.

Prefetch

Only a demand miss triggers prefetch, after the demanded page is inserted. The device inserts pages `[lpn + prefetch-stride, lpn + prefetch-stride + degree)`, where the degree is `prefetch-degree` capped at `cache-pages`. Resident, out-of-range and uncached pages, and pages whose set is fully pinned, are skipped. A prefetch performs no NAND read and adds no media time, but a dirty page it evicts is still programmed. If an insert cannot evict, prefetching stops for that access without failing it. Prefetched pages may be mapped directly. A small cache may evict the demanded page itself while prefetching, following insertion order.

Run-time changes

`cache-ways`, `prefetch-degree` and `prefetch-stride` can change while the guest runs (`qom-set`, or control commands 3, 5 and 7). A `cache-ways` change takes the guard exclusively, refuses before touching anything if the pinned pages do not fit the new geometry, then revokes mappings, writes dirty pages back, rebuilds the cache with the pinned pages still resident, and maps a direct ratio again.

4.19.7 Dirty data and write-back to the FTL

The cache is write-back. A dirty page is programmed to NAND when:

Event	Who pays
Eviction of a dirty victim by an insert	The access that caused the insert, or the `femu-cxl-cca` thread for a PIN fill
`flush-cache=true`, control commands 2, 9, 11	The caller (the main loop, waiting with the locks dropped)
`cache-ways` change	The caller
Caching API INVALIDATE and CACHE_DISABLE	The `femu-cxl-cca` thread, per chunk

`cylon-free-writeback=on` skips the program on eviction and flush, as the published Cylon experiments do.

What counts as dirty depends on the `der` mode, because a direct store never reaches QEMU:

- `der=off`: a store through MMIO sets `dirty`.
- `der=memslot`: QEMU cannot see stores through an alias, so every page that gets an alias is marked dirty when it is mapped. Its eviction always costs a program.

[78] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#media-and-cache`

- `der=cylon`: revocation samples the EPT dirty bit. A page read but never written through the direct entry is evicted clean, with no program.

The FTL worker (`cxl_worker()` in `cxlssd.c`, thread `femu-cxl-ftl`) is the only thread of the device itself that changes FTL and NAND state; with a linked NVMe controller, that controller's FTL thread is the other one, and both run under `s->lock`. Callers queue a `FemuCxlWork` on `s->work` under `s->lock`, wake the worker on `s->worker_cond` and wait for `done` on the request's own `done_cond`, which lives on the caller's stack. The worker signals only that condition, so each request is woken individually and no waiter wakes for another request. The worker calls `bb_ftl_process_req()` with an 8-sector request (one 4 KiB page of 512-byte sectors) and returns its latency. Requests are served in arrival order, and the NAND model overlaps them where they reach different LUNs. `cylon-first-touch-program=on` turns a read of an unmapped page into a program.

The FTL is a private BBSSD instance built in `femu_cxl_start()`: 512-byte sectors, eight per page, one plane per LUN, and the geometry, timing and GC thresholds of the device properties. It has no PCI queues and none of the optional NVMe features. With `ftl=off` there is no FTL and no worker: accesses have no media time and write-back is metadata only. GC, mapping and NAND timing are those of the BBSSD FTL, which the timing model (§6.2) and the architecture page's BBSSD section (§3.1.5) describe.

Realize refuses NAND that leaves less than 7% of the media as spare beyond the forced collection reserve and two working lines. Collection then always frees a line, and its victims hold invalid pages. A write that finds the free lines at the forced threshold waits until the collection erases end on every LUN; `gc-stalls` and `gc-stall-ns` count the waits. `media-full` counts programs that still find no page. Such a program is not timed, and it does not stop an eviction or an insert. A full NAND never sends a cacheable access uncached. See Full NAND[79].

4.19.8 Direct mapping

`der` selects how cached pages may be served without an exit. The design note's Direct Endpoint Remapping[80] section is the full reference.

Mode	Mechanism	Host needs	Mapping limit
`off`	None: every access is MMIO	Nothing	n/a
`memslot`	One-page `memory_region_init_alias()` RAM aliases over the window; KVM makes each one an ordinary memslot	KVM; refused under TCG at realize	1024 aliases shared by all devices, and never more than KVM's free slots minus 8
`cylon`	A dual-mode KVM slot over the window, and direct leaves written into KVM's EPT through the Cylon kernel's linear SPT	The fixed Cylon host kernel, Intel EPT with A/D bits, TDP MMU, MMIO caching, a shared preallocated hugetlbfs backend, root (for `mlock()` and `/proc/self/pagemap`), `cylon-kernel-ack=on`, `smm=off`, identical CPUID on all vCPUs	No per-page alias cost; the window is at most 120 GiB (60 SPT chunks of 4 MiB at 8 bytes per page)

Every mapping's HPA is checked against the endpoint decoders before it is installed (`femu_cxl_der_map()`), so a prefetched page whose derived HPA decodes to another DPA, or to none, is not mapped. Pages in

[79] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#full-nand`

[80] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#direct-endpoint-remapping`

caching API uncached ranges are never mapped. `der-ratio` maps a fixed fraction of all pages, cached or not, to reproduce Cylon's direct ratio experiments; see direct ratios[81].

`der=memslot`: the alias budget and replacement

Each alias, and the MMIO gap beside it, is a separate section of the system address space, and QEMU aborts once an address space needs 4096 sections. So all `femu-cxl-ssd` devices share one budget of 1024 one-page aliases (`FEMU_CXL_DER_ALIASES`), further capped at KVM's free slots minus 8 (`der_alias_budget()`). Ratio runs use the same budget. A cached page that finds the budget full stays on MMIO and counts in `der-fallbacks`.

When the hot set moves, the aliases may hold cold pages. Replacement (`der_replace_due()`, `der_replace_victim()`, `der_displace()`):

```
MMIO access to a cached page, budget full
      |
      v
der-replace-rate == 0 ? ---- yes ------------------------> stay on MMIO
      | no                                                  (no counting)
      v
entry->der_hits += 1  ---- < 256 (FEMU_CXL_DER_HOT) ---->  stay on MMIO
      | >= 256           (the miss that inserted the page counts too)
      v
first replacement, or time since the last one
>= (1 s / rate) << backoff ? ---------------- no -------> stay on MMIO
      | yes
      v
victim = oldest alias of this device whose page is not pinned
      | none ------------------------------------------> stay on MMIO
      v
one memory transaction: remove victim alias, add this page's alias
der-replacements +1, der-remaps +1, der-revocations +1
      |
      v
backoff: this page was displaced before -> backoff + 1 (max 8, so the
         interval grows up to 256x); 8 consecutive promotions of pages
         not displaced before -> backoff - 1
```

Accesses through an alias never reach QEMU, so installation order is the only recency the device can see. Pinned pages keep their aliases.

`der=cylon`: dual-mode slot and SPTE protocol

At realize, `femu_cylon_prepare()` probes the two Cylon ioctls (`KVM_GET_LINEAR_SPT`, VM ioctl 0xde; `KVM_SET_SPTE_FLAG`, vCPU ioctl 0xdd), checks the KVM module parameters, the hugetlbfs backend, `mlock()` and the pagemap frames. Any failure prints one warning, `FEMU CXL DER unavailable: <reason>; using MMIO`, and the device continues as with `der=off`. `cylon-kernel-ack=on` is required even then, because the device cannot tell the fixed kernel from the published one.

The first eligible decoded access schedules `cylon_install_bh()`, which pauses all vCPUs, reserves a slot ID through the generic reservation hook, and registers a slot over the window with flag `KVM_CYLON_DUAL_MODE` (bit 17). Its host address is the payload backend itself. The kernel then hands back the slot's linear SPT in at most 60 chunks of 4 MiB. Installation needs at least 8 free memslots.

Per page, the SPTE moves through these states:

[81] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#direct-ratios

```
         KVM's first fault on the page
(empty) ----------------------------> MMIO SPTE (kernel-created,
   |                                   generation bits saved by FEMU)
   | ratio only: CAS empty -> direct               |
   v                                               | access completes in QEMU:
DIRECT SPTE  <--------------------------------------+ CAS MMIO -> direct
RWX, WB, A=0, D=0                                    (der-remaps +1)
   |
   | revoke (eviction, invalidation, teardown)
   v
A bit clear? -- yes --> CAS direct -> saved MMIO, no TLB flush
   |                    (der-quiet-revocations +1, der-revocations +1,
   |                    page clean)
   | no (or the CAS lost to the CPU setting A)
   v
CAS clear W and MMU-writable; flush this GFN
read D; CAS -> saved MMIO; flush this GFN
D set -> cache entry dirty      (der-revocations +1)

KVM replaced the entry itself (zero, MMIO, frozen): treated as a KVM
revocation: entry made dirty, record dropped after a flush.
Unexpected entry or ioctl failure: restore saved MMIO entries, delete
the slot, disable Cylon until the next device reset.
```

An entry whose accessed bit is still clear has not been used by any page walk since it was installed, so no TLB holds it: that is the quiet path. The full path costs two single-GFN flushes per page. `der-revocations` counts every revocation, quiet or not. A whole-slot invalidation still samples each page (one flush per accessed page) but skips the final flush, because the slot deletion that follows flushes everything; later accesses reinstall the slot and pages map again lazily. Global dirty logging (migration) revokes everything and disables Cylon. A failed slot deletion aborts QEMU, because continuing could leave the kernel using a slot whose backing FEMU has released.

The Cylon host kernel. The published CylonLinux 6.4.6 corrupts host memory with this interface. The fixes are on MoatLab/Cylon `master`; the design note's host kernel[82] section lists them and the requirements the kernel still does not check (SMM off, identical CPUID, no move or flag change of the dual slot). Its host and guest restrictions[83] section explains why hugetlb migration, memory offline, `page_idle` and DAMON must not touch the QEMU process during a run.

Known kernel-side limitation: first touch per 2 MiB under a ratio. A direct ratio installs direct leaves into empty SPT entries, including pages in 2 MiB guest regions the guest has never touched. For such a region the level above the leaf is still empty. The first guest access in the region faults, KVM links the page table, and in doing so replaces FEMU's direct leaf with an MMIO SPTE. FEMU sees a KVM revocation (`der-revocations` +1), serves that access by MMIO and maps the page again (`der-remaps` +1); the rest of the region stays direct. Expect one exit per untouched 2 MiB region under a `der=cylon` ratio. This is kernel behaviour, not a device bug; the fix belongs in the kernel (keep a present direct leaf when a fault only links its parent table). Ordinary cache mappings are not affected, because they are installed only over MMIO SPTEs KVM has already created.

[82] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#host-kernel`

[83] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#host-and-guest-restrictions`

4.19.9 Invalidation

Direct mappings are revoked whenever the guest could change what an address means, and whenever the device needs to observe a page again. `cxl_invalidate()` revokes all mappings and bumps the `invalidations` generation; an access in flight that sees the generation move during its media delay does not install the mapping it translated before.

Trigger	Where	Effect
PCI configuration write	`adapter_config_write()`	Revoke all, the parent's method, revoke all again
Component register write (HDM decoders)	`adapter_component_write()` through `component_overlay`	Revoke all, then the parent's registers, under the CXL lock
Any CCI command on any transport	`adapter_pre_command()`, `adapter_post_command()`	Revoke all, then the command, under the CXL lock; except Get LSA on the primary mailbox with `lsa-control=on`, which carries a control command and revokes nothing
Device reset	`adapter_reset_hold()`	Revoke all, let a failed Cylon retry, caching API reset (unpin all, end uncached ranges), free CCI background state, then the parent's hold phase
Unplug	`cxl_exit()`	Tell the caching API thread to stop, mark the device closing, disable DER, and free the media now or leave that to the last guard holder
Eviction	`femu_cxl_evict()`	Revoke that page only (a ratio-selected page keeps its mapping; its dirty bit is sampled)
Flush, way change, commands 2, 3, 9, 11	`cxl_flush()`, `cxl_runtime_set()`	Revoke all, write dirty pages back, drop unpinned entries, map a ratio again
Caching API INVALIDATE, CACHE_DISABLE	`cca.c`	Revoke the chunk's pages, or all mappings once for more than 64 candidates
Linked NVMe write, zeroes, copy, deallocate	`femu_cxl_nvme_bh()`	Revoke and drop the pages without write-back; pinned pages stay, made clean; ratio-selected pages keep their mapping; a range over 64 pages revokes all mappings at once when no ratio is set
Global dirty logging (Cylon)	`cylon_log_start()`	Revoke all, disable Cylon
Memory map change overlapping the window (Cylon)	`cylon_region_change()`	Delete the dual slot; re-admit after fresh checks

Invalidation never waits for the guard. Configuration writes, component writes and CCI commands run inside another owner's re-entrancy guard, and waiting there would refuse other vCPUs' accesses. Revocation therefore touches only DER state, which is safe under the CXL lock alone. Warm reset keeps the volatile payload and the cache and FTL contents; only mappings, pins and uncached ranges go. Pages map again on their next access.

4.19.10 The LSA control channel

With `lsa-control=on` the device serves an internal, zero-filled 128 MiB label storage area (`FEMU_CXL_LSA_SIZE`) instead of an `lsa` backend, and a guest `Get LSA` mailbox command (opcode 0x4102) carries an experiment command: the read length is the command and the read offset its argument. This is the convention of Cylon's scripts: `cxl read-labels mem0 -s COMMAND -O ARGUMENT`. Byte 0 of the returned data is the status: 0 success, 1 error, 2 queued.

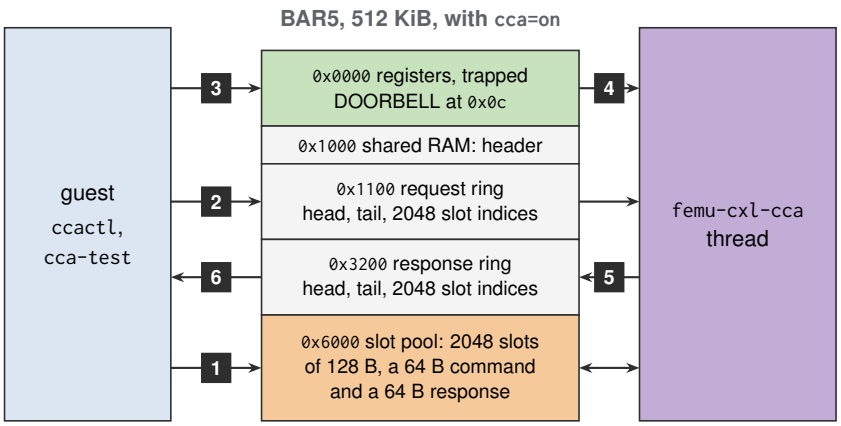
```
guest: cxl read-labels mem0 -s CMD -O ARG
      |
      v  mailbox Get LSA (0x4102), vCPU thread, device re-entrancy guard held
cxl_get_lsa()
      |
      +-- CMD in {1,5,7,13,15,17,80,81,90,91}, guard free, queue empty
      |        -> run inline (cxl_command()), byte 0 = 0 or 1
      |
      +-- otherwise (2, 3, 9, 11, unknown, or device busy)
               -> append to lsa_queue, byte 0 = 2
                  main-loop BH cxl_lsa_bh() runs the queue in order;
                  control-status reads 2 until it drains
```

The command table is in the guide (§4.18.8) and the design note[84]. The same commands are available from the host through `control-argument` and `control-command`, whatever `lsa-control` says.

Trust. The channel lets the guest act on the host. Commands 1, 3, 5, 7, 13 and 17 create or write files (`cxlssd-stats.log`, `cxlssd-io-N.log`, `cxlssd-spt.log`) in `log-dir`, QEMU's working directory by default. When `tracefs-dir` is set, command 91 truncates the host's tracefs `trace` file and writes `tracing_on`, and 81 writes `tracing_on`. The device bounds this: each file stops at `log-limit`, statistics appends pass a token bucket (a burst of 64, then 100 per second), I/O log names cycle through 64 files, and `log-dropped` counts what was refused. It does not authenticate the guest. Keep `lsa-control` off for guests you do not trust; it is off by default on the device and on in `run-cxlssd.sh`. The security page (§5.13.1) says more.

4.19.11 The caching API (CCA)

`cca=on` gives the guest control of the cache through PCI BAR5: pin and unpin pages, write back and drop ranges, mark ranges uncached, and query residency. The device side is `hw/femu/cxlssd/cca.c` and `cca-ring.c`; the ABI is one header, `hw/femu/cxlssd/cca-abi.h`, compiled by both the device and the guest library in `hw/femu/tools/cca/`.



1 The guest writes a command (`cmd`, `flags`, `lpn_start`, `lpn_count`, `tag`) into a free slot. **2** It pushes the slot index on the request ring. **3** It writes DOORBELL. **4** The thread wakes, takes the BQL, pops up to 64 indices, reads each command and works it in chunks of at most 256 pages, waiting out the media time with the BQL dropped. **5** It writes the response into the slot and pushes the index on the response ring. **6** The guest pops the response. Commands: NOP, CACHE_ENABLE, CACHE_DISABLE, PIN, UNPIN, INVALIDATE, QUERY. Ranges are in 4 KiB device pages.

Figure 24: The CXL caching API in BAR5: the guest puts a command in a slot, queues the slot on the request ring and rings the doorbell; the `femu-cxl-cca` thread runs it and queues the response.

BAR5 layout

```
BAR5, 512 KiB, 32-bit memory BAR
offset
0x00000 +-------------------------------------------------+
        | registers, 4 KiB, trapped (4- and 8-byte access) |
        |  0x00 MAGIC 0x43434131 "CCA1"   0x04 VERSION 2   |
        |  0x08 STATUS  READY|FATAL|CACHE|BUSY             |
        |  0x0c DOORBELL (any write)  0x10 RESET (1, 2)    |
        |  0x14 FATAL_REASON  0x18 MEDIA_PAGES (64-bit)    |
        |  0x20 CACHE_PAGES  0x24 CACHE_WAYS               |
        |  0x28 PIN_LIMIT  0x30 COMPLETED (64)  0x38 EPOCH |
0x01000 +-------------------------------------------------+
        | RAM "femu-cxl-cca-shm" (offsets below are BAR5)  |
        |  0x01000 header (magic, version, ring offsets;   |
        |          data ring fields stay 0)                |
        |  0x01100 request ring:  head | tail | 2048 u32   |
        |  0x03200 response ring: head | tail | 2048 u32   |
        |  0x06000 slot pool: 2048 slots x 128 B           |
        |          slot = 64 B command + 64 B response     |
0x46000 |  (unused to the end of the BAR)                  |
0x80000 +-------------------------------------------------+

command  (64 B): cmd | flags | lpn_start | lpn_count | tag | rsvd[4] = 0
response (64 B): status (0 or -errno) | lpn_start | lpn_count (pages acted
                 on) | tag | resident | dirty | pinned | uncached
```

The rings are single producer, single consumer, with free-running 32-bit head and tail; their size is a constant, never read from shared memory. The device keeps its own cursors, reads each shared index once, and copies a command out of its slot before validating it. A head too far ahead, a slot index out of range, or a response ring the guest never drains sets STATUS.FATAL with a reason and stops the device side until RESET.

[84] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#experiment-controls`

Commands

Command	Effect
NOP	Status 0
PIN	Room is checked for the whole range first (`-ENOSPC`). Resident pages are pinned; others are filled by a NAND read, then pinned. A later fill failure (`-EIO`) or a way change between chunks (`-EAGAIN`) leaves the pages already pinned. `-EBUSY` on an uncached page, `-EOPNOTSUPP` without a cache
UNPIN	Returns pinned pages to `main` under S3-FIFO with more than one way, else to the tail of `small`; dirty state is kept
INVALIDATE	Revoke mappings, write dirty pages back, drop them; pinned pages need `CCA_F_FORCE`
CACHE_DISABLE	Mark pages uncached, then drop resident ones as INVALIDATE does
CACHE_ENABLE	End the uncached marking
QUERY	Count resident, dirty, pinned and uncached pages

Ranges are 4 KiB device pages; `CCA_F_ALL` selects the whole medium. Every way of a set may be pinned; a miss to such a set is served uncached and counted in `cca-pinned-set-misses`. `der-ratio` and uncached ranges exclude each other.

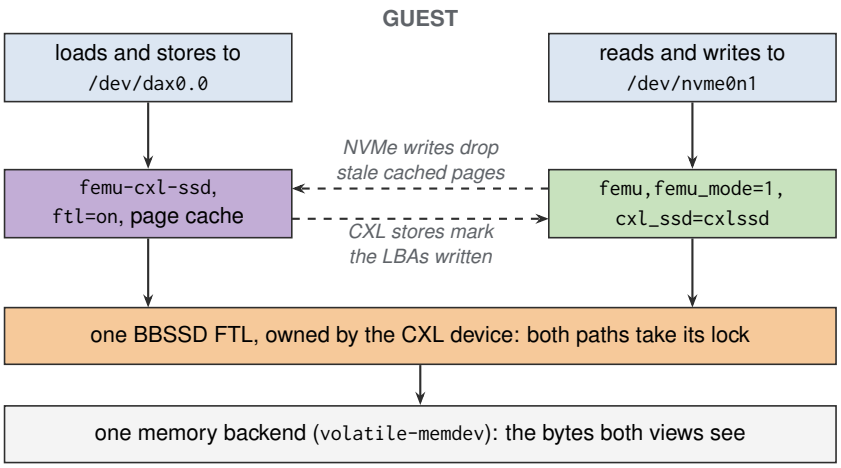
Thread and chunking

A doorbell write signals the detached `femu-cxl-cca` thread and never waits. The thread takes the BQL and the CXL lock, runs up to 64 commands in submission order, then gives them up and wakes itself again, so a guest that keeps the ring full cannot hold them. Each command runs in chunks of at most 256 pages of work and 4096 lookups. A chunk holds the guard exclusively, then waits out its media time with the locks released; before each chunk the thread lets one waiting access take the guard first. Guest accesses therefore run between chunks, and the worst stall a chunk adds to an access is about 256 programs. Device reset reformats the rings, bumps EPOCH and drops a command in flight without a completion. The caching API section[85] of the design note covers the reset and unplug races.

4.19.12 The NVMe front end

`-device femu,femu_mode=1,cxl_ssd=<id>` puts a BBSSD NVMe controller in front of the same medium. Its one namespace uses the device's memory backend as payload and the device's FTL as its own, so both front ends see one set of bytes and one mapping table.

[85] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#caching-api-cca`



The `femu-cxl-ssd` comes first on the command line; the `femu` controller sits on `pcie.0` or a PCIe root port, not under `pxb-cxl`, has one namespace and uses the CXL device's geometry and timing. Avoid mounting a file system on the namespace while the CXL region is in use.

Figure 25: One medium, two views: a linked BBSSD controller serves the CXL SSD's medium as an NVMe namespace, sharing its FTL and memory backend, and each path keeps the other's view consistent.

```
     guest CXL driver (devdax / system RAM)     guest NVMe driver
                 |                                      |
      loads/stores (MMIO or direct)          SQ/CQ, poller thread
                 |                                      |
+----------------v------------------+     +------------v-------------+
| femu-cxl-ssd                      |     | femu (bbssd, cxl_ssd=)   |
| cache metadata, DER mappings      |     | DMA copies payload at    |
+----------------+------------------+     | submission               |
                 |                         +------------+-------------+
                 |   payload: one host memory backend   |
                 +----------------+---------------------+
                                  |
  femu-cxl-ftl worker             |        NVMe FTL thread
  (CXL fills, write-backs)        |        (femu_cxl_nvme_ftl())
                 \                |               /
                  +--- s->lock (worker mutex) ---+
                                  |
                    one BBSSD FTL + NAND model
                                  |
NVMe Write / Write Zeroes / Copy dst / Deallocate:
  FTL thread appends page ranges to nvme_ranges, tags request cxl_seq
                                  |
                                  v
  main-loop BH femu_cxl_nvme_bh(): takes the guard (or sets nvme_kick
  and lets the holder reschedule it), revokes mappings, drops the
  pages WITHOUT write-back (pinned pages stay, made clean),
  publishes nvme_done
                                  |
                                  v
  poller posts the completion only when nvme_done >= cxl_seq
```

Consistency rules (the full list is in the design note's coherence[86] section):

[86] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#coherence

1. Payload needs no code: CXL accesses, direct mappings and NVMe DMA all touch the same host memory. An NVMe read returns the latest CXL store whether the page is clean, dirty in the cache or direct-mapped.
2. Write, Write Zeroes, Copy (destination) and Deallocate make each page they cover non-resident before the command completes. The entry is dropped without write-back, because the command already programmed or unmapped the page.
3. NVMe reads leave the cache alone and charge a NAND read even for resident pages.
4. A CXL store, and every direct mapping, marks the namespace's written-block bitmap, so DULBE and Get LBA Status see CXL writes.
5. NVMe Flush does not flush the cache; the medium is volatile memory.
6. Flips 1 to 4 change the shared timings and so affect CXL misses too; flip 3 restores the device's own `read-ns`, `program-ns`, `erase-ns` and `channel-ns`.

Lock order is BQL, CXL lock (the guard is a condition under it), worker mutex. The NVMe FTL thread never takes the BQL or the CXL lock. The controller must be BBSSD with one namespace and none of namespace management, a subsystem, streams, `power_loss`, `buffer_size`, `op_pcent`, metadata or protection information (`femu_cxl_link_check()` in `femu.c`). Format NVM is not advertised and Sanitize is refused, since both would rewrite the medium behind the cache. While linked, `device_del` of the medium fails with an unplug blocker; the NVMe link guide (§4.21) covers the slot power-off case and the realize messages.

4.19.13 Threads, locks and where latency is charged

Thread	Runs	Takes
vCPU, MMIO exit	MMIO overlay callback, the whole access path, Get LSA	BQL, CXL lock, guard, page holds; drops both locks to wait for the worker and for the media delay
vCPU, Cylon fault exit	The fault service and its fill	CXL lock, guard, page holds; drops it to wait; takes the BQL first only when a step needs it (`der-fault-bql`)
`femu-cxl-ftl`	`cxl_worker()`: every FTL request of the medium, device DMA work queued before the last `fast-load=false` first	`s->lock`, then `post_lock` briefly; never the BQL or the CXL lock, never guest memory
`femu-cxl-cca`	Caching API commands	BQL, CXL lock, guard per chunk, `cca->lock` for the doorbell flag
QEMU main loop	QMP `qom-set` (flush, way change, control commands), queued LSA commands, NVMe drop BH, Cylon install BH, teardown and reference BHs left by fault exits	BQL, CXL lock; the guard, except the Cylon install BH, which pauses all vCPUs before it takes the CXL lock
QEMU main loop, device DMA	Copies of other devices into the window, guarded or not (block layer completions), except qtest commands; identified by the thread that realized the device, so an IOThread's or another non-vCPU thread's copy, which also holds the BQL, keeps the full model	BQL, CXL lock, `post_lock`; never the guard or `s->lock`, never waits (`femu_cxl_access_nowait()`)
NVMe FTL thread (linked controller)	NVMe I/O on the shared FTL (`femu_cxl_nvme_ftl()`), after the device DMA work before the last `fast-load=false` is booked	`s->lock`, waits on `posted_cond` under `post_lock` with `s->lock` dropped; never the BQL or the CXL lock
NVMe pollers (linked controller)	Hold a completion until its cache drop is published	Read `nvme_done` only

Latency model (femu_cxl_media(), femu_cxl_delay() in cxlssd.c):

- Each FTL request is stamped now + op.ns, so the media operations of one access (a fill, then the write-back of the victim its insert evicts) are serialized in modelled time, and the returned latencies accumulate in op.ns and in media-time-ns.
- After its media work, the access waits op.ns minus the time already spent, with the locks released. The wait sleeps until 100 us before the deadline and spins on the realtime clock for the last 100 us (FEMU_CXL_SPIN_NS), which absorbs timer slack without holding a host CPU for long waits such as a flush.
- The stall is charged to the vCPU inside the MMIO exit: the guest's load or store does not complete until the media time has passed.
- Direct hits never enter QEMU and have no modelled time. flush-cache and way changes wait on the main loop. Caching API chunks wait on the CCA thread.

The timing model (§6.2.10) lists what each kind of access costs, and the architecture page's CXL load walkthrough (§3.1.11) follows one load end to end.

4.19.14 Parameters

The property reference (§A.1.3) lists every property with its type, default and range; it is generated from the binary. This table explains how they interact.

Group	Properties	Interactions
Backend (§A.1.3)	volatile-memdev	Required and the only backend accepted: memdev, persistent-memdev, volatile-dc-memdev and num-dc-regions are refused, and lsa only with lsa-control=off. Size a nonzero multiple of 256 MiB, at most 120 GiB. der=cylon needs it hugetlbfs, share=on, prealloc=on
Cache (§A.1.3)	cache-pages, cache-policy	cache-pages=0 disables the cache. Otherwise at most the media page count and divisible by cache-ways
Cache tunables (§A.2.2)	cache-ways, prefetch-degree, prefetch-stride	Accepted on -device and changeable with qom-set. Ways must be nonzero and divide cache-pages. Prefetch values are bounded by the media page count; the effective degree is capped at cache-pages
NAND geometry and timing (§A.1.3)	ftl, channels, luns-per-channel, pages-per-block, blocks-per-plane, gc-threshold, gc-threshold-high, read-ns, program-ns, erase-ns, channel-ns	Feed the private BBSSD FTL. blocks-per-plane=0 sizes the NAND to 5/4 of the media plus 4 blocks per plane; an explicit value must leave the media plus 7% beyond the forced collection reserve and two working lines. ftl=off drops all media timing and forbids an NVMe link. When linked, these also apply to the NVMe namespace and the controller's own geometry and timing properties are ignored
Cylon media switches (same table)	cylon-first-touch-program, cylon-free-writeback	Change the media model to match published Cylon experiments; off for normal use
Direct mapping (§A.1.3)	der, der-replace-rate, cylon-kernel-ack, concurrent-misses	der=memslot is refused under TCG. der=cylon without cylon-kernel-ack=on is refused. der-replace-rate matters only for memslot. concurrent-misses=auto follows whether DER is available
Caching API, control channel and logs (§A.1.3)	cca, lsa-control, log-dir, tracefs-dir, log-limit	cca=off registers no BAR5. lsa-control=on refuses an lsa backend. log-limit=0 opens no I/O log and takes no statistics appends. tracefs-dir unset makes commands 91 and 81 no-ops on the host

Group	Properties	Interactions
Actions and control (§A.2.2)	der-ratio, control-command, control-argument, control-status, flush-cache, stats-reset, fast-load, fast-load-drain-ns, fast-load-switch-ns, nand-idle-ns	Run time only, except fast-load, which -device also accepts. der-ratio needs a direct mode and no uncached ranges; memslot refuses a ratio that needs more aliases than are free. fast-load=false holds the guard alone while it sets the post_barrier and waits at most 100 ms for the worker to book the device DMA work before it and then tries lock to read the NAND horizon, with the BQL and the CXL lock dropped; it never waits for lock or for the horizon. nand-idle-ns only tries lock

run-cxlssd.sh uses defaults that follow Cylon's launch script and differ from the device's (a cache of 1/20 of the media, direct mapped, 8 by 8 channels and LUNs, lsa-control=on, and the least NAND the device accepts for the 48G and 96G presets); the guide (§4.18.4) has the table.

4.19.15 Counters

All counters are read-only QOM properties on the device (/machine/peripheral/<id>). The runtime property reference (§A.2.2) describes each one; this table says where each family comes from.

Family	Counted where	Notes
Cache (§A.2.2): cache-hits, cache-misses, read-*, write-*, cache-inserts, cache-evictions, cache-entries, prefetch-inserts	femu_cxl_access() and the cache library	Trapped lookups only; direct hits are invisible
Snapshots (§A.2.2): last-*	stats-reset, control command 1	Copies taken before the event counters are cleared
Media (§A.2.2): media-reads, media-writes, media-time-ns, media-full, gc-stalls, gc-stall-ns	femu_cxl_media()	Never cleared by stats-reset; take differences. media-writes comes from the FTL and includes linked NVMe programs
Direct mapping (§A.2.2): der-active, der-probes, der-mapped, der-remaps, der-revocations, der-quiet-revocations, der-replacements, der-fallbacks, der-emul-exit, der-emul-fills, der-emul-failures	DER code in qemu-adapter.c	der-mapped is a gauge and the evidence that mapping is active
Caching API (§A.2.2): cca-*	cca.c	cca-pinned and cca-uncached are gauges
Other (§A.2.2): invalidations, nvme-drops, log-dropped	Invalidation sites, NVMe drop BH, log writers	invalidations is a generation, not an error count

Host-side files, all bounded by log-limit and written in log-dir: cxlssd-stats.log (command 1 snapshots, ways and prefetch changes), cxlssd-io-N.log (one line per trapped access: realtime start, R/W, DPA, length, modelled ns) and cxlssd-spt.log (command 17 dump of tracked direct mappings and Cylon SPTE values).

4.19.16 Validation

Layer	What runs	What it covers
Unit tests (`hw/femu/tests/unit/`)	`test-cxl-cache.c`, `test-cxl-spte.c`, `test-cxl-cca-ring.c`, built by `hw/femu/tests/Makefile`	Policy ordering, S3-FIFO promotion and ghost admission, full-width keys, dirty accounting at 1 to 32 ways and a large fully associative cache; SPTE encodings, MMIO generation range and huge-page address arithmetic; the guest ring library against the device ring consumer, with every guest-writable index fuzzed
qtests (`hw/femu/tests/qtest/femu-test.c`)	About a hundred `cxl-*` cases under `qos-test`	Real PCI bridges and HDM registers programmed, then accesses through the window: all policies, readback, dirty programming, media timing, invalid configuration, `ftl=off`, mixed-window forwarding, interleaved, switched and multi-root-port topologies, memslot mappings, budget and replacement, ratios, invalidation, unplug during a wait, concurrency stress, LSA and QOM control, every caching API command, and the NVMe link (`cxl-nvme-*`)
Doc examples	`hw/femu/scripts/check-doc-examples.py`	Each tagged command line in the guides starts under qtest; `der=cylon` is checked to fall back with its warning
Guest runs on real hosts	`hw/femu/tools/cca/run-guest-tests.sh` in the guest, plus the runs recorded in the design note	A bare-metal Intel host on the fixed Cylon kernel, and a nested run under KASAN, passed write/readback across eviction cycles in all three `der` modes; caching API self-test; fio on a linked NVMe namespace next to devdax traffic with checksums on both paths

Under qtest Cylon is deliberately inert (no KVM), so qtests cannot validate the custom kernel: dual-slot registration, SPT mapping, SPTE revocation and the quiet path are covered only by guest runs on a Cylon host. Long workloads, sustained traffic past the GC threshold in a guest, hugetlb migration during a run, and guests that online the range as system RAM for long periods are not covered. The design note's scope and validation limits[87] has the exact list.

4.19.17 Limits

- No migration or snapshot: the device's vmstate is `unmigratable`, because the FTL and cache have no migration state. `migrate` fails with "State blocked by non-migratable device".
- Volatile only: no persistent memory, no dynamic capacity. Data is lost when QEMU exits.
- TCG: `der=memslot` is refused at realize (other vCPUs could keep TLB entries for a revoked alias). `der=cylon` needs KVM and falls back to MMIO under TCG and qtest. `der=off` works everywhere.
- Direct mapping only in the single-endpoint topology described above.
- `der=off` costs an exit per access: a hit takes microseconds, so large workloads are slow.
- With `der=off`, `lock`-prefixed operations to the window are not atomic.
- Direct hits are untimed and uncounted, and do not update CLOCK or S3-FIFO reference state.
- `der=memslot` holds at most 1024 aliases at once across all devices: one page each for cache mappings, while a ratio run counts as one alias.
- `der=cylon` needs the fixed host kernel and has the host restrictions and the 2 MiB first-touch effect described above.
- The caching API data rings are not implemented; their header fields are 0.

4.19.18 Extending the component

[87] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#scope-and-validation-limits`

To add	Change	Check
A cache policy	`FemuCxlPolicy` in `cache.h`, the name table in `femu_cxl_policy()`, insert and victim rules in `femu_cxl_cache_insert()` and `cache_evict()` and the requeue rule in `femu_cxl_cache_unpin()` in `cache.c`, the name table in `cxl_policy_name()` and the realize error message in `qemu-adapter.c`, the `cache-policy` description in `props.c`	`test-cxl-cache.c` builds `cache.c` without QEMU; add ordering cases there first
A counter	A QOM property in `cxl_init()` in `qemu-adapter.c` and its description in `cxl_runtime_descs` in `props.c`	`gen-property-docs.py --check` fails until the description exists and `reference/runtime-properties.md` is regenerated
A control command	`cxl_command()` in `qemu-adapter.c`; add it to `cxl_lsa_inline()` only if it never drops the locks (no media wait, flush or way change)	qtests `cxl-control-qom` and `cxl-control-lsa` run the same table both ways
A caching API command	The command enum in `cca-abi.h` (raise `CCA_LAYOUT_VERSION` if the layout changes), `cca_validate()`, `cca_prepare()` and the per-page handler table in `cca_exec()` in `cca.c`, the guest library in `hw/femu/tools/cca/`	`test-cxl-cca-ring.c` and the `cxl-cca-*` qtests
A direct mapping mode	The `der` check in `cxl_realize()`, `femu_cxl_der_init()`, and the map, remove, sample and clear entry points in `qemu-adapter.c`	Every invalidation trigger in the table above must reach it
Another media model	`cxl_worker()` calls `bb_ftl_process_req()`; replace the call and the FTL construction in `femu_cxl_start()`	Keep the worker free of guest memory: payload copies stay on the vCPU thread, and any future worker-side copy must use `femu_dma_rw()`

Keep new QEMU or KVM dependencies in `qemu-adapter.c`, so `cache.c`, `cca-ring.c`, `spte.h` and `spt.h` stay testable without QEMU and `cxlssd.c` stays free of CXL and KVM details.

4.19.19 Source map

File	Contents
`hw/femu/cxlssd/qemu-adapter.c`	QOM type `femu-cxl-ssd`, realize and exit, window overlay and routing, invalidation hooks, reset chain, LSA control channel and logs, QOM properties, DER (memslot aliases, budget, replacement, ratios; Cylon prepare, install, SPTE map and revoke, listener), NVMe link ops
`hw/femu/cxlssd/qemu-adapter.h`	`FemuCxlMedia`, `FemuCxlOp`, `FemuCxlWork`, guard and access API
`hw/femu/cxlssd/cxlssd.c`	Guard, media delay, FTL worker, `femu_cxl_access()`, eviction callback, geometry checks, FTL start and stop, NVMe link side (`femu_cxl_nvme_ftl()`, drop BH, flips)
`hw/femu/cxlssd/cache.c`, `cache.h`	Set-associative cache and the four policies, pinning
`hw/femu/cxlssd/der.h`	`FemuCxlDer` and the ratio selection helpers
`hw/femu/cxlssd/spte.h`, `spt.h`	Cylon EPT SPTE encodings and SPT chunk limits
`hw/femu/cxlssd/cca.c`, `cca.h`	Caching API device side: BAR5, thread, commands
`hw/femu/cxlssd/cca-ring.c`, `cca-ring.h`, `cca-abi.h`	Ring consumer and the shared ABI
`hw/femu/cxlssd/props.c`	Property help text used by the generated reference
`hw/femu/tools/cca/`	Guest library, `ccactl`, `cca-test`, `run-guest-tests.sh`

File	Contents
hw/femu/scripts/run-cxlssd.sh	One-device launcher
hw/femu/femu.c	cxl_ssd link property, femu_cxl_link_check(), calls through femu_cxl_nvme_ops
accel/kvm/kvm-all.c, include/system/kvm.h	Generic memslot reservation hook
hw/cxl/cxl-mailbox-utils.c, include/hw/cxl/cxl_device.h	Generic CCI pre-command hook

4.19.20 Related pages

- CXL SSD design note[88]
- CXL SSD guide (§4.18)
- CXL caching API (§4.20)
- CXL NVMe link (§4.21)
- Timing model (§6.2.10)
- Architecture (§3.1)
- Security and limits (§5.13)

4.20 CXL caching API (CCA)

The caching API lets software in the guest control the DRAM cache of a femu-cxl-ssd (§4.18): pin pages so they stay cached, write back and drop ranges, mark ranges uncached so every access goes to the media, and ask what is resident. Ranges are in 4 KiB device pages. It follows the caching API of the Cylon CXL SSD emulator.

The guest talks to the device through PCI BAR5 of the CXL function. FEMU ships a small C library, the ccactl command-line tool and a self-test in hw/femu/tools/cca/. The design note's caching API section[89] describes the register layout and the rings in full.

4.20.1 Turning it on

cca=on on the device (default off). With it off there is no BAR5 and the device behaves exactly as without the feature. run-cxlssd.sh has no variable for it, so pass a -global:

```
../femu-scripts/run-cxlssd.sh -global femu-cxl-ssd.cca=on \
    -drive file=$HOME/images/u20s.qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::8080-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0
```

On a full command line, add cca=on to the device:

```
./qemu-system-x86_64 -machine q35,cxl=on,smm=off -accel kvm -cpu host -smp 4 -m 4G \
    -object memory-backend-ram,id=cxlmem,size=256M \
    -device pxb-cxl,id=cxl.0,bus=pcie.0,bus_nr=52 \
    -device cxl-rp,id=rp0,bus=cxl.0,chassis=0,slot=0 \
```

[88] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md

[89] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#caching-api-cca

```
    -device
        femu-cxl-ssd,id=cxlssd,bus=rp0,volatile-memdev=cxlmem,cache-pages=1024,cache-ways=16,cca=on
        \
    -M cxl-fmw.0.targets.0=cxl.0,cxl-fmw.0.size=256M
```

The device starts a `femu-cxl-cca` thread that runs the commands. PIN and UNPIN need a cache (`cache-pages` above 0).

4.20.2 Guest setup

The tools map `/sys/bus/pci/devices/<function>/resource5`. That needs root and a guest kernel without PCI lockdown; no kernel module or UIO driver is needed. Check that firmware assigned BAR5 under the CXL host bridge:

```
fn=$(basename "$(readlink -f /sys/bus/cxl/devices/mem0/..)")   # PCI function of mem0
lspci -vv -s "$fn" | grep Region            # a Region 5 line must be present
ls -l /sys/bus/pci/devices/$fn/resource5
```

Build the tools inside the guest from a copy of the FEMU source tree:

```
sudo apt install build-essential
make -C hw/femu/tools/cca
```

This builds `libcca.a`, `ccactl` and `cca-test`. Commands work without a region. The address helpers of the library and most self-test cases need a devdax region on the memdev; create one as in creating the region (§4.18.5) and keep it in devdax mode.

4.20.3 `ccactl`

```
ccactl [-d DEV] [-f] [-t MS] COMMAND [LPN COUNT | all]
```

Option	Meaning
`-d DEV`	A memdev name (`mem0`), a PCI address or a `resource5` path. Without it, the only CCA device is used
`-f`	FORCE: lets `invalidate` and `disable` act on pinned pages, unpinning them
`-t MS`	Command timeout in milliseconds, default 10000; negative waits forever
`LPN COUNT`	First device page (DPA / 4096) and number of pages, decimal, `0x` hexadecimal or, with a leading 0, octal; `all` is the whole media

Command	What it does
`info`	Print the layout version, media pages, cache pages, cache ways, pins allowed per set and commands completed
`nop`	Round trip with no effect
`pin LPN COUNT`	Keep the pages cached until unpinned
`unpin LPN COUNT`	Let the pages be evicted again
`invalidate LPN COUNT`	Write dirty pages back to NAND and drop them from the cache
`disable LPN COUNT`	Mark the pages uncached and drop them
`enable LPN COUNT`	End the uncached marking
`query LPN COUNT`	Count resident, dirty, pinned and uncached pages
`reset`	Reset the rings only
`reset all`	Reset the rings, unpin every page and end every uncached range

```
sudo ./ccactl -d mem0 info
sudo ./ccactl -d mem0 pin 0 16            # pin device pages 0..15
sudo ./ccactl -d mem0 query all
sudo ./ccactl -d mem0 -f invalidate all   # unpin, write back and drop all
sudo ./ccactl -d mem0 disable 100 8       # pages 100..107 uncached
sudo ./ccactl -d mem0 enable all
```

Range commands print `status N (text)` and `pages N`, the number of pages acted on; `query` adds `resident`, `dirty`, `pinned` and `uncached`. The exit status is 0 when the device returns status 0 (always for `info`), 1 on an error status or when the device cannot be opened, and 2 on a usage error. `ccactl` opens the device before it checks the command, so with no device a usage error also exits 1.

4.20.4 What each command does

PIN : Refused as a whole with `-ENOSPC` when the pins cannot fit. Resident pages are pinned. Other pages are read from NAND as a miss would read them (counted in `media-reads` and `cca-pin-fills`, not as guest misses), possibly evicting an unpinned page, then pinned. Pinned pages leave the eviction queues, so eviction and prefetch never touch them. Every way of a set may be pinned; a miss to a set whose ways are all pinned is served from NAND without caching and counted in `cca-pinned-set-misses`. On `-EIO` or `-EAGAIN` partway through, the pages pinned before the failure stay pinned; `pages` says how many.

UNPIN : Returns pinned pages to the replacement queue a newly inserted page would join. Dirty pages stay dirty.

INVALIDATE : Writes dirty pages back to NAND and drops them from the cache. Pinned pages need FORCE; without it nothing changes and the status is `-EBUSY`. Direct mappings of the pages are revoked first.

CACHE_DISABLE (`disable`) : Marks the pages uncached, then drops resident ones as INVALIDATE does. An uncached page is never inserted, prefetched or direct-mapped: every read is a NAND read and every write a NAND program. A failed command clears the mark from the pages it could not drop, so an uncached page is never resident. A lock-prefixed read-modify-write on an uncached page depends on the mode:

- With `der=off`, or with `der=cylon` while no slot is installed, it reaches the device as a read and a separate write. So it is not atomic between vCPUs (see thread ownership[90]). A slot is never installed when every page is uncached from the start.
- With an installed `der=cylon` slot, a Cylon kernel with commit 768e5d4d0 or later does the write as an atomic exchange on the payload. FEMU then charges the read, but no program for the write.

CACHE_ENABLE (`enable`) : Ends the uncached marking; the next access caches the page again.

QUERY : Counts resident, dirty, pinned and uncached pages. Dirty comes from the cache metadata; with `der=cylon` a page written through a direct mapping may not show as dirty until its EPT dirty bit is sampled.

Commands run one at a time in submission order, in chunks of at most 256 pages, and each chunk waits out its media time. Guest accesses run between chunks. A page is changed atomically within its chunk, but a range is not: if a range must stay non-resident, stop accessing it while the command runs.

Pins and uncached ranges survive a cache flush (`flush-cache`, control commands 2, 9 and 11) and a `cache-ways` change; a flush writes pinned dirty pages back and keeps them resident. A way change that cannot fit the pinned pages is refused. A device reset, including a guest reboot, unpins everything and ends every uncached range. A direct ratio (`der-ratio`) and uncached ranges exclude each other.

[90] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#thread-ownership`

4.20.5 Errors

Status values from the device:

Status	Meaning
`-EINVAL`	Unknown command or flag, nonzero reserved field, or an empty range
`-ERANGE`	The range goes past the end of the media
`-ENOSPC`	PIN: a set has no way left to pin
`-EBUSY`	INVALIDATE or CACHE_DISABLE over pinned pages without FORCE; PIN over an uncached page; CACHE_DISABLE while a direct ratio is set
`-EOPNOTSUPP`	PIN or UNPIN without a cache
`-ENODEV`	The CXL media is disabled; every command but NOP fails. Nothing a guest does through the CXL mailbox in this QEMU disables it
`-EIO`	NAND refused a read or write. INVALIDATE leaves that page and the rest of the range resident
`-EAGAIN`	A `cache-ways` change during a PIN left no room for the rest

The library adds its own:

Status	Meaning
`-ETIMEDOUT`	No answer within the timeout
`-ECANCELED`	A device reset cancelled the command while it waited
`-EPROTO`	The rings hit a fatal error; every call fails until `cca_reset()`, or the next `cca_open()`
`-EBUSY` (from `cca_open()`)	Another process holds the device

4.20.6 Library

`hw/femu/tools/cca/cca.h` documents every call. Link with `libcca.a` and `-pthread`. A short example:

```
#include "cca.h"

struct cca_dev *d;
struct cca_result r;

if (cca_open("mem0", &d) == 0) {
    cca_pin(d, 0, 16, &r);          /* returns r.status */
    cca_query(d, 0, 16, &r);        /* r.resident, r.pinned, ... */
    cca_unpin(d, 0, 16, &r);
    cca_close(d);
}
```

- Synchronous calls (`cca_pin`, `cca_unpin`, `cca_invalidate`, `cca_cache_disable`, `cca_cache_enable`, `cca_query`) return the device status and fill `struct cca_result`; `pages` is the number of pages acted on. `cca_nop` returns the status only. Pass `CCA_FLAG_FORCE` to `cca_invalidate` and `cca_cache_disable`, and `CCA_WHOLE` as the count with a start of 0 for the whole media.
- `cca_submit()` and `cca_reap()` keep up to 2048 commands in flight; `CCA_SUBMIT_DEFER` batches doorbells, and `cca_kick()` rings once.
- `cca_attach_dax()` takes a devdax mapping of a non-interleaved region on the memdev; afterwards `cca_pin_addr()` and the other `*_addr` calls take virtual addresses in the mapping and widen them to whole pages.
- All calls are thread safe. The library locks `resource5` with `flock()`, so one process uses a device at a

time. A process that dies holding it leaves its ring state behind; the next `cca_open()` resets the rings. Its pins and uncached ranges stay; `ccactl reset all` clears them.
- `cca_set_timeout()` changes the 10 second default; negative waits forever.

4.20.7 Guest tests

`run-guest-tests.sh` builds the tools, records the kernel and whether BAR5 was assigned, finds the devdax device of the memdev and runs `cca-test`. Run it as root inside the guest:

```
cd hw/femu/tools/cca
sudo ./run-guest-tests.sh -d mem0
```

`-x dax0.0` names the devdax device and `-o LOG` the log file (default `cca-guest-<date>-<time>.log`). Cases can be named after the options. Each case prints `PASS`, `FAIL` or `SKIP`, and the script exits 1 if any failed. The `query`, `thrash`, `invalidate` and `disable` cases map the devdax device and are skipped without one of at least 2 MiB. The tools README[91] lists every case.

The `thrash` and `disable` cases compare access times and expect a factor of 5. With `der=off` a cache hit already costs microseconds of emulation, so `disable` can fail that ratio without anything being wrong; with `der=memslot` and `der=cylon` all cases passed in FEMU's guest runs. `invalidate` brackets its step with `CCA-MARK` lines; between them the host's `media-writes` counter must rise by 64.

4.20.8 Counters

Read with `qom-get` as described in counters (§4.18.9):

Counter	Meaning
`cca-commands`, `cca-errors`	Commands completed, and those with a nonzero status
`cca-pinned`, `cca-uncached`	Pages pinned now, pages in uncached ranges now
`cca-pin-fills`	Pages PIN read from NAND
`cca-writebacks`, `cca-dropped`	Dirty pages programmed, and resident pages dropped, by INVALIDATE and CACHE_DISABLE
`cca-pinned-set-misses`	Misses served from NAND because every way of the set is pinned

`stats-reset` clears the event counters and keeps the two gauges.

4.20.9 Related pages

- CXL SSD (§4.18)
- CCA guest tools README[92]
- CXL SSD design note[93]

4.21 CXL NVMe link

A BBSSD NVMe controller can serve the medium of a `femu-cxl-ssd` (§4.18) as its namespace. The guest then sees the same bytes twice: as CXL memory and as an NVMe block device. Both front ends share one

[91] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tools/cca/README.md#guest-tests
[92] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tools/cca/README.md
[93] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#caching-api-cca

memory backend and one FTL, so one mapping table, one garbage collector and one set of NAND counters cover both.

Use it to study software that moves data between a memory path and a block path on one device. For most other uses, give the guest one view at a time.

4.21.1 Turning it on

Add a `femu` controller with `femu_mode=1` and `cxl_ssd=<id>` after the `femu-cxl-ssd` on the command line. With `run-cxlssd.sh`, whose device id is `cxlssd`:

```
../femu-scripts/run-cxlssd.sh \
    -device femu,id=nvme0,bus=pcie.0,femu_mode=1,cxl_ssd=cxlssd \
    -drive file=$HOME/images/u20s.qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::8080-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0
```

On a full command line:

```
./qemu-system-x86_64 -machine q35,cxl=on,smm=off -accel kvm -cpu host -smp 4 -m 4G \
    -object memory-backend-ram,id=cxlmem,size=256M \
    -device pxb-cxl,id=cxl.0,bus=pcie.0,bus_nr=52 \
    -device cxl-rp,id=rp0,bus=cxl.0,chassis=0,slot=0 \
    -device femu-cxl-ssd,id=cxlssd,bus=rp0,volatile-memdev=cxlmem \
    -device femu,id=nvme0,bus=pcie.0,femu_mode=1,cxl_ssd=cxlssd \
    -M cxl-fmw.0.targets.0=cxl.0,cxl-fmw.0.size=256M
```

Give the controller a bus outside the CXL hierarchy. Without `bus=`, QEMU picks the `pxb-cxl` bus and refuses with "Only PCI/PCIe bridges can be plugged into pxb-cxl". `bus=pcie.0` is the simplest choice; use a `pcie-root-port` instead if you want to unplug the controller at run time (see Unplugging (§4.21.6)).

In the guest the namespace appears as `/dev/nvme0n1` (with the NVMe driver), next to the CXL memdev. The namespace size is the medium's size.

4.21.2 Rules at realize

Realize fails with a message naming the rule when one is broken:

Rule	Message
The `femu-cxl-ssd` exists and comes first on the command line	`Device 'cxlssd' not found`
`cxl_ssd` names a `femu-cxl-ssd`	`Invalid parameter type for 'cxl_ssd', expected: femu-cxl-ssd`
The `femu-cxl-ssd` is realized and not being removed	`cxl_ssd must name a realized femu-cxl-ssd`
The `femu-cxl-ssd` has `ftl=on` (the default)	`cxl_ssd requires the femu-cxl-ssd to have ftl=on`
One controller per medium	`the femu-cxl-ssd already serves an NVMe controller`
`femu_mode=1`	`cxl_ssd requires femu_mode=1`
One namespace; no `namespace_modes` or `namespace_sizes`	`cxl_ssd requires a single bbssd namespace`
No `ns_mgmt` and no `subsys`	`cxl_ssd requires no namespace management or subsystem`
No `streams`, `power_loss`, `buffer_size` or `op_pcent`	`cxl_ssd requires no streams, power_loss, buffer_size or op_pcent`

Rule	Message
No `meta`, `pi` or `dps`	`cxl_ssd requires no metadata or protection information`
`devsz_mb` unset, 1024 or the medium's size in MiB	`devsz_mb must be unset or N, the size of the femu-cxl-ssd`

The medium's geometry and NAND timings apply. The controller's own geometry and timing properties are ignored.

4.21.3 What the controller offers

- Read, Write, Flush, and the optional commands that `oncs` sets. The default sets Dataset Management (Deallocate), Write Zeroes, Copy, Compare and Verify.
- Format NVM is not advertised and Sanitize is refused. Both would rewrite the whole medium behind the CXL cache.
- SMART, vendor log page C0h and the write amplification factor report the shared FTL. CXL traffic does not count as NVMe host I/O.
- The timing admin command 0xEF works. Codes 1 and 2 (GC delay on and off) and 3 and 4 (NAND times) act on the shared FTL and so affect CXL misses too; code 3 restores the medium's `read-ns`, `program-ns`, `erase-ns` and `channel-ns`. Codes 5 to 7 affect the NVMe side only.

4.21.4 Consistency between the two views

The data is consistent without any action from you:

- An NVMe read returns the most recent CXL store, whether the page is clean, dirty in the cache or direct-mapped.
- A CXL access after an NVMe write completes sees the NVMe data.
- Accesses that overlap in time with no synchronization in the guest have no defined order, as on real hardware.

What happens to the cache and the counters:

1. Write, Write Zeroes, Copy (its destination) and Deallocate drop every page they touch from the CXL cache before the command completes, without writing it back: the command already programmed or unmapped the page. `nvme-drops` counts these pages, and the pinned pages of rule 2.
2. A page pinned through the caching API (§4.20) stays pinned and resident but becomes clean.
3. NVMe reads leave the cache alone and are charged a NAND read even when the page is cached.
4. CXL stores mark the blocks they cover as written, so Deallocated or Unwritten Logical Block errors (DULBE) and Get LBA Status see them. A trapped CXL load marks nothing, but with `der=memslot` or `der=cylon` any access that maps a page, a load or a prefetch included, marks the whole page, since stores through the mapping are invisible. Linking to a medium that was already accessed marks every block. A Deallocate does not clear pages that a direct ratio maps; they stay marked as written.
5. NVMe Flush does not flush the CXL cache. The medium is volatile memory.
6. A long cache operation (a flush, a `cache-ways` change, a caching API chunk) delays NVMe write completions until it ends. Reads do not wait.

Do not mount a filesystem on the namespace while the guest also uses the CXL region. The filesystem would overwrite CXL-resident data, and kernel memory when the region is onlined as system RAM. Use one view

at a time unless your workload coordinates the two.

`media-writes` on the `femu-cxl-ssd` includes NVMe writes; it is updated before each NVMe write, Write Zeroes, Copy or Deallocate completes.

4.21.5 Concurrent misses and atomicity

`concurrent-misses` on the `femu-cxl-ssd` controls whether CXL misses to different pages wait for the media together:

Value	Behaviour
`auto` (default)	Together while a direct mode (`der=memslot` or `der=cylon`) is active, one at a time otherwise
`on`	Always together
`off`	Always one at a time

With `der=off`, a guest `lock`-prefixed read-modify-write on CXL memory reaches the device as a separate read and write, so it is never atomic. With misses one at a time, other vCPUs rarely get between the two (under 1% of `lock add` increments were lost with four vCPUs in FEMU's tests). With `concurrent-misses=on` about half were lost. Do not combine `der=off` with `concurrent-misses=on` for anything that uses atomic operations on the region. In the direct modes, on a page that is direct-mapped the write lands on the page the read mapped, and none were lost either way. A `memslot` page left on MMIO because the alias budget is full behaves as with `der=off`.

```
DER=memslot ../femu-scripts/run-cxlssd.sh \
    -global femu-cxl-ssd.concurrent-misses=on \
    -device femu,id=nvme0,bus=pcie.0,femu_mode=1,cxl_ssd=cxlssd
```

4.21.6 Unplugging

While the link exists, `device_del` of the `femu-cxl-ssd` fails with "femu-cxl-ssd is in use by NVMe controller nvme0". Remove the controller first, then the medium. A controller on `pcie.0` cannot be unplugged ("Bus 'pcie.0' does not support hotplugging"), so for this put it on a root port:

```
../femu-scripts/run-cxlssd.sh \
    -device pcie-root-port,id=hp0,bus=pcie.0,chassis=5,slot=5 \
    -device femu,id=nvme0,bus=hp0,femu_mode=1,cxl_ssd=cxlssd
```

Then, over QMP, with the guest running so it can acknowledge each removal:

```
{"execute": "device_del", "arguments": {"id": "nvme0"}}
{"execute": "device_del", "arguments": {"id": "cxlssd"}}
```

The guest can still remove the medium by powering off its root port slot, which QEMU allows without asking. The controller then keeps working as a plain BBSSD over the same bytes until you remove it; the CXL cache goes away with the device. A CXL reset or disabled CXL media leaves the NVMe path working.

4.21.7 Checking it in a guest

FEMU's guest runs checked, in all three `der` modes, that data written through devdax reads back through NVMe and the reverse, that Deallocate reports DULBE errors and reads zeros through devdax, and that `fio` random writes on one half of the namespace run alongside a devdax writer on the other half with both halves verified through both paths. Long runs past the garbage collection threshold have not been checked in a guest.

The guest needs the NVMe driver (`CONFIG_BLK_DEV_NVME`) in addition to the CXL options in guest requirements (§4.18.3). Its `nvme-cli` reads the logs as on any BBSSD:

```
sudo nvme list
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | xxd | head
```

4.21.8 Related pages

- CXL SSD (§4.18)
- CXL caching API (§4.20)
- CXL SSD design note[94]
- Log pages and counters (§B)

4.22 Computational Programs and Subsystem Local Memory

A CSD controller (`femu_mode=4`) can also offer the two ratified NVMe command sets for computational storage:

- Subsystem Local Memory (SLM) 1.2, command set identifier (CSI) 03h: a memory namespace of volatile, byte addressable device memory.
- Computational Programs (CP) 1.2, CSI 04h: a compute namespace that loads, activates and runs programs on ranges of that memory.

Both are opt-in. Without `csd_slm_size` and `csd_cp` the controller is the CSD controller of CSD mode (§4.13), with its CEMU commands only. The CEMU commands stay on and keep their opcodes when the new sets are on.

4.22.1 Requirements

- `femu_mode=4` and one CSD namespace (from `femu_mode` or `namespace_modes`).
- The host enables every I/O command set: CC.CSS 110b. Linux does that when the controller reports more than one command set.
- Linux and nvme-cli have no CP or SLM support. Send the commands through NVMe passthrough (`nvme io-passthru`, `nvme admin-passthru`).
- A `femu-subsys`, with or without FDP, works when the controller is its only controller. The block namespace is in endurance group 1. The memory and compute namespaces report ENDGID 0, because SLM 1.2 2.1.2 and CP 1.2 2.1.1 keep them out of every endurance group.
- Downloaded programs need `csd_program_dir`, as in CSD mode (§4.13.2).

4.22.2 Launch

The shortest device line with both namespaces:

```
-device femu,devsz_mb=64,femu_mode=4,fdm_size=16,csd_slm_size=4,csd_cp=on
```

The controller then has three namespaces:

```
NSID 1   CSD block namespace, CSI 00h   NVM commands, CEMU commands (b0h..f8h)
NSID 2   memory namespace,    CSI 03h   4 MiB of device memory
NSID 3   compute namespace,   CSI 04h   programs
```

[94] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md#nvme-front-end

With `namespaces=N`, the memory namespace is N+1 and the compute namespace N+2. A memory namespace alone (`csd_slm_size` without `csd_cp`) is allowed.

4.22.3 Configuration

Property	Unit	Default	Meaning
`csd_slm_size`	MiB	0	Size of the memory namespace, 1 to 1024. 0 adds none.
`csd_cp`	bool	off	Add the compute namespace. Needs `csd_slm_size`.

`nr_cu` is also the most programs the compute namespace activates at once (MAXACT). `csf_runtime_scale` sets the run time of every CP program, as in CSD mode (§4.13.4).

4.22.4 How a host uses it

```
host                    compute ns (3)              memory ns (2)        block ns (1)
 |  Memory Copy, format 2h --------------------------> SLM <---- read ---- LBAs
 |  Execute Program ----> program on a compute unit
 |                          reads and writes ranges --> SLM
 |  <--- completion: 64-bit return value
 |  Memory Read 02h ----------------------------------> SLM --> host buffer
 |  Copy, format 4h, to namespace 1 -------------------------- SLM ---> LBAs
```

1. Write the input to the memory namespace with Memory Write, or copy it from blocks with Memory Copy, format 2h.
2. Create a Memory Range Set on the compute namespace.
3. Activate a program.
4. Run it with Execute Program, naming the set.
5. Read the result with Memory Read, or copy it to blocks with Copy, format 4h.

Memory Read replaces the CEMU READ_AFDM (f2h) second step.

4.22.5 Commands

Memory namespace (SLM 1.2)

Opcode	Command	Notes
00h	Flush	No effect.
01h	Memory Copy	Formats 2h (blocks) and 4h (memory).
02h	Memory Read	SB in CDW10-11, length in CDW12.
04h	Memory Fill	Fills zeros.
05h	Memory Write	SB in CDW10-11, length in CDW12.

- Addresses and lengths are dword aligned.
- Memory Copy formats 2h and 4h each need their bit in the Copy Descriptor Formats Enable field of Host Behavior Support (feature 16h).
- Each command is atomic as a whole: a lock serializes all memory access, and data moves through a buffer, so a failed transfer changes nothing.

Compute namespace (CP 1.2)

Opcode	Command
00h (I/O)	Flush, no effect
01h (I/O)	Execute Program
85h (admin)	Load Program
88h (admin)	Program Activation Management
89h (admin)	Memory Range Set Management

The admin commands name the compute namespace in NSID.

Programs

PIND	Kind	What it does
0	device-defined	Copies Memory Range 1 into Memory Range 2. Returns the bytes copied.
1	device-defined	FNV-1a, 64 bits, over every range in order, then the program data. Returns the hash.
2 to 15	downloadable	Load Program, type C3h (FEMU shared library) or C1h (FEMU eBPF, with uBPF).

- A downloaded program has the CEMU program format and ABI: the file name, a NUL, the symbol, a NUL. CEMU programs therefore run unchanged.
- A program sees copies of its ranges. The device writes the changed bytes back when the program returns. A queue deletion or a controller reset while the program runs drops the command, and then nothing reaches the memory. A command dropped after its program returned keeps what the program wrote.
- Native programs still run inside QEMU with all of its rights. A program that writes outside its ranges damages QEMU memory, not the memory namespace.

In use

```
Execute fetched ---> program runs ---> completion waits ---> completion posted
|<-------------------- program and range set are in use --------------->|
```

While an Execute is in use, Load Program unload and Program Activation Management deactivate of its program give Program In Use (99h). Memory Range Set Management delete of its set gives Memory Range Set In Use (95h). The forms that name all (FFFFh) change nothing when any entry is in use.

4.22.6 Identify and log pages

Structure	Value
Identify Controller NN	namespaces plus 1 or 2
CRCAP (byte 134)	3h: Reachability Reporting, group identifiers fixed
OAES bit 17	Reachability change notices; the topology never changes, so none is sent
OCFS	bits 2 and 4 with a memory namespace
CNS 1Ch	bit 3 (SLM), bit 4 (CP)
CNS 05h, CSI 03h	NSZE in bytes, NF 0, NOWG 0, MCMCL and MCMSSRL 1 MiB, MCMSRC 127
CNS 05h, CSI 04h	MAXACT `nr_cu`, MAXMEMRS 64, MRSG 2, MAXMEMR 128, MAXPB 64 MiB, LPG 2

Structure	Value
CNS 06h, CSI 03h and 04h	version 1.2
CNS 08h	RGRPID 1 for every namespace

Log pages: log pages and counters (§B.2).

4.22.7 Limits and refusals

Message	Cause
`csd_slm_size and csd_cp need femu_mode=4`	The controller is not a CSD controller.
`csd_slm_size and csd_cp need a CSD namespace`	No namespace runs CSD.
`csd_cp needs a memory namespace: set csd_slm_size`	`csd_cp=on` alone.
`csd_slm_size must be at most 1024 MiB`	Too large.
`csd_slm_size and csd_cp cannot be used with ns_mgmt, power_loss or a CXL link`	The added namespaces have no place in a managed or shared namespace table.
`a subsystem with csd_slm_size takes a single controller`	A second controller joins the subsystem of a controller with a memory namespace, or the reverse. Base 2.3 8.1.21 needs Reachability Reporting on every controller of the subsystem, and only this controller has it.
`CSD: cannot allocate csd_slm_size`	The host has no memory for it.

Where FEMU differs from the specifications:

Specification	FEMU
Several compute and memory namespaces	One of each per controller.
Memory Copy format 3h	Invalid Field: no 16-byte protection information.
A format 2h source with protection information and PRACT=1 is copied without it	Incompatible Namespace or Format for any source with protection information.
A ZNS namespace is a valid format 2h source	Invalid Namespace or Format: no ZNS read timing here.
Fast copy	Not offered: FCO gives Fast Copy Not Possible (186h).
Program Unique Identifiers of NVMe programs	None registered yet; device-defined programs report PIT 000b.
A failed program run	No status is defined; FEMU returns the program's own error, Invalid Field.
A Memory Range outside the memory namespace	No status is defined; FEMU returns Invalid Memory Range Set (8Ch) for a set and Invalid Field for Execute.
CP Figure 53 gives 8Ch for an invalid PIND	FEMU returns 8Fh, as CP Figure 14 does.
Program run time	CP has no field for it; every run takes host time times `csf_runtime_scale`.

4.22.8 Timing

- Memory Read, Write and Fill have no media time. With the host link model on, the link charges the bytes each command moves to or from the host.
- Memory Copy, format 2h, charges the NAND reads of its block ranges on the FTL thread, as a Copy does.
- Copy, format 4h, charges only the write of its destination blocks.

- Execute Program takes the CSD program time: host run time times `csf_runtime_scale` on the first free compute unit.

4.22.9 Verify

The qtests in `hw/femu/tests/qtest/femu-test.c` named `csd-cpslm-*`, `csd-slm-*` and `csd-cp-*` send every command. `caps-csd-cpslm` checks every opcode and log page against the logs. Run them from `build-femu/`:

```
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
  -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests/csd-cp-exec
```

4.22.10 Related pages

- CSD mode (§4.13): the controller, CEMU commands and programs
- CSD design (§4.14.13)
- Several namespaces (§4.15)

4.23 LSM-tree key index for KV namespaces

A key-value namespace keeps a device index from each key to the place of its value. By default this index is an open-addressed hash in device DRAM. With `kv_index=lsm`, the index is an LSM tree instead. The model follows PinK: Im et al., "PinK: High-speed In-storage Key-value Store with Bounded Tails", USENIX ATC '20. The tree changes what a command costs: lookups read meta pages from flash, and merges read and program them.

Use it to study how the index of a KV-SSD shapes latency. It shows the tail latency of flash lookups, the write interference of merges, and the effect of the DRAM that pins the top levels.

4.23.1 Requirements

- A KV namespace: `femu_mode=5`, or `kvssd` in `namespace_modes` (KV mode (§4.11)).
- A host that enables the controller with CC.CSS 110b, as Linux does. KV I/O fails with Invalid Command Opcode under 000b.
- Nothing in the guest beyond what KV mode needs. The index is not visible to the host, except through the time of each command and the order of List.

4.23.2 Launch

```
-device femu,femu_mode=5,devsz_mb=64,kv_index=lsm
```

Small knobs make the tree deep with few keys, which suits a short test:

```
-device femu,femu_mode=5,devsz_mb=64,kv_index=lsm,kv_lsm_memtable_keys=16,kv_lsm_size_ratio=2,kv_lsm_pinned_levels=0
```

`run-kvssd.sh` takes the index from the environment: `KV_INDEX=lsm`, and `KV_LSM_OPTS` for the knobs. The guest recipe `kv-bench-guest.sh` runs the same against both indexes.

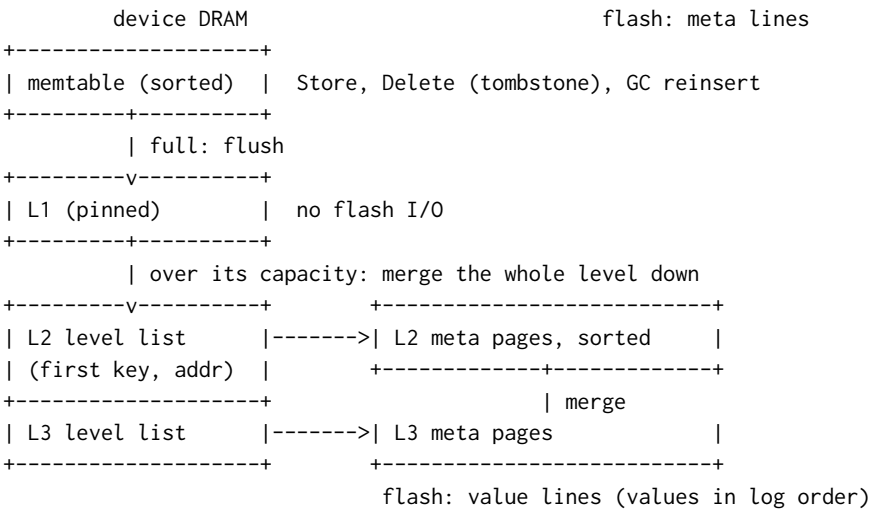
4.23.3 Properties

Property	Default	Range	Meaning
`kv_index`	`hash`	`hash`, `lsm`	the key index of every KV namespace of the controller

Property	Default	Range	Meaning
`kv_lsm_memtable_keys`	1024	16 to 65536	records the DRAM memtable holds before it flushes
`kv_lsm_size_ratio`	10	2 to 64	level `i` holds `memtable_keys * ratio^i` records
`kv_lsm_pinned_levels`	1	0 to 24	top levels held whole in DRAM, with no flash I/O

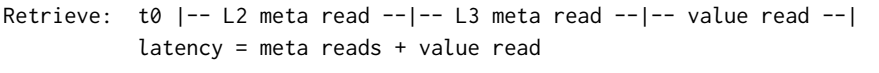
The default memtable size is the 1024 keys of PinK. Each KV namespace has its own tree.

4.23.4 The model



- A record holds a key and a value pointer. A meta page holds `page size / 32` records (128 for 4 KiB pages).
- Meta pages are real NAND pages, on their own write pointer. Their lines hold no value, as PinK keeps meta segments apart from data segments.
- A merge programs new pages and releases the pages it replaces. Value GC also collects meta lines.
- A level list in DRAM holds the first key of each meta page. Thus a lookup reads at most one meta page in each unpinned level.
- A merge takes the whole upper level. It reads and rewrites only the pages of the lower level that its key range overlaps.
- An overwrite adds a new record; a Delete adds a tombstone. Merges drop the older records. A tombstone drops in the deepest level.
- The value space and the values are the same as with the hash index.

4.23.5 What each command costs



Command	Index cost
Retrieve, Exist	a lookup: one meta page read per unpinned level whose key range holds the key, one after the other, until a record is found
Store	none (a blind insert into the memtable); with the SIKE or SINKE option, a lookup
Delete	none (a blind tombstone, also for an absent key); with EDNEK set, a lookup, and no tombstone for an absent key

Command	Index cost
List	each meta page its walk enters; the pages of one level one after the other, the levels side by side
a flushing Store or Delete	the flush and merges, in the background (below)

A command pays only the flash that the tree reads or programs. It pays no base read: that read is the cost model of the hash index, which keeps it. Thus an Exist that the memtable answers takes no NAND time. A blind Delete takes none either, unless it runs value GC.

Merges run in the background

```
Store k (memtable full)  |--value program--|                 latency of Store k
merges, queued:                     reads |-r-|-r-|  programs |-p-|-p-|
next command on a busy LUN                             ...waits...|----|
```

- A merge reads its input pages, then programs its output pages.
- The next merge of a cascade starts when the one before has programmed.
- A flush comes before a new key goes into a full memtable. With 16 memtable keys, the 17th distinct key flushes. An update of a key in the memtable does not flush.
- The I/O starts at the arrival time of the command that flushes. It is not in that command's latency.
- Later commands on the same LUNs wait behind it. A device that cannot merge as fast as the host writes shows this in every command.

Old values stay on flash until the tree drops them

A blind Store or Delete does not look the key up, so the device does not know the old value yet. Its pages stay valid on flash. They become garbage when the tree drops the old record:

```
Store k (blind)     new record (k, v2) in the memtable
                    v1 pages: still VALID on flash
merge over k        drops (k, v1): v1 pages INVALID
value GC finds v1   newest record of k is v2: v1 page INVALID now
```

- An overwrite of a key that the memtable holds replaces the record there, so the old pages go at once.
- NUSE drops at the Delete or the overwrite, as the KV specification defines it. Only the flash keeps the old pages.
- `valid-value-pages` in `query-femu` counts the pages that are still valid, with these old ones.

Value garbage collection

Value GC starts in a Store or a Delete when the free lines fall to the GC threshold that `gc_thres_pcent` sets. As PinK does (section 4.5 of the paper), it collects one victim line at a time:

1. The victim is the line with the fewest valid pages. With no invalid page anywhere, a Store with no room takes the full value lines.
2. For each value on the victim, look its key up. This reads the meta pages of the unpinned levels, once each, level by level.
3. A value that the newest record names moves to a new page, after its read. A stale value is released, with no move.
4. A moved key with its record in an unpinned level goes back into the memtable. Its merges wait for the moves.

5. The victim is erased.

A meta line moves its valid meta pages, and the level list follows them. The Store that runs the collection waits for it. GC ends when the free lines are above the threshold and the Store has room. A victim that gives no free page also ends it, unless a Store still needs room. Then GC tries more queued victims, at most as many as the queue held at the start, and each full line once.

A Store can still have no room while Deletes or old records sit in the levels. The device then merges every run into the deepest level at once. The tombstones and the records they hide drop there. GC then collects the meta pages that this frees.

4.23.6 List order

List returns the keys in byte order, from the first key at or after the start key, then from the smallest key (the wrap). Each live key comes back once. A live start key comes first. A merge keeps the order, so a host that walks the namespace page by page sees each key once. The hash index returns the same keys in table order.

4.23.7 With kv_timing=model

`kv_timing=model` replaces the time of a Store and of a Retrieve that finds its key (workload-mix timing (§4.11.3)). The tree still runs: its reads, programs and counters are the same. Its I/O loads the LUN timelines, so an Exist, a Delete or a List, which keep the NAND time, can wait behind it. The model time does not change.

4.23.8 Counters

The `lsm` member of the namespace in `query-femu` shows the tree (query-femu[95]). It has the memtable, each level, and counters of each kind of meta page I/O. `femu-stat.py` prints them with their change over each interval.

A merge program counts as a page programmed (`nand_write_pages`), not as a page the host asked for. In `gc_write_pages`, GC counts each meta page or value it moves as a page relocated. Thus the write amplification of the health log and of log page C0h includes the index.

4.23.9 Limits

- Meta pages take NAND pages, but no value space: NUSE counts keys and values only. A Store leaves free a reserve: two lines, and the pages of a merge that rewrites every record. A Store that would use the reserve gets Capacity Exceeded.
- A merge that finds too few free pages changes nothing. Every merge leaves one line free, for value GC. A Store whose flush fails gets Capacity Exceeded. A Delete keeps its tombstone in the memtable over its capacity, so it succeeds; the next put flushes again.
- There is no Bloom filter. PinK also has none.
- With `kv_timing=model`, the model time also hides the time of value GC, as it hides the other NAND time of a Store.

Message	Cause and fix
`kv_index=X is not hash or lsm`	Use `hash` or `lsm`.
`kv_lsm_memtable_keys, kv_lsm_size_ratio and kv_lsm_pinned_levels need kv_index=lsm`	A knob other than its default with the hash index.

[95] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md#kv-lsm-index`

Message	Cause and fix
`kv_lsm_memtable_keys=N is not 16 to 65536` (and the other ranges)	Use a value in the range of the table above.
`kv_index=lsm and the kv_lsm_* knobs need a key-value namespace ...`	The device has no KV namespace.
`FEMU kvssd: kv_index=lsm needs a page of at least 32 bytes`	Use a larger `secsz` or `secs_per_pg`.

4.23.10 Verify

1. `query-femu` on the namespace has an `lsm` member.
2. After a load, its `levels` list grows and `flushes` counts up.
3. `kv-bench` passes in the guest, as with the hash index.

4.23.11 Related pages

- KV mode (§4.11)
- KV design: the key index (§4.12.5)
- query-femu[96]

4.24 KV value packing

A key-value namespace packs small values into shared NAND pages, as real KV-SSDs do. PinK (Im et al., USENIX ATC '20) appends values to a value log. The Samsung KV-SSD firmware, KAML and iLSM also put several small values in one page. Packing is on by default (`kv_pack=on`).

Without packing, a 100 byte value programs a whole 4 KiB page. That model wastes 97% of the page, so small-value WAF, GC load and capacity are far from a device. `kv_pack=off` keeps that model for old results.

4.24.1 Requirements

- A KV namespace: `femu_mode=5`, or `kvssd` in `namespace_modes` (KV mode (§4.11)).
- Both key indexes: the hash (default) and `kv_index=lsm` (LSM-tree key index (§4.23)).
- Nothing in the guest. The host sees packing only in the time of each command and in the counters.

4.24.2 Launch

Packing needs no property:

```
-device femu,femu_mode=5,devsz_mb=64
```

With a volatile write cache, so that Flush programs the open page:

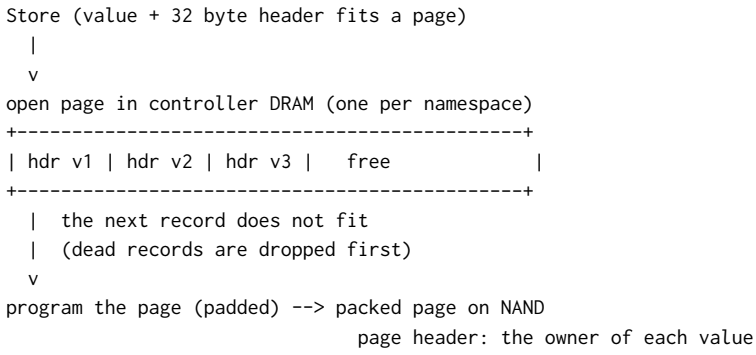
```
-device femu,femu_mode=5,devsz_mb=64,vwc=1
```

The old model, one value or more per page:

```
-device femu,femu_mode=5,devsz_mb=64,kv_pack=off
```

[96] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md

4.24.3 The model



- A value packs when its record fits a page. A record is the value and a 32 byte header: the key, the length, a version and the offset. A value log record holds the key, so that GC can find the index entry.
- A value never crosses a page. Thus a Retrieve reads one page.
- A larger value takes whole pages, as before. A zero-length value takes no page.
- The index entry holds the page and the offset of a packed value.
- With the hash index, an overwrite or a Delete makes the old record dead at once. With `kv_index=lsm`, a new record that replaces one in the memtable kills the old record at once. An old record in a level stays live until a merge drops it or value GC finds it stale (as for whole pages). A page stays valid until its last record dies.
- Dead records in the open page are dropped before every program, as a write buffer drops a replaced page. The old and the new record must fit the page together: `2 * (length + 32) <= page size`, so up to 2016 bytes with 4 KiB pages. Then 200 overwrites of one key program nothing. This needs the open page to stay in DRAM (not `vwc=1` with WCE cleared), no Flush and no other key. With the LSM tree, the key must also stay in its memtable.
- The Store whose record fills the page with live records programs it after the record it replaced is released.

4.24.4 Durability

KV Store has no FUA bit (KV 1.3, Figure 30). The Base spec rules for the volatile write cache apply:

Configuration	The open page	Store time
`vwc=0` (default)	stays in DRAM; the cache is non-volatile (Base 2.3, 5.2.26.1.4 note)	the DRAM time
`vwc=1`, WCE set	stays in DRAM until Flush or shutdown	the DRAM time
`vwc=1`, WCE cleared	programmed with each Store, padded (Base 2.3, 5.2.26.1.4)	the program time

Event	Action
Flush, `vwc=1`, WCE set	programs the open page; waits for every earlier program of packed values (Base 2.3, 7.2)
Flush otherwise	no effect, as the spec asks
Set Features 06h clears WCE	programs the open page of every KV namespace first
CC.SHN	programs the open page of every KV namespace
the next record does not fit, or the page is full of live records	programs the open page; the Store that needs the room, or fills the page, waits for it

The DRAM time is `pg_rd_lat / 16`, the time a bbssd write buffer charges. The program at Set Features and at shutdown loads the LUNs; the admin command and the shutdown status do not wait for it. FEMU models no power loss for KV, so the index in device DRAM needs no journal.

4.24.5 What each command costs

Command	Packed value
Store, record in the open page	the DRAM time, and the index cost
Store whose record does not fit, or fills the page	the program of the full page
Retrieve, value in the open page	the DRAM time, and the index cost
Retrieve, value on NAND	one page read (the whole page, as the media model reads whole pages)
Flush	the open page program, and the earlier programs

The index cost is the hash base read, or the LSM lookup. With `kv_timing=model`, Store and Retrieve take the model time; packing then changes only the NAND load, the space and the counters.

4.24.6 Garbage collection

```
victim = closed line with the fewest live bytes
         (each closed line once per GC run)
  |
  v
packed page: read it once
  each live record --> GC page (programmed when full, and before the erase)
  lsm: a stale version --> released, no move
whole page or meta page --> moved as before
  |
  v
erase the victim
```

- Live bytes count a record of each live packed value, and a full page for a whole value or a meta page.
- A full line can hold dead records. Thus full lines are victims too.
- The GC page is programmed before the victim is erased. A moved value is never only in DRAM.

4.24.7 Counters

Counter	With packing
C0h pages programmed (offset 24)	each packed page and each whole value page
C0h host pages (offset 8)	the value bytes of the Stores, in whole pages
C0h GC pages (offset 16)	each GC page and each moved whole page
C0h WAF	programmed pages over host pages

The host page count rounds down. Thus the WAF is exact only after many pages. The `pack` member of `query-femu`[97] gives the host bytes for an exact ratio, and the open page, pad bytes and GC moves. `femu-stat.py` prints them with their change since the sample before.

[97] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md#kv-value-packing`

4.24.8 Properties

Property	Type	Default	Meaning
kv_pack	bool	on	on: values whose record fits a page share pages; off: each value takes whole pages

`kv_pack=off` with no KV namespace is refused: `kv_pack=off needs a key-value namespace (femu_mode=5 or kvssd in namespace_modes)`.

4.24.9 Limits

- One open page per namespace. A device can hold more pages in DRAM; that changes only when programs happen.
- The Store that forces a program waits for it, as a bbssd write waits for the destage it forces.
- A Store needs one free page more while the open page holds a live value, so that a Flush always finds its page.

4.24.10 Validation

The qtests `kv-pack-pages`, `kv-pack-off`, `kv-pack-overwrite`, `kv-pack-overwrite-off`, `kv-pack-flush`, `kv-pack-flush-wait`, `kv-pack-read-time`, `kv-pack-gc` and `kv-pack-gc-lsm` cover packing. `FEMU_KV_CHECK` checks each page header, each owner and the live bytes of each line.

4.24.11 Related pages

- KV mode (§4.11)
- KV design (§4.12.4)
- LSM-tree key index (§4.23)
- Log pages and counters (§B.1)

4.25 Tiered NAND: a pSLC pool in front of dense NAND

Many TLC and QLC SSDs write host data to a fast cache first. The cache is ordinary NAND that the drive programs with one bit per NAND unit (pSLC). Later the drive moves the cached data to the dense blocks. This move is a fold.

FEMU can model this in a BlackBox (§4.4) SSD. The tier is off by default. With the tier off, FEMU allocates no tier state, and timing, counters and garbage collection do not change.

Use it to study the write cliff of a full cache, the cost of a fold, and the read speed of data in the cache.

4.25.1 What it models

```
host write --> pSLC frontier --> pool line full --> fold FIFO
                  |                                    |
    pool empty:   |                                    v
    direct  --> dense frontier           read pSLC page (tier_pslc_read_ns)
    through --> fold first, then pool    program dense page (TLC/QLC time)
                                         erase pool blocks (tier_pslc_erase_ns)
                                         line back to the free pool
```

- The pool is static. `tier_static_mb` of host data sets its size in whole lines. The pool lines come from the over-provisioning and stay pSLC for their life.
- A pool block holds `pgs_per_blk / b` pages, rounded down per block. `b` is 2 for MLC, 3 for TLC and 4 for QLC (`nand_cell_type`).
- Host writes, FUA writes, Write Zeroes and Copy destinations go to the pool while it has a free page.
- A folded pool line is free again only when its erase ends.
- When the pool has no free page, `tier_full_policy` decides:
  - `direct` (default): the page goes to the dense lines. This also applies while the free pool lines still erase.
  - `through`: the write waits for the erase of a free pool line. With no free line, it folds the oldest pool line first and waits for that fold.
- With `direct`, the end of a request folds one pool line when the pool has no free page left. Version 1 has no other fold trigger.
- A fold takes the oldest full pool line. Each valid page is read in pSLC mode, then programmed on the dense frontier. The program starts only when the read data is ready.
- The erase of the pool line waits for the last program of the fold. Pages that the host deallocated or overwrote before the fold do not move.
- Reads of pool pages take `tier_pslc_read_ns`. Programs take `tier_pslc_prog_ns`. The built-in SLC times of `nand_cell_type=1` do not change.

The pool never enters the dense free list or the garbage collection victim queue. Dense garbage collection works as before on the dense lines.

4.25.2 Launch

The tier needs `nand_cell_type` 2 (MLC), 3 (TLC) or 4 (QLC). A 1 GiB TLC device with a 64 MiB pool:

```
-device femu,femu_mode=1,devsz_mb=1024,nand_cell_type=3,tier_mode=static,tier_static_mb=64
```

With the default geometry a pool line holds 64 blocks of 85 pages, so the pool takes 4 lines.

4.25.3 Properties

Property	Default	Meaning
`tier_mode`	unset (off)	`static` or `dynamic` turns the tier on
`tier_static_mb`	`0`	host data in MiB the static pool holds
`tier_full_policy`	unset (`direct`)	`direct` or `through`
`tier_pslc_read_ns`	`25000`	pSLC page read time in ns
`tier_pslc_prog_ns`	`200000`	pSLC page program time in ns
`tier_pslc_erase_ns`	`3500000`	pSLC block erase time in ns
`tier_pslc_pe_cycles`	`0`	pSLC erase rating; 0 is 30 times the dense rating
`tier_max_mb`	`0`	`dynamic`: cap of the borrowed lines in MiB; 0 sets no cap
`tier_shrink_pct`	`50`	`dynamic`: fill in % where the pool starts to shrink
`tier_idle_ms`	`0`	quiet time in ms before an idle fold; 0 turns idle folds off
`tier_lazy_pct`	`0`	idle folds: 0 folds every closed line; else only while use is above this %

The full descriptions are in the property reference (§A.1.1).

4.25.4 Dynamic pool

With `tier_mode=dynamic`, the pool also borrows free dense lines while the device has room. `tier_static_mb` is then a floor and can be 0. A borrowed line goes back to the dense free list after its fold:

```
dense free --borrow--> open --> closed (FIFO) --fold--> dense free
```

The number of borrowed lines the pool may hold falls as the device fills:

```
lines
max |________
    |        \
    |         \
  0 +---------+--\---> logical fill (valid host pages / exported pages)
    0      shrink   100%
```

- `max` is `tier_max_mb` in whole lines, rounded down, or every line when it is 0. Realize refuses a cap below one pool line.
- The fill counts mapped FTL pages. A deallocate of part of a page unmaps the whole FTL page, as without the tier.
- `shrink` is `tier_shrink_pct`. Above it, the allowance falls in a straight line to 0 at a full device.
- After each request, a pool that holds more closed borrowed lines than the fill allows folds the oldest ones until it does not. The open line folds after it fills.
- When the pool has no line and may borrow none, the end of the request folds the oldest pool line, as in a static pool.

FEMU borrows a line only when three rules hold:

1. The fill allows one more borrowed line.
2. The dense lines that stay still hold every valid page, plus one full pool line.
3. The dense free lines stay above the forced garbage collection threshold.

Before each host page, FEMU checks rule 2 again for the page. When the page would break it, a borrowed line folds first. So a full device always has dense room for every page, and the pool shrinks to its floor.

`tier_full_policy=through` needs a static floor, so `dynamic` with `through` also needs `tier_static_mb` above 0.

A 4 GiB TLC device with no fixed pool and a cache of up to 1 GiB:

```
-device femu,femu_mode=1,devsz_mb=4096,nchs=8,luns_per_ch=8,pgs_per_blk=256,blks_per_pl=256,nand_cell_type=3,tier_mode=dynamic,tier_max_mb=1024
```

4.25.5 Idle folds

By default a pool line folds only when a host write needs room. With `tier_idle_ms` above 0, the device also folds in idle time, so the next burst finds free pool lines:

```
burst --> pool full --> quiet for tier_idle_ms --> step, step, ... --> next burst
                                                    |
                          each step: at most one page per LUN, then wait
                          until the work of that step ends
```

- The idle time counts from the end of the last host request and of the NAND work it queued, such as GC. The PCIe transfer time is not part of it.

- A step moves at most one page from each LUN. The next step starts only when the programs (and, at the end of a line, the erases) of the last one end. So idle work adds at most one step on a LUN to a host command.
- A host request between two steps runs as usual. A fold that the request needs takes the half-moved line first and moves all of it.
- At the forced GC mark (`gc_thres_pcent_high`), idle folds stop until a host write runs GC. An idle step never runs dense GC.
- The open pool line does not fold at idle, so one open line can stay.
- `tier_lazy_pct=0` (eager) folds every closed line. A value of 1 to 99 (lazy) folds only while the closed and open lines are above that percent of the pool. The pool is the static lines plus the borrowed lines the fill allows. Lazy keeps more recent data in pSLC.
- An idle fold of a borrowed line gives it back to the dense free list.
- Idle folds charge their time, as other folds do.

A 4 GiB TLC device with a 512 MiB static pool that folds after 100 ms of quiet time:

```
-device femu,femu_mode=1,devsz_mb=4096,nchs=8,luns_per_ch=8,pgs_per_blk=256,blks_per_pl=256,nand_cell_type=3,␣
    tier_mode=static,tier_static_mb=512,tier_idle_ms=100
```

4.25.6 Capacity

The exported capacity does not change. Realize checks that the dense lines alone can hold the whole namespace plus the garbage collection reserve. The check counts the static pool only; a dynamic pool keeps the same rule at run time (see Dynamic pool (§4.25.4)):

```
(lines - pool lines - reserve lines) x dense pages per line  >=  exported pages
```

Each valid pool page counts as a mapped page, so the dense lines can always take a fold of the whole pool.

When the check fails, realize refuses the device and names the pool. Lower `tier_static_mb`, or raise `op_pcent` or `blks_per_pl`.

4.25.7 Wear

FEMU counts the pSLC erases of each block. The wear of a block is the share of each rating it used:

```
wear = dense erases / dense rating + pSLC erases / pSLC rating
```

The dense rating is `pe_cycles_rated`, or the rating of `nand_cell_type`. In version 1 this figure is for tests only. SMART Percentage Used still counts every erase against the dense rating, so it shows more wear than the figure. The ECC wear tiers would make the same error, so realize refuses `ecc_step_ns` with the tier.

4.25.8 Counters

- Log page C0h offset 16 (pages relocated) includes the fold pages. A fold page is never counted twice.
- Log page C0h offset 24 (pages programmed) counts host pages in the pool and in the dense lines, but not fold pages.
- `query-femu` does not report the tier yet. Pool lines show as unlisted.

4.25.9 Example: the write cliff

One run on a CloudLab d760 host wrote 3 GiB once, in sequence, at 128 KiB and queue depth 32. The device was a 4 GiB TLC BlackBox SSD with 64 dies and a 1 GiB pool:

```
-device femu,femu_mode=1,devsz_mb=4096,nchs=8,luns_per_ch=8,pgs_per_blk=256,blks_per_pl=256,nand_cell_type=3,tier_mode=static,tier_static_mb=1024
```

The pool takes 49 lines (1041 MiB). The write rate drops when the pool is full:

```
MiB/s
1231 |##########
     |          #
  30 |           ##########################
     +----------+--------------------------> MiB written
               ~1024 (pool full)

policy   drop at    after       folds   relocated pages
direct   1024 MiB   29.3 MiB/s  81      440640 (81 x 5440)
through  1008 MiB   30.1 MiB/s  96      522240 (96 x 5440)
```

Without the tier, the same write runs at about 85 MiB/s from start to end. After the drop, host pages and fold work share the same dies, so the rate falls below the plain TLC rate. The rates depend on the host.

4.25.10 Presets

These are starting points, not values from one datasheet. Each needs a device with enough over-provisioning for the pool.

```
consumer TLC    nand_cell_type=3,tier_mode=static,tier_static_mb=6144
consumer QLC    nand_cell_type=4,tier_mode=static,tier_static_mb=24576,tier_full_policy=through
industrial TLC  nand_cell_type=3,tier_mode=static,tier_static_mb=2048
```

With the default zero bus times, a fold costs only array time. To charge the channel too, set a bus profile such as `pg_xfer_lat` or `onfi_mode`.

4.25.11 Limits

- bbssd only, with one namespace and page mapping.
- Realize refuses the tier with FDP, KV, ZNS, CSD, OCSSD, `mapping` other than `page`, `buffer_size`, `power_loss`, Streams, `hot_cold_sep`, `gc_pace` and `gc_copyback`.
- Realize also refuses it with `blk_pe_limit`, `wl_spread`, `read_reclaim_limit`, `retention_limit_sec`, the `pd_*` properties, `mp_program`, `onfi_cache_read`, `pe_suspend` and `ecc_step_ns`.
- With `pe_suspend`, a read could move the end of a fold program past the time the erase waits for, so the tier refuses it.
- `tier_full_policy=through` needs `tier_static_mb` above 0. A static pool always needs it. A dynamic pool needs it only with `through`.
- Folds always charge their time. `enable_gc_delay` off (the GC delay flip) does not remove the fold cost.
- Dense programs use the FEMU page model: one program per page, timed by its page type. A one-shot word line program is a later option.
- A fold that a host write needs moves a whole pool line in one step. Host requests do not run between its page moves. Idle folds move one step at a time.
- No read promotion yet.

4.25.12 Credit

The design studies Hybrid_FEMU by Funny-Boy-Max (GitHub #113) and the thesis 'Research on Read Performance Optimization Techniques for Hybrid Solid-State Drives' (Zhao Yumiao, 2024). FEMU uses no code from Hybrid_FEMU.

5 Running and configuring

5.1 Parameter manual

This manual explains how to configure an emulated device: the ways a parameter reaches FEMU, then every group of parameters by component, with what each one means, its unit, the values it accepts, and how it interacts with the others. It ends with worked configurations.

The property reference (§A.1) is generated from the binary and is the authority on names, types and defaults. This manual never states a default; each group links to its table there. Run-time properties and counters are in runtime-properties.md (§A.2).

5.1.1 Contents

1. How parameters reach FEMU (§5.1.2)
2. Devices and how they combine (§5.1.3)
3. Controller (§5.1.4)
4. Namespaces and subsystems (§5.1.5)
5. NAND geometry (§5.1.6)
6. NAND timing (§5.1.7)
7. FTL: mapping, caches and write buffer (§5.1.8)
8. Garbage collection (§5.1.9)
9. Reliability, wear and fault insertion (§5.1.10)
10. Host link and controller firmware (§5.1.11)
11. ZNS (§5.1.12)
12. FDP (§5.1.13)
13. OCSSD, CSD and KV (§5.1.14)
14. CXL SSD (§5.1.15)
15. Accepted for compatibility, no effect (§5.1.16)
16. Worked configurations (§5.1.17)

5.1.2 How parameters reach FEMU

1.1 Device options on the QEMU command line

Every FEMU parameter is a property of one of three QEMU devices, given on the command line when QEMU starts:

```
-device femu-subsys,id=SUB,prop=value,...     NVMe subsystem (FDP, shared namespaces)
-device femu-cxl-ssd,id=CXL,prop=value,...    CXL Type-3 SSD
-device femu,id=CTRL,prop=value,...           NVMe controller and its namespaces
```

The rules of QEMU's option syntax apply:

- Options are separated by commas. A comma inside a value is written twice: `namespace_modes=bbssd,,znssd` is the list `bbssd,znssd`.
- Booleans take `on` or `off` (QEMU also accepts `yes`/`no` and `true`/`false`).
- Properties of type `size` take a number of bytes or a QEMU size with a suffix (`64M`, `4G`). Integer properties take plain numbers, decimal or `0x` hexadecimal; their unit is in the property name or in this manual (`_mb` MiB, `_ns` and `_lat` nanoseconds, `_pcent` percent).

- A property that links to another device (`subsys`, `cxl_ssd`) names the other device's `id`, and that device must come earlier on the command line.
- A misspelled name stops QEMU with `Property 'femu.NAME' not found`.

Static properties are read once, when QEMU creates (realizes) the device. Realize checks the values and their combinations and stops QEMU with a message naming the rule that failed; the "Limits and refusals" section of each mode page (§4.4.5) lists them. `./qemu-system-x86_64 -device femu,help` prints every property with its type, default and description; `femu-subsys,help` and `femu-cxl-ssd,help` do the same for the other two devices.

1.2 Configuration files

`hw/femu/scripts/ssd-config.sh` expands a file of `key = value` lines into the same `-device` options, so a configuration can be read, commented and kept in version control. Keys are property names, `mode = bbssd` stands for `femu_mode`, and a `[subsys]` section becomes a `femu-subsys` device that the controller joins. The script checks names against `-device femu,help` (and `femu-subsys,help` for `[subsys]` keys) of the binary in `FEMU_BIN`, or of one it finds in the build directories; without a binary it only warns. It also rejects an unknown `mode` and malformed lines. QEMU still checks the values. Tutorial 09 (§5.11) walks through it, and scripts and tools (§C.4) describes the shipped files.

1.3 Run-time properties over QMP

A few properties can be read or changed while the guest runs, with QMP `qom-get` and `qom-set` on `/machine/peripheral/<id>`:

- `femu`: `simulate-power-loss`, a write-only trigger for the power-cut model (power loss (§5.1.8)).
- `femu-cxl-ssd`: the cache tunables `cache-ways`, `prefetch-degree` and `prefetch-stride` (also accepted on `-device`), the actions `flush-cache`, `stats-reset`, `der-ratio` and `control-command`, and every counter.

The full list is runtime-properties.md (§A.2). `femu` and `femu-subsys` have no counter properties; their counters are NVMe log pages (log pages and counters (§B)).

1.4 Run-time switches from the guest

These are not parameters, but they change the model while the guest runs:

- Vendor admin command 0xEF on a BlackBox controller turns GC time and the flat NAND times off and on, and counts completions posted after their due time. Code 3 sets the built-in flat times, not the ones on your command line; code 4 sets them to 0 (timing model (§6.2.11)).
- Set Features 06h (Volatile Write Cache) turns the write buffer off and on when `vwc=1`; Set Features 20h (Key Value Configuration) sets EDNEK on a KV namespace; Set Features 04h and 0Bh set the temperature threshold and the asynchronous events the host wants.

1.5 Environment variables

A few debugging and host-placement aids are environment variables of the QEMU process, not properties: environment variables (§A.1.5).

5.1.3 Devices and how they combine

```
femu-subsys (optional)               femu-cxl-ssd (optional)
FDP, shared namespaces               CXL memory over NAND
      ^ subsys=SUB                         ^ cxl_ssd=CXL
      |                                    |
femu (NVMe controller) ----------------------+
  controller parameters        section 3
  femu_mode, namespaces        section 4
  per-mode parameters:
    bbssd, CSD   geometry 5, timing 6, FTL 7, GC 8, reliability 9
    KV           geometry 5, timing 6, part of 8
    ZNS          zns_* 11
    OCSSD, CSD   section 13
  host link, firmware          section 10
```

A `femu` controller runs one mode per namespace. `femu_mode` picks it for every namespace, and `namespace_modes` per namespace. Only the parameters of the modes in use have an effect; the description of each property in properties.md (§A.1) names the modes it applies to. A property of a mode that no namespace runs has no effect. The properties of section 15 (§5.1.16) have no effect in any mode and warn when set.

5.1.4 Controller

These shape the NVMe controller the guest driver sees, whatever the mode.

3.1 Queues and pollers

Reference: queues, pollers and interrupts (§A.1.1).

Parameter	Unit	Valid values	Meaning
`queues`	count	1 to 2047	I/O submission and completion queue pairs the controller offers; MSI-X vectors are `queues` + 1
`entries`	count, 0's based	1 to 65534	CAP.MQES; queues may have up to `entries` + 1 entries
`multipoller_enabled`	0 or 1	0, 1	0: one poller thread for every I/O queue; 1: several, see `poller_ratio`
`poller_ratio`	queues per poller	0 or more (0 counts as 1)	with `multipoller_enabled=1`, `ceil(queues / poller_ratio)` pollers, each serving a round-robin share of the queues
`stride`	power of two	0 to 12	doorbell stride CAP.DSTRD: doorbells are `4 << stride` bytes apart
`max_sqes`, `max_cqes`	power of two	6 and 4 only	Identify SQES and CQES; the only accepted values are the 64-byte and 16-byte entries
`aerl`	count, 0's based	0 to 255	outstanding Asynchronous Event Requests the controller holds
`elpe`	count, 0's based	0 to 255	Error Information log entries kept
`hiops_inline`	bool	`on`, `off`	NoSSD only: performance option, on by default; set off only when debugging
`intc`, `intc_thresh`, `intc_time`	as named	`intc` 0 or 1	initial values of the Interrupt Coalescing and Interrupt Vector Configuration features; coalescing runs when `intc_thresh` and `intc_time` are both non-zero

Interactions:

- Every poller spins on a host core while the controller is enabled. The number of pollers follows `queues`, not the number of queues the guest creates, so set `queues` to the guest's vCPU count when you turn on more pollers (performance tuning (§5.12.2)).

3.2 Identity, capabilities and transfers

Reference: controller identity and capabilities (§A.1.1).

Parameter	Unit	Valid values	Meaning
`vid`, `did`	PCI IDs	16-bit	PCI vendor and device ID; `vid` is also Identify VID
`mdts`	power of two	0 to 255; 0 = no limit	largest transfer, 2^(12 + `mpsmin` + `mdts`) bytes
`mpsmin`, `mpsmax`	power of two	`mpsmin` <= `mpsmax` <= 15	host memory page sizes the controller accepts, 2^(12 + n) bytes
`cqr`	0 or 1	0, 1	1 requires physically contiguous queues
`vwc`	0 or 1	0, 1	advertise a volatile write cache and make feature 06h usable; Flush drains the write buffer either way
`oacs`	bit mask	only bit 1 (0x2)	Format NVM support; clearing it refuses Format NVM
`oncs`	bit mask	0x1 Compare, 0x2 Write Uncorrectable, 0x4 Dataset Management, 0x8 Write Zeroes, 0x10 Save/Select, 0x80 Verify, 0x100 Copy	optional NVM commands; the default 0x19d sets all but Write Uncorrectable; Timestamp is always added
`sgl`	bool	`on`, `off`	accept scatter gather lists; OCSSD ignores it
`cmbsz`, `cmbloc`	registers	`cmbsz` size x unit a power of two; `cmbloc` BAR field 2	Controller Memory Buffer; 0 means none
`temperature`	kelvin	16-bit	composite temperature in the SMART log, compared with the threshold feature
`pel_file`	host path	a writable path	keeps the Persistent Event log and power cycle count across QEMU runs (retention (§B.6))

Interactions:

- Write Zeroes, Compare, Copy and Verify are on by default; `oncs=0x14` gives the old default, with Dataset Management and Save/Select only. Write Uncorrectable needs bit 0x2. Open-Channel SSD refuses all the optional commands. Flush is always accepted and drains the bbssd write buffer even with `vwc=0`, though Linux sends no Flush to a controller that advertises no cache.
- With ZNS, `mdts` also caps Zone Append when `zns_zasl_bs=0`.

3.3 LBA formats, metadata and protection

Reference: LBA formats, metadata and protection (§A.1.1).

Parameter	Unit	Valid values	Meaning
nlbaf	count	1 to 16, at most 8 with meta	LBA formats offered: 512 bytes, doubling with each
lba_index	index	below nlbaf	format the namespaces boot with; ZNS needs 4 KiB or less
meta	bytes per block	NoSSD and bbssd only	metadata per logical block; each format is then also offered with metadata
mc	bit mask	bit 0 interleaved, bit 1 separate	Metadata Capabilities; required with meta
extended	0 or 1	needs mc bit 0	boot with metadata interleaved (extended LBAs)
pi	bool	takes effect only with meta >= 8	offer protection information types 1 to 3 to Format and Create
dpc, dps	bit masks	dpc 0 with meta; leave dps 0	Identify fields; use pi instead

Interactions: meta is refused with FDP, with dpc or dps, and with any namespace that is not NoSSD or bbssd; pi is refused with power_loss and cxl_ssd (namespace management, metadata and PI (§4.17)).

5.1.5 Namespaces and subsystems

Reference: mode, capacity and namespaces (§A.1.1), namespace management, streams and power loss (§A.1.1), shared namespaces (§A.1.2). Design: subsystems, controllers and namespaces (§4.16).

Parameter	Unit	Valid values	Meaning
femu_mode	mode number	0 OCSSD, 1 bbssd, 2 NoSSD, 3 ZNS, 4 CSD, 5 KV	mode of every namespace unless namespace_modes is set
devsz_mb	MiB	32-bit	host memory backend, split across the namespaces; ignored with op_pcent
namespaces	count	1 to 256; OCSSD 1	namespaces present from boot
namespace_sizes	bytes, QEMU sizes	one entry per namespace, each >= 512 bytes, sum <= backend	size of each namespace; unset splits the backend evenly
namespace_modes	list	nossd, bbssd, znssd, ocssd, csd, kvssd, one per namespace	mode of each namespace
namespace_phls	list of lists	FDP only; one entry per namespace, each 1 to MIN(fdp.nruh, 128) handle IDs below fdp.nruh, joined with :, none twice	placement handle list of each namespace; unset gives every handle
op_pcent	percent	bbssd; not with cxl_ssd	back the device with the full NAND and expose NAND / (1 + op_pcent/100)
ns_mgmt (femu)	bool	standalone NoSSD or bbssd controller	Namespace Management and Attachment from the guest
bbssd_ns_limit	count	1 to 256, >= namespaces	most bbssd namespaces ns_mgmt may allocate, each with its own FTL
streams, streams.max	bool, count	streams.max 1 to 32	the Streams directive; bbssd separates streams per FTL page
power_loss	bool	see interactions	roll back writes still in the write buffer on a simulated power cut
subsys	device id	a femu-subsys listed earlier	join a subsystem

Parameter	Unit	Valid values	Meaning
ns_mgmt (femu-subsys)	bool	NoSSD and bbssd controllers, not with fdp	one namespace table and backend shared by every controller of the subsystem
nqn (femu-subsys)	string	any	subsystem NQN suffix

Interactions:

- Namespaces are packed one after another in the backend. Each bbssd or CSD namespace has its own FTL built from the whole NAND geometry, so each must fit that geometry on its own with room for GC (the reserve (§3.4.12)). Each ZNS namespace builds its zones from its own size.
- op_pcent overrides devsz_mb: the backend becomes the raw NAND, and each namespace gets its share of NAND x 100 / (100 + op_pcent).
- A controller may hold at most one CSD namespace; OCSSD takes the whole controller.
- ns_mgmt on femu stays off without an error unless every namespace runs femu_mode with dps 0; with a shared subsystem use the subsystem's ns_mgmt.
- streams needs bbssd with mapping page or dftl (or NoSSD, where it has no placement effect). It is refused with FDP, with shared namespaces (ns_mgmt on the subsystem), and when a second controller joins the subsystem. bbssd reserves streams.max + 1 lines for it.
- power_loss needs bbssd (femu_mode=1), buffer_size > 0 and page-aligned namespaces. Without vwc=1 it is accepted but the write buffer is off, so there is nothing to roll back. It is refused with meta, pi, ns_mgmt, subsys, namespace_modes and cxl_ssd.

5.1.6 NAND geometry

Applies to bbssd, CSD and KV. Reference: NAND geometry (§A.1.1). Design: geometry (§3.5.3).

Parameter	Unit	Valid values	Meaning
secsz	bytes	> 0	sector size
secs_per_pg	sectors	1 to 256	sectors per NAND page; page size = secs_per_pg x secsz
pgs_per_blk	pages	1 to 65536; <= 512 with nand_cell_type	pages per block
blks_per_pl	blocks	1 to 65536	blocks per plane; also the number of lines
pls_per_lun	planes	1 to 16	planes per LUN
luns_per_ch	LUNs	1 to 128	LUNs (dies) per channel
nchs	channels	1 to 4096	channels

Derived quantities:

```
page           = secs_per_pg x secsz bytes
NAND capacity  = nchs x luns_per_ch x pls_per_lun x blks_per_pl x pgs_per_blk x page
lines          = blks_per_pl          (a line is one block on every plane of every LUN)
line size      = nchs x luns_per_ch x pls_per_lun x pgs_per_blk x page
parallel units = nchs x luns_per_ch   (LUNs work in parallel; planes of a LUN together)
```

Interactions:

- The total sector count must fit in a signed 32-bit integer.

- Without op_pcent, the namespace is devsz_mb and must leave GC its reserve of lines; the reserve grows with gc_thres_pcent_high, hot_cold_sep, log-block mapping and Streams (the reserve (§3.4.12)).
- Fewer, larger lines make GC coarser; more LUNs and channels raise throughput, not single-request latency.

5.1.7 NAND timing

Applies to bbssd, CSD and KV; ZNS has its own in section 11 (§5.1.12). Reference: NAND timing (§A.1.1). Design: NAND media and timing (§3.5).

Parameter	Unit	Valid values	Meaning
pg_rd_lat, pg_wr_lat, blk_er_lat	ns	32-bit	flat page read, page program and block erase times, used when nand_cell_type is 0
nand_cell_type	type	0 flat, 1 SLC, 2 MLC, 3 TLC, 4 QLC; others fall back to 0	built-in per-page-type tables instead of the flat times
pgtype_lat, cell_pages	flag, bits per cell	cell_pages 0 to 5	with nand_cell_type 0, scale the program time by the page's position in its wordline
cmd_addr_lat	ns	>= 0	channel bus command and address phase
pg_xfer_lat	ns	>= 0	channel bus data phase per page; 0 uses ch_xfer_lat
ch_xfer_lat	ns	>= 0	data phase when pg_xfer_lat is 0; also the OCSSD 1.2 transfer time
status_lat	ns	>= 0	channel bus status phase
onfi_mode	mode	-1 (off), 0 to 26	ONFI 6.0 NV-LPDDR4 timing mode; sets every bus phase, refused with the four phase knobs above and with pe_suspend
onfi_mts	MT/s	0 to 100000	with onfi_mode, the page rate instead of the mode's; 0 uses the mode
onfi_spare_bytes	bytes	>= 0; the page on the bus must be even	with onfi_mode, spare bytes moved with each page
onfi_tccs_ns	ns	>= 0	with onfi_mode, tCCS before a data out on a shared channel; 0 charges nothing
onfi_sca	0 or 1	0 (default)	with the ONFI profile, 1 selects the SCA protocol: commands, addresses and status use a separate CA bus
onfi_cache_read	0 or 1	0 (default)	with the ONFI profile, 1 reads the pages of one read command on one LUN as a read cache sequence; refused with mp_read, mapping=dftl and read_cache_mb
trcbsy	ns	>= 0	with onfi_cache_read, tRCBSY of each copy; 0 takes 3000 (Table 4-30 typical); without it, a value warns
onfi_param_page	path	an ONFI 6.0 parameter page image	turns the profile on; gives the mode when onfi_mode is -1, and the spare bytes and address cycles; refused when the page does not match the device
tplebsy	ns	>= 0	busy time between the planes of a multi-plane erase (GC with pls_per_lun > 1)

Parameter	Unit	Valid values	Meaning
pe_suspend, tsusp_ns	flag, ns	tsusp_ns >= 0	let a read suspend a program or erase on its LUN, at a cost of tsusp_ns
trim_lat_ns	ns per range	>= 0; bbssd and CSD, also under FDP	time charged per Dataset Management deallocate range

Interactions:

- The channel bus is modelled only when cmd_addr_lat, pg_xfer_lat (or ch_xfer_lat) or status_lat is non-zero; phases on one channel then run one at a time.
- Vendor command 0xEF codes 3 and 4 change only the flat times: code 3 sets the built-in values, not the configured ones, and code 4 sets them to 0. Neither has an effect while nand_cell_type is set.
- Read time also grows with ecc_step_ns (section 9 (§5.1.10)).
- Tutorial 05 (§5.7) measures each of these.

5.1.8 FTL: mapping, caches and write buffer

Applies to bbssd and CSD. Reference: garbage collection, mapping and caches (§A.1.1). Design: the BlackBox FTL (§3.4).

Parameter	Unit	Valid values	Meaning
mapping	name	page, dftl, hybrid, fast	logical-to-physical scheme: a full page table, a cached page table, BAST or FAST log-block mapping
mapping_cache_mb	MiB	32-bit; 0 picks the built-in size (4 MiB)	with mapping=dftl, the cache of whole translation pages; a miss reads the translation page, and a dirty eviction programs it
read_cache_mb	MiB	0 disables	DRAM read cache; a hit costs DRAM time instead of a NAND read
cache_evict	name	clock, random, lru, arc	read cache eviction policy
hot_cold_sep	bool	acts with page or dftl; ignored under hybrid and fast, which still reserve its line; refused with FDP	write overwrites of mapped pages to separate hot lines
buffer_size	NAND pages, not bytes	>= 0; under FDP each page keeps its placement handle	DRAM write buffer; 0 programs every write directly
buffer_thres_pcent	percent	1 to 100 when buffer_size > 0	fill level at which buffered pages are written to NAND
debug_ftl	bool		print FTL invariant violations and log-block merge counts

Interactions:

- The write buffer models timing; the data lives in the backend. With vwc=1 the guest sees a volatile write cache and feature 06h turns it off. Flush drains the buffer and FUA writes skip it with either setting. With power_loss=on the buffer also holds data, which simulate-power-loss drops.
- hybrid and fast reserve one more line and count merges in log page C0h; hot_cold_sep reserves one more line.
- mapping_cache_mb has no effect except under dftl.

- `mapping=dftl` without FDP keeps its translation pages on flash. Realize keeps lines for them and two forced free lines, so the largest namespace a geometry exposes is smaller than with `page`.
- Unknown names for `mapping`, `gc_policy` and `cache_evict` are refused.

5.1.9 Garbage collection

Applies to bbssd and CSD; KV uses `gc_thres_pcent` only (but see KV (§5.1.14)). Reference: garbage collection, mapping and caches (§A.1.1). Design: garbage collection (§3.4.6).

Parameter	Unit	Valid values	Meaning
`gc_thres_pcent`	percent of lines in use	1 to 100	background GC starts; KV: share of the NAND usable for values
`gc_thres_pcent_high`	percent of lines in use	`gc_thres_pcent` to 100	forced GC inside writes; also sizes the reserve
`gc_policy`	name	`greedy`, `random`, `cost-benefit`, `fifo`, `d-choice`; under FDP a name other than `greedy` needs `gc_strategy=0`	victim line policy; under FDP the reclaim unit policy
`gc_strategy`	number	0 greedy, 1 cost-benefit, 2 random, 4 per-handle; FDP only	victim reclaim unit policy
`fdp_trim_erase_all`	flag	FDP only	a deallocate resets every reclaim unit instead of the given ranges

Interactions:

- Background GC takes only a line with at least 1/8 of its pages invalid; forced GC takes the best line whatever it holds.
- Set `gc_thres_pcent` above the share of the NAND the namespace fills. Below it, background GC never stops and the WAF settles near 8 (tutorial 02 (§5.4.4)).
- The reserve is `(1 - gc_thres_pcent_high / 100) x blks_per_pl` lines, rounded down, plus the write pointers; the namespace must fit in the rest.

5.1.10 Reliability, wear and fault insertion

Applies to bbssd, CSD and KV unless noted. Reference: reliability and wear (§A.1.1). Design: wear, read reclaim and retention refresh (§3.4.10).

Parameter	Unit	Valid values	Meaning
`ecc_step_ns`	ns per tier	0 disables	extra read time per ECC tier: one per 750 erases of the block, one per `ecc_retention_sec` of data age, at most 4
`ecc_retention_sec`	seconds	0 counts wear only	data age that adds one tier
`pd_edge_wls`	wordlines	0 disables; set with `pd_edge_tiers` and `ecc_step_ns`; below the wordlines of a block	bottom wordlines of each block that pay `pd_edge_tiers` on a read
`pd_edge_tiers`	ECC tiers	0 to 4	program disturb tiers for a read of a `pd_edge_wls` wordline
`pd_open_tiers`	ECC tiers	0 to 4; needs `ecc_step_ns`	program disturb tiers for a read of a part-programmed block

Parameter	Unit	Valid values	Meaning
pd_open_age	on/off	off (default); needs ecc_step_ns and ecc_retention_sec	data age counts from the first program of the line
pd_open_read_limit	reads	0 disables; bbssd and CSD, also FDP; not above a non-zero read_reclaim_limit	a part-programmed block read this often gets its line or unit rewritten on a following Write; a frontier line is sealed first, a frontier unit retired with its free pages unprogrammed
pd_open_close_sec	seconds of device time (age_scale)	0 disables; bbssd and CSD, also FDP	a frontier line or unit open this long after its first program is padded and closed at the next request; under FDP the handle gets a fresh unit and event 1h
pe_cycles_rated	P/E cycles	0 takes the rating of nand_cell_type	denominator of SMART Percentage Used
nand_bad_blocks	blocks	capped at the block count	blocks bad from the start; lowers SMART Available Spare
blk_pe_limit	P/E cycles	0 disables; bbssd and CSD, one namespace, page or dftl mapping, one plane per LUN, no buffer, Streams, hot/cold separation or nand_bad_blocks; FDP takes more namespaces and planes, but not spare_lines or fdp_trim_erase_all	erases a block survives; its line (FDP: reclaim unit) then retires while enough remain; else the media goes read-only (eol_read_only), or the block stays in service and sets SMART critical warning bit 2 (eol_read_only=off)
blk_pe_spread	percent	0 to 90	each block's limit varies by up to this much from blk_pe_limit
blk_pe_seed		any	seed of the per-block limits; one seed gives the same limits on every run
spare_lines	lines	below blks_per_pl; needs blk_pe_limit	lines held back; their blocks replace worn-out blocks of the same plane; they hold no data, so the namespace must fit without them
eol_read_only	on/off	on (default); has an effect only with blk_pe_limit; on needs mdts 1 or more and queues x (entries + 1) of at most 65535; under FDP each reclaim group holds the reserve, in units	on: the media goes read-only when no block can replace a worn-out one, and writes then fail with status 182h; reserve lines stay erased for the writes in flight then, a write waits in its queue until the pages it may program fit in the reserve, Write Zeroes (WZSL) is limited to mdts, and Copy (MCL) to mdts or one block of the largest format; off: the version 1 wear model
err_read_unc_ppm	per million	0 disables; bbssd and CSD	reads that fail as Unrecovered Read Error, at a fixed period
err_write_fail_ppm	per million	0 disables; bbssd, CSD and ZNS	writes that fail; a ZNS zone then becomes read only
read_reclaim_limit	reads	0 disables; bbssd and CSD; under FDP a whole reclaim unit	a block read this often since its erase gets its line rewritten on a following write

Parameter	Unit	Valid values	Meaning
retention_limit_sec	seconds	0 disables; bbssd and CSD; under FDP a whole reclaim unit	a read that hits a line filled at least this long ago queues the line, which is rewritten on a following write
thermal_tau_ms, thermal_r, idle_mw, thermal_step_ms	ms, mK per mW, mW, ms	thermal_tau_ms 0 (default) turns the model off; it needs thermal_r; step 1 to 60000, default 100	the SMART temperature follows a first-order model heated by idle_mw and the NAND energy (thermal model (§B.5))
energy_read_nj, energy_prog_nj, energy_erase_nj	nJ per plane operation	0 (default) gives no figure	price the plane reads, programs and erases log page C0h counts; the total is reported in uJ at offset 184
wl_spread	erases	0 disables; the settings blk_pe_limit refuses are refused too	static wear levelling: when the erases of lines in service differ by more than this, the least worn full line moves into the most worn free line, at most a quarter of the host pages; it stops at half the gap
gc_pace	on/off	off (default); the settings blk_pe_limit refuses are refused too	paced collection: the victim is copied in step with host writes, V0 / (pages per line - V0) copies per host page, instead of in one pass; log page C0h offsets 224, 232 and 240 count it
gc_copyback	on/off	off (default); bbssd, CSD and FDP, not KV; refused with gc_pace, pe_suspend, cxl_ssd, namespace_modes, ns_mgmt, shared namespaces and the hybrid and fast mappings	collection moves a page on its die when the destination shares its channel, LUN and plane; the walk takes victim pages in the order of the destination dies; saves the channel data phases. A copyback skips ECC: on a real SSD its bit errors are not corrected and can add up. FEMU models no bit errors, so only timing changes; pages with extra ECC time and refresh moves stay off-chip for that reason. Log page C0h offset 248 counts it
age_scale	factor	1 (default) or more	data ages this many times faster than wall time for retention_limit_sec and ecc_retention_sec; I/O timing and collection order do not change

Interactions: faults come at a fixed period, so a run repeats exactly. Read reclaim and retention refresh act only when the host reads and then writes; their cost appears as write amplification.

5.1.11 Host link and controller firmware

Applies to every NVMe mode. Reference: host link and controller firmware (§A.1.1). Design: host link and firmware CPU (§3.5.13).

Parameter	Unit	Valid values	Meaning
pcie_bandwidth_mbps	MB/s (10^6 bytes)	0 disables	each Read and Write is charged its size at this rate, on one queue per direction
pcie_prop_delay_ns	ns	0 disables	fixed delay added to each Read and Write after its transfer

Parameter	Unit	Valid values	Meaning
fw_cpu_ns	ns per command	0 disables	firmware time per Read, Write and Zone Append on one modelled core; caps the rate at about one command per fw_cpu_ns

5.1.12 ZNS

Reference: ZNS (§A.1.1). Design: ZNS (§4.8).

Parameter	Unit	Valid values	Meaning
zns_num_ch	channels	1 to 128	channels
zns_num_lun	LUNs	>= 1	LUNs per channel
zns_num_plane	planes	1 to 8	planes per LUN; the program unit grows with it
zns_num_blk	blocks	>= 1	blocks per plane
zns_chnls_per_zone	channels	divides zns_num_ch; 0 = all	zone width
zns_luns_per_zone	LUNs	divides zns_num_lun; 0 = all	LUNs of a zone on each of its channels
zns_zone_order	name	ch (default) or lun	axis a zone fills first
zns_zone_cap	bytes	one block to the zone size; 0 = zone size	writable part of each zone
zns_flash_type	type	1 SLC, 2 MLC, 3 TLC, 4 QLC, 5 PLC	cell type; MLC and PLC need the three times below
zns_pg_rd_lat, zns_pg_wr_lat, zns_blk_er_lat	ns	>= 0; 0 = built-in	override the cell type's times
zns_cmd_addr_lat, zns_pg_xfer_lat, zns_status_lat	ns	>= 0	channel bus phases
zns_pe_suspend, zns_tsusp_ns	flag, ns		reads suspend a program or erase on their plane
zns_lun_contention	flag	0 = off, non-zero = on	the planes of one LUN take turns
zns_reset_by_fill	flag	0 = off, non-zero = on	a zone reset erases only programmed blocks
zns_fzr_sec	seconds	0 = off	a written zone not finished this long gets Finish Zone Recommended
zns_frl_sec	seconds	0 = no excursion; needs zns_fzr_sec	FRL: after it the controller finishes an FZR zone (Zone Active Excursion, ZFC)
zns_rzr_sec	seconds	0 = off	a zone Full this long gets Reset Zone Recommended
zns_rrl_sec	seconds	0 = no refresh; needs zns_rzr_sec	RRL: after it the controller refreshes an RZR zone and clears RZR
zns_max_open, zns_max_active	zones	<= zone count, open <= active; 0 = no limit	Maximum Open and Active Resources
zns_num_wc	write caches	<= zone count; 0 picks from zns_max_open	zone write caches
zns_zasl_bs	bytes	power-of-two multiple of 4 KiB; 0 follows mdts	Zone Append size limit
zns_zd_ext_size	bytes	multiple of 64, up to 16320	zone descriptor extension size

Parameter	Unit	Valid values	Meaning
zns_num_conv_zones	zones	capped at the zone count	leading conventional zones; Linux rejects them
zns_zrwa_size, zns_zrwafg_size, zns_zrwa_num	blocks, blocks, zones	all three set or all 0	Zone Random Write Area window, flush granularity and resources
zns_cross_zone_read	bool		allow reads across zone boundaries

Derived quantities (when the divisions are exact):

```
pages per block = namespace size / 16 KiB / (zns_num_ch x zns_num_lun x zns_num_blk)
zone width      = zns_chnls_per_zone, or zns_num_ch when 0
zone LUNs       = zns_luns_per_zone, or zns_num_lun when 0
zone size       = zone width x zone LUNs x zns_num_plane x pages per block x 16 KiB
zone count      = zns_num_ch x zns_num_lun x zns_num_blk / (zone width x zone LUNs x zns_num_plane)
```

Interactions:

- The zone count does not depend on the namespace size; the zone size grows with it. Linux uses a zoned namespace only when the zone size is a power of two, so keep the size and the geometry powers of two.
- lba_index must select a block of 4 KiB or less.
- zns_zrwa_size must be a multiple of zns_zrwafg_size, and the zone capacity a multiple of zns_zrwafg_size.
- The BlackBox geometry and timing properties do not apply to ZNS.

5.1.13 FDP

Set on femu-subsys, which a bbssd controller joins with subsys=. Reference: Flexible Data Placement (§A.1.2). Design: FDP (§4.6).

Parameter	Unit	Valid values	Meaning
fdp	bool		FDP in endurance group 1 for the joining controller
fdp.nruh	handles	1 to fdp.nru, at most 16366; required with fdp=on	reclaim unit handles, the placement identifiers the host can name
fdp.nru	reclaim units	fdp.nruh to 65536	reclaim units per group; bbssd uses at most one per block index and needs 2 x fdp.nruh + 1 in each group
fdp.nrg	groups	1 to 128; with bbssd, a divisor of nchs, or a multiple of nchs that divides nchs x luns_per_ch	reclaim groups; with bbssd each group is a set of dies (reclaim groups (§4.5.3))
fdp.runs	bytes	0, or with bbssd one line's size divided by fdp.nrg	reclaim unit size; 0 lets FEMU choose
fdp.isolation_mode	number	0, or any other value	0: every handle Persistently Isolated; otherwise the last one Initially Isolated

Interactions:

- The subsystem takes one controller with one namespace, and must come first on the command line.
- The controller's gc_strategy, gc_policy, fdp_trim_erase_all, read_reclaim_limit, retention_limit_sec, ecc_retention_sec and trim_lat_ns apply. A gc_policy other than greedy together

with a gc_strategy other than 0, mapping=hybrid or fast, hot_cold_sep, meta, streams and ns_mgmt are refused.
- KV is refused under FDP; NoSSD, ZNS and OCSSD report FDP but ignore it for placement.

5.1.14 OCSSD, CSD and KV

OCSSD

Reference: OCSSD (§A.1.1).

Parameter	Unit	Valid values	Meaning
lver	version	1 (1.2) or 2 (2.0)	Open-Channel version
flash_type	type	1 SLC, 2 MLC, 3 TLC, 4 QLC	cell type for the built-in timing tables
lnum_ch, lnum_lun	count	lnum_ch 1 to 32, lnum_ch x lnum_lun <= 128	channels (2.0 groups) and LUNs (parallel units)
lnum_pln	planes	> 0; 1.2: 1, 2 or 4	planes per LUN
lpgs_per_blk, lsecs_per_pg	count	> 0; 1.2: pages <= 512	pages per block, sectors per page
lsec_size, lmetasize, lmax_sec_per_rq	bytes, bytes, sectors	1.2 only	sector size, out-of-band bytes, most sectors per vector command
oc12_channel_timing	bool	1.2 only	charge channel transfer per page, ch_xfer_lat or the flash_type value
learly_reset	flag	2.0 only	report the early reset capability

OCSSD takes the whole controller: one namespace, no namespace_modes neighbours.

CSD

Reference: CSD (§A.1.1). CSD builds the bbssd FTL, so sections 5 to 9 apply.

Parameter	Unit	Valid values	Meaning
fdm_size	MiB	> 0, required	functional data memory
nr_cu	compute units	1 to 64	programs run on the first free unit
csf_runtime_scale	multiplier	> 0	host run time multiplier for programs that declare no run time
csd_program_dir	host directory		where shared-object and uBPF programs load from; unset allows only the built-in program type
csd_slm_size	MiB	0 to 1024; 0 adds none	adds a memory namespace (SLM 1.2) of this size
csd_cp	bool	needs csd_slm_size	adds a compute namespace (CP 1.2)
csd_legacy_opcodes	bool	needs a CSD namespace	old CEMU admin opcodes 21h to 25h instead of C5h to CDh
csd_pause	bool	needs a CSD namespace, KVM on x86	stop the guest and its clocks while a program runs
csd_pause_max_ms	ms	1 to 60000	the longest such stop (default 1000)

csd_slm_size and csd_cp refuse ns_mgmt, power_loss and a CXL link. csd_pause refuses namespaces of other modes than bbssd and CSD, and shared namespaces (CSD (§4.13.4)). With subsys, the controller must

be the only one in its subsystem. Guide: CP and SLM (§4.22).

KV

KV uses the geometry (section 5), the NAND timing (section 6), and `gc_thres_pcent` as the share of the NAND usable for values. The rest of the NAND is free space: value GC starts when the free lines fall to it. The FTL, mapping, cache, write buffer and other GC properties have no effect on KV, but KV runs the same validation as bbssd, so an out-of-range or unknown value is still refused. The GC capacity check of bbssd is not made: a namespace larger than the NAND starts, with its value space clamped. Design: KV (§4.12).

Parameter	Unit	Valid values	Meaning
`kv_timing`	name	`media`, `model`	NAND time for every command, or the workload-mix model for Store and Retrieve
`kv_model_coeffs`	36 numbers	finite	the model fit, intercept first
`kv_index`	name	`hash`, `lsm`	the key index: a DRAM hash, or an LSM tree with flash meta pages
`kv_lsm_memtable_keys`	records	16 to 65536, `kv_index=lsm`	memtable size before a flush
`kv_lsm_size_ratio`	ratio	2 to 64, `kv_index=lsm`	capacity of a level over the one above
`kv_lsm_pinned_levels`	levels	0 to 24, `kv_index=lsm`	top levels held in DRAM, with no flash I/O
`kv_pack`	bool	`on` (default), `off`	values whose record fits a page share pages through an open page in DRAM; `off`: each value takes whole pages

Interactions:

- An LSM knob other than its default needs `kv_index=lsm`, and `kv_index=lsm` needs a KV namespace.
- With `kv_timing=model`, the tree still loads the NAND; only the Store and Retrieve times come from the model (LSM-tree key index (§4.23.7)).
- `kv_pack=off` needs a KV namespace. With `vwc=1`, Flush programs the open page of packed values; with WCE cleared, each Store programs it (KV value packing (§4.24.4)).

5.1.15 CXL SSD

`femu-cxl-ssd` is a separate device below a CXL root port. Reference: `femu-cxl-ssd` (§A.1.3) and its runtime properties (§A.2.2). Design: CXL SSD (§4.19).

Parameter	Unit	Valid values	Meaning
`volatile-memdev`	memory backend id	required; a non-zero multiple of 256 MiB, at most 120 GiB	the backend that holds the data; its size is the device capacity
`cache-pages`	4 KiB pages	0, or up to the media page count and divisible by `cache-ways`	DRAM page cache; 0 sends every access to the media
`cache-ways`	ways	non-zero, divides `cache-pages`	set associativity; 1 is direct mapped; changeable at run time
`cache-policy`	name	`fifo`, `lifo`, `clock`, `s3-fifo`	replacement within a set
`prefetch-degree`, `prefetch-stride`	pages	up to the media page count	pages inserted after a miss, and their distance; changeable at run time

Parameter	Unit	Valid values	Meaning
ftl	bool		off charges no media time and cannot be linked to an NVMe controller
channels, luns-per-channel	count	1 to 4096, 1 to 128	NAND channels and LUNs (one plane each)
pages-per-block, blocks-per-plane	count	1 to 65536; blocks 2 to 65536, or 0 to size it	NAND blocks; 0 leaves spare room for GC; an explicit value must leave the media plus 7% beyond the lines gc-threshold-high keeps free and two more
read-ns, program-ns, erase-ns, channel-ns	ns	at most one second	NAND times
gc-threshold, gc-threshold-high	percent	1 to 100, high >= low	GC watermarks
der	name	off, memslot, cylon	direct mapping of cached pages into the guest
der-replace-rate	per second	0 disables	memslot alias replacements a hot page may cause
cylon-kernel-ack	bool	must be on with der=cylon	states that the host runs a fixed Cylon kernel
concurrent-misses	on, off, auto		misses to different pages wait for the media together
cylon-first-touch-program, cylon-free-writeback	bool		media rules of the published Cylon experiments
cca	bool		the caching API on BAR 5
lsa-control	bool	not with an lsa backend	experiment commands through Get LSA; trusted guests only
log-dir, tracefs-dir, log-limit	path, path, bytes	log-limit 0 opens no I/O log	where the control channel writes, and how much

Interactions:

- The machine needs cxl=on, a pxb-cxl host bridge, a cxl-rp root port and a cxl-fmw window at least as large as the backend; run-cxlssd.sh builds them.
- der=memslot needs KVM. der=cylon needs a Cylon host kernel and a shared, preallocated hugetlbfs backend; without them, or when the memory cannot be locked, it falls back to MMIO with a warning. It also needs -machine smm=off, which the device does not check (der=cylon (§4.18.7)).
- A femu controller with cxl_ssd=<id> serves the same medium as an NVMe namespace. It needs femu_mode=1, one namespace, ftl=on on the CXL device, devsz_mb unset or equal to the medium's size, and none of ns_mgmt, subsys, streams, power_loss, buffer_size, op_pcent, meta, pi or dps. The medium's geometry and timing apply, not the controller's (CXL NVMe link (§4.21)).

5.1.16 Accepted for compatibility, no effect

These properties exist so that old command lines still start. Setting one to anything but its default prints a warning at realize.

Device	Properties
femu	serial, ms, ms_max, dlfeat, nr_thread, time_slice, context_switch_time

nr_thread is still refused at 0 by CSD. Identify Controller reports a serial number FEMU generates, whatever serial says.

5.1.17 Worked configurations

Each configuration below is started by the documentation checks, which also write and read one block through it.

A TLC drive with a channel bus and read suspend

A 4 GiB namespace over the default geometry, TLC page-type timing, a 10 us page transfer and 2 us command and status phases on each channel, and reads that suspend a program or erase for 15 us of overhead:

```
-device femu,femu_mode=1,devsz_mb=4096,nand_cell_type=3,cmd_addr_lat=2000,pg_xfer_lat=10000,status_lat=2000,pe_suspend=1,tsusp_ns=15000
```

DFTL with caches and a write buffer

A DFTL table with 8 MiB cached, a 64 MiB read cache with LRU eviction, and a write buffer of 4096 NAND pages that the guest sees as a volatile write cache:

```
-device femu,femu_mode=1,devsz_mb=2048,mapping=dftl,mapping_cache_mb=8,read_cache_mb=64,cache_evict=lru,buffer_size=4096,buffer_thres_pcent=75,vwc=1
```

A GC study drive

512 MiB of NAND exposed as NAND / 1.25, background and forced GC both starting at 95% of lines in use, cost-benefit victims and hot/cold separation, as in tutorial 02 (§5.4):

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,gc_thres_pcent=95,gc_thres_pcent_high=95,gc_policy=cost-benefit,hot_cold_sep=on
```

A ZNS drive with 4 KiB blocks, limits and ZRWA

64 zones of 64 MiB, 4 KiB logical blocks, 8 open and 16 active zones, and a ZRWA of 64 blocks flushed in units of 8 on up to 4 zones at once:

```
-device femu,femu_mode=3,devsz_mb=4096,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=128,lba_index=
    3,zns_max_open=8,zns_max_active=16,zns_zrwa_size=64,zns_zrwafg_size=8,zns_zrwa_num=4
```

FDP with eight handles

Eight placement handles over 256 lines (blks_per_pl=256), with cost-benefit reclaim unit selection:

```
-device femu-subsys,id=fdp0,fdp=on,fdp.nruh=8 -device femu,femu_mode=1,devsz_mb=2048,blks_per_pl=256,subsys=fdp0,gc_strategy=1
```

Mixed namespaces on a modelled link

A 3 GiB BlackBox namespace and a 1 GiB ZNS namespace on one controller, with a 3500 MB/s host link, 1 us of propagation delay and 2 us of firmware time per command:

```
-device femu,femu_mode=1,devsz_mb=4096,namespaces=2,namespace_sizes=3G,,1G,namespace_modes=bbssd,,znssd,pcie_
    bandwidth_mbps=3500,pcie_prop_delay_ns=1000,fw_cpu_ns=2000
```

Wear and faults

A drive rated for 3000 program/erase cycles, with 100 bad blocks, an ECC step of 5 us, one read in 100,000 uncorrectable and one write in 1,000,000 failed:

```
-device femu,femu_mode=1,devsz_mb=1024,pe_cycles_rated=3000,nand_bad_blocks=100,ecc_step_ns=5000,err_read_unc_ppm=10,err_write_fail_ppm=1
```

Namespace management

A controller that starts with one bbssd namespace and lets the guest allocate up to four bbssd namespaces in all, the boot one included, so three more:

```
-device femu,femu_mode=1,devsz_mb=2048,ns_mgmt=on,bbssd_ns_limit=4
```

A CXL SSD and an NVMe view of it

A 256 MiB `femu-cxl-ssd` with a 1024-page, 4-way `s3-fifo` cache below a CXL host bridge, and a `femu` controller that serves the same medium as an NVMe namespace:

```
./qemu-system-x86_64 -machine q35,cxl=on \
    -object memory-backend-ram,id=cxlmem,size=256M \
    -device pxb-cxl,id=cxl.0,bus=pcie.0,bus_nr=52 \
    -device cxl-rp,id=cxl-rp0,bus=cxl.0,chassis=0,slot=0 \
    -device femu-cxl-ssd,id=cxlssd,bus=cxl-rp0,volatile-memdev=cxlmem,cache-pages=1024,cache-ways=
        4,cache-policy=s3-fifo
        \
    -device femu,id=nvme0,bus=pcie.0,femu_mode=1,cxl_ssd=cxlssd \
    -M cxl-fmw.0.targets.0=cxl.0,cxl-fmw.0.size=256M
```

5.1.18 Related pages

- Device property reference (§A.1) and runtime properties (§A.2), generated from the binary
- Tutorials (§5.2)
- Design (§3.2)
- Log pages and counters (§B)

5.2 Tutorials

Hands-on walkthroughs. Each one starts a guest with an emulated device, runs a workload inside it, and reads what the device reports. Every tutorial stands on its own, but they build on each other in this order.

Tutorial	You learn to	Device
01 Your first SSD (§5.3)	boot a BlackBox SSD from your own QEMU command line, measure read and write latency, read the write amplification factor (WAF)	BBSSD, 2 GiB of NAND
02 Garbage collection and WAF (§5.4)	provoke garbage collection (GC), measure the WAF of one run, and see how over-provisioning, the GC threshold, the victim policy and hot/cold separation change it	BBSSD
03 Zoned namespaces (§5.5)	read the zone report, move zones through their states, use Zone Append, fio's zoned mode and zonefs	ZNS
04 Flexible Data Placement (§5.6)	write through placement identifiers, read the FDP log pages, and measure what placement does to the WAF	BBSSD with FDP
05 Latency tuning (§5.7)	change NAND times, the cell type, the channel bus, the host link and the firmware cost, and measure each with fio	BBSSD
06 Several namespaces (§5.8)	run a BlackBox, a ZNS, a NoSSD and a KV namespace on one controller	mixed
07 Key-value SSD (§5.9)	store, retrieve, list and delete values with nvme-cli	KV

Tutorial	You learn to	Device
08 CXL SSD as memory (§5.10)	start a `femu-cxl-ssd`, find it in the guest, use it as a DAX device or a NUMA node, and read its counters	CXL Type-3 SSD
09 Configuration files (§5.11)	describe a device in a file and expand it with `ssd-config.sh` instead of writing a long `-device` line	any

5.2.1 Before you start

You need a built FEMU and a guest image. The quick start (§2.4) does both in about five minutes of work: `femu-compile.sh` in `build-femu/` and `make-guest-image.sh` for the image.

The tutorials use three shell variables on the host. Set them once in the terminal you start QEMU from and in the one you use to reach the guest:

```
cd build-femu                       # the directory femu-compile.sh built in
export IMGDIR=$HOME/images          # where make-guest-image.sh put the image
export OSIMGF=$IMGDIR/u20s.qcow2    # the guest image
export SSH_PORT=8080                # host port forwarded to the guest's SSH
```

`run-guest-ssh.sh` reads `IMGDIR` and `SSH_PORT`, so with these set, `./run-guest-ssh.sh` opens a shell in the guest and `./run-guest-ssh.sh CMD` runs one command there.

5.2.2 How to read the tutorials

- Blocks marked as a FEMU example hold a QEMU command line or `-device` options. The documentation checks start each of them under QEMU's `qtest` accelerator, so they are known to be accepted by the current code. The checks do not boot a guest.
- `sh` blocks run inside the guest unless the text says "on the host". Run them in the shell `./run-guest-ssh.sh` gives you.
- `text` blocks show output. Unless a block says it is illustrative, it was captured from a real run: FEMU at commit 1a03ded0f, the Ubuntu 24.04 guest from `make-guest-image.sh` (Linux 6.8, nvme-cli 2.8, fio 3.36), on a 20-core host. Latencies and throughput depend on the host. Counter values such as the WAF depend only on the workload, and fio's random offsets change between runs, so expect them within a few percent of the values shown.
- The QEMU command lines run without `sudo`. That works when your user is in the `kvm` group; otherwise put `sudo` in front, as the `run-*.sh` launchers do. Without root, FEMU usually cannot lock the device memory (`ulimit -l` is too small) and says so at start; the device works, but host paging can add jitter to latency, so use `sudo` or raise the limit for latency measurements.
- Each emulated device lives in host memory: plan for the device size plus the guest's `-m` in free host RAM.

5.2.3 Where to go next

- Measuring (§6.1) and performance tuning (§5.12) explain how to get numbers that repeat.
- The parameter manual (§5.1) explains every group of device parameters and how they interact.
- The design pages (§3.2) explain what happens inside the device.

5.3 Tutorial 01: your first SSD

You start a guest with a small BlackBox SSD (BBSSD) from a QEMU command line you write yourself, look at the device the guest sees, measure read and write latency with fio, and read the write amplification factor (WAF) from the device. It takes about ten minutes.

The quick start (§2.4) does something similar with `run-blackbox.sh`, which builds a 12 GiB device. This tutorial uses a 2 GiB device so that the whole drive can be filled in seconds, and it explains each option.

You need: a built FEMU and guest image, the variables from Before you start (§5.2.1), and about 7 GiB of free host memory (2 GiB for the device, 4 GiB for the guest).

5.3.1 Choose the geometry

A BBSSD is NAND flash organised as channels, LUNs (dies) per channel, planes per LUN, blocks per plane and pages per block. This tutorial uses 4 channels of 4 LUNs, one plane, 128 blocks per plane and the default 256 pages of 4 KiB per block:

```
NAND     = 4 channels x 4 LUNs x 1 plane x 128 blocks x 256 pages x 4 KiB = 2 GiB
line     = one block on every LUN = 16 blocks = 16 MiB; there are 128 lines
```

A line (a superblock) is the unit garbage collection (GC) reclaims. The guest must not see all of the NAND: GC needs spare room to move data into. `op_pcent=25` exposes `NAND / 1.25`, about 1.6 GiB, and keeps the rest as spare. With `op_pcent` set, `devsz_mb` is ignored and the device is sized from the geometry.

5.3.2 Start the guest (on the host)

From `build-femu/`:

```
./qemu-system-x86_64 -name femu-tut01,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    -device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25 \
    -nographic
```

Everything except the last `-device` line is an ordinary guest: KVM, 4 vCPUs, 4 GiB of RAM, the guest image on a virtio-scsi disk, and user networking that forwards `SSH_PORT` to the guest's SSH port. `debug-threads=on` names FEMU's threads, which performance tuning (§5.12.1) needs. The FEMU device:

Option	Meaning
`femu_mode=1`	BlackBox mode: the device runs its own FTL, GC and NAND timing
`nchs=4,luns_per_ch=4,blks_per_pl=128`	the geometry from step 1; the page size and pages per block keep their defaults
`op_pcent=25`	25% over-provisioning

Wait for the `femu-guest login:` prompt, about 30 seconds. Leave this terminal running. To stop QEMU, press `Ctrl-a` then `x`.

5.3.3 Look at the device

In a second terminal, open a shell in the guest with `./run-guest-ssh.sh`, then:

```
sudo nvme list
```

```
Node             Generic          SN               Model                                Namespace  Usage                    Format           FW Rev
---------------- ---------------- ---------------- ------------------------------------ ---------- ------------------------ ---------------- --------
/dev/nvme0n1     /dev/ng0n1       vSSD0            FEMU BlackBox-SSD Controller         0x1         1.72  GB /   1.72  GB    512   B +  0 B   1.0
```

1.72 GB is 2 GiB / 1.25. The guest's own disk is `/dev/sda`. The namespace uses 512-byte blocks; `cat /sys/block/nvme0n1/size` prints `3355443`, the size in 512-byte sectors.

5.3.4 Read before writing: no NAND time

Measure 4 KiB random reads at queue depth 1 on the empty drive:

```
sudo fio --name=rr0 --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randread --bs=4k --iodepth=1 --runtime=10 --time_based
```

The median completion latency (`clat` 50.00th) is about 8 us. A page that was never written has no mapping, so the FTL charges it no NAND read: what you see is the guest's and FEMU's own overhead. A read benchmark on a fresh device measures nothing, so always write the range first.

5.3.5 Fill the drive

Write the whole namespace once, sequentially:

```
sudo fio --name=fill --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=write --bs=128k --iodepth=16
```

```
  write: IOPS=2499, BW=312MiB/s (328MB/s)(1638MiB/5244msec); 0 zone resets
```

5.3.6 Read the WAF

The vendor log page C0h holds FEMU's media counters. The first 4 bytes are the WAF times 1000. The three 8-byte counters from byte 8 are the pages the host wrote, the pages GC moved, and the pages programmed for the host (log pages and counters (§B.1)):

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
```

```
       1000
               419424                    0               419424
```

The WAF is `(pages programmed + pages GC moved) x 1000 / pages the host wrote`. 1000 means 1.000: each 4 KiB page the host wrote was programmed once, and GC has moved nothing. 419424 pages are the 13107 whole 128 KiB blocks fio wrote.

5.3.7 Measure written data

Now read the written data at queue depth 1:

```
sudo fio --name=rr --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randread --bs=4k --iodepth=1 --runtime=15 --time_based
```

```
  clat percentiles (usec):
   |  1.00th=[   44],  5.00th=[   45], 10.00th=[   45], 20.00th=[   45],
   | 30.00th=[   45], 40.00th=[   45], 50.00th=[   46], 60.00th=[   47],
   | 70.00th=[   48], 80.00th=[   49], 90.00th=[   49], 95.00th=[   50],
   | 99.00th=[   58], 99.50th=[   66], 99.90th=[  116], 99.95th=[  141],
```

The median is 46 us: the default NAND page read time `pg_rd_lat` of 40 us plus about 6 us of guest and poller overhead. Then random writes:

```
sudo fio --name=rw --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=1 --runtime=15 --time_based
```

```
clat percentiles (usec):
 |  1.00th=[  204],  5.00th=[  206], 10.00th=[  206], 20.00th=[  208],
 | 30.00th=[  208], 40.00th=[  208], 50.00th=[  208], 60.00th=[  210],
 | 70.00th=[  210], 80.00th=[  212], 90.00th=[  215], 95.00th=[  219],
 | 99.00th=[  269], 99.50th=[  297], 99.90th=[54789], 99.95th=[55313],
```

The median is 208 us, the program time `pg_wr_lat` of 200 us plus overhead. The 99.9th percentile is 55 ms. After the fill, more than 75% of the lines are in use, so GC is armed. Every overwrite leaves an invalid page behind, and once a line has 1/8 of its pages invalid, GC reads its valid pages, programs them elsewhere and erases it. This run gets there towards its end, and a write that needs a LUN GC is busy with waits for it. That is why GC shows in the slowest 0.1% and not in the median. Read the counters again:

```
       1477
               472826              225792              472826
```

GC has moved 225792 pages for 472826 host pages: WAF 1.477.

5.3.8 Shut down

```
sudo poweroff
```

QEMU exits in the first terminal. The data on the emulated SSD is gone: FEMU keeps it only in host memory. A guest reboot keeps it.

5.3.9 What you learned

- A FEMU SSD is one `-device femu` option on an ordinary QEMU command line.
- `op_pcent` sets the spare area directly; the geometry sets the NAND size.
- Unwritten pages cost no NAND time; write before you read.
- Queue depth 1 latency is the NAND time plus a few microseconds of overhead; GC shows up in the tail.
- Log page C0h gives the WAF and the page counters since the device started.

5.3.10 Next

- Tutorial 02 (§5.4) makes GC run on purpose and changes what it does.
- The BlackBox FTL (§3.4) explains lines, write pointers and GC; NAND media and timing (§3.5) explains where the 40 and 200 us come from.
- BlackBox SSD (§4.4) lists every option of the mode.

5.4 Tutorial 02: garbage collection and write amplification

You make garbage collection (GC) run on a BlackBox SSD, measure the write amplification factor (WAF) of one run from the counters in log page C0h, and then change four things and watch the WAF move: the GC threshold, the over-provisioning, the victim policy, and hot/cold separation. Each measurement takes under a minute.

You need: tutorial 01 (§5.3) done once, the variables from Before you start (§5.2.1), and about 7 GiB of free host memory.

5.4.1 Background

The FTL writes every host page to a fresh NAND page and marks the old copy invalid. A line (one block on every LUN) fills up with a mix of valid and invalid pages. GC picks a victim line, copies its valid pages elsewhere and erases it. Every copy is a NAND program the host did not ask for, so

```
WAF = (pages programmed for the host + pages GC copied) / pages the host wrote
```

A WAF of 1 means GC copied nothing. The C0h counters hold the three page counts since the device started. The BlackBox FTL (§3.4.6) describes GC in full; this tutorial needs two facts from it:

- **Background GC** runs after a request once the share of lines in use reaches `gc_thres_pcent`. It collects one line at a time, and only a line with at least 1/8 of its pages invalid.
- **Forced GC** runs inside a write once the share reaches `gc_thres_pcent_high`. It takes the policy's victim however few invalid pages it holds, and repeats until enough lines are free.

5.4.2 The device

Every run in this tutorial uses the drive from tutorial 01, 2 GiB of NAND in 128 lines of 16 MiB, and changes one or two options:

```
-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25
```

Start QEMU with the command line of tutorial 01, step 2 (§5.3.2), with this `-device` line in place of its own. You restart QEMU for every configuration: the counters, the mapping and the NAND state start from zero only when QEMU starts.

5.4.3 The measurement script

In the guest, save this as `waf-run.sh`:

```
cat > waf-run.sh <<'EOF'
# Usage: bash waf-run.sh [extra fio options]
c0() {
    sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b |
        od -An -t u8 -j 8 -N 24 -w24
}
# Optional: GC and NAND operations take no time, so the runs finish in
# seconds. The page counts do not depend on time.
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=2 >/dev/null
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=4 >/dev/null
# 1. Fill the whole drive once.
sudo fio --name=fill --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=write --bs=128k --iodepth=16 >/dev/null
# 2. Age it: random overwrites until GC is in a steady state.
sudo fio --name=age --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=32 --io_size=4G --randrepeat=0 "$@" >/dev/null
# 3. Measure one run as the difference of two counter reads.
c0 > before.txt
sudo fio --name=run --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=32 --io_size=4G --randrepeat=0 "$@" >/dev/null
```

```
c0 > after.txt
paste before.txt after.txt | awk '{ h = $4 - $1; g = $5 - $2; n = $6 - $3;
    printf "host %d  gc %d  nand %d  WAF %.3f\n", h, g, n, (n + g) / h }'
EOF
```

Three things in it matter:

- **Steady state.** The first pass over a fresh drive copies nothing, so its WAF is 1. The script fills the drive, then ages it with 4 GiB of random writes (about 2.5 times its size) before it measures.
- **Differences, not totals.** The counters count since QEMU started. The script reads them before and after the measured run and divides the differences.
- **Time switched off.** Vendor command 0xEF with code 2 makes GC take no time and code 4 sets the flat NAND times to 0. GC still copies the same pages, so the WAF is unchanged, and a run takes seconds instead of minutes. Code 3 would restore the built-in times, not the ones on your command line, so restart QEMU before you measure latency again (run-time controls (§3.4.13)). Leave the two lines out if you want the latency as well.

Run it:

```
bash waf-run.sh
```

```
host 1048576  gc 7357952  nand 1048576  WAF 8.017
```

The measured run wrote 1048576 pages (4 GiB) and GC copied 7.36 million: eight NAND programs for each host write. That is far more than an SSD with 25% spare should need. The next step shows why.

5.4.4 The GC threshold

`gc_thres_pcent` defaults to 75. This drive exposes 1 / 1.25 = 80% of its NAND, so once it is full, more than 75% of its lines always hold data and background GC never stops. It collects any line that is at least 1/8 invalid, so it takes lines that are still 7/8 valid: it copies 7 valid pages for each invalid page it frees. Seven GC copies plus the host's own program per page written is the WAF of 8.

Raise the threshold above the fill level. Restart QEMU for each line, then run `bash waf-run.sh`:

```
-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=90

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95
```

`gc_thres_pcent`	WAF
75 (default)	8.017
90	4.590
95	3.273

At 95, `gc_thres_pcent` equals `gc_thres_pcent_high` in this configuration, so background GC starts only where forced GC does. By then the lines hold more invalid pages, and each collection frees more space per copy. Set `gc_thres_pcent` above the share of the NAND your namespace fills, or the background GC rate is what you measure.

5.4.5 Over-provisioning

Keep `gc_thres_pcent=90` and change `op_pcent`:

```
-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=7,gc_thres_pcent=90

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=15,gc_thres_pcent=90

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=50,gc_thres_pcent=90

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=100,gc_thres_pcent=90
```

op_pcent	Share of NAND exposed	WAF
7	93%	43.945
15	87%	8.000
25	80%	4.590
50	67%	1.970
100	50%	1.028

Read the table this way:

- At 15% the namespace fills 87% of the NAND. With the open lines and the invalid pages on top of that, the share of lines in use stays at the 90% threshold, background GC keeps running, and it is the case of step 3 again: WAF 8.
- At 7% almost no line is free, forced GC runs inside most writes, and it takes victims that are nearly all valid. Forced GC has no 1/8 filter, so the WAF has no ceiling: 44 programs per host write.
- From 25% up, more spare means emptier victims and a WAF that falls towards 1.

FEMU refuses a namespace that leaves GC too few lines at start-up (the reserve (§3.4.12)); 7% is close to the smallest this geometry accepts.

5.4.6 The victim policy

Go back to `op_pcent=25` with `gc_thres_pcent=95`, and try each `gc_policy`:

```
-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=greedy

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=fifo

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=random

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=cost-benefit

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=d-choice
```

Run the uniform workload (`bash waf-run.sh`) and a skewed one, where a few pages take most of the writes:

```
bash waf-run.sh --random_distribution=zipf:1.2
```

gc_policy	WAF, uniform	WAF, zipf 1.2
greedy	3.277	5.961
fifo	3.277	6.052
random	5.087	6.122
cost-benefit	3.269	5.798
d-choice	3.660	5.898

With uniform writes, every line ages the same way, so the line with the fewest valid pages is also the oldest,

and `greedy`, `fifo` and `cost-benefit` choose alike. `random` and `d-choice` (best of 4 random lines) pick worse victims. With skewed writes the WAF is higher for every policy: hot and cold pages share lines, and each collection copies cold pages that will not change again. The policy cannot fix that; separating the data can.

5.4.7 Hot/cold separation

`hot_cold_sep=on` writes overwrites (pages that already had a mapping) to their own lines, so hot pages are invalidated together and cold pages stay together (hot/cold separation (§3.4.5)):

```
-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,hot_cold_sep=on

-device femu,femu_mode=1,nchs=4,luns_per_ch=4,blks_per_pl=128,op_pcent=25,gc_thres_pcent=95,gc_policy=cost-benefit,hot_cold_sep=on
```

Configuration, zipf 1.2	WAF
`greedy`	5.961
`greedy`, `hot_cold_sep=on`	2.394
`cost-benefit`, `hot_cold_sep=on`	1.646

Separation more than halves the WAF. With hot and cold data in different lines, the age term of `cost-benefit` now pays off: it leaves young hot lines alone until more of their pages are invalid.

5.4.8 What you learned

- Measure the WAF of a run as the ratio of counter differences, after the drive is full and aged.
- Background GC with a threshold below the fill level runs all the time and holds the WAF near 8; set `gc_thres_pcent` above the share of NAND you expose.
- Spare capacity is the strongest lever. Below a few percent, forced GC has no limit.
- Victim policies differ little under uniform writes; under skew, separate hot from cold data first.

5.4.9 Next

- Tutorial 04 (§5.6) lets the host do the separating, with Flexible Data Placement.
- Tutorial 05 (§5.7) measures latency instead of page counts.
- Measuring (§6.1.2) covers the counters and repeatable runs in more depth.

5.5 Tutorial 03: zoned namespaces

You start a guest with a Zoned Namespace (ZNS) SSD, read its zone report, move one zone through every state by hand, append data and let the device pick the address, hit the open-zone limit on purpose, then write with fio's zoned mode and mount zonefs. It takes about fifteen minutes.

You need: the variables from Before you start (§5.2.1) and about 5 GiB of free host memory. The guest image from `make-guest-image.sh` has a kernel with ZNS support (Linux 6.8), nvme-cli, `blkzone` and fio. The zonefs step installs two more packages, so the guest needs network access.

5.5.1 Background

A zoned namespace is divided into zones. Each zone has a write pointer: the host writes the zone only at its write pointer (or with Zone Append, where the device picks the address), and must reset the zone before it writes it again. The device runs no garbage collection; reclaiming space is the host's job. Each zone is in one state, and these are the moves you make in this tutorial:

From	Event	To
Empty or Closed	a write or an append	Implicitly Opened
Empty, Implicitly Opened or Closed	Open Zone	Explicitly Opened
Implicitly or Explicitly Opened	Close Zone	Closed
Empty, Opened or Closed	Finish Zone, or the write pointer reaches the zone capacity	Full
Opened, Closed or Full	Reset Zone	Empty

Opened zones count against Maximum Open Resources, and opened plus closed zones against Maximum Active Resources. The ZNS design page (§4.8.5) has the full state machine.

5.5.2 Start the guest (on the host)

```
./qemu-system-x86_64 -name femu-tut03,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    -device femu,femu_mode=3,devsz_mb=1024,zns_max_open=4,zns_max_active=6 \
    -nographic
```

Option	Meaning
`femu_mode=3`	ZNS
`devsz_mb=1024`	a 1 GiB namespace
`zns_max_open=4`, `zns_max_active=6`	at most 4 open zones and 6 open or closed ones; 0, the default, means no limit

With the default ZNS geometry (2 channels, 4 LUNs, 2 planes, 32 blocks) the namespace has 16 zones; their size grows with the namespace, so 1 GiB gives 64 MiB zones (zone size (§4.7.3)). The default cell type is QLC.

5.5.3 Check that Linux sees a zoned device

Open a shell in the guest with `./run-guest-ssh.sh`, then:

```
sudo nvme list
cd /sys/block/nvme0n1/queue
cat zoned nr_zones chunk_sectors max_open_zones max_active_zones
cd
```

```
/dev/nvme0n1          /dev/ng0n1            vZNSSD0              FEMU ZMS-SSD Controller [by Misao]       0x1          1.07  GB /   1.07  GB    512   B +  0 B   1.0
host-managed
16
131072
4
6
```

`chunk_sectors` is the zone size in 512-byte sectors: 131072 x 512 bytes = 64 MiB. The ZNS Identify Namespace data reports the limits 0's based:

```
sudo nvme zns id-ns /dev/nvme0n1 | grep -E '^(mar|mor) '
```

```
mar     : 0x5
mor     : 0x3
```

5.5.4 Read the zone report

```
sudo nvme zns report-zones /dev/nvme0n1 -d 3
sudo blkzone report -c 3 /dev/nvme0n1
```

```
nr_zones: 16
SLBA: 0         WP: 0         Cap: 0x20000    State: 0x10 Type: 0x2  Attrs: 0    AttrsInfo: 0
SLBA: 0x20000   WP: 0x20000   Cap: 0x20000    State: 0x10 Type: 0x2  Attrs: 0    AttrsInfo: 0
SLBA: 0x40000   WP: 0x40000   Cap: 0x20000    State: 0x10 Type: 0x2  Attrs: 0    AttrsInfo: 0
  start: 0x000000000, len 0x020000, cap 0x020000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000020000, len 0x020000, cap 0x020000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000040000, len 0x020000, cap 0x020000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
```

nvme-cli prints the write pointer as an absolute LBA, `blkzone` relative to the zone start. The two tools name the states differently:

State	nvme-cli `State`	`blkzone` zcond
Empty	0x10	1 (em)
Implicitly Opened	0x20	2 (oi)
Explicitly Opened	0x30	3 (oe)
Closed	0x40	4 (cl)
Read Only	0xd0	13 (ro)
Full	0xe0	14 (fu)
Offline	0xf0	15 (of)

Every zone is sequential-write-required (type 2) and starts Empty.

5.5.5 Append, then move a zone through its states

Append 4 KiB to zone 0 twice. The device writes at the write pointer and reports where the data went:

```
head -c 4096 /dev/urandom > data.bin
sudo nvme zns zone-append /dev/nvme0n1 -s 0 -z 4096 -d data.bin
sudo nvme zns zone-append /dev/nvme0n1 -s 0 -z 4096 -d data.bin
sudo nvme zns report-zones /dev/nvme0n1 -d 1
```

```
Success appended data to LBA 0
Success appended data to LBA 8
nr_zones: 16
SLBA: 0          WP: 0x10       Cap: 0x20000    State: 0x20 Type: 0x2  Attrs: 0    AttrsInfo: 0
```

The write pointer moved 16 blocks (2 x 4 KiB), and the write opened the zone implicitly (0x20). Read the data back and compare:

```
sudo nvme read /dev/nvme0n1 -s 0 -c 7 -z 4096 -d out.bin
cmp data.bin out.bin && echo same
```

Close it, then explicitly open four other zones, and try a fifth:

```
sudo nvme zns close-zone /dev/nvme0n1 -s 0
for z in 1 2 3 4; do
    sudo nvme zns open-zone /dev/nvme0n1 -s $((z * 0x20000))
done
sudo nvme zns open-zone /dev/nvme0n1 -s 0xa0000
```

```
zns-close-zone: Success, action:1 zone:0 all:0 zcapc:0 nsid:1
zns-open-zone: Success zone slba:20000 nsid:1
zns-open-zone: Success zone slba:40000 nsid:1
```

```
zns-open-zone: Success zone slba:60000 nsid:1
zns-open-zone: Success zone slba:80000 nsid:1
NVMe status: Too Many Open Zones: The controller does not allow additional open zones(0x41be)
```

Four zones are open, the limit `zns_max_open=4` allows no fifth. Zone 0 is Closed (0x40): it no longer counts as open, but it still counts as active. Finish one of the open zones, which makes it Full without writing it:

```
sudo nvme zns finish-zone /dev/nvme0n1 -s 0x20000
sudo nvme zns report-zones /dev/nvme0n1 -d 2 | tail -1
```

```
zns-finish-zone: Success, action:2 zone:20000 all:0 zcapc:0 nsid:1
SLBA: 0x20000    WP: 0xffffffffffffffff Cap: 0x20000    State: 0xe0 Type: 0x2  Attrs: 0    AttrsInfo: 0
```

A Full zone has no write pointer; it reads as all ones. Reset every zone to start over:

```
sudo nvme zns reset-zone /dev/nvme0n1 -a
```

5.5.6 Write with fio's zoned mode

`--zonemode=zbd` makes fio write each zone at its write pointer and respect the open-zone limit:

```
sudo fio --name=zw --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=write --bs=128k --size=256M
sudo blkzone report -c 5 /dev/nvme0n1
```

```
  write: IOPS=650, BW=81.3MiB/s (85.3MB/s)(256MiB/3148msec); 0 zone resets
  start: 0x000000000, len 0x020000, cap 0x020000, wptr 0x020000 reset:0 non-seq:0, zcond:14(fu) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000020000, len 0x020000, cap 0x020000, wptr 0x020000 reset:0 non-seq:0, zcond:14(fu) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000040000, len 0x020000, cap 0x020000, wptr 0x020000 reset:0 non-seq:0, zcond:14(fu) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000060000, len 0x020000, cap 0x020000, wptr 0x020000 reset:0 non-seq:0, zcond:14(fu) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000080000, len 0x020000, cap 0x020000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
```

256 MiB filled four 64 MiB zones, which are now Full. Read them back at random:

```
sudo fio --name=zr --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=randread --bs=4k --size=256M --runtime=8 --time_based
```

The average completion latency is about 89 us: the built-in QLC page read time of 85 us plus overhead. A ZNS write goes to a zone write cache at 1 us per 4 KiB page. The cached pages are programmed when a write finds the cache full, or when another zone takes the cache over, and that write pays for the program (write cache (§4.8.8)). Reset the zones before the next step:

```
sudo blkzone reset /dev/nvme0n1
```

5.5.7 Mount zonefs

zonefs exposes each zone as a file. The cloud kernel ships the module in `linux-modules-extra`:

```
sudo apt-get install -y zonefs-tools linux-modules-extra-$(uname -r)
sudo mkzonefs -f /dev/nvme0n1
sudo modprobe zonefs
sudo mkdir -p /mnt/zonefs
sudo mount -t zonefs /dev/nvme0n1 /mnt/zonefs
ls /mnt/zonefs; ls /mnt/zonefs/seq | wc -l
```

```
seq
15
```

zonefs keeps its super block in zone 0, and the other 15 zones are the files `seq/0` to `seq/14`. A file can only grow by appending, with direct I/O:

```
sudo dd if=/dev/urandom of=/mnt/zonefs/seq/0 bs=1M count=8 oflag=direct,append conv=notrunc
ls -l /mnt/zonefs/seq/0
sudo blkzone report -c 2 /dev/nvme0n1
```

```
8388608 bytes (8.4 MB, 8.0 MiB) copied, 0.109279 s, 76.8 MB/s
-rw-r----- 1 root root 8388608 Oct  1 16:28 /mnt/zonefs/seq/0
  start: 0x000000000, len 0x020000, cap 0x020000, wptr 0x020000 reset:0 non-seq:0, zcond:14(fu) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000020000, len 0x020000, cap 0x020000, wptr 0x004000 reset:0 non-seq:0, zcond: 2(oi) [type: 2(SEQ_WRITE_REQUIRED)]
```

The file size is the zone's write pointer (0x4000 sectors = 8 MiB), and the zone is implicitly open. Truncating the file to 0 resets the zone:

```
sudo truncate -s 0 /mnt/zonefs/seq/0
sudo blkzone report -c 2 /dev/nvme0n1 | tail -1
sudo umount /mnt/zonefs
```

```
  start: 0x000020000, len 0x020000, cap 0x020000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
```

Do not run `mkfs.ext4` on a zoned namespace: it needs random writes. A zoned file system such as f2fs also needs a randomly writable device for its metadata; tutorial 06 (§5.8) shows how to put a BlackBox namespace next to the zoned one. That combination was not run for this page.

5.5.8 What you learned

- A ZNS namespace is a host-managed zoned block device; the zone count follows from the `zns_` geometry, and the zone size from the geometry and the namespace size.
- Writes and appends open zones implicitly; open, close, finish and reset move them by hand; the device enforces the open and active limits.
- fio (`--zonemode=zbd`) and zonefs work on FEMU's ZNS as on real hardware.

5.5.9 Next

- ZNS mode (§4.7) covers ZRWA, zone capacity, conventional zones and the other options.
- The ZNS design page (§4.8) explains the zone layout over NAND and where time is charged.
- Tutorial 04 (§5.6) shows the other way to let the host place data: Flexible Data Placement.

5.6 Tutorial 04: Flexible Data Placement

You start a BlackBox SSD with Flexible Data Placement (FDP), read its FDP configuration and status with nvme-cli, write through a placement identifier and watch the reclaim unit fill, and then run a workload with data of two lifetimes, with and without placement, to see what placement does to the write amplification factor (WAF). It takes about fifteen minutes.

You need: tutorial 02 (§5.4) is helpful background, the variables from Before you start (§5.2.1), and about 5 GiB of free host memory. The guest image has nvme-cli 2.8, which has the `fdp` commands, and fio 3.36, which can place writes through its `io_uring_cmd` engine.

5.6.1 Background

FDP lets the host say which writes belong together. The device groups its NAND into reclaim units and offers a few reclaim unit handles. A write may carry a placement identifier that names a handle; all writes of one handle go into that handle's open reclaim unit. If data that dies together shares a reclaim unit, garbage collection (GC) finds it mostly invalid and copies little.

In FEMU, FDP is not a mode. It is a property of an NVMe subsystem (`femu-subsys`) that a BlackBox controller joins, and a reclaim unit is one line (superblock) of the BlackBox FTL. See the FDP design page (§4.6.7).

5.6.2 Start the guest (on the host)

```
./qemu-system-x86_64 -name femu-tut04,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    -device femu-subsys,id=fdp0,fdp=on,fdp.nruh=4 \
    -device femu,femu_mode=1,devsz_mb=384,nchs=2,luns_per_ch=4,blks_per_pl=64,subsys=fdp0 \
    -nographic
```

Option	Meaning
`femu-subsys,id=fdp0,fdp=on`	a subsystem with FDP in endurance group 1; it must come before the controller that names it
`fdp.nruh=4`	four reclaim unit handles, so placement identifiers 0 to 3
`nchs=2,luns_per_ch=4,blks_per_pl=64`	512 MiB of NAND in 64 lines of 8 MiB; each line is one reclaim unit
`devsz_mb=384`	a 384 MiB namespace, 75% of the NAND
`subsys=fdp0`	the controller joins the subsystem

`fdp.nru` keeps its default of 128 reclaim units, but FEMU uses at most one per line, so this device has 64.

5.6.3 Check FDP from the guest

In the guest (`./run-guest-ssh.sh`):

```
sudo nvme id-ctrl /dev/nvme0 | grep ctratt
sudo nvme fdp configs /dev/nvme0 -e 1
```

```
ctratt    : 0x80010
FDP Attributes: 0x80
Vendor Specific Size: 0
Number of Reclaim Groups: 1
Number of Reclaim Unit Handles: 4
Number of Namespaces Supported: 256
Reclaim Unit Nominal Size: 8388608
Estimated Reclaim Unit Time Limit: 0
Reclaim Unit Handle List:
  [0]: Persistently Isolated
  [1]: Persistently Isolated
  [2]: Persistently Isolated
  [3]: Persistently Isolated
```

Bit 19 (0x80000) of CTRATT says the controller supports FDP. The reclaim unit is 8 MiB, one line of this geometry. The namespace's placement identifiers and the room left in each one's reclaim unit:

```
sudo nvme fdp status /dev/nvme0n1
```

```
Placement Identifier 0; Reclaim Unit Handle Identifier 0
  Estimated Active Reclaim Unit Time Remaining (EARUTR): 0
```

```
  Reclaim Unit Available Media Writes (RUAMW): 16384
...
```

RUAMW counts logical blocks: 16384 x 512 bytes = 8 MiB, an empty reclaim unit. The output repeats for identifiers 1 to 3.

5.6.4 Write through a placement identifier

A write with directive type 2 (data placement) carries its placement identifier in the directive specific field (`-S`). Write 4 KiB three times through identifier 1:

```
head -c 4096 /dev/urandom > d.bin
for i in 1 2 3; do
    sudo nvme write /dev/nvme0n1 -s $((i * 8)) -c 7 -z 4096 -d d.bin -T 2 -S 1
done
sudo nvme fdp status /dev/nvme0n1 | grep RUAMW
```

```
write: Success
write: Success
write: Success
  Reclaim Unit Available Media Writes (RUAMW): 16384
  Reclaim Unit Available Media Writes (RUAMW): 16360
  Reclaim Unit Available Media Writes (RUAMW): 16384
  Reclaim Unit Available Media Writes (RUAMW): 16384
```

Only identifier 1's reclaim unit lost room: 3 writes of 8 blocks. A write without a directive, or with an identifier the namespace does not have, goes to handle 0; an identifier the namespace does not have also logs an FDP event (`sudo nvme fdp events /dev/nvme0 -e 1 -E` lists them). The endurance group statistics count bytes:

```
sudo nvme fdp stats /dev/nvme0 -e 1
```

```
Host Bytes with Metadata Written (HBMW): 12288
Media Bytes with Metadata Written (MBMW): 12288
Media Bytes Erased (MBE): 0
```

On a device that has seen only these three writes, HBMW is the 12 KiB the host wrote, and no reclaim unit has been erased yet. HBMW counts what the host wrote; MBMW also counts what GC copied, so MBMW / HBMW is the WAF over the endurance group.

5.6.5 Measure what placement does

The workload mixes two lifetimes: a hot 64 MiB region that takes random overwrites, and a cold 320 MiB region rewritten sequentially. Without placement both go into the same reclaim units, and GC copies cold pages whenever it collects a unit full of dead hot pages. With placement, hot writes use identifier 0 and cold writes identifier 1.

Save this in the guest as `fdp-run.sh`. It follows the method of tutorial 02 (§5.4.3): switch time off, precondition, then measure the counter differences of one run.

```
cat > fdp-run.sh <<'EOF'
# Usage: FDP=0 bash fdp-run.sh, or FDP=1 bash fdp-run.sh
c0() {
    sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b |
        od -An -t u8 -j 8 -N 24 -w24
```

```
}
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=2 >/dev/null
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=4 >/dev/null
P= H= C=
if [ "$FDP" = 1 ]; then P=--fdp=1 H=--fdp_pli=0 C=--fdp_pli=1; fi
J="--filename=/dev/ng0n1 --ioengine=io_uring_cmd --cmd_type=nvme --bs=4k --iodepth=8 $P"
sudo fio --name=prefill $J $C --rw=write --size=384M >/dev/null
c0 > before.txt
sudo fio $J --randrepeat=0 \
    --name=hot $H --rw=randwrite --offset=0 --size=64M --io_size=1G \
    --name=cold $C --rw=write --offset=64M --size=320M --io_size=1G >/dev/null
c0 > after.txt
paste before.txt after.txt | awk '{ h = $4 - $1; g = $5 - $2; n = $6 - $3;
    printf "host %d  gc %d  nand %d  WAF %.3f\n", h, g, n, (n + g) / h }'
sudo nvme fdp stats /dev/nvme0 -e 1
EOF
```

fio's `io_uring_cmd` engine sends NVMe commands through the generic node `/dev/ng0n1`. With `--fdp=1` it reads the namespace's placement identifiers and uses the ones whose indexes `--fdp_pli` lists, one per job here.

Restart QEMU (step 1) before each run, so that both start from an empty device, and run once each way:

```
FDP=0 bash fdp-run.sh
```

```
host 524288  gc 3524608  nand 524288  WAF 7.723
Host Bytes with Metadata Written (HBMW): 2550136832
Media Bytes with Metadata Written (MBMW): 16986931200
Media Bytes Erased (MBE): 16567500800
```

```
FDP=1 bash fdp-run.sh
```

```
host 524288  gc 1977855  nand 524288  WAF 4.772
Host Bytes with Metadata Written (HBMW): 2550136832
Media Bytes with Metadata Written (MBMW): 10651430912
Media Bytes Erased (MBE): 10234101760
```

The host wrote the same 2.55 GB (384 MiB of prefill plus 2 GiB) in both runs. Placement cut the pages GC copied by 44% and the WAF of the measured run from 7.72 to 4.77. The FDP statistics count since the device started, prefill included: MBMW / HBMW falls from 6.66 to 4.18.

The WAF with placement is still well above 1 because the hot region is written at random: its own reclaim units still hold a mix of live and dead pages when GC takes them. Try `gc_strategy=1` (cost-benefit) on the controller, or more handles and a third lifetime.

5.6.6 What you learned

- FDP lives on `femu-subsys`; the controller joins it with `subsys=`.
- `nvme fdp configs`, `status` and `stats` show the handles, the room left in each reclaim unit, and host against media bytes.
- Placement only helps when the writes carry it: `nvme write -T 2 -S N`, or fio with `--ioengine=io_uring_cmd --fdp=1` on the generic node.
- Separating lifetimes by placement cuts GC copies.

5.6.7 Next

- Flexible Data Placement (§4.5) lists every option, refusal and the other log pages.
- The FDP design page (§4.6) explains how reclaim units map to lines and how GC chooses one.
- Tutorial 05 (§5.7) turns from page counts to latency.

5.7 Tutorial 05: latency tuning

You change the parts of FEMU's timing model one at a time: the flat NAND times, the cell type, the channel bus, the host link and the controller firmware cost. For each one you predict the effect, measure it with fio, and compare. It takes about twenty minutes.

You need: tutorial 01 (§5.3), the variables from Before you start (§5.2.1), and about 5 GiB of free host memory. Read performance tuning (§5.12) first if you want numbers that repeat to the microsecond: a poller or FTL thread that has to share a core adds delay the model did not ask for.

5.7.1 Background

FEMU copies the data of a command at once, computes when the command would have finished on the emulated device, and posts the completion no earlier than that (the timing model (§6.2.1)). A queue depth 1 latency is therefore the model's time plus a fixed overhead of the guest and of FEMU itself. Each step below changes one term of the model; NAND media and timing (§3.5) has the formulas.

5.7.2 The device and the measurement

Every run uses a small BlackBox SSD, 512 MiB of NAND in 2 channels of 4 LUNs, and adds one option to it:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25
```

Start QEMU with the command line of tutorial 01, step 2 (§5.3.2) and this `-device` line, restarting QEMU for each configuration. In the guest, fill the first 256 MiB (a read of a page never written costs no NAND time), then run four measurements:

```
sudo fio --name=fill --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=write --bs=128k --iodepth=16 --size=256M
F="--filename=/dev/nvme0n1 --direct=1 --ioengine=libaio --size=256M --runtime=8 --time_based"
sudo fio --name=rd4k   $F --rw=randread  --bs=4k   --iodepth=1
sudo fio --name=rd128k $F --rw=randread  --bs=128k --iodepth=1
sudo fio --name=qd32   $F --rw=randread  --bs=4k   --iodepth=32
sudo fio --name=wr4k   $F --rw=randwrite --bs=4k   --iodepth=1
```

Read the median (`clat` 50.00th percentile) of the QD1 tests and the IOPS of the QD32 test. Keep the write test last. Random overwrites move pages to new places, so a 128 KiB read after them finds its 32 pages spread unevenly over the LUNs and takes longer than the model of a freshly filled range (in a first run with the write test second, it took 289 us instead of 165 us). On this small drive, background garbage collection may also start in the last second of the write test; that moves its tail, not its median.

5.7.3 The baseline

With the default timing (40 us read, 200 us program, no channel bus):

Test	Result
4 KiB read, QD1	44.3 us
4 KiB write, QD1	203.8 us
128 KiB read, QD1	164.9 us
4 KiB read, QD32	176,880 IOPS

A 4 KiB read is `pg_rd_lat` plus about 4 us. A 128 KiB read is 32 pages. The FTL spreads consecutive pages over the channels and then the LUNs, so the 32 pages sit on all 8 LUNs, 4 each. The LUNs work in parallel and each reads its 4 pages one after another: 4 x 40 us = 160 us. Copying the 128 KiB takes about 30 us (measured with NAND times set to 0), but FEMU copies at once and holds only the completion, so the copy overlaps the modelled time instead of adding to it. At queue depth 32, 8 LUNs reading in parallel allow at most 8 / 40 us = 200,000 reads per second; FEMU delivers 88% of that.

5.7.4 Flat NAND times

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,pg_rd_lat=80000,pg_wr_lat=400000
```

Prediction: reads and writes take 40 us and 200 us longer, and QD32 throughput halves.

Test	Baseline	80 us / 400 us
4 KiB read, QD1	44.3 us	84.5 us
4 KiB write, QD1	203.8 us	423.9 us
4 KiB read, QD32	176,880 IOPS	89,317 IOPS

5.7.5 Cell type

`nand_cell_type` replaces the flat times with built-in tables that give each page type in a wordline its own time (cell types (§3.5.6)). TLC (3) has lower, center and upper pages read in 56.5, 77.5 and 106 us:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,nand_cell_type=3
```

Test	Baseline	TLC
4 KiB read, QD1	44.3 us	81.4 us
4 KiB read, QD1, 99th percentile	52.0 us	114.2 us
4 KiB write, QD1	203.8 us	2310.1 us
4 KiB read, QD32	176,880 IOPS	89,449 IOPS

The pages of a block are split about equally over the lower, center and upper types, so the read median is the center page's 77.5 us plus overhead, and the 99th percentile is near the upper page. TLC programs take milliseconds. `pg_rd_lat`, `pg_wr_lat` and `blk_er_lat` have no effect while `nand_cell_type` is set.

5.7.6 The channel bus

By default, moving a page between controller and NAND takes no time. Any non-zero bus phase turns the channel bus on; `pg_xfer_lat` is the data transfer per page, and transfers on one channel run one at a time:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,pg_xfer_lat=20000
```

Test	Baseline	20 us transfer
4 KiB read, QD1	44.3 us	63.7 us
128 KiB read, QD1	164.9 us	383.0 us
4 KiB read, QD32	176,880 IOPS	98,056 IOPS

A single page read gains exactly one transfer. A 128 KiB read moves 16 pages over each channel, one at a time: 320 us of transfers, which now dominate the read (with NAND times set to 0 it still takes 342 us). Under load the two channels become the limit: each moves one page per 20 us, 100,000 pages per second for both, which is where QD32 lands.

5.7.7 The host link

`pcie_bandwidth_mbps` charges each Read and Write its size divided by the link bandwidth, on one queue per direction (host link (§3.5.13)):

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,pcie_bandwidth_mbps=1000
```

Test	Baseline	1000 MB/s link
4 KiB read, QD1	44.3 us	47.9 us
128 KiB read, QD1	164.9 us	305.2 us

128 KiB at 1000 MB/s is 131 us, and the 128 KiB read grew by 140 us. A 4 KiB read pays 4 us. `pcie_prop_delay_ns` adds a fixed delay on top.

5.7.8 Controller firmware

`fw_cpu_ns` charges a fixed cost per Read, Write and Zone Append on one modelled controller core, so commands queue behind each other there:

```
-device femu,femu_mode=1,nchs=2,luns_per_ch=4,blks_per_pl=64,op_pcent=25,fw_cpu_ns=20000
```

Test	Baseline	20 us firmware
4 KiB read, QD1	44.3 us	63.7 us
4 KiB write, QD1	203.8 us	230.4 us
4 KiB read, QD32	176,880 IOPS	49,534 IOPS

At queue depth 1 the cost simply adds. Under load one core finishes one command per 20 us, 50,000 per second, and that becomes the limit no matter how many LUNs the NAND has.

5.7.9 What you learned

- Queue depth 1 latency is the modelled time plus a few microseconds; each timing option moves it by what you configured.
- Throughput is set by whichever resource saturates first: LUNs, channels with the bus on, or the firmware core.
- `nand_cell_type` gives a spread of read times rather than one value.

The numbers above come from one run per configuration on a 20-core host where other work was running, so expect yours to differ by a few microseconds. The relations between them should hold.

5.7.10 Next

- NAND media and timing (§3.5) has the rules these measurements follow, including program suspend and ECC read time.
- Measuring (§6.1.6) and performance tuning (§5.12) explain how to pin FEMU's threads and get repeatable numbers.
- Tutorial 06 (§5.8) puts several modes on one controller.

5.8 Tutorial 06: several namespaces, several modes

You start one FEMU controller with four namespaces, each in a different mode: a BlackBox SSD, a zoned namespace, a NoSSD namespace with no media timing, and a key-value namespace. You find each one in the guest, use it, and see which counters belong to which. It takes about ten minutes.

You need: the variables from Before you start (§5.2.1) and about 6 GiB of free host memory. Tutorials 01 (§5.3), 03 (§5.5) and 07 (§5.9) explain each mode on its own.

5.8.1 Background

A controller's `femu_mode` sets its default mode. `namespaces` sets how many namespaces exist from boot, `namespace_sizes` how big each is (the default splits `devsz_mb` evenly), and `namespace_modes` the mode of each. The namespaces are packed one after another in the controller's memory. Each BlackBox namespace gets its own FTL built from the whole NAND geometry; each ZNS namespace builds its own zones; each KV namespace has its own key space (namespaces design page (§4.16.5)).

5.8.2 Start the guest (on the host)

```
./qemu-system-x86_64 -name femu-tut06,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    -device femu,femu_mode=1,devsz_mb=2048,namespaces=4,namespace_modes=bbssd,,znssd,,nossd,,kvssd
        \
    -nographic
```

A comma inside a property value is written twice on the QEMU command line, so `namespace_modes=bbssd,,znssd,,nossd,,kvssd` is the list `bbssd,znssd,nossd,kvssd`. A single comma would end the property, and QEMU would read `znssd` as the name of the next one. The 2 GiB backend is split into four 512 MiB namespaces.

5.8.3 Find the namespaces

In the guest (`./run-guest-ssh.sh`):

```
sudo nvme list
sudo nvme list-ns /dev/nvme0
```

```
Node                 Generic              SN                   Model                                    Namespace  Usage                      Format           FW Rev
-------------------- -------------------- -------------------- ---------------------------------------- ---------- -------------------------- ---------------- --------
/dev/nvme0n1         /dev/ng0n1           vSSD0                FEMU BlackBox-SSD Controller             0x1        536.87  MB / 536.87  MB    512   B +  0 B   1.0
/dev/nvme0n2         /dev/ng0n2           vSSD0                FEMU BlackBox-SSD Controller             0x2        536.87  MB / 536.87  MB    512   B +  0 B   1.0
/dev/nvme0n3         /dev/ng0n3           vSSD0                FEMU BlackBox-SSD Controller             0x3        536.87  MB / 536.87  MB    512   B +  0 B   1.0
nvme0n4              /dev/ng0n4           vSSD0                FEMU BlackBox-SSD Controller             0x4        536.87  MB /   0.00   B    512   B +  0 B   1.0
[   0]:0x1
[   1]:0x2
[   2]:0x3
[   3]:0x4
```

Namespaces 1 to 3 have block devices. Namespace 4, the KV one, has only the generic character device `/dev/ng0n4`: Linux has no block driver for the key-value command set.

The model and serial number are the controller's, and they come from its own `femu_mode` (BlackBox here), not from the namespaces' modes. Tell the namespaces apart by what they report:

```
for n in 1 2 3; do
    echo nvme0n$n $(cat /sys/block/nvme0n$n/queue/zoned) $(cat /sys/block/nvme0n$n/size)
done
```

```
nvme0n1 none 1048576
nvme0n2 host-managed 1048576
nvme0n3 none 1048576
```

Namespace 2 is the zoned one. Each is 1048576 sectors of 512 bytes, 512 MiB.

5.8.4 Use each one

The BlackBox namespace pays NAND program time, the NoSSD namespace pays none:

```
sudo fio --name=n1 --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=1 --size=64M --runtime=5 --time_based
sudo fio --name=n3 --filename=/dev/nvme0n3 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=1 --size=64M --runtime=5 --time_based
```

The average completion latency is about 206 us on `nvme0n1` (the 200 us program) and about 5.5 us on `nvme0n3`.

The zoned namespace builds its zones from its own size: 16 zones of 32 MiB, half the size of the 1 GiB namespace in tutorial 03 (§5.5):

```
sudo blkzone report -c 2 /dev/nvme0n2
```

```
  start: 0x000000000, len 0x010000, cap 0x010000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
  start: 0x000010000, len 0x010000, cap 0x010000, wptr 0x000000 reset:0 non-seq:0, zcond: 1(em) [type: 2(SEQ_WRITE_REQUIRED)]
```

Store and retrieve a value in the KV namespace. With several namespaces, Linux refuses I/O passthrough on the controller node `/dev/nvme0`, so use the generic node, and name the namespace with `-n 4`:

```
head -c 64 /dev/urandom > value.bin
sudo nvme io-passthru /dev/ng0n4 -O 0x01 -n 4 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -w -i value.bin
sudo nvme io-passthru /dev/ng0n4 -O 0x02 -n 4 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -r -b > out.bin
cmp value.bin out.bin && echo match
```

```
IO Command Write is Success and result: 0x00000040
IO Command Read is Success and result: 0x00000040
match
```

The same Retrieve sent to `/dev/nvme0` fails with `passthru: Invalid argument`. Tutorial 07 (§5.9) explains the command fields.

5.8.5 Whose counters

Log page C0h is per controller. It sums the BlackBox, CSD and KV namespaces, and ignores NoSSD and ZNS (counters (§B.1)). After the two fio runs above, before the KV store:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
                24046                   0               24046
```

24046 pages are the BlackBox run alone (5 seconds at about 4,800 writes per second); the 2.7 GiB written to the NoSSD namespace in the same time is not counted. The SMART log (`sudo nvme smart-log /dev/nvme0`) counts the successful Read, Write and Zone Append commands of every namespace, NoSSD and ZNS included; KV Store and Retrieve share those opcodes and are counted too.

5.8.6 What you learned

- `namespaces`, `namespace_sizes` and `namespace_modes` build several namespaces on one controller; commas inside a value are doubled.
- Each mode keeps its own behaviour: zone geometry, NAND timing, key space.
- A KV namespace has only a generic node, and on a controller with several namespaces passthrough needs that node.
- C0h counts only BlackBox, CSD and KV namespaces; NoSSD and ZNS do not add to it.

5.8.7 Next

- Several namespaces and devices (§4.15) lists the limits and refusals, and how to start several controllers.
- Namespace management (§4.17) creates and deletes namespaces from the guest while it runs.
- Subsystems, controllers and namespaces (§4.16) explains how the backend is partitioned.

5.9 Tutorial 07: key-value SSD

You start an SSD with the NVMe Key Value command set, then store, retrieve, check, list and delete values with nvme-cli's passthrough command, and read the status codes the device returns. It takes about ten minutes.

You need: the variables from Before you start (§5.2.1), about 5 GiB of free host memory, and a guest kernel of Linux 6.0 or newer. Older kernels skip a namespace whose command set they do not know, so the commands below fail. The Ubuntu 24.04 image from `make-guest-image.sh` runs Linux 6.8.

5.9.1 Background

A KV namespace stores values under keys instead of blocks at addresses. Linux has no driver for the key-value command set, so the namespace gets no block device, only the generic node `/dev/ngXnY`, and you send each command as a passthrough command. The fields of a command:

Field	Holds
opcode	0x01 Store, 0x02 Retrieve, 0x06 List, 0x10 Delete, 0x14 Exist
CDW2, CDW3	bytes 0 to 7 of the key, little-endian
CDW14, CDW15	bytes 8 to 15 of the key
CDW11 bits 7:0	key length, 1 to 16 bytes
CDW11 bits 15:8	Store only: bit 8 stores only if the key exists, bit 9 only if it does not
CDW10	the value size for Store, the buffer size for Retrieve and List

Field	Holds
NSID	the namespace; it must be given

Values are up to 2 MiB and live on emulated NAND with the BlackBox geometry and timing (the KV design page (§4.12.7)).

5.9.2 Start the guest (on the host)

```
./qemu-system-x86_64 -name femu-tut07,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    -device femu,femu_mode=5,devsz_mb=1024 \
    -nographic
```

`femu_mode=5` is KV; the namespace is 1 GiB.

5.9.3 Find the namespace

In the guest (`./run-guest-ssh.sh`):

```
sudo nvme list
ls /dev/ng0n1 /dev/nvme0n1
```

```
nvme0n1              /dev/ng0n1           vKVSSD0              FEMU KV-SSD Controller                 0x1         1.07  GB /   0.00   B    512   B +  0 B   1.0
/dev/ng0n1
ls: cannot access '/dev/nvme0n1': No such file or directory
```

There is a generic node and no block device. On a controller with one namespace you may also send passthrough commands to the controller node `/dev/nvme0`, which this tutorial does. With several namespaces use `/dev/ngXnY` (tutorial 06 (§5.8.4)).

5.9.4 Store and retrieve

Store a 64-byte value under the 4-byte key `BBBB` (0x42424242), then read it back:

```
head -c 64 /dev/urandom > value.bin
sudo nvme io-passthru /dev/nvme0 -O 0x01 -n 1 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -w -i value.bin
sudo nvme io-passthru /dev/nvme0 -O 0x02 -n 1 --cdw10=64 --cdw11=4 \
    --cdw2=0x42424242 -l 64 -r -b > out.bin
cmp value.bin out.bin && echo match
```

```
IO Command Write is Success and result: 0x00000040
IO Command Read is Success and result: 0x00000040
match
```

nvme-cli names opcodes 0x01 and 0x02 after the NVM command set's Write and Read; the device treats them as Store and Retrieve. The result (completion Dword 0) of a Retrieve is the full size of the value, 0x40. `-n 1` is required: nvme-cli sends namespace 0 by default, which Linux refuses with `Invalid argument`.

A buffer smaller than the value gets the first bytes, and the result still reports the full size, so the host can retry with a larger buffer:

```
sudo nvme io-passthru /dev/nvme0 -O 0x02 -n 1 --cdw10=16 --cdw11=4 \
    --cdw2=0x42424242 -l 16 -r -b | od -An -tx1
```

The command prints `result: 0x00000040` and 16 bytes of the value.

5.9.5 Conditional stores, Exist and List

Check that the key exists, then try to store it again with bit 9 set ("only if the key does not exist"), which CDW11 `0x204` encodes with the key length 4:

```
sudo nvme io-passthru /dev/nvme0 -O 0x14 -n 1 --cdw11=4 --cdw2=0x42424242
printf hello > v2.bin
sudo nvme io-passthru /dev/nvme0 -O 0x01 -n 1 --cdw10=5 --cdw11=0x204 \
    --cdw2=0x42424242 -l 5 -w -i v2.bin
```

```
IO Command Vendor Specific is Success and result: 0x00000000
NVMe status: unrecognized(0x4089)
```

nvme-cli does not know the key-value status codes. 0x4089 is status 0x89, Key Exists, with the Do Not Retry bit (0x4000). Store a second key, `CCCC`, and list the keys. List walks the device's key index from its first slot, so keys come back in index order, not sorted:

```
sudo nvme io-passthru /dev/nvme0 -O 0x01 -n 1 --cdw10=5 --cdw11=4 \
    --cdw2=0x43434343 -l 5 -w -i v2.bin
sudo nvme io-passthru /dev/nvme0 -O 0x06 -n 1 --cdw10=4096 --cdw11=0 \
    -l 4096 -r -b | od -An -tx1 -N 24
```

```
 02 00 00 00 04 00 43 43 43 43 00 00 04 00 42 42
 42 42 00 00 00 00 00 00
```

The buffer starts with a 4-byte key count (2), then one entry per key: a 2-byte key length (4), the key, and padding to a multiple of 4 bytes.

5.9.6 Delete

```
sudo nvme io-passthru /dev/nvme0 -O 0x10 -n 1 --cdw11=4 --cdw2=0x42424242
sudo nvme io-passthru /dev/nvme0 -O 0x14 -n 1 --cdw11=4 --cdw2=0x42424242
```

```
IO Command Vendor Specific is Success and result: 0x00000000
NVMe status: unrecognized(0x4087)
```

0x87 is Key Does Not Exist. Deleting a key that does not exist succeeds unless the host sets the EDNEK bit of the Key Value Configuration feature (20h).

Status	Meaning
0x85	Invalid Value Size: the value is larger than 2 MiB
0x86	Invalid Key Size: a key length of 0 on Store, Retrieve, Delete or Exist
0x87	Key Does Not Exist
0x89	Key Exists, from a Store with bit 9 set

5.9.7 Counters

The KV namespace adds to log page C0h like a BlackBox one:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
```

```
                2                   0                   2
```

Two Stores of small values, two pages programmed. The key index lives in device DRAM, but each command is charged a base cost of one NAND page read time, which overlaps the media time of its value (KV timing (§4.12.9)).

5.9.8 What you learned

- A KV namespace has a generic node and no block device; Linux 6.0 or newer is needed to see it.
- `nvme io-passthru` with the opcode, the key in CDW2 and CDW3 (and CDW14 and CDW15), the key length in CDW11 and an explicit namespace drives every command.
- nvme-cli prints KV status codes as `unrecognized`; read the low byte.

5.9.9 Next

- KV mode (§4.11) has `kv-probe.c`, a C program that runs the whole command cycle and checks the results.
- The KV design page (§4.12) explains the key index and the value store.

5.10 Tutorial 08: a CXL SSD as memory

You start a guest with `femu-cxl-ssd`, a CXL Type-3 memory device whose capacity is backed by emulated NAND behind a DRAM page cache. You find the device in the guest, turn it into a memory region, use the region as a DAX device and as a NUMA node, and read the device's counters from the host over QMP. It takes about twenty minutes.

You need: the variables from Before you start (§5.2.1), about 3 GiB of free host memory, and `socat` on the host. The guest needs the CXL tools (`cxl`, `daxctl`) and a kernel that can commit a CXL region inside a virtual machine; step 4 explains what that takes.

5.10.1 Background

The guest sees ordinary CXL memory. Behind it, FEMU keeps a cache of 4 KiB pages in front of the BlackBox FTL and NAND model. A cache hit costs no media time, a miss costs a NAND page read (nothing for a page never written), and evicting a dirty page costs a NAND program. With the default `der=off`, every guest load or store to the device exits to QEMU, and the vCPU waits out the media time before the access completes (CXL SSD design (§4.19.5)).

The device needs a CXL topology: a CXL host bridge, a root port below it, the device below the root port, and a fixed memory window on the machine. `run-cxlssd.sh` builds all of it.

5.10.2 Start the guest (on the host)

`run-cxlssd.sh` adds no guest disk and no network; you pass them after the script name. Put them on the main PCIe bus with `bus=pcie.0`: without it, QEMU may place them on the CXL host bridge, which accepts only bridges, and stops with `Only PCI/PCIe bridges can be plugged into pxb-cxl`. From `build-femu/`:

```
RAM=2G ../femu-scripts/run-cxlssd.sh \
    -object memory-backend-ram,id=ram0,size=2G \
    -numa node,nodeid=0,cpus=0-3,memdev=ram0 \
    -drive file=$OSIMGF,if=none,id=hd0 \
```

```
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::$SSH_PORT-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0 \
    -qmp unix:$PWD/qmp-cxl.sock,server=on,wait=off
```

Part	Why
`RAM=2G`	the guest's RAM; the script's default is 4G
`-object memory-backend-ram,...` and `-numa node,...`	describe the guest RAM as NUMA node 0, so QEMU builds an ACPI SRAT and the CXL memory can become a node of its own; the backend size must equal the guest RAM
`-drive`, `-device virtio-blk-pci`	the guest disk
`-netdev`, `-device virtio-net-pci`	SSH to the guest on `SSH_PORT`
`-qmp unix:...`	the socket you read the counters through in step 6

The script's defaults give a 256 MiB device with a 3072-page (12 MiB) direct-mapped cache, 8 channels of 8 LUNs, and `lsa-control=on`, which lets the guest send experiment commands that write files on the host (run-cxlssd.sh (§C.3) lists every variable). `DRY_RUN=1` in front prints the QEMU command instead of running it.

5.10.3 Install the guest tools

The image from `make-guest-image.sh --cxl` already has them. Otherwise, in the guest (`./run-guest-ssh.sh`):

```
sudo apt-get install -y cxl daxctl ndctl linux-modules-extra-$(uname -r)
sudo modprobe cxl_acpi
sudo modprobe cxl_pci
```

5.10.4 Find the device

```
sudo cxl list -M
sudo cxl list -D -d root
```

```
[
  {
    "memdev":"mem0",
    "ram_size":268435456,
    "serial":0,
    "host":"0000:35:00.0"
  }
]
[
  {
    "decoder":"decoder0.0",
    "resource":4563402752,
    "size":268435456,
    "interleave_ways":1,
    "max_available_extent":268435456,
    "pmem_capable":true,
    "volatile_capable":true,
    "accelmem_capable":true,
    "nr_targets":1
```

```
  }
]
```

`mem0` is the `femu-cxl-ssd` with 256 MiB of volatile capacity, and `decoder0.0` is the root decoder of the fixed memory window.

5.10.5 Create a region

```
sudo cxl create-region -d decoder0.0 -t ram -w 1 -m mem0
```

With the stock Ubuntu 24.04 kernel this fails:

```
cxl region: create_region: region0: failed to commit decode: No such device or address
cxl region: cmd_create_region: created 0 regions
```

and `dmesg` says `cxl region0: Failed to synchronize CPU cache state`. Linux invalidates the CPU caches over the new range before it commits a region, and a virtual machine cannot do that. A guest kernel built with `CONFIG_CXL_REGION_INVALIDATION_TEST=y` skips the step. Check yours with:

```
grep -E 'CONFIG_(CXL_REGION|CXL_REGION_INVALIDATION_TEST|DEV_DAX_CXL|DEV_DAX_KMEM)=' \
    /boot/config-$(uname -r)
```

CXL SSD guest requirements (§4.18.3) lists every option the guest kernel needs. The rest of this tutorial assumes such a kernel. Its output is illustrative: it was not captured for this page. FEMU's own guest runs used Linux 6.12 with these options and wrote and verified data on the region as a NUMA node.

On such a kernel the region is created and appears as a DAX device:

```
sudo daxctl list
```

```
[
  {
    "chardev":"dax0.0",
    "size":268435456,
    "target_node":1,
    "align":2097152,
    "mode":"devdax"
  }
]
```

If `mode` is `system-ram`, the kmem driver claimed the region on its own; `sudo daxctl reconfigure-device --mode=devdax --force dax0.0` gives it back.

5.10.6 Use the region

As a DAX device

A program maps `/dev/dax0.0` and loads and stores directly. Offsets and lengths must be multiples of `align` (2 MiB). This writes and reads back one page:

```
sudo python3 -c '
import mmap, os
fd = os.open("/dev/dax0.0", os.O_RDWR)
m = mmap.mmap(fd, 2 << 20)
m[0:4096] = b"femu" * 1024
print(m[0:8])
'
```

Every access traps to QEMU (`der=off`), so each costs microseconds even on a cache hit; keep test working sets small.

As a NUMA node

```
sudo daxctl reconfigure-device --mode=system-ram --force dax0.0
numactl -H
numactl --membind=1 dd if=/dev/zero of=/dev/null bs=1M count=64
```

`numactl -H` shows a node 1 with memory and no CPUs: the CXL SSD. Memory a program allocates with `--membind=1` lives on the emulated NAND behind the cache. Expect such a program to run orders of magnitude slower than on node 0 with `der=off`; `der=memslot` maps cached pages into the guest so hits run at DRAM speed (direct mapping (§4.18.7)).

5.10.7 Read the counters (on the host)

The counters are QOM properties of the device, whose `id` is `cxlssd`. QEMU's `scripts/qmp/qom-get` reads one; run it from the FEMU source tree:

```
export QMP_SOCKET=build-femu/qmp-cxl.sock
for p in cache-pages cache-ways cache-hits cache-misses media-reads media-writes media-time-ns
    media-full der-active; do
    echo "$p: $(python3 scripts/qmp/qom-get /machine/peripheral/cxlssd.$p)"
done
```

On the guest of this tutorial, which could not create a region, every access counter is still 0:

```
cache-pages: 3072
cache-ways: 1
cache-hits: 0
cache-misses: 0
media-reads: 0
media-writes: 0
media-time-ns: 0
media-full: 0
der-active: False
```

After a workload on the region, `cache-misses`, `media-reads` and `media-time-ns` grow, and `cache-hits` counts the accesses the cache served. `media-full` must stay 0: a non-zero value means some NAND programs found no free page and were not timed, and the run is not valid.

To change a setting or start from a cold cache, send raw QMP, for example with `socat`. `flush-cache` writes dirty pages back and drops every page that is not pinned; `cache-ways` can change while the guest runs:

```
printf '%s\n' '{"execute":"qmp_capabilities"}' \
  '{"execute":"qom-set","arguments":{"path":"/machine/peripheral/cxlssd","property":"flush-cache",␣
     "value":true}}'
     \
  '{"execute":"qom-set","arguments":{"path":"/machine/peripheral/cxlssd","property":"cache-ways",␣
     "value":4}}'
     \
  '{"execute":"qom-get","arguments":{"path":"/machine/peripheral/cxlssd","property":"cache-ways"}}'
     |
  socat -t 2 - UNIX-CONNECT:build-femu/qmp-cxl.sock
```

```
{"QMP": {"version": {"qemu": {"micro": 0, "minor": 1, "major": 10}, "package": ""}, "capabilities": ["oob"]}}
{"return": {}}
{"return": {}}
{"return": {}}
{"return": 4}
```

`stats-reset` (also a `qom-set` to `true`) clears the cache event counters but not the media counters, so measure those as the difference of two reads (runtime properties (§A.2.2)).

5.10.8 What you learned

- `run-cxlssd.sh` builds the CXL topology; guest devices you add go on `bus=pcie.0`.
- The guest sees `mem0` and a root decoder; committing a region inside a VM needs `CONFIG_CXL_REGION_INVALIDATION_TEST`.
- The region works as a DAX device or, with an SRAT from `-numa`, as a CPU-less NUMA node.
- The counters are QOM properties read over QMP; `media-full` must stay 0.

5.10.9 Next

- CXL SSD (§4.18) covers the cache options, prefetch, direct mapping and the control channel.
- The CXL SSD design page (§4.19) explains the cache, the write-back path and direct mapping.
- The caching API (§4.20) lets the guest pin and drop pages itself.

5.11 Tutorial 09: configuration files

You describe a device in a short INI-style file instead of a long `-device femu,...` line, expand it with `ssd-config.sh`, let the script catch a misspelled property, and boot the result. It takes about ten minutes.

You need: a built FEMU and the variables from Before you start (§5.2.1). Steps 1 to 4 run on the host and start no guest.

5.11.1 Background

`hw/femu/scripts/ssd-config.sh` turns a file of `key = value` lines into QEMU `-device` arguments. Keys are the `femu` device's property names and mean exactly what the property reference (§A.1) says. Two additions:

- `mode = bbssd` (or `znssd`, `nossd`, `ocssd`, `csd`, `kvssd`) stands for `femu_mode`.
- Keys under a `[subsys]` section become a separate `-device femu-subsys,...`, which the `femu` device is joined to. FDP lives there.

Other section headers are labels for the reader. `#` and `;` start a comment, and a key with an empty value is ignored, so a file can list a property and leave it at its default. The script checks every key against `-device femu,help` (`femu-subsys,help` for `[subsys]` keys) of the binary named by `FEMU_BIN`, or of one it finds in `build-femu/`, `build/` or `build-official/` of the source tree.

5.11.2 Write a configuration

Save the drive of tutorial 02, step 6 (§5.4.7) as `gc-study.conf` in `build-femu/`:

```
# Tutorial 02's skewed-workload drive: 2 GiB of NAND, 25% spare,
# GC from 95% of lines in use, cost-benefit victims, hot and cold separated.
```

```
[device]
mode     = bbssd
op_pcent = 25

[geometry]
nchs        = 4
luns_per_ch = 4
blks_per_pl = 128

[gc]
gc_thres_pcent = 95
gc_policy      = cost-benefit
hot_cold_sep   = on
```

5.11.3 Expand it

From `build-femu/`, on the host:

```
export FEMU_BIN=$PWD/qemu-system-x86_64
../femu-scripts/ssd-config.sh gc-study.conf
```

```
-device femu,id=nvme0,op_pcent=25,nchs=4,luns_per_ch=4,blks_per_pl=128,gc_thres_pcent=95,gc_policy=cost-benefit,hot_cold_sep=on,femu_mode=1
```

That is the line you would otherwise type, and the documentation checks start it under QEMU.

The script adds `id=nvme0` unless the file sets an `id`. `--device-only` prints the arguments without the `-device` words, for a launcher that adds them itself. Set `FEMU_BIN`: run through the `../femu-scripts` link, the script cannot find the binary on its own and then skips the key check with a warning.

5.11.4 Let it catch mistakes

Misspell a key:

```
printf '[device]\nmode = bbssd\ngc_polcy = greedy\n' > bad.conf
../femu-scripts/ssd-config.sh bad.conf; echo "exit $?"
```

```
ssd-config: unknown property 'gc_polcy' -- not one FEMU accepts
ssd-config: config rejected; see the warnings above
exit 1
```

Put a subsystem property on the controller:

```
printf '[device]\nmode = bbssd\nfdp = on\n' > bad.conf
../femu-scripts/ssd-config.sh bad.conf
```

```
ssd-config: 'fdp' is a subsystem property; move it under [subsys]
ssd-config: config rejected; see the warnings above
```

`--check` validates a file without printing the arguments: it prints the same warnings for a bad file, nothing for a good one, and its exit status says which. Without a binary to check against, it only warns and exits 0. Besides names, the script rejects an unknown `mode`, `mode` together with `femu_mode`, and lines that are not `key = value`. Values, ranges and combinations are checked when QEMU creates the device, which stops with a message; the "Limits and refusals" section of each mode page lists them, for example BlackBox (§4.4.5).

5.11.5 Lists and subsystems

The shipped files in `hw/femu/scripts/configs/` show the two conveniences. A list value is written with plain commas, and the script doubles them for QEMU:

```
../femu-scripts/ssd-config.sh ../femu-scripts/configs/heterogeneous.conf
```

```
-device femu,id=nvme0,devsz_mb=6144,namespaces=3,namespace_modes=bbssd,,znssd,,nossd,secsz=512,secs_per_pg=8,␣
    pgs_per_blk=256,blks_per_pl=256,pls_per_lun=1,luns_per_ch=8,nchs=8,femu_mode=1
```

A `[subsys]` section becomes its own device, placed first, with the controller joined to it:

```
../femu-scripts/ssd-config.sh ../femu-scripts/configs/fdp.conf
```

```
-device femu-subsys,id=femu-subsys-0,nqn=subsys0,fdp=on,fdp.nruh=4,fdp.nrg=1,fdp.nru=256 -device
    femu,id=nvme0,devsz_mb=4096,namespaces=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,pls␣
    per_lun=1,luns_per_ch=8,nchs=8,femu_mode=1,subsys=femu-subsys-0
```

Every mode has a file to start from: `bbssd.conf`, `zns.conf`, `nossd.conf`, `ocssd.conf`, `csd.conf` and `kvssd.conf`. The scripts reference (§C.4) lists them all.

5.11.6 Boot it

The launchers take no arguments, so put the expansion on your own QEMU command line, the one from tutorial 01, step 2 (§5.3.2), in place of its `-device femu` option. In bash:

```
QEMU_ARGS=$(../femu-scripts/ssd-config.sh gc-study.conf)
./qemu-system-x86_64 -name femu-tut09,debug-threads=on \
    -enable-kvm -cpu host -smp 4 -m 4G \
    -device virtio-scsi-pci,id=scsi0 -device scsi-hd,drive=hd0 \
    -drive file=$OSIMGF,if=none,cache=none,format=qcow2,id=hd0 \
    -net user,hostfwd=tcp::$SSH_PORT-:22 -net nic,model=virtio \
    $QEMU_ARGS -nographic
```

`$QEMU_ARGS` is left unquoted on purpose: bash splits it into the `-device` word and its options. zsh does not split an unquoted variable, and QEMU then stops with `invalid option`; in zsh write `${=QEMU_ARGS}`, or run the command under bash. In the guest, `sudo nvme list` shows the 1.72 GB BlackBox drive of tutorial 01.

5.11.7 What you learned

- A configuration file names the same properties as `-device femu`, with `mode` for `femu_mode` and a `[subsys]` section for FDP.
- `ssd-config.sh` checks names against the binary in `FEMU_BIN`; QEMU checks values when it starts.
- List values need no doubled commas in the file.

5.11.8 Next

- Scripts and tools (§C.4) describes `ssd-config.sh` and the shipped files.
- The parameter manual (§5.1) explains what to put in a configuration, group by group.

5.12 Performance tuning

FEMU's timing model computes when each command should complete, and a poller thread posts the completion once the host clock passes that time (timing model (§6.2.1)). The model gives a lower bound: if a poller does not get a host CPU when a completion is due, the guest waits longer than the model says. Tuning

is mostly about giving FEMU's threads the CPUs and memory they need, so that the latency you measure is the model's and not the host's.

5.12.1 Threads and cores

These threads run inside QEMU:

Thread name	How many	Started	Uses a core
`CPU N/KVM`	one per guest vCPU (`-smp`)	at start	while the vCPU runs
`femu-poller`	1, or `ceil(queues / poller_ratio)` with `multipoller_enabled=1`	when the guest first enables the controller	always: it spins while the controller is enabled
`FEMU-FTL-Thread`	one per controller with a BlackBox, ZNS or CSD namespace	at start	always: it spins while the controller is enabled
`femu-cxl-ftl`	one per `femu-cxl-ssd` with `ftl=on` (the default)	at start	only while it serves a miss
`femu-cxl-cca`	one per `femu-cxl-ssd` with `cca=on`	at start	only while it serves a command
`femu-csd-cu`	`nr_cu` per controller with a CSD namespace	at start	only while it runs a program

Linux shows these names only when QEMU runs with `-name NAME,debug-threads=on`. The `run-*.sh` launchers pass it, for example `-name "FEMU-BBSSD-VM",debug-threads=on` in `run-blackbox.sh`. Without it, every thread is called `qemu-system-x86`, so add it to any QEMU command line of your own.

Plan one host core for every poller and FTL thread, on top of one per vCPU. `run-blackbox.sh` (4 vCPUs, one poller, one FTL thread) needs at least 6 cores to itself; 8 is comfortable. When cores are short, the first sign is latency above the configured NAND time and run-to-run variation.

5.12.2 Pollers and queues

Properties: queues, pollers and interrupts (§A.1.1).

Setting	Pollers	Use it when
`multipoller_enabled=0` (default)	1, serving every I/O queue	You measure latency at low queue depth, or have few spare cores
`multipoller_enabled=1`, `poller_ratio=1`	one per I/O queue	You need throughput from several guest jobs and have a core per queue
`multipoller_enabled=1`, `poller_ratio=R`	`ceil(queues / R)`, each serving R queues	You need more than one poller but have fewer spare cores than queues

Other values of `multipoller_enabled` are refused at start. `poller_ratio=0` counts as 1.

The number of pollers comes from the `queues` property (default 8), not from the number of queues the guest creates. Linux creates about one I/O queue pair per guest CPU, up to `queues`. A poller whose queues the guest never created still spins. So set `queues` to the guest's vCPU count when you turn on more pollers. With 4 vCPUs and a poller per queue:

```
-device femu,devsz_mb=4096,femu_mode=1,queues=4,multipoller_enabled=1
```

What you trade:

- One poller walks every queue on each pass. It needs only one core, but its pass gets longer with each busy queue, and all completions share it.
- A poller per queue keeps each queue's path short and scales with guest jobs, at the cost of one spinning core per queue.
- `poller_ratio` sits in between. Each poller serves its queues round-robin: poller i serves queues i, i + P, i + 2P and so on, where P is the number of pollers.

5.12.3 Pin the threads

Unpinned, the scheduler moves vCPUs and pollers between cores and lets them share a core with other work. Pin them to separate cores. With `debug-threads=on` on the command line (see above), boot the guest, then on the host:

```
pid=$(pgrep -x qemu-system-x86)
ps -T -p "$pid" -o tid=,comm=
```

Pin the vCPUs with the QMP helper that `femu-copy-scripts.sh` copies to `build-femu/ftk/`. This puts vCPU 0 to 3 on host CPUs 0 to 3:

```
sudo ./ftk/qmp-vcpu-pin -s ./qmp-sock 0 1 2 3
```

Then give each poller and the FTL thread a core of its own, here starting at host CPU 4. The pollers exist only after the guest has enabled the controller, so run this after the guest has booted:

```
cpu=4
for tid in $(ps -T -p "$pid" -o tid=,comm= |
             awk '$2 == "femu-poller" || $2 == "FEMU-FTL-Thread" {print $1}'); do
    sudo taskset -pc "$cpu" "$tid"
    cpu=$((cpu + 1))
done
```

Before you pin anything, move every existing QEMU thread off the cores you reserve for FEMU, for example with `sudo taskset -apc 8-15 "$pid"`, then pin the vCPUs and FEMU's threads as above. A thread created later inherits the affinity of the thread that creates it. Choose cores that are not hardware-thread siblings of each other (`lscpu -e` shows the core of each CPU), so that two spinning threads do not share one physical core.

`pin.sh` does all of this in one step. From `build-femu/`, after the guest has booted:

```
./pin.sh 4      # vCPUs, then pollers and FTL threads, from host CPU 4
```

It gives each vCPU, `femu-poller`, `FEMU-FTL-Thread`, `femu-cxl-ftl` and `femu-cxl-cca` a CPU of its own, starting at the CPU you name (default 0), and moves the other QEMU threads, `femu-csd-cu` included, to the CPUs after those. It finds the threads by name, so it stops with an error when QEMU runs without `debug-threads=on`. Set `QEMU_PID` when more than one QEMU runs (scripts reference (§C.7)).

5.12.4 Hugepages

FEMU's device memory is ordinary anonymous memory; FEMU does not request hugepages for it (a host with transparent hugepages set to `always` may still use them). You can back the guest's RAM with hugepages, which reduces TLB misses in the guest. Reserve them on the host (2048 pages of 2 MiB for a 4 GiB guest):

```
echo 2048 | sudo tee /proc/sys/vm/nr_hugepages
grep HugePages_Free /proc/meminfo
```

Then, in the launcher, replace `-m 4G` with these options:

```
-m 4G -object memory-backend-memfd,id=ram0,size=4G,hugetlb=on,prealloc=on \
-machine memory-backend=ram0
```

If the host has too few free hugepages, QEMU stops at start with `unable to map backing store for guest RAM: Cannot allocate memory`. CI does not test this setup.

`femu-cxl-ssd` with `der=cylon` needs its CXL memory on a shared, preallocated hugetlb backend; see the CXL SSD guide (§4.18.7).

5.12.5 NUMA

On a host with more than one NUMA node, a thread that reads memory on the other node pays the inter-socket latency on every access. Each NVMe command copies its data between the guest's RAM and FEMU's device memory, so keep the vCPUs, the pollers, the guest RAM and the device memory on one node, unless you mean to study the other case.

To keep the whole QEMU process on node 0, start the launcher under `numactl`:

```
sudo numactl --cpunodebind=0 --membind=0 ./run-blackbox.sh
```

To place only the device memory, set `FEMU_MBE_INTERLEAVE` in QEMU's environment: `0` or `1` binds it to that node, `on` interleaves it across nodes 0 and 1 (environment variables (§A.1.5)). The launchers other than `run-cxlssd.sh` start QEMU with `sudo`, which drops your environment, so put the variable on the launcher's `sudo` line:

```
sudo FEMU_MBE_INTERLEAVE=1 ./qemu-system-x86_64 \
```

FEMU prints `backend: N MB bound via FEMU_MBE_INTERLEAVE=1` when the binding worked. One layout that keeps the guest and the emulator from competing for memory bandwidth puts the vCPUs and guest RAM on one node, and the pollers, the FTL thread and the device memory on the other.

5.12.6 Host settings

- **CPU frequency.** A core that changes frequency changes how long FEMU's own work takes. Set every core to the performance policy: `sudo cpupower frequency-set -g performance`, or `sudo ../femu-scripts/set_cpu_perf_mode.sh`.
- **Locked device memory.** Under `sudo`, FEMU locks its device memory so that page faults do not add latency. As a normal user it needs `ulimit -l unlimited` or a matching `/etc/security/limits.conf` entry; without it FEMU prints `cannot pin the N MiB memory backend` and runs with less precise latency.
- **Other work.** Keep other busy processes off the cores you gave FEMU. The `isolcpus=` kernel parameter keeps the scheduler from placing other tasks there.
- **Physical host.** Nested virtualization and WSL add their own delays (requirements (§2.1.1)).
- **Priority.** `sudo nice -n -10 ./run-blackbox.sh` starts QEMU, and every thread it creates, at a higher scheduling priority.

Inside the guest, services that wake up on their own add noise to a measurement, and a guest that suspends stops it. For example:

```
sudo systemctl disable cups bluetooth
sudo systemctl mask sleep.target suspend.target
```

```
cat /sys/block/nvme0n1/queue/scheduler          # the guest's I/O scheduler
echo mq-deadline | sudo tee /sys/block/nvme0n1/queue/scheduler
```

The guest's I/O scheduler queues requests before they reach FEMU, so state which one a measurement used.

5.12.7 What each knob trades

Knob	Gains	Costs
`multipoller_enabled=1`	throughput that scales with guest jobs	one spinning core per poller
`poller_ratio` above 1	fewer cores used	longer passes, more completion delay per poller
`queues`	one queue per guest CPU, no queue sharing in the guest	with `multipoller_enabled=1` and `poller_ratio=1`, each extra queue adds a spinning poller
Pinning	stable latency, no migrations	cores reserved for FEMU
Guest RAM on hugepages	fewer TLB misses in the guest	memory reserved up front
One NUMA node	no cross-socket copies	that node's cores and memory bandwidth only
Performance CPU frequency	stable service times	power and heat
Locked device memory	no page faults in the I/O path	memory resident from start-up

5.12.8 Related pages

- Measuring (§6.1): repeatable numbers once the host is tuned
- Timing model (§6.2)
- Security and limits: host sizing (§5.13.4)
- Requirements (§2.1)

Related issues: #7, #69, #77, #93, #101.

5.13 Security and limits

What a guest can do to the host through FEMU, what QEMU features FEMU does not support, how device properties change between versions, and how much host memory a device takes. To report a vulnerability, follow SECURITY.md[98].

5.13.1 Security model

FEMU runs inside the QEMU process on the host. Everything a FEMU device does, it does with QEMU's privileges. The NVMe `run-*.sh` launchers start QEMU with `sudo`, so by default that means root; `run-cxlssd.sh` runs QEMU as you.

Treat the guest as trusted unless you have turned off every feature below. For a guest you do not trust:

- Run QEMU as a normal user in the `kvm` group, not with `sudo`. FEMU then cannot pin the device memory unless you raise the memory lock limit (requirements (§2.1.1)). It starts anyway and prints `[FEMU] Err: cannot pin the N MiB memory backend`, which is a warning, not a failure.
- Do not use CSD mode with `csd_program_dir`.

[98] `https://github.com/MoatLab/FEMU/blob/master/SECURITY.md`

- Keep `lsa-control` off on `femu-cxl-ssd`.
- Do not put a FEMU controller behind a virtual IOMMU. Its pollers copy guest memory without the BQL, but an IOMMU walk can take it: to report a translation fault by MSI (AMD-Vi), or to read page tables the guest put in device memory. If a controller reset holds the BQL meanwhile, the VM hangs.

CSD programs

In CSD mode (`femu_mode=4`), the guest can ask the device to load a program by file name. For a shared-library program, QEMU opens that file with `g_module_open()` and runs its code inside the QEMU process. An eBPF program is read from the same directory and run by the uBPF library, when FEMU is built with it.

FEMU limits which files the guest can name:

- With `csd_program_dir` unset (the default, and what `run-csd.sh` does), only the built-in program type loads.
- With it set, a name must be a file in that directory. FEMU refuses an empty name, a name containing `/`, the names `.` and `..`, and a name whose path leaves the directory after symbolic links are resolved.

This keeps the guest away from the rest of the host file system. It does not make the programs in the directory safe: any shared object there runs as QEMU, which is root under the launchers. Put only programs you built in the directory, and make sure no other user can write to it. See the CSD guide (§4.13.2).

CXL SSD control channel

`femu-cxl-ssd,lsa-control=on` lets the guest send experiment commands through the CXL Get LSA mailbox command (CXL SSD guide (§4.18.8)). With it on, the guest can:

- create and write `cxlssd-stats.log`, `cxlssd-io-N.log` and `cxlssd-spt.log` in `log-dir`, which is QEMU's working directory when unset;
- write the host's `trace` and `tracing_on` files in `tracefs-dir`, which clears the host trace buffer and turns host tracing on or off, when `tracefs-dir` is set;
- flush and reconfigure the device cache, reset the statistics, and set or remove a direct-mapping ratio.

Each file is capped at `log-limit` (64 MiB by default) and FEMU reuses a fixed set of 64 I/O log names. With the statistics and mapping dump files, a guest can therefore make FEMU write about 66 times `log-limit` (about 4.1 GiB with the default) in `log-dir`, and no more.

`lsa-control` is off by default on the device. `run-cxlssd.sh` turns it on, because Cylon's scripts use it. Turn it off for a guest you do not trust:

```
LSA_CONTROL=off ../femu-scripts/run-cxlssd.sh
```

The same commands stay available from the host through QMP (`control-command`), whatever `lsa-control` says.

Other host-side files and output

These are chosen on the host, not by the guest:

- `pel_file` keeps the Persistent Event Log in a host file that QEMU creates if it is missing.
- `FEMU_DUMP_LPN` (BlackBox) hex-dumps the contents of one logical page to QEMU's standard error on every read, which `run-blackbox.sh` copies into its log file. `FEMU_EXP_LOG` traces only the addresses of pages that contain the `FEMU_SECRET` marker. Leave `FEMU_DUMP_LPN` unset when the guest's data is private.

5.13.2 Migration and snapshots

FEMU devices cannot be migrated or saved. Both `femu` and `femu-cxl-ssd` mark their state unmigratable, so QEMU refuses these operations and the guest keeps running:

Operation	What QEMU reports
QMP `migrate`	`State blocked by non-migratable device '0000:00:04.0/femu'`
HMP `savevm NAME`	`Error: State blocked by non-migratable device '0000:00:04.0/femu'`
Either, with a CXL SSD	`State blocked by non-migratable device '<path>/femu-cxl-ssd'`
`-only-migratable` with a FEMU device	`Device femu is not migratable, but --only-migratable was specified,` and QEMU does not start

The part before `/femu` is the device's PCI address, so it changes with your command line.

Device data lives only in host memory:

- A guest reboot or a controller reset keeps the data.
- QEMU exiting, for any reason, loses it. FEMU never writes an NVMe namespace to a file, and there is no option to do so. (`femu-cxl-ssd` keeps its data in the memory backend you give it; a file-backed `memory-backend-file` leaves the bytes in that file.)
- A snapshot of the guest's boot disk (for example with `qemu-img`) does not include the FEMU device.

To start every run from the same state, write the data again after boot, or run a fixed preconditioning workload. Related issue: #52.

5.13.3 Property compatibility

FEMU's device properties are its configuration interface, but they are not versioned: FEMU has no machine-type compatibility settings for them, so a command line means what the current code says it means.

Values refused at realize

Recent versions check more property values when the device is created and stop QEMU with a message, where older versions accepted the value and then misbehaved. A command line that used to start may therefore stop. The most recent additions:

Property	Now refused
`femu_mode`	above 5
`multipoller_enabled`	other than 0 or 1
`lver` (OCSSD)	other than 1 or 2
`flash_type` (OCSSD)	outside 1 to 4
`zns_chnls_per_zone`	a value that does not divide `zns_num_ch`
`zns_luns_per_zone`	a value that does not divide `zns_num_lun`
`zns_zone_order`	a value other than `ch` or `lun`

The message names the property, for example:

```
qemu-system-x86_64: -device femu,devsz_mb=64,femu_mode=7: femu_mode must be 0 (OpenChannel), 1 (black-box), 2
    (no-SSD), 3 (zoned), 4 (computational storage) or 5 (key-value)
```

The changelog (§D.1.3) lists these and other refusals, and the changes that move numbers without stopping a

run. Most mode guides list their own refusals under "Limits and refusals"; the CXL SSD guide has them under "Limits".

Properties that are accepted but do nothing

These are kept so that old command lines still start. Nothing reads them, so setting them changes nothing the guest can observe. A value other than the default prints one warning at realize, for example `warning: femu: dlfeat has no effect and is accepted only for compatibility`:

Property	Why
`serial`	Identify Controller reports a serial number FEMU generates
`ms`	the metadata size comes from `meta` (the warning says so)
`dlfeat`	Identify Namespace always reports 0x9
`ms_max`	OCSSD 2.0 reports a single LBA format (NLBAF 0), and no other mode reads it
`nr_thread`, `time_slice`, `context_switch_time` (CSD)	accepted for CEMU configurations; CSD still refuses `nr_thread=0`

The property reference (§A.1) says this in each description, and `-device femu,help` prints the same text.

5.13.4 Host sizing

Memory

A FEMU device keeps its whole capacity in host DRAM. For each device, plan for:

- **The backend**: `devsz_mb` MiB, or the raw NAND capacity with `op_pcent`. Under `sudo`, FEMU locks it in memory at start-up, so all of it is resident from the first second. Without the lock (a normal user with the default limit), pages become resident as the guest first touches them.
- **FTL tables**: allocated and filled at start-up. With the default BlackBox geometry (16 GiB of NAND in 4 KiB pages), QEMU's resident size was 116 MiB against 35 MiB for a NoSSD device of the same size, so the FTL took about 80 MiB. It grows with the number of NAND pages.
- **The guest**: its `-m` size.
- **QEMU itself**: the requirements page budgets about 1 GiB.

The requirements page (§2.1.1) gives the totals for each launcher. If the allocation is more than the kernel will commit, QEMU aborts at start with `failed to allocate N bytes`.

In BlackBox, CSD and KV mode the NAND geometry as a whole (every namespace plus spare space) may hold at most 2^31 - 1 sectors of `secsz` bytes, just under 1 TiB with the default 512-byte sectors.

Hugepages

FEMU allocates its backend from ordinary anonymous memory, not hugetlbfs, so the NVMe modes need no hugepages. You can back the guest's RAM with hugepages to reduce TLB misses in the guest; see performance tuning (§5.12.4). `femu-cxl-ssd` with `der=cylon` is the exception: it needs a shared, preallocated hugetlb memory backend (CXL SSD guide (§4.18.7)).

CPU

Each `femu-poller` thread, and the `FEMU-FTL-Thread` of a controller with a BlackBox, ZNS or CSD namespace, spin on a host core while the guest has the controller enabled. Plan one core for each of them on top of the

guest's vCPUs (performance tuning (§5.12.1)).

5.13.5 Related pages

- Architecture (§3.1)
- Performance tuning (§5.12)
- Troubleshooting (§7.1)
- Changelog (§D)

6 Measuring and validation

6.1 Measuring

How to read FEMU's counters, measure latency and throughput from the guest, and get numbers that repeat from run to run. Commands in `sh` blocks run inside the guest unless they say otherwise. From the host, prefix a single command with `./run-guest-ssh.sh` as in the quick start (§2.4); run multi-line blocks in a shell inside the guest (`./run-guest-ssh.sh` with no arguments).

Before you measure anything, tune the host as in performance tuning (§5.12). Most surprising numbers come from a FEMU thread that did not get a core.

6.1.1 What FEMU can tell you

Source	Read with	Modes	What it holds
Vendor log page C0h	`nvme get-log --log-id=0xc0`	BlackBox (with or without FDP), CSD, KV	WAF, host, GC and NAND page counts, write buffer hits, read reclaim, log-block merges
SMART log	`nvme smart-log`	every NVMe mode	data read and written, command counts, Percentage Used, Available Spare, media errors
FDP statistics	`nvme fdp stats`	BlackBox with FDP	host and media bytes written
`query-femu`, `query-femu-stages`	QMP on the host, or `femu-stat.py` (§6.1.4)	BlackBox (with or without FDP), ZNS; NAND timing for CSD, KV, OCSSD	host, NAND, GC and pad pages, WAF, line, reclaim unit and zone counts, I/O stage times
QOM counters	QMP `qom-get` on the host	`femu-cxl-ssd`	cache hits and misses, media reads, programs and time, direct mapping
fio	in the guest	every block mode	latency and throughput as the guest sees them

ZNS, NoSSD and OCSSD leave the C0h counters at zero. ZNS has no device garbage collection: the host resets zones.

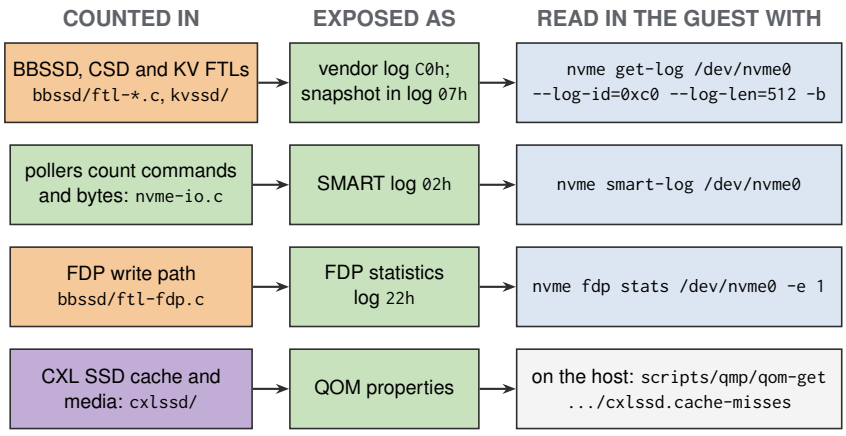
The guest's own tools show I/O from the guest's side: `iostat -x 1` per device, `iotop` per process, `dstat -cdn` for CPU, disk and network together.

6.1.2 Write amplification and media counters (C0h)

The first 4 bytes of log page C0h are the write amplification factor (WAF) times 1000. The three 8-byte counters from byte 8 are pages the host wrote, pages garbage collection moved, and pages programmed for the host:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u8 -j 8 -N 24 -w24
```

FEMU computes the WAF as `(pages programmed + pages GC moved) x 1000 / pages the host wrote`. The counters are summed over every BlackBox, CSD and KV namespace of the controller, and they count from the moment QEMU started the device (or from when namespace management created the namespace). The WAF in the log is therefore the WAF since start, not the WAF of your last run. To measure one run, read the counters before and after it and use the differences:



C0h, little endian: bytes 0 to 3 hold the WAF times 1000, (NAND pages + GC pages) × 1000 / host pages; 8-byte host, GC and NAND page counts follow at bytes 8, 16 and 24, then the most reads of any block since its erase, read reclaims, retention refreshes, write buffer and merge counts. It stays zero in ZNS, NoSSD and OCSSD modes. fio in the guest gives the latency and bandwidth the guest sees.

Figure 26: Where FEMU's counters come from and how you read them: FTL media counters in vendor log C0h, command and byte counts in the SMART log, FDP statistics, and CXL SSD counters as QOM properties on the host.

```
c0() {
    sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b |
        od -An -t u8 -j 8 -N 24 -w24
}
c0 > before.txt
sudo fio --name=run --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randwrite --bs=4k --iodepth=32 --size=4G
c0 > after.txt
paste before.txt after.txt | awk '{ h = $4 - $1; g = $5 - $2; n = $6 - $3;
    printf "host %d  gc %d  nand %d  WAF %.3f\n", h, g, n, (n + g) / h }'
```

Pages the write buffer absorbs (`buffer_size`) count as host pages but not as programmed pages until they leave the buffer, so with a buffer the WAF of a short run can be below 1. The other fields (read reclaim, write buffer hits, log-block merges) are listed in log pages and counters (§B.1).

GC starts when `gc_thres_pcent` (75% by default) of the lines are in use. A WAF of 1.000 after a run means GC has not run yet, or found no valid pages to move. To make GC run, write more than the free space, as in the BlackBox guide (§4.4.4).

6.1.3 SMART and FDP statistics

```
sudo nvme smart-log /dev/nvme0
```

FEMU fills these SMART fields:

- `data_units_read`, `data_units_written`: thousands of 512-byte units the host moved, rounded up.
- `host_read_commands`, `host_write_commands`.
- `percentage_used`: wear of the most worn namespace, from erase counts and `pe_cycles_rated` (or the rating of `nand_cell_type`); 0 when neither is set.
- `available_spare`: the worst namespace, from `nand_bad_blocks`.

- `media_errors`, `num_err_log_entries`, `temperature` (the `temperature` property), power-on hours and unsafe shutdowns.

With FDP, the endurance group statistics give host and media bytes written. Their ratio is the WAF over the endurance group, every FDP namespace of the subsystem:

```
sudo nvme fdp stats /dev/nvme0 -e 1
```

6.1.4 Live counters on the host: femu-stat

`hw/femu/scripts/femu-stat.py` shows the FTL counters of a running VM from the host, like `top`. It needs no guest tools. It polls the QMP commands `query-femu`[99] and `query-femu-stages`[100] on the QMP socket of QEMU. It never resets the counts.

```
 femu-stat.py --connect--> QMP socket of QEMU
      every --interval s:  query-femu          per namespace state
                           query-femu-stages   I/O stage histograms
      this sample - last sample --> view (or one JSON line)
```

Start QEMU with a QMP socket. `io_stages=on` adds the stage times:

```
-device femu,id=femu0,devsz_mb=1024,femu_mode=1,io_stages=on -qmp unix:/tmp/femu-qmp.sock,server=on,wait=off
```

Then, on the host:

```
python3 hw/femu/scripts/femu-stat.py /tmp/femu-qmp.sock
python3 hw/femu/scripts/femu-stat.py /tmp/femu-qmp.sock --once
python3 hw/femu/scripts/femu-stat.py localhost:4444 --json --count 60 > stat.jsonl
```

This is a BlackBox namespace after the guest wrote 200 pages in one second:

```
femu-stat  2026-10-14T05:40:20  interval 1.00 s  /tmp/femu-qmp.sock
/machine/peripheral/femu0  bbssd
  ns 1 bbssd
    total:    host 264  nand 264  gc 0  pad 0  erases 0
    interval: host +200  nand +200  gc +0  pad +0  erases +0  (host 200.0/s, gc 0.0/s, erases 0.0/s)
    WAF: 1.000 since start, 1.000 interval
    lines: free 255 of 256, victim 0, full 0, retired 0, spare 0
    media: bus phases not charged
  stages (interval):
    stage          count      mean         p50         p99      max*
    submit           200     2.4us     <=2.0us     <=8.2us    41.0us
    ftl              200     829ns     <=1.0us     <=2.0us    94.9ms
    hold             200   196.8us   <=197.8us   <=197.8us   197.8us
    post-delay       200     235ns     <=255ns     <=1.0us   274.1us
    total            200   200.2us   <=262.1us   <=262.1us    94.9ms
    past-due         200     235ns     <=255ns     <=1.0us    94.7ms
    * max since the counts were reset; p50 and p99 are bucket bounds
```

What each namespace shows depends on what `query-femu` returns for it:

[99] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md

[100] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md#io-stage-times-query-femu-stages

Mode	Shown
`bbssd`	host, NAND, GC and pad pages and block erases, with the change over the interval; host, GC and erase rates; WAF since start and over the interval; line counts
`bbssd` with FDP	the same counters, reclaim unit counts, and host and media bytes per handle with the WAF of each handle over the interval
`znssd`	zone counts by state, and the open and active zone limits
`csd`, `kvssd`, `ocssd`	the NAND bus timing only
`nossd`, or a namespace on a `femu-cxl-ssd` medium	the error that `query-femu` gives; the other namespaces still show
a mode or counter that the tool does not know	its members by their QMP names

Read the figures as follows:

- The interval WAF is (NAND + GC + pad pages) / host pages over the interval. It is - when the host wrote no page in the interval.
- Host pages count NAND pages. A 512-byte write to one page counts as one page, so the page rate is not a byte rate.
- p50 and p99 are the bounds of the histogram bucket that holds them. A bucket spans a factor of two. The tool caps the upper bound at the largest time counted, so a bound can be narrower than its bucket. The last bucket has no upper bound of its own; without the cap it shows as >=.
- `max*` is the largest time since the stage counts were last reset. The histograms do not give the largest time of one interval.
- The first sample has no interval. Its figures are totals since the device started, or since the last reset of the stage counts.
- A counter that goes down starts a new baseline for its group: the namespace counters, the stages, or one FDP handle. This detection is best effort. A namespace that is deleted and created again between two samples can show a wrong interval.
- Each namespace and each command is a separate copy, so two namespaces can show slightly different times. The stage counts of a busy device can differ by the commands in flight.
- A `bbssd` namespace reports nothing until the guest enables the controller.

Options: `--interval SEC` (default 1), `--count N`, `--once`, `--json` (one object per sample on one line), `--ns 1,2` (namespaces to show) and `--path QOMPATH` (one device, repeatable). With `--json`, the keys are the QMP member names. The tool exits with status 2 when it cannot connect, finds no `femu` device, or loses the connection. On a terminal it redraws the screen; in a pipe it prints one block per sample.

6.1.5 CXL SSD counters

The `femu-cxl-ssd` counters are QOM properties, read on the host through QMP. Add a QMP socket to the `run-cxlssd.sh` command line (`-qmp unix:/tmp/qmp.sock,server=on,wait=off`), then on the host, from the QEMU source tree:

```
export QMP_SOCKET=/tmp/qmp.sock
scripts/qmp/qom-get /machine/peripheral/cxlssd.cache-misses
scripts/qmp/qom-get /machine/peripheral/cxlssd.media-time-ns
scripts/qmp/qom-get /machine/peripheral/cxlssd.media-full
```

`cxlssd` is the `id=` that `run-cxlssd.sh` gives the device.

- `stats-reset` clears the cache, prefetch and caching API (`cca-*`) event counters. It copies the cache and prefetch counters to the `last-*` properties first. It does not clear the media counters (`media-reads`, `media-writes`, `media-time-ns`) or the `der-*` counters, so measure those as differences between two reads.
- `media-full` must stay 0. A non-zero value means some NAND programs found no free page and were not timed, and the run is not valid. The over-provisioning rule at realize keeps it at 0.
- `gc-stalls` and `gc-stall-ns` show how often and how long writes waited for forced garbage collection. Take differences, as for the media counters. `gc-stall-max-ns` is the longest single wait. Waits of a second or more come from the NAND timing or queued NAND work; more `blocks-per-plane` make them shorter.
- Accesses served through a direct mapping (`der=memslot` or `cylon`) never reach QEMU and are not counted as hits.

The full list is in runtime properties (§A.2.2).

6.1.6 Latency and throughput with fio

Use `--direct=1` so the guest page cache does not answer for the device. Report the completion latency (`clat`) percentiles, not only the mean.

NoSSD

NoSSD has no media time, so it measures FEMU's own path. Unwritten blocks read as fast as written ones.

```
sudo fio --name=lat --filename=/dev/nvme0n1 --direct=1 --ioengine=io_uring \
    --rw=randread --bs=4k --iodepth=1 --runtime=30 --time_based
sudo fio --name=tput --filename=/dev/nvme0n1 --direct=1 --ioengine=io_uring \
    --rw=randread --bs=4k --iodepth=64 --numjobs=4 --group_reporting \
    --runtime=30 --time_based
```

For throughput beyond one poller, see pollers and queues (§5.12.2).

BlackBox and CSD

A read of a page that was never written costs no NAND time. Fill the range first, then read it:

```
sudo fio --name=fill --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=write --bs=128k --iodepth=16 --size=4G
sudo fio --name=rr --filename=/dev/nvme0n1 --direct=1 --ioengine=libaio \
    --rw=randread --bs=4k --iodepth=1 --size=4G --runtime=30 --time_based
```

On an idle device at queue depth 1, a 4 KiB read takes about `pg_rd_lat` (40 us by default) plus the guest's and the poller's overhead, and a 4 KiB write about `pg_wr_lat` (200 us) when there is no write buffer. Measure the overhead on a NoSSD device with the same guest and subtract it.

To see GC in the latency, age the device first: random writes over the whole namespace, several times, until the C0h WAF stops rising. Then measure. The tail percentiles (`clat` 99th and above) show GC; the median mostly does not.

ZNS

Write sequentially inside zones, and reset the zones between runs:

```
sudo fio --name=zns --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=write --bs=128k --size=1G
```

```
sudo fio --name=zr --filename=/dev/nvme0n1 --direct=1 --ioengine=psync \
    --zonemode=zbd --rw=randread --bs=4k --size=1G --runtime=30 --time_based
sudo blkzone reset /dev/nvme0n1
```

A write costs 1 us per 4 KiB page while it fits in the zone's write cache; the write that fills the cache pays for programming it (timing model (§6.2.6)).

FDP

fio places data with its `io_uring_cmd` engine on the generic node. Run the same job with and without `--fdp=1` and compare the WAF:

```
sudo fio --name=fdp --filename=/dev/ng0n1 --ioengine=io_uring_cmd --cmd_type=nvme \
    --fdp=1 --fdp_pli=0,1,2,3 --rw=randwrite --bs=4k --iodepth=16 --size=4G
```

KV, OCSSD and CXL

- KV has no block device, and fio has no key-value engine. Time the passthrough commands in your own program; `kv-probe.c` is a starting point (KV guide (§4.11.4)).
- OCSSD needs a guest kernel older than 5.15 with pblk, or SPDK (OCSSD guide (§4.9)).
- For the CXL SSD, run your workload on the DAX device or NUMA node of the region (CXL SSD guide (§4.18.5)), and read the QOM counters above before and after.

6.1.7 Repeatable numbers

1. **Fix the host.** Pin vCPUs, pollers and the FTL thread to their own cores, set the CPU frequency policy to performance, and keep other work off those cores (performance tuning (§5.12)).

2. **Record the setup.** The FEMU commit, the `-device` line (`run-blackbox.sh`, `run-blackbox-fdp.sh` and `run-csd.sh` print it; `DRY_RUN=1 ../femu-scripts/run-cxlssd.sh` prints the CXL command), the guest kernel and the fio job file. A number without these cannot be compared.

3. **Start from a known state.** Restart QEMU between configurations. The device memory and every counter start from zero when QEMU starts; a guest reboot or controller reset keeps both. A namespace created with namespace management starts its counters at zero, and deleting one removes its counts from C0h. Then precondition: fill the range you will read, and for steady-state write numbers, overwrite at random until the WAF stops changing.

4. **Measure intervals, not totals.** Read C0h, SMART or the QOM counters before and after each run and use the differences.

5. **Run long enough, more than once.** Use `--time_based` with a `--ramp_time`, run each point at least three times, and report the median and the spread.

6. **Check FEMU kept up.** On a BlackBox controller, vendor command 0xEF with CDW10 5 prints to QEMU's console how many completions were posted 20 us or more after they were due, out of all completions, and resets both counts. `run-blackbox.sh` copies the console to `build-femu/log`. Its standard output goes through a pipe there, so the line can appear only later; put `stdbuf -oL` in front of `./qemu-system-x86_64` on the launcher's `sudo` line to see it at once. Reset the counts before a run, read them after:

   ```
   sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=5
   ```

The line ends with the two counts as `A/B`. If A is more than a small share of B, the pollers did not get enough CPU and the latencies are longer than the model.

7. **Keep the timing switches in mind.** 0xEF codes 2 and 4 turn GC time and NAND time off until you turn them back on with 1 and 3, and code 3 restores the built-in times, not the ones on your command line (timing model (§6.2.11)). Restart QEMU after using them.

6.1.8 Related pages

- Timing model (§6.2)
- Log pages and counters (§B)
- Runtime properties (§A.2)
- Performance tuning (§5.12)

Related issues: #7, #15, #92, #130, #137, #151.

6.2 Timing model

FEMU makes an emulated SSD take as long as a real one would. This page explains how it computes that time, how it makes the guest wait for it, which properties change it, and how to check what it did. The architecture page (§3.1) shows where each step runs.

6.2.1 Compute, then hold

FEMU does not slow down the data copy. An NVMe command's data is copied between guest memory and the memory backend as soon as the command is parsed. Separately, a model computes when the command would have finished on the emulated device, and the poller does not post the completion before that time:

1. The poller stamps the command with the current host time, `stime`, read from `QEMU_CLOCK_REALTIME`. Its completion time starts equal to it.
2. The mode's model adds the command's latency to the completion time.
3. The poller keeps completed commands in a priority queue ordered by completion time and posts each one once the host clock passes it (`nvme_process_cq_cpl()` in `hw/femu/nvme-io.c`).

Two things follow from this:

- Time in the model is host wall-clock time, and the guest sees the same clock. A 200 us program makes the guest wait 200 us.
- The model gives a lower bound. A completion is posted on the first poller sweep after it is due, so a poller that does not get a host CPU (too few cores, no pinning) adds delay on top. Give each `femu-poller` and the `FEMU-FTL-Thread` its own core when you measure latency.

The CXL SSD works differently: the vCPU thread that made the access waits out the media time itself before the access completes (see CXL SSD (§6.2.10)).

6.2.2 Which mode charges what

Mode	Time charged for	Computed in
NoSSD	nothing	n/a

Mode	Time charged for	Computed in
BBSSD	NAND reads, programs and erases; write buffer and read cache hits; DFTL mapping misses; GC; deallocate (`trim_lat_ns`)	`FEMU-FTL-Thread`
CSD	as BBSSD, plus program run time on a compute unit and copies into device memory	FTL thread; compute units on the `femu-csd-cu` threads; copies in the poller
ZNS	NAND reads, programs and erases; write cache accesses	`FEMU-FTL-Thread`
KV	NAND reads and programs for values, value GC and erases; the hash index adds one page read per command, the LSM index its meta page reads	poller
OCSSD	NAND reads, programs and erases, optional channel transfer	poller

Optional host-link and firmware-CPU models apply on top of any NVMe mode (see Host link and controller firmware (§6.2.9)).

6.2.3 NAND operations

`nand_media_op()` in `hw/femu/nand/nand-media.c` computes one read, program or erase. BBSSD, CSD, KV, ZNS and both Open-Channel versions use it.

Operation times

The array time of an operation comes from one of two sources:

- **Flat times** (BBSSD, CSD, KV with `nand_cell_type=0`, the default): every read takes `pg_rd_lat`, every program `pg_wr_lat`, every erase `blk_er_lat`, in nanoseconds. Defaults are 40 us, 200 us and 2 ms. `pgtype_lat` with `cell_pages` scales the program time by the page's position in its wordline (lower to upper).
- **Cell-type tables**: `nand_cell_type` 1 to 4 (SLC, MLC, TLC, QLC) uses the built-in per-page-type read and program times and erase time in `hw/femu/nand/nand.h`, and ignores the flat times.
- **ZNS** uses one built-in value per cell type for `zns_flash_type` (`hw/femu/zns/zns.h`; SLC, TLC and QLC have one), unless `zns_pg_rd_lat`, `zns_pg_wr_lat` or `zns_blk_er_lat` overrides it.

Properties: NAND timing (§A.1.1), ZNS (§A.1.1).

Parallelism

The media layer keeps a busy-until time for each unit that can work on its own:

- **BBSSD, CSD, KV**: one per LUN. An operation starts at the later of its own start time and the LUN's busy-until time, and the LUN is busy until the operation ends. All planes of a LUN are busy together. Erasing a whole line (GC, FDP GC and KV reclaim) with `pls_per_lun` > 1 erases the same block on every plane of a LUN as one operation, which takes one erase time instead of one per plane.
- **ZNS**: one per plane, so the planes of a LUN work in parallel.

Different LUNs (or ZNS planes) never wait for each other. A command that covers several pages issues them all at its start time, and its latency is that of the slowest page. By default the BBSSD write pointer moves across channels first and then LUNs, so consecutive pages of a large write fall on different LUNs and are programmed in parallel. With `plane_first`, pages in a row fill the planes of one LUN first.

Properties: NAND geometry (§A.1.1).

Channel bus

The channel bus is off by default: transfers between controller and NAND take no time and channels never contend. It is turned on when any bus phase has a non-zero time:

Phase	BBSSD, CSD, KV	ZNS
Command and address	`cmd_addr_lat`	`zns_cmd_addr_lat`
Page data transfer	`pg_xfer_lat`, or `ch_xfer_lat` when `pg_xfer_lat` is 0	`zns_pg_xfer_lat`
Status read	`status_lat`	`zns_status_lat`

With the bus on, phases on one channel run one at a time. A program sends its command and data over the bus before the array starts. A read sends its command, waits for the array, then moves its data out; the data-out is booked for when it will happen, so other LUNs can use the bus in the meantime.

Program and erase suspend, ECC

- `pe_suspend` (`zns_pe_suspend` for ZNS) lets a read go ahead of a program or erase that is running on its LUN (plane for ZNS). The first such read pays `tsusp_ns` (`zns_tsusp_ns`); the suspended operation then ends later by `tsusp_ns` plus the time of every read that went ahead of it.
- `ecc_step_ns` adds read time for worn or old blocks: one step per 750 erases of the block, plus one per `ecc_retention_sec` seconds since its line was filled, at most four steps.

Properties: NAND timing (§A.1.1), Reliability and wear (§A.1.1).

6.2.4 BBSSD FTL costs

On top of the NAND operations, the BBSSD FTL (also used by CSD) charges:

- **Write buffer** (`buffer_size` > 0, in pages): a write the buffer accepts costs one DRAM access, `pg_rd_lat` / 16 (2.5 us with the defaults). When the buffer passes `buffer_thres_pcent`, the write that pushed it there also pays for programming a batch of buffered pages. A read of a buffered page costs the same DRAM access and does not reach NAND. Flush programs everything buffered and pays for it. FUA writes and stream writes skip the buffer.
- **Read cache** (`read_cache_mb` > 0): a hit costs a DRAM access instead of a NAND read.
- **Mapping cache** (`mapping=dftl`): a miss in the cached mapping table of `mapping_cache_mb` adds a NAND read of the translation page, after a NAND program if the entry it replaces is dirty.
- **Deallocate**: `trim_lat_ns` per Dataset Management range.

Properties: Garbage collection, mapping and caches (§A.1.1).

6.2.5 Garbage collection

GC moves valid pages out of a victim line and erases it. Its NAND reads, programs and erases go through the same per-LUN timelines as host commands, so a host command that needs a LUN that GC is using waits for it. That is how GC shows up in host latency.

- **Background GC** runs one line after a request, on the FTL thread, when the share of lines in use has reached `gc_thres_pcent` (default 75).
- **Foreground GC** runs inside a write when the share has reached `gc_thres_pcent_high` (default 95), and repeats until it drops below. The write waits for the LUNs it needs, which GC has just made busy.
- `gc_policy` picks the victim line. Write amplification depends on it and on how full the device is.

GC time can be switched off at run time with the vendor command below; GC still moves the pages, but its NAND operations then take no time.

ZNS has no device GC: the host resets zones, and a reset charges the erase of the zone's blocks.

6.2.6 ZNS write cache

A ZNS write goes into a per-zone SRAM write cache and costs 1 us per 4 KiB page. When the cache is full, or another zone needs it, the cached pages are programmed across the zone's planes and the write that triggered it pays for the program. `zns_num_wc` sets the number of caches (default: `zns_max_open`, or 3 when that is 0).

6.2.7 OCSSD

Both Open-Channel versions keep their busy-until times in the shared media layer above, with the LUN rule. Open-Channel 1.2 keeps one per chip (LUN) and one per channel; Open-Channel 2.0 keeps one per LUN, with the bus off. A write first moves its data over the channel, then programs the chip; a read occupies the chip, then moves its data out. Open-Channel 1.2 charges channel transfer only with `oc12_channel_timing=on`, using `ch_xfer_lat` per page or the `flash_type` table value when that is 0. Open-Channel 2.0 charges no channel time. Read, program and erase times come from the `flash_type` table (SLC, MLC, TLC, QLC) and can be changed at run time with vendor admin command 0xEE. `hw/femu/ocssd/oc-timing.c` copies the table times at init and handles 0xEE.

Properties: OCSSD (§A.1.1).

6.2.8 KV and CSD

KV computes its latency in the poller with the BBSSD NAND model. It reads and programs value pages. The Store that triggers value GC and reclaim erases pays for them. With the default hash index, every command pays one page read for the index lookup, on a LUN chosen by the key's hash. A command completes when its last NAND operation does.

With `kv_timing=model`, Store and Retrieve take their time from a fitted workload-mix model instead (KV mode (§4.11.3)).

With `kv_index=lsm`, the index is an LSM tree. A lookup reads one meta page per unpinned level, in sequence, before the value, and pays no hash base read. Merges load the LUNs in the background (LSM-tree key index (§4.23.5)).

Small values share pages (`kv_pack=on`, the default). A Store goes to an open page in DRAM and costs `pg_rd_lat / 16`; the Store that does not fit, or that fills the page, programs it. A Retrieve reads the one page of its value, or nothing while the value is still in DRAM (KV value packing (§4.24.5)).

CSD runs a program on the compute unit that frees up first. A program holds its unit for its declared run time, or for its measured host run time multiplied by the program's own scale, or by `csf_runtime_scale` when it gives none. `nr_cu` sets how many units run at once. A copy from the namespace into device memory costs `pg_rd_lat` once. The completion cannot come before the host has run the program. The opt-in `csd_pause` stops the guest and its clocks for the run, which hides the host run (CSD (§4.13.4)).

Properties: CSD (§A.1.1).

6.2.9 Host link and controller firmware

Three optional models apply to every NVMe mode. They are added in the poller after the media time and before the completion is queued:

- `pcie_bandwidth_mbps`: each Read and Write is charged its size divided by this bandwidth, on one queue per direction, so transfers in the same direction wait for each other.
- `pcie_prop_delay_ns`: a fixed delay added to each Read and Write after its transfer.
- `fw_cpu_ns`: a fixed cost per Read, Write and Zone Append on one modelled controller core, which caps the command rate at about one per `fw_cpu_ns`.

All three are 0 (off) by default. With any of them on, NoSSD also completes through the priority queue instead of inside the sweep.

Properties: Host link and controller firmware (§A.1.1).

6.2.10 CXL SSD

A `femu-cxl-ssd` access is timed on the vCPU thread that made it:

Access	Media time
Cache hit	none
Read miss	one NAND page read; nothing if the FTL has never mapped the page
Write miss	one NAND page read to fill the page, which then becomes dirty
Eviction of a dirty page	one NAND page program, charged to the access that caused it
Prefetched page	none for the page itself; a dirty page it evicts is written back at the access's cost
Any access with `cache-pages=0`	a NAND read or program for every access
Access through a direct mapping (`der=memslot` or `cylon`)	none, and it is not counted; it never reaches QEMU. A `memslot` mapped page counts as dirty, so its eviction costs a program

The access hands the page to the `femu-cxl-ftl` worker, which runs the same BBSSD FTL and NAND model as above and returns the latency. The vCPU then waits out what is left of it with the BQL released: it sleeps until 100 us before the deadline and spins for the rest. Other properties:

- Geometry and NAND times: `channels`, `luns-per-channel`, `pages-per-block`, `blocks-per-plane`, `read-ns`, `program-ns`, `erase-ns`, `channel-ns`, and GC thresholds `gc-threshold`, `gc-threshold-high`.
- `ftl=off` charges no media time at all.
- `cylon-first-touch-program=on` charges a program instead of a free read on the first access to an unmapped page; `cylon-free-writeback=on` makes dirty write-backs free. Both reproduce published Cylon experiments and are off by default.
- `concurrent-misses` decides whether misses to different pages wait for the media together or one after another.

`flush-cache` (QMP) and cache control commands on BAR5 also wait for the media time of the write-backs they cause, on the thread that runs them.

`fast-load=true` removes only the wait at the end of an access; the FTL and the NAND timelines still advance. Setting it back to false does not wait for that backlog: the next accesses wait behind it. `nand-idle-ns` reads 0 once the latest LUN and channel busy-until time has passed, so a harness polls it before it measures. See Fast load (§4.18.6).

Properties: femu-cxl-ssd cache (§A.1.3), NAND geometry and timing (§A.1.3).

6.2.11 Changing timing at run time

A BBSSD controller (`femu_mode=1`) accepts the vendor admin command 0xEF, with the action in CDW10. It applies to every namespace of the controller that has a BBSSD FTL. From the guest:

```
sudo nvme admin-passthru /dev/nvme0 --opcode=0xef --cdw10=2
```

CDW10	Effect
1	GC NAND operations take time (the default)
2	GC NAND operations take no time
3	Set the flat read, program and erase times to the built-in 40 us, 200 us and 2 ms
4	Set the flat read, program and erase times to 0
5	Print the number of completions posted 20 us or more after they were due and the total number of completions to QEMU's standard output, then reset both
6, 7	Turn per-command debug logging on or off; it prints only in a build with `FEMU_DEBUG_NVME` defined

Code 3 restores the built-in values, not the ones given on the command line. Codes 3 and 4 change only the flat times: channel bus phases, the `nand_cell_type` tables, `ecc_step_ns` and `trim_lat_ns` stay in force, and write buffer and read cache hits still cost 1 us when `pg_rd_lat` is 0. For a controller linked to a CXL SSD, it restores the medium's `read-ns`, `program-ns`, `erase-ns` and `channel-ns` instead.

6.2.12 Measuring

- **Latency seen by the guest**: run `fio` with a direct I/O engine and read the completion latency (`clat`). With an idle device and queue depth 1, a 4 KiB random read on BBSSD should take about `pg_rd_lat` plus guest and poller overhead. Write the range first: a read of a page that was never written is not charged any NAND time.
- **Write amplification and media counters**: the vendor log page C0h holds the write amplification factor, host and NAND page counts, GC copies, write buffer hits and more. Read it with `sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b`. The layout is in log pages and counters (§B.1).
- **Health**: `sudo nvme smart-log /dev/nvme0` reports Percentage Used and Available Spare from the wear model, and host read and write totals.
- **CXL SSD**: the device's QOM counters, read with `qom-get` through QMP, include `media-time-ns` (total media time charged), `media-reads`, `media-writes`, `cache-hits`, `cache-misses` and the direct mapping counters. `media-full` must stay 0 for a valid measurement. See runtime properties (§A.2.2).

6.2.13 Related pages

- Architecture (§3.1)
- Choosing a mode (§4.1)
- Device property reference (§A.1)
- CXL SSD design note[101]

[101] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/cxlssd.md`

7 Troubleshooting

7.1 Troubleshooting and FAQ

Answers to the questions that come up most often in FEMU's issue tracker. Each one gives a short answer, the fix or workaround, and the issues where it came up. For build errors, see build.md (§2.2.6). For crashes and debug output, see debugging (§7.2).

1. Which device and mode do I want? (§7.1.1)
2. Which guest kernel does each mode need? (§7.1.2)
3. Where do I get a guest image? (§7.1.3)
4. The build fails (§7.1.4)
5. How large an SSD can I emulate? (§7.1.5)
6. How do I change the geometry or latency without recompiling? (§7.1.6)
7. How do I set the ZNS zone size, zone count and limits? (§7.1.7)
8. How do I choose the NAND cell type? (§7.1.8)
9. Why are reads impossibly fast? (§7.1.9)
10. How do I see garbage collection and write amplification? (§7.1.10)
11. What does FEMU model, and what not? (§7.1.11)
12. How do I run several SSDs or namespaces? (§7.1.12)
13. How do I run FDP? (§7.1.13)
14. Can I still use OCSSD after LightNVM was removed? (§7.1.14)
15. How do I get debug output? (§7.1.15)
16. How do I tune for performance? (§7.1.16)
17. Where is the code for X, and how do I add a command? (§7.1.17)
18. Does data survive a reboot? (§7.1.18)

7.1.1 Which device and mode do I want?

`-device femu` is an NVMe SSD; `femu_mode` picks what is inside it. With no `femu_mode`, you get NoSSD (`femu_mode=2`): an NVMe drive with no media timing. For a conventional SSD with an FTL, garbage collection and write amplification, use BlackBox (`femu_mode=1`, `run-blackbox.sh`). ZNS is `femu_mode=3`, Open-Channel `0`, computational storage `4`, key-value `5`. FDP is not a mode: it is BlackBox with a `femu-subsys,fdp=on`. A CXL memory-semantic SSD is a different device, `femu-cxl-ssd`.

Fix: pick from the decision table in choosing a mode (§4.1), then follow that mode's guide.

Related issues: #21, #36, #60, #68, #153.

7.1.2 Which guest kernel does each mode need?

Mode	Guest kernel
NoSSD, BlackBox, CSD, FDP	any with the NVMe driver
ZNS	5.9 or newer, with `CONFIG_BLK_DEV_ZONED=y`
KV	6.0 or newer; the namespace appears only as `/dev/ngXnY`
OCSSD	older than 5.15, with LightNVM; 1.2 needs 4.16, 2.0 needs 4.17
CXL SSD	CXL region support (`CONFIG_CXL_REGION` and related options)

The image from `make-guest-image.sh` runs Linux 6.8 and covers every mode except OCSSD. Build it with `--cxl` for the CXL SSD tools.

Fix: if `nvme list` shows the controller but no usable namespace, check `uname -r` in the guest against this table and `dmesg | grep nvme`. requirements.md (§2.1.2) has the details.

Related issues: #20, #57, #64, #112, #115, #161.

7.1.3 Where do I get a guest image?

Build one. `make-guest-image.sh` makes an Ubuntu 24.04 image from the official cloud image, with a `femu` user, an SSH key, `nvme-cli` and `fio`, and writes it where the launchers look for it, `$HOME/images/u20s.qcow2`:

```
cd build-femu
./make-guest-image.sh
```

Fixes for older instructions:

- `qemu-system-x86_64: -localtime: invalid option`: QEMU removed `-localtime` in version 3.1. Use `-rtc base=localtime`.
- A launcher says `VM disk image couldn't be found`: set `OSIMGF` to your image, or `IMGDIR` to its directory.
- WSL is not supported, and in WSL the guest may use only one or two cores. Use a Linux host.

See guest-image.md (§2.3).

Related issues: #1, #36, #88, #89, #93.

7.1.4 The build fails

FEMU is based on QEMU 10.1, which needs Python 3.9 and GLib 2.66 or newer, so Ubuntu 20.04 and older cannot build it with their own packages.

Fixes for the common errors:

- `-Werror` stops the build on a new compiler warning: add `--disable-werror` to the `configure` line, and report the warning.
- Errors around `nfs_pread_async`: older FEMU trees failed against libnfs 6, whose API changed. The current QEMU base ignores libnfs 6, and `femu-compile.sh` passes `--disable-libnfs` as well.
- `Cannot find Ninja` or `python venv creation failed`: install `ninja-build` and `python3-venv`.
- Errors such as `memfd_create` declared twice, or syntax errors in the ZNS code, were reported against older FEMU versions. Build current `master`.

The full table is in build.md (§2.2.6).

Related issues: #2, #56, #136, #150, #168.

7.1.5 How large an SSD can I emulate?

As large as the host's free memory. FEMU keeps the whole device in host DRAM, so a 256 GiB device needs 256 GiB of free host RAM, plus the guest's RAM and the FTL tables. In BlackBox, CSD and KV mode the NAND geometry as a whole (every namespace plus spare space) may hold at most 2^31 - 1 sectors of `secsz` bytes, just under 1 TiB with the default 512-byte sectors; a larger geometry is refused at start.

FEMU allocates the whole device at start-up and tries to lock it in memory. Under `sudo` the lock normally succeeds, so all of it must be free when QEMU starts. If the allocation itself is more than the kernel will commit, QEMU aborts with `failed to allocate N bytes`. If only the lock fails, FEMU prints `[FEMU] Err: cannot pin the N MiB memory backend` and runs anyway.

Fix: for NoSSD and ZNS, set `devsz_mb`. For BlackBox, CSD and KV, also size the geometry (`nchs`, `luns_per_ch`, `blks_per_pl` and so on) to hold the namespace plus spare space; with `op_pcent`, the host memory used is the full NAND capacity. Host sizing (§5.13.4) explains the memory a device takes.

Related issues: #19, #33, #52, #73, #144; discussion #119.

7.1.6 How do I change the geometry or latency without recompiling?

Every setting is a device property on the QEMU command line. Nothing needs a code change, and the old `vssd1.conf` file is gone.

Fix:

- Edit the variables at the top of the launcher (`run-blackbox.sh` and others), or write the `-device femu,...` line yourself.
- Or describe the device in a config file and expand it with `ssd-config.sh` (scripts reference (§C.4)).
- BlackBox: `pg_rd_lat`, `pg_wr_lat`, `blk_er_lat` (ns) and the geometry properties. ZNS: `zns_pg_rd_lat`, `zns_pg_wr_lat`, `zns_blk_er_lat` and `zns_num_ch`, `zns_num_lun`, `zns_num_plane`, `zns_num_blk`.
- `qemu-system-x86_64 -device femu,help` lists every property, and the property reference (§A.1) explains each one.

This example sets a 4 GiB BlackBox device with 50 us reads:

```
-device femu,devsz_mb=4096,femu_mode=1,pg_rd_lat=50000,pg_wr_lat=500000,blk_er_lat=3000000
```

Related issues: #19, #21, #73, #193.

7.1.7 How do I set the ZNS zone size, zone count and limits?

There is no zone size property. The zone size follows from the namespace size and `zns_num_ch`, `zns_num_lun`, `zns_num_plane`, `zns_num_blk` and `zns_chnls_per_zone`. Raising `zns_num_blk` gives more, smaller zones. Linux needs a power-of-two zone size.

Fix:

- Use the formula and table in the ZNS guide (§4.7.3).
- For 4 KiB logical blocks, set `lba_index=3`; no code change is needed.
- `zns_max_open` and `zns_max_active` set the open and active zone limits (0 means no limit).

```
-device femu,devsz_mb=4096,femu_mode=3,lba_index=3,zns_num_ch=8,zns_num_lun=4,zns_num_plane=2,zns_num_blk=128,zns_max_active=16,zns_max_open=8
```

Related issues: #76, #126, #144, #177, #182.

7.1.8 How do I choose the NAND cell type?

Each mode has its own numeric property. Names such as `tlc` are not accepted:

Mode	Property	Values
BlackBox, CSD, KV	`nand_cell_type`	0 flat times (default), 1 SLC, 2 MLC, 3 TLC, 4 QLC

Mode	Property	Values
ZNS	`zns_flash_type`	1 SLC, 3 TLC, 4 QLC; 2 MLC and 5 PLC only with explicit `zns_pg_rd_lat`, `zns_pg_wr_lat` and `zns_blk_er_lat`
OCSSD	`flash_type`	1 SLC, 2 MLC, 3 TLC, 4 QLC

`Property 'femu.cell_type' not found` means the name is wrong: use `nand_cell_type`. FEMU has no mode that mixes cell types, such as an SLC cache in front of TLC.

```
-device femu,devsz_mb=1024,femu_mode=1,nand_cell_type=4
```

Related issues: #50, #113, #132.

7.1.9 Why are reads impossibly fast?

In BlackBox and CSD, a read of a page that was never written has no mapping, so it costs no NAND time and returns zeroes. A read benchmark on a fresh device therefore measures only FEMU's overhead. Old versions printed `ppn[-1] not mapped` for such reads; current versions print nothing.

Fix: write the range before you read it, for example with a sequential fio write job, as in measuring (§6.1.6).

Related issues: #16, #92.

7.1.10 How do I see garbage collection and write amplification?

Read the vendor log page C0h in the guest. Its first 4 bytes are the write amplification factor times 1000, followed by the host, GC and NAND page counts:

```
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
```

The old SMART vendor bytes no longer hold these counters. GC starts when 75% of the lines are in use (`gc_thres_pcent`), so on a fresh device you must write more than the free space before the WAF rises above 1.000. Random overwrites leave valid pages in every line, which GC must move; sequential overwrites free whole lines, so the WAF stays near 1.

Fix: see measuring (§6.1.2) for per-run WAF and the BlackBox guide (§4.4.4) for a workload that makes GC run.

Related issues: #130, #137.

7.1.11 What does FEMU model, and what not?

FEMU models:

- NAND read, program and erase times per LUN (per plane for ZNS), with optional channel bus, program and erase suspend, and ECC retry time;
- the BlackBox FTL: page or cached mapping, GC, write buffer, read cache, wear and errors;
- optional PCIe bandwidth, propagation delay and controller firmware time (`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`, `fw_cpu_ns`).

It does not model:

- the time to copy data between guest memory and the device; the copy happens at once, and the model only delays the completion;

- persistence: the device lives in host memory;
- power, heat or temperature changes (the reported temperature is the `temperature` property).

The latency is a lower bound. A completion is posted on the first poller pass after it is due, so a poller without a host core adds delay. The guest's own driver and block layer add time that the model does not include.

Fix: read the timing model (§6.2), and measure FEMU's own overhead on a NoSSD device before you compare against the model.

Related issues: #7, #15, #69, #151.

7.1.12 How do I run several SSDs or namespaces?

For several SSDs, add one `-device femu` per SSD, each with its own `id` and `devsz_mb`. For several namespaces on one controller, set `namespaces`. `namespace_sizes` and `namespace_modes` give each namespace its own size and mode; double the commas inside them on the QEMU command line:

```
-device femu,id=nvme0,devsz_mb=4096,femu_mode=1,namespaces=2,namespace_sizes=3G,,1G -device femu,id=nvme1,devsz_mb=1024,femu_mode=3
```

The `serial` property has no effect, and setting it prints a warning at realize. Each controller reports a serial number made of a mode prefix (`vSSD`, `vZNSSD`, `vNoSSD` and so on) and that mode's own counter, which advances once for each namespace the mode sets up. In the example above the serials are `vSSD1` and `vZNSSD0`. Names under `/dev/disk/by-id` therefore depend on the order of the `-device` options.

Fix: see several namespaces and devices (§4.15). OCSSD controllers have one namespace each. FDP controllers take several BBSSD or NoSSD namespaces (FDP (§4.5.3)).

Related issues: #26, #121.

7.1.13 How do I run FDP?

FDP is configured on an NVMe subsystem, not on the controller. Create a `femu-subsys` with `fdp=on` first, then a BlackBox controller that joins it. `run-blackbox-fdp.sh` does both.

```
-device femu-subsys,id=subsys0,nqn=subsys0,fdp=on,fdp.nruh=4 -device femu,devsz_mb=4096,femu_mode=1,subsys=subsys0
```

In the guest, `sudo nvme fdp configs /dev/nvme0 -e 1` shows the configuration. See the FDP guide (§4.5).

Related issues: #153.

7.1.14 Can I still use OCSSD after LightNVM was removed?

Linux removed LightNVM, its Open-Channel driver, in 5.15. On a newer guest kernel the OCSSD controller appears, but nothing in the kernel can use it.

Fix: use a guest kernel older than 5.15 with `CONFIG_NVM` and pblk, or drive the device from user space with SPDK. Open-Channel 2.0 in current FEMU also accepts plain NVMe Read and Write, which SPDK uses. nvme-cli 2.x dropped `nvme lnvm`; build nvme-cli 1.x in the guest for it. See the OCSSD guide (§4.9).

Related issues: #4, #48, #139.

7.1.15 How do I get debug output?

FEMU prints lines that start with `[FEMU]` (`Log:`, `Err:`, `FTL-Log:`, `FTL-Err:`) and, for the ZNS FTL, `[Misao] ZFTL-` on QEMU's console. `run-blackbox.sh`, `run-zns.sh` and `run-csd.sh` save the console in `build-femu/`

log; run-blackbox-fdp.sh writes /tmp/femu-fdp.log. There is no FEMU_DEBUG environment variable, and FEMU defines no QEMU trace events.

Fix: compile the debug messages in with --extra-cflags="-DFEMU_DEBUG_NVME -DFEMU_DEBUG_FTL", or use the run-time variables such as FEMU_FDP_DEBUG. See debugging (§7.2).

Related issues: #18, #105; discussion #133.

7.1.16 How do I tune for performance?

Give every FEMU thread a core. The femu-poller threads and, with a BlackBox, ZNS or CSD namespace, the FEMU-FTL-Thread spin while the guest has the controller enabled, so each needs a host core of its own, on top of the vCPUs. One poller serves every queue by default; multipoller_enabled=1 starts one per queue, or one per poller_ratio queues.

Fix: follow performance tuning (§5.12): pin the vCPUs, the pollers and the FTL thread, set the CPU frequency policy to performance, and keep the threads on one NUMA node. pin.sh pins the vCPUs, the pollers and the FTL threads (CXL ones included) in one step.

Related issues: #69, #77, #101.

7.1.17 Where is the code for X, and how do I add a command?

All FEMU code is under hw/femu/. Architecture (§3.1) maps each layer to its files: femu.c creates the device, nvme-admin.c and nvme-io.c handle NVMe commands, and each mode has its own directory (bbssd/, zns/, ocssd/, nossd/, kvssd/, csd/, cxlssd/).

To add a command:

- An I/O command for every mode: add a case to nvme_io_cmd() in nvme-io.c. Opcodes it does not handle go to the namespace's mode, for example bb_io_cmd() in bbssd/bb.c.
- An admin command: add it to nvme_admin_cmd() in nvme-admin.c, or to the mode's admin_cmd hook, as bb_admin_cmd() does for the vendor command 0xEF.
- To fail a command, return an NVMe status from the handler, for example return NVME_INVALID_FIELD | NVME_DNR;. A status found later, in the FTL, goes in req->status.

Add a qtest for it (testing (§8.2.7)).

Related issues: #72, #114, #162.

7.1.18 Does data survive a reboot?

A guest reboot keeps the data, unless you trigger simulate-power-loss on a power_loss=on device, which drops the write buffer. Stopping QEMU loses it. FEMU keeps every NVMe namespace in host memory and never writes it to a file; there is no option to make it persistent. Snapshots and migration are refused (security and limits (§5.13.2)).

Workaround: keep data you need on the guest's boot disk or copy it out before you stop QEMU, and recreate the device contents after each start.

Related issues: #52.

7.2 Debugging

Where FEMU reports problems, how to get more output, and how to run it under a debugger. For answers to common problems, see troubleshooting (§7.1).

7.2.1 Where messages go

FEMU prints its messages on QEMU's console:

Prefix	Stream	Meaning
`[FEMU] Log:`	stdout	information, such as a vendor command taking effect
`[FEMU] Err:`	stderr	an error FEMU recovered from, such as a CSD program that failed to load
`[FEMU] FTL-Log:`, `[FEMU] FTL-Err:`	stdout, stderr	the BlackBox FTL
`[Misao] ZFTL-Log:`, `[Misao] ZFTL-Err:`	stdout, stderr	the ZNS FTL
`[FEMU] FDP-Log:`, `[FEMU] FDP-Trace:`	stderr	FDP setup; placement and reclaim traces with `FEMU_FDP_DEBUG`
`qemu-system-x86_64: -device femu,...: MESSAGE`	stderr	the device refused its options and QEMU did not start

`run-blackbox.sh`, `run-zns.sh` and `run-csd.sh` copy the console to `build-femu/log`, and `run-blackbox-fdp.sh` to `/tmp/femu-fdp.log`. `run-nossd.sh`, `run-whitebox.sh` and `run-cxlssd.sh` print only to the terminal. Through the `tee` pipe, standard output is buffered, so stdout lines can reach the log later than stderr lines. Put `stdbuf -oL` in front of `./qemu-system-x86_64` on the launcher's `sudo` line to write them at once.

Check the guest's side too. A guest that gives up on the device logs it in `dmesg`:

```
sudo dmesg | grep -i nvme
```

FEMU defines no QEMU trace events and does not use QEMU's `-d` log categories, so `-d` and `-trace` show nothing from FEMU itself. They still help with the QEMU code around it. `-d guest_errors,unimp -D qemu.log` logs bad register accesses that QEMU's MSI-X and CXL code detect. Add them to the QEMU command line in the launcher:

```
-d guest_errors,unimp -D qemu.log
```

7.2.2 Run FEMU under gdb

Do not use `gdb-run.sh`; it starts QEMU's stock `nvme` device, not FEMU. Instead, run a launcher's own command line under gdb. From `build-femu/`, make a copy of the launcher that starts QEMU through gdb, and run it:

```
sed -e 's|\./qemu-system-x86_64|gdb -ex "handle SIGUSR1 nostop noprint pass" --args
    ./qemu-system-x86_64|' \
    -e 's/ 2>&1 | tee .*$//' run-blackbox.sh > gdb-blackbox.sh
bash gdb-blackbox.sh
```

At the gdb prompt, set breakpoints and start QEMU:

```
(gdb) break femu_realize
(gdb) run
```

KVM uses SIGUSR1 to kick vCPU threads, which is why gdb is told to pass it on. The guest's serial console shares the terminal with gdb; press Ctrl-C to get back to the gdb prompt.

To attach to a QEMU that is already running instead, on the host:

```
sudo gdb -p "$(pgrep -x qemu-system-x86)" -ex "handle SIGUSR1 nostop noprint pass"
```

When QEMU crashes, `thread apply all bt` in gdb prints every thread's stack. The launchers pass `-name NAME,debug-threads=on`, so `info threads` shows FEMU's thread names (`femu-poller`, `FEMU-FTL-Thread`, `femu-csd-cu`, `femu-cxl-ftl`, `femu-cxl-cca`); add it to a QEMU command line of your own (performance tuning (§5.12.1)).

7.2.3 Debug builds and compile-time switches

`femu-compile.sh` builds with `-O2 -g`. gdb works on that build, but many variables are optimized out. To single-step, configure an unoptimized build yourself from `build-femu/`:

```
../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp \
    --disable-libnfs --disable-libiscsi --disable-curl \
    --enable-debug --enable-debug-info
make -j"$(nproc)"
```

The debug messages (`Dbg:`) have no run-time switch. These macros, passed with `--extra-cflags`, compile them in:

Macro	Effect
`FEMU_DEBUG_NVME`	controller debug messages (`[FEMU] Dbg:`), and the per-command log that vendor command 0xEF with CDW10 6 and 7 turns on and off
`FEMU_DEBUG_FTL`	BlackBox FTL debug messages (`[FEMU] FTL-Dbg:`), and the FTL invariant checks of BlackBox and ZNS
`FEMU_DEBUG_ZFTL`	ZNS FTL debug messages (`[Misao] ZFTL-Dbg:`)
`FEMU_FTL_ASSERT`	the FTL invariant checks only, without the messages

For example:

```
../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp \
    --disable-libnfs --disable-libiscsi --disable-curl \
    --extra-cflags="-DFEMU_DEBUG_FTL -DFEMU_DEBUG_NVME"
make -j"$(nproc)"
```

The invariant checks (`ftl_assert`) are compiled out of a normal build. A mapping or bound error there corrupts FTL state quietly instead of stopping QEMU. When a BlackBox or ZNS run produces wrong data or impossible counters, rebuild with `-DFEMU_FTL_ASSERT` first. The debug messages print on the I/O path, so use them with small workloads.

The sanitizer build (§8.2.3) adds AddressSanitizer and UndefinedBehaviorSanitizer on top of the FTL checks. Run the failing workload on it to find memory errors.

7.2.4 Run-time switches

FEMU reads a few environment variables (environment variables (§A.1.5)):

Variable	Prints to stderr
FEMU_FDP_DEBUG (any value)	FDP placement and reclaim traces
FEMU_EXP_LOG=1 with FEMU_SECRET=TEXT	writes, overwrites, deallocations, GC moves and erases of pages whose data contains TEXT
FEMU_DUMP_LPN=N	a hex dump of logical page N on every read not served from the write buffer
FEMU_KV_SELFTEST (any value)	the result of a KV FTL self-test run once at start
FEMU_KV_CHECK (any value)	stops QEMU with a message when a KV page owner check fails

The launchers other than run-cxlssd.sh start QEMU with sudo, which drops your environment. run-blackbox.sh passes FEMU_EXP_LOG, FEMU_SECRET and FEMU_DUMP_LPN through, so this works:

```
FEMU_EXP_LOG=1 FEMU_SECRET=MARKER ./run-blackbox.sh
```

For the others, add the variable to the launcher's sudo line, for example sudo FEMU_FDP_DEBUG=1 ./qemu-system-x86_64 \ in run-blackbox-fdp.sh.

On a running BlackBox controller, the vendor admin command 0xEF switches GC time and NAND time on and off and prints poller counts (timing model (§6.2.11)).

7.2.5 Check a configuration without a guest

hw/femu/scripts/femu-config-check.py tells you, in about a second, whether FEMU accepts a set of device options, and what it builds from them. It needs no guest image and no KVM.

```
 options --> qemu-system-x86_64 -accel qtest (no guest)
               realize fails? --> FEMU's message               exit 1
               realize works  --> enable each controller (qtest)
                                  Identify, query-femu
                                  --> derived configuration    exit 0
```

Give the options as on the QEMU command line, or the value of one -device option. From build-femu/:

```
../femu-scripts/femu-config-check.py -device femu,femu_mode=1,devsz_mb=4096
```

```
femu-config-check: accepted
device /machine/peripheral-anon/device[0]  bbssd  PCI 00:02.0
  controller: FEMU BlackBox-SSD Controller, enabled with CC 0x460061 (4096 B memory page, command sets all), admin queues of 64 entries
  ns 1 bbssd, NVM command set: 4.0GiB (8388608 blocks of 512 B)
    NAND: 8 channels x 8 LUNs x 1 planes x 256 blocks x 256 pages x 4096 B = 16.0GiB
    over-provisioning: 300.0% (NAND / capacity - 1)
    lines: 256 of 64.0MiB, free 255, spare 0
    media: bus phases not charged
```

A configuration that FEMU refuses gives FEMU's message and exit status 1. The example uses a small geometry, because FEMU allocates the memory backend before it checks the size:

```
../femu-scripts/femu-config-check.py -device
    femu,femu_mode=1,devsz_mb=80,nchs=2,luns_per_ch=2,blks_per_pl=64,pgs_per_blk=64
```

```
femu-config-check: REFUSED
  qemu-system-x86_64: -device femu,femu_mode=1,devsz_mb=80,nchs=2,luns_per_ch=2,blks_per_pl=64,pgs_per_blk=64:
      FEMU bbssd: namespace 1 exposes 80 MiB of the 64 MiB this geometry has, leaving garbage collection no
      room; expose at most 60 MiB per namespace (lower devsz_mb, or set op_pcent)
```

FDP needs its femu-subsys, and ZNS takes the bare -device value:

```
../femu-scripts/femu-config-check.py -device femu-subsys,id=fdp0,fdp=on,fdp.nruh=4 -device
    femu,femu_mode=1,devsz_mb=4096,subsys=fdp0
```

```
../femu-scripts/femu-config-check.py femu,femu_mode=3,devsz_mb=4096
```

```
femu-config-check: accepted
device /machine/peripheral-anon/device[0]  znssd  PCI 00:02.0
  controller: FEMU ZMS-SSD Controller [by Misao], enabled with CC 0x460061 (4096 B memory page, command sets all), admin queues of 64 entries
  ns 1 znssd, zoned command set: 4.0GiB (8388608 blocks of 512 B)
    zones: 16 of 256.0MiB, capacity 256.0MiB, max open 0, max active 0
    media: bus phases not charged
```

The report shows, for each namespace:

Item	Source	Modes
mode, command set, capacity, block size	Identify	all
NAND geometry, NAND size, over-provisioning, line size, line counts	`query-femu` and Identify	`bbssd`
reclaim groups, units, unit size, handles	`query-femu`	`bbssd` with FDP
zone count, zone size and capacity, open and active limits	`query-femu`	`znssd`
NAND bus timing	`query-femu`	all except `nossd`
warnings that QEMU printed	stderr	all

Over-provisioning is NAND size / capacity - 1, per namespace. Each `bbssd` namespace has its own FTL and NAND.

How the check runs:

1. It drops the options that need a host resource or a monitor, for example KVM, `-cpu`, `-qmp`, `-serial` and `-kernel`.
2. It replaces each `-drive` by a null block device and each `-netdev` by user networking. The report lists each change.
3. It keeps `-machine`, `-m`, `-smp`, `-numa`, `-object` and `-global`. Without `-machine`, it uses `pc`.
4. It enables each controller as a guest driver does. It uses the smallest memory page the controller takes, and all command sets.

The configuration it reports is the one after the enable. A controller behind a PCI bridge (a root port, a CXL port) is not enabled. Then the check reports what it can and exits with status 2.

The exit status is one of these:

- 0: QEMU accepts the configuration and the controllers come ready.
- 1: QEMU or a controller refuses the configuration.
- 2: the check itself fails. A QEMU that dies of a signal is a check failure.

Options: `--json` prints one JSON object, `--qemu BINARY` names QEMU (default `./qemu-system-x86_64`, then `build-femu/` in the source tree), and `--timeout SEC` (default 60). Put `--` before QEMU options when you give tool options.

7.2.6 Inspect a running device

QEMU's monitor shows what QEMU built. With the QMP socket the launchers other than `run-cxlssd.sh` create (`build-femu/qmp-sock`), on the host:

```
printf '%s\n' '{"execute":"qmp_capabilities"}' '{"execute":"query-pci"}' |
    sudo socat - UNIX-CONNECT:./qmp-sock
```

`qom-list` and `qom-get` on `/machine/peripheral/<id>` read a device's properties and counters (runtime properties (§A.2)). `run-nossd.sh` names its controller `nvme0` and `run-cxlssd.sh` names its device `cxlssd`. The other launchers give the controller no `id=`, so add one to address it by name.

7.2.7 Common crash reports

Symptom	What it means	What to do
`qemu-system-x86_64: -device femu,...: MESSAGE` and QEMU exits	A property value was refused at start	Read the message; the mode guide's "Limits and refusals" table explains it
`[FEMU] Err: cannot pin the N MiB memory backend`	Not a crash. QEMU runs without sudo and the memory lock limit is too low	Ignore it, or raise `ulimit -l` (requirements (§2.1.1))
`failed to allocate N bytes`, then QEMU stops with a trap or abort and may dump core	The host has less free memory than the device needs	Lower `devsz_mb` or free memory (host sizing (§5.13.4))
Guest `dmesg` shows `nvme nvme0: I/O ... timeout`, then a controller reset	The guest's NVMe driver waited too long for a completion and reset the controller	Check the QEMU console for an error at the same time. If there is none, check that the pollers have cores (performance tuning (§5.12)). If it repeats on an idle host, report it
The device is missing in the guest, and the console shows `[FEMU] Err:` lines	FEMU refused a command or a configuration the guest used	Read the console; for ZNS, see the ZNS troubleshooting (§4.7.7)
QEMU exits with a segmentation fault or an assertion	A FEMU bug	Run it under gdb or the sanitizer build, and report it with the steps below

Older reports of crashes in FDP garbage collection (#186, #189, #191) and of controllers disabled under load (#22, #30) were made against older FEMU versions. Reproduce on current `master` before you report one.

7.2.8 Reporting a bug

Open an issue with:

- the FEMU commit (`git log -1 --oneline`) and how you built it;
- the full QEMU command line, or the launcher and every change you made to it;
- the guest kernel (`uname -r`) and the commands you ran in the guest;
- the QEMU console output and the guest's `dmesg`;
- for a crash, the `thread apply all bt` output from gdb, or the sanitizer report.

Report a security problem privately, as SECURITY.md[102] says.

7.2.9 Related pages

- Testing (§8.2)
- Troubleshooting (§7.1)
- Architecture (§3.1)

Related issues: #18, #22, #30, #105; discussion #133.

[102] `https://github.com/MoatLab/FEMU/blob/master/SECURITY.md`

8 Contributing

8.1 Code structure

All FEMU code, scripts, tests and documentation live under `hw/femu/`, so the rest of the tree stays QEMU. A top-level `femu-scripts` link points to `hw/femu/scripts/`, which keeps the `cd build-femu && ../femu-scripts/...` workflow working. How the pieces fit together at run time, layer by layer, is in architecture (§3.1).

8.1.1 Directory map

Path	What is there
`femu.c`[103]	QOM types `femu` and `femu-subsys`, property definitions, realize and exit, the FTL thread, mode registration
`femu-props.c`[104]	Help text for every `femu` and `femu-subsys` property (what `-device femu,help` prints)
`nvme.h`[105]	NVMe structures, the `femu_mode` enum and the controller state `FemuCtrl`
`nvme-admin.c`[106]	Admin commands, starting the pollers, namespace management, asynchronous events
`nvme-caps.c`[107]	What the controller advertises: the Commands Supported and Effects and Supported Log Pages entries, and the Identify bits derived from them
`nvme-io.c`[108]	I/O commands and the poller loop that fetches submissions and posts completions
`nvme-util.c`[109]	Deallocation state per LBA (TRIM, Write Zeroes with deallocate, DULBE), queue head and tail and completion posting helpers, poller pause and resume, the Timestamp feature
`nvme-pel.c`[110]	Persistent Event log and its `pel_file`
`nvme-pi.c`[111]	Metadata and protection information
`nvme-streams.c`[112]	Streams directive
`dma.c`[113]	PRP and SGL mapping, copies between guest memory and the device
`intr.c`[114]	MSI-X, MSI and pin interrupts
`bbssd/`[115]	BlackBox mode (`bb.c`) and its FTL: geometry, data path, mapping schemes, read cache, GC and lines, FDP, the bridge to the NAND media layer
`zns/`[116]	ZNS mode (`zns.c`) and its zone FTL (`zftl.c`)
`ocssd/`[117]	Open-Channel 1.2 (`oc12.c`) and 2.0 (`oc20.c`); their `flash_type` times, geometry check and vendor command 0xEE (`oc-timing.c`)
`nossd/`[118]	NoSSD mode (`nop.c`)

[103] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu.c
[104] https://github.com/MoatLab/FEMU/blob/master/hw/femu/femu-props.c
[105] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme.h
[106] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-admin.c
[107] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-caps.c
[108] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-io.c
[109] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-util.c
[110] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-pel.c
[111] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-pi.c
[112] https://github.com/MoatLab/FEMU/blob/master/hw/femu/nvme-streams.c
[113] https://github.com/MoatLab/FEMU/blob/master/hw/femu/dma.c
[114] https://github.com/MoatLab/FEMU/blob/master/hw/femu/intr.c
[115] https://github.com/MoatLab/FEMU/tree/master/hw/femu/bbssd
[116] https://github.com/MoatLab/FEMU/tree/master/hw/femu/zns
[117] https://github.com/MoatLab/FEMU/tree/master/hw/femu/ocssd
[118] https://github.com/MoatLab/FEMU/tree/master/hw/femu/nossd

Path	What is there
`kvssd/`[119]	Key-value mode: commands, its FTL, Identify and features
`csd/`[120]	Computational storage mode and its private commands
`cxlssd/`[121]	`femu-cxl-ssd`: QOM glue, the page cache, the DER modes, the caching API
`nand/`[122]	NAND media layer: per-cell-type timing tables and the timing of each operation
`backend/`[123]	The DRAM backend that holds the emulated medium
`lib/`[124], `inc/`[125]	Lock-free rings and the priority queue, and their headers
`scripts/`[126]	Build and launch scripts, configs, guest tools, documentation tooling (scripts reference (§C))
`tools/`[127]	Guest tools for the CXL caching API
`tests/`[128]	Unit tests, the qtest file, CSD guest tests (testing (§8.2))
`docs/`[129]	This documentation (doc map[130])

8.1.2 Making a change

CONTRIBUTING.md[131] has the process: style (`checkpatch.pl`), tests, sign-off and pull requests. For a new property, add its help text in `femu-props.c` and regenerate the property reference (keeping the documentation correct (§8.3)). For a new mode, add it to the `femu_mode` enum in `nvme.h`, register its handlers the way the existing modes do in `femu.c`, and add an entry to `docs/modes.py`.

8.2 Testing

FEMU has five kinds of tests. The first four run without a guest and run in CI; the last runs inside a booted guest.

Tests	Where	What they cover	Run with
Unit tests	`hw/femu/tests/unit/`	the NAND timing math, the completion priority queue, the hybrid mapping, the CXL cache, page table entries and caching API ring	`make -C hw/femu/tests check`, and `meson test --suite femu-unit` in a build; each runs a different subset
Device tests (qtest)	`hw/femu/tests/qtest/femu-test.c`	about 400 cases that drive the controller through its registers with no guest: every mode, admin and I/O commands, error paths, fuzzers, the CXL SSD	`qos-test`

[119] https://github.com/MoatLab/FEMU/tree/master/hw/femu/kvssd
[120] https://github.com/MoatLab/FEMU/tree/master/hw/femu/csd
[121] https://github.com/MoatLab/FEMU/tree/master/hw/femu/cxlssd
[122] https://github.com/MoatLab/FEMU/tree/master/hw/femu/nand
[123] https://github.com/MoatLab/FEMU/tree/master/hw/femu/backend
[124] https://github.com/MoatLab/FEMU/tree/master/hw/femu/lib
[125] https://github.com/MoatLab/FEMU/tree/master/hw/femu/inc
[126] https://github.com/MoatLab/FEMU/tree/master/hw/femu/scripts
[127] https://github.com/MoatLab/FEMU/tree/master/hw/femu/tools
[128] https://github.com/MoatLab/FEMU/tree/master/hw/femu/tests
[129] https://github.com/MoatLab/FEMU/tree/master/hw/femu/docs
[130] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/README.md
[131] https://github.com/MoatLab/FEMU/blob/master/CONTRIBUTING.md

Tests	Where	What they cover	Run with
Documentation checks	`hw/femu/scripts/`	the property reference, mode tables, links and every example in the docs	`make -C hw/femu/tests check-docs`
Host tool checks	`hw/femu/scripts/femu-stat.py`, `femu-config-check.py`	the host tools against QEMU under qtest, with planted faults	`make -C hw/femu/tests check-tools`
Configuration files	`hw/femu/scripts/ssd-config-test.sh`	every file in `scripts/configs/` starts QEMU	`ssd-config-test.sh BINARY`
Guest-side tests	`hw/femu/scripts/`, `hw/femu/tests/csd/`, `hw/femu/tools/cca/`	the device as Linux sees it, with blktests and the nvme-cli suite among them	inside the guest, or `guest-conformance.sh` from the host

CONTRIBUTING.md asks for a test that fails without your change and passes with it.

8.2.1 Build for testing

The qtests and the documentation checks need QEMU and `qos-test`. A debug build is the usual choice:

```
mkdir build && cd build
../configure --target-list=x86_64-softmmu --disable-docs --enable-debug --enable-slirp
ninja qemu-system-x86_64 tests/qtest/qos-test
```

8.2.2 Unit tests

Without a QEMU build, from the source tree:

```
make -C hw/femu/tests check
make -C hw/femu/tests clean
```

This compiles the tests that need no GLib test framework against a stub `qemu/osdep.h` and runs them, in about a second. The CXL cache test needs GLib development files and is skipped without them. In a configured build directory, the `femu-unit` meson suite runs every test in `hw/femu/tests/meson.build` against QEMU's real headers. CI runs both:

```
cd build
./pyvenv/bin/meson test --suite femu-unit --print-errorlogs
```

A new test goes in `tests/meson.build`, in `tests/Makefile`, or in both. `hw/femu/scripts/check-unit-tests.py` fails CI for a test source that is in neither list.

8.2.3 Device tests (qtest)

From the build directory, run every FEMU qtest:

```
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
    -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests
```

List them, or run one by its full path:

```
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -l |
    sed -n 's|^# \(.*femu-tests/.*\)|\1|p'
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
    -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests/cc-states
```

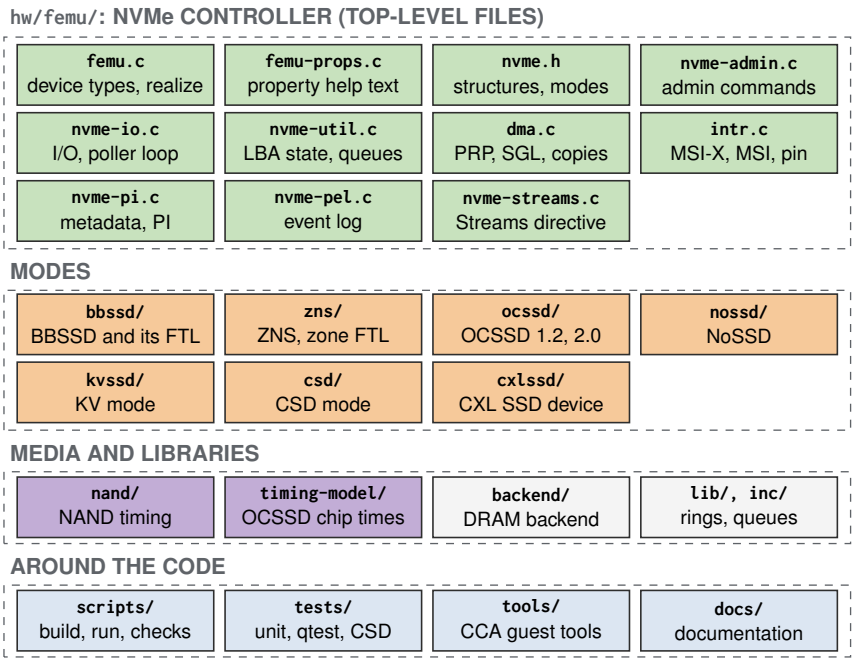


Figure 27: The FEMU source tree under `hw/femu/`: the NVMe controller files at the top level, one directory per mode, the shared media and library code, and the scripts, tests, tools and docs around them.

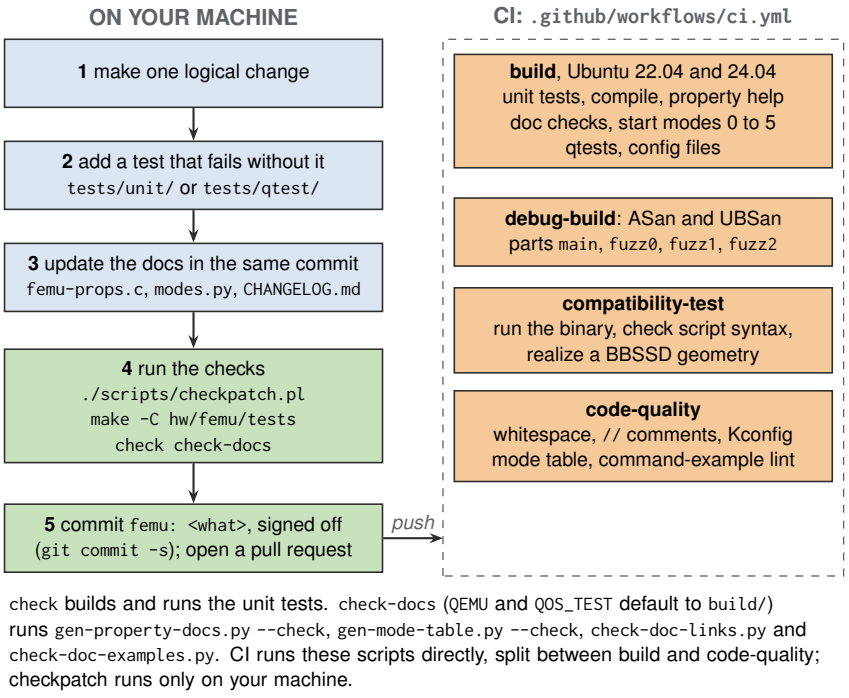


Figure 28: Contributing a change: one logical change with a failing test and its docs, checked locally with checkpatch, `make check` and `make check-docs`, then the four CI jobs run on the pull request.

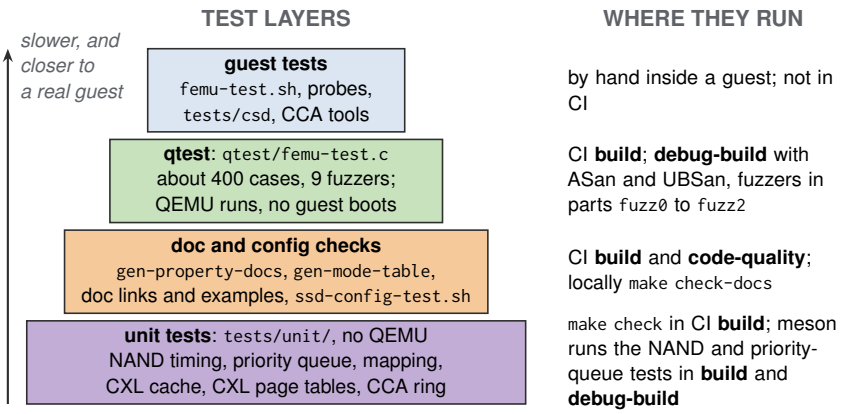


Figure 29: FEMU's test layers, fastest at the bottom: unit tests without QEMU, doc and config checks, qtests that run QEMU without a guest, and guest tests run by hand.

Each case starts its own QEMU under the qtest accelerator, so no KVM and no guest are needed. The default device is a 64 MiB NoSSD controller at PCI address 04.0; a case can add its own device options. The CXL cases start their own machine with a `femu-cxl-ssd`, which the `x86_64-softmmu` build includes by default.

Sanitizer build

CI also runs the unit tests and qtests in a build with AddressSanitizer, UndefinedBehaviorSanitizer and the FTL invariant checks on. A bound violation that the normal build silently survives aborts there:

```
mkdir build-debug && cd build-debug
../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp \
    --disable-libnfs --disable-libiscsi --disable-curl \
    --enable-asan --enable-ubsan --extra-cflags=-DFEMU_FTL_ASSERT
make -j"$(nproc)"
```

Run the qtests in that build with the sanitizer options CI uses. Without `abort_on_error=1`, a sanitizer report does not fail the test:

```
export ASAN_OPTIONS=detect_leaks=0:abort_on_error=1
export UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
    -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests
```

The fuzz cases (names containing `fuzz`) take most of the sanitized run time, and CI splits them over three jobs.

Each fuzz case uses a fixed seed, so CI sees the same inputs on each run. For a long run with new inputs, set two environment variables:

```
FEMU_FUZZ_SEED=N    each seed becomes its fixed value XOR N
FEMU_FUZZ_SCALE=K   each round count is multiplied by K (1 to 1000)
neither set         fixed seeds and default rounds, as in CI
```

The test log prints the seed of each case. To reproduce a failure, run the same case again with the same `FEMU_FUZZ_SEED`:

```
FEMU_FUZZ_SEED=7 FEMU_FUZZ_SCALE=10 \
QTEST_QEMU_BINARY=./qemu-system-x86_64 ./tests/qtest/qos-test -m quick \
    -p /x86_64/pc/i440FX-pcihost/pci-bus-pc/pci-bus/femu/femu-tests/io-fuzz
```

8.2.4 Documentation checks

From the source tree, with the build above in `build/`:

```
make -C hw/femu/tests check-docs
```

It regenerates the property reference and mode tables and compares them, checks every relative link, and runs every example tagged `femu-example` under qtest. Set `QEMU=` and `QOS_TEST=` for a build elsewhere. docs-maintenance.md (§8.3) says how to tag a new example.

8.2.5 Host tool checks

The host tools check themselves under qtest, with QEMU and no guest:

- `femu-stat.py` writes pages through the qtest protocol and checks the counters, WAF and stage times it shows.
- `femu-config-check.py` enables the controllers and checks the configuration it derives. It also checks its exit status on refused configurations.

Each self-test also plants faults in its tool and requires each fault to fail its check:

```
make -C hw/femu/tests check-tools
```

Set `QEMU=` for a build elsewhere. CI runs the same self-tests.

8.2.6 Guest-side tests

These need a booted guest; CI does not run them. Copy the files into the guest first, for example with `scp -P 8080` and the key from `make-guest-image.sh`.

`femu-test.sh` checks one namespace end to end: data written comes back, the counters move, deallocate works, and zones or key-value commands answer when the namespace has them. **It overwrites the whole namespace**:

```
sudo bash femu-test.sh --yes /dev/nvme0n1
```

It prints `PASS`, `FAIL` and `SKIP` lines and a summary, and exits non-zero when a check failed, so a script can check it. A read that fails counts as a failure, the same as a bad checksum. It refuses a mounted device. A key-value namespace has no block node, so it is driven by passthrough through its generic node, `/dev/ngXnY`; name it as `/dev/nvmeXnY` or `/dev/ngXnY`. It makes no timing claims. A BlackBox run prints lines like these (`N` stands for the counts; ... for lines left out here). `max_block_reads` is the read count of the most-read block since its erase:

```
== device ==
  PASS  controller answers Identify
== data survives the FTL ==
  PASS  random write then verify (crc32c)
== deallocate ==
  PASS  deallocate accepted
  ...
== counters ==
  PASS  SMART log readable
  data_units_written=N host_write_commands=N
  ...
  waf_x1000=N host_pages=N nand_pages=N max_block_reads=N read_reclaims=N
```

```
  PASS  host writes counted
  PASS  write amplification reported

FEMU_TEST pass=N fail=0 skip=N
```

Smaller probes, built in the guest with `gcc -O2 -o NAME NAME.c`:

Probe	Run	Checks
`kv-probe.c`	`sudo ./kv-probe /dev/nvme0`	key-value Store, Exist, Retrieve and Delete
`kv-bench.c`	`sudo ./kv-bench /dev/ng0n1` (build with `-pthread`)	a key-value workload, with each answer checked against a model of the keys (KV benchmark (§8.2.6))
`aer-probe.c`	`sudo ./aer-probe /dev/nvme0`	a temperature event completes an Asynchronous Event Request
`zone-aen-probe.c`	`sudo ./zone-aen-probe /dev/nvme0 /dev/nvme0n1`	Zone Descriptor Changed notices, on a ZNS device with `err_write_fail_ppm` set

Mode-specific suites:

- FDP: `fdp-test-nvme-admin.sh` checks the FDP admin commands against the device `run-blackbox-fdp.sh` creates.
- CSD: the programs and guest tool in `hw/femu/tests/csd/` (CSD guide (§4.13.5)).
- CXL caching API: `hw/femu/tools/cca/run-guest-tests.sh` (CCA guide (§4.20.7)).

All of them are listed in the scripts reference (§C.5).

Conformance suites

Two external suites check FEMU as Linux sees it: blktests[132] and the end-to-end tests of nvme-cli[133]. Two more suites check FEMU under real workloads. `zenfs-crash` runs a RocksDB crash test on ZenFS[134] on a ZNS namespace. `kv-bench` drives a key-value namespace with the NVMe-KV commands and checks every answer. `guest-conformance.sh` runs the suites on the host. For each suite it boots a fresh guest with one FEMU device, runs the suite inside, and stops the guest:

```
host: guest-conformance.sh SUITE...
  for each suite:
    qemu-img overlay of the guest image (the image does not change)
      |
      v
    QEMU + KVM + FEMU (suite device options) --ssh--> guest
                                                        |
                                    SUITE's guest script
                                    (apt-get, git clone, build, run)
      |
      v
    OUTDIR/SUITE.log, SUITE.qemu.log, SUITE.serial.log --> RESULT SUITE PASS|FAIL
```

Suite	Device	Runs
`blktests-block`	BlackBox SSD, 768 MiB	blktests `block` group with `TEST_DEVS=(/dev/nvme0n1)`
`blktests-zbd`	ZNS SSD, 1 GiB	blktests `zbd` group

[132] https://github.com/linux-blktests/blktests

[133] https://github.com/linux-nvme/nvme-cli

[134] https://github.com/westerndigitalcorporation/zenfs

Suite	Device	Runs
nvme-cli	BlackBox SSD with oncs=415,vwc=1 (Compare, Write Uncorrectable, Dataset Management, Write Zeroes, Verify, Copy and Flush)	nvme-cli tests/nvme-cli-e2e, tag v3.1 by default
zenfs-crash	ZNS SSD, 4 GiB in 64 zones of 64 MiB; the guest gets 16 vCPUs and 16 GiB	zenfs-crash-guest.sh: RocksDB v9.10.0 with ZenFS 919c2eb, a blackbox crash test
kv-bench	KV SSD, 1 GiB	kv-bench-guest.sh: kv-probe, then two kv-bench runs

The guest is the image that make-guest-image.sh builds. The suites fetch packages and sources, so the guest needs network access; QEMU's user networking gives it. Run from build-femu/. A suite takes 5 to 60 minutes, and zenfs-crash builds RocksDB first:

```
../femu-scripts/guest-conformance.sh all
../femu-scripts/guest-conformance.sh --outdir /tmp/conf nvme-cli
../femu-scripts/guest-conformance.sh --keep zenfs-crash kv-bench
```

--help lists the options. They set the QEMU binary, the image, the SSH key and port, and the result directory. Others set the time limits, the guest size and a device for control runs.

BLKTESTS_REF and NVMECLI_REF select a blktests commit and an nvme-cli tag. ROCKSDB_REF and ZENFS_REF select the RocksDB tag and the ZenFS commit.

The script prints RESULT SUITE PASS, FAIL or SETUP-ERROR for each suite. The exit status is 0 when every suite passed and 1 when a suite failed. It is 2 when a suite could not run and no suite failed.

A blktests suite fails when a test that ran on the FEMU namespace failed. It also fails when the run stopped before every test reported a status. Some tests use null_blk, scsi_debug or device-mapper instead of the namespace. Their failures count in FAILED but stay out of the DEVICE_FAILED count. A test that needs a feature FEMU or the guest kernel lacks shows as not run, and the suite log gives the reason.

The nvme-cli suite fails when a test fails, with one exception. In nvme-cli v3.1, test_get_lba_status passes the namespace device path as the namespace ID. When that test fails, the script runs the same command with the numeric namespace ID. If the command passes, the log shows the test as KNOWN_DEFECT and the suite still passes. If it fails, FEMU fails the test.

ZenFS crash test. zenfs-crash-guest.sh builds RocksDB with the ZenFS plugin in the guest and runs the RocksDB blackbox crash test on the zoned namespace:

```
zenfs mkfs --zbd=nvme0n1
  |
  v
round 1..6:  db_stress (60 s) --SIGKILL-->  next round opens the database
             and checks it against the expected state on the host
  |
  v
final:       db_stress --verification_only=1 checks every key
  |
  v
blkzone report: data zones past the 3 ZenFS metadata zones hold data
```

The test passes when all of these are true:

- zenfs mkfs succeeds.

- Each round runs until the SIGKILL of its timeout and writes nothing to stderr. Each round opens the database and checks it first.
- In each round, a `db_stress` thread reports at least 1000 finished operations, and the SMART host write count grows.
- The final run exits 0, prints `Verification successful` and writes nothing to stderr.
- ZenFS wrote data zones on the device.

The options are those of ZenFS's own crash test, with zone-sized SST files and no injected RocksDB faults. ZenFS keeps unsynced WAL data in the process, and a SIGKILL loses it. Thus `--sync=1` syncs each write, and every write that completed must survive the kill. ZenFS's own driver passes `--expected_values_path`, a flag that RocksDB removed before 7.0. Thus the script starts `db_stress` itself, as RocksDB's `db_crashtest.py` does. The build goes to `/opt/femu-zenfs`. A kept overlay (`--keep`) can serve as the `--image` of a later run, which then skips the build.

Two control runs show that the test can fail:

- `ZENFS_CONTROL=reset-zones` resets every data zone that holds data after round 1. Round 2 must then fail. A control run that passes ends as a setup error.
- `--device` with BlackBox SSD properties gives a conventional namespace. `zenfs mkfs` must then refuse it.

KV benchmark. `kv-bench.c` sends the NVMe-KV commands through the passthrough ioctl of `/dev/ngXnY`: Store (01h), Retrieve (02h), List (06h), Delete (10h) and Exist (14h). It keeps a model of every key and checks each answer against it:

```
load          store every key
  |
mixed         Store, Retrieve, Delete and Exist by the --mix shares
  |
verify        Exist and Retrieve of every key
  |
List          the keys == the live keys of the model
  |
delete half   Delete; then Exist and Retrieve give 0x87
  |
verify, List
  |
cleanup       Delete the rest; List returns no key
```

Each Retrieve compares the value bytes and the size in the completion. Each command on a deleted key must return Key Does Not Exist (0x87). The List walk must return each live key once and no other key. The namespace must hold no key at the start. The workload options follow Samsung's kvbench: key count, key length, value sizes and their shares, and the operation shares. The program also prints the rate of each phase and the latency of each command.

`kv-bench-guest.sh` runs `kv-probe`, then `kv-bench` with 20000 keys of 4 to 16 bytes and values of 512, 2048 and 4096 bytes. A second run uses 256 keys with values of 4 KiB to 256 KiB. The suite passes when `kv-probe` reports no failure and each `kv-bench` run reports no error and no mismatch.

Two control runs show that the checks can fail:

- `KVBENCH_CONTROL=legacy-opcodes` sends Samsung's driver opcodes (81h, 90h, A1h, B3h, B2h). FEMU rejects them, so the run must fail.

- `KVBENCH_CONTROL=corrupt-model` changes one byte of the expected value of one key. The checks must report that key as a mismatch.

Guest scripts on their own. The guest scripts also run on their own in any guest:

```
sudo bash blktests-guest.sh block /dev/nvme0n1
sudo bash nvme-cli-e2e-guest.sh /dev/nvme0 /dev/nvme0n1
sudo bash zenfs-crash-guest.sh nvme0n1
sudo bash kv-bench-guest.sh /dev/ng0n1
```

All of them write to the namespace. The nvme-cli suite can format it, and `zenfs-crash-guest.sh` formats it with ZenFS. `kv-bench-guest.sh` needs `kv-probe.c` and `kv-bench.c` in its own directory.

8.2.7 Adding a test

A qtest

1. Write a function in `hw/femu/tests/qtest/femu-test.c` with the signature `static void femu_test_NAME(void *obj, void *data, QGuestAllocator *alloc)`. `obj` is the `QFemu` device; the existing helpers enable the controller, create queues and submit commands. Read a similar case first.
2. Register it in `femu_register_nodes()`. For the default device:

   ```
   qos_add_test("my-case", "femu", femu_test_my_case, NULL);
   ```

   For other device options:

   ```
   qos_add_test("my-bbssd-case", "femu", femu_test_my_case,
                &(QOSGraphTestOptions) {
       .edge.extra_device_opts =
           "serial=my-bbssd-case,devsz_mb=4,femu_mode=1,secs_per_pg=8,"
           "pgs_per_blk=4,blks_per_pl=128,pls_per_lun=1,luns_per_ch=2,nchs=2"
   });
   ```
3. Rebuild `tests/qtest/qos-test` and run the case by its path.
4. Show that it can fail: revert your fix, or plant the bug it is meant to catch, and check that the case goes red.

A unit test

Put it in `hw/femu/tests/unit/` and add it to `UNIT_TESTS`, a build rule and the `clean` rule in `hw/femu/tests/Makefile`. If it should also build against QEMU's headers, add it to `femu_unit_tests` in `hw/femu/tests/meson.build`.

A documentation example

Put the command line in a code block with a `femu-example` tag. The check starts it, enables each controller, and writes and reads one block. See docs-maintenance.md (§8.3.2).

8.2.8 Related pages

- Debugging (§7.2)
- Scripts reference (§C)
- CONTRIBUTING.md[135]

[135] `https://github.com/MoatLab/FEMU/blob/master/CONTRIBUTING.md`

8.3 Keeping the documentation correct

Four checks keep the documentation in step with the code. CI runs all of them, and so does

```
make -C hw/femu/tests check-docs QEMU=$PWD/build/qemu-system-x86_64 \
    QOS_TEST=$PWD/build/tests/qtest/qos-test
```

Check	Script	Fails when
Property reference	`hw/femu/scripts/gen-property-docs.py --check`	a property has no description, or `reference/properties.md` or `reference/runtime-properties.md` differs from the binary
Mode table	`hw/femu/scripts/gen-mode-table.py --check`	a mode table differs from `modes.py`, or `modes.py` disagrees with the tree
Links	`hw/femu/scripts/check-doc-links.py`	a relative link or heading anchor does not exist
CHANGELOG commits	`hw/femu/scripts/check-changelog-hashes.py`	a commit that the CHANGELOG names is not in the history of HEAD, for example a hash from before a rebase
Examples	`hw/femu/scripts/check-doc-examples.py`	a code block is not tagged, or a tagged example does not work

Only two reference pages are generated: `reference/properties.md` and `reference/runtime-properties.md`, from the built binary plus `reference/property-topics.py` and `reference/environment.inc.md`. The others, `reference/log-pages-and-counters.md` and `reference/scripts.md`, are written by hand and no check compares them with the code, so update them in the same commit as the change they describe. A user-visible change also needs an entry in CHANGELOG.md (§D).

8.3.1 Per-mode facts: `modes.py`

`hw/femu/docs/modes.py`[136] holds one entry per mode or feature: how to turn it on, a minimal example, the guest kernel and tools it needs, host requirements, its launcher and the page that documents it. Change a fact there, never in a table, then run

```
python3 hw/femu/scripts/gen-mode-table.py
```

The script rewrites the text between `<!-- modes-table:start -->` and `<!-- modes-table:end -->` in README.md and in any page under `hw/femu/docs/` that has the two markers. To put the table on a new page, add the markers and run the script. README.md must keep its markers.

The same script writes a checkbox list of the entries between `<!-- modes-list:start -->` and `<!-- modes-list:end -->` in `.github/ISSUE_TEMPLATE/bug_report.md` and `.github/pull_request_template.md`. Both files must keep these markers, so a new mode appears in the bug report and pull request forms with no manual edit.

`--check` also compares `modes.py` with the code. Each `symbol` must have the stated `femu_mode` value in `hw/femu/nvme.h`, every mode in that enum must have an entry, each launcher must exist in `hw/femu/scripts/`, and each guide link must resolve. Each entry's `example` is run by the example check below, so the "Checked" column states what CI really does with it.

[136] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/modes.py

8.3.2 Code blocks in the documentation

Every fenced code block in README.md and `hw/femu/docs/**` falls into one of these classes:

Block	Meaning	Checked
preceded by `<!-- femu-example: NAME -->`	a QEMU command line, `-device` options, a `run-*.sh` launcher or a `femu-config-check.py` command	run under qtest
preceded by `<!-- femu-untested: REASON -->`	a command that cannot run in CI	the reason must have at least three words
` ```sh `	shell commands that do not start FEMU: git, apt, configure, commands inside the guest	must not start FEMU
` ```bash `, ` ```shell `, ` ```console `, ` ```zsh `	not allowed untagged	fails
any other info string, or none	output, configuration files, code	must not start FEMU

A block "starts FEMU" when, outside shell comments, it runs `qemu-system-*` as a command, passes `-device femu...`, or names a `run-*.sh` launcher other than `run-guest-ssh.sh`. The tag comment goes on the line right before the fence. GitHub does not show it and still highlights the block.

Writing an example

```
<!-- femu-example: zns-small -->
-device femu,devsz_mb=1024,femu_mode=3
```

- One example per block, or several separated by blank lines; each is named `NAME`, `NAME.2`, and so on in the report.
- `...` stands for options left out and is dropped, so `-device femu,...,femu_mode=1,read_reclaim_limit=100000` is tested as `-device femu,femu_mode=1,read_reclaim_limit=100000`.
- A launcher such as `./run-zns.sh` (with optional `VAR=value` settings in front) is run with stand-ins for `sudo` and QEMU that record the command line it builds, so the launcher's own options are what gets tested. `OSIMGF` is replaced by an empty file, and lines that run `run-guest-ssh.sh` are skipped.
- A list of `key=value` lines can be tested as one device: `<!-- femu-example: zns-params; device: femu,femu_mode=3 -->`.
- `io: rw`, `io: kv`, `io: identify` or `io: none` overrides what the I/O stage does; by default it follows the mode, from `modes.py`.
- `allow-warning: TEXT` allows one expected warning, for example `allow-warning: FEMU CXL DER unavailable` for `der=cylon`, which falls back to MMIO without a Cylon host.
- A `femu-config-check.py` command is run as written, with `--qemu` added. It must exit with status 0, or with the status that `exit: 1` names for a configuration the documentation shows as refused. A warning in its report fails the example unless `allow-warning` allows it.

What the example check does

1. QEMU starts with every `-device` of the example, in its order, plus the memory backends and `-machine` options (q35 when none is given), under `-accel qtest -S`. Guest disks and NICs are realized too, so one the machine cannot plug where it lands fails here: on a `cxl=on` machine a `virtio-net-pci` or `virtio-blk-pci` without `bus=pcie.0` fails with "Only PCI/PCIe bridges can be plugged into pxb-cxl". Their backends are replaced by stand-ins that touch nothing on the host: each `-drive` by a `null-co` block device with the same `id` and `if`, each `-netdev` and `-net user` by user networking with no forwarded ports (so the QEMU under test needs `--enable-slirp`). The other options are dropped, among them

`-enable-kvm`, `-cpu`, `-smp`, `-m`, `-numa`, `-name` and `-qmp`. QMP must show every `femu`, `femu-subsys` and `femu-cxl-ssd` created, and `query-pci` must list each `femu` as an NVMe controller. Anything on stderr fails the example, except FEMU's `[FEMU] Log:` lines, the notice that the memory backend could not be pinned (the check lowers `RLIMIT_MEMLOCK` so that it never pins), and allowed warnings. Test-only properties (`x-...`) are refused.

2. For an example with an NVMe controller, the `doc-examples` case in `hw/femu/tests/qtest/femu-test.c` starts the FEMU devices, the CXL topology and the memory backends from step 1, without the guest's disks and NICs, enables each controller, sends Identify, and writes and reads back one block of namespace 1 (stores and retrieves one value in KV mode). Open-Channel controllers stop after Identify. A controller whose namespace 1 is not attached stops after Identify, but at least one controller in the example must move data.

The check does not boot a guest. Guest kernel versions, guest tools and anything done inside the guest are outside what it can see.

`--self-test` runs planted mistakes (an untagged block, a misspelled property, `femu_mode=9`, a warning-only `der=cylon`, an I/O-only failure, a test-only property, a `virtio-net-pci` without `bus=` on a `cxl=on` machine, a guest disk naming a drive that does not exist) and requires each to fail with the expected message, and two good examples, one with guest devices on a CXL machine, to pass, so a check that stopped catching them fails rather than passing everything.

```
python3 hw/femu/scripts/check-doc-examples.py --lint          # tags only
python3 hw/femu/scripts/check-doc-examples.py --list          # what would run
python3 hw/femu/scripts/check-doc-examples.py \
    --qemu build/qemu-system-x86_64 --qos-test build/tests/qtest/qos-test \
    --only quick-start-bbssd
```

A Property reference

A.1 Device property reference

Every property of the FEMU devices, as the binary reports it. `./qemu-system-x86_64 -device femu,help` prints the same descriptions at the terminal. Properties are set on the command line, for example `-device femu,femu_mode=1,devsz_mb=4096`, and only before the device is realized; the QOM properties that can be read or changed while the guest runs are in runtime-properties.md (§A.2).

Mode names follow `femu_mode`: OCSSD (0), bbssd (1, black box), NoSSD (2), ZNS (3), CSD (4) and KV (5). CSD builds the bbssd FTL, so the bbssd geometry, timing and FTL properties apply to it; KV uses only the bbssd geometry and NAND timing. Each description names the modes it applies to.

Devices:

- `femu`: NVMe controller (§A.1.1)
- `femu-subsys`: NVMe subsystem (§A.1.2)
- `femu-cxl-ssd`: CXL Type-3 SSD (§A.1.3)
- Test-only properties (§A.1.4)
- Environment variables (§A.1.5)

A.1.1 `femu`: NVMe controller

`-device femu` has 219 properties of its own and 1 QOM property listed in runtime-properties.md (§A.2).

Mode, capacity and namespaces

Property	Type	Default	Description
`femu_mode`	`uint8`	`2`	Emulated SSD type: 0 OCSSD (Open-Channel), 1 bbssd (black-box FTL), 2 NoSSD (no media timing), 3 ZNS, 4 CSD (computational), 5 KV; other values fail realize
`devsz_mb`	`uint32`	`1024`	Size of the host memory backend in MiB, split across namespaces unless namespace_sizes is set; bbssd with op_pcent sizes from the NAND geometry instead
`namespaces`	`uint32`	`1`	Number of namespaces created at boot, 1 to 256; OCSSD supports only 1, and FDP more than 1 only when all are bbssd or all nossd
`namespace_sizes`	`str`	unset	Comma-separated size in bytes of each namespace (QEMU size syntax such as 4G,2G), one non-empty entry per namespace, summing to at most the backend; unset splits the backend evenly
`namespace_modes`	`str`	unset	Comma-separated mode of each namespace from nossd, bbssd, znssd, ocssd, csd and kvssd, one entry per namespace; unset gives every namespace femu_mode
`namespace_phls`	`str`	unset	FDP placement handle list of each namespace: comma-separated entries, one per namespace, each a ':'-separated list of reclaim unit handle IDs such as 0:1,,2 (placement handle i selects entry i); needs fdp=on, and unset gives every namespace every handle
`op_pcent`	`uint32`	`0`	Over-provisioning in percent for bbssd: back the device with the full NAND capacity and expose capacity/(1 + op_pcent/100); 0 keeps devsz_mb sizing, not with cxl_ssd

Property	Type	Default	Description
subsys	link<femu-subsys>	unset	ID of a femu-subsys device to join, created before this controller; needed for FDP, shared namespaces and the Endurance Group log
cxl_ssd	link<femu-cxl-ssd>	unset	ID of a femu-cxl-ssd, listed before this controller, whose memory and FTL this bbssd controller serves as its one namespace
pel_file	str	unset	Host file that keeps the Persistent Event Log and power cycle count across runs, created if missing; a corrupt or incompatible file fails realize
serial	str	unset	No effect, kept for compatibility; Identify Controller reports a serial number FEMU generates. Setting it warns at realize

Queues, pollers and interrupts

Property	Type	Default	Description
queues	uint32	8	Number of I/O submission and completion queue pairs, 1 to 2047; MSI-X vectors are queues + 1
entries	uint32	2047	Value reported as CAP.MQES (0's based), 1 to 65534; the controller accepts queues of up to entries + 1 entries
max_sqes	uint8	6	Submission queue entry size as a power of two (Identify SQES); must be 6, 64-byte entries
max_cqes	uint8	4	Completion queue entry size as a power of two (Identify CQES); must be 4, 16-byte entries
stride	uint8	0	Doorbell stride (CAP.DSTRD): doorbells are 4 « stride bytes apart, 0 to 12
multipoller_enabled	uint8	0	0 runs one poller thread for all I/O queues; 1 runs ceil(queues / poller_ratio) pollers, each owning a round-robin share of the queues; other values fail realize
poller_ratio	uint32	1	I/O queues per poller thread when multipoller_enabled=1; 0 is treated as 1, one poller per queue
hiops_inline	bool	on	NoSSD only: performance option, on by default; set off only when debugging
io_stages	bool	off	Time each I/O command in five stages (submit, ftl, hold, post-delay, total) for the QMP command query-femu-stages; off by default
aerl	uint8	3	Asynchronous Event Request Limit (Identify AERL, 0's based): the controller holds up to aerl + 1 outstanding requests
elpe	uint8	3	Error Log Page Entries (Identify ELPE, 0's based): the Error Information log keeps the newest elpe + 1 entries
mdts	uint8	10	Maximum Data Transfer Size as a power of two of the minimum memory page size (2^(12 + mpsmin + mdts) bytes); 0 means no limit
intc	uint8	0	Initial Coalescing Disable bit (0 or 1) of the Interrupt Vector Configuration feature for every vector
intc_thresh	uint8	0	Initial aggregation threshold (0's based entries) of the Interrupt Coalescing feature; coalescing runs when it and intc_time are non-zero
intc_time	uint8	0	Initial aggregation time, in 100 microsecond units, of the Interrupt Coalescing feature; coalescing runs when it and intc_thresh are non-zero

Controller identity and capabilities

Property	Type	Default	Description
vid	uint16	7453	PCI vendor ID, also reported as the Identify Controller PCI Vendor ID
did	uint16	7967	PCI device ID of the controller function
acl	uint8	3	Abort Command Limit reported in Identify Controller (0's based); an Abort run with more than acl others queued behind it fails
cqr	uint8	1	CAP.CQR: 1 requires physically contiguous queues, 0 allows PRP-list queues
vwc	uint8	0	1 advertises a volatile write cache, which the host can turn off with feature 06h to stop bbssd buffering writes; 0 advertises none and refuses feature 06h. Flush drains the bbssd write buffer either way. On KV with 1, Flush programs the open page of packed values, and with the cache off each Store programs it; 0 or 1
temperature	uint16	323	Composite temperature in kelvin reported by the SMART log and compared with the temperature threshold feature; default 323 (50 C)
mpsmin	uint8	0	CAP.MPSMIN: smallest host memory page size as 2^(12 + mpsmin) bytes; must not exceed mpsmax
mpsmax	uint8	0	CAP.MPSMAX: largest host memory page size as 2^(12 + mpsmax) bytes, from mpsmin to 15
oacs	uint16	2	Optional Admin Command Support; only bit 1 (Format NVM, 0x2) may be set, and clearing it refuses Format NVM
oncs	uint16	413	Optional NVM Command Support bit mask: 0x1 Compare, 0x2 Write Uncorrectable, 0x4 Dataset Management, 0x8 Write Zeroes, 0x10 Save/Select, 0x80 Verify, 0x100 Copy; the default 0x19d sets all but Write Uncorrectable; Timestamp is always added
sgl	bool	off	Advertise and accept address scatter gather lists for data transfer; OCSSD ignores it
cmbsz	uint32	0	Controller Memory Buffer size register; 0 means no buffer, otherwise the size field times the unit must be a non-zero power of two
cmbloc	uint32	0	Controller Memory Buffer location register; its BAR field must be 2 when cmbsz is set

LBA formats, metadata and protection

Property	Type	Default	Description
nlbaf	uint8	5	Number of LBA formats, 1 to 16 (at most 8 with meta): 512-byte blocks doubling with each format
lba_index	uint8	0	LBA format the namespaces boot with, below nlbaf; ZNS needs a block size of 4 KiB or less
extended	uint8	0	1 boots with metadata interleaved with the data (extended LBAs); needs bit 0 of mc; 0 or 1
meta	uint8	0	Metadata bytes per logical block, NoSSD and bbssd only, not with FDP, dpc or dps; each format is then also offered with metadata
mc	uint8	0	Metadata Capabilities bit mask: bit 0 interleaved (extended LBAs), bit 1 separate buffer; required when meta is set
pi	bool	off	Offer end-to-end protection information types 1 to 3, in the last 8 bytes of metadata, when meta is at least 8 bytes, and allow Format and Create to select them; not with power_loss or cxl_ssd

Property	Type	Default	Description
dpc	uint8	0	Data Protection Capabilities reported in Identify Namespace when pi is off; must be 0 with meta, and bit 3 (first 8 bytes) is refused except on OCSSD
dps	uint8	0	Data Protection Type Settings reported in Identify Namespace; a non-zero value needs 8 bytes of metadata, which meta refuses, so leave it 0 and use pi
ms	uint8	16	No effect, kept for compatibility; meta sets the metadata size. A value other than the default warns at realize
ms_max	uint8	64	No effect, kept for compatibility; OCSSD 2.0 reports a single LBA format. A value other than the default warns at realize
dlfeat	uint8	1	No effect, kept for compatibility; Identify Namespace always reports DLFEAT 0x9 (deallocated blocks read as zeroes). A value other than the default warns at realize

Namespace management, streams and power loss

Property	Type	Default	Description
ns_mgmt	bool	off	Enable Namespace Management and Attachment on a standalone NoSSD or bbssd controller whose namespaces all run femu_mode with dps 0, otherwise it stays off without an error; with a shared subsystem use femu-subsys ns_mgmt
bbssd_ns_limit	uint32	4	Most bbssd namespaces that may be allocated with ns_mgmt, including detached ones, 1 to 256 and at least namespaces; each has its own FTL
streams	bool	off	Enable the Streams directive; bbssd separates streams per FTL page (SWS) and needs page or dftl mapping, NoSSD has no placement effect; not with FDP or a shared subsystem
streams.max	uint16	8	Number of stream slots with streams=on, 1 to 32; bbssd reserves streams.max + 1 lines for them
power_loss	bool	off	bbssd: roll back writes still in the write buffer on a simulated power cut and enable simulate-power-loss; needs buffer_size > 0, vwc=1, page-aligned namespaces, and no meta, pi, ns_mgmt, subsys, namespace_modes or cxl_ssd

NAND geometry (bbssd, CSD, KV)

Property	Type	Default	Description
secsz	int32	512	bbssd, CSD, KV: sector size in bytes, greater than 0
secs_per_pg	int32	8	bbssd, CSD, KV: sectors per NAND page, 1 to 256
pgs_per_blk	int32	256	bbssd, CSD, KV: pages per NAND block, 1 to 65536, at most 512 with nand_cell_type
blks_per_pl	int32	256	bbssd, CSD, KV: blocks per plane, 1 to 65536; also the number of lines (superblocks)
pls_per_lun	int32	1	bbssd, CSD, KV: planes per LUN, 1 to 16
luns_per_ch	int32	8	bbssd, CSD, KV: LUNs (dies) per channel, 1 to 128
nchs	int32	8	bbssd, CSD, KV: number of channels, 1 to 4096; the total sector count must fit in a signed 32-bit integer

NAND timing (bbssd, CSD, KV)

Property	Type	Default	Description
pg_rd_lat	int32	40000	bbssd, CSD, KV: NAND page read time in ns when nand_cell_type is 0
pg_wr_lat	int32	200000	bbssd, CSD, KV: NAND page program time in ns when nand_cell_type is 0
blk_er_lat	int32	2000000	bbssd, CSD, KV: NAND block erase time in ns when nand_cell_type is 0
ch_xfer_lat	int32	0	Channel transfer time per page in ns: the data phase for bbssd, CSD and KV when pg_xfer_lat is 0, and the OCSSD 1.2 transfer time with oc12_channel_timing
cmd_addr_lat	int32	0	bbssd, CSD, KV: command and address phase on the channel bus in ns; the bus is modelled only when this, pg_xfer_lat (or ch_xfer_lat) or status_lat is non-zero
pg_xfer_lat	int32	0	bbssd, CSD, KV: page data transfer phase on the channel bus in ns; 0 uses ch_xfer_lat
status_lat	int32	0	bbssd, CSD, KV: status read phase on the channel bus in ns
onfi_mode	int32	-1	bbssd, CSD, KV: ONFI 6.0 NV-LPDDR4 timing mode 0 to 26; the channel bus phases then follow the spec for that mode and pg_rd_lat, pg_wr_lat and blk_er_lat stay the array times; -1 (default) is off. Refused with cmd_addr_lat, pg_xfer_lat, ch_xfer_lat, status_lat or pe_suspend
onfi_mts	int32	0	bbssd, CSD, KV: with onfi_mode, the page payload rate in MT/s in place of the mode's own rate, 1 to 100000; 0 (default) uses the mode
onfi_spare_bytes	int32	0	bbssd, CSD, KV: with onfi_mode, the spare bytes moved with each page on the bus; the page on the bus must be an even number of bytes; 0 (default) moves the data bytes only
onfi_tccs_ns	int32	0	bbssd, CSD, KV: with onfi_mode, tCCS in ns from a datasheet, paid before a data out when several LUNs share a channel and per plane of a multi-plane read; ONFI 6.0 gives no value, so 0 (default) pays nothing
onfi_param_page	str	unset	bbssd, CSD, KV: path of an ONFI 6.0 parameter page image (256 bytes per copy, CRC checked, redundant copies tried). It turns the ONFI profile on, gives the mode when onfi_mode is -1, and sets the spare bytes and address cycles. Realize refuses a page whose geometry, features or modes do not match the device; unset (default) uses none
onfi_sca	int32	0	bbssd, CSD, KV: with the ONFI profile, non-zero selects the SCA protocol (ONFI 6.0 Section 5): commands, addresses and status use a separate CA bus, and the channel bus carries only the data bursts; 0 (default) is the conventional protocol
onfi_cache_read	int32	0	bbssd, CSD: with the ONFI profile, non-zero reads the pages of one read command that land on one LUN as a read cache sequence (31h, 00h-addr-31h, 3Fh; ONFI 6.0 Section 6.15), or as plain reads when the sequence would end later on an idle LUN. Refused with mp_read, mapping=dftl and read_cache_mb; 0 (default) is off
tplpbsy	int32	0	bbssd, CSD: busy time in ns between the planes of a multi-plane program; only with mp_program, and a value without it warns at realize. Negative is refused
tplrbsy	int32	0	bbssd, CSD: busy time in ns between the planes of a multi-plane read; only with mp_read, and a value without it warns at realize. Negative is refused
tplebsy	int32	0	bbssd, CSD, KV: busy time in ns between the planes of a multi-plane erase, which garbage collection issues when pls_per_lun > 1
mp_program	int32	0	bbssd, CSD: non-zero charges programs of the same page on several planes of a LUN as one multi-plane command (one array time); needs pls_per_lun > 1 and no FDP, and warns at realize otherwise

Property	Type	Default	Description
mp_read	int32	0	bbssd, CSD: non-zero charges host reads of the same page on several planes of a LUN as one multi-plane command (one array time); needs pls_per_lun > 1 and no FDP, and warns at realize otherwise
plane_first	int32	0	bbssd, CSD: non-zero makes write pointers fill the planes of a LUN before the next channel, so consecutive pages form one multi-plane program with mp_program; needs pls_per_lun > 1 and no FDP, and warns at realize otherwise
trcbsy	int32	0	bbssd, CSD: tRCBSY in ns, the busy time of each copy in a read cache sequence; only with onfi_cache_read, and a value without it warns at realize. 0 (default) takes 3000, the ONFI 6.0 typical value. Negative is refused with onfi_cache_read
trim_lat_ns	int32	0	bbssd, CSD: time in ns charged per Dataset Management deallocate range, with or without FDP
pe_suspend	int32	0	bbssd, CSD, KV: non-zero lets a read suspend a program or erase on its LUN instead of waiting for it to finish
tsusp_ns	int32	0	bbssd, CSD, KV: overhead in ns added to a read that suspends a program or erase, 0 or more
nand_cell_type	uint8	0	bbssd, CSD, KV: 0 uses the flat pg_rd_lat, pg_wr_lat and blk_er_lat; 1 SLC, 2 MLC, 3 TLC or 4 QLC uses built-in per-page-type timing (other values fall back to 0)
cell_pages	int32	0	bbssd, CSD, KV: bits per cell (pages per wordline) for the pgtype_lat model, 0 to 5; 0 with pgtype_lat means 3
pgtype_lat	int32	0	bbssd, CSD, KV: non-zero scales the program time by page type (lower to upper) using cell_pages, when nand_cell_type is 0

Reliability and wear

Property	Type	Default	Description
ecc_step_ns	int32	0	bbssd, CSD, KV: extra read time in ns per ECC tier, one tier per 750 erases of the block plus one per ecc_retention_sec of data age, at most 4 tiers; 0 turns the model off
ecc_retention_sec	int32	0	bbssd, CSD, KV: data age in seconds that adds one ECC tier, with ecc_step_ns; 0 counts wear only
pd_edge_wls	int32	0	bbssd, CSD, KV: program disturb, bottom wordlines of each block whose reads take pd_edge_tiers more ECC tiers; set with pd_edge_tiers and ecc_step_ns; 0 (default) turns the term off. A wordline is the pages per wordline of nand_cell_type (else the effective cell_pages, 3 with pgtype_lat, else 1) contiguous pages
pd_edge_tiers	int32	0	bbssd, CSD, KV: program disturb, ECC tiers added to a read of a pd_edge_wls wordline, 0 to 4, under the shared tier cap; chosen values for sensitivity studies, not fitted to error rates
pd_open_tiers	int32	0	bbssd, CSD, KV: program disturb, ECC tiers added to a read of a block that is only part programmed, 0 to 4, under the shared tier cap; needs ecc_step_ns; 0 (default) turns the term off
pd_open_age	bool	off	bbssd, CSD, KV: data age for the ECC age tiers counts from the first program of the line instead of the line close, so open data ages too; one time per line; needs ecc_step_ns and ecc_retention_sec; off (default)

Property	Type	Default	Description
pd_open_read_limit	int32	0	bbssd, CSD, FDP: program disturb, reads of a part-programmed block that queue its line or reclaim unit; the next Write rewrites it, after it seals a frontier line or retires a frontier unit with its free pages unprogrammed; not above a non-zero read_reclaim_limit; 0 (default) turns it off
pd_open_close_sec	int32	0	bbssd, CSD, FDP: program disturb, seconds of device time (age_scale) a frontier line or reclaim unit may stay open after its first program; the next request pads its free pages and closes it; under FDP the handle gets a fresh unit and event 1h; 0 (default) turns it off
pe_cycles_rated	uint32	0	bbssd, CSD, KV, ZNS: rated program/erase cycles used for SMART Percentage Used; 0 takes the rating of nand_cell_type (ZNS: zns_flash_type), or reports none
nand_bad_blocks	uint32	0	bbssd, CSD, KV: blocks marked bad at start, capped at the block count, which lowers SMART Available Spare
blk_pe_limit	uint32	0	bbssd, CSD, also FDP: mean program/erase cycles a block survives before it wears out; 0 (default) turns wear-out off. When no block can replace a worn-out one, the media goes read-only (eol_read_only). Under FDP a worn-out block retires its reclaim unit
blk_pe_spread	uint32	0	bbssd, CSD, also FDP: how far each block's limit varies from blk_pe_limit, in percent, 0 to 90
blk_pe_seed	uint64	0	bbssd, CSD, also FDP: seed of the per-block erase limits; one seed gives the same limits on every run
spare_lines	uint32	0	bbssd, CSD: lines held back, needs blk_pe_limit; their blocks replace worn-out blocks of the same plane, and SMART Available Spare is the spare blocks the emptiest plane has left, in percent
eol_read_only	bool	on	bbssd, CSD, with blk_pe_limit: on (default) puts the media in read-only mode when no block can replace a worn-out one, with reserve lines for the writes in flight; needs mdts 1 or more and limits Write Zeroes (WZSL) to mdts and Copy (MCL) to mdts or one block. off keeps a worn-out block in service with the SMART reliability warning. Under FDP each reclaim group holds reserve units
energy_read_nj	uint32	0	bbssd, CSD, KV: energy of one plane read in nJ, for the energy total in log page C0h; 0 (default) gives no figure
energy_prog_nj	uint32	0	bbssd, CSD, KV: energy of one plane program in nJ, for the energy total in log page C0h; 0 (default) gives no figure
energy_erase_nj	uint32	0	bbssd, CSD, KV: energy of one plane erase in nJ, for the energy total in log page C0h; 0 (default) gives no figure
thermal_tau_ms	uint32	0	time constant of the package, ms: the composite temperature moves towards temperature + power * thermal_r at this rate, where power is idle_mw plus the energy of the NAND plane operations (energy_*_nj) per second; 0 (default) turns the model off
thermal_r	uint32	0	thermal resistance from the package to ambient, mK per mW (K per W); needed with thermal_tau_ms
idle_mw	uint32	0	power with no NAND operations, mW, for the thermal model
thermal_step_ms	uint32	100	how often the thermal model updates the temperature, ms, 1 to 60000; 100 (default)
err_read_unc_ppm	uint32	0	bbssd, CSD: reads per million that fail as Unrecovered Read Error, injected at a fixed period; 0 disables
err_write_fail_ppm	uint32	0	bbssd, CSD and ZNS: writes per million that fail, injected at a fixed period (a ZNS zone then goes read-only); 0 disables

Property	Type	Default	Description
read_reclaim_limit	int32	0	bbssd, CSD: when a host read finds its block has taken this many reads since its erase, that line is queued and rewritten on a following write, one line at a time (under FDP, its reclaim unit); 0 disables
retention_limit_sec	int32	0	bbssd, CSD: when a host read hits a line filled at least this many seconds earlier, the line is queued and rewritten on a following write (under FDP, its reclaim unit); 0 disables
age_scale	int32	1	bbssd, CSD: data ages this many times faster than wall time, for retention_limit_sec and ecc_retention_sec only; 1 (default) or more. I/O timing and garbage collection order do not change

Garbage collection, mapping and caches

Property	Type	Default	Description
gc_thres_pcent	int32	75	bbssd, CSD: percent of lines in use at which background garbage collection starts, 1 to 100; KV uses it only as the fraction of NAND usable for values
gc_thres_pcent_high	int32	95	bbssd, CSD: percent of lines in use at which garbage collection is forced, from gc_thres_pcent to 100; under FDP it keeps at least one reclaim unit free unless it is 100
gc_policy	str	unset	bbssd, CSD: victim policy, one of greedy, random, cost-benefit, fifo or d-choice; unset is greedy. Under FDP it picks the reclaim unit strategy, and a name other than greedy needs gc_strategy=0
gc_seed	uint64	0	bbssd, CSD: seed for the victims the random and d-choice gc_policy and the FDP random gc_strategy pick; the same seed and workload pick the same victims, so vary it to vary runs
gc_strategy	int32	0	bbssd with FDP: reclaim unit victim strategy, 0 greedy, 1 cost-benefit, 2 random or 4 per-handle; a gc_policy other than greedy picks it instead
wl_spread	int32	0	bbssd, CSD: static wear levelling starts when the erases of lines in service differ by more than this and stops at half of it; it moves the least worn full line into the most worn free line and copies at most a quarter of the host pages; 0 (default) turns it off
gc_pace	bool	off	bbssd, CSD: collect the victim line a few pages per host page written, in proportion to its valid pages, instead of in one pass; the forced pass stays as the last resort; off (default)
gc_copyback	bool	off	bbssd, CSD, FDP: collection moves a page on its die (copyback) when the destination is on the same channel, LUN and plane; collection takes victim pages in the order of the destination dies. A copyback skips ECC: the data never reaches the controller, so a real SSD does not correct its bit errors, and they can add up over repeated copybacks. FEMU models no bit errors, so here only the timing changes. For that reason a page whose read has extra ECC time, and every read reclaim or retention refresh, moves off-chip. Not for KV. Refused with gc_pace, pe_suspend, cxl_ssd, namespace_modes, ns_mgmt, shared namespaces and the hybrid and fast mappings; off (default)
mapping	str	unset	bbssd, CSD: logical-to-physical mapping scheme, one of page, dftl, hybrid or fast; unset is page, and FDP takes page or dftl
mapping_cache_mb	uint32	0	bbssd, CSD with mapping=dftl: size of the cached mapping table in MiB; 0 means 4
read_cache_mb	uint32	0	bbssd, CSD: size of the DRAM read cache in MiB; 0 disables it

Property	Type	Default	Description
cache_evict	str	unset	bbssd, CSD: read cache eviction policy, one of clock, random, lru or arc; unset is clock
hot_cold_sep	bool	off	bbssd, CSD with mapping page or dftl: write overwrites of mapped pages to separate hot lines; refused with FDP
buffer_size	int32	0	bbssd, CSD: DRAM write buffer capacity in NAND pages, not bytes; 0 programs every write directly
buffer_thres_pcent	int32	90	bbssd, CSD: buffer fill level in percent at which buffered pages are written to NAND, 1 to 100 when buffer_size > 0
fdp_trim_erase_all	int32	0	bbssd with FDP: non-zero makes a deallocate reset every reclaim unit instead of the given ranges
debug_ftl	bool	off	bbssd, CSD, KV: print a message when a page is programmed while not free or invalidated while not valid, and print merge counts for hybrid and fast mapping

pSLC tier (bbssd)

Property	Type	Default	Description
tier_mode	str	unset	bbssd: off (unset), static or dynamic; static keeps a pool of lines in pSLC mode in front of the dense lines, and host writes go there first. dynamic also borrows free dense lines for the pool while the device has room. Needs nand_cell_type 2, 3 or 4 and page mapping; refused with FDP, more than one namespace, buffer_size, Streams and the wear, refresh and pd_* features
tier_full_policy	str	unset	bbssd with a tier_mode: what a host page does when the pool has no free page; direct (unset) writes it to the dense lines, through folds the oldest pool line first and writes it to the pool
tier_static_mb	uint32	0	bbssd with a tier_mode: host data in MiB the static pSLC pool holds; the pool takes enough whole lines out of the over-provisioning, and realize refuses a pool that leaves the namespace too little room. Above 0 for static and for through; with dynamic it is the floor the pool keeps
tier_pslc_read_ns	uint64	25000	bbssd with a tier_mode: time in ns of one pSLC page read, at most 10 s
tier_pslc_prog_ns	uint64	200000	bbssd with a tier_mode: time in ns of one pSLC page program, at most 10 s
tier_pslc_erase_ns	uint64	3500000	bbssd with a tier_mode: time in ns of one pSLC block erase, at most 10 s
tier_pslc_pe_cycles	uint32	0	bbssd with a tier_mode: rated erase cycles of a block in pSLC mode, for the tier wear figure; 0 (default) takes 30 times the dense rating
tier_max_mb	uint32	0	bbssd with tier_mode=dynamic: host data in MiB the borrowed lines may hold at most; 0 (default) sets no cap, and the free space alone limits the pool
tier_shrink_pct	uint32	50	bbssd with tier_mode=dynamic: logical fill in percent at which the pool starts to give borrowed lines back, 0 to 99; above it the allowance falls in a line to 0 at a full device
tier_idle_ms	uint32	0	bbssd with a tier_mode: quiet time in ms after which the device folds pool lines in the background, one bounded step at a time; 0 (default) folds only when a write needs room

Property	Type	Default	Description
tier_lazy_pct	uint32	0	bbssd with tier_idle_ms: 0 (default) folds every closed pool line at idle; 1 to 99 folds only while the pool lines in use are above this percent of the pool

Host link and controller firmware

Property	Type	Default	Description
pcie_bandwidth_mbps	uint32	0	Host link bandwidth in MB/s (10^6 bytes); each Read or Write is charged its transfer time on a per-direction link queue; 0 disables the bandwidth charge
pcie_prop_delay_ns	uint32	0	Host link propagation delay in ns added to each Read or Write after its link transfer; 0 disables it
fw_cpu_ns	uint64	0	Controller firmware time in ns charged to each Read, Write and Zone Append, serialized on one modelled core; 0 disables it
fw_read_ns	uint64	0	Controller firmware time in ns for each Read; 0 (default) takes fw_cpu_ns
fw_write_ns	uint64	0	Controller firmware time in ns for each Write and Zone Append; 0 (default) takes fw_cpu_ns
fw_other_ns	uint64	0	Controller firmware time in ns for each I/O command other than Read, Write and Zone Append; 0 (default) charges none
fw_cores	uint32	1	Controller firmware cores, 1 (default) to 64; each command takes the core that is free soonest

ZNS

Property	Type	Default	Description
zns_num_ch	uint8	2	ZNS: number of channels, 1 to 128
zns_num_lun	uint8	4	ZNS: LUNs (dies) per channel, 1 or more
zns_num_plane	uint8	2	ZNS: planes per LUN, 1 to 8; the program unit grows with it
zns_num_blk	uint8	32	ZNS: blocks per plane, 1 or more; the zone count is zns_num_blk times zns_num_ch times zns_num_lun, divided by the zone width, the zone LUNs and zns_num_plane
zns_flash_type	int32	4	ZNS: cell type, 1 SLC, 2 MLC, 3 TLC, 4 QLC or 5 PLC; MLC and PLC have no built-in timing and need zns_pg_rd_lat, zns_pg_wr_lat and zns_blk_er_lat
zns_pg_rd_lat	int64	0	ZNS: page read time in ns, 0 or more; 0 uses the built-in time of zns_flash_type
zns_pg_wr_lat	int64	0	ZNS: page program time in ns, 0 or more; 0 uses the built-in time of zns_flash_type
zns_blk_er_lat	int64	0	ZNS: block erase time in ns, 0 or more; 0 uses the built-in time of zns_flash_type
zns_cmd_addr_lat	int64	0	ZNS: command and address phase on the channel bus in ns, 0 or more
zns_pg_xfer_lat	int64	0	ZNS: page data transfer phase on the channel bus in ns, 0 or more
zns_status_lat	int64	0	ZNS: status read phase on the channel bus in ns, 0 or more
zns_pe_suspend	int32	0	ZNS: non-zero lets a read suspend a program or erase on its plane

Property	Type	Default	Description
zns_lun_contention	int32	0	ZNS: non-zero makes the planes of one LUN share the die, so their operations run one at a time; a zone reset erases each LUN's planes in one multi-plane operation
zns_reset_by_fill	int32	0	ZNS: non-zero makes a zone reset erase only the blocks that hold programmed pages, so its time and wear follow the zone's fill
zns_fzr_sec	uint32	0	ZNS: seconds a zone may stay written but not finished before the controller sets Finish Zone Recommended and reports the zone as changed; 0 disables
zns_frl_sec	uint32	0	ZNS: Finish Recommended Limit in seconds: after it, the controller finishes a zone with Finish Zone Recommended itself (a Zone Active Excursion); 0 means no excursion; needs zns_fzr_sec
zns_rzr_sec	uint32	0	ZNS: seconds a zone may stay Full before the controller sets Reset Zone Recommended and reports the zone as changed; 0 disables
zns_rrl_sec	uint32	0	ZNS: Reset Recommended Limit in seconds: after it, the controller refreshes a zone with Reset Zone Recommended itself and clears it; 0 means no refresh; needs zns_rzr_sec
zns_page_types	int32	0	ZNS: non-zero gives each read the latency of its page type (lower to upper page), spread around the read latency of the flash type; programs keep one latency per wordline
zns_ecc_step_ns	int64	0	ZNS: ECC read time in ns added per 750 erases of the block, up to 4 steps, as bbssd ecc_step_ns; 0 or more
zns_tsusp_ns	int64	0	ZNS: overhead in ns added to a read that suspends a program or erase, 0 or more
zns_max_active	uint32	0	ZNS: Maximum Active Resources (zones), at most the zone count; 0 means no limit
zns_max_open	uint32	0	ZNS: Maximum Open Resources (zones), at most the zone count and zns_max_active; 0 means no limit
zns_num_wc	uint32	0	ZNS: number of zone write caches; 0 uses zns_max_open, or 3 when that is 0
zns_zd_ext_size	uint32	0	ZNS: zone descriptor extension size in bytes, a multiple of 64 up to 16320; 0 means none
zns_num_conv_zones	uint32	0	ZNS: number of leading conventional zones, which take random writes; capped at the zone count
zns_zone_cap	size	0	ZNS: zone capacity in bytes, at least one logical block and at most the zone size; 0 means the zone size
zns_chnls_per_zone	uint32	0	ZNS: channels one zone spans (zone width), dividing zns_num_ch; 0 means all channels
zns_luns_per_zone	uint32	0	ZNS: LUNs one zone spans on each of its channels, dividing zns_num_lun; 0 means all LUNs
zns_zone_order	str	unset	ZNS: ch (default) fills a zone channel first, then LUN; lun fills it LUN first, so consecutive programs share a channel
zns_zrwa_size	uint64	0	ZNS: Zone Random Write Area size in logical blocks, at most 65535 and a multiple of zns_zrwafg_size; 0 disables ZRWA
zns_zrwafg_size	uint64	0	ZNS: ZRWA flush granularity in logical blocks, 1 to 65535 with ZRWA and 0 without; the zone capacity must be a multiple of it
zns_zrwa_num	uint32	0	ZNS: number of zones that may hold a ZRWA at once, 1 or more with ZRWA and 0 without
zns_cross_zone_read	bool	off	ZNS: allow reads that cross zone boundaries (Read Across Zone Boundaries)

Property	Type	Default	Description
zns_zasl_bs	uint32	131072	ZNS: Zone Append size limit in bytes, a power-of-two multiple of 4 KiB; 0 follows mdts

OCSSD (Open-Channel)

Property	Type	Default	Description
lver	uint8	2	OCSSD: Open-Channel version, 1 for 1.2 or 2 for 2.0; other values fail realize
flash_type	uint8	2	OCSSD: cell type for the built-in timing tables, 1 SLC, 2 MLC, 3 TLC or 4 QLC; other values fail realize
oc12_channel_timing	bool	off	OCSSD 1.2: charge channel transfer time for each page accessed, scaled by the sectors used out of lsecs_per_pg; ch_xfer_lat sets ns per page and 0 uses the flash_type value; off, transfers take no time
oc20_page_types	bool	off	OCSSD 2.0: charge each read and program the time of its page type (lower to upper page) from the flash_type tables, as OCSSD 1.2 does; off, every page takes the lower page time
lsec_size	uint16	4096	OCSSD 1.2: sector size in bytes reported in the geometry, greater than 0 in both versions; data moves at the namespace block size, and OCSSD 2.0 always uses 4096
lsecs_per_pg	uint8	4	OCSSD: sectors per page, greater than 0
lpgs_per_blk	uint16	512	OCSSD: pages per block, greater than 0 and at most 512 for OCSSD 1.2
lmax_sec_per_rq	uint8	64	OCSSD 1.2: most sectors in one vector command; OCSSD 2.0 uses 64
lnum_ch	uint8	2	OCSSD: channels (2.0 groups), 1 to 32, with lnum_ch * lnum_lun at most 128
lnum_lun	uint8	8	OCSSD: LUNs (2.0 parallel units) per channel, 1 or more, with lnum_ch * lnum_lun at most 128
lnum_pln	uint8	2	OCSSD: planes per LUN, greater than 0; OCSSD 1.2 accepts 1, 2 or 4
lmetasize	uint16	16	OCSSD 1.2: out-of-band metadata bytes per sector; OCSSD 2.0 uses 16
learly_reset	uint8	0	OCSSD 2.0: non-zero reports the early reset capability, so the host may reset a chunk it has not filled

CSD (computational storage)

Property	Type	Default	Description
fdm_size	uint64	0	CSD: functional data memory size in MiB; required, greater than 0
nr_cu	uint8	4	CSD: number of compute units, 1 to 64; programs wait for the first free unit
nr_thread	uint8	4	No effect, kept for CEMU compatibility; CSD still refuses 0. A value other than the default warns at realize
time_slice	uint64	200000	No effect, kept for CEMU compatibility. A value other than the default warns at realize
context_switch_time	uint64	200	No effect, kept for CEMU compatibility. A value other than the default warns at realize
csf_runtime_scale	uint16	3	CSD: non-zero multiplier applied to the host run time of a program that sets neither a runtime nor its own scale

Property	Type	Default	Description
csd_program_dir	str	unset	CSD: host directory that shared-object and uBPF programs are loaded from, named by a file name with no slash that must resolve inside it; unset allows only the built-in phantom program type
csd_slm_size	uint32	0	CSD: MiB of a memory namespace (NVMe SLM 1.2, CSI 03h) added after the configured namespaces, 1 to 1024; 0 adds none
csd_cp	bool	off	CSD: add a compute namespace (NVMe CP 1.2, CSI 04h) after the memory namespace; needs csd_slm_size
csd_legacy_opcodes	bool	off	CSD: take the old CEMU admin opcodes 21h, 22h, 23h and 25h instead of the vendor specific C5h, C8h, C9h and CDh; needs a CSD namespace
csd_pause	bool	off	CSD: stop the guest, its vCPUs and clocks, while the host runs a program, so the guest sees only the modelled run time; needs KVM on x86 (host Linux 6.0 or later), the guest TSC at the host rate, and only bbssd and CSD namespaces, none shared
csd_pause_max_ms	uint32	1000	CSD with csd_pause: the longest stop, 1 to 60000 ms; past it the guest runs again and sees the rest of the host run

KV timing

Property	Type	Default	Description
kv_timing	str	unset	KV: media (default) times every command on the NAND model; model times Store and Retrieve from the workload mix of the last 16 of them, with the Samsung KVSSD emulator fit
kv_model_coeffs	str	unset	KV with kv_timing=model: 36 comma-separated numbers, the intercept then the 35 polynomial coefficients; unset uses the Samsung fit
kv_index	str	unset	KV: the key index; hash (default) is an open-addressed hash in device DRAM, lsm a PinK-style LSM tree whose unpinned levels cost flash reads on lookup and flash reads and programs on compaction
kv_lsm_memtable_keys	uint32	1024	KV with kv_index=lsm: keys the DRAM memtable holds before it flushes to level 1, 16 to 65536
kv_lsm_size_ratio	uint32	10	KV with kv_index=lsm: how many times more entries a level holds than the one above, 2 to 64; level i holds memtable keys times ratio^i entries
kv_lsm_pinned_levels	uint32	1	KV with kv_index=lsm: top levels held whole in DRAM, with no flash I/O, 0 to 24
kv_pack	bool	on	KV: on (default) packs values below a page into shared NAND pages through an open page in controller DRAM; off gives each value whole pages, the model before packing
kv_max_keys	uint32	0	KV: the key limit of each KV namespace, reported as MNK; 0 (default) sizes it from the flash: with kv_pack the 1-byte values it holds, at most 4194303, else the old two index slots per 4 KiB; 1 to 1610612735 sets it, at 85 bytes of hash index DRAM a key

Generic PCI device properties

Common to every QEMU PCI device and documented by QEMU: acpi-index, addr, busnr, failover_pair_id, multifunction, rombar, romfile, romsize, sriov-pf. QEMU's internal compatibility properties (x-max-bounce-buffer-size, x-pcie-ari-nextfn-1, x-pcie-err-unc-mask, x-pcie-ext-tag, x-pcie-extcap-init,

x-pcie-lnksta-dllla) are not listed.

A.1.2 femu-subsys: NVMe subsystem

-device femu-subsys has 8 properties of its own.

Shared namespaces

Property	Type	Default	Description
ns_mgmt	bool	off	Keep one namespace table and one backend in the subsystem, shared by every controller that names it with subsys=; NoSSD and bbssd controllers only, and not with fdp
nqn	str	unset	Subsystem name reported as nqn.2019-08.org.qemu:<nqn> by controllers that share namespaces through this subsystem; unset uses the device id

Flexible Data Placement

Property	Type	Default	Description
fdp	bool	off	Enable Flexible Data Placement in endurance group 1 for controllers that join this subsystem; only bbssd places data by reclaim unit
fdp.runs	size	0	Reclaim unit size in bytes; 0 means 96 MiB, and a bbssd controller accepts only 0 or the size of one superblock of the dies of a reclaim group, which it then uses
fdp.nrg	uint32	1	Number of FDP reclaim groups, from 1 to 128; with bbssd a group is a set of dies (whole channels, or LUNs of one channel), nrg must divide nchs or be a multiple of it that divides nchs * luns_per_ch, a logical block must be whole NAND pages, and each group must hold the whole namespace
fdp.nruh	uint16	0	Number of FDP reclaim unit handles (placement handles), from 1 to fdp.nru and at most 16366; must be set when fdp=on
fdp.nru	uint64	128	Number of reclaim units in each reclaim group, from fdp.nruh to 65536; bbssd uses at most one per superblock and needs at least 2 * fdp.nruh + 1 of them, and the namespace must fit in the units left once each handle has one open, each Persistently Isolated handle one to collect into, and forced collection its free ones
fdp.isolation_mode	uint32	0	0 makes every reclaim unit handle Persistently Isolated; any other value makes the last handle Initially Isolated

A.1.3 femu-cxl-ssd: CXL Type-3 SSD

-device femu-cxl-ssd has 28 properties of its own, 10 inherited from cxl-type3 and 80 QOM properties listed in runtime-properties.md (§A.2).

Cache

Property	Type	Default	Description
cache-pages	uint32	1024	Number of 4 KiB pages the device cache holds; 0 sends every access to the media, otherwise at most the media page count and divisible by cache-ways
cache-policy	str	unset	Cache replacement policy, one of fifo, lifo, clock or s3-fifo; unset is fifo

NAND geometry and timing

Property	Type	Default	Description
ftl	bool	on	Charge cache misses and write-backs to the FTL and NAND model; off keeps memory behaviour with no media timing and cannot be linked to an NVMe controller
channels	uint32	4	Number of NAND channels, 1 to 4096
luns-per-channel	uint32	4	NAND LUNs per channel, 1 to 128, with one plane per LUN
pages-per-block	uint32	256	4 KiB pages per NAND block, 1 to 65536
blocks-per-plane	uint32	0	NAND blocks per plane, 2 to 65536; with the FTL on, the lines beyond the free lines gc-threshold-high keeps and two more must hold the media plus 7%; 0 sizes it to 5/4 of the media plus 4 blocks per plane, or more if that rule needs it
gc-threshold	uint32	75	Percent of lines in use at which background garbage collection starts, 1 to 100
gc-threshold-high	uint32	95	Percent of lines in use at which garbage collection is forced, from gc-threshold to 100
read-ns	uint64	40000	NAND page read time in ns, at most one second
program-ns	uint64	200000	NAND page program time in ns, at most one second
erase-ns	uint64	2000000	NAND block erase time in ns, at most one second
channel-ns	uint64	0	NAND channel transfer time per page in ns, at most one second
cylon-first-touch-program	bool	off	Charge a NAND program instead of a free read when a read reaches a page the FTL has never mapped, as the Cylon experiments do
cylon-free-writeback	bool	off	Write dirty pages back on eviction and flush with no NAND program and no media time, as the Cylon experiments do

Direct mapping (DER)

Property	Type	Default	Description
der	str	unset	Direct mapping of cached pages into the guest: off (MMIO only, the default), memslot (KVM memory slot aliases, not under TCG) or cylon (a Cylon host kernel)
der-replace-rate	uint32	64	With der=memslot and no free alias (1024 shared by all devices, fewer if KVM has fewer free slots), the most aliases per second a repeatedly missing page may displace; 0 disables replacement
cylon-kernel-ack	bool	off	Must be on with der=cylon to state that the host runs a Cylon kernel with the dual-slot fixes; the device does not check it
cylon-emul-exit	bool	on	With der=cylon, ask the host kernel to return accesses it cannot decode on unmapped pages to FEMU, which maps the page; off keeps stock KVM behaviour (a guest #UD or an internal error) only if no other device of the VM turned the VM-wide capability on
cylon-never-emulate	OnOffAuto	auto	With der=cylon and cylon-emul-exit on, version 2 of the Cylon fault exit: an access to a cold page exits with its type and FEMU maps the page, so KVM emulates only pages FEMU cannot map; auto (the default) uses it when the host kernel offers it and otherwise warns once and uses version 1, on also warns when it cannot, off keeps version 1; per VM, set by the first Cylon device that installs its slot

Property	Type	Default	Description
cylon-fault-stop	uint32	100000	With der=cylon, stop the VM after this many consecutive fault exits of one vCPU at one RIP on pages already filled for that RIP; a watchdog that counts exits, not instructions, so a healthy loop over more pages than the cache holds can reach it; 0 only warns; default 100000
cylon-revoke-batch	uint32	64	With cylon-never-emulate on, the most pages one full revocation takes, 1 to 256 (default 64): an eviction that must flush the VM's TLBs also revokes the mappings of the pages the policy evicts next, which stay cached; 1 revokes one page per two flushes
concurrent-misses	OnOffAuto	auto	Let misses to different pages wait for the media together; auto does so only while direct mapping is active

Caching API, control channel and logs

Property	Type	Default	Description
cca	bool	off	Expose the caching API on BAR 5 (pin, unpin, invalidate, uncached ranges, query) and start its worker thread
lsa-control	bool	off	Accept experiment control commands through Get LSA on an internal 128 MiB label area; trusted guests only, and not with an lsa backend
log-dir	str	unset	Host directory for cxlssd-stats.log, cxlssd-io-N.log and cxlssd-spt.log; unset is the working directory
tracefs-dir	str	unset	Host tracefs directory whose tracing control commands 91 and 81 write; unset, those commands change nothing on the host
log-limit	size	67108864 (64 MiB)	Size limit in bytes for each log file the device writes; 0 opens no I/O log and takes no statistics appends

Inherited from `cxl-type3`

These belong to QEMU's `cxl-type3` device. The descriptions say how `femu-cxl-ssd` treats them.

Property	Type	Default	Description
cdat	str	unset	Host file holding the CDAT table returned over DOE; unset builds a table from the backends; femu-cxl-ssd leaves it unchanged
lsa	link<memory-backend>	unset	Label storage area backend served by Get and Set LSA; femu-cxl-ssd accepts it only with lsa-control=off
memdev	link<memory-backend>	unset	Legacy persistent memory backend of cxl-type3; femu-cxl-ssd refuses it at realize
num-dc-regions	uint8	0	Number of dynamic capacity regions; femu-cxl-ssd requires it to stay 0
persistent-memdev	link<memory-backend>	unset	Persistent memory backend of cxl-type3; femu-cxl-ssd refuses it at realize
sn	uint64	18446744073709551615	PCIe Device Serial Number; the default (2^64 - 1) means unset, which gives no serial number capability; femu-cxl-ssd leaves it unchanged
volatile-dc-memdev	link<memory-backend>	unset	Dynamic capacity memory backend; femu-cxl-ssd refuses it at realize

Property	Type	Default	Description
volatile-memdev	link<memory-backend>	unset	Required: ID of the host memory backend that holds the device data, a non-zero multiple of 256 MiB, at most 120 GiB; der=cylon needs a hugetlbfs file backend with share=on and prealloc=on, or direct mapping falls back to MMIO with a warning
x-speed	PCIELinkSpeed	32	PCIe link speed the device reports; femu-cxl-ssd leaves it unchanged and its timing does not depend on it
x-width	PCIELinkWidth	16	PCIe link width the device reports; femu-cxl-ssd leaves it unchanged and its timing does not depend on it

Generic PCI device properties

Common to every QEMU PCI device and documented by QEMU: acpi-index, addr, busnr, failover_pair_id, multifunction, rombar, romfile, romsize, sriov-pf. QEMU's internal compatibility properties (x-max-bounce-buffer-size, x-pcie-ari-nextfn-1, x-pcie-err-unc-mask, x-pcie-ext-tag, x-pcie-extcap-init, x-pcie-lnksta-dllla) are not listed.

A.1.4 Test-only properties

These exist only when QEMU runs under qtest (-accel qtest) and serve FEMU's own tests. They are not part of the user interface.

- femu: x-cmt-slots, x-copy-held, x-copy-hold, x-csd-pause-stats, x-eol-enter, x-fdp-check, x-fdp-ns-mapped, x-ftl-check, x-ftl-hold, x-ftl-trace, x-ns-test, x-oc12-clock, x-oc12-trace, x-query-delay-ms, x-range-waits, x-stream-test, x-tier-check, x-tier-fold-stop, x-tp-hold, x-zone-draining, x-zone-held, x-zone-lock-hold-ms, x-zone-log-changes, x-zone-log-hold-ms, x-zone-report-hold-ms, x-zone-write-hold-ms
- femu-cxl-ssd: test-change-dpa, test-fault, test-fault-decode, test-fault-delivery, test-fault-fill, test-fault-repeats, test-fault-version, test-fault-walk, test-fill, test-fill-race, test-ftl-delay, test-ftl-hold, test-ftl-holding, test-guarded-write, test-lock-mutex, test-lock-wanted, test-map, test-media-disabled, test-nvme-after-posted, test-owner, test-posted-done, test-prefetch-race, test-prefetch-race-end, test-protect, test-protect-window, test-revoke-ahead, test-revoke-ahead-keep, test-rip, test-slot-reservation, test-storm, test-storm-ns, test-storm-served, test-storm-stops, test-storm-wait, test-thread-read, test-unprotect

A.1.5 Environment variables

FEMU reads these variables from the environment of the QEMU process. They are debugging and host-placement aids, not device configuration, so they have no -device property. When QEMU runs under sudo, pass them through, for example sudo FEMU_EXP_LOG=1 ./run-blackbox.sh or sudo -E.

Variable	Read by	Effect
FEMU_MBE_INTERLEAVE	memory backend, every mode (hw/femu/backend/dram.c)	on interleaves the backend memory across NUMA nodes 0 and 1; 0 or 1 binds it to that node. Other values are ignored with a message. Unset leaves the host default policy.
FEMU_FDP_DEBUG	bbssd FTL (hw/femu/bbssd/ftl.c)	Any value, even empty, prints FDP placement and reclaim traces to stderr.
FEMU_EXP_LOG	bbssd FTL (hw/femu/bbssd/ftl-exp.c)	A non-empty value prints [EXP] lines to stderr that trace the writes, overwrites, deallocations, garbage collection moves and erases of pages whose data contains FEMU_SECRET; without FEMU_SECRET nothing is traced.
FEMU_SECRET	bbssd FTL (hw/femu/bbssd/ftl-exp.c)	A non-empty marker string that selects the pages FEMU_EXP_LOG traces.
FEMU_DUMP_LPN	bbssd FTL (hw/femu/bbssd/ftl-exp.c)	A logical page number (decimal or 0x hex) whose backend page is hex-dumped to stderr on every read not served from the write buffer, independent of FEMU_EXP_LOG.
FEMU_KV_SELFTEST	KV FTL (hw/femu/kvssd/kvssd-ftl.c)	Any value runs the KV FTL self-test once at realize and logs the result.
FEMU_KV_CHECK	KV FTL (hw/femu/kvssd/kvssd-ftl.c)	Any value checks the owner of every NAND page after each value GC and every 64 commands, and stops QEMU on a broken invariant. The qtests set it. It is slow on a large device.

A.2 Runtime properties and counters

QOM properties that a running device exposes. Read and set them through QMP at /machine/peripheral/<id>, where <id> is the id= given on -device:

```
{"execute": "qom-get", "arguments": {"path": "/machine/peripheral/cxl-ssd0", "property": "read-hits"}}
{"execute": "qom-set", "arguments": {"path": "/machine/peripheral/cxl-ssd0", "property": "stats-reset", "value": true}}
```

or in the HMP monitor with qom-get and qom-set. Event counters are cleared by stats-reset; gauges and totals keep their value. Descriptions are the ones -device <type>,help prints.

A.2.1 femu: NVMe controller

Power loss trigger

Property	Type	Description
simulate-power-loss	bool	Write-only: setting true on a realized controller with power_loss=on drops the write buffer and resets the controller as a power cut would

A.2.2 femu-cxl-ssd: CXL Type-3 SSD

Cache tunables (also accepted on -device)

Property	Type	Description
cache-ways	uint32	Entries per cache set, non-zero and dividing cache-pages (1 is direct mapped); default 16, and a qom-set at run time writes back and rebuilds the cache
prefetch-degree	uint32	Up to this many pages inserted into the cache after each demand miss, at most the media page count, with no NAND read; default 0, changeable with qom-set

Property	Type	Description
prefetch-stride	uint32	Distance in pages from a missed page to the first prefetched page, at most the media page count; default 1, changeable with qom-set

Actions and control

Property	Type	Description
der-ratio	uint64	Run time only: direct-map this fraction of pages, one of 0, 50, 75, 90, 95, 97, 98, 99, 995 (99.5%), 999 (99.9%) or 100 percent; needs der other than off and no uncached ranges, and der=memslot refuses a ratio whose unmapped gaps outnumber the free aliases
control-command	uint64	Run time only: writing runs that experiment control command with control-argument before qom-set returns; reading gives the last command
control-argument	uint64	Argument for the next control-command write; reading gives the last argument used, which a Get LSA control command also sets
control-status	uint64	Read-only result of the last control command: 0 success, 1 error, 2 a Get LSA command still queued
flush-cache	bool	Write-only: true revokes direct mappings, writes dirty pages back, drops unpinned pages and waits for the modelled media time
stats-reset	bool	Write-only: true copies the counters to the last-* properties, then clears the event counters
fast-load	bool	Accesses skip only their wait for the modelled media time; the FTL, cache and counters still run. For warmup and loading, not for measurement. Setting false does not wait for the queued NAND work (see nand-idle-ns); default off, changeable with qom-set
fast-load-drain-ns	uint64	Read-only: the NAND backlog, in ns from when fast-load last went to false, including device DMA work queued before the switch; the switch does not wait for it. At least 1 until that work is booked
fast-load-switch-ns	uint64	Read-only: ns the last setting of fast-load to false took inside the device, for checking that the switch does not wait for the backlog; 0 before the first
nand-idle-ns	uint64	Read-only: ns until the NAND timelines and queued NAND work are idle, 0 when idle; poll it after fast-load=false before measuring. Never blocks

Cache counters

Property	Type	Description
cache-entries	uint64	Read-only gauge: pages resident in the cache, pinned ones included
cache-hits	uint64	Read-only event counter: MMIO page lookups that found a resident page
cache-misses	uint64	Read-only event counter: MMIO page lookups that missed, including accesses with no cache
read-hits	uint64	Read-only event counter: cache-hits caused by reads
read-misses	uint64	Read-only event counter: cache-misses caused by reads
write-hits	uint64	Read-only event counter: cache-hits caused by writes
write-misses	uint64	Read-only event counter: cache-misses caused by writes
cache-inserts	uint64	Read-only event counter: pages admitted by demand misses, prefetch and PIN fills
cache-evictions	uint64	Read-only event counter: pages the replacement policy removed, including those a flush or a cache-ways change drops

Property	Type	Description
prefetch-inserts	uint64	Read-only event counter: pages inserted by prefetch

Snapshot counters

Property	Type	Description
last-read-hits	uint64	Read-only: read-hits at the last stats-reset or control command 1
last-read-misses	uint64	Read-only: read-misses at the last stats-reset or control command 1
last-write-hits	uint64	Read-only: write-hits at the last stats-reset or control command 1
last-write-misses	uint64	Read-only: write-misses at the last stats-reset or control command 1
last-inserts	uint64	Read-only: cache-inserts at the last stats-reset or control command 1
last-evictions	uint64	Read-only: cache-evictions at the last stats-reset or control command 1
last-entries	uint64	Read-only: cache-entries at the last stats-reset or control command 1
last-prefetch-inserts	uint64	Read-only: prefetch-inserts at the last stats-reset or control command 1

Media counters

Property	Type	Description
media-time-ns	uint64	Read-only: total modelled media time in ns returned by FTL requests, including contention
media-reads	uint64	Read-only: page reads the FTL performed for cache fills, uncached reads and PIN fills, not counting reads that cylon-first-touch-program turned into programs; stays 0 with ftl=off
media-writes	uint64	Read-only: user page programs counted by the FTL, garbage collection copies excluded, refreshed at each media request of this device, so writes from a linked NVMe controller appear after the next one
media-full	uint64	Read-only: NAND programs that found no free page after garbage collection; the program is not timed but does not stop the eviction or insert, and the first one reports an error; the over-provisioning rule keeps it at 0, stats-reset keeps it, and a measurement is valid only while it is 0
gc-stalls	uint64	Read-only: media requests, linked NVMe and device DMA ones included, that waited for timed forced garbage collection to free a line; refreshed with media-writes; stats-reset keeps it
gc-stall-ns	uint64	Read-only: total ns from the start of those requests to the end of the collection they waited for, on every LUN; stats-reset keeps it
gc-stall-max-ns	uint64	Read-only: the longest of those waits in ns; one over a second warns once per device; stats-reset keeps it
dma-accesses	uint64	Read-only: accesses made inside another device's MMIO handler or bottom half, typically its DMA; served without waiting, they never fill the cache and are not cache hits or misses
dma-media-ops	uint64	Read-only: media operations queued for dma-accesses to uncached pages, one per run of consecutive accesses to a page within one MMIO handler or bottom half, plus one per run of writes to a page whose write-back is in progress; nobody waits for them; stays 0 with ftl=off
dma-media-time-ns	uint64	Read-only: modelled media time of dma-media-ops, not part of media-time-ns; a refused program counts in media-full

Direct mapping counters

Property	Type	Description
der-active	bool	Read-only: whether direct mapping is available on this device
der-probes	uint64	Read-only: probe attempts, one per realize with der=cylon
der-mapped	uint64	Read-only gauge: pages currently mapped for direct guest access
der-remaps	uint64	Read-only: direct page mappings installed
der-revocations	uint64	Read-only: direct page mappings removed
der-quiet-revocations	uint64	Read-only: Cylon revocations of entries whose accessed bit was clear, done without a TLB flush
der-replacements	uint64	Read-only: memslot aliases displaced by a hotter page
der-fallbacks	uint64	Read-only: refused direct mapping attempts and device disablements
der-emul-exit	bool	Read-only: whether the host kernel returns Cylon accesses it cannot emulate to FEMU (KVM_CAP_CYLON_FAULT_EXIT)
der-emul-v2	bool	Read-only: whether version 2 of the Cylon fault exit is on for the VM (cylon-never-emulate)
der-fault-reads	uint64	Read-only: version 2 exits for a data read of a cold page; stats-reset keeps it
der-fault-writes	uint64	Read-only: version 2 exits for a data write to a cold page; stats-reset keeps it
der-fault-fetches	uint64	Read-only: version 2 exits for an instruction fetch from a cold page; stats-reset keeps it
der-fault-page-walks	uint64	Read-only: version 2 exits for a guest page walk that read a cold page-table page; stats-reset keeps it
der-fault-emulated	uint64	Read-only: version 2 pages handed back to KVM's emulator for a data access: only pages that the caching API keeps uncached (uncached range, every way of the set pinned); stats-reset keeps it
der-fault-unprotected	uint64	Read-only: version 2 protections released early because the instruction already held 64 pages; stats-reset keeps it
der-fault-conflicts	uint64	Read-only: fills refused because pages that the same instruction filled hold every way of the set; the page maps outside the cache (der-fault-overflows); stats-reset keeps it
der-fault-overflows	uint64	Read-only: pages mapped without a cache way for one instruction: in version 2 every page that cannot keep a way, in version 1 an instruction the emulator cannot run whose pages do not fit in their set; stats-reset keeps it
der-fault-forced	uint64	Read-only: of der-fault-overflows, pages that the caching API keeps uncached, mapped because a guest page walk or an event delivery touched them, which the emulator cannot serve; stats-reset keeps it
der-fault-deliveries	uint64	Read-only: version 2 exits made while the CPU delivered an interrupt or exception (KVM_CYLON_FAULT_DELIVERY; host kernel candidate 3); stats-reset keeps it
der-fault-marker-refused	uint64	Read-only: version 2 emulation markers that FEMU refused to write because the rule allows them only for data accesses to uncached pages; nonzero is a defect, and the exit stops the VM; stats-reset keeps it
der-revoke-flushes	uint64	Read-only: TLB flushes that Cylon full revocations attempted (two, or one when KVM revoked every page first or the slot is deleted next), shared by every page a revocation takes; flushes after a KVM revocation found outside a full revocation are not counted; stats-reset keeps it
der-revoked-ahead	uint64	Read-only: version 2 pages whose mapping a revocation took before their own eviction; they stay cached; stats-reset keeps it
der-ahead-remaps	uint64	Read-only: of der-revoked-ahead, pages accessed and mapped again before their eviction; stats-reset keeps it

Property	Type	Description
der-fault-bql	uint64	Read-only: Cylon fault exits that FEMU served under the BQL instead of the CXL lock alone (the first exit after an invalidation, a route through a switch or to another device); stats-reset keeps it
der-emul-fills	uint64	Read-only: exits for an access KVM could not decode that FEMU served by a fill and a mapping, repeats and cache hits included; not instructions or unique pages; stats-reset keeps it
der-emul-fetch-fills	uint64	Read-only: the der-emul-fills exits for code the guest executed from an unmapped page; stats-reset keeps it
der-emul-failures	uint64	Read-only: such exits FEMU could not serve (unmappable page or no progress); each one stops the VM; stats-reset keeps it

Caching API counters

Property	Type	Description
cca-commands	uint64	Read-only event counter: caching API commands completed, whatever their status
cca-errors	uint64	Read-only event counter: caching API commands completed with a non-zero status
cca-pinned	uint64	Read-only gauge: pages currently pinned in the cache
cca-uncached	uint64	Read-only gauge: pages currently in caching API uncached ranges
cca-pin-fills	uint64	Read-only event counter: pages PIN read from the media
cca-writebacks	uint64	Read-only event counter: dirty pages programmed by caching API commands
cca-dropped	uint64	Read-only event counter: resident pages dropped by INVALIDATE and CACHE_DISABLE
cca-pinned-set-misses	uint64	Read-only event counter: misses served from the media because every way of the set is pinned

Other counters

Property	Type	Description
invalidations	uint64	Read-only: generation count bumped by PCI configuration writes, component register writes, every mailbox command other than a control Get LSA, and reset; each revokes all direct mappings
nvme-drops	uint64	Read-only: resident pages a linked NVMe write, copy or deallocate dropped or cleaned
log-dropped	uint64	Read-only: statistics appends refused, I/O logs closed at log-limit and SPT dumps truncated

B Log pages and counters

How to read FEMU's own counters from the guest. Device properties are in properties.md (§A.1), and the counters of `femu-cxl-ssd` are QOM properties listed in runtime-properties.md (§A.2).

B.1 Vendor log page C0h

`nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b` returns the emulator's own media counters, little-endian at these offsets (`FemuStatsLog` in `hw/femu/nvme.h`):

Offset	Size	Field
0	4	Write amplification factor, scaled by 1000: (pages programmed + pages relocated by garbage collection + pad pages at offset 264 + translation pages at offset 280) x 1000 / pages the host asked to program
8	8	Pages the host asked to program
16	8	Data pages relocated by garbage collection
24	8	Pages actually programmed
32	8	Reads of the most-read block since its erase
40	8	Lines rewritten because of read stress (reclaim units under FDP)
48	8	Lines rewritten because of retention age (reclaim units under FDP)
56	8	Host read pages the write buffer saw
64	8	Of those, pages the buffer held
72	8	Host write pages the write buffer saw
80	8	Of those, pages the buffer already held
88	8	Log-block switch merges (`mapping=hybrid` only)
96	8	Log-block full merges
104	8	Erases charged to log-block merges
112	8	Blocks in service past their erase limit (`blk_pe_limit`)
120	8	Worn-out blocks taken out of service
128	8	Lines taken out of service (under FDP, the lines of retired reclaim units)
136	8	Lines moved by static wear levelling (`wl_spread`)
144	8	Pages those moves copied (also counted at offset 16)
152	8	NAND pages host writes covered only in part (a device reads such a page to program it again; FEMU does not charge that read)
160	8	Plane reads charged to NAND (a multi-plane command counts each plane)
168	8	Plane programs
176	8	Plane erases
184	8	Energy in uJ: the three counts above times `energy_read_nj`, `energy_prog_nj` and `energy_erase_nj`
192	8	Host writes that waited for forced garbage collection to make room
200	8	Forced collection passes run inside those writes
208	8	Host writes that emptied a full write buffer to make room
216	8	Completions held because the host's completion queue was full (summed over pollers)
224	8	Pages paced collection copied (`gc_pace`; also counted at offset 16)
232	8	Lines paced collection finished, freed or retired
240	8	Paced lines the forced pass or the budget finished in one pass
248	8	Pages collection moved by copyback on their die (`gc_copyback`; also counted at offset 16)

Offset	Size	Field
256	8	Lines (reclaim units under FDP) rewritten for the open-block read limit (`pd_open_read_limit`); not counted at offset 40
264	8	Pages programmed with pad data to close lines or units left open (`pd_open_close_sec`)
272	8	Lines or units those pads closed
280	8	Translation pages programmed (`mapping=dftl` without FDP): dirty CMT evictions, published changes, and translation pages that collection, refresh or wear levelling moved
288	8	Translation pages read from flash (`mapping=dftl` without FDP)

Bytes 4-7 and 296-511 are reserved and read as zero. The counters are summed over the controller's bbssd, CSD and KV namespaces (the block read count is the largest of them); other modes leave them zero. The write amplification factor stays zero until the host has written a page. Bytes 8 and 24 are equal when no write buffer is configured; a buffer that absorbs repeated writes to one page makes the factor drop below 1.

A KV namespace with `kv_index=lsm` adds its merge programs to offset 24. It adds the meta pages that value GC moves to offset 16. Thus its factor includes the index.

With value packing (`kv_pack=on`, the default), a KV namespace counts host data at offset 8. The count is the value bytes of its Stores, in whole pages, rounded down. At offset 24 it counts each packed page and each whole value page it programs. Thus the factor is programmed pages over host data, and it drops below 1 when the open page drops overwritten values. With `kv_pack=off`, offset 8 counts the pages of each value, as before (KV value packing (§4.24.7)).

They were previously written into the SMART log from byte 192, which NVMe Base 2.0 assigned to the composite temperature times, the temperature sensors and the thermal transition counts.

The host can read the same page counters per namespace without the guest, together with the line states, through the QMP command query-femu[137].

The same counters can be captured through the standard Telemetry Host-Initiated log (07h): `nvme telemetry-log /dev/nvme0 --output-file=telemetry.bin` takes a snapshot and saves it. Data Area 1 is one 512-byte block laid out as above, and it stays as captured until the next capture. The Controller-Initiated log (08h) never holds data, because the controller does not capture on its own.

`nvme get-log /dev/nvme0 --log-id=0 --log-len=1024 -b` lists every log page the controller answers, four bytes per identifier with bit 0 set for the ones it supports, so this page can be discovered rather than assumed.

B.2 Computational programs and reachability

A CSD controller with `csd_slm_size` or `csd_cp` answers these pages (CP and SLM (§4.22)):

LID	Page	CSI	NSID	Content
1Ah	Reachability Groups	any	any	one group, RGID 1, with every namespace; LSP bit 0 (RGO) leaves out the NSIDs
1Bh	Reachability Associations	any	any	one association, RASID 1, RAC 01h, holding RGID 1; LSP bit 0 (RAO) leaves out the RGIDs
82h	Program List	04h	the compute namespace	16 descriptors, one per PIND

[137] `https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md`

LID	Page	CSI	NSID	Content
83h	Downloadable Program Types List	04h	the compute namespace	C3h with `csd_program_dir`; C1h also with uBPF
84h	Memory Range Set List	04h	the compute namespace	RSID 0h, then every set by RSID; LSP bit 0 (RIO) leaves out the ranges

Both change counts of 1Ah and 1Bh stay 0h, because the groups never change.

B.3 Asynchronous events

The controller completes an outstanding Asynchronous Event Request when one of these happens:

Event	Raised when	Log page it names
SMART temperature warning	the host has enabled it with Async Event Configuration and the reported temperature (`temperature`, in Kelvin, default 323, which is 50 C) is at or above the over threshold (default 343 K, the warning temperature WCTEMP) or at or below the under threshold (default 0); with the thermal model on, also when the modelled temperature crosses one	SMART / Health (02h)
Error	the host writes a doorbell that does not exist, or a value past the end of its queue	Error Information (01h)
Namespace Attribute Changed	with `ns_mgmt=on`, a namespace is attached, detached, deleted or formatted, and the host enabled the notice	Changed Namespace List (04h)
SMART spare below threshold (02h), reliability (00h)	with `blk_pe_limit`: the spare falls below 20, the first block stays in service past its limit, or the media goes read-only; bit 0, 2 or 3 of Async Event Configuration enables each	SMART / Health (02h)
Zone Descriptor Changed	an injected write fault (`err_write_fail_ppm`) made a ZNS zone read only, and the host enabled Zone Descriptor Changed notices (bit 27)	Changed Zone List (BFh)

An event of a given type is reported once and then held back until the host reads the log page it named without Retain Asynchronous Event (RAE), so the same condition is not reported again before the host has looked. A controller reset drops anything outstanding. `hw/femu/scripts/aer-probe.c` checks the temperature path from inside the guest:

```
gcc -O2 -o aer-probe femu-scripts/aer-probe.c   # inside the guest
sudo ./aer-probe /dev/nvme0
```

B.4 End of life

With `blk_pe_limit` and `eol_read_only` (on by default), the media goes read-only when a worn-out block has no replacement. These fields then show it:

Where	Field	Value
SMART / Health (02h)	Critical Warning bit 3 (All Media Read-Only)	1
SMART / Health (02h)	Available Spare	0

Where	Field	Value
SMART / Health (02h)	Endurance Group Critical Warning Summary (byte 6)	bits 0, 2 and 3 of the endurance group's warning; needs a `femu-subsys`
Endurance Group Information (09h)	Critical Warning bit 3 (Endurance Group Read-Only)	1
Identify Namespace, CNS 00h byte 99 and CNS 08h byte 8	NSATTR bit 0 (Currently Write Protected)	1
Asynchronous event	SMART, information 00h (reliability), log page 02h	once, if bit 3 of Async Event Configuration is set
Persistent Event log (0Dh)	hardware error event, code 06h (critical warning)	once, with the warning byte when the media went read-only

Writes fail with Attempted Write to Read Only Range (status 182h, DNR). The same holds for Write Zeroes, Write Uncorrectable, Copy and a deallocating Dataset Management. The NVM Identify Controller (CNS 06h, CSI 00h) reports WZSL in byte 1, equal to MDTS, and VER 1.1 in bytes 23:20. ONCS bit 3 is clear, so WZSL is a hard limit.

B.5 Thermal model

With `thermal_tau_ms` set, the composite temperature in the SMART log follows the NAND work instead of staying at `temperature`. Every constant comes from the user; FEMU has no built-in figures.

```
power (mW)  = idle_mw + NAND energy since the last step / the step
              (plane reads, programs, erases x energy_*_nj; bbssd, CSD, KV)
target (K)  = temperature + power x thermal_r / 1000     (thermal_r: mK per mW)
every thermal_step_ms of virtual time:
  T = target + (T - target) x exp(-step / thermal_tau_ms)
  SMART temperature = T rounded to a Kelvin
  T reaches the over threshold, or falls to the under threshold:
    SMART critical warning bit 1, Persistent Event log entry,
    and one SMART temperature event if the host enabled it
```

- The clock is QEMU's virtual clock, so a paused VM does not cool. NAND work that FEMU does while the VM is paused counts in the first step after it.
- The model does not throttle I/O, and the SMART time-over-threshold fields stay 0.
- One event is raised each time the condition starts while the host has temperature events enabled, including when it enables them during an excursion. A SMART read without RAE discards queued SMART events, so an excursion that ends and starts again before the host reads the log is not reported twice.
- Refused with namespace management, shared namespaces, `cxl_ssd` and `-icount`. `thermal_tau_ms` needs `thermal_r`; `thermal_step_ms` is 1 to 60000.

B.6 Persistent Event log retention

The Persistent Event log (0Dh) lives in memory and is lost when QEMU exits. Set `pel_file=/absolute/path/events.pel` on a `femu` device to keep it across QEMU runs. A missing file starts an empty history. A corrupt or incompatible file refuses device creation and is left unchanged.

The file holds the encoded events, the log generation and the power cycle count. Each realize adds one

power cycle, a power-on event and a SMART snapshot. The PEL header's power cycle count (PWRCC), the Controller Power Cycle and SMART Power Cycles all use the retained count. Reporting contexts, the other SMART counters, namespace data and power-on hours are not retained.

The file is written on the main loop after events and generation changes, and again at controller reset, device removal and normal process exit. Each write goes to a temporary file in the same directory, which is synced, renamed over the old file, and followed by a sync of the directory. At normal exit, event collection closes under the log mutex before the last write: events added before that point are saved, and later ones, including completions of outstanding I/O, are dropped. The exit path does not wait for the pollers, whose MMIO DMA may need the main thread's lock. Killing QEMU abruptly can lose changes still queued for the main loop. A write error is printed to stderr, and the next event or lifecycle save tries again.

Use one file per device and one QEMU writing it. The file is not a way to share a log between devices or to migrate one.

C Scripts and tools

Every script and tool shipped under `hw/femu/scripts/` and `hw/femu/tools/`: what it does, its arguments and the environment variables it reads. The top-level `femu-scripts` link points to `hw/femu/scripts/`, so from `build-femu/` you can also run them as `../femu-scripts/NAME`.

Scripts marked **legacy** do not start FEMU. They start QEMU's stock `nvme` device or another disk on the old `x86_64-softmmu/` binary path, set thread affinity by hand, or prepare a specific lab machine. They are kept for reference. Do not use them.

C.1 Build and setup

Script	Run from	What it does
`pkgdep.sh`	anywhere, as root	Installs the build dependencies with `apt-get` on Debian and Ubuntu. Exits with `pkgdep: unsupported system type` elsewhere. CI does not run it.
`femu-compile.sh`	`build-femu/`	Runs `make clean`, then `../configure --enable-kvm --target-list=x86_64-softmmu --enable-slirp --disable-libnfs --disable-libiscsi --disable-curl`, then `make` with one job per CPU. `--enable-csd-ubpf` adds uBPF CSD programs, `--enable-csd-ubpf=PATH` uses the uBPF tree at `PATH`. Any other argument is an error. See build.md (§2.2).
`femu-copy-scripts.sh`	`build-femu/`	Copies `pkgdep.sh`, `femu-compile.sh`, `make-guest-image.sh`, `run-guest-ssh.sh`, the `run-blackbox.sh`, `run-blackbox-fdp.sh`, `run-whitebox.sh`, `run-nossd.sh`, `run-zns.sh`, `run-csd.sh` and `run-kvssd.sh` launchers, `pin.sh` and `ftk/` into the current directory, overwriting earlier copies. It does not copy `run-cxlssd.sh`, `ssd-config.sh`, `configs/` or the guest tools; run those from `../femu-scripts/`.

C.2 Guest image and access

Script	What it does	Arguments and environment
`make-guest-image.sh`	Builds an Ubuntu 24.04 guest image from the official cloud image. It checks the download's SHA256 sum, and its signature when `gpgv` and the Ubuntu keyring are installed, with user `femu`, an SSH key, a serial console, `nvme-cli` and `fio`. Needs no root, but needs access to `/dev/kvm` unless you pass `--no-kvm`. See guest-image.md (§2.3).	`-o DIR` output directory, `-n FILE` image name (default `u20s.qcow2`, the name the launchers expect), `-s SIZE` (default `32G`), `-k FILE` public key, `-p PW` console password, `--cxl` adds `ndctl`, `daxctl` and `cxl-cli`, `--packages LIST` extra packages, `--qemu PATH`, `--qemu-img PATH`, `--no-kvm`, `--timeout SEC` (default 1800), `-f` replace an existing image. Reads `IMGDIR` (default output directory `$HOME/images`), `QEMU` and `QEMU_IMG`.
`run-guest-ssh.sh`	Logs in to a running guest built by `make-guest-image.sh`, or runs one command in it: `./run-guest-ssh.sh sudo nvme list`.	`IMGDIR` (default `$HOME/images`), `SSH_KEY` (default `$IMGDIR/femu-guest-key`), `SSH_PORT` (default 8080), `GUEST_USER` (default `femu`).

C.3 Launchers

Run each NVMe launcher from `build-femu/` after `femu-copy-scripts.sh`. They start `./qemu-system-x86_64` with `sudo`, KVM, `-cpu host`, `-nographic`, `-name NAME,debug-threads=on` (so the host shows FEMU's thread

names), a virtio-scsi boot disk and user networking that forwards host port SSH_PORT to the guest's port 22. run-cxlssd.sh is different: it is not copied, so run it as ../femu-scripts/run-cxlssd.sh, and it adds no disk, no network and no sudo (see below). All except run-cxlssd.sh read:

Variable	Default	Meaning
IMGDIR	$HOME/images	Directory of the guest image
OSIMGF	$IMGDIR/u20s.qcow2	Guest image file; the script exits with status 1 if it does not exist
SSH_PORT	8080	Host port forwarded to the guest's SSH port; use a different one for each VM you run at the same time

The device geometry, size and timing are shell variables at the top of each script. Edit them there; they are not read from the environment.

Launcher	Device	Guest	Output files
run-blackbox.sh	BlackBox SSD, 12 GiB namespace on 16 GiB of NAND (BlackBox (§4.4))	4 vCPUs, 4 GiB	log, qmp-sock
run-blackbox-fdp.sh	BlackBox with a femu-subsys that has FDP on, 4 handles, 1 reclaim group (FDP (§4.5))	4 vCPUs, 4 GiB	/tmp/femu-fdp.log, qmp-sock
run-nossd.sh	NoSSD, 4 GiB (NoSSD (§4.2))	4 vCPUs, 4 GiB	qmp-sock
run-zns.sh	ZNS, 4 GiB, QLC timing, 16 zones of 256 MiB (ZNS (§4.7))	4 vCPUs, 4 GiB	log, qmp-sock
run-whitebox.sh	Open-Channel 2.0 (OCVER=2 in the script; 1 selects 1.2), 4 GiB (OCSSD (§4.9))	4 vCPUs, 4 GiB	qmp-sock
run-csd.sh	Computational storage, 4 GiB, 4 compute units (CSD (§4.13)). It does not set csd_program_dir, so only the built-in program type loads	4 vCPUs, 4 GiB	log, qmp-sock
run-kvssd.sh	Key-value SSD, 4 GiB of value space on the BlackBox geometry (KV (§4.11)); the guest needs Linux 6.0 or newer	4 vCPUs, 4 GiB	log, qmp-sock
run-cxlssd.sh	One femu-cxl-ssd below a CXL host bridge (CXL SSD (§4.18))	4 vCPUs, 4 GiB, no disk and no network unless you add them	cxlssd-stats.log, cxlssd-io-N.log and cxlssd-spt.log in LOG_DIR when the guest asks for them through lsa-control

run-blackbox.sh also passes FEMU_EXP_LOG, FEMU_SECRET and FEMU_DUMP_LPN through sudo to QEMU (environment variables (§A.1.5)). The other launchers pass no variables to QEMU.

qmp-sock is created by root in the current directory; log is written by tee as you. Two launchers started from the same directory share them, so start a second VM from another directory.

run-cxlssd.sh runs QEMU without sudo and adds its own arguments after the ones it builds, so you append a boot disk, a network and a QMP socket on its command line. Its settings are environment variables:

Variable	Default	Sets
QEMU	./qemu-system-x86_64	the QEMU binary
CXL_SIZE	256M	media size, a number followed by M or G
CACHE_PAGES	size in MiB / 20 x 256	cache-pages

Variable	Default	Sets
CACHE_WAYS	1	cache-ways; full means CACHE_PAGES
BLOCKS_PER_PLANE	867 for 48G, 1732 for 96G, else 0	blocks-per-plane
CACHE_POLICY	fifo	cache-policy
DER	off	der
CYLON_KERNEL_ACK	off	cylon-kernel-ack
PREFETCH_DEGREE, PREFETCH_STRIDE	0, 1	prefetch-degree, prefetch-stride
CHANNELS, LUNS_PER_CHANNEL, PAGES_PER_BLOCK	8, 8, 256	NAND geometry
READ_NS, PROGRAM_NS, ERASE_NS, CHANNEL_NS	40000, 200000, 2000000, 0	NAND timing
GC_THRESHOLD, GC_THRESHOLD_HIGH	75, 95	GC thresholds
FTL	on	ftl
LSA_CONTROL	on	lsa-control; see security (§5.13.1)
CYLON_FIRST_TOUCH_PROGRAM, CYLON_FREE_WRITEBACK	off	the matching properties
LOG_DIR, LOG_LIMIT	., 64M	log-dir, log-limit
TRACEFS_DIR	unset	tracefs-dir, only when set
CXL_BACKEND	memory-backend-ram	memory backend type and options
ACCEL, CPU, CPUS, RAM	kvm, host, 4, 4G	accelerator, CPU model, vCPUs, guest RAM
DRY_RUN	0	1 prints the command instead of running it

These defaults follow Cylon's launch script and differ from the device's own defaults: one cache way instead of 16, a cache of size / 20 (3072 pages for 256 MiB) instead of 1024 pages, 8x8 channels and LUNs instead of 4x4, fixed blocks-per-plane for the 48G and 96G sizes (the least the device accepts; Cylon's sizes have no spare blocks), and lsa-control on instead of off.

C.4 Configuration files

Script	What it does
ssd-config.sh CONFIG [--device-only | --check]	Expands an INI-style file into -device femu,... arguments (and a -device femu-subsys,... for a [subsys] section). Keys are device property names; mode = bbssd and the like stand for femu_mode. With --device-only it omits the -device words; with --check it only validates. It checks keys against -device femu,help of the binary in FEMU_BIN, or of build-femu/, build/ or build-official/ under the source tree.
ssd-config-test.sh [QEMU]	Expands every file in configs/, starts QEMU with each and requires the device to come up, and checks that the parser rejects bad input. The binary is the argument, FEMU_BIN, or the first one found as above. CI runs it.

The files in configs/:

File	Device
bbssd.conf	BlackBox, 4 GiB, 8 channels of 8 LUNs
bbssd-overprovisioned.conf	BlackBox sized with op_pcent=10

File	Device
csd.conf	Computational storage, the device of run-csd.sh without its no-effect scheduler properties
fdp.conf	BlackBox with FDP on a [subsys] section
heterogeneous.conf	One controller with a BlackBox, a ZNS and a NoSSD namespace
kvssd.conf	Key-value SSD, the device of run-kvssd.sh
nossd.conf	NoSSD with link and firmware time from the NVMeCHA controller (NoSSD (§4.2.3))
ocssd.conf	Open-Channel 2.0, the device of run-whitebox.sh; lver = 1 selects 1.2
qlc.conf	BlackBox with QLC per-page-type timing
write-buffer.conf	BlackBox with a 2048-page write buffer, vwc=1 and Write Zeroes
zns.conf	ZNS, 16 zones of 256 MiB, at most 16 active and 8 open

Run it from build-femu/ with FEMU_BIN set. Through the ../femu-scripts link the script cannot find the binary on its own, and then it skips the key check:

```
FEMU_BIN=./qemu-system-x86_64 ../femu-scripts/ssd-config.sh ../femu-scripts/configs/zns.conf
```

The launchers take no arguments. To boot a config, copy a launcher and put the output in place of its -device femu option, or call QEMU yourself:

```
QEMU_ARGS=$(FEMU_BIN=./qemu-system-x86_64 ../femu-scripts/ssd-config.sh my-ssd.conf)
./qemu-system-x86_64 -enable-kvm -cpu host -smp 4 -m 4G $QEMU_ARGS ...
```

The file format: keys are femu device properties and mean what -device femu,help says. # and ; start comments, section headers are labels for the reader (except [subsys]), and a key with an empty value is ignored. List values such as namespace_modes = bbssd,znssd,nossd get their commas doubled for QEMU automatically.

```
[device]
mode        = bbssd        # friendly name for femu_mode
devsz_mb    = 4096

[geometry]
secs_per_pg = 8            # 4 KiB pages
luns_per_ch = 8
nchs        = 8

[timing]
pg_rd_lat   = 40000        # ns
pg_wr_lat   = 200000
```

A misspelled key stops the expansion when the binary is found:

```
ssd-config: unknown property 'gc_polcy' -- not one FEMU accepts
ssd-config: config rejected; see the warnings above
```

C.5 Guest-side test tools

Copy these into the guest and run them there. The C programs build with gcc -O2 -o NAME NAME.c and need no headers beyond libc.

Tool	Arguments	What it checks
femu-test.sh	--yes [DEVICE], default /dev/nvme0n1	Data integrity, counters, deallocate, and zone or key-value commands, chosen by what the namespace reports. **It overwrites the whole namespace**, so --yes is required, and it refuses a mounted device. See testing (§8.2.6).
kv-probe.c	[CONTROLLER], default /dev/nvme0	A key-value Store, Exist, Retrieve, Delete and a Retrieve of the deleted key (KV (§4.11))
kv-bench.c	[OPTIONS] NODE, the generic node such as /dev/ng0n1; build with gcc -O2 -pthread	A key-value workload of Store, Retrieve, Delete, Exist and List with the kvbench knobs (--keys, --key-len, --values, --ratio, --mix, --threads). It checks each answer against a model of the keys and prints the rate and latency of each command. --control legacy-opcodes or corrupt-model makes it fail on purpose. Needs an empty namespace and deletes its keys at the end. Exits 1 on an error or a mismatch. See KV benchmark (§8.2.6).
aer-probe.c	[CONTROLLER], default /dev/nvme0	That crossing the temperature threshold completes an Asynchronous Event Request, and that the event is re-armed after the log is read
zone-aen-probe.c	[CONTROLLER] [NAMESPACE], defaults /dev/nvme0 /dev/nvme0n1	That a zone taken read only raises the Zone Descriptor Changed notice; needs a ZNS device with err_write_fail_ppm set (ZNS (§4.7))
fdp-test-nvme-admin.sh	none; uses /dev/nvme0, /dev/nvme0n1 and /dev/ng0n1	The FDP admin commands of nvme-cli against the configuration run-blackbox-fdp.sh creates (4 handles, 1 reclaim group); written for a Linux 6.12 guest
blktests-guest.sh	GROUP [DEVICE], default device /dev/nvme0n1; reads BLKTESTS_REPO, BLKTESTS_REF and WORKDIR (parent of a new scratch directory, default /tmp)	Clones, builds and runs one blktests group against the namespace, and prints the counts of passed, failed and not run tests. Exits 1 when a test that ran on the namespace failed or the run stopped early, and 2 when the setup failed. Needs network access and writes to the namespace. See conformance suites (§8.2.6).
nvme-cli-e2e-guest.sh	[CONTROLLER [NAMESPACE]], defaults /dev/nvme0 /dev/nvme0n1; reads NVMECLI_REPO, NVMECLI_REF (default v3.1), E2E_TIMEOUT and WORKDIR (parent of a new scratch directory, default /tmp)	Builds nvme-cli and runs its end-to-end suite. Exits 1 when a test failed or the suite ended abnormally, and 2 when the setup failed. A failure of test_get_lba_status counts as a known nvme-cli defect only when the same command passes with the numeric namespace ID. Needs network access, writes to the namespace and can format it.
zenfs-crash-guest.sh	[NAMESPACE], default the first host-managed zoned namespace; reads ROCKSDB_REPO, ROCKSDB_REF (default v9.10.0), ZENFS_REPO, ZENFS_REF (default 919c2eb), ZENFS_PREFIX, ZENFS_JOBS, ZENFS_ROUNDS, ZENFS_INTERVAL, ZENFS_THREADS, ZENFS_KEYS, ZENFS_CONTROL and WORKDIR	Builds RocksDB with ZenFS and runs the RocksDB blackbox crash test on the namespace: rounds of db_stress killed with SIGKILL, then a final check of every key. Exits 1 when a round or the final check failed, and 2 when the setup failed. ZENFS_CONTROL=reset-zones makes it fail on purpose. Needs network access and formats the namespace with ZenFS. See ZenFS crash test (§8.2.6).
kv-bench-guest.sh	[NODE], default the one /dev/ngXnY without a block device; reads KVBENCH_KEYS, KVBENCH_OPS, KVBENCH_THREADS, KVBENCH_SEED, KVBENCH_CONTROL and WORKDIR	Builds kv-probe.c and kv-bench.c from its own directory, runs kv-probe, then a small-value and a large-value kv-bench run. Exits 1 when a check failed and 2 when the setup failed. See KV benchmark (§8.2.6).

Run this one on the host:

Script	What it does	Arguments and environment
guest-conformance.sh	Boots a guest for each suite with a fresh overlay of the guest image, runs the suite's guest script in it over SSH, stops the guest and prints RESULT SUITE PASS, FAIL or SETUP-ERROR. An SMBIOS string checks that the SSH port reaches this guest. It refuses an overlay or PID file that an earlier run left in the result directory. CI does not run it. See conformance suites (§8.2.6).	Suites blktests-block, blktests-zbd, nvme-cli, zenfs-crash, kv-bench or all. Options --qemu PATH (default ./qemu-system-x86_64), --image FILE, --key FILE, --port N (default 8090), --outdir DIR, --timeout SEC (default 7200 for zenfs-crash, else 3600), --boot-timeout SEC (default 300), --device PROPS (the FEMU properties for a control run), --smp N and --mem SIZE (default 16 and 16G for zenfs-crash, else 4 and 4G), --sudo, --keep. Reads QEMU, IMGDIR, OSIMGF, SSH_KEY and SSH_PORT, and passes the BLKTESTS_*, NVMECLI_*, ROCKSDB_*, ZENFS_* and KVBENCH_* variables that --help lists to the guest. Exits 0 when every suite passed, 1 when a suite failed, and 2 when a suite could not run and no suite failed.
csd-pause-check/run.sh	Boots a small KVM guest from a kernel and a busybox initramfs twice, with csd_pause off and on. In the guest, csdlat.c times Execute of a program that sleeps on the host, and clockcheck.c counts clock steps back on each vCPU, sleeps that end early, and the gap between the TSC and the kvmclock. Prints both results; it checks nothing itself. See CSD (§4.13.4).	QEMU KERNEL MODDIR OUTDIR [VCPUS]: a FEMU binary, a guest bzImage, its /lib/modules/<version> for nvme.ko or none, an output directory, and the vCPU count (default 4). Needs gcc with static glibc, a static busybox, cpio and gzip.

C.6 Host monitoring tools

Run these on the host, against a running or a dry-run QEMU. They need Python 3 and no other package.

Script	What it does	Arguments
femu-config-check.py	Starts QEMU under qtest with the given options and no guest, then reports either FEMU's refusal or the configuration it builds: capacity, geometry, over-provisioning, line or reclaim unit size, zones, mode and NAND timing per namespace, and QEMU's warnings. See check a configuration (§7.2.5).	[--json] [--qemu BINARY] [--timeout SEC] [--] QEMU-OPTIONS, or the value of one -device option; --self-test checks the tool. Exits 0 accepted, 1 refused, 2 when the check failed. It loads femu-stat.py from its own directory.
femu-stat.py	Shows the FTL counters of each femu device of a running VM, like top: pages, WAF since start and over the interval, rates, line, reclaim unit and zone counts, and I/O stage times. It polls QMP query-femu and query-femu-stages and never resets the counts. See live counters (§6.1.4).	SOCKET (a Unix socket path or HOST:PORT), --interval SEC, --count N, --once, --json, --ns LIST, --path QOMPATH; --self-test --qemu BINARY checks the tool under qtest. Exits 2 on a connection error.

C.7 Host tuning helpers

Script	What it does
ftk/qmp-vcpu-pin -s SOCKET CPU...	Pins each vCPU thread to a host CPU with taskset, using QMP query-cpus-fast on SOCKET; vCPU i goes to the i-th CPU in the list, wrapping around. It imports ftk/qmp.py. Run it with sudo when QEMU runs as root. A Unix socket path longer than about 107 bytes fails with AF_UNIX path too long.
pin.sh [FIRST_CPU]	Pins each vCPU thread, then each femu-poller, FEMU-FTL-Thread, femu-cxl-ftl and femu-cxl-cca, to its own host CPU, starting at FIRST_CPU (default 0), and moves the other QEMU threads, femu-csd-cu included, to the CPUs after those. It finds the threads by name, so QEMU must run with -name NAME,debug-threads=on (the launchers pass it), and it finds QEMU with pgrep -x qemu-system-x86; set QEMU_PID when several run. Run it after the guest has booted: the pollers start when the guest enables the controller. It stops if the host has too few CPUs. See performance tuning (§5.12.3).
set_cpu_perf_mode.sh	Sets every CPU's cpufreq scaling policy to performance through sysfs. Run it as root.

C.8 Documentation tooling

These run in CI. docs-maintenance.md (§8.3) explains each.

Script	Arguments
gen-property-docs.py	--qemu BINARY regenerates reference/properties.md and runtime-properties.md from the binary; --check compares instead of writing (--qemu is still required)
gen-mode-table.py	rewrites the mode tables from docs/modes.py; --check compares and checks modes.py against the code
check-doc-links.py	[--root DIR] [PATH...]; checks every relative link and anchor
check-changelog-hashes.py	[--head COMMIT] [FILE]; checks that every commit the CHANGELOG names is in the history of --head (needs a full clone)
check-c-style.py	[DIR] (default hw/femu) or --self-test; finds C99 comments and trailing whitespace in .c and .h files, and fails on a tree with no source file
check-property-tests.py	no argument or --self-test; checks that a file under tests/ or scripts/ names each property in reference/properties.md, or that tests/untested-properties.list lists it with a reason
check-unit-tests.py	no argument, --meson-names, --listed FILE or --self-test; checks that each tests/unit/test-*.c is in tests/meson.build or tests/Makefile, and that the femu-unit suite lists every meson unit test
check-doc-examples.py	--lint, --list, --self-test, --qemu BINARY, --qos-test BINARY, --only NAME, --timeout SEC, [PATH...]; checks every code block's tag and runs the tagged examples

C.9 CXL caching API tools: hw/femu/tools/cca/

Guest code for femu-cxl-ssd,cca=on. make -C hw/femu/tools/cca in the guest builds libcca.a, the ccactl command and the cca-test self-test. run-guest-tests.sh [-d MEMDEV] [-x DAX] [-o LOG] [CASE...] builds them and runs the self-test as root, logging to cca-guest-YYYYMMDD-HHMMSS.log by default. See the caching API guide (§4.20) and the tool README[138].

[138] https://github.com/MoatLab/FEMU/blob/master/hw/femu/tools/cca/README.md

C.10 Legacy scripts

None of these start FEMU. Several hard-code paths on one lab machine (`/home/huaicheng/images`), the old `x86_64-softmmu/qemu-system-x86_64` binary, or a `u14s.qcow2` image.

Script	What it was for
`femu-run.sh`, `m.sh`, `s1.sh`, `s2.sh`	Start a VM with QEMU's stock `-device nvme`
`f.sh`, `dp.sh`, `dp-run.sh`, `ide-run.sh`, `virtio-run.sh`, `null-run.sh`	Start a VM with a raw, tmpfs-backed, IDE, virtio-blk or null data disk instead of FEMU, for comparison runs
`gdb-run.sh`	Start the stock `nvme` device under gdb; see debugging (§7.2.2) for a version that runs FEMU
`valgrind-run.sh`	Start the stock `nvme` device under valgrind, with properties that no longer exist
`aff.sh PID`	Pin 20 consecutive thread IDs starting at `PID` to CPUs 5 to 24
`getaff.sh`	Print the CPU affinity of every thread of every `qemu` process
`pre.sh`	Turn off address space randomization, format `/dev/nvme0n1` with ext4 and mount it for one lab user. **Destroys data**
`pre-all.sh`	Run `pre.sh` and network setup over SSH on three named lab hosts
`tuning.sh`	Stop a list of services on one lab machine

One more NoSSD launcher, and a benchmark harness in a subdirectory of `hw/femu/scripts/` with its own README, are not covered here.

C.11 Related pages

- Build (§2.2)
- Guest image (§2.3)
- Performance tuning (§5.12)
- Testing (§8.2)

D Changelog

User-visible changes to FEMU. Each entry names the commits it comes from; `git show <hash>` has the details.

D.1 Unreleased (since femu-v9.0.1)

655 commits after the `femu-v9.0.1` tag (2024-06-20), dated 2024-08-07 to 2026-10-01, plus the documentation and script fixes that close this list. Tests, CI-only changes, internal refactoring and wording fixes are not listed one by one.

D.1.1 QEMU base

- FEMU now builds on QEMU 10.1.0 instead of 9.0.1, with device registration moved to the current QEMU APIs (eafddfdd0).
- Two QEMU TCG fixes are carried from upstream QEMU. Without them a vCPU could resolve a stale memory section after a device was unplugged and stop QEMU with an assertion (ef516ed58, c66409e3b).
- Two QEMU CXL fixes are carried: the background command timer is freed with its CCI, and a CXL Type 3 device releases its mapped backends when realize fails (43040a04c, 6153becf8).
- QEMU's network block drivers are left out of the FEMU build, which fixes compilation against libnfs 6 (81c4385fc).

D.1.2 New devices, modes and features

CXL SSD (new device)

- New `femu-cxl-ssd` device: a CXL memory device whose accesses go through a DRAM buffer cache and FEMU's FTL and NAND timing (bb9aea670, a8627fcf9, 7777c0d53, 6ffe34f17).
- The buffer cache gained runtime controls, scalable associativity, per-page pin and remove, and NAND geometry properties for larger capacities (f5ba3ff12, 38b314181, 76f180b9d).
- Direct mapping modes `der=memslot` and `der=cylon` map hot pages straight into the guest, with configurable direct mapping ratios and LSA mailbox commands to control them (80803505b, b3e042a84, 1f26fb2df, 8266a291e, a1e550885).
- A CXL caching API is served on BAR5, with a guest library, the `ccactl` tool and a guest self-test (924714c18, 1235e87d2, ec3af4ba4, 0a357d11e, defd999b8).
- Cache misses to different pages wait for the media together, by default only while a direct mode is active, and long media waits sleep instead of spinning (b72d14dbe, 64d424dd2, 133c553ca, 145fcfe4b).
- A bbssd NVMe controller can share a `femu-cxl-ssd` medium through `cxl_ssd=<id>`, with writes, deallocates and flips kept consistent between the two front ends (146a240dc, d73a5b780, a7a3c5678, 353bbe107, 287e5c885, 6783d50af).
- A runtime `fast-load` switch on `femu-cxl-ssd` lets warmup and data loading skip the wait for modelled media time while the FTL, cache and counters still run. Switching it off leaves the queued NAND work on the timelines and reports it in `fast-load-drain-ns`; a harness polls the new `nand-idle-ns` until it reads 0 before it measures.
- With `der=cylon` on a host kernel that has `KVM_CAP_CYLON_FAULT_EXIT`, an instruction KVM cannot decode (VEX, EVEX, most SSE with a memory operand) on an unmapped page no longer hangs its vCPU: FEMU fills and maps the page and the guest runs the instruction natively, or stops the

VM with a report when the page cannot be mapped or the instruction makes no progress (100,000 consecutive refills at one RIP by default; the new property `cylon-fault-stop` sets the bound, and 0 only warns, because the bound counts exits, not instructions, and a healthy loop over more pages than the cache holds can reach it). Instruction fetches from an unmapped page (for example a shared library cached on the CXL node) are mapped, not emulated, so code such as `endbr64` runs natively. New property `cylon-emul-exit` and counters `der-emul-exit`, `der-emul-fills`, `der-emul-fetch-fills` and `der-emul-failures`.

- With version 2 of `KVM_CAP_CYLON_FAULT_EXIT`, KVM does not emulate accesses to cold `der=cylon` pages: it exits with the exact access type and FEMU maps the page, so every instruction runs natively. New property `cylon-never-emulate` defaults to `auto`, which turns version 2 on when the host kernel reports it and otherwise warns once and uses version 1; `on` warns when the kernel lacks version 2, and `off` keeps version 1. Pages FEMU cannot map go back to the emulator. The pages kept in the cache for an instruction bind other vCPUs' fills for only 1 ms, so a vCPU that stops faulting cannot close a cache set to the others. An instruction whose pages do not fit in their cache set is completed by the emulator, or mapped outside the cache when the emulator cannot run it, instead of refilling its pages for ever; new counters `der-fault-conflicts` and `der-fault-overflows`. New counters `der-emul-v2`, `der-fault-reads`, `der-fault-writes`, `der-fault-fetches`, `der-fault-page-walks`, `der-fault-emulated` and `der-fault-unprotected`. `der=cylon` now refuses a machine without `smm=off`, which it always required.
- With version 2 on, an eviction that must flush the VM's TLBs also revokes the mappings of the next pages the cache evicts, so up to `cylon-revoke-batch` pages (default 64, at most 256) share the two flushes. On a d760 run with random reads, 64 gave 8 vCPUs 111.7k operations per second against 104.2k at 32 (bf2197a37). Version 2 maps every filled page and the guest then uses it, so nearly every eviction needed the two flushes, and with several busy vCPUs they serialized all vCPUs. The pages revoked ahead stay cached; an access maps them again as a hit. New counters `der-revoke-flushes`, `der-revoked-ahead` and `der-ahead-remaps`.
- With version 2 on, FEMU serves the fault exits of cold `der=cylon` pages without the BQL, under a lock of its own, so misses of different vCPUs no longer queue for the BQL. A fill also waits out its media time and checks the frame it maps while that lock is released. On the R0 pilot with 8 vCPUs the exits waited about 90 us of a 136 us miss for the BQL. New counter `der-fault-bql` counts the exits that still needed the BQL (28f80da22). The lock becomes a mutex only when version 2 is on; until then its holders hold the BQL, so version 1 MMIO pays nothing for it.
- A `femu-cxl-ssd` with the FTL on no longer goes uncached when its NAND is full. Realize refuses NAND that leaves less than 7% of the media as spare beyond the forced collection reserve and two working lines. The error names the smallest `blocks-per-plane` that meets the rule, and automatic sizing meets it. A write at the forced threshold now waits until the collection erases end on every LUN, counted in the new `gc-stalls` and `gc-stall-ns`; `gc-stall-max-ns` is the longest wait, and a wait over one second warns once per device. A program that still finds no page loses only its timing and counts in `media-full`; it no longer stops an eviction or an insert. `run-cxlssd.sh` presets for 48G and 96G use the least NAND the rule accepts (867 and 1732 blocks per plane).
- A device that copies into `femu-cxl-ssd` memory from its own MMIO handler or bottom half, such as QEMU's NVMe controller when the guest page cache is on the CXL node, no longer loses doorbell writes. The copy used to release the BQL while that device's re-entrancy guard was engaged, so a vCPU's doorbell write was refused ("Blocked re-entrant IO on MemoryRegion: nvme") and the guest timed out. Such copies, and NIC receive copies, now complete without releasing the BQL and queue their media time instead, without waiting for a garbage collection in progress, and the FTL worker still lets waiting accesses in between them; new counters `dma-accesses`, `dma-media-ops` and `dma-media-time-ns`. Devices that run in an IOThread are not covered. `nand-idle-ns` stays above 0 while these operations are queued.

- The `femu-cxl-ssd` FTL worker wakes only the access whose request it finished, instead of every waiting access. On a `der=cylon` test with 8 and 16 vCPUs doing random reads that all miss a 256-page cache, throughput rose about 40% (65k to 91k and 70k to 98k accesses/s), because waiters no longer wake for other requests.
- `der=cylon` also accepts slot tables that the host kernel maps as refcounted pages (`mm` in `/proc/self/smaps`), not only as raw PFNs (`pf`). A Cylon kernel that maps them this way can free the tables of a deleted slot once FEMU unmaps them.

New modes and data placement

- Flexible Data Placement (FDP) on a new `femu-subsys` device, with reclaim unit handles, FDP log pages, I/O Management commands and several GC strategies (c966d341a, 6df00de01).
- FDP placement features can be read and enabled by a Linux host, and the host sees its active reclaim unit and remaining space (8f00c7117, 9a66fe52d).
- FDP accepts `trim_lat_ns` and `ecc_retention_sec`. A deallocate pays the same time per range as without FDP. The age of a reclaim unit counts from its retirement and starts again when collection frees it.
- FDP accepts `read_reclaim_limit` and `retention_limit_sec`. A read queues the reclaim unit of its line. The next write moves the valid pages of that unit to where collection would move them. The moves count in MBMW and in the C0h refresh counters.
- FDP accepts `gc_policy`. Each name picks the reclaim unit version of its policy: `cost-benefit` and `random` are `gc_strategy` 1 and 2, `fifo` takes the unit retired first, and `d-choice` the fewest valid pages of 4 random units. A name other than `greedy` needs `gc_strategy=0`.
- FDP accepts `mapping=dftl`. A placed write charges the translation page of each page it programs, as a write without FDP does.
- FDP accepts `buffer_size`. Each held page keeps the placement handle of its last write and goes to that handle's reclaim unit when it is written back. RUH Update, Flush and disabling the volatile write cache write the buffer back first, and fail with Capacity Exceeded if a page finds no room.
- FDP still refuses `hot_cold_sep` and the `hybrid` and `fast` mappings, and the message now says why: they write host data through pointers of their own, past the unit the placement handle names.
- New `namespace_phls` property: a placement handle list for each FDP namespace. Placement handle `i` selects entry `i`. A handle no list names is reported unused in log 21h. Unset, every namespace gets every handle, as before.
- FDP takes more than one namespace. The reclaim units of an endurance group are shared, so every bbssd namespace of the controller places into one FTL; NoSSD namespaces report their lists only. Namespaces that share a handle keep one format, and events from collection name the namespace of the data they moved.
- query-femu reports the placement handle list of each FDP namespace (`placement-handles`). It also reports the namespace whose FTL holds the units (`ftl-nsid`). `femu-stat.py` and `femu-config-check.py` show both, and the latter counts over-provisioning over the namespaces that share one FTL.
- FDP HBMW and the host bytes of each handle count the logical blocks the host wrote, as Base 2.3 Figure 293 defines HBMW. A 512-byte write on a 4 KiB page now adds 512, not 4096; MBMW still counts the programmed page.
- FDP takes more than one reclaim group (`fdp.nrg` up to 128). On bbssd and CSD a group is a set of dies: whole channels, or LUNs of one channel. A reclaim unit is one block of each plane of each die of its group, so RUNS is a line divided by the group count. Each handle has an active unit and a collection unit in every group. Collection, pads and refreshes stay in their group.
- With more than one reclaim group, a write without placement goes to the group with the most free

units. Each group must hold the whole namespace, since a host may place all of its data in one. A logical block must be whole NAND pages.
- Under FDP, a buffered write that finds the write buffer full of pages with no room programs its page at once when its own handle has room, instead of failing with Capacity Exceeded. RUH Update writes back only the held pages of the handles it names.
- query-femu lists the reclaim groups of an FDP namespace (`groups`) with their dies and free units; `femu-stat.py` and `femu-config-check.py` show them.
- Computational Storage Drive mode (`femu_mode=4`) with program load and execute commands, routed through the bbssd FTL (eb01bb73b, 50b65ca91).
- CSD `nr_cu` and `csf_runtime_scale` now take effect (202ab7afb).
- A CSD runtime shorter than the program's own run on the host cannot be reached, since the completion carries the result; QEMU now warns once when that happens (b8e8d6b45).
- Opt-in `csd_pause` stops the guest while a CSD program runs on the host. Under KVM the kvmclock and the TSC skip the stop. The controller times its I/O on a clock that stops too, so the guest sees the modelled run time. `csd_pause_max_ms` bounds a stop.
- Key-value SSD mode (`femu_mode=5`) with store, retrieve, list, delete and exist commands (e72465909).
- Every KV namespace has its own command set and key space, the KV command set is advertised to the host, and the KV Configuration feature is served (39664d242, e62db84f2, e4949509c).
- Opt-in KV workload-mix timing (`kv_timing=model`): Store and Retrieve take their time from the Samsung KVSSD emulator fit over the last 16 of them, or from coefficients in `kv_model_coeffs`. A device with no KV namespace refuses both.
- Opt-in LSM-tree key index for KV namespaces (`kv_index=lsm`), after PinK (ATC '20): a DRAM memtable, levels by `kv_lsm_size_ratio`, `kv_lsm_pinned_levels` held in DRAM. Lookups read a meta page per unpinned level, and merges load the NAND in the background. Value GC puts moved keys back in the memtable, and List returns keys in byte order. The default hash index is unchanged. LSM knobs without `kv_index=lsm`, and `kv_index=lsm` with no KV namespace, are refused.
- `query-femu` reports the LSM-tree key index of a KV namespace with `kv_index=lsm` in an `lsm` member: the memtable, each level, merges and meta page I/O. `femu-stat.py` and `femu-config-check.py` show it.
- `query-femu` reports KV value packing in a `pack` member: the open page, packed pages, pad bytes, GC moves of values and the host bytes. `femu-stat.py` prints it, and `femu-config-check.py` shows whether packing is on.

Namespaces

- Multiple namespaces per controller via `namespaces` and `namespace_sizes` (140b2891e, 2e25897fe).
- Namespaces on one controller can run different modes via `namespace_modes` (053fd8936).
- Opt-in Namespace Management and Attachment (`ns_mgmt=on`) for NoSSD and bbssd, with a bbssd namespace cap (`bbssd_ns_limit`) (b12bc8754, d9c9f2251, b48799800, e0fa92f19, 42369e0bd).
- Namespace attribute changes are reported through AER and log page 04h, and recorded in the Persistent Event log (3630d6fa0, cc9e271b4).
- Namespaces can be created with protection information (c213b120c).
- Shared namespace management across controllers of one `femu-subsys,ns_mgmt=on`, with the subsystem owning namespace storage (ca80e3dc5, 23844afaa, feebe635c, 76cc58f63, 97fc6ae0a, 349f12d40).

Metadata, protection information and Streams

- Separate LBA metadata (`meta`, `mc`) on bbssd and NoSSD namespaces (62d149dd5).
- Metadata interleaved with data (extended LBAs, `extended=1` or Format with MSET) (ba36f0f68).

- Opt-in protection information formats (`pi=on`), with generation and checking on Read, Write, Verify, Compare, Write Zeroes and Copy (bb2f3fb06, d1d8e1593, 617f09c10, 1854c1940, f6dc00f46, 98fcaedea, 8f7382c2a, 9661e5adb).
- Opt-in Streams directive (`streams`, `streams.max`), with stream writes placed on separate bbssd write frontiers (623637942, 8e7387268, cb459a65b, e3529d48f).

NVMe commands, features and log pages

- With `csd_slm_size`, CSD mode adds an NVMe Subsystem Local Memory 1.2 namespace. It takes Memory Read, Write, Fill and Copy. With `csd_cp=on`, CSD mode also adds an NVMe Computational Programs 1.2 namespace. It takes Load Program, Program Activation Management, Memory Range Set Management and Execute Program. Two device-defined programs (copy and a 64-bit FNV-1a hash) and the CEMU shared-library and eBPF programs run on it.
- The memory and compute namespaces of `csd_slm_size` and `csd_cp` work in a `femu-subsys`, so FDP and these namespaces can run together. The controller must be alone in its subsystem, because Reachability Reporting on one controller needs it on all of them.
- A Copy with descriptor format 4h moves memory namespace bytes to a block namespace. The CEMU commands stay on (guide (§4.22)).
- With those namespaces the controller reports Reachability Reporting: the Reachability Groups (1Ah) and Reachability Associations (1Bh) log pages, CRCAP and RGRPID (log pages (§B.2)).
- SGL data transfers (`sgl`), now honoured on every I/O command that carries data (1fa43530d, d24705851).
- DULBE and Write Zeroes with Deallocate report deallocated blocks (98247b96f).
- TRIM (Dataset Management deallocate) on bbssd (d749928e9).
- Verify command (e8c2c8933).
- Copy command, including copies between namespaces (descriptor format 2) (b16b3b3ae, 04b3da484).
- Device Self-test and its log (b3e5965d4).
- Sanitize block erase and the Sanitize Status log (4997dbdd9).
- Telemetry Host-Initiated and Controller-Initiated logs (d859ad4d6).
- Get LBA Status and the LBA Status Information log (bfc02d6a3).
- Timestamp feature (5323f363b).
- Persistent Event log, with format, sanitize, telemetry, error and Set Features events, optionally kept across runs in `pel_file` (87ea8154c, 93dddf1d2, 276c93e23, 3a368e984).
- Asynchronous Event Requests are accepted and events are reported (7d5d2f84f, b53ec7122).
- Supported Log Pages (log 00h) and the I/O Command Set Profile feature are answered (1b15f38df, 481eb6df2).
- SMART reports host data units and command counts, available spare (with an optional `nand_bad_blocks` model), percentage used (from the cell type or `pe_cycles_rated`) and media errors (f07d117f1, a20d9ac94, a8a28d451, df8a3492a, e25ddf7be).
- The Endurance Group log is filled in (074b63f4f).
- Media counters (write amplification, host and relocated pages, write buffer hits, read reclaims, retention refreshes) are reported in vendor log page C0h (e57448e41, 4cb1f9ba7, e9746f6bf, e7f182b87).
- Interrupt Coalescing (Feature 08h) now holds I/O completion interrupts until the threshold or the aggregation time is reached. Feature 09h Coalescing Disable, the admin queue and error completions are not held. The default settings leave it off.
- Opt-in I/O stage times (`io_stages`): the QMP command `query-femu-stages` returns histograms of the submit, FTL, hold, post-delay and total time of each I/O command. post-delay shows how far completions land after the model's time.
- A QMP command, `query-femu`, reports the geometry, write counters, line counts and per-line state of

bbssd namespaces to the host. The FTL thread copies the state between two requests (reference[139]).

- `query-femu` also reports zoned namespaces: the zone geometry, the zone counts per state and, with kind `zones`, the state, write pointer and capacity of each zone. The main loop copies them under the zone lock, so a zoned namespace needs no enabled controller. Each namespace entry now carries only the members of its mode.
- `query-femu` reports the NAND bus timing of each namespace in a `media` member. It gives the page transfer and the command, status and burst phases after their fallbacks. It also gives the multi-plane gaps, the ONFI mode and the ordering rules. CSD, KV and OCSSD namespaces now report this member only (reference[140]).
- `query-femu` also reports FDP namespaces: the reclaim group geometry, unit counts per state, and the host and media bytes of each reclaim unit handle. Kind `reclaim-units` lists the state, handle, valid pages and wear of each unit. The handle byte counters sum to the FDP Statistics log.
- Opt-in power loss model (`power_loss`) that drops data still held in the volatile write buffer and records the event in SMART and the event log (46602b0e7, 303c77aef, 7679c443d).

ZNS

- Write buffer and FTL for the ZNS SSD, and zone reset charged with erase time (0326ab489, b3272c013).
- Configurable zone resource limits (`zns_max_active`, `zns_max_open`) and zone descriptor extension size (6404dd233).
- Optional conventional zones (`zns_num_conv_zones`) (6e7395b55).
- Configurable zone width (`zns_chnls_per_zone`) (e04e4e901).
- A zone can span fewer LUNs (`zns_luns_per_zone`), and `zns_zone_order=lun` fills a zone LUN first instead of channel first. The defaults keep the full-width, channel-first zone.
- Zone Random Write Area support (`zns_zrwa_*`) (b18f56b2b, 9f51854d7).
- An injected write fault (`err_write_fail_ppm`) takes the zone read only and lists it in the Changed Zone List (ccdae5d3b).
- Changed Zone List log page (569608a41).
- Configurable reads across zone boundaries (`zns_cross_zone_read`) and Zone Append size limit (`zns_zasl_bs`) (78caee058, bca834133).
- ZNS takes NAND timing from the shared media layer, with configurable read, program and erase times and an opt-in channel bus (3d6056020, 34bbe45fa, 2a4ebc2cb).
- Configurable write cache count (`zns_num_wc`) (66f432117).
- Optional program/erase suspend for reads (`zns_pe_suspend`, `zns_tsusp_ns`) (9ec423985).
- Optional LUN contention (`zns_lun_contention`): the planes of one LUN run their operations one at a time, and a zone reset erases each LUN's planes in one multi-plane operation. Off by default, so the timing does not change.
- Optional zone reset by fill (`zns_reset_by_fill`): a zone reset erases only the blocks that hold programmed pages, so its time and the SMART wear follow the zone's fill. Off by default, so a reset of a non-empty zone still erases every block.
- Opt-in ZNS read timing by page type (`zns_page_types`) and ECC read time by block wear (`zns_ecc_step_ns`). The page types keep the mean read latency of the flash type.
- ZNS Reset Zone Recommended (`zns_rzr_sec`, `zns_rrl_sec`, off by default). A zone Full for `zns_rzr_sec` seconds of guest time gets RZR at the next check, and Identify reports RRL. With a nonzero RRL, the controller refreshes the zone itself at the first check once RZR has been set for RRL seconds. It clears RZR, and the zone ages again. RRL 0 means no refresh. Both changes enter the Changed Zone

[139] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md

[140] https://github.com/MoatLab/FEMU/blob/master/hw/femu/docs/reference/query-femu.md#media-timing

List; a host Reset clears RZR with no event.

BlackBox SSD (bbssd) and NAND media

- NAND operation timing moved into a shared media layer used by bbssd and ZNS (46cb582c9, 711d537bc).
- Pluggable GC victim selection (`gc_policy`: greedy, random, cost-benefit, fifo, d-choice) (7222d6744).
- Opt-in DRAM read cache (`read_cache_mb`, `cache_evict`) (3b21542de, fc5d3ded0).
- Pluggable L2P mapping (`mapping`: page, dftl, hybrid, fast) with a modelled mapping cache (`mapping_cache_mb`) (64a19a6f3, 46e4261f9, 0f9b6aeab).
- Write amplification accounting, and a `debug_ftl` switch that prints page-state violations found on the GC path (e57448e41).
- Optional cell-type NAND timing (`nand_cell_type`, `cell_pages`, `pgtype_lat`, ONFI phase times, `ecc_step_ns`) (e405dbcbc).
- Optional modelled TRIM time (`trim_lat_ns`) and explicit over-provisioning (`op_pcent`) (1027eb240, 6a4559ee4).
- DRAM write buffer (`buffer_size`, `buffer_thres_pcent`) with Flush, FUA and volatile write cache support (6a8daec7a, d4d03f127, 0ee6d211b).
- Hot/cold separation of overwritten pages (`hot_cold_sep`) (4ded2e928).
- Log page C0h counts NAND pages that host writes cover only in part, whose read-modify-write the model does not charge (c80a23e88).
- Log page C0h counts plane reads, programs and erases and, with `energy_read_nj`, `energy_prog_nj` and `energy_erase_nj`, their energy (1f5620a1e).
- Firmware time per opcode (`fw_read_ns`, `fw_write_ns`, `fw_other_ns`) and on several firmware cores (`fw_cores`) (8370f6f59).
- Log page C0h counts host writes that waited for forced collection or a full write buffer, and completions held for a full completion queue (5631b3c07).
- Opt-in paced collection (`gc_pace`): the victim line is copied a few pages per host page written instead of in one pass, so a write waits for about one copy instead of a whole line (ff5e4b5ce).
- Opt-in copyback for line and FDP collection (`gc_copyback`, off by default). A page moves on its die when the destination shares its channel, LUN and plane, so the move puts no data on the channel. Collection then takes victim pages in the order of the destination dies. A copyback skips ECC, so on a real SSD its bit errors are not corrected and can add up; FEMU models no bit errors, so only timing changes, and pages with extra ECC time and refresh moves stay off-chip for that reason. Log page C0h offset 248 counts the pages moved this way.
- Opt-in ONFI 6.0 bus timing (`onfi_mode` 0 to 26, with `onfi_mts`, `onfi_spare_bytes` and `onfi_tccs_ns`) for bbssd, CSD and KV. The timing mode sets the page transfer, and the ONFI command, status and burst times set the other bus phases. Status comes before a read's data out and after a program, and a page register holds its LUN to the end of the op. The array times stay with `pg_rd_lat`, `pg_wr_lat` and `blk_er_lat`.
- With the ONFI profile, an op with an older start time can use an idle bus gap before the latest start time on its channel. The channel keeps about 1024 past bus windows for this. An op older than those starts no earlier than the end of the newest dropped window.
- `onfi_param_page` loads an ONFI 6.0 parameter page image. It turns the ONFI profile on and gives the mode, the spare bytes and the address cycles. Realize refuses a page with a bad CRC or one that does not match the device geometry and features. `query-femu` reports the address cycles.
- `onfi_sca=1` selects the ONFI SCA protocol. Commands, addresses and status use a separate CA bus per channel, and the channel bus carries only the data bursts. A burst owns the channel bus from its

SCE packet to the hold after its SCT packet. `query-femu` reports `sca` and the burst times.

- `onfi_cache_read=1` reads the pages of one read command that land on one LUN as an ONFI read cache sequence. It sends 31h for the next page of a block, 00h-addr-31h for any other page, and 3Fh after the last. `trcbsy`, which had no effect, now sets the busy time of each copy. The array reads the next page while the bus moves the current one.
- With `onfi_cache_read`, a LUN group reads as plain reads when a read cache sequence would end later on an idle LUN. Mode 26 with 4 KiB pages reads 8 pages in 332328 ns, not 345912 ns. The choice uses only timing fields, so it is the same for every run.
- Opt-in program disturb terms for the ECC read time (`pd_edge_wls`, `pd_edge_tiers`, `pd_open_tiers`, `pd_open_age`) in bbssd, CSD, FDP and KV. Reads of bottom wordlines and of part-programmed blocks pay extra tiers, and open data can age from its first program. The tier counts are chosen values for sensitivity studies.
- Opt-in program disturb actions in bbssd, CSD and FDP: `pd_open_read_limit` rewrites a line or reclaim unit whose part-programmed block takes too many reads, and takes it out of its write frontier first, and `pd_open_close_sec` pads and closes one left open too long. Log page C0h counts both at offsets 256 to 279, and pad pages count in the write amplification. Under FDP the pad gives the handle a fresh unit and logs event 1h.
- Opt-in thermal model (`thermal_tau_ms`, `thermal_r`, `idle_mw`, `thermal_step_ms`): the SMART composite temperature follows the NAND energy and idle power through a first-order model, and crossing a threshold raises the SMART temperature warning and event (eda6b6ffe).
- Opt-in static wear levelling (`wl_spread`): the least worn full line moves into the most worn free line, without a write pointer of its own (2e1ef35ba).
- Read reclaim (`read_reclaim_limit`) and retention refresh (`retention_limit_sec`) (7f9b4f6af, 4e09c4797, e7f182b87).
- Data age feeds the ECC read model (`ecc_retention_sec`) (f9433e9ab, 13f2b85de).
- `age_scale` ages data faster than wall time for retention refresh and the ECC retention tier, so a retention study runs in minutes (9988153d3).
- More than one plane per LUN (`pls_per_lun`) in bbssd, FDP and KV, with a line's planes erased in one operation (0f554fb7d, 3699e980d, 9c8228d28, 7276af2d8).
- Channel bus phases (`cmd_addr_lat`, `pg_xfer_lat`, `status_lat`, `ch_xfer_lat`) are added to the timing when set (c274ba7d9).
- A read can suspend an in-flight program or erase (`pe_suspend`, `tsusp_ns`) (9ec423985, 6a5c498a8).
- Opt-in multi-plane program and read (`mp_program`, `mp_read`, with `pls_per_lun > 1`): host programs or reads of the same page on several planes of a LUN take one array time, plus `tplpbsy` or `tplrbsy` between planes. Placement and page counts do not change. The defaults leave the timing unchanged, FDP is not affected, and a negative busy time is refused.
- Opt-in `plane_first` makes bbssd and CSD write pointers fill the planes of a LUN before the next channel. With `mp_program`, pages in a row then form one multi-plane program.
- Optional debug logging to study whether deleted data remains on the device (18ba6557c, 45e61ae41).
- Opt-in block erase limits (`blk_pe_limit`, `blk_pe_spread`, `blk_pe_seed`): a block past its limit stays in service, is counted in log page C0h and sets SMART critical warning bit 2 (12f1c582c). The block's line leaves service while enough lines remain (af99ecb21). `spare_lines` holds lines back whose blocks replace worn-out ones, and SMART Available Spare follows what is left (ee6afccd4). Crossing into either warning raises a SMART asynchronous event when the host enabled it (c9b3d3ad8). SMART Percentage Used then counts against the blocks' limits, and `query-femu` reports retired and spare lines (b6af86af1).
- With `blk_pe_limit`, the new `eol_read_only` (on by default) keeps erased reserve lines for the writes in flight at the end of life. Each write takes credits for the NAND pages it may program before it changes

data, and waits in its queue while the reserve is full. `query-femu` reports the reserve lines and the credits.

- With `eol_read_only`, the media goes read-only when no block can replace a worn-out one, as on a real SSD. Writes, Write Zeroes, Write Uncorrectable, Copy and a deallocating Dataset Management fail with Attempted Write to Read Only Range (182h) and change nothing. Reads, Flush, Format NVM and Sanitize still run, and the writes in flight finish from the reserve. SMART shows critical warning bit 3 and Available Spare 0, the Endurance Group log bit 3, and Identify Namespace NSATTR bit 0. A SMART event with information 00h follows when the host enabled bit 3. Format NVM fails with Command Sequence Error while admitted writes wait for the FTL.
- FDP takes `blk_pe_limit` with `eol_read_only=off`. A worn-out block retires its reclaim unit while the unit's group keeps the units the namespaces and placement need. Otherwise the block stays in service with SMART critical warning bit 2. SMART and the Endurance Group Information log report the warnings, Available Spare (the worst group) and Percentage Used, and `query-femu` shows retired units. FDP still refuses `eol_read_only=on`, `spare_lines` and `fdp_trim_erase_all` with it.
- FDP also takes `eol_read_only` (on by default with `blk_pe_limit`). Each reclaim group holds erased units for the writes in flight, the media goes read-only when no unit can replace a worn-out one, and writes then fail with 182h. Collection, refresh and padding stop then, and RUH Update fails with Capacity Exceeded.
- Opt-in pSLC tier in front of MLC, TLC or QLC NAND. The properties `tier_mode`, `tier_static_mb`, `tier_full_policy`, `tier_pslc_read_ns`, `tier_pslc_prog_ns`, `tier_pslc_erase_ns` and `tier_pslc_pe_cycles` control it. Host writes go to a static pool of pSLC lines first. A fold moves the oldest full pool line to the dense lines. The tier is off by default, and with it off nothing changes. See tiered NAND (§4.25).
- `tier_mode=dynamic` lets the pSLC pool borrow free dense lines while the device has room, with `tier_max_mb` as a cap and `tier_shrink_pct` as the fill where the pool starts to shrink. A borrowed line goes back to the dense free list after its fold, and the dense lines always keep room for every valid page. See tiered NAND (§4.25.4).
- `tier_idle_ms` lets the pSLC tier fold pool lines in idle time, one bounded step at a time, so the next burst finds free pool lines. `tier_lazy_pct` folds only while the pool use is above a percent. Both are off by default. See tiered NAND (§4.25.5).

Timing and fault models

- Fault insertion (`err_read_unc_ppm`, `err_write_fail_ppm`), a host link model (`pcie_bandwidth_mbps`, `pcie_prop_delay_ns`) and a controller CPU model (`fw_cpu_ns`), now also applied to NoSSD (7bfac4c3a, 2cdd55c52).

Open-Channel SSD

- Opt-in OC 1.2 channel transfer timing (`oc12_channel_timing`) (d174f9551, 5cf3996c0).
- Opt-in OC 2.0 page type timing (`oc20_page_types`): reads and programs take the time of their page type, as on OC 1.2, instead of the lower page time.
- OC 2.0 gets media timings, chunk resets are charged as erases, an unwritten block reads as the predefined pattern, and a host can reset a chunk it has not filled (b652c3d87, 9d176f891, 00daad11c, 90ff859c1).

NoSSD

- Poller threads decoupled from queues (`poller_ratio`), and I/O counters kept per poller (0b192893b, 6b1592746).
- The backend memory can be bound to or interleaved across NUMA nodes with `FEMU_MBE_INTERLEAVE`

(3594d27c6).

- Under FDP a NoSSD namespace counts its writes: HBMW, MBMW and the RUAMW of its reclaim units, per handle and per endurance group. It logs event 03h for an invalid placement identifier. Write Zeroes with Deallocate and Write Uncorrectable count in HBMW only. A unit fills by the blocks written, and RUH Update logs event 00h only for a part-written unit. A logical block larger than `fdp.runs` is refused.
- ZNS Finish Zone Recommended and Zone Active Excursions (`zns_fzr_sec`, `zns_frl_sec`, off by default). A zone written but not finished gets FZR after `zns_fzr_sec` seconds of guest time. FRL seconds later the controller finishes it and sets ZFC. Both changes enter the Changed Zone List.

D.1.3 Configuration changes and new refusals

A command line that ran before may stop at realize, with a message naming the property. Each of these was accepted before and then ignored, or failed a check without saying so. Nothing here changes a configuration that was already being honoured. If one of these stops a run, remove the property: it was not doing anything.

Refused at realize (previously accepted and ignored)

Property	Why it is refused	Commits
any violated controller constraint	The check ran but returned silently, leaving QEMU up with no FEMU PCI device and no namespaces. The reason is now reported.	9716c0a9f
`mpsmax` below `mpsmin` or above 15	The test used to be inverted, so `mpsmax=1` was rejected and `mpsmin=1,mpsmax=0` accepted, advertising CAP.MPSMIN above CAP.MPSMAX.	9716c0a9f
`meta` with `dpc` or `dps`, with `nlbaf` above 8, or without a matching `mc`	Metadata is implemented for NoSSD and bbssd (separate buffer or interleaved with the data), but not together with the legacy protection settings; use `pi` for protection information. Before metadata was implemented, any non-zero `meta` was refused.	d589ce2b8, 62d149dd5, ba36f0f68, bb2f3fb06
`cell_pages` above 5	Indexed past the page-type multiplier table.	d589ce2b8
`nand_cell_type` with `pgs_per_blk` above 512	Read past the page-type latency tables.	d589ce2b8
`gc_strategy` outside {0,1,2,4}	Other values silently fell back to greedy or never collected at all.	d589ce2b8
`gc_policy` other than greedy with `femu_mode=5`	KV reclaims by taking the emptiest line off the shared victim queue; another policy reorders that queue (fifo by a close order KV never records), so KV took a line that was not the emptiest.	
`zns_flash_type` 0, 6 or above, or MLC/PLC without explicit latencies	0 gives a zero-length write cache and an endless flush loop; 6+ indexes past the timing tables; MLC and PLC have no built-in figures, so every NAND operation cost nothing.	b234d27c8
`femu_mode` above 5	No mode registers command handlers for it (6 was a SmartSSD placeholder), so the controller came up with none.	c3b8e88ef
`multipoller_enabled` other than 0 or 1	Values above 1 started several pollers that each swept every queue, so two pollers could run and complete the same command.	de5fcd972
`lver` other than 1 or 2 with `femu_mode=0`	No Open-Channel handlers were registered for it.	498afdd7a

Property	Why it is refused	Commits
`flash_type` outside 1 to 4 with `femu_mode=0`	OCSSD 2.0 indexed the SLC to PLC timing tables with it unchecked, so 6 or above read past them; 0 and 5 have no built-in figures. OCSSD 1.2 already refused them.	4891d870e
`zns_num_plane` above 8, `zns_num_ch` above 128, or a page count above 65536	Wrapped and aliased onto lower indices in the PPA.	b234d27c8
`zns_chnls_per_zone` that does not divide `zns_num_ch`	Was silently replaced by the full channel width.	b234d27c8
bbssd knobs under FDP: `buffer_size`, `hot_cold_sep`, `read_reclaim_limit`, `retention_limit_sec`, `ecc_retention_sec`, `trim_lat_ns`, non-default `mapping` or `gc_policy`	FDP keeps its own write and reclaim path; none of these reach it.	d589ce2b8

Other new refusals

- Invalid bbssd and zoned geometries, and a bbssd namespace with no room left for garbage collection; CSD and KV, and every mode a controller serves, now run the same checks (b4bbd93e0, 1ac931d86, ceb2b5bf7, 5ccbe3f24, bf8961b9b, 0b82aa58a).
- Invalid ZRWA configurations, page sizes ZNS cannot serve, and more write caches than zones (b89cde46d, 154775e62, fcfa0d581).
- Unknown `mapping` or `gc_policy` names, and a negative suspend overhead (29c397f38, 2ae58aa55).
- A CSD controller with more than one namespace (b425007c2).
- FDP with more than one reclaim group, and other placement configurations the device cannot serve (f08762450, 8ed17a495).
- ZNS and OCSSD namespaces in an FDP subsystem, and the memory and compute namespaces of CSD. An FDP endurance group holds NVM command set namespaces only (Base 2.3, 8.1.10). They were accepted, and placement had no effect.
- `fdp.nrg` above 128, and `fdp.nruh` above 16366, whose configuration descriptor would not fit the 16-bit size field of log 20h.
- An FDP namespace larger than what is left once each handle has a reclaim unit open, each Persistently Isolated handle one to collect into, and forced GC its free units. Such a namespace was accepted and then failed most random writes. A partly exposed last page now counts as a whole one in this check, with or without placement (89f34939f).
- Open-Channel namespaces on a controller in another mode, and OCSSD geometries the timing model cannot index (8cd92eec5, 598498e93, f4e85634a, 707d8c8c1).
- Controller memory buffer settings that do not fit, including any CMB BAR other than 2 (1c3a5a2a2, 9d6e13cd3).
- Queue entry sizes other than 64 and 16 bytes (23b21cc38).
- A format with protection information but no metadata room for it (1c9808bcd).
- On `femu-cxl-ssd`, a direct ratio without DER, and `der=memslot` under TCG (34b9f8f9e, d03b0606e, f1182b72e).
- On `femu-cxl-ssd` with the FTL on, NAND with less than 7% of the media as spare beyond the forced collection reserve and two working lines. The old rule counted the reserve as spare. At the old 96 GiB preset (1644 blocks per plane), collection had 1.6% to work in. A request could then wait minutes for queued line copies. The refusal names the new least value (1732 at 96 GiB, 867 at 48 GiB).

- A dpc property with bit 3 set is refused, except on OCSSD. NVM 1.1 does not allow protection information in the first bytes of metadata.

Accepted with a warning (still no effect)

- serial, ms, ms_max, dlfeat, and the CSD nr_thread, time_slice and context_switch_time are still accepted, but nothing reads them, so a value other than the default now prints one warning at realize naming the property; for ms it points to meta (03da6e9d2). tplpbsy and tplrbsy were on this list; they now set the multi-plane program and read busy times, and warn only when set without mp_program or mp_read. trcbsy was on it too; it now sets tRCBSY and warns only without onfi_cache_read.

Behaviour changes (still boots, numbers move)

- Small KV values share NAND pages, as real KV-SSDs pack them (kv_pack=on, the default). A Store goes to an open page in controller DRAM and costs the DRAM time. The Store that does not fit, or that fills the page, programs it and waits for it. Flush with a volatile write cache, clearing WCE and shutdown program the open page (Base 2.3, 5.2.26.1.4 and 7.2). A packed Retrieve reads one page, GC moves values one by one, and log C0h counts host pages from value bytes. kv_pack=off gives the old model, one value or more per page (KV value packing (§4.24)).
- With kv_pack=on, the key limit (MNK) of a KV namespace is the number of 1-byte values its usable flash holds, at most 4194303. Small values are no longer held to two keys for each 4 KiB of value space. The index takes up to about 341 MiB of host DRAM, only as keys use it. New property kv_max_keys sets the limit (KV mode (§4.11.3)).
- A device with blk_pe_limit keeps reserve lines out of service, so it needs one or more lines more. It also refuses mdts=0, reports WZSL equal to MDTS with ONCS bit 3 (NVMWZSV) clear, and limits MCL so a Copy writes no more than MDTS. eol_read_only=off gives the old model.
- With blk_pe_limit, the Persistent Event log records the spare, reliability and read-only warnings when the FTL sees them start. The event carries the warning byte of that moment. Before, the log recorded them only when the host next read the SMART log.
- With blk_pe_limit, the media goes read-only at the end of life by default. A worn-out block with no replacement no longer stays in service with critical warning bit 2. With spare lines, a line no longer retires while the media stays writable. The worn line retires only as the media goes read-only. eol_read_only=off gives the old behaviour.
- A device with blk_pe_limit keeps reserve lines out of service, so it needs one or more lines more. It also refuses mdts=0 and reports WZSL equal to MDTS with ONCS bit 3 (NVMWZSV) clear. MCL keeps a Copy within MDTS, or within one block of the largest LBA format when that is larger. eol_read_only=off gives the old model.
- KV value GC collects one victim line at a time when the free lines fall to the GC threshold, with both key indexes. It replaces the old compaction, which rewrote every live value when the value arena filled: emulator memory set that trigger, not the NAND. The arena now packs its bytes with no NAND time. A Store leaves free a reserve for GC, so a full device returns Capacity Exceeded sooner. FEMU_KV_CHECK checks the owner of every page.
- With kv_index=lsm, meta pages are real NAND pages in lines of their own. Merges program them and value GC moves them. The health log and log C0h count them, so the WAF includes the index. An old value stays valid on flash until a merge drops its record or value GC finds it stale (PinK, section 4.5). Commands pay only the flash the tree reads, with no hash base read. Thus a memtable hit takes no NAND time.
- With kv_index=lsm, a Store keeps room for a merge of every record. When it finds no room, the device merges every level down, so Deletes give back their space. query-femu adds gc-meta-pages, gc-stale-

`value-pages`, `valid-value-pages` and `versions`.

- Compare, Write Zeroes, Verify and Copy are on by default: the `oncs` default is now 0x19d instead of 0x14. A guest that sees Write Zeroes in ONCS can send it, and on bbssd the FTL programs the range unless Deallocate is set. `oncs=0x14` gives the old set. Write Uncorrectable still needs bit 0x2.
- Compare and Verify take the NAND read time of a Read on bbssd, CSD and ZNS namespaces, also after a miscompare. Before, both completed with no media read.
- A Copy on bbssd issues its destination write when its source reads end. On one LUN, a one-page Copy took the read time twice: 600 ms instead of 500 ms with a 100 ms read and a 400 ms program.
- A zone write cache holds one program unit for each die of its zone. A cache sized from the whole device made a narrow zone (`zns_chnls_per_zone` below `zns_num_ch`) program each of its dies several times in series per flush. Full-width zones keep their timing.
- A zoned namespace counts the block erases of Zone Reset. SMART Percentage Used (and the Endurance Group log) now reports them against `pe_cycles_rated`, or the rating of `zns_flash_type`; it used to stay 0. Each block passes its erase count to the media layer.
- Log 20h reports MAXPIDS as MIN(128, NRG x NRUH) - 1, below NRG x NRUH as Base 2.3 Figure 287 asks. It was 127 for every configuration; four handles in one group now report 3, and RUH Update still takes that many identifiers.
- A RUH Update of a handle whose reclaim unit is empty keeps the unit and logs no event 00h. It used to log one, which says that the handle moved.
- The FDP Statistics log keeps 128-bit sums, which stop at the 128-bit maximum, instead of 64-bit ones.
- FDP HBMW also counts Write Zeroes with the Deallocate bit and Write Uncorrectable (NVM Command Set 1.2, 4.1.4.6). MBMW does not, since no page is programmed.
- The default placement handle list of an FDP namespace holds the first 128 handles when `fdp.nruh` is larger: a list holds at most 128 (NVM Command Set 1.2, Figure 125).
- A CSD namespace now goes through its FTL, so reads and writes take NAND time instead of completing instantly. A pure-CSD device previously timed out on its first I/O and the kernel disabled the controller (50b65ca91).
- A namespace whose mode differs from the controller's is routed by its own mode. A bbssd namespace on a NoSSD controller no longer completes inline (50b65ca91).
- `pcie_bandwidth_mbps`, `pcie_prop_delay_ns` and `fw_cpu_ns` now apply to NoSSD. Setting any of them takes the request off the inline completion path, which costs throughput; leaving them unset keeps the previous path (2cdd55c52).
- `cmd_addr_lat`, `pg_xfer_lat`, `status_lat` and `ch_xfer_lat` now add channel bus time on bbssd. The bundled run scripts pass 0 and are unaffected (c274ba7d9).
- `zns_cmd_addr_lat`, `zns_pg_xfer_lat` and `zns_status_lat` add the same channel bus to ZNS. They default to 0, which leaves ZNS timing exactly as it was; a negative value is refused at realize (2a4ebc2cb).
- Temperature threshold Set Features accepts only TMPSEL 0 and Fh; other selectors are rejected rather than stored as part of the value (c7eaa6b4a).
- An aborted command completes as Command Abort Requested rather than Invalid Opcode (03a4eee3c).
- FDP RUAMW counts down in LBAs, so it drains at the documented rate and reaches zero; cost-benefit victim selection now orders by age and utilisation rather than insertion (9a66fe52d).
- The CEMU admin commands of CSD mode moved to vendor specific opcodes: 0x21 to 0xc9, 0x22 to 0xc5, 0x23 to 0xc8 and 0x25 to 0xcd. NVMe Base 2.3 gives 0x21, 0x22 and 0x25 to fabrics commands and reserves 0x23, and 0x22 declared the wrong data direction. The old opcodes now fail with Invalid Command Opcode; `csd_legacy_opcodes=on` restores them for old tools.
- The data plane starts when the host enables the controller rather than on Doorbell Buffer Config, so hosts that never send that command (SPDK, FreeBSD, Windows, older Linux) now work (04f04b86e).
- Failure to lock the backend memory is a warning instead of a fatal error (847864637).

- The media counters moved from the SMART log's temperature fields to vendor log page C0h (4cb1f9ba7).
- The register BAR is at least 16 KiB (01c79c442).
- Under FDP, forced GC keeps at least one reclaim unit free unless `gc_thres_pcent_high` is 100. Below 20 units the watermark rounded to none, and a pass that filled its destination part way had no unit to go on with (89f34939f).
- With `hot_cold_sep`, Streams, or a `hybrid` or `fast` mapping, forced GC keeps at least one line free. Below 20 lines, where the watermark used to round to zero, such a namespace can expose one line less (5a81d02cd).
- OC 1.2 Identify no longer advertises hybrid commands, and unsupported OC 1.2 block opcodes are rejected (f5fc7d573, 6b5081821).
- `mapping=dftl` without FDP keeps its translation pages on flash, in lines of their own. Collection moves them, and dirty evictions and the changes that collection and deallocate make program them. FDP keeps the timing-only model.
- Log page C0h (bytes 280 and 288) and `query-femu` count the translation page programs and reads of `mapping=dftl`, and the write amplification factor includes the programs. Realize keeps lines for these pages, so the largest namespace a geometry exposes is smaller than before. `x-ftl-check` adds the translation page checks.
- With `mapping=dftl` and no FDP, a read starts its media read when its translation is ready. So a CMT miss adds the translation read to the page read; before, the two overlapped. With `hot_cold_sep` a write also waits for its translation.
- With `mapping=dftl` and no FDP, DSM Deallocate and Write Zeroes with Deallocate write the translation pages they change once each, and charge that to the command. Write Zeroes without Deallocate pays the translation of each page, as a write does. Format and Sanitize drop the translation pages with no write-back.
- With `mapping=dftl` and `eol_read_only`, read-only media opens no new translation page line, so the reserve lines stay for the writes in flight.
- ZNS: a zone Finish, by the host or by the controller (`zns_frl_sec`), programs the pages the zone's write cache still holds. A host Finish completes after these programs.
- The `run-*.sh` launchers name QEMU's threads (`-name ...,debug-threads=on`), so `ps -T` and `top -H` show `femu-poller`, `FEMU-FTL-Thread` and `CPU N/KVM` (e29fe6ee2).

The shared namespace model behind `femu-subsys,ns_mgmt=on` is described in namespace management (§4.17.2).

- Identify CNS 06h, DPC with `pi=on` and the Error Information entries return other bytes on a default command line; the lines under Spec conformance give each change.

D.1.4 Fixes

Crashes, memory safety and teardown

- A `der=cylon` guest that onlines the CXL memory in a normal zone, which guest memory auto-online does by default, no longer ends with "KVM internal error. Suberror: 3". Its page tables, stacks and descriptor tables land on cold pages, and version 2 used to hand a page that could not keep a cache way to KVM's emulator; the MMIO entry this leaves cannot pass an interrupt delivery. Version 2 now maps such a page outside the cache and emulates only data accesses to pages the caching API keeps uncached; a guest page walk or an event delivery maps even those. A marked page gets its leaf back to zero when `CACHE_ENABLE` or an unpin makes it admissible again. A vCPU keeps up to 64 pages outside

the cache for one instruction (was 16), and an instruction that needs more stops the VM after `cylon-fault-stop` cycles instead of looping. New counters `der-fault-forced`, `der-fault-deliveries` and `der-fault-marker-refused`; `der-fault-conflicts` now leads to `der-fault-overflows` instead of `der-fault-emulated`. Version 1 keeps every cold page as an MMIO entry, so it needs the CXL memory online as movable; FEMU warns once when the VM runs without version 2. The host kernel side, which exits with `KVM_CYLON_FAULT_DELIVERY` instead of ending the VM, is Cylon kernel candidate 3 (e3c51d264).

- Fixed a memory leak and build errors from the previous release (b11ce9631, 1c2d358c1).
- Fixed ZNS plane allocation and the DSM buffer size (f863d7a8e, 805b9c8e6).
- Fixed a submission queue setup race and a poller use-after-free on controller reset or guest reboot (7100e6179, ac89117b2).
- Fixed priority queue heap repair, which affected FDP victim selection (e2d5413ff, 66067fabc, a4ed1d8c9).
- I/O past the device geometry is refused instead of accessing out of bounds (8848d724d).
- Widened bbssd and ZNS physical address fields so large channel, LUN and plane counts no longer alias (2e4c7fe82, d7c5aff58).
- Fixed an out-of-bounds write in ZNS zone reset with a narrow zone width, and ZNS backend addressing within a namespace (02a11cc51, 316ecf9df, cba095fb5).
- Fixed OCSSD writes past the backing store and many OCSSD bounds checks on vector lists, chunk info, bad block tables, offsets and addresses (fca17a8c6, f2ccb06ab, e8d415a07, 11aafda03, b37e7dd58, ff6e8bb4b, 58ba9f515, 98641558d, d44f0c4c1, 761f78757, 42ae8360e, 1f135b961, d5a5b788b, 91412b74d, 3838cf6bf, 1c9f737fe, 0d6d5926f).
- The FTL thread and pollers are stopped and every mode's state released on device removal, controller reset and failed realize (da76e206a, 06c7579b2, 81fa5697a, b3608d3f4, d0998ea16, 5a4107c14, 6610ecba1, f050a8bce, 3ba1ddec4, 42ba3cf4a, 817d34363, 36d1ef4b0, 52a579830, 8e8dc4431, 8c1fb192f, 0b7c53826, 9b0dd6eb0).
- Fixed leaks of aborted requests, DSM range lists, PRP lists and dropped command lists (03a4eee3c, f3fdbe7c0, 1c498adad, 10aaf11e2, c501af95e).
- A host address is no longer written into guest memory when setting up a non-contiguous queue, and Identify transfers the structure rather than a whole memory page (037d9c9c2, 14f4224d3).
- Queues with 64 KiB pages no longer crash QEMU, and queues are mapped by their length in bytes (cbebe5e10, 49880d0a0).
- Shadow doorbell values are bounds-checked as ring positions and the buffers are one page each (e082da2c1, b95d305b5, 33584d6b8).
- Controller DMA from device threads is restricted to directly reachable memory, which avoids a VM hang on data pointers into device registers (fbfe674c2, 60e4b2d5d).
- Report and log buffers are bounded and their lengths checked before transfer (608749b8d, 40656e5a1, 062bcdb02, 0ddcc33bd, 3106c1205).
- CSD program loads are bounded and confined to `csd_program_dir` (f1d5f3736).
- A CSD controller with no CSD namespace (`femu_mode=4,namespace_modes=nossd`) no longer crashes QEMU on a CEMU admin command with valid fields. It has no program tables, so it now refuses the commands with Invalid Command Opcode and log 05h does not list them.
- CSD programs run on `nr_cu` compute unit threads (`femu-csd-cu`) instead of the poller, so a long program no longer stalls I/O on every queue or the vCPU that sends a CSD admin command; a loaded program can no longer be reloaded in place while it runs, and deleting a queue or resetting the controller under a running program drops its result safely (4596ba498).
- Running out of lines refuses the write instead of aborting, and FDP reports device full instead of asserting or following a null reclaim unit (49ada8834, e1c4e9174, 59128ab66, e7913d89c, c0ae29cb4,

cbda78bf6, 37b9b84f9, f05f128a9).

- Zone state is locked, reset zones are erased on the media thread, and asynchronous events are raised from the main loop (fe8e931e3, 5001ae15c, 89e1c4b04).
- FDP event rings are serialized and the written and uncorrectable bitmaps are updated atomically across pollers (645424c86, daaa6a3db).
- The BlackBox flip command applies to every namespace of the controller (3b7d88d63).
- `femu-cxl-ssd` no longer advertises that unmapped pages read as zeros (3c85bfd86).
- Switching `femu-cxl-ssd` `fast-load` off no longer sleeps out the NAND backlog on the QEMU main loop. After a fast fill the backlog can be minutes, and the guest's boot disk, served by the main loop, timed out and the guest lost its root file system. The switch now returns at once and the backlog stays on the NAND timelines; the new `nand-idle-ns` tells a harness when the NAND is idle. The switch does not wait for the FTL mutex either, which the worker can hold for a whole forced collection. Device DMA work queued before the switch books its NAND time before any later access, linked NVMe requests included, and `fast-load-drain-ns` includes it once it is booked.
- A QEMU NVMe read into `femu-cxl-ssd` memory no longer stops the QEMU main loop for the media time of the CXL pages it fills. The block layer completes such a read on the main loop, outside any re-entrancy guard, so the copy took the waiting path and slept up to the whole NAND backlog. Accesses from the main-loop thread now queue their media time as guarded device DMA does; an IOThread's copies, and those of other threads such as RCU's, which also hold the BQL, keep the full model.
- An Open-Channel SSD no longer runs or advertises Dataset Management, Write Zeroes, Compare, Verify, Copy or Write Uncorrectable. Their common handlers used the sparse Open-Channel address as a flat LBA. The default `oncs` allowed Dataset Management, and a deallocate of a high address zeroed memory past the backing store and stopped QEMU. Flush is still accepted.
- Many `femu-cxl-ssd` lifecycle, decoder and mapping fixes, including teardown while callers wait on invalidation, little-endian forwarding, and keeping stores when NAND has no page for write-back (9e0223c19, bb7e28722, 51c420143, d31c48d92, ec8b1ac7d, 02a872661, 4505cd9a7, 86ba15078, 842d4ff72, b322efb8f, 2d0c1404e).
- Device threads copy guest memory through the translation they checked. A guest that mapped a ROM over a buffer between the check and the copy could make a poller wait for the BQL while a pause held it.
- A ZNS zone report is built under the zone lock and sent after the lock is released, to directly reachable memory only. A report into ROM used to hang the VM when a Changed Zone List read waited for that lock.

Data correctness

- A CXL memory device (`cxl-type3` and `femu-cxl-ssd`) disables its media while a Sanitize command runs, as the CXL specification says. The media check read the first mailbox register instead of the memory device status, so media was never disabled; it now reads the status atomically, so `femu-cxl-ssd` fault exits without the BQL read it without a data race.
- With more than one poller, a read no longer sees a write, a Copy destination or a Write Zeroes range half done. Each NVMe access to the memory backend now holds the locks of the 4 KiB granules it touches, so a write is atomic as AWUN 0 promises. CXL.mem accesses of a `femu-cxl-ssd` medium do not take these locks.
- Compare reads the stored data instead of a zeroed buffer (7589ee012).
- FDP TRIM honours the requested LBA ranges instead of erasing the whole device (0cc20e897).
- Fixed FDP victim accounting, foreground GC under write pressure, the initial reclaim unit, and the FDP Events log offset (cee9670a5, 247151a16, f9fc94d4f, 438e485a1).
- bbssd maps blocks by their real size, so 4 KiB formats no longer share logical pages (bb63df5a3).

- Fixed ZNS zone append, Zone Append bounds, zone open on write, zone resource counts, multi-zone reset, and zone write caches keyed by zone (07acf491a, 27949b51f, 3fd4f34e6, 8129cfd16, 340522142, 99ce4fb2b, 9a3449362, 347a8a691).
- A ZNS zone whose write cache is evicted before it fills now continues its next flush on the first free plane, not on plane 0. With more zones written at once than write caches, plane 0's block ran past its last page and reads of those blocks finished faster than tR (8bcaa0623).
- A ZNS write refused for its data pointer no longer moves the write pointer (6343c4d22).
- FDP Write Zeroes is placed by its own directive fields and counted in HBMW and MBMW, instead of reusing the placement of an earlier command (834abe2cb).
- Under FDP, the Directive commands report the Data Placement Directive, and the host can enable it per namespace. The enabled state survives a controller reset.
- Write Zeroes and deallocate address the backend per namespace, program the media where required, and drop buffered copies (f44ce6499, 9005cf952, 373de039b, 783131b7d, 22988e07b).
- The write buffer no longer resurrects deallocated pages and is bounded (d4d03f127, 0ee6d211b).
- A Format rebuilds per-block state (a79474e73, 7e7e36161).
- Writes clear the invalid status set by Write Uncorrectable (026eb0ef0).
- A KV Retrieve or List that returns less than the host buffer holds follows the PRP layout of the whole buffer. A transfer of 4 to 8 KiB into a larger buffer took PRP2, a list pointer, as a data pointer, and wrote the data over the host's PRP list.
- CSD and KV commands address the namespace they name, and each namespace gets its own mode state (a93de9d54, aa6a5a8f7, d2a770601, ce5ad913f).
- A KV command completes when its last NAND operation does: a Store that compacts no longer counts the compaction wait twice, and still pays for it when it then fails. The per-command index read is charged on a LUN chosen by the key's hash instead of always on the LUN at address 0, which had every KV command queue on one LUN (4ca11c2a0, 1e5385ae4).
- Vendor admin command 0xEE sets an Open-Channel controller's read, program, erase and channel times; it used to write fields nothing read. Other modes refuse it with Invalid Field (949d01eb9, cdbd5bb36).
- `namespace_sizes` may add up to all of `devsz_mb` when the namespace count does not divide it, and each size is rounded down to whole logical blocks, so TNVMCAP counts only addressable capacity (d8cdd4d22).
- A CSD controller in an FDP subsystem refuses the same knobs as a BlackBox one and reports the superblock as its reclaim unit size, instead of 96 MiB (207bb12d4).
- bbssd GC no longer erases a line it could not empty, which left mappings pointing at erased pages and made every later write fail. Forced GC runs before every page a command programs, so a large write on a device with fewer than 20 lines no longer runs it out of space (5a81d02cd).
- FDP GC moves all of a reclaim unit's pages before erasing any block, retires each moved page's old copy, and runs foreground GC per page. A pass that stopped part way used to leave the unit with erased blocks counted again later (c7b373186).
- FDP GC takes a new unit for a collection destination that was dropped when it filled with nothing free, and collects a unit with no valid pages without one. GC used to stop for good once the destination was gone, so a full device stayed full even after the host deallocated everything (22a0fc5ee).
- An FDP handle whose last unit filled with nothing free reports no room in RUH Status. It used to report the room of its retired unit, which GC could free and give to another handle (4e7a06666).
- FDP RUAMW counts every page programmed into a reclaim unit, also the pages that collection or a refresh moves into the active unit of an Initially Isolated handle. That unit used to report the room of the moved pages as free. A Format keeps the programmed pages, so RUAMW no longer returns to a full unit after it.

- RUH Update on a CSD namespace moves the handle to a fresh reclaim unit, as on a BlackBox one. It used to reset only the RUAMW, while the FTL went on writing into the old unit.
- The `random` and `d-choice` GC policies and FDP's random reclaim strategy draw victims from a generator seeded by the new `gc_seed` property instead of the wall clock and `rand()`, so the same configuration and workload give the same victims and WAF on every run. `fifo` finds its victim at the top of a queue ordered by close order instead of scanning every line, with the same victims as before (04ba1c0aa).
- FDP background GC counts the pages a unit never wrote as reclaimable. A unit that RUH Update retired holding a few valid pages had none invalid, so each background pass took it from the top of the victim queue, refused it and stopped; units behind it were collected only under the foreground watermark (beb9878ac).
- FDP cost-benefit GC ages a unit retired with no page invalidated from its retirement. It used to count from time zero, so a part-written unit left by RUH Update outscored every other victim; with few pages to free, background GC refused it on every pass and collected nothing. (a9e0e1af6)
- FDP RUH Update runs foreground collection first, as a write does, while the reclaim group is at or below its forced watermark. It used to take the last free units, so collection had no unit to move pages into, and a write that fit the namespace failed with Capacity Exceeded.
- FDP RUH Update gives a handle with no reclaim unit an empty one, or fails with Capacity Exceeded when collection frees none. A handle is left with no unit when its last unit fills with nothing free. The update used to succeed and leave the handle with no unit (Base 2.3, 7.4.1.1).
- FDP cost-benefit GC measures a unit's utilization against all its pages at every invalidation, as it already did at retirement. The first overwrite in a part-written unit used to measure it against the pages written only, so a unit holding 3 valid pages of 16 scored as 3 of 4 full and lost to units that cost more to collect. (529d70960)
- FDP background cost-benefit GC takes the best-scoring unit with enough to free. A unit with one invalid page and an old invalidation could outscore every other victim; the pass refused it and collected nothing until the others aged past it or the foreground watermark was reached (fe29da90b).
- A ZNS Zone Management Send waits for the writes to its zones that the controller already accepted. A Reset in the gap between a write's acceptance and its data copy left an Empty zone with a moved write pointer and the old data.

Spec conformance and host compatibility

- With a `femu-subsys`, SMART byte 6 (Endurance Group Critical Warning Summary) repeats bits 0, 2 and 3 of the endurance group's warning. It read 0.
- A KV namespace runs its I/O commands only when the host enables the controller with CC.CSS 110b. Under 000b, which selects only the NVM command set, they fail with Invalid Command Opcode (Base 2.3, Figure 41). Linux selects 110b. Set Features I/O Command Set Profile (19h) now succeeds with no effect when CC.CSS is not 110b (Base 2.3, 5.2.26.1.17).
- Write Zeroes reads its directive type from CDW12 bits 22:20 (NVM Command Set 1.2, Figure 82). Bit 23 is NSZ: with it set, a Write Zeroes failed with Invalid Field once the Data Placement Directive was enabled, or lost its placement.
- FDP event 03h sets LV, with the reclaim group and handle that the controller selected for the write (Base 2.3, Figure 296).
- The placement handle of a placement identifier is bits 15 - RGIF to 0 (Base 2.3, Figure 290). FEMU read one bit less, which did not matter with one group. RGIF is now the fewest bits that hold `fdp.nrg` - 1.
- A Copy descriptor format 2 range that names the command's own namespace is accepted in every mode that accepts Copy. A CSD namespace refused it with Invalid Namespace, although the controller

advertised format 2.

- Identify CNS 19h (Endurance Group List) lists endurance group 1 on a controller in a subsystem, from a starting identifier of 0 or 1, as `nvme list-endgrp` asks. Without a subsystem it is refused.
- Commands complete on the completion queue they were bound to, and submission queues that share a completion queue are served (7a3f1d9fa, dc4289f91).
- Completions are not posted into a full queue and the phase tag is written last (6f998742f, 04ce8b1a7).
- I/O interrupts are delivered without a KVM route, and pin interrupts work with shadow doorbells (8ebffbaa4, 34621540e, 4ca1be945).
- Create SQ/CQ reject out-of-range queue IDs and invalid interrupt vectors, and return the specified statuses (b3bbaedb0, 7580dd341, edefcd7c5).
- Fixed controller configuration and admin queue registers, feature reset on controller reset, and the fatal status cleared on disable (330ed2aab, f2ea41361, ab63717c5).
- Get Log Page reads its identifier and offset correctly and serves every log from the requested offset (a6b0d5897, ef8b755ee, 23fc538ea).
- Feature identifier, selector and save bit are decoded, and namespace-scoped features take a namespace (2380e4ecc, 1729244c2).
- Temperature Threshold: Get Features refuses the reserved THSEL values and "all sensors", Set Features refuses the reserved THSEL values, the over threshold starts at the warning temperature (343 K) as Identify reports, and the under threshold sets SMART critical warning bit 1 and raises the temperature event (7ff2172cc).
- Identify reports total NVM capacity and the fields a host relies on (907ad6ff1, 8530bc287).
- Fixed the error and SMART logs, the log directories, Format secure erase decoding, data pointer and transfer size statuses, and bad doorbell reporting (325e02cfd, c52d9ff9c, adfc12e93, bdb6f6b59, 225c6bdee).
- Fixed the AER count, Zone Append limit, KV MDTS and vector masking (7b5dbbc77).
- A failed data transfer is reported as an error, PRP lists starting inside a page are followed, and SGL errors use their own status codes (29175cb05, ecbef8416, c9344bd4c, e310fdb43).
- The Changed Zone List reports only unsolicited changes, ZNS follows the zone resource, compare and notice rules, and commands that rewrite a zone behind its back are refused (7a6f01f8a, 41a6c189f, 99d2194e0, ed38b0728).
- A reset zone holds no data, an offline zone returns its resources, and zone geometry is reported for the namespace's format (bce5e1c3e, 2c7062ed0, 901d0908f, b4d9585ca, f0f989c21).
- KV list entries are padded to 4 bytes, NVM-only commands are refused on KV namespaces, and KV Identify requires a KV namespace (240e21c43, f5a7d7bc8, 5ec9ee859).
- FDP reports the reclaim unit size the FTL uses and moves a handle to a new reclaim unit on update (7fe32c556, 78de60448).
- FDP reports reclaim moves with the controller events of the specification. A collection or refresh of Initially Isolated data logs 80h with the blocks moved, and 81h when the moves give that handle a new unit. Moves of Persistently Isolated data log nothing; collection used to log 81h for every unit.
- The FDP Reclaim Unit Handle Usage log uses 8 byte descriptors, so `nvme fdp usage` reports every handle as host specified instead of reading some as unused, and the placement logs store their fields little endian (1bc8508a6, 31bd13259).
- With `namespace_modes`, the controller's model number and serial come from its own `femu_mode` instead of the namespace brought up last (efdb98643, c3f968029).
- SMART wear counters are summed across namespaces (4a4f0d9bf).
- Counters fixed to move in every mode: FDP and KV write amplification and bytes, KV relocations, FDP erases (f4b3ac376, 980fb7886, 37464e4bd, ce20d8c06, 4c7d9c22a, 749e09fef, 2a16553c9, ba0c49338).
- OC 1.2 enforces its bad block table: a write or erase that names a factory bad, grown bad or device

reserved block fails with Write Fault. Set Bad Block Table marks the plane it is given instead of another block's plane, and keeps the table's counts current (933addb72).

- Abort no longer rewrites the aborted command in the host's submission queue: the controller marks it and completes it with Command Abort Requested when fetched. An Abort run with more than `acl` others queued behind it fails with Abort Command Limit Exceeded (7b7eaf133).
- Logs 12h (Feature Identifiers Supported and Effects) and 13h (NVMe-MI Commands Supported and Effects) are answered and listed in log 00h. A feature that needs something absent, such as Volatile Write Cache without `vwc=1`, is an invalid field for every selector (7b36b4c25).
- Enabling with a CC.CSS value that CAP.CSS does not offer fails the controller, and admin commands that name SGLs are refused (992dc9911, 2ff5e3925).
- Every Unrecovered Read Error and Compare Failure sets Do Not Retry (f8efff85a).
- Identify reports one read-only firmware slot, matching the firmware log (989df3cea).
- OACS, ONCS, OCFS, LPA, SANICAP and logs 00h and 05h are built from one capability registry, and a qtest checks them against what every mode answers (a1c67d97f, 933fb3f15).
- Log 05h lists each mode's own commands (BBSSD 0xEF, the Open-Channel and CSD commands, Flush on KV namespaces) and no longer lists Read and Write for Open-Channel 1.2, which refuses them (99eabe2c1).
- Log 00h lists the Open-Channel 2.0 chunk information page (CAh) (209b499b6).
- Get Log Page answers a page log 00h lists for no command set with Invalid Log Page, so the endurance group and FDP pages without a subsystem no longer answer Invalid Field (87d5f1372).
- A zoned namespace reports no Copy limits in Identify Namespace, since it refuses Copy (92b3763a3).
- Vendor admin command 0xEE outside Open-Channel mode answers Invalid Command Opcode instead of Invalid Field, since log 05h does not list it there.
- A subsystem with FDP off has no FDP capability, as CTRATT.FDPS already said. The FDP pages 20h to 23h answer Invalid Log Page and Features 1Dh and 1Eh answer Invalid Field; some used to answer FDP Disabled. Where log 05h does not list I/O Management Send and Receive, they answer Invalid Command Opcode instead of their no-operation.
- Log 00h and the pages of one command set follow CC.CSS: with CC.CSS 000b they serve the NVM set whatever CDW14.CSI names. The Changed Zone List (BFh) answers only for the zoned set. Open-Channel 2.0 Chunk Information (CAh) answers only for NVM. Another set gets Invalid Log Page. BFh with an unknown CSI answers I/O Command Set Not Supported.
- An Open-Channel 2.0 Vector Chunk Reset handles every entry of its LBA list. An entry that fails no longer stops the entries after it. Dword 0 and 1 of the completion mark the failed entries, and the metadata buffer gets a chunk descriptor for each entry. Each chunk that is reset is charged one erase, also when another entry fails.
- The controller reports NVMe 2.1 instead of 1.4, with what that requires: CAP.CRMS and the CRTO register, BPCAP 01b, Identify CNS 1Fh, CNS 00h refused for a Key Value namespace (Invalid I/O Command Set), CSI-specific log pages refused for an unknown command set, and CNS 07h refused for a set CC.CSS does not enable. Open-Channel stays at 1.4 (409db7bb3).
- CAP.AMS no longer claims weighted round robin, which nothing arbitrated by, and enabling with another CC.AMS fails (409db7bb3).
- A controller in a subsystem reports its endurance group (CTRATT bit 4, ENDGIDMAX, each namespace's ENDGID, the Key Value Identify structure included) whether or not FDP is on, as log 09h already did (409db7bb3).
- The spare critical warning in the SMART and Endurance Group logs is set when the available spare falls below the threshold, not when it equals it (4faaff90a).
- A memory backend that the host cannot allocate now refuses the device with a message that names its size. Before, QEMU aborted in the allocator.

- An asynchronous event identical to one already queued (type, information, log page, namespace) is not queued again. Copies used to fill the 16-entry queue and push out other namespaces' events.
- Reading one zoned namespace's Changed Zone List no longer unmasks Notice events while another namespace's reported notice is unread. That read used to send the other namespace's queued notice before its log was read (Base 2.3, 5.2.2).
- A Changed Zone List read without RAE clears the list and its event only after a successful transfer, as SMART and Error Information already did. A failed transfer used to lose the changes. A zone that changes during the read keeps its entry.
- An asynchronous event about a namespace no longer counts against the 16 queued controller events. With more than 16 zoned namespaces, a Zone Descriptor Changed notice used to be dropped and never raised again.
- Identify CNS 06h reports the version of each command set (TP4135): NVM 1.1 in bytes 23:20 and ZNS 1.2 in bytes 35:32. Both read 0. KV stays 1.3, SLM and CP stay 1.2, and an Open-Channel controller is unchanged.
- The NVM Identify CNS 06h sets AOCS bit 0, as Get LBA Status serves action type 02h. It read 0.
- With `pi=on`, DPC is 17h instead of 1Fh: protection information sits in the last 8 bytes of metadata only (NVM 1.1, Figure 114). Format and Namespace Management refuse PIL with a protection type, and DPS never reports PIL.
- Error Information entries name the command (TP4113a): byte 30 holds its command set and byte 31 its opcode. Every returned entry has Log Page Version 1h in byte 63. Open-Channel entries are unchanged.
- Identify CNS 0Ah for key value ignores the NSID and reports only format capabilities: NSZE and the key limit read 0. Indexes other than 0 fail with Invalid Field.
- With Namespace Management, Identify CNS 09h and CNS 0Ah answer for the NVM Command Set (TP4095a). Without it they stay refused.

D.1.5 Documentation and tooling

- `femu-stat.py` prints the end-of-life state of a namespace with `eol_read_only`: read-only or writable, and the reserve pages that admitted writes hold.
- Documentation moved under `hw/femu/docs/` with getting-started, concepts, mode and feature guides, an FAQ, security notes and a doc map (e00c70f99, 034670c4c, 62a565ae0, fb1cf3ce6, 28aa224b7, 0bcb5cae2).
- CXL SSD, caching API and NVMe link guides, and a reference for every `femu-cxl-ssd` property and counter (41b318ef2, 9ec16a701, 93bfbf5c9, ccaa5d190, dcea2b9a8).
- A property reference generated from the binary and checked in CI, with every device property described in its help text (a6118d051, 9489cf692, 37aaccf27, c2a12a2eb).
- The mode table is generated from `modes.py`, and every documented command line is tested (6a97f8755, 12361d7f7, 573da0a1b).
- Refusals and behaviour changes that affect existing command lines are listed (9e93cb94f); that list is now part of this changelog.
- The `vwc` description no longer claims `vwc=0` makes Flush a no-op: Flush drains the bbssd write buffer either way (f48a6bc6f).
- Documentation of the SMART vendor area, vendor log page, endurance rating, mapping schemes, GC and cache options (4e1f5bce5, 77fa23981, 9f52a098d, 658e4c9e6).
- README refreshed: build hosts for the QEMU 10.1 base, binary path, guest image steps, namespace and plane limits, debugging switches, and links to the new docs (4f327242c, a967b15fa, 4dbeea46d, b5c68a96f, f2d9dee99, b2ae45483, 65332b21a, 10f30e231).

- Added CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, CITATION.cff and ROADMAP files (599a36676, da692f5d9).
- `ssd-config.sh` expands an INI-style config file into the device arguments, checking keys against the binary (7f605016e).
- `femu-test.sh` checks a device from inside the guest for block, zoned, KV and CSD namespaces (9fee8190c, 2244a1b9c, c17a31ea9).
- `guest-conformance.sh` boots a guest for each conformance suite and runs blktests (the `block` and `zbd` groups) or the nvme-cli end-to-end suite against FEMU. The guest scripts `blktests-guest.sh` and `nvme-cli-e2e-guest.sh` also run on their own inside a guest. CI does not run them.
- `guest-conformance.sh` runs a `zenfs-crash` suite: `zenfs-crash-guest.sh` builds RocksDB with ZenFS in the guest and runs a crash test on a ZNS namespace. The new `--device`, `--smp` and `--mem` options serve control runs and larger guests.
- `kv-bench.c` drives a KV namespace with the NVMe-KV commands and checks each answer against a model of the keys. The `kv-bench` suite of `guest-conformance.sh` runs it and `kv-probe` in a guest.
- `femu-config-check.py` starts QEMU under qtest with no guest and reports either the refusal or the configuration FEMU builds: capacity, geometry, over-provisioning, line, reclaim unit and zone sizes, mode and NAND timing per namespace, and warnings. The documentation example check runs its documented commands.
- `femu-stat.py` shows the FTL counters of a running VM from the host, like `top`: pages, WAF since start and over the interval, line, reclaim unit and zone counts, and I/O stage times, from QMP `query-femu` and `query-femu-stages`.
- `make-guest-image.sh` builds an Ubuntu 24.04 guest image, and the run scripts accept `IMGDIR`, `OSIMGF` and another SSH port (e877f2dc0, e74535012, 66bb3068e).
- The build script fails on a compile error, and the config self-test fails when FEMU does not survive (a1bf37caf, da7c1fcfe).
- The config self-test requires each device to come up, not only its property names to be accepted. That caught `zns.conf`, which asked for more active zones than it has and was refused at every size (b509a23a5).
- Config presets for OCSSD, CSD, KV and NoSSD (NoSSD with link and firmware time from the NVMeCHA controller), and a `run-kvssd.sh` launcher (04eb9b06d).
- A key-value probe tool and a corrected KV wire format description (9ab75f5dc).
- `pin.sh` no longer names a CPU past the last one, and now pins the pollers, the FTL thread and the CXL SSD's `femu-cxl-ftl` and `femu-cxl-cca` threads as well as the vCPUs (e29fe6ee2, 56e0e2f27).
- Every launcher, the legacy scripts and `make-guest-image.sh` name QEMU's threads with `debug-threads=on` (e29fe6ee2, 56e0e2f27).
- The documentation example check realizes the guest's disks and NICs too, with stand-in backends, so a device the machine cannot plug (a NIC without `bus=pcie.0` on a `cxl=on` machine) fails the check (ac5a152fe).
- `kv-probe.c` comments describe what it checks and which device nodes it can use (dfc1bd606).
- GitHub Actions CI on several Ubuntu releases, with pinned actions, a read-only token, sanitizer builds, link checks and steps that can actually fail (3b4708763, daf890b7f, c613ae5ac, 4164e551b, 8d4762608, 89ff3ed38, e3700422c).
- qtests and unit tests for FEMU now live under `hw/femu/tests/`, including a NAND media unit test run by meson and FTL checks turned on in CI (d72e30a34, 6549d6094, cc926e1e9, 8241ccc7f).
- Seeded fuzzers for admin, I/O, zoned, FDP, KV, Open-Channel 2.0 and CSD commands, run on sanitizer builds in CI (8a57a3e27, 6d6d927d0, 9798a6574, dff3d9f72, 28eaf2afa, 2dbb18a82, 243fc0b66, 2f1208402, 692d9a106).

D.1.6 Removed or legacy

- `femu_mode=6` (an unimplemented SmartSSD placeholder) is no longer accepted (c3b8e88ef).
- QEMU network block drivers are no longer built (81c4385fc).
- Three NAND media policy flags that nothing read were dropped (2e29d7efb).
- OC 1.2 no longer advertises hybrid commands it does not implement (f5fc7d573).